클로드코드로 하는

바이브 코딩

한국폴리텍대학 영남융합기술캠퍼스

임성국

목차

- 01: 나의 첫 바이브 코딩
- 02: 효과적인 프롬프트로 AI 200% 활용하기
- 03: 클로드 코드 시작하기
- 04: 클로드 코드 실전 활용
- 05: 게임 제작으로 배우는 체계적인 개발과 관리
- 06: 클로드 코드에 API 날개 달기
- 07: 클로드 코드 AI 에이전트로 개발팀 구성하기
- 08: MCP로 클로드 코드의 한계 넘어서기

SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(1)

- 거대 언어 모델(Large Language Model, LLM) - 대규모 언어 AI 모델
  - 자연어뿐만 아니라 방대한 프로그래밍 코드와 기술 문서들을 함께 학습
  - 사람의 언어를 이해하고 자연스럽게 대화하듯, 프로그래밍 언어의 문법과 패턴, 그리고 개발자들의 코딩 스타일까지 학습하여 뛰어난 코드 생성 능력을 갖춤
- 바이브 코딩
  - 새로운 개발 패러다임의 시작점 - LLM을 실제 코딩에 활용

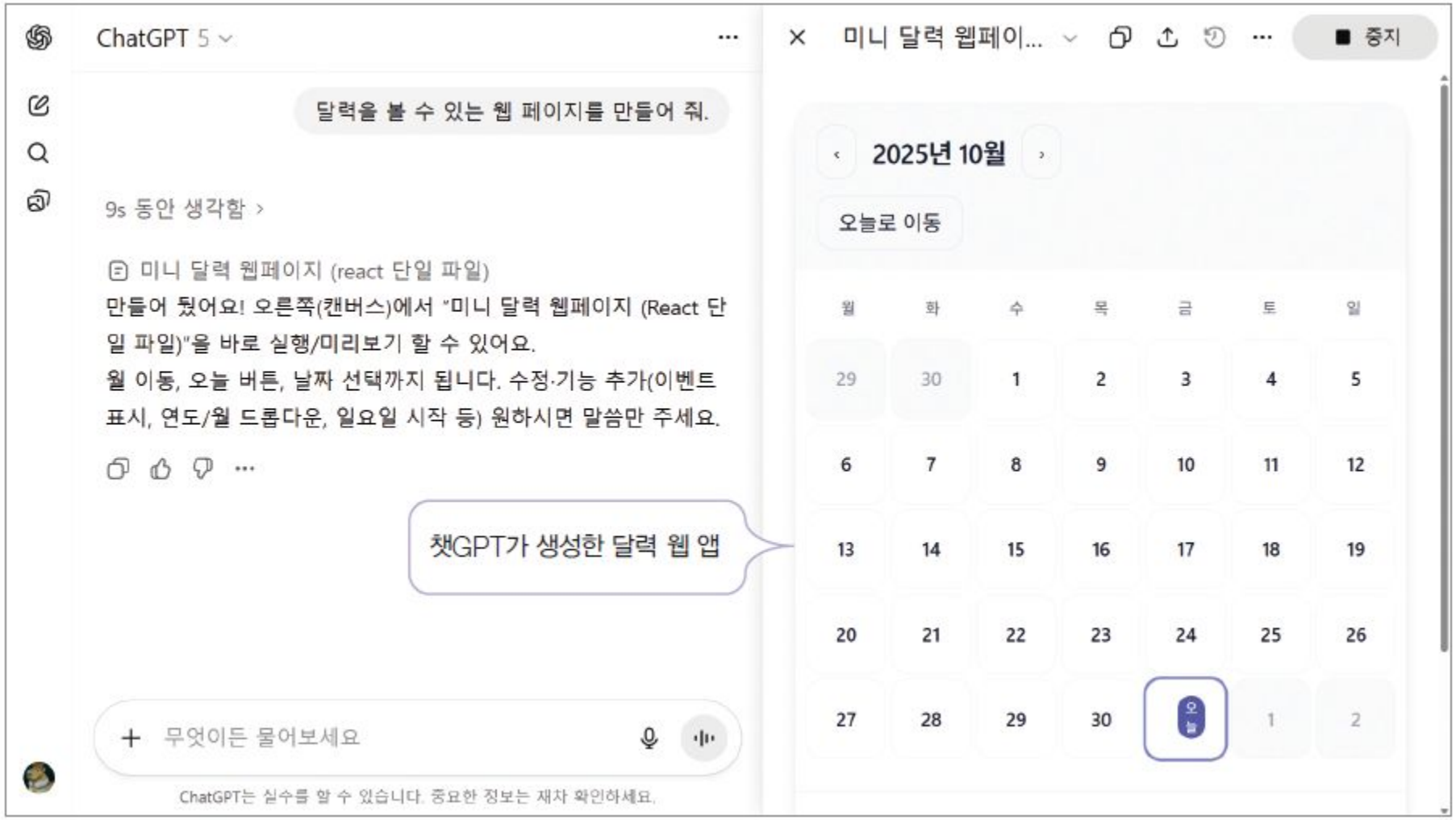
SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(2)

◦ 바이브 코딩(vibe coding)

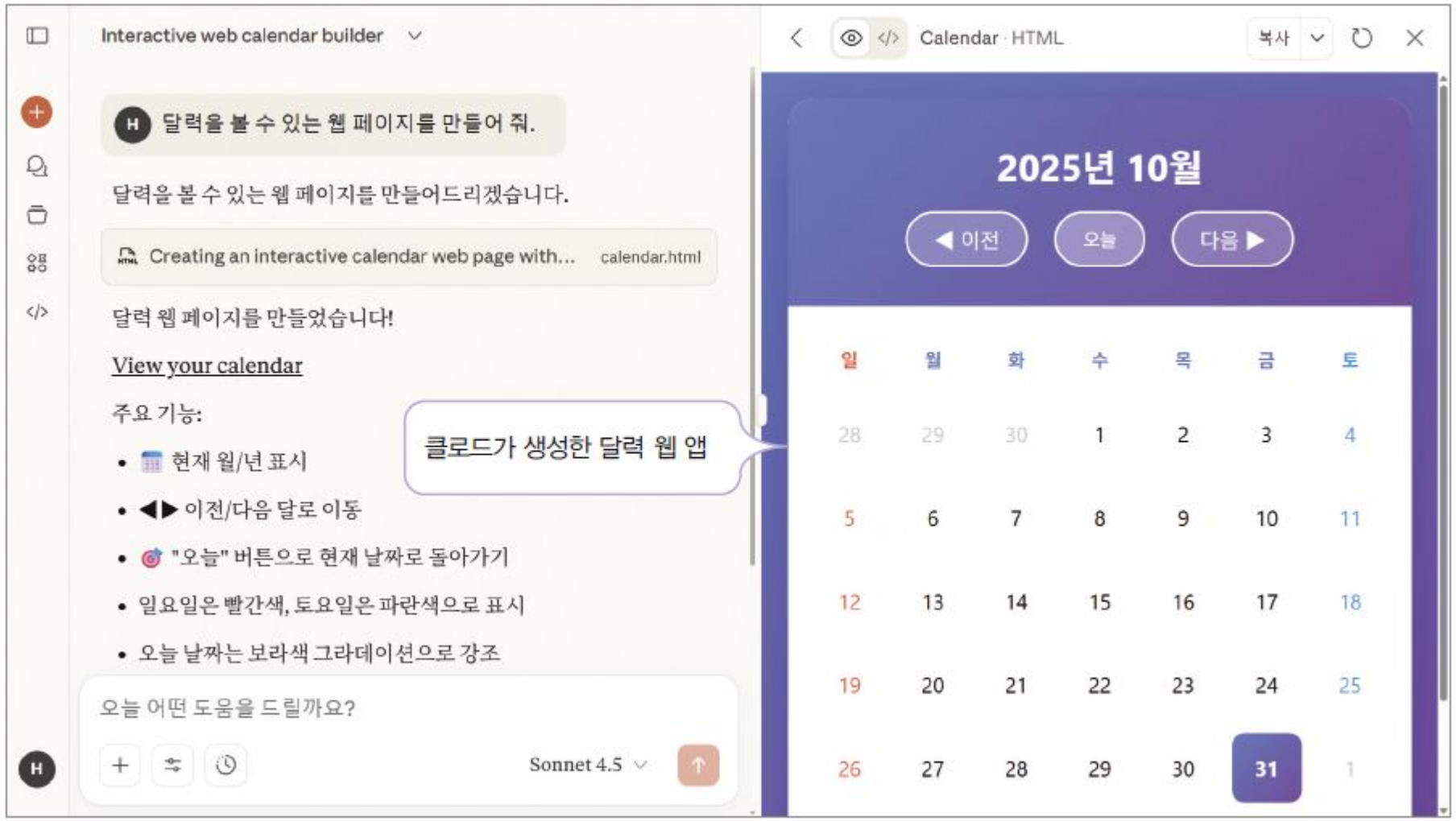
- 사람이 직접 코드를 한 줄 한 줄 작성하는 전통적인 프로그래밍 방식에서 벗어나, AI와 대화를 주고받으며 프로그램을 구현하는 새로운 개발 패러다임
- 2025년 2월 오픈AI 공동 창업자인 안드레이 카파시(Andrej Karpathy)가 제안
- AI에게 아이디어를 자연어로 설명하면 AI가 이를 해석해 코드와 실행 방법을 제시하고, 사람이 이를 검토 및 수정하며 점차 완성도를 높여 나가는 과정
- 바이브 코딩의 가장 큰 특징 - 누구나 개발의 주체가 될 수 있음
  - LLM이 방대한 텍스트와 소스 코드를 학습하여 사람의 요청을 이해하고 맥락에 맞는 코드를 생성
  - 오랜 시간을 들여 개발 지식을 전문가 수준까지 학습하지 않아도 아이디어만 있다면 AI와의 대화를 통해 손쉽게 프로그램을 완성
  - 초보자에게는 새로운 기회를, 숙련된 개발자에게는 생산성을 획기적으로 높이는 방법을 제공

SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(3)

- 챗GPT와 클로드에게 달력 웹 앱을 만들어 달라고 요청한 결과



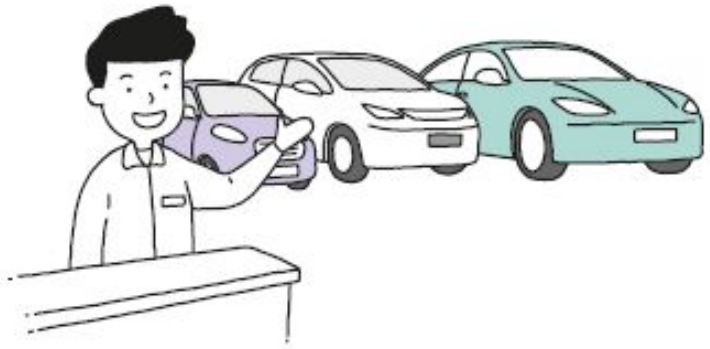
SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(4)



SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(5)

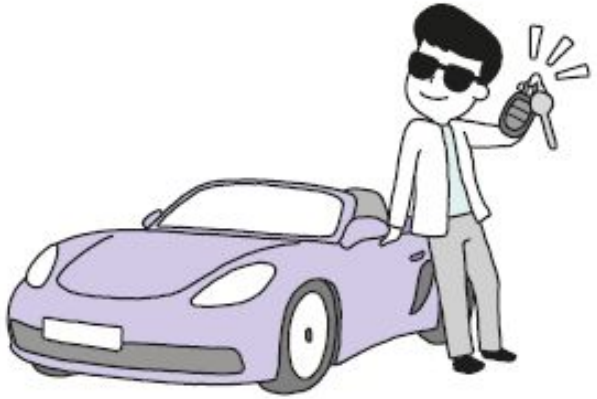
◦ AI 통합 도구의 등장

- 단순히 코드를 생성하는 데서 그치지 않고, 개발 전 과정을 AI가 지원할 수 있도록 설계
  - AI 모델 선택형
  - 전용 AI 도구형



AI 모델 선택형
(커서, 리플릿 등)

VS



전용 AI 도구형
(클로드 코드, 제미나이 CLI 등)

SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(6)

◦ 모델 선택형

- 렌터카를 빌리듯, 프로그램에서 여러 LLM을 선택해 사용할 수 있도록 만든 방식
  - 코드 편집기와 AI가 통합되어 있어 사용자 인터페이스에서 복사-붙여넣기 없이 코드를 작성하고 수정
  - 커서(Cursor), 리플릿(replit), 윈드서프(Windsurf) 등
- 장점
  - 여러 언어 모델 중 특성에 맞는 하나를 골라 편리하게 사용
- 단점
  - 렌터카를 쓸 때 기름값과 더불어 렌터카 업체 수수료를 부담하듯, 모델과 더불어 도구 자체의 비용을 이중으로 부담
  - 언어 모델마다 성능의 차이가 크지 않아서 대부분 한두 가지 모델만 주로 쓰게 되는 경우가 많음

SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(7)

◦ 전용 모델형

- AI 언어 모델을 직접 내 컴퓨터에 설치해서 사용하는 방식 - 마치 자기 차를 소유하는 것과 비슷
  - 클로드 기반의 클로드 코드(Claude Code), 구글의 제미나이 CLI(Gemini CLI) 등
- 장점
  - 해당 AI 모델 제작사의 전용 프로그램을 설치하므로 추가 비용 없이 모델 사용료만 지불
  - 로컬 파일에 직접 접근할 수 있어 프로젝트 전체를 효과적으로 관리

- 사용자와 컴퓨터가 상호작용하는 방식
  - **GUI**(Graphical User Interface) - 화면에서 아이콘, 버튼, 창 같은 그래픽 요소를 마우스로 클릭해 사용
  - **CLI(Command Line Interface)** - 명령 프롬프트cmd라고 하는 창에 텍스트로 직접 명령어를 입력해서
    컴퓨터를 움직이는 방식
    - 예) mkdir temp는 'temp'라는 이름의 새 디렉터리를 만들라는 명령어
- CLI는 겉보기에는 어렵지만, 빠르고 가벼워서 특히 전문가가 서버를 관리하거나 반복 작업을 자동으로 처리할 때는 GUI보다 훨씬 효율적

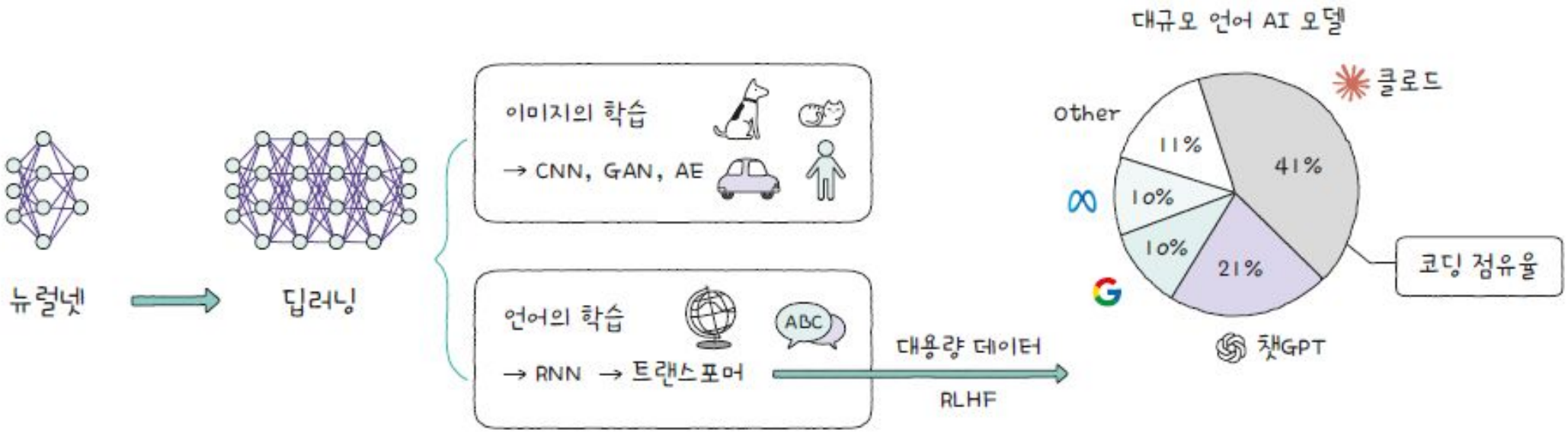
SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(8)

좀더알아보기 **AI 어시스턴트의 탄생과 발전**

- 인공지능(Artificial Intelligence, AI)의 역사는 1940년대 뉴럴 네트워크(neural network) 방식에서 시작
  - 기계가 주어진 데이터를 학습하고, 그 속에서 패턴을 찾아 예측하려는 시도
- 2010년 무렵부터는 딥러닝이라는 이름으로 더욱 널리 알려지기 시작
  - 초기 딥러닝 모델
    - 합성곱 신경망(Convolutional Neural Network, CNN)
    - 생성적 적대 신경망(Generative Adversarial Networks, GAN)
    - 오토인코더(Autoencoder, AE)
    - 순환 신경망(Recurrent Neural Network, RNN)
- 트랜스포머가 어텐션 메커니즘(attention mechanism)으로 RNN의 한계를 극복하며, 현재 챗GPT와 클로드 같은 대규모 언어 AI 모델의기반 기술이 됨
- 언어를 학습해 대화하는 언어 모델(Language Model, LM)의 발전
  - 2017년 구글 연구팀이 발표한 트랜스포머(transformer)
- 오픈AI(OpenAI)
  - GPTGenerative Pre-trained Transformer, 생성적 사전 학습 트랜스포머 시리즈를 발표
  - GPT-3.5 버전에 이르러 챗GPT(ChatGPT) 상용화
- 앤트로픽(Anthropic)
  - 2021년 오픈AI 출신 연구진들이 AI 안전성을 최우선 가치로 내세우며 설립
  - 클로드(Claude)를 개발해 현재 코딩 분야에서 가장 인기 있는 AI 도구로 자리매김
- 구글이 제미나이Gemini를, 일론 머스크Elon Musk가 이끄는 xAI가 그록Grok 같은 대규모 언어 AI 모델들을 속속 발표

SECTION 1-1 나의 코딩 파트너, AI 어시스턴트(9)

좀더알아보기 **AI 어시스턴트의 탄생과 발전**

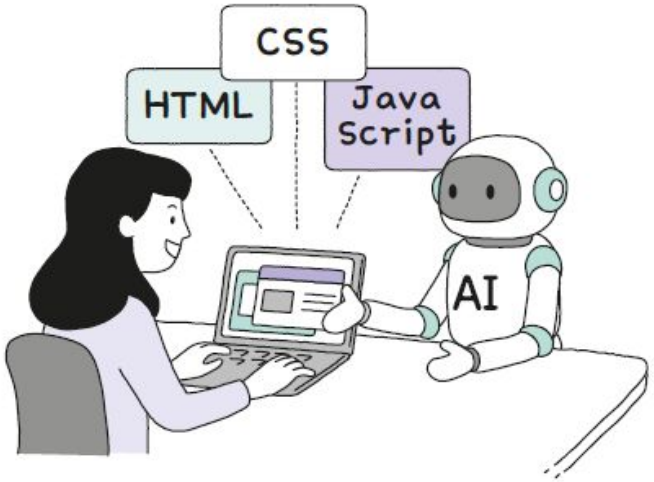


[마무리]

- 3가지 키워드로 정리하는 핵심 포인트
  - **바이브 코딩**은 AI와 자연스럽게 대화하며 코드를 만드는 새로운 개발 방식
  - **모델 선택형 도구**는 여러 언어 모델을 골라 쓸 수 있다는 장점
  - **전용 모델형 도구**는 특정 언어 모델을 설치해 깊이 활용할 수 있다는 장점

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(1)

시작하기전에



- 기본적인 웹페이지의 구성 요소
  - HTML/CSS와 자바스크립트(JavaScript)
  - HTML - 뼈대
  - CSS - 색상과 배치
  - 자바스크립트 - 동작과 상호작용
- AI에게 원하는 기능과 디자인을 구체적으로 제시
  - AI가 이 세 요소를 알아서 조합해 실제로 작동하는 웹페이지 만들기

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(2)

◦ 클로드 시작하기

- 웹 브라우저를 열고 클로드 공식 페이지에 접속한 뒤 로그인
  클로드를 처음 이용한다면 안내에 따라 회원가입이 필요
  - https://claude.ai

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(3)

- 로그인 후의 화면

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(4)

◦ 클로드 아티팩트로 웹페이지 만들기

- 아티팩트(artifacts)
  - 대화창과 별도로 구분되는 패널
  - 단순 텍스트 의사소통을 넘어 실제 작동하는 결과물을 즉시 확인 가능
  - 보고서 작성이나 차트, 간단한 웹페이지 제작에 많이 활용되어 아이디어를 바로 검증하기에 좋음
  - 매 수정마다 이전 버전이 저장되므로 다양한 시도를 부담 없이 해볼 수 있고, 이를 통해 프롬프트를 단계적으로 발전시키는 법을 자연스럽게 익힐 수 있음

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(5)

새 아티팩트 열기

01 클로드 채팅 창의 왼쪽 사이드바에 있는 [아티팩트( )] 아이콘을 클릭

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(6)

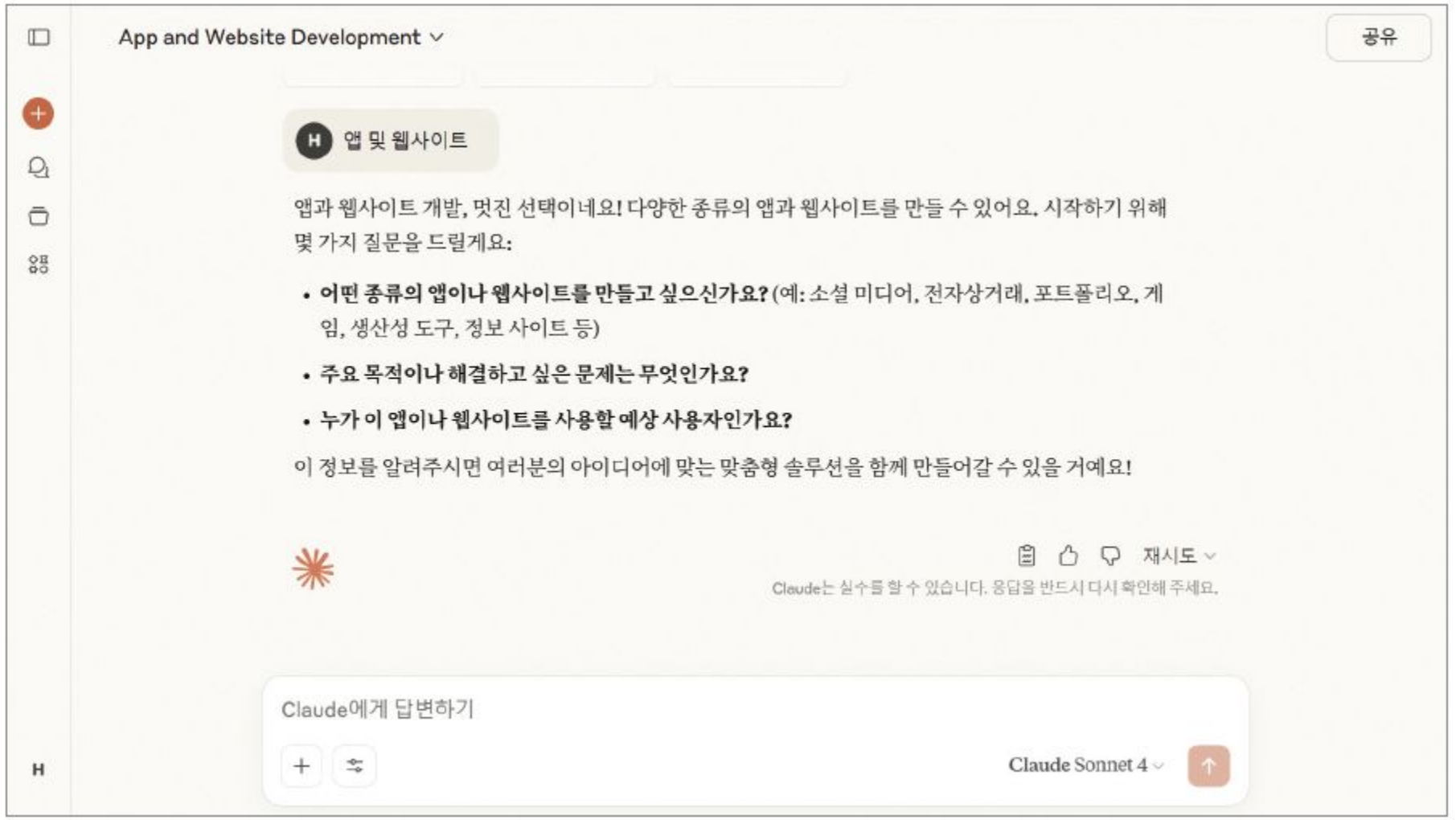
02 [+ 새 아티팩트] 버튼을 클릭

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(7)

03 아티팩트 카테고리 중 [앱 및 웹사이트] 버튼을 클릭

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(8)

04 클로드가 어떤 종류의 앱이나 웹사이트를 만들고 싶은지 질문을 하면,
이제 AI와 대화할 준비가 됨



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(9)

프롬프트 입력하기

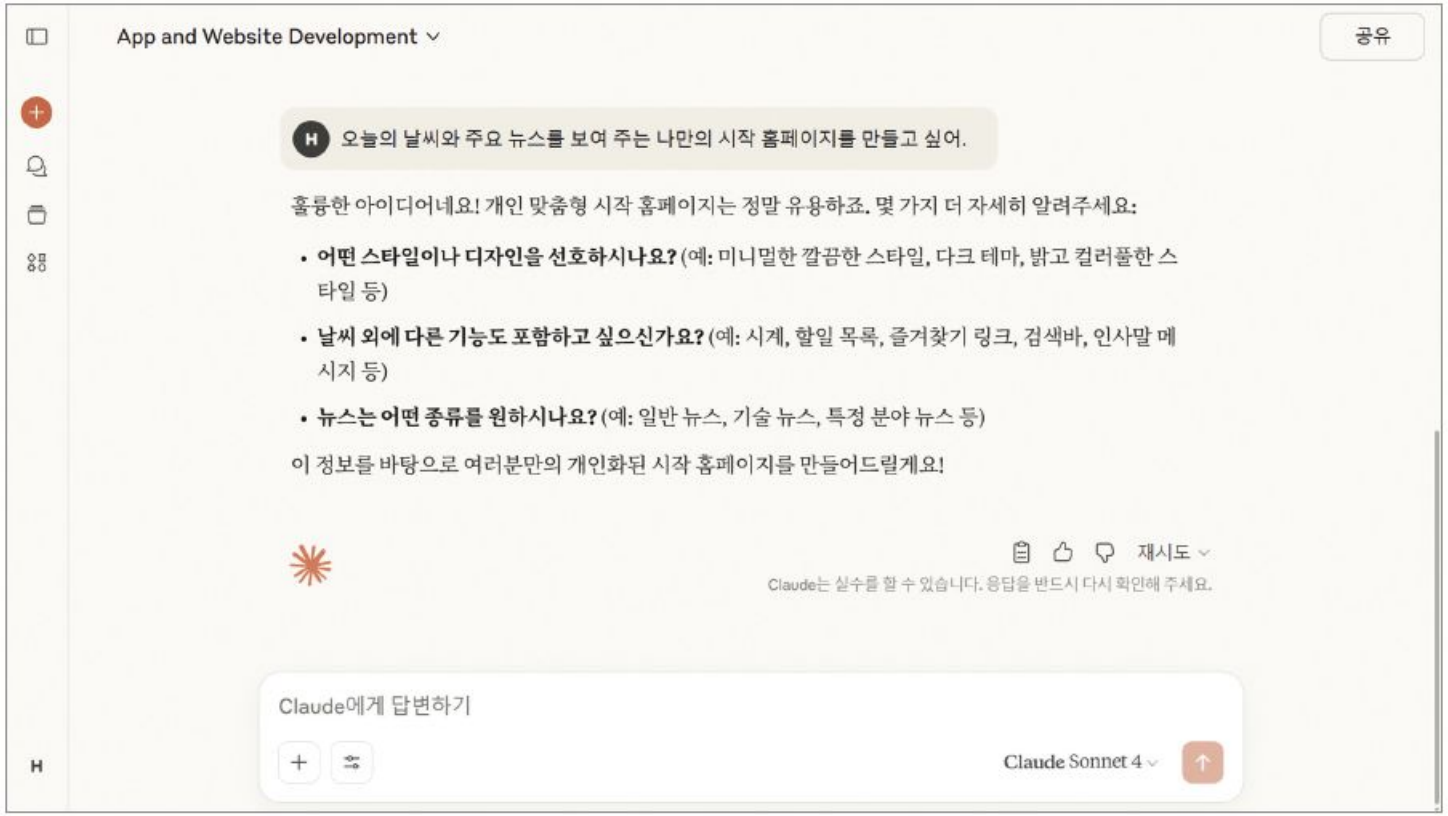
- 프롬프트(prompt)
  - 바이브 코딩의 가장 강력한 장점은 프로그래밍 용어 없이 자연어로 AI에게 코딩을 요청
  - 프롬프트는 AI에게 전달하는 요청이나 지시 사항
  - 개발자가 원하는 결과물의 방향과 품질을 결정하는 핵심 자원
  - 프롬프트가 구체적이고 맥락을 잘 반영할수록 AI는 의도에 맞는 코드와 아이디어를 생성
  - 모호하거나 추상적인 프롬프트는 불필요한 수정과 재작업을 초래하여 효율성이 떨어짐

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(10)

01 채팅창의 프롬프트 입력란에 다음과 같이 입력하고 [Enter] 키를 눌러 요청 프롬프트를 받은 클로드가 답변을 생성

프롬프트

오늘의 날씨와 주요 뉴스를 보여 주는 나만의 시작 홈페이지를 만들고 싶어.



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(11)

02 더 자세한 프롬프트를 요청하고 있음
다음과 같이 추가로 입력한 뒤 [Enter] 키를 누름

프롬프트

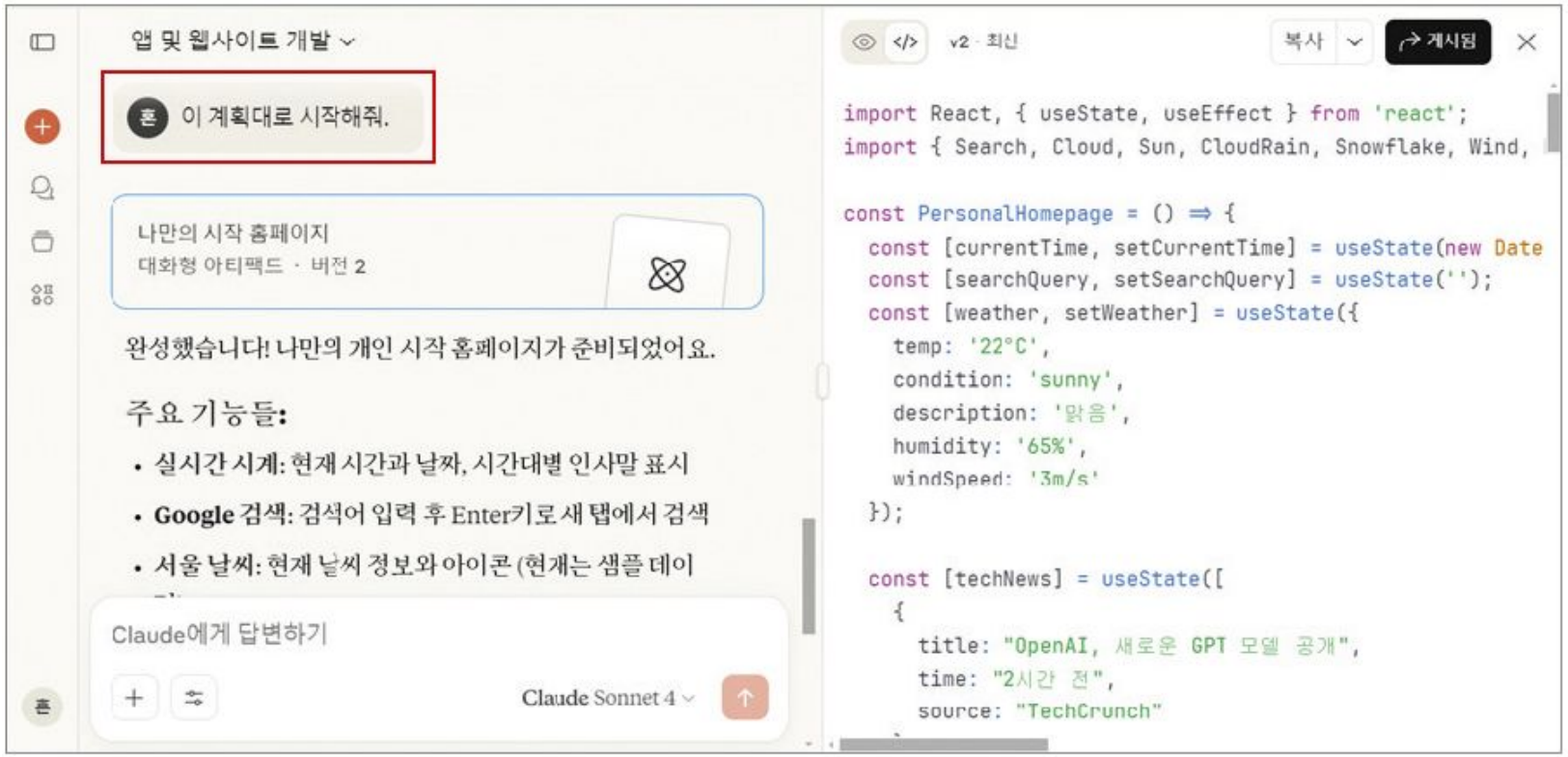
```
밝고 깔끔한 스타일을 선호해. 시계와 검색바도 추가해 줘. 기술 관련 뉴스를 보여 줘.
```

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(12)

03 클로드가 프롬프트를 이해하고 시작 페이지 제작 계획을 제시
계획이 요청한 바와 일치하는지 확인하고, 다음과 같이 프롬프트를 입력해 웹페이지를 제작

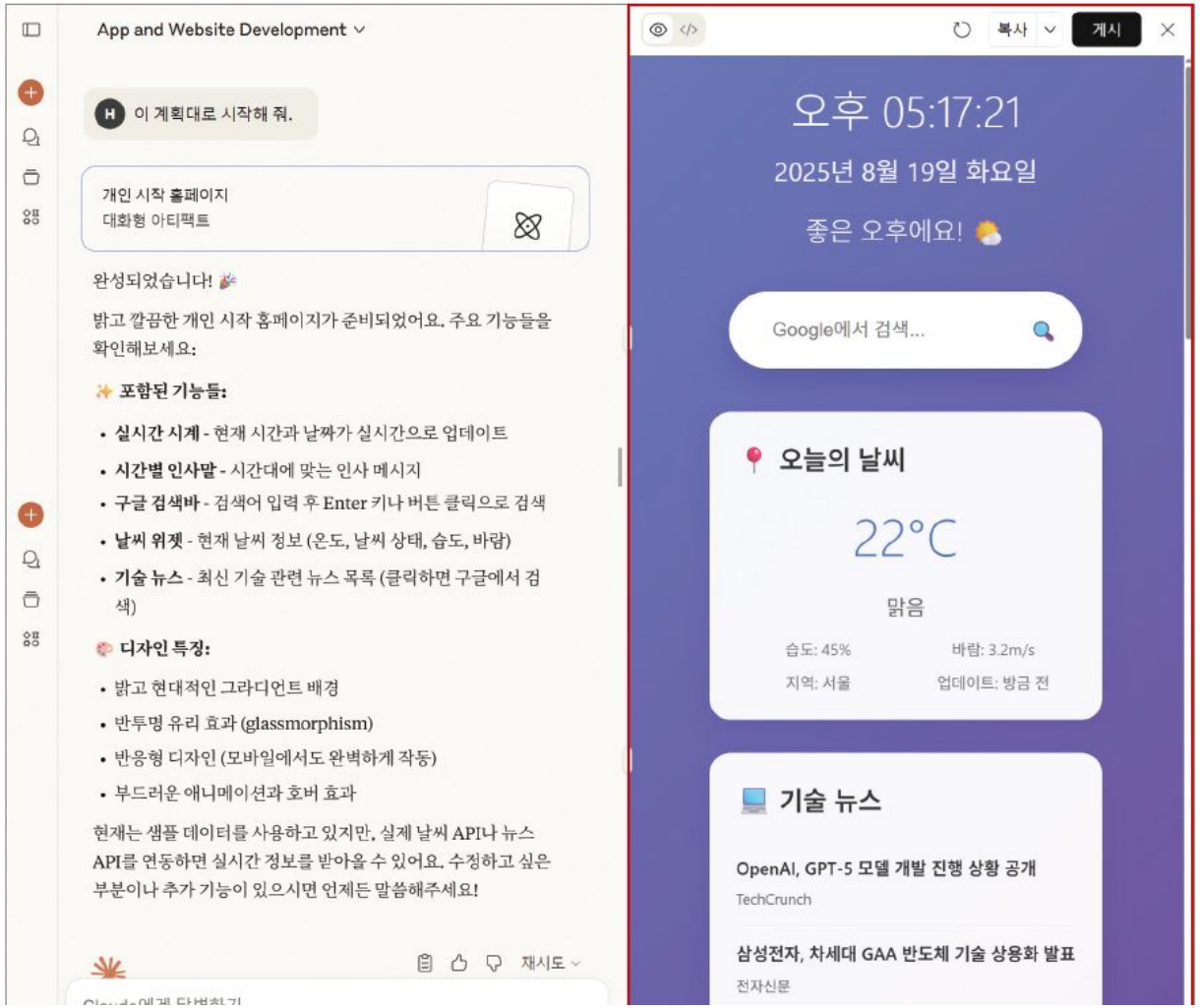
프롬프트

이 계획대로 시작해 줘.



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(13)

04 오른쪽 미리보기 창에 HTML, CSS, 자바스크립트를 작성해 주고, 잠시 기다리면 완성된 시작 페이지가 나타남 - 요청한 대로 시계와 날씨, 검색창, 기술 뉴스가 포함됨

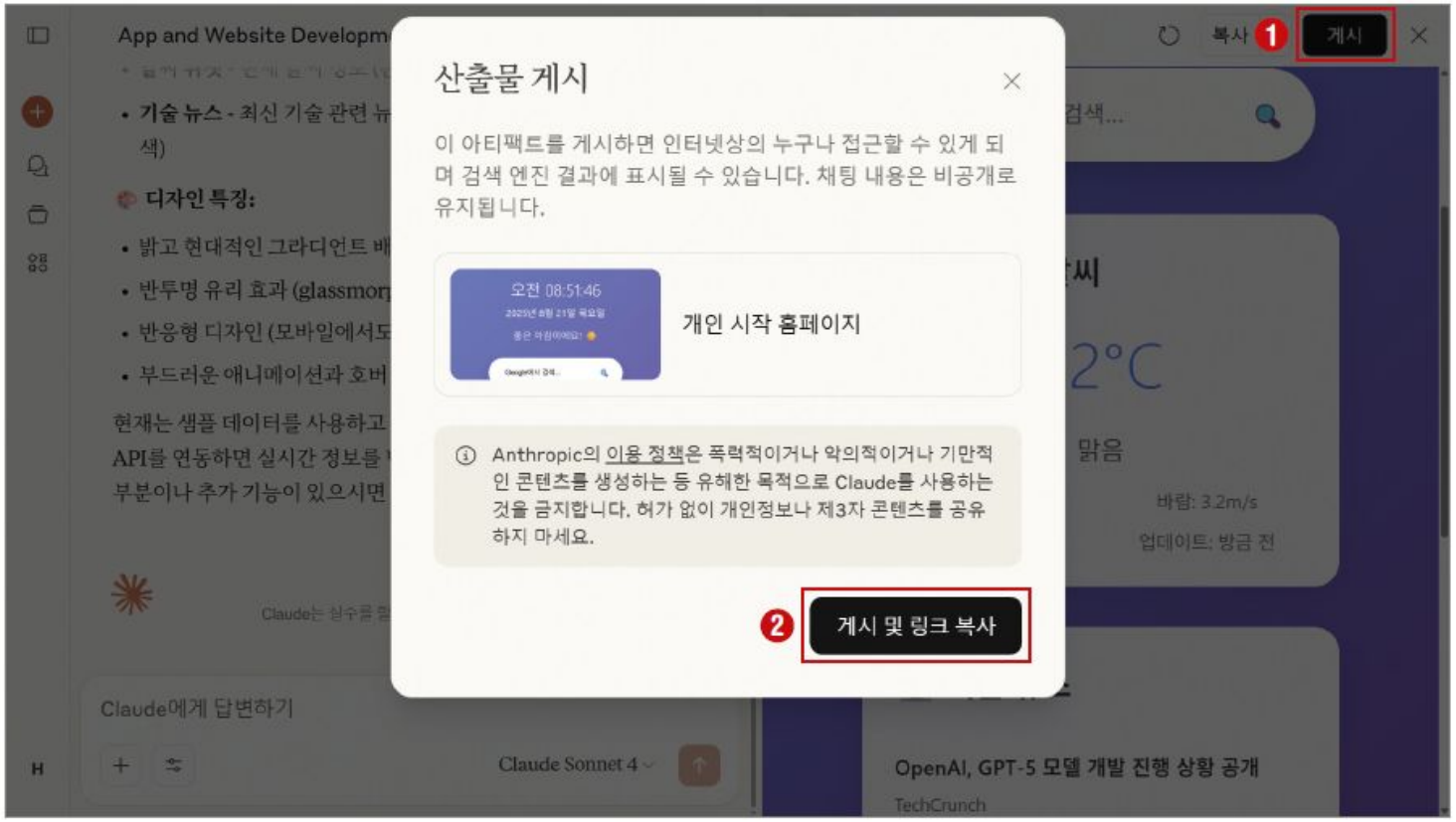


SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(14)

◦ 웹페이지 게시하고 검토하기

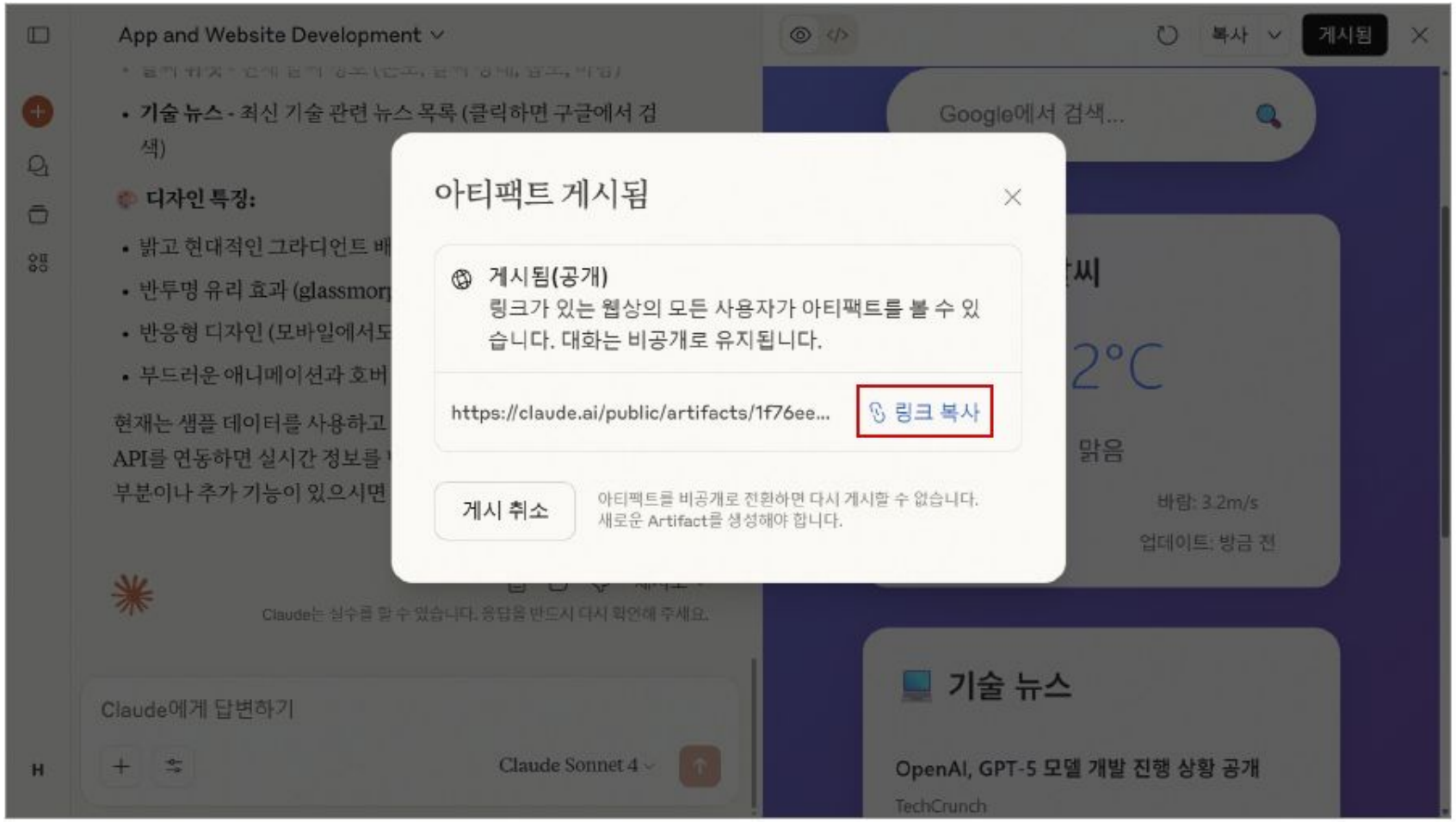
웹페이지 공유하기

01 아티팩트 상단의 [게시] 버튼을 클릭하고, 이어서 나타난 창 하단의 [게시 및 링크 복사] 버튼을 클릭
웹페이지가 게시되면서 링크를 복사할 수 있게 됨



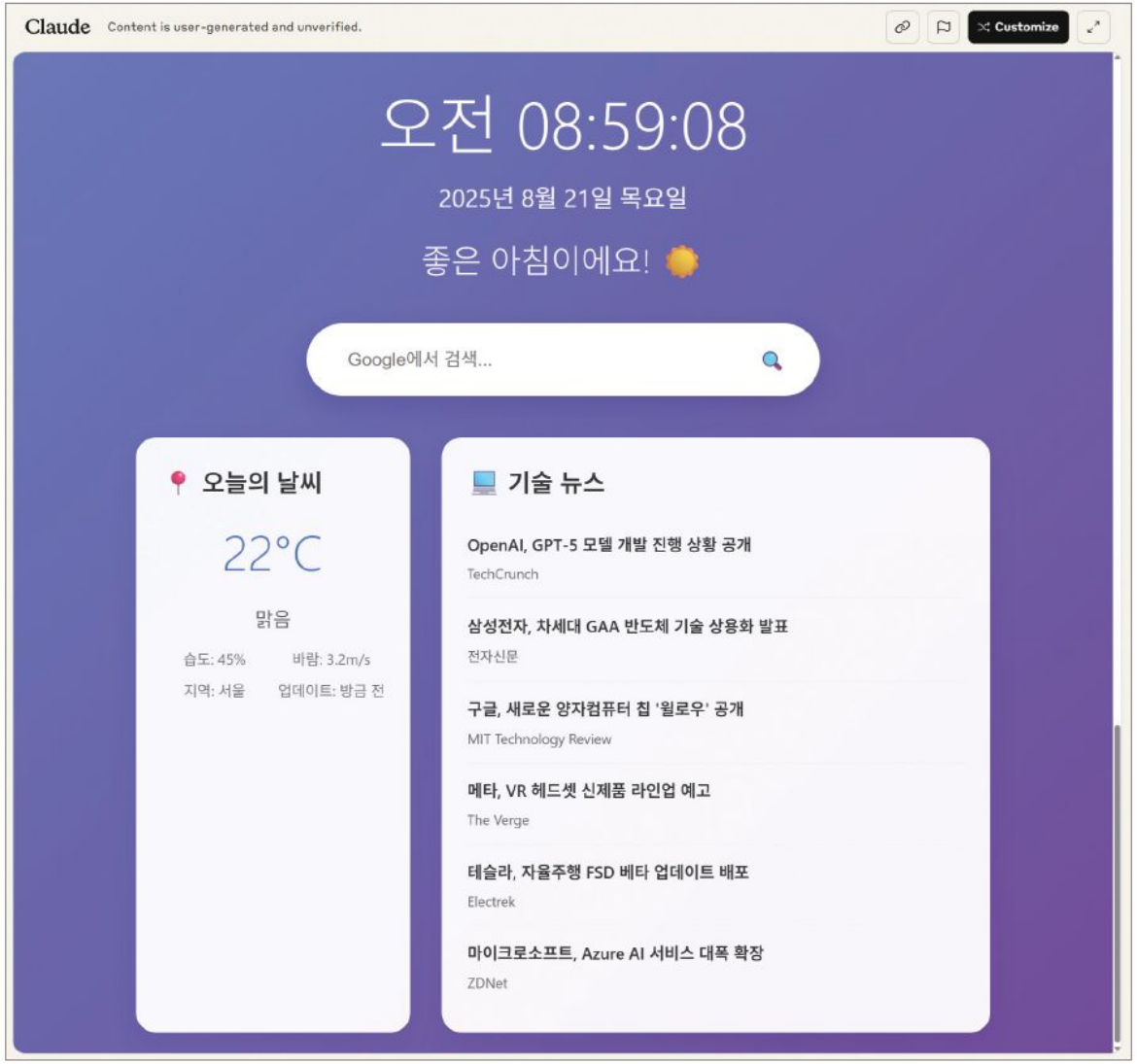
SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(15)

02 이제 링크가 있는 사용자라면 누구나 웹페이지를 방문할 수 있게 됨
[링크 복사] 버튼을 눌러 웹페이지 주소를 복사



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(16)

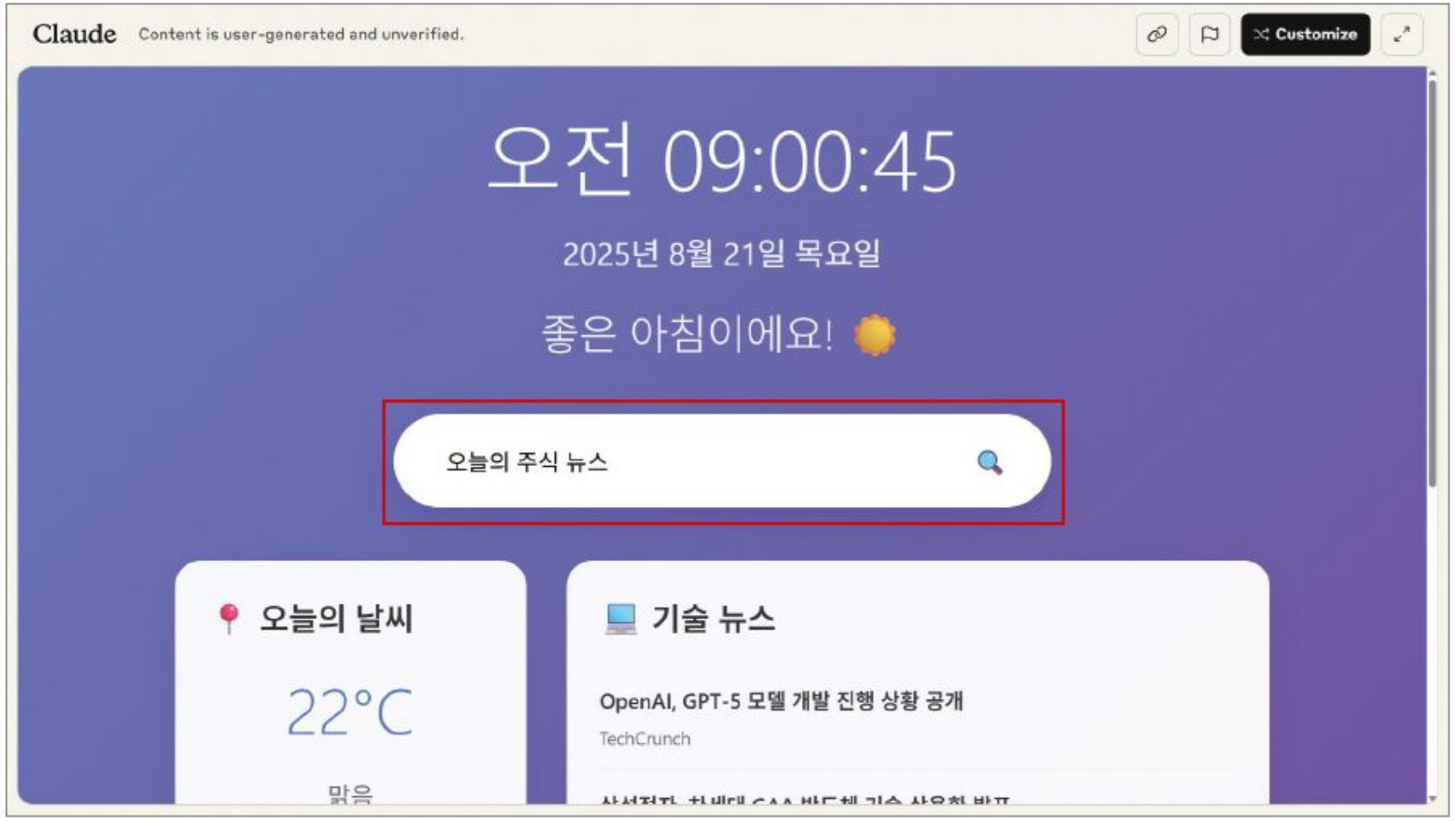
03 복사한 링크를 브라우저의 주소 탭에 붙여 넣으면, 직접 만든 시작 페이지를 볼 수 있음



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(17)

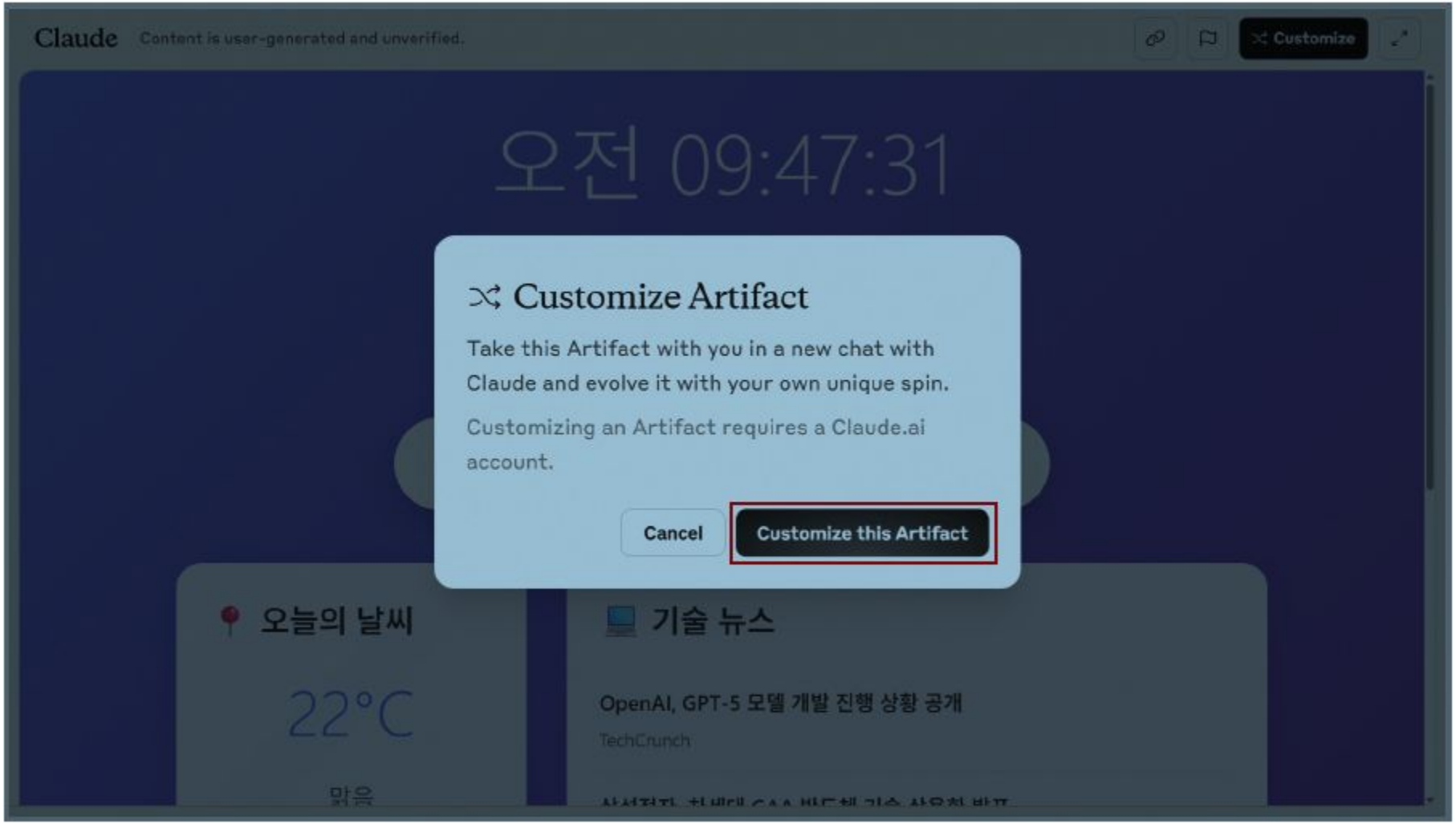
검색 기능 확인하기

01 홈페이지의 검색창에 무언가를 입력하고 [Enter] 키를 누름



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(18)

02 이어서 나타난 창에서 [Customize this Artifact] 버튼을 클릭



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(19)

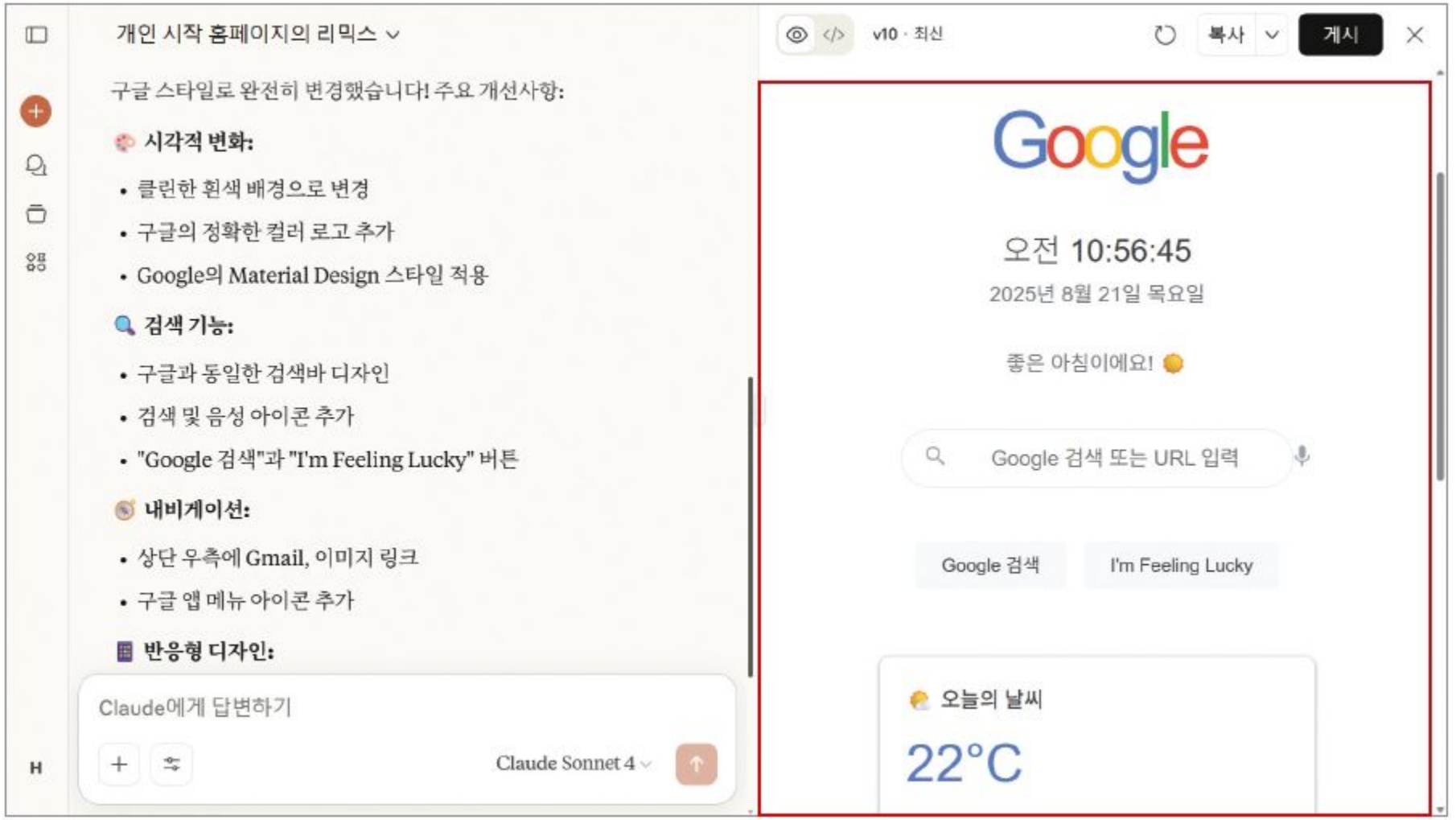
03 클로드 대화창으로 돌아오면 채팅을 통해 웹페이지의 디자인을 수정할 수 있음
프롬프트 입력란에 다음과 같이 요청 - 잠시 기다리면 클로드가 웹페이지의 스크립트를 수정

프롬프트

구글 스타일로 디자인을 수정해 줘.

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(20)

04 잠시 기다리면 미리보기 창에서 수정된 웹페이지를 확인



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(21)

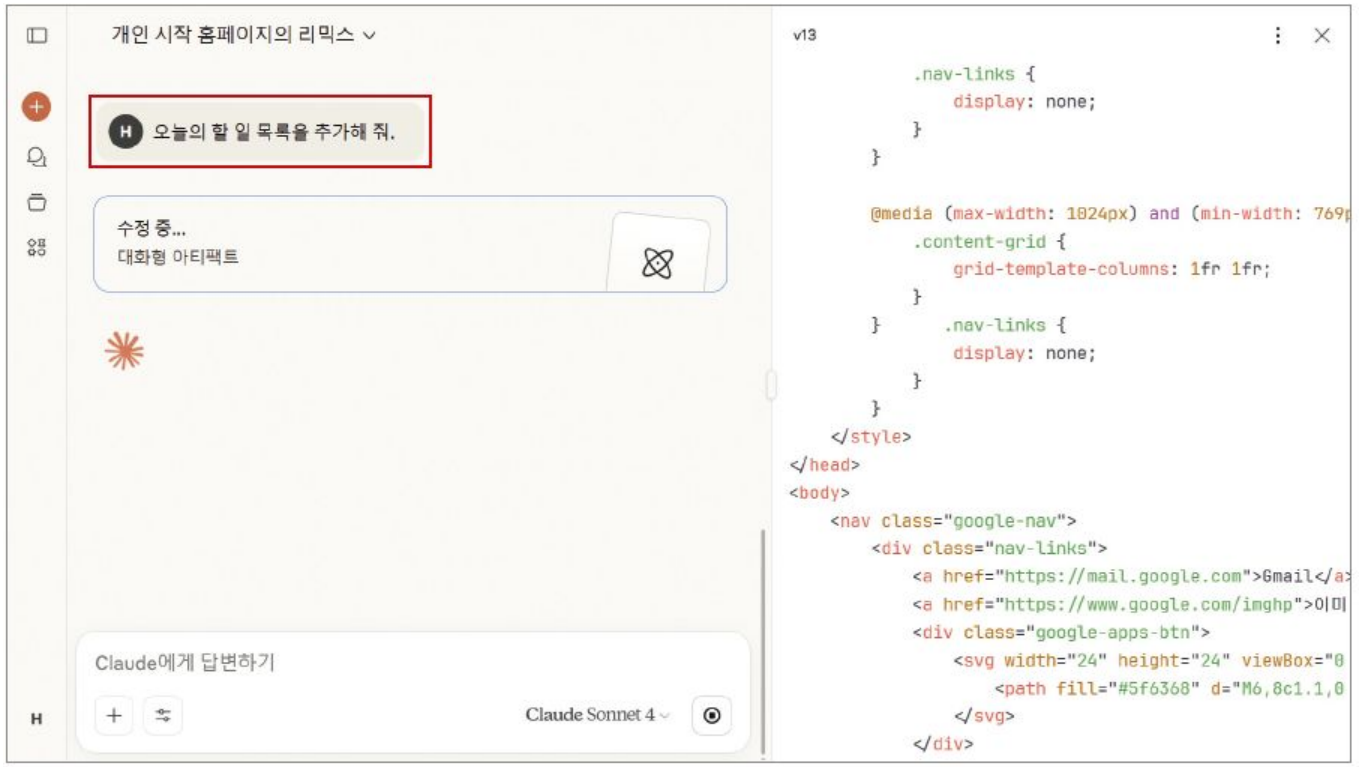
기능 추가하기

- 해야 할 일을 메모한 투두 리스트(to-do list) 추가

01 프롬프트 입력 창에 다음과 같이 입력해서 [오늘의 할 일] 목록 추가를 요청

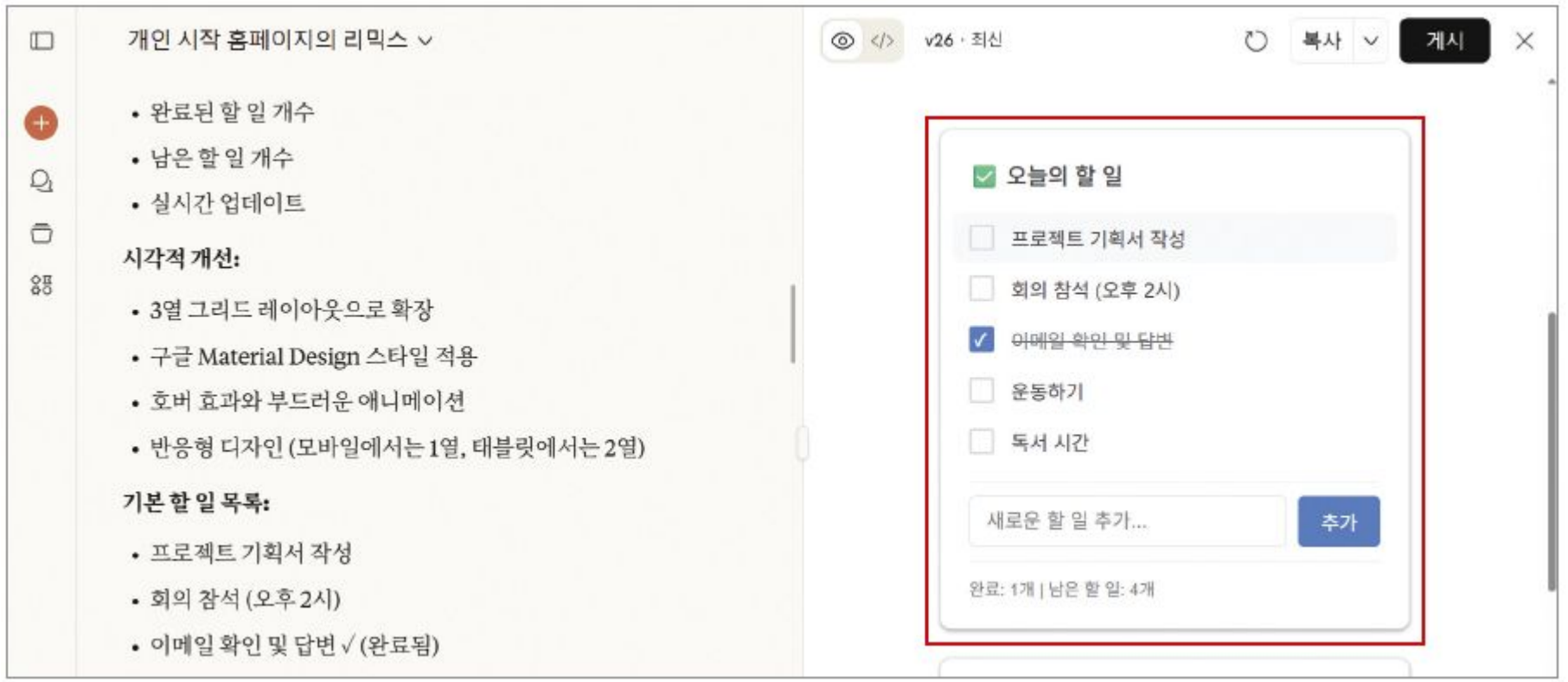
프롬프트

오늘의 할 일 목록을 추가해 줘.



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(22)

02 [오늘의 할 일] 목록이 추가됨



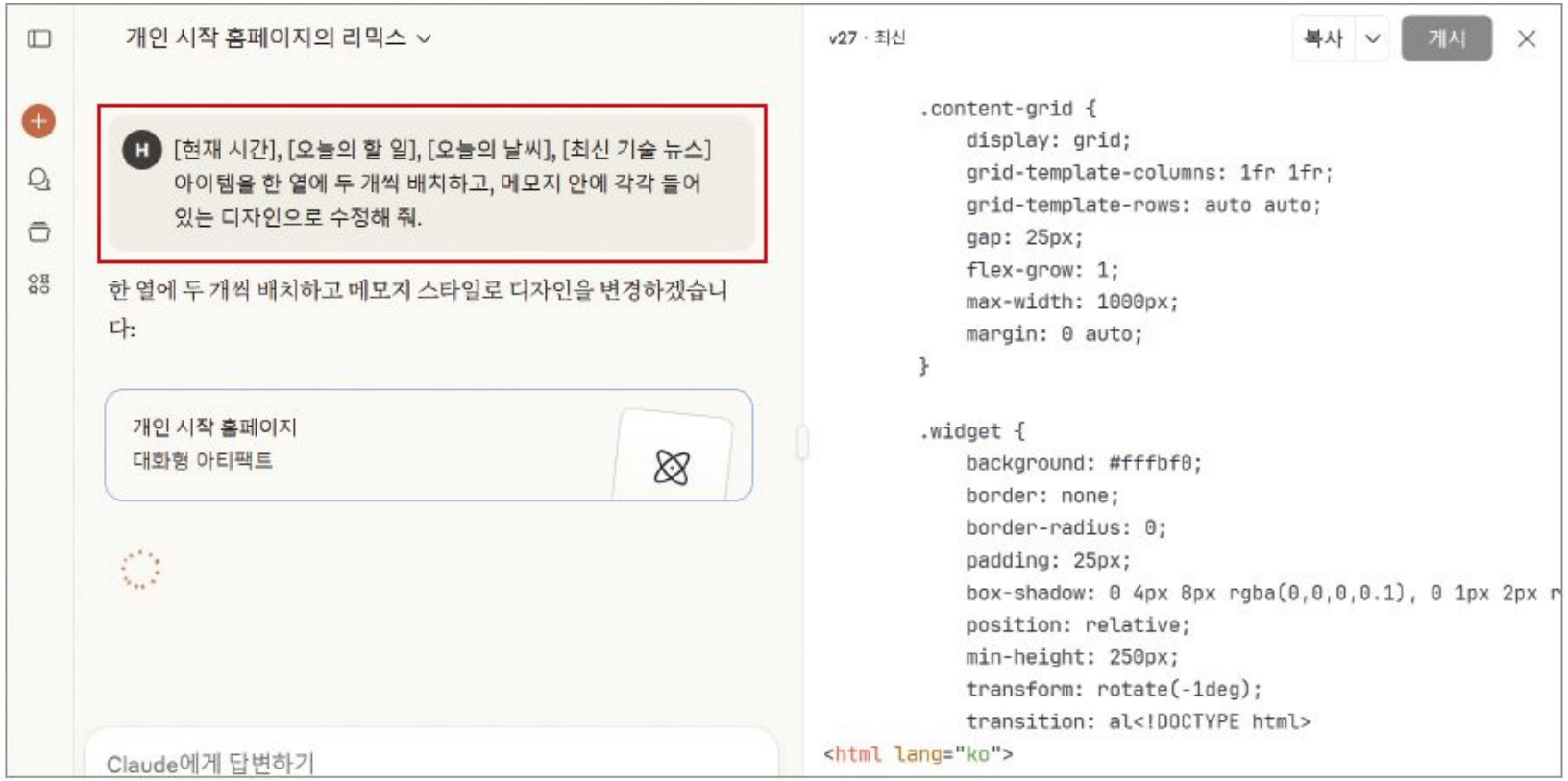
SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(23)

03 세로로 긴 웹페이지의 모든 위젯을 한눈에 보기 쉽도록 디자인을 수정

프롬프트 입력창에 다음과 같이 요청

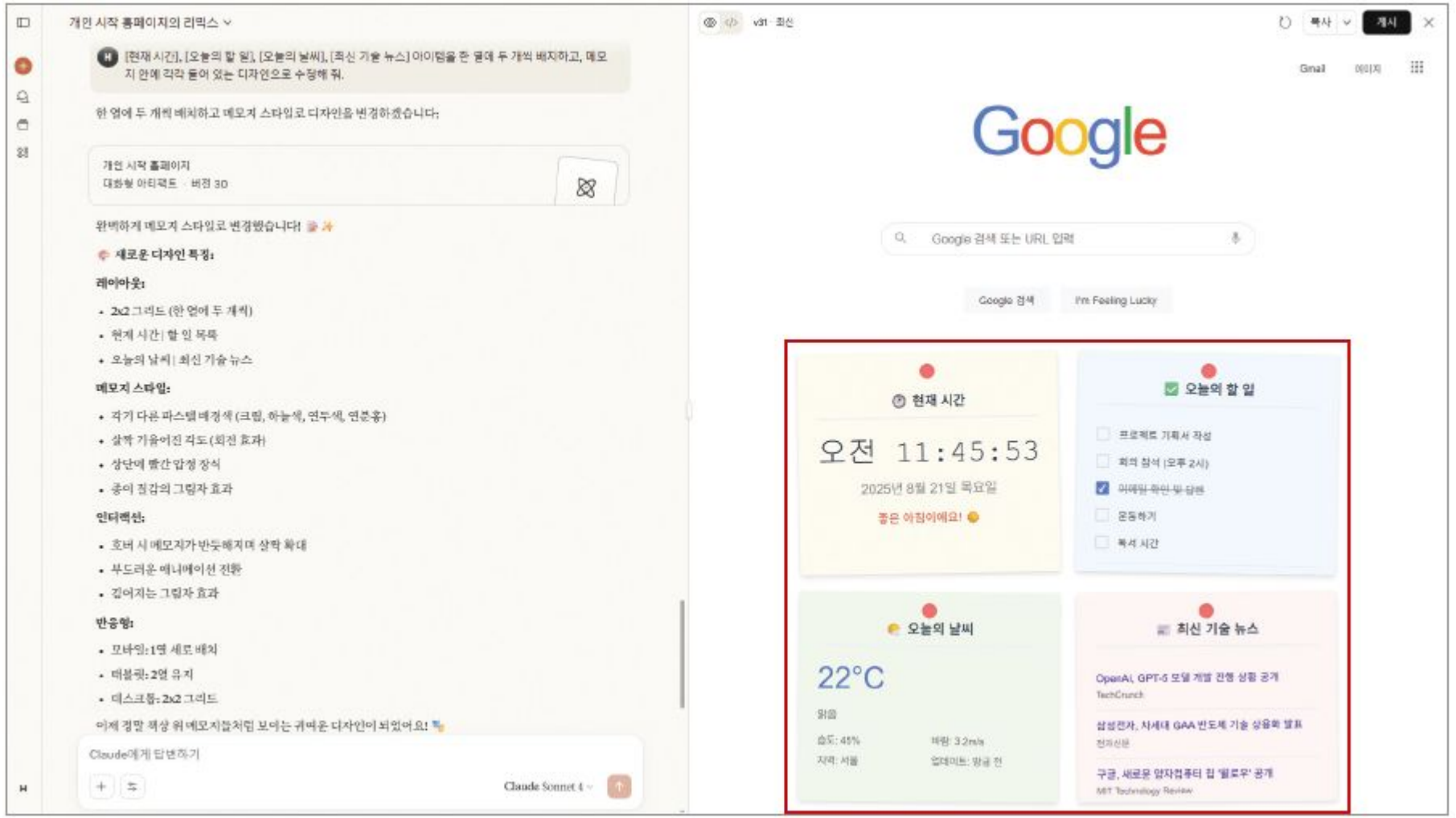
프롬프트

[현재 시간], [오늘의 할 일], [오늘의 날씨], [최신 기술 뉴스] 아이템을 한 열에 두 개씩 배치하고, 메모지 안에 각각 들어 있는 디자인으로 수정해 줘.



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(24)

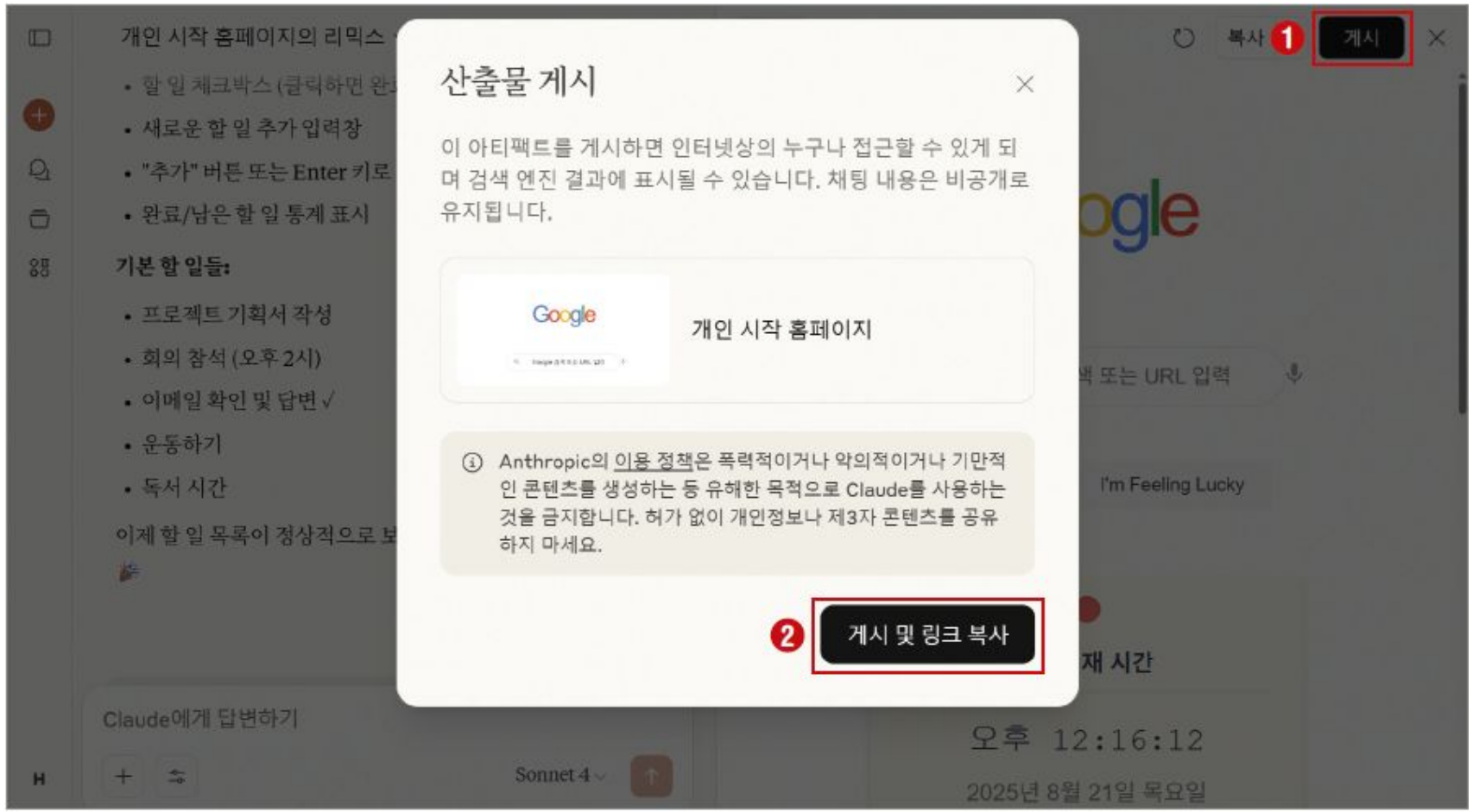
04 깔끔하게 완성



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(25)

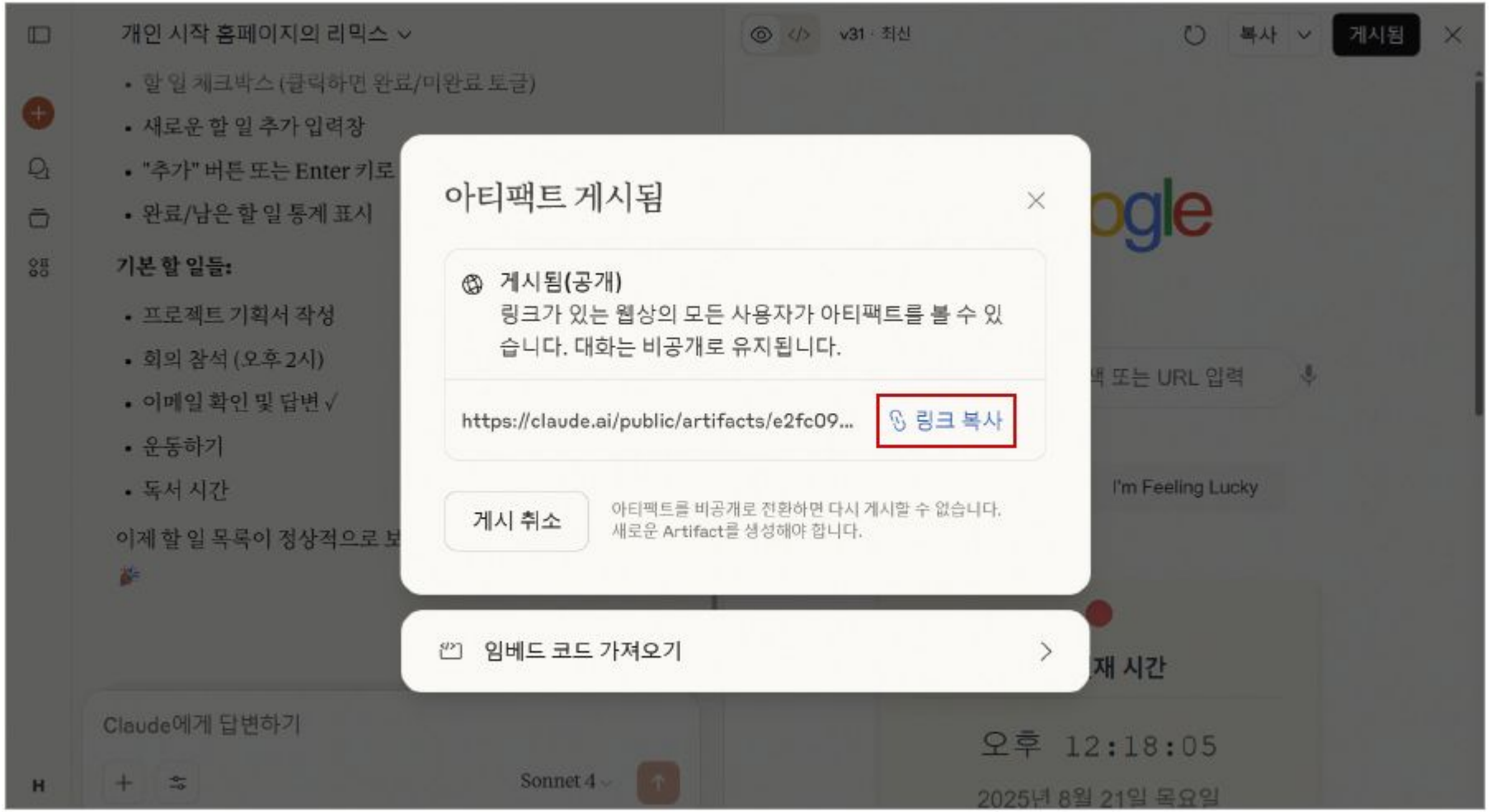
게시하기

01 클로드 아티팩트 상단의 [게시] 버튼과 [게시 및 링크 복사] 버튼을 차례로 클릭



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(26)

02 게시된 웹페이지의 링크를 복사하기 위해 [링크 복사]를 클릭



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(27)

03 웹 브라우저에 복사한 링크를 붙여 넣음 - 시작 페이지 완성

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(28)

✚ 여기서 잠깐 **프롬프트의 중요성**

- “홈페이지 만들어 줘”라는 막연한 요청보다 “서울 날씨와 기술 뉴스를 보여주는 시작 페이지”라는 구체적인 요청이 더 만족스러운 결과를 가져옴
- AI가 우리의 의도를 정확히 파악하려면 구체적인 지시가 필요
- 효과적인 프롬프트 작성법 - AI의 잠재력을 최대한 끌어내는 전략적 소통

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(29)

좀더알아보기 **클로드 구독으로 더 강력한 AI 파트너 만들기**

- 지금까지의 실습은 클로드 무료(Free) 플랜으로도 충분히 따라 할 수 있도록 구성
- 무료 플랜은 사용량이 제한적
  - 하루 약 40~50개 이상의 메시지를 주고받은 뒤에는 대화가 중단될 수 있음
  - 트래픽이 몰리는 시간대에는 사용 우선순위가 밀리면서 속도가 느려지거나 접속이 제한
- 2장부터는 더 복잡한 프로젝트의 원활한 학습을 위해 Pro 이상의 클로드 유료 플랜 구독을 권장
  - 긴 대화도 끊김 없이 진행할 수 있고, 이미지 파일 업로드, 더 정교한 코드 생성 등이 가능
  - 트래픽이 많은 시간대라도 우선 접근 권한이 있어 언제든 대화

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(30)

클로드 유료 플랜 구독하기

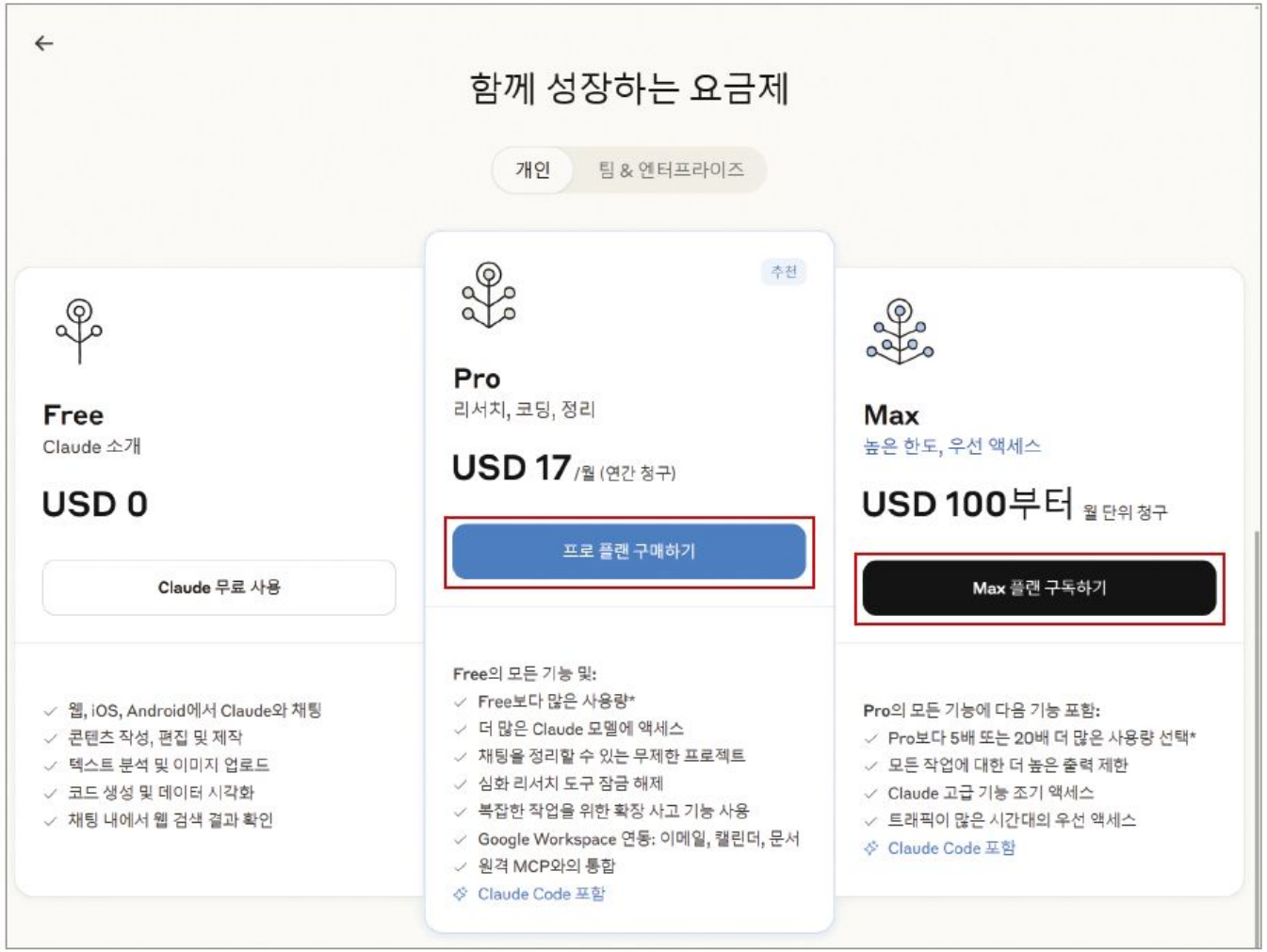
01 클로드 대화창 하단의 프로필을 클릭한 뒤 나타나는 목록에서 [요금제 업그레이드] 메뉴를 선택

- **Pro 플랜 - 클로드 코드를 포함한 모든 기능을 써볼 수 있는 모델**
- **Max 플랜 - 조금 더 큰 프로젝트나 긴 대화를 위한 모델**
- **Max Ultimate 플랜 - 클로드를 거의 무제한으로 사용**

SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(31)

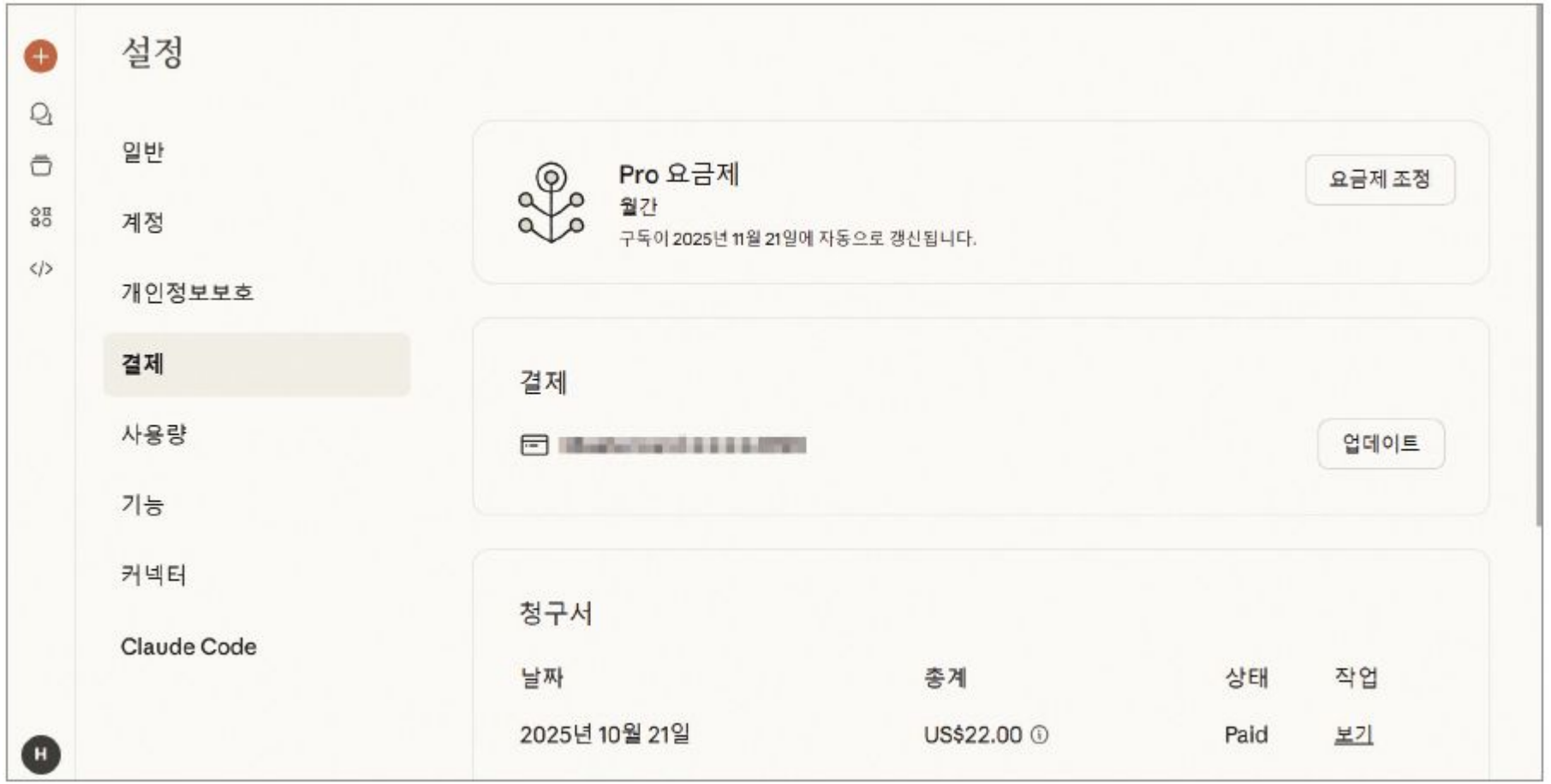
02 Pro 또는 Max 버전을 선택

Pro로 시작하는 경우 [프로 플랜 구매하기], Max를 구매할 경우 [Max 플랜 구독하기] 버튼을 클릭



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(32)

03 결제 정보를 입력하면 구독이 완료



SECTION 1-2 AI로 나만의 첫 웹페이지 만들기(33)

유료 플랜 비교

	Free	Pro	Max	Max Ultimate
월 단위 요금	$0	$20	$100	$200
사용량	제한적 사용량	Free 대비 약 5배	Pro 대비 약 5배	Pro 대비 약 20배
특징	고급 모델 선택 불가	모델 선택 가능, 클로드 코드 사용 가능	Pro 기능 + 우선 접근, 웹 검색 사용 가능	Max 기능 + 최대 사용량
적합한 사용자 유형	가끔 가벼운 대화를 시도하는 사용자	정기적으로 클로드를 업무에 활용하는 일반 사용자	클로드 코드를 통해 다양한 주제, 길게 이어지는 작업을 하려는 사용자	거의 무제한으로 클로드 코드를 사용하려는 사용자

[마무리]

- 4가지 키워드로 정리하는 핵심 포인트
  - **아티팩트** - 클로드가 코드를 생성하고 미리보기를 제공하는 작업 공간
  - **프롬프트** - AI에게 전달하는 요청이나 지시 사항
  - **게시 기능** - 만든 작품을 온라인에 공유하고 링크를 생성하는 기능
  - **커스터마이즈** - 기존 디자인을 바탕으로 추가 수정하는 과정

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(1)

시작하기전에

- AI는 방대한 언어 데이터를 학습한 거대한 지식 저장소
  - 이 거대한 지식 창고와 소통할 수 있는 통로는 작은 프롬프트 창 하나뿐
  - 프롬프트는 내비게이션에 목적지를 입력하는 것과 비슷함
    - “좋은 데로 데려다 줘”라는 명령어에는 정확한 안내를 받을 수 없지만,
      “서울역에서 강남역까지 가장 빠른 길”이라고 입력하면 효율적인 루트를 통해 목적지에 도달
- 제한된 언어로 AI의 잠재력을 최대한 끌어내는 프롬프트
  - 명확하고 구체적인 프롬프트 작성
  - 잠들어 있는 AI라는 거인을 깨우는 첫 번째 주문은 바로 잘 만들어진 프롬프트

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(2)

◦ 프롬프트의 중요성

좋은 프롬프트의 조건

- 명확성: 모호한 표현을 피하고, 원하는 결과를 구체적으로 전달
- 구체성: 결과물에 필요한 기능이나 조건을 세부적으로 지시
- 맥락 제공: 왜 필요한지, 어떤 상황에서 활용되는지를 알려야 AI가 더 정밀한 답변을 할 수 있음

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(3)

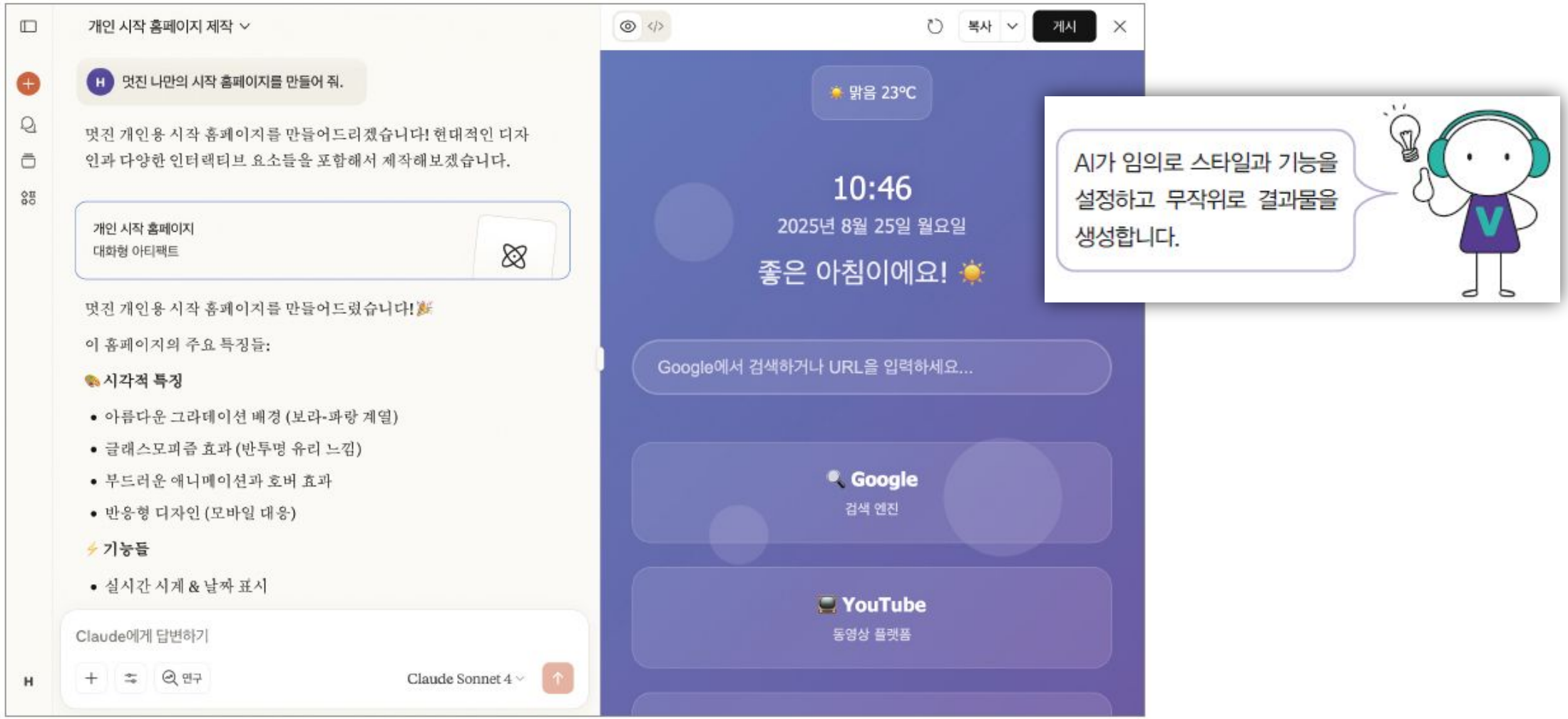
모호한 프롬프트의 한계

프롬프트

멋진 나만의 시작 홈페이지를 만들어 줘.

- '멋지다'는 표현이 지나치게 모호함
  - 세련된 디자인을 원하는지, 강력한 기능을 원하는지, 아니면 둘 다 포함하는 것인지 알 수 없음
  - 막연한 요청을 받은 AI는 수백 가지 가능성 중에서 무엇을 선택해야 할지 갈피를 잡지 못함
- 구체적인 요구사항이 빠져 있으면, AI가 모든 요소를 임의로 결정할 수밖에 없고, 사용자의 기대나 취향과 다른 결과물이 나올 가능성이 큼

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(4)



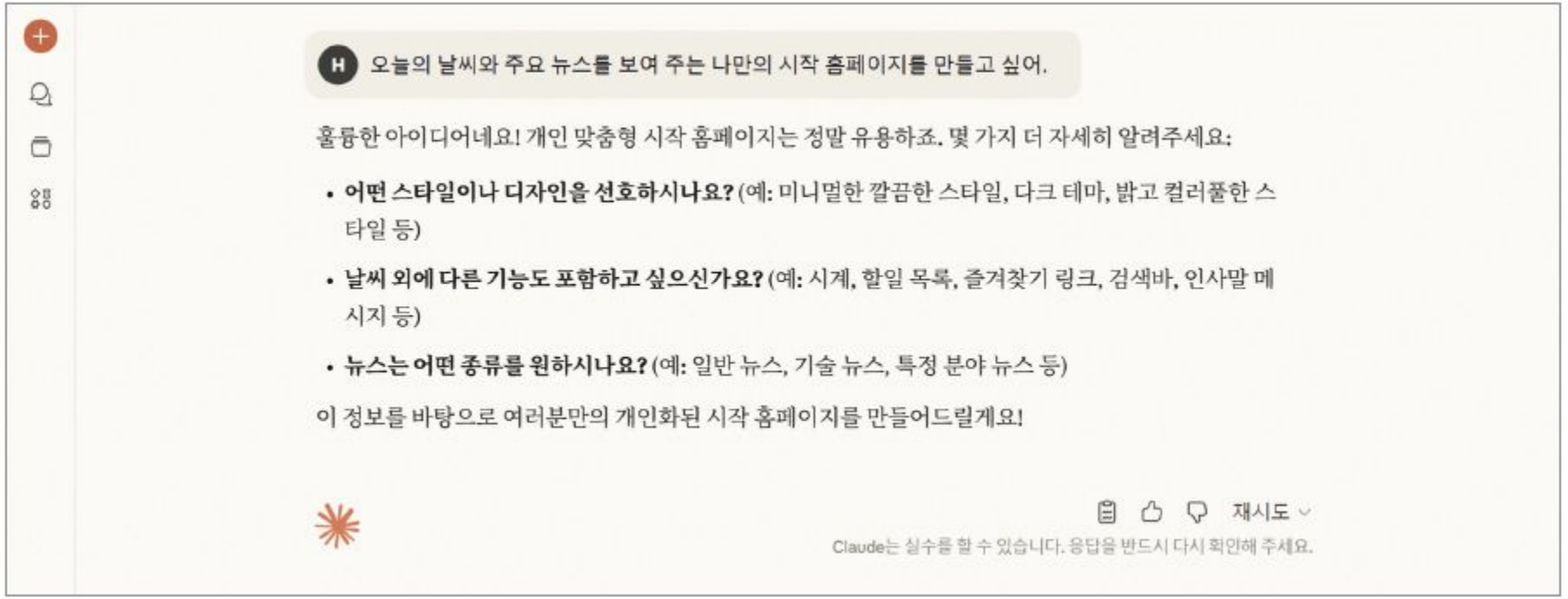
SECTION 2-1 AI를 깨우는 프롬프트의 비밀(5)

AI와 프롬프트 개선하기

프롬프트

오늘의 날씨와 주요 뉴스를 보여 주는 나만의 시작 홈페이지를 만들고 싶어.

- '더 구체적으로 쓰고 싶지만, 무엇을 어떻게 지시해야 할지 막막하다'할 때
  - AI가 스스로 추가 정보를 되물으며 프롬프트를 보완할 기회를 제시

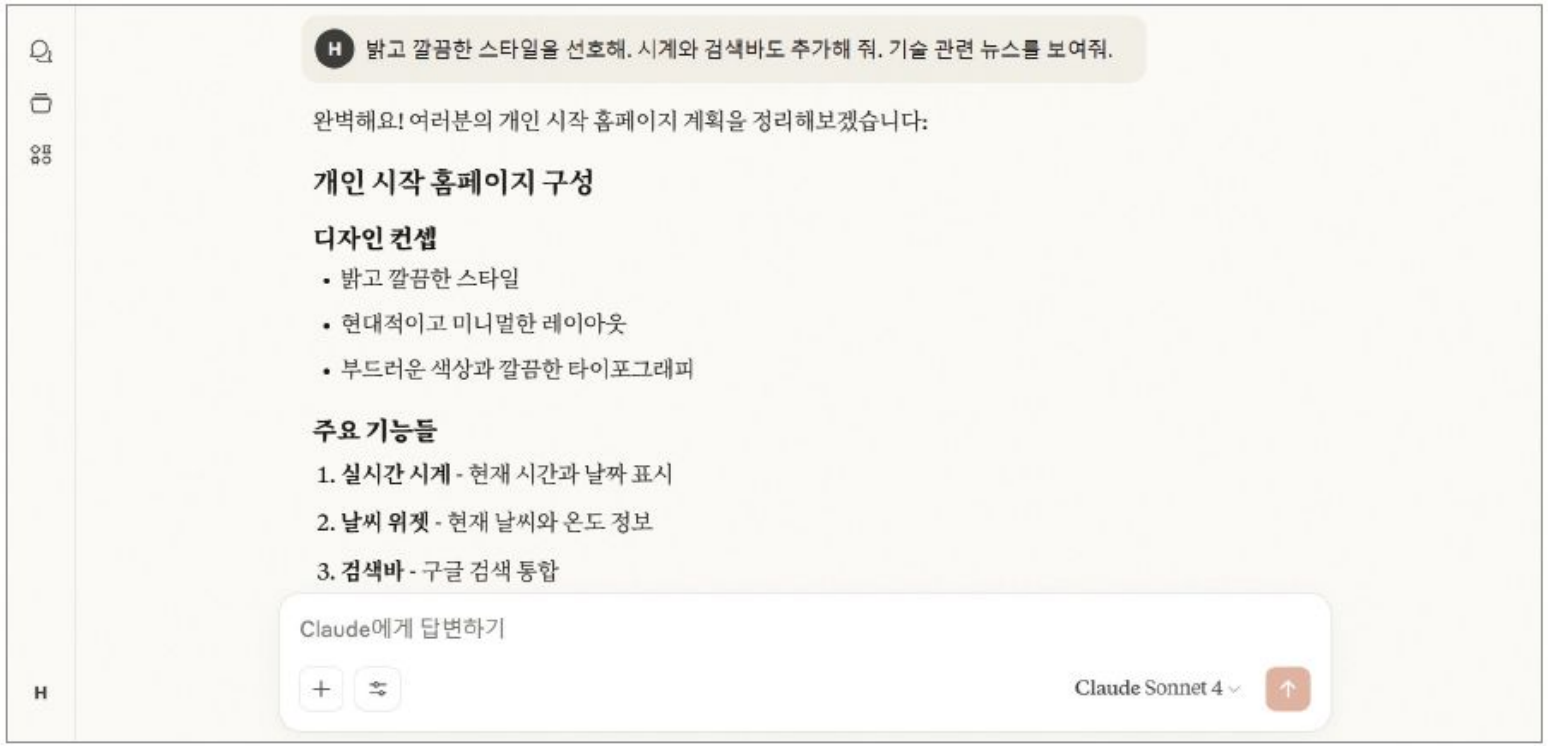


SECTION 2-1 AI를 깨우는 프롬프트의 비밀(6)

- 세부 조건을 추가하여 완성한 프롬프트
  - ‘서울 날씨’, ‘기술 뉴스’, ‘시계와 검색창’ 등
  - 완성된 프롬프트는 훨씬 구체적이고 명확해짐

프롬프트

밝고 깔끔한 스타일을 선호해. 시계와 검색바도 추가해 줘. 기술 관련 뉴스를 보여 줘.



SECTION 2-1 AI를 깨우는 프롬프트의 비밀(7)

◦ 제품 기획서 초안 작성하기

- PRD(Product Requirements Document) - 제품 기획서

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(8)

기획의 핵심 요소

- 5W1H
  - (목적) Why: 왜 만드는가?
  - (대상) Who: 누구를 위한 것인가?
  - (기능) What: 무엇을 만들 것인가?
  - (사용맥락) When: 언제 사용되는가? Where: 어디서 사용되는가?
  - (구현 방법) How: 어떻게 만들 것인가?

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(9)

대화하며 완성하는 제품 기획서

- 기본 질문 템플릿의 요소
  - 5W1H의 모든 요소가 적절히 포함
  - 구현 방향을 구체적으로 요구
  - AI를 위해 구체적인 기능을 기술적으로 설명

- 효과적인 프롬프트를 위한 기본 질문 템플릿

```
[무엇]을 만들려고 해.
주요 기능은 [구체적인 기능 설명]이야.
[누구]를 위한 것이고,
[왜/어떤 문제를 해결]하기 위해 필요해.
어떻게 구현하면 좋을지 기술적인 방향과 함께 구체적인 PRD를 작성해 줘.
```

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(10)

- 기본 질문 템플릿을 기반으로 작성한 프롬프트의 예시
  - 취업을 준비 중인 프리랜서 마케터이고, 포트폴리오 웹페이지가 필요하다고 가정

프롬프트

포트폴리오 웹페이지를 만들려고 해. ← 무엇

주요 기능은 채용 담당자가 30초 안에 내 역량을 파악할 수 있는 임팩트 있는 구성이야. ← 구체적인 기능 설명

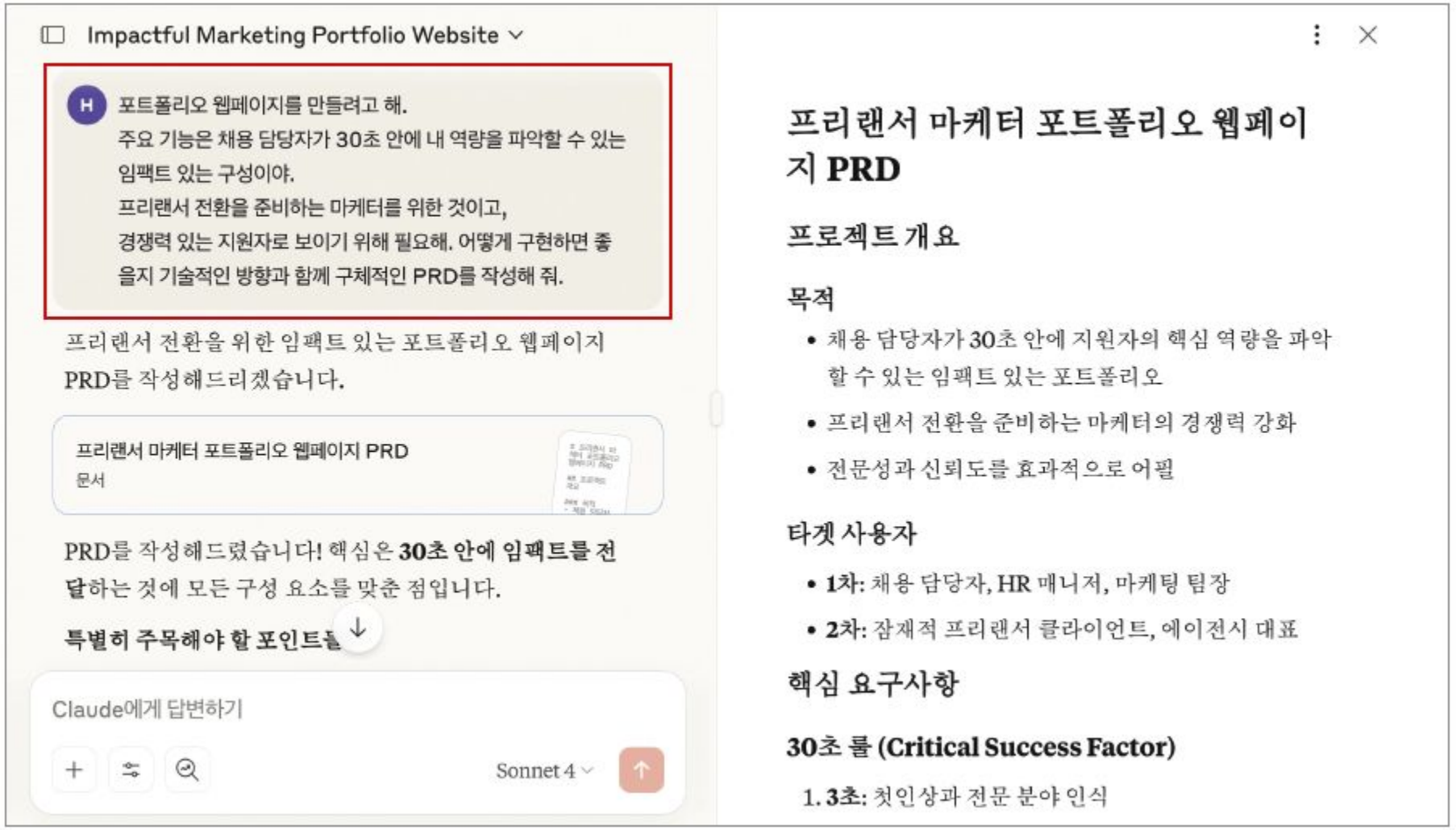
프리랜서 전환을 준비하는 마케터를 위한 것이고, ← 누구

경쟁력 있는 지원자로 보이기 위해 필요해. ← 왜/어떤 문제를 해결

어떻게 구현하면 좋을지 기술적인 방향과 함께 구체적인 PRD를 작성해 줘. ← 구현 방안

SECTION 2-1 AI를 깨우는 프롬프트의 비밀(11)

- 앞의 내용을 클로드에 입력



[마무리]

◦ 4가지 키워드로 정리하는 핵심 포인트

- **프롬프트**는 AI와 소통하는 유일한 창구이자, AI의 성능을 좌우하는 핵심
- Why, Who, What, When, Where, How로 구성된 체계적 사고의 틀을 줄여서 **5W1H**
- 개발자들이 사용하는 제품 기획서를 **PRD**라고 함
- **템플릿**은 효과적인 프롬프트 작성을 위해 재사용 가능한 질문 구조

[확인 문제]

1. 다음 중 좋은 프롬프트의 조건이 아닌 것을 고르세요.
    1. 명확성: 원하는 결과를 구체적으로 전달한다.
    2. 구체성: 필요한 기능이나 조건을 세부적으로 지시한다.
    3. 창의성: 결과물이 놀라움을 주도록 요구한다.
    4. 맥락 제공: 왜 필요한지, 어떤 상황에서 활용되는지 알려 준다.

2. 다음 빈칸을 채우세요.

> 프롬프트와 PRD는 모두 ( )원칙을 기반으로 한다.
> 이 원칙은 Why, Who, What, When, Where, How 여섯 가지 질문으로 구성된다.

[확인 문제]

3. 소프트웨어 개발을 위한 PRD에서 특히 강조되어야 하는 두 가지 질문은 무엇인가요?
    1. Why(왜), Who(누구)
    2. What(무엇을), How(어떻게)
    3. When(언제), Where(어디서)
    4. Who(누구), Where(어디서)
4. 아래 두 프롬프트 중 더 좋은 것은 어느 것인가요? 그 이유는 무엇인가요?
    1. “멋진 나만의 홈페이지를 만들어 줘.”
    2. “오늘의 날씨와 주요 뉴스를 보여 주는 나만의 시작 홈페이지를 만들고 싶어.”

SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(1)

시작하기전에

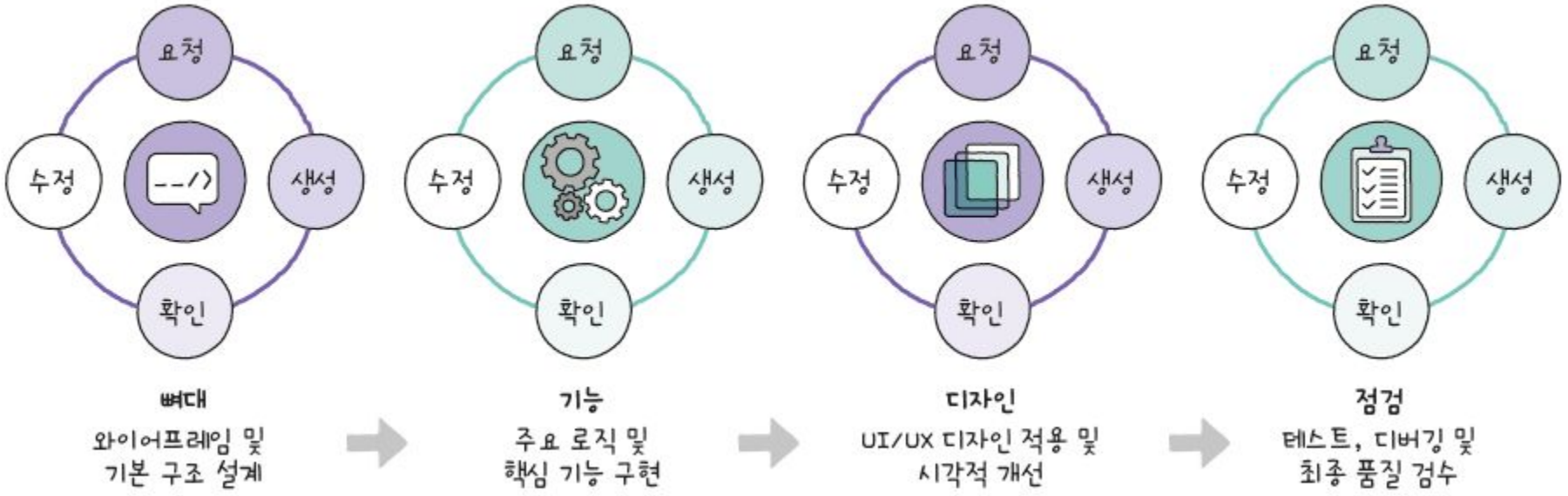
- HTML은 섹션(section)으로 이루어진 문서
  - <section> 태그를 이용해 각각의 구역을 나눔
  - 실습에서는 PRD의 내용을 기반으로 이런 섹션을 먼저 나눈 뒤, 섹션 단위로 웹페이지를 완성
  - AI에게 많은 요구사항을 한 번에 코드로 변환하면 HTML, CSS, 자바스크립트가 뒤섞인 복잡한 코드가 생성될 가능성이 높음
    - PRD의 요구사항을 기능별로 분해하여 단계적으로 구현
    - 각 단계에서 명확한 목표를 달성하고 검증
- 단계별 프롬프트를 준비하고, 순차적으로 진행

```
주어진 PRD를 클로드 코드에서 사용할 5단계 프롬프트로 변환해 줘.
```

SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(2)

◦ 4단계 포트폴리오 전략

- 마케팅 포트폴리오(portfolio)의 경우, 채용 목적으로 사용되므로 첫인상이 중요
  - 마케팅이라는 직무적 특성상 시각적 임팩트와 정보 전달력을 모두 갖춰야 함
- ① 뼈대, ② 기능, ③ 디자인, ④ 점검 순의 4단계 포트폴리오 전략
- 각 단계는 또 다시 요청 → 생성 → 확인 → 수정의 짧은 사이클을 반복해 변경 위험을 줄이고, 구조와 품질을 차근차근 확보



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(3)

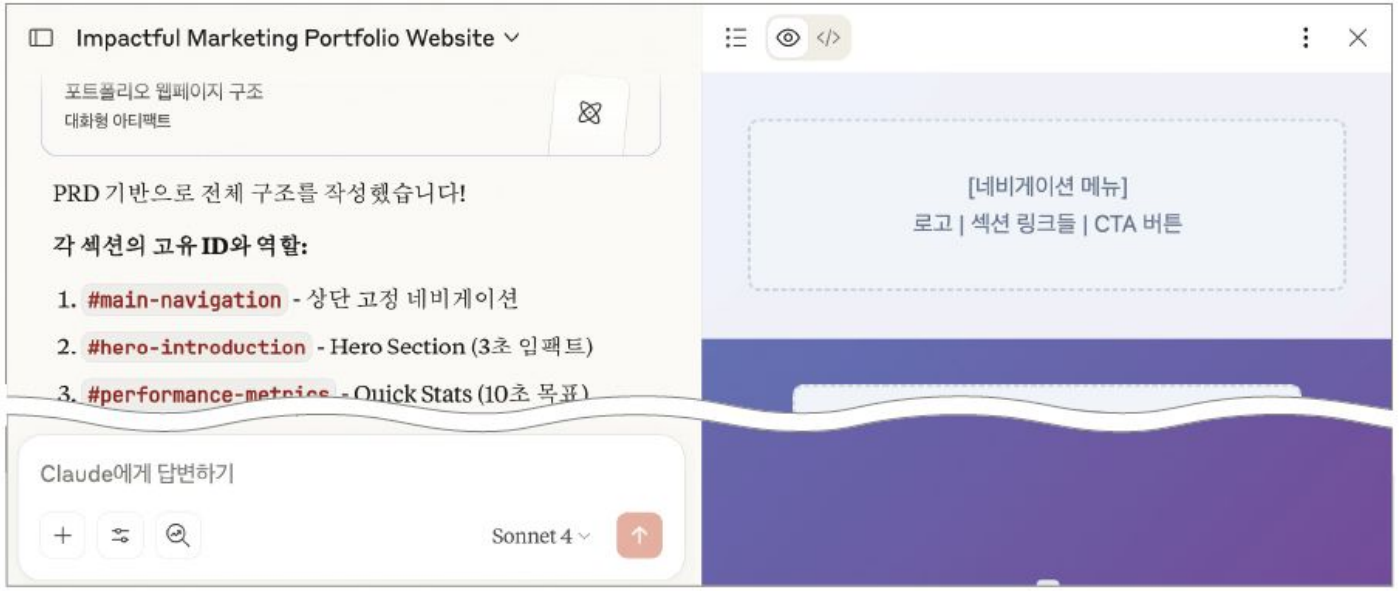
◦ 1단계: 뼈대

- 요청 → 생성 → 확인 → 수정의 과정을 반복

01 [요청] [생성] - 생성한 PRD를 불러온 뒤, 다음과 같이 입력해서 웹페이지의 뼈대를 생성
HTML 문서 내부의 요소를 섹션으로 묶어 주제별로 구성하고, 각 섹션에 고유한 이름을 부여

프롬프트

```
이 PRD대로 진행할 거야. 먼저 PRD를 기반으로 HTML 문서를 작성해 줘.
섹션을 나누고, 수정을 대비해 각 섹션에 고유한 이름을 붙여 줘.
아직 기능은 구현하지 말고, 전체 구조만 보여 줘.
```



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(4)

02 [확인] - 각 섹션에 붙은 고유한 이름을 확인(기능별로 구분)

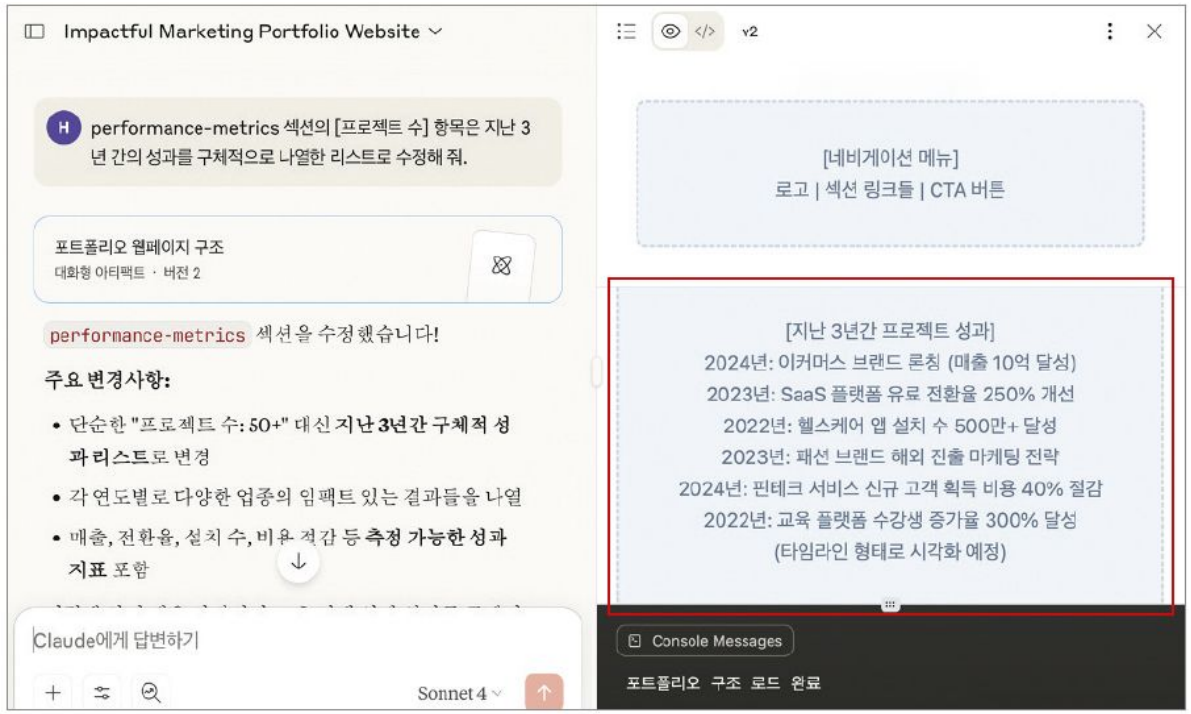
SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(5)

03 [수정] - 특정 섹션을 수정

스펙 요약(Quick Stats) 섹션에 지난 3년 동안의 성과를 제시하기 위해 클로드의 입력창에 다음과 같이 프롬프트를 작성

프롬프트

```
performance-metrics 섹션의 [프로젝트 수] 항목은 지난 3년 간의 성과를 구체적으로 나열한 리스트로 수정
해 줘.
```

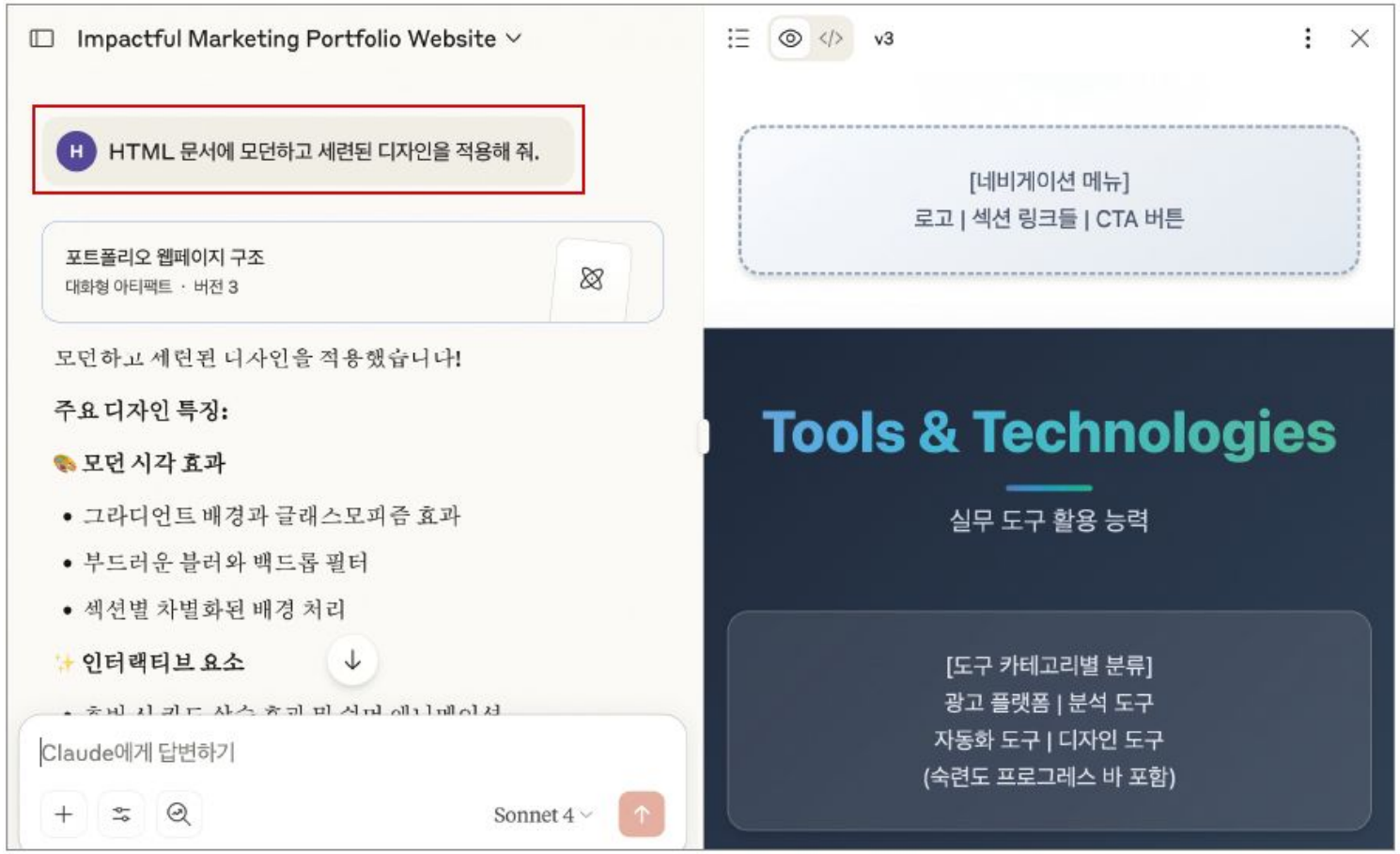


SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(6)

04 [수정] - 디자인을 더하여 뼈대를 완성(상세한 디자인은 <3단계: 디자인 입히기>에서 요청)

프롬프트

```
HTML 문서에 모던하고 세련된 디자인을 적용해 줘.
```



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(7)

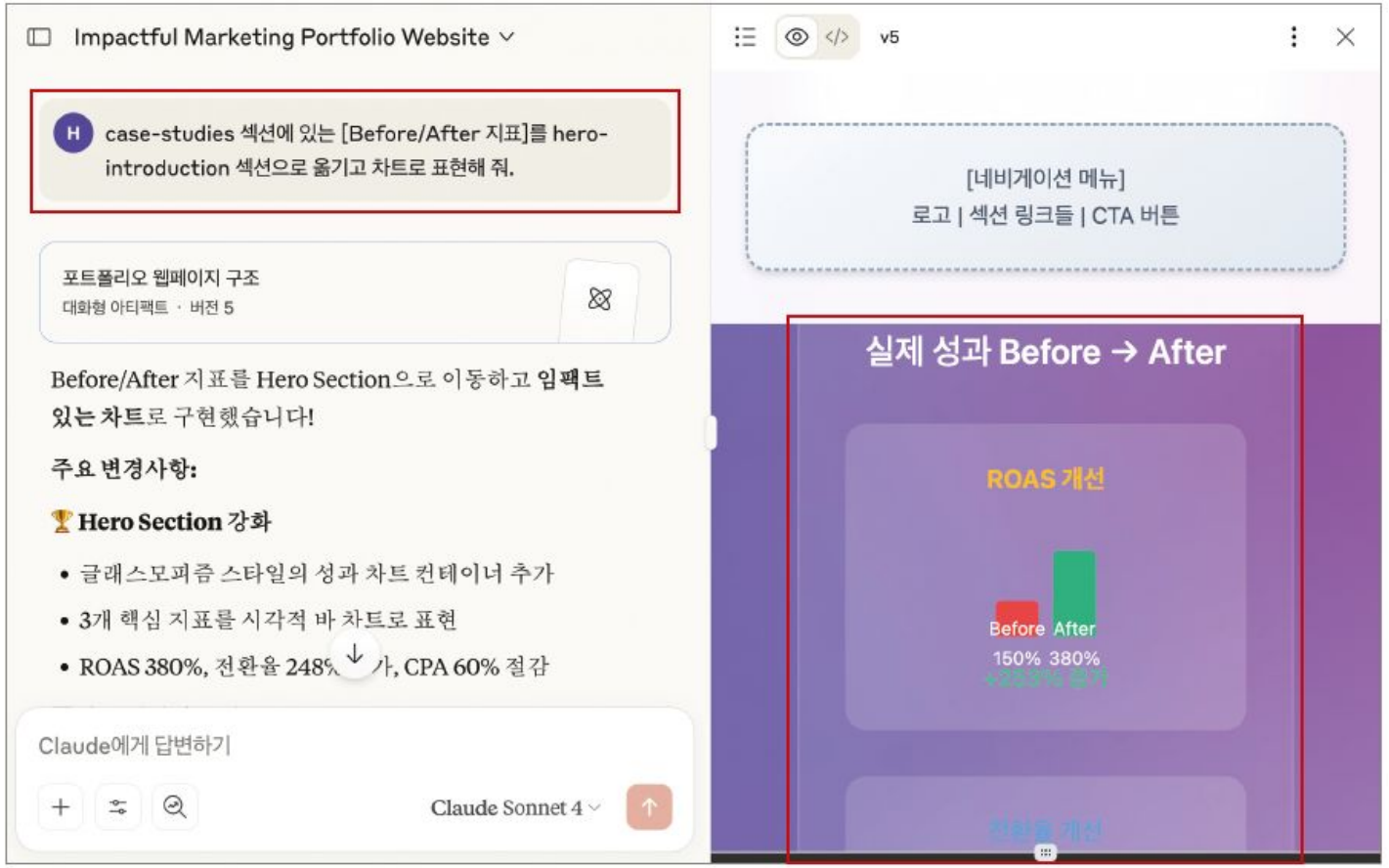
05 [확인] - 스크롤을 내려 다른 섹션들도 확인

SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(8)

06 [수정] - 생성된 웹페이지와 PRD를 비교하면서 부족하거나 잘못된 섹션을 찾아 수정
여기서는 웹페이지 메인 화면의 첫 번째 섹션에 매출 비교 차트를 추가

프롬프트

```
case-studies 섹션에 있는 [Before/After 지표]를 hero-introduction 섹션으로 옮기고 차트로 표현해 줘.
```



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(9)

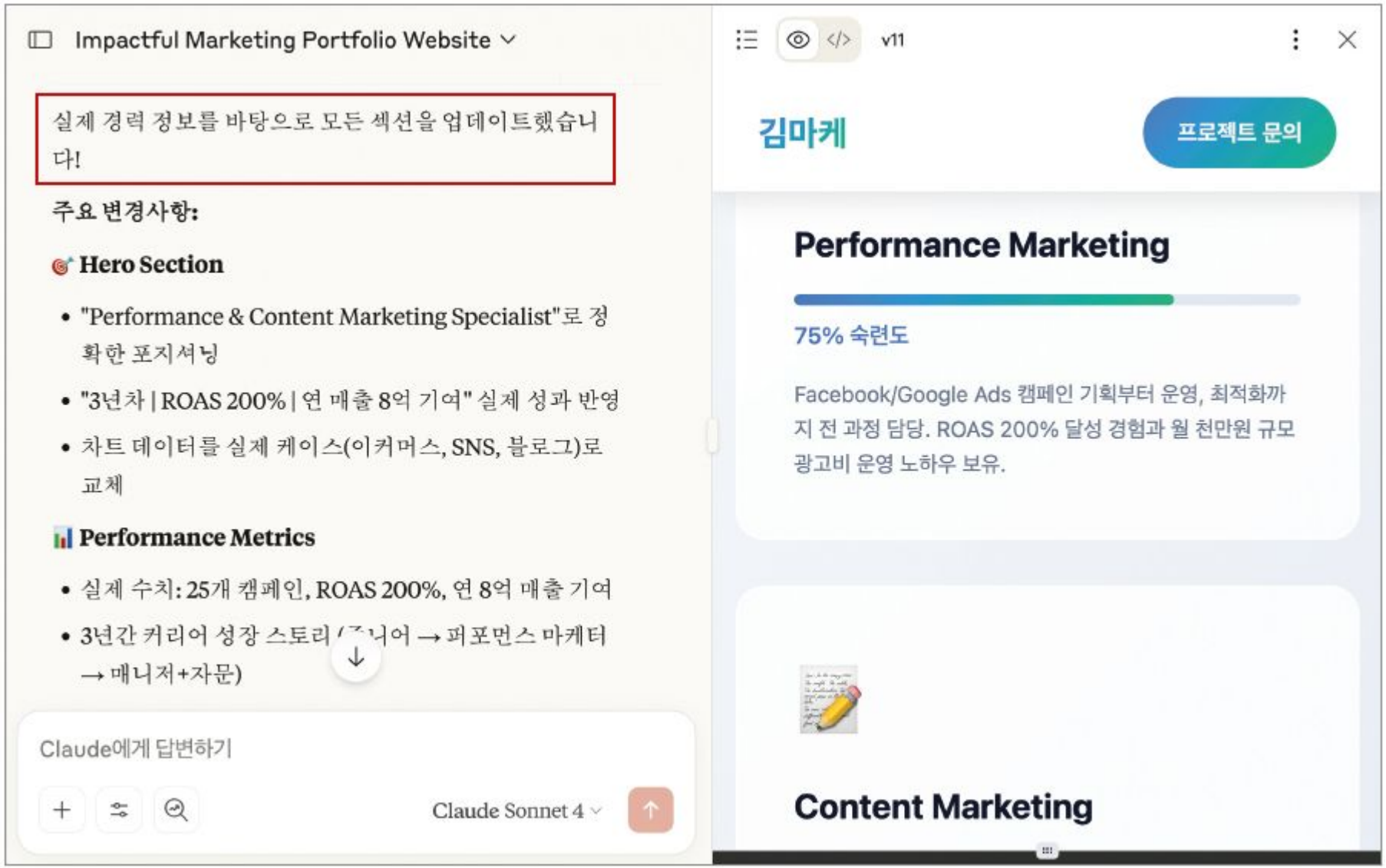
◦ 2단계: 기능

01 [요청] [생성] - 클로드 입력창에 실제 포트폴리오를 전달(구체적으로 제공)

프롬프트

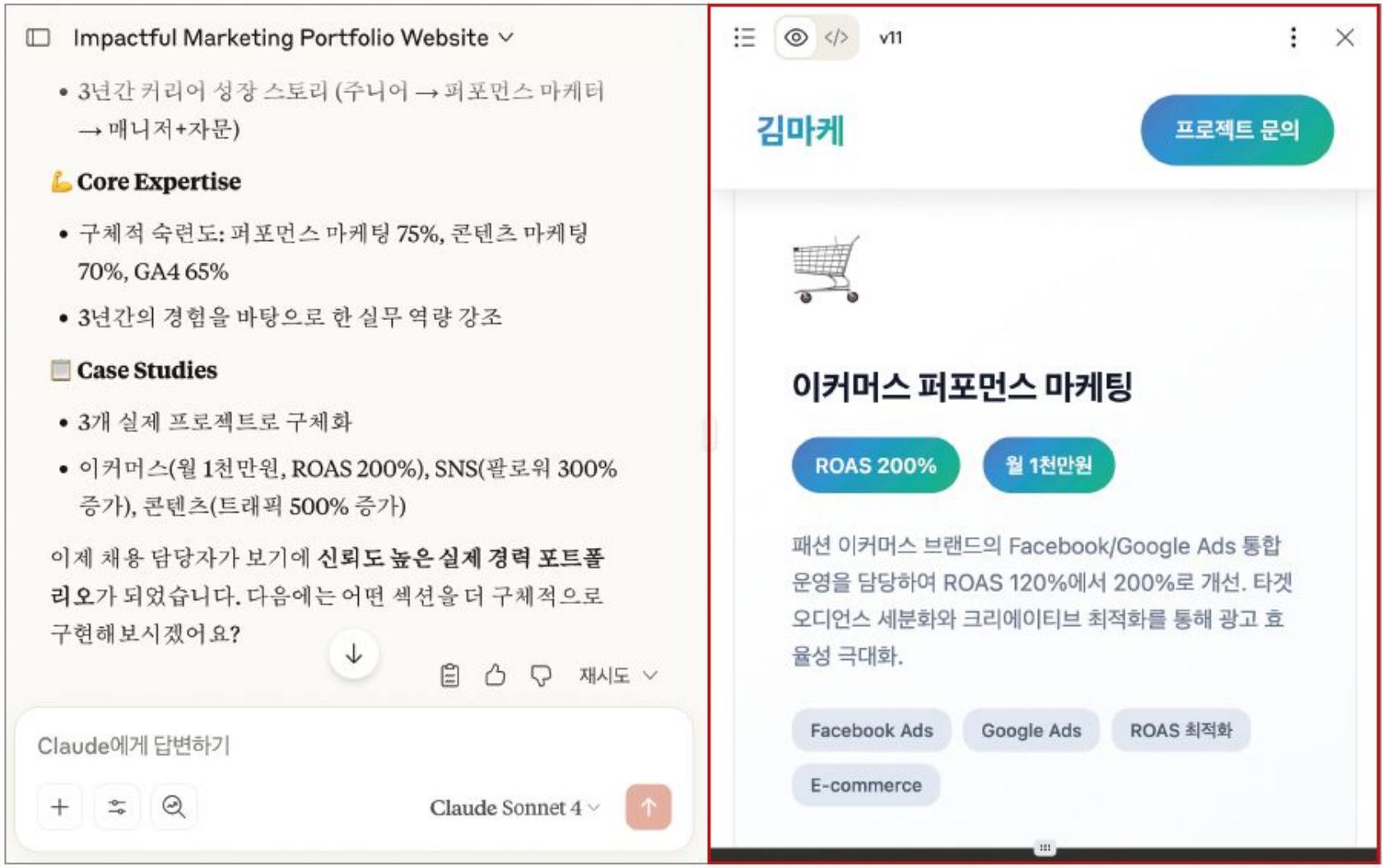
```
나의 경력 포트폴리오는 다음과 같아. 섹션별로 다음 내용을 참고해서 업데이트해 줘.

나는 3년차, 퍼포먼스/콘텐츠 마케팅 전문가야.
- 핵심 성과: ROAS 200%, 25개 캠페인, 연 8억 매출 기여한 거고,
- 3년간 포지션 변화: 주니어(2023) → 퍼포먼스 마케터(2024) → 매니저+자문(2025)
- 스킬 레벨: 퍼포먼스 마케팅: 75%, 콘텐츠 마케팅: 70%, 데이터 분석 (GA4): 65%

다음은 나의 대표 케이스 스터디 이력이야.
- 이커머스: 월 광고비 1천만원, ROAS 200%
- 스타트업: 인스타 팔로워 2천→8천(6개월) 증가
- 로컬 브랜드: 블로그 방문자 500→3천(4개월) 증가
```

SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(10)



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(11)

02 [확인] - 내용이 적절히 업데이트되었는지 확인



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(12)

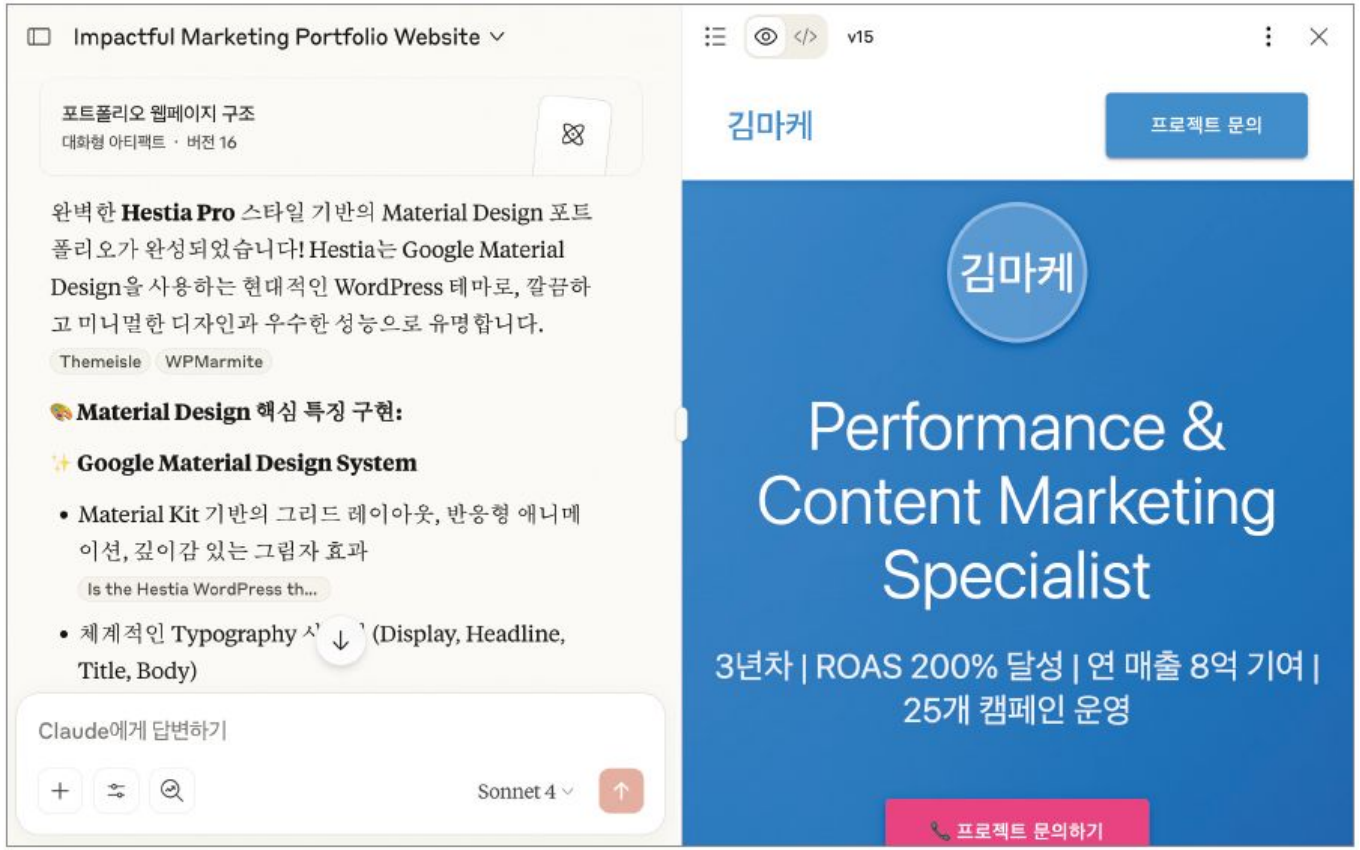
- 3단계: 디자인
  - 3단계에서 활용할 프롬프트 노하우
    - 더 좋은 디자인을 위해 AI에게 레퍼런스(reference)를 제공
    - 레퍼런스 선정이 어렵다면 AI에게 참고할 만한 디자인을 추천해 달라고 요청
  - 이 책에서는 워드프레스(WordPress)의 테마(theme)를 참고해 웹페이지를 구성
    - https://wordpress.com/ko/themes/filter/subject:portfolio

SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(13)

01 [요청] [생성] - 다음과 같이 프롬프트를 작성

프롬프트

워드프레스 테마 중에서 이 포트폴리오와 가장 잘 어울리는 것을 하나 선택해서 디자인을 수정해 줘. 섹션별로 색상 톤을 달리 해서 구분이 잘 되게 해 줘.



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(14)

02 [확인] - 아티팩트의 [게시] 버튼을 클릭해서 포트폴리오 웹페이지의 링크를 생성
웹 브라우저 주소창에 복사한 링크를 붙여 넣은 뒤, 요청한 대로 디자인이 잘 구현되었는지 확인

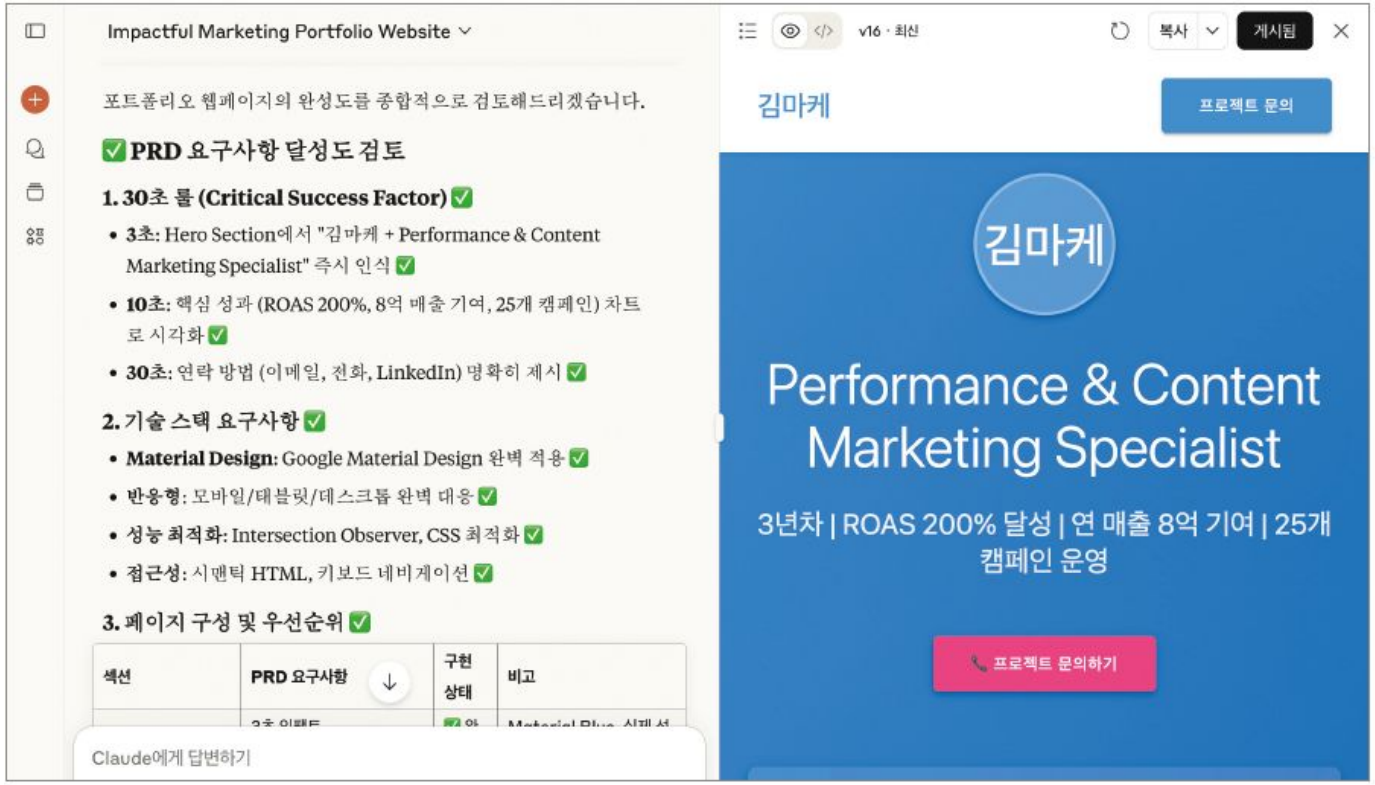
SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(15)

◦ 4단계: 점검

01 [요청] - 기능 점검을 위해 클로드 채팅창으로 돌아와서 다음과 같이 프롬프트를 작성

프롬프트

포트폴리오 웹페이지의 모든 부분이 잘 만들어졌는지 검토해 줘. PRD대로 제작되었는지, 각 링크가 문제없이 작동하는지 확인해.



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(16)

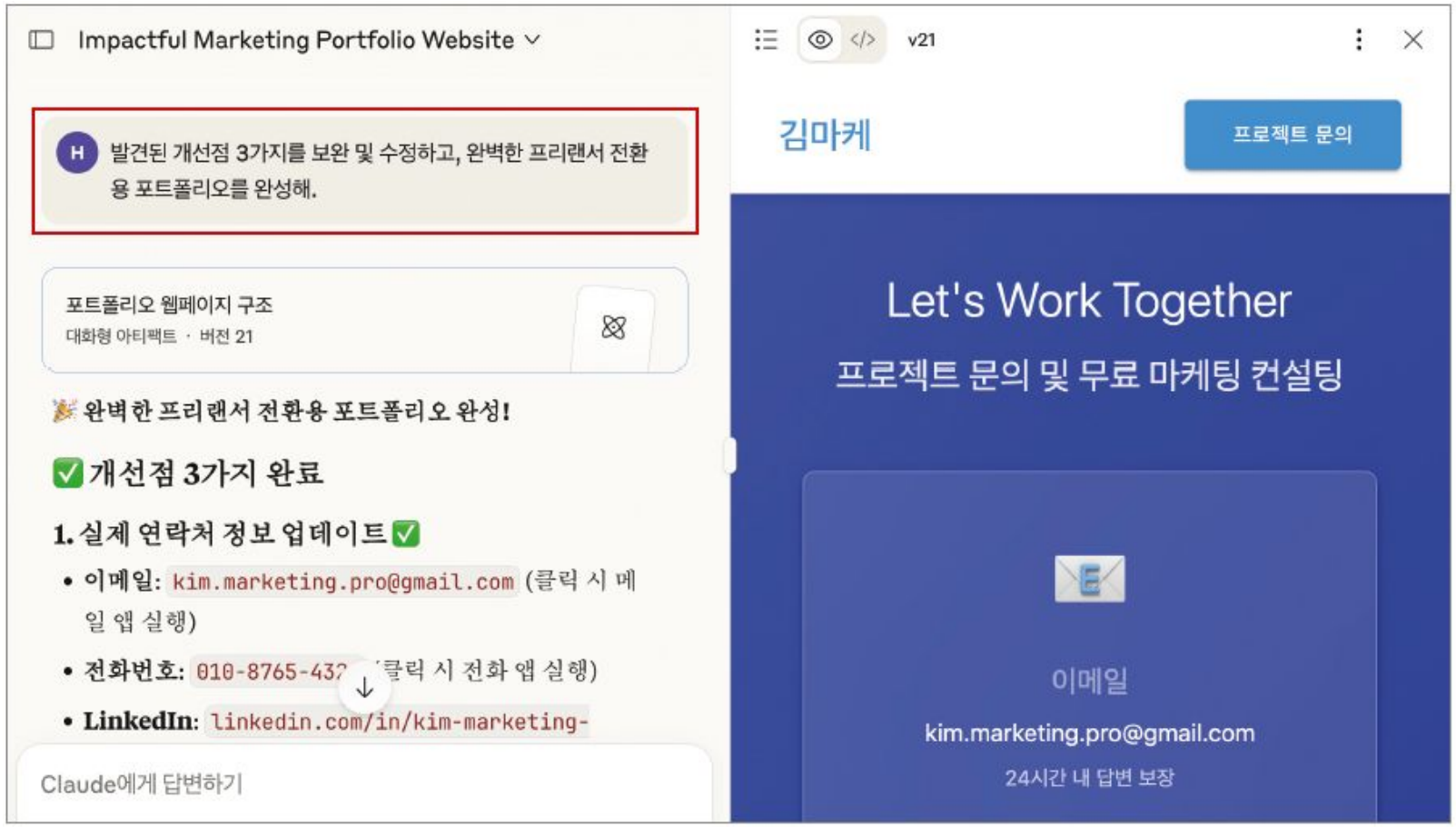
02 [생성] - 클로드가 발견한 개선점을 확인

SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(17)

03 [수정] - 클로드에게 수정을 요청

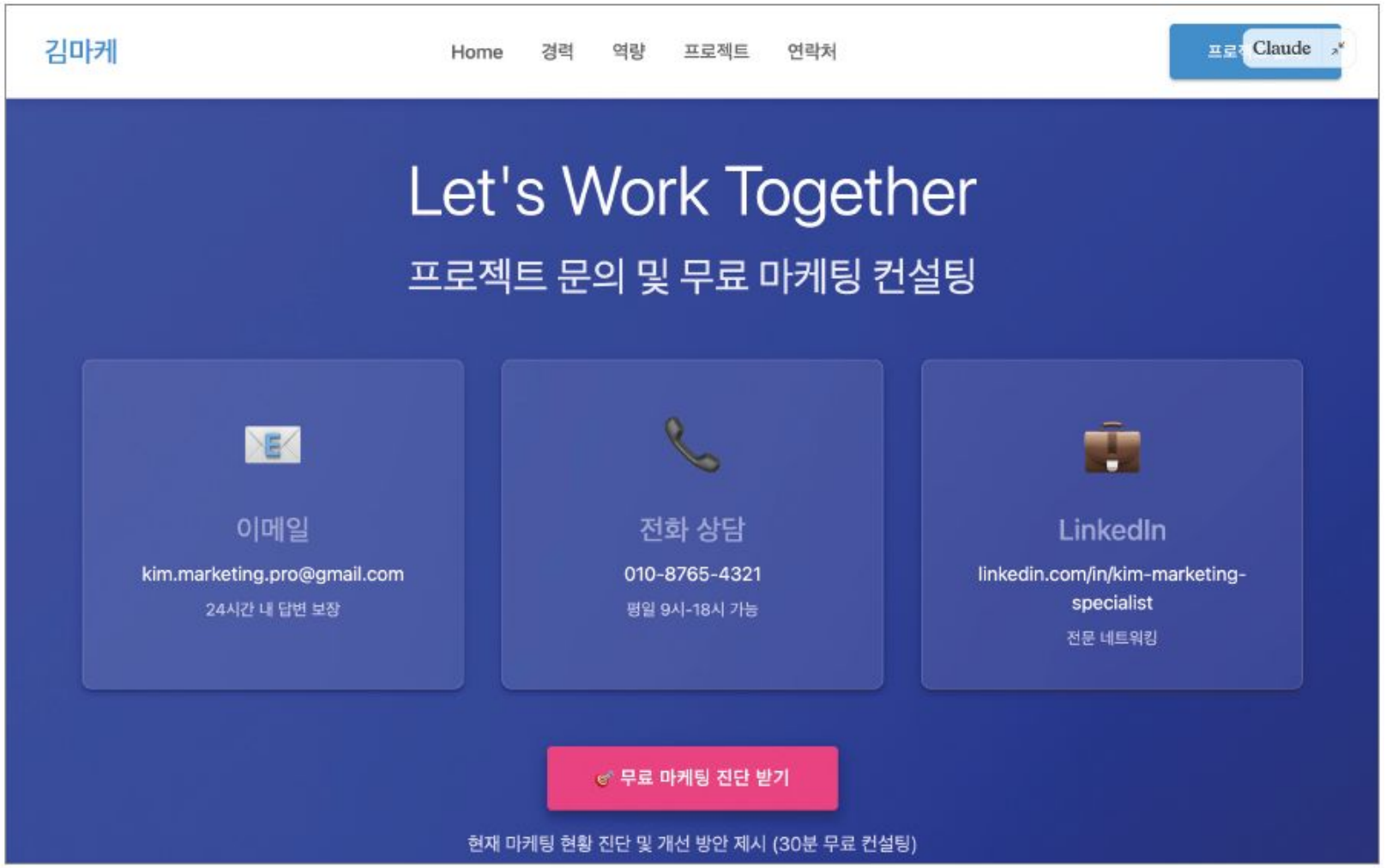
프롬프트

발견된 개선점 3가지를 보완 및 수정하고, 완벽한 프리랜서 전환용 포트폴리오를 완성해.



SECTION 2-2 실전! 마케팅 포트폴리오 완성하기(18)

04 [확인] - 실제 사용 가능한 포트폴리오 웹페이지를 완성



[마무리]

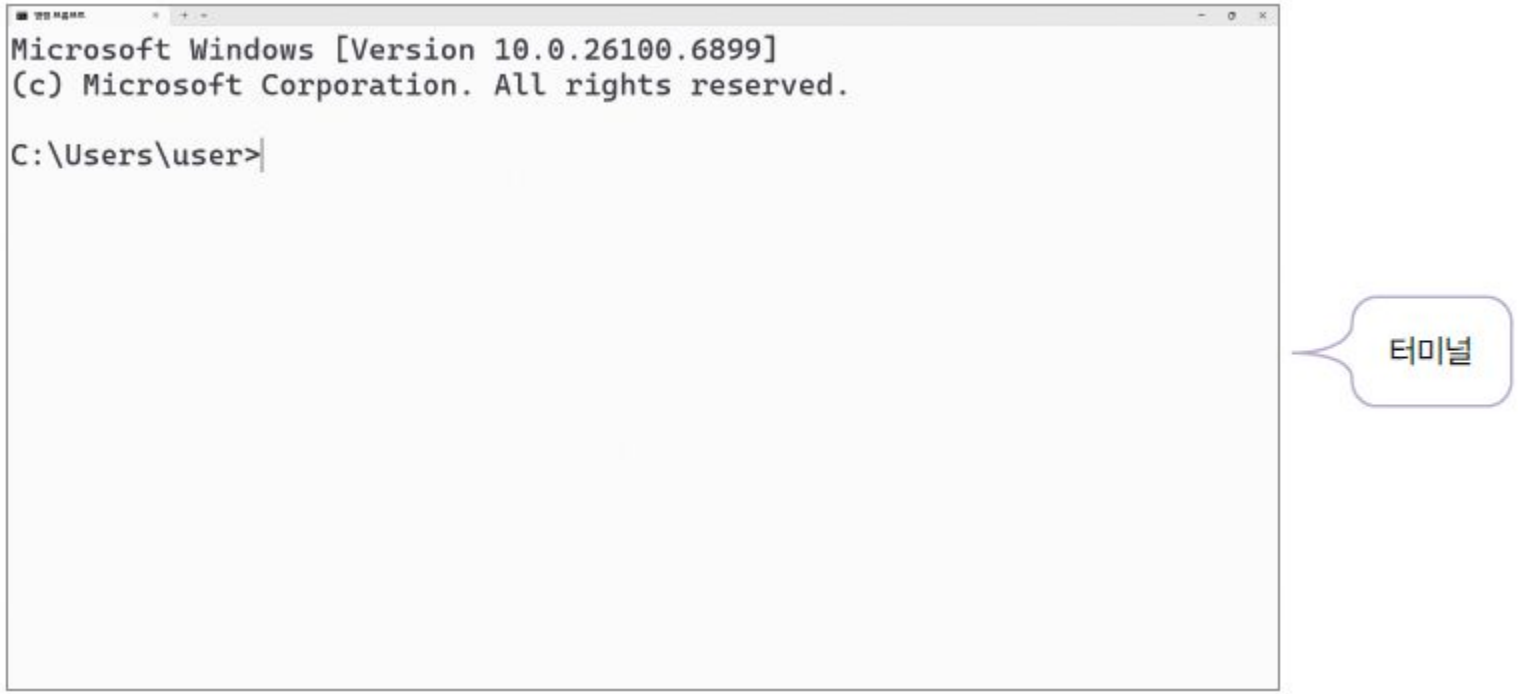
◦ 6가지 키워드로 정리하는 핵심 포인트

- **4단계 포트폴리오 전략**
  - 웹페이지를 뼈대 → 기능 → 디자인 → 점검 순으로 AI에게 요청
- 웹페이지는 주제별로 구분된 **섹션(section)** 단위로 구성
- **뼈대** 단계 - 웹페이지에 기능이나 디자인을 넣기 전, 기본적인 HTML 구조만으로 틀을 잡음
- **기능** 단계 - 실제 데이터를 채워 넣고 동작을 추가
- **디자인** 단계 - 색상, 레이아웃, 시각적 요소를 적용하여 완성도를 높임
- **점검** 단계 - 완성된 웹페이지를 검토하며 누락된 기능이나 오류를 수정

SECTION 3-1 클로드 코드 설치하기(1)

시작하기전에

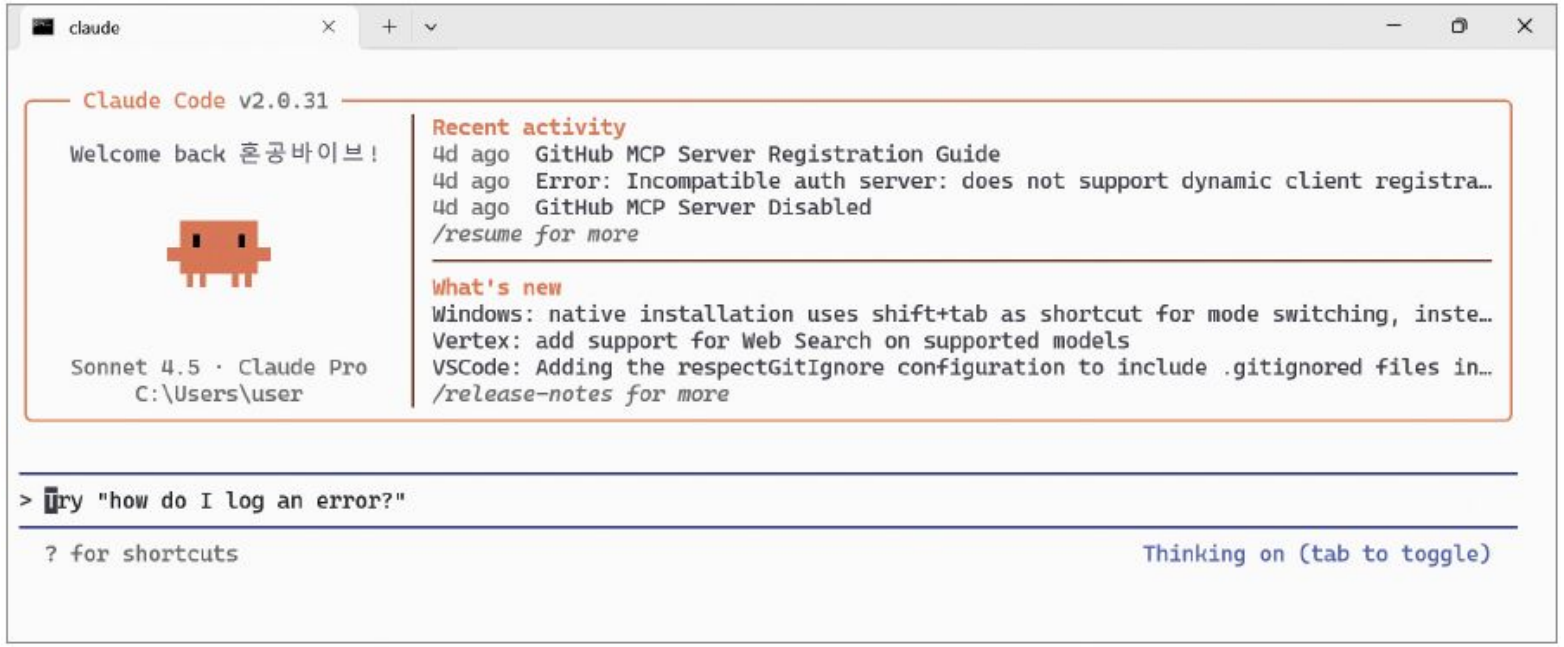
- 터미널(terminal)
  - 명령어를 입력해 컴퓨터와 직접 소통할 수 있는 창
  - 화면에 있는 메뉴, 아이콘을 마우스로 클릭하는 대신 터미널에서는 텍스트 명령어를 입력해 프로그램 실행, 파일 관리, 시스템 설정과 같은 작업을 수행



SECTION 3-1 클로드 코드 설치하기(2)

◦ 클로드 코드(Claude Code)

- 내 컴퓨터 터미널에서 직접 실행되는 도구
- 클로드 - 아이디어를 떠올리고 초안을 만드는 데 편리
- 클로드 코드 - 실제 개발 환경에서 실행 가능한 결과물을 만들어 내는 데 강점
- 프로젝트별로 직접 파일을 생성·수정할 수 있어 개발자의 실무 환경과 유사한 방식으로 작업



SECTION 3-1 클로드 코드 설치하기(3)

- 실습 환경 준비하기
  - 클로드 코드는 윈도우(Windows), 맥OS(macOS), 리눅스(Linux) 등 주요 운영체제에 모두 설치 가능
    - 이 책에서는 접근성이 높은 윈도우 환경을 기준으로 설명
  - 클로드 코드를 윈도우 환경에서 실행하기 위한 두 가지 프로그램
    - 노드(JS Node.js)
      - 자바스크립트 실행 환경으로, 서버를 만들거나 다양한 프로그램을 작성
      - 클로드가 동작할 환경을 제공
    - 깃(Git)
      - 소스 코드를 체계적으로 관리하는 버전 관리 도구
      - 프로젝트의 규모가 커지거나 여러 파일이 얽힐 때, 변경 이력을 기록하고 필요할 때 과거 상태로 되돌릴 수 있어 안정적인 개발에 필수적

SECTION 3-1 클로드 코드 설치하기(4)

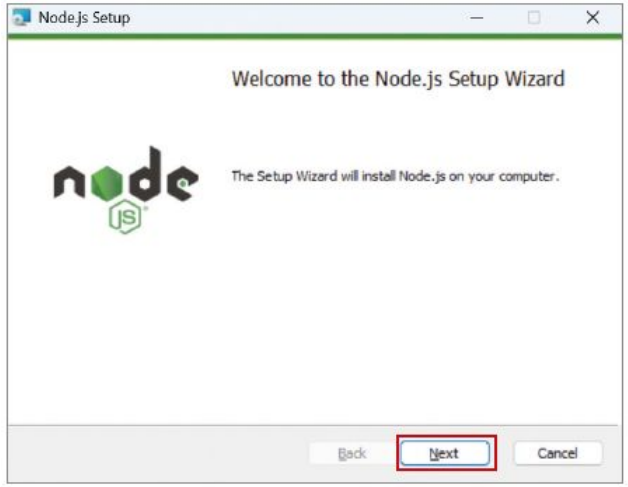
노드JS 설치

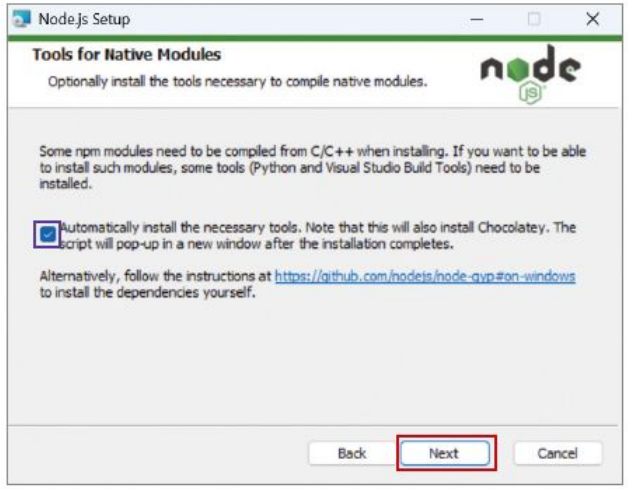
01 노드JS 공식 홈페이지의 상단 [다운로드] 탭을 클릭, [Windows 설치 프로그램(.msi)]를 다운로드

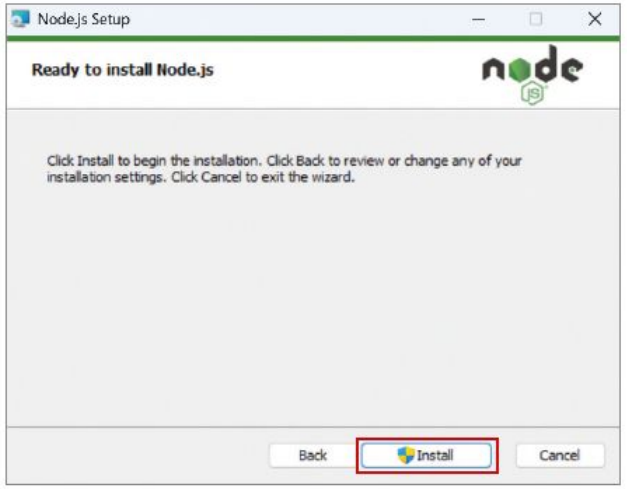
- https://nodejs.org/ko

SECTION 3-1 클로드 코드 설치하기(5)

02 다운로드한 'node-v22.19.0-x64.msi' 파일을 파일 탐색기에서 더블 클릭한 뒤, 설치 마법사를 따라 모든 옵션을 기본 설정으로 두고 [Next] 버튼을 눌러 진행. [Install] 버튼을 누르면 설치를 시작

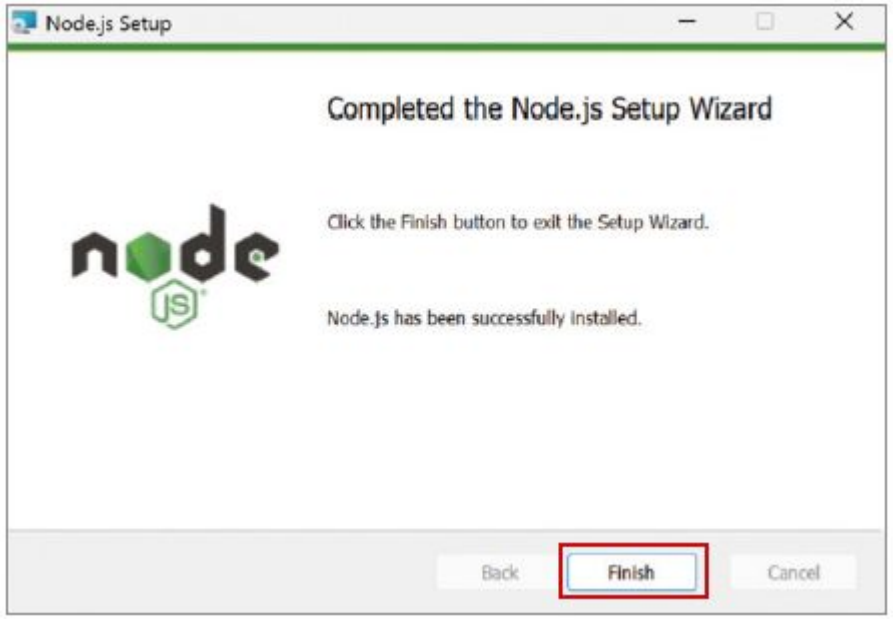






SECTION 3-1 클로드 코드 설치하기(6)

03 [Finish] 버튼을 클릭해 설치를 완료



[note]

- 노드JS 설치가 완료되면 추가적인 도구를 설치할 수 있는 터미널이 나타남
- 우선 아무것도 입력하지 않고 [창 닫기(X)]를 눌러 종료

SECTION 3-1 클로드 코드 설치하기(7)

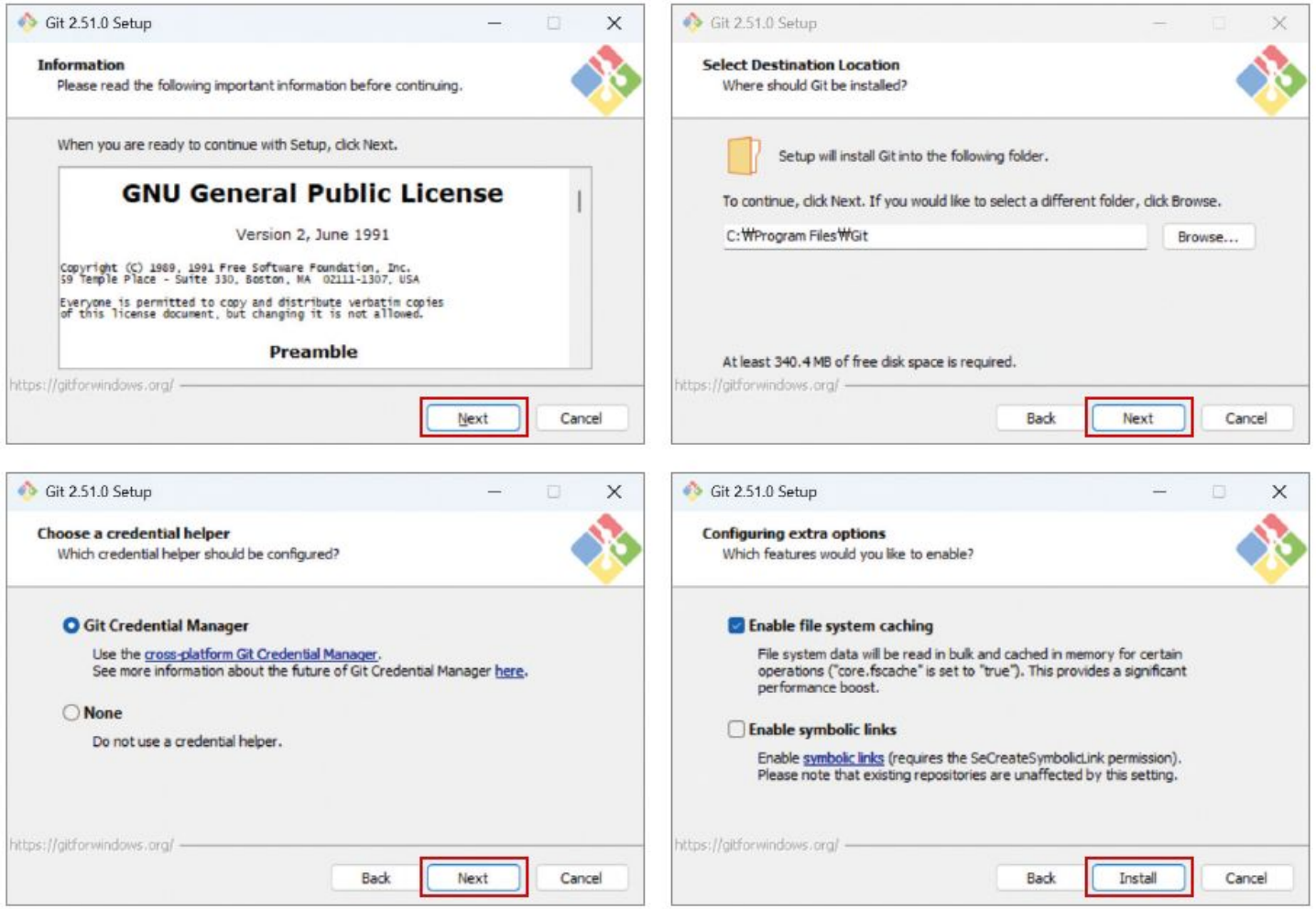
깃 설치

01 깃 홈페이지의 [Downloads] 메뉴에서 ‘Standalone Installer’ 하단의 ‘Git for Windows /x64 Setup.’을 클릭해 다운로드

- https://git-scm.com

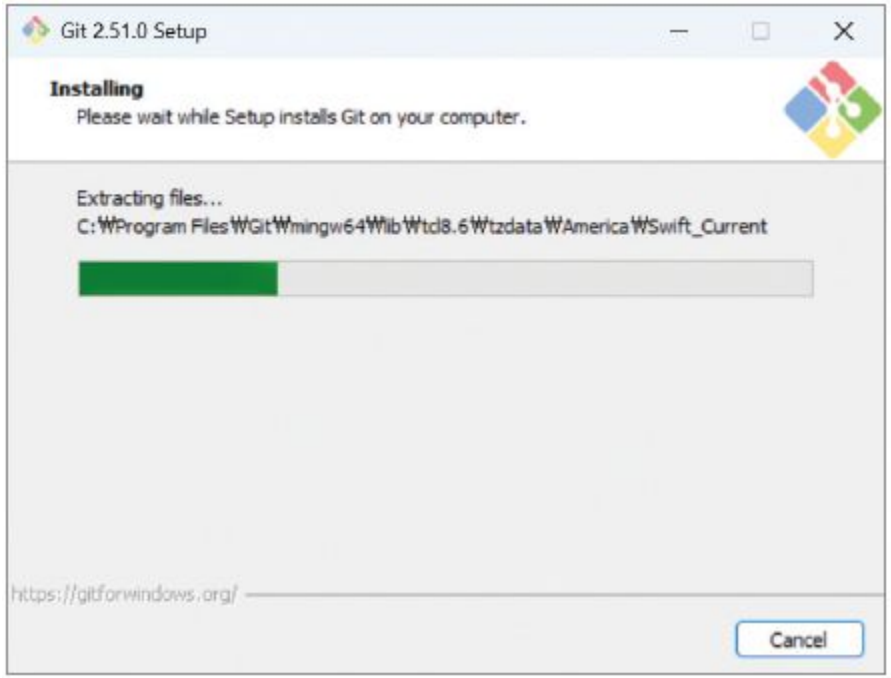
SECTION 3-1 클로드 코드 설치하기(8)

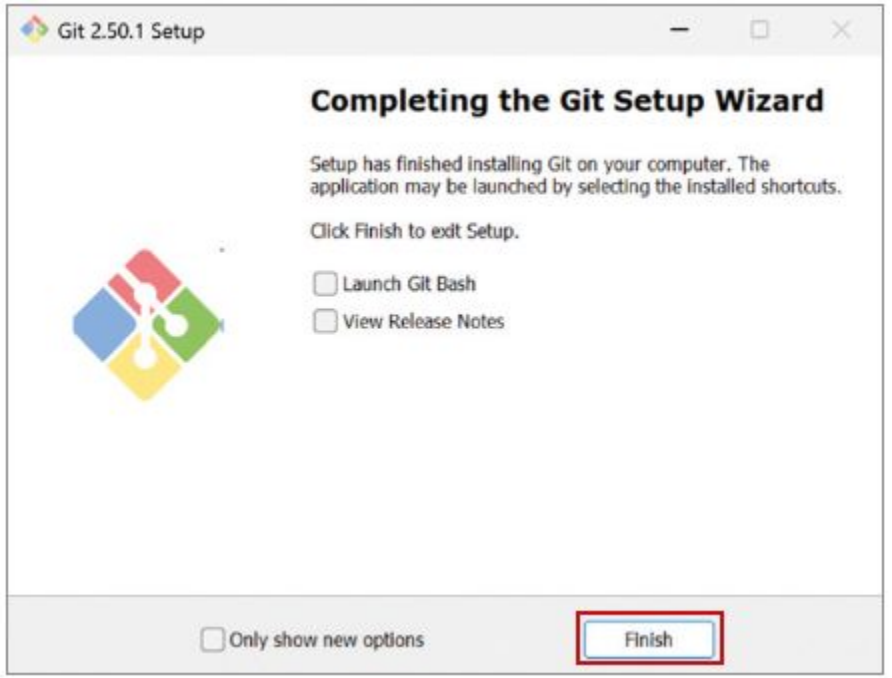
02 다운로드한 'Git-2.51.0-64-bit.exe' 파일을 파일 탐색기에서 더블 클릭해 실행하고, 설치 마법사를 따라 진행 - [Install] 버튼을 클릭하면 설치를 시작



SECTION 3-1 클로드 코드 설치하기(9)

03 [Finish] 버튼을 눌러 설치를 완료



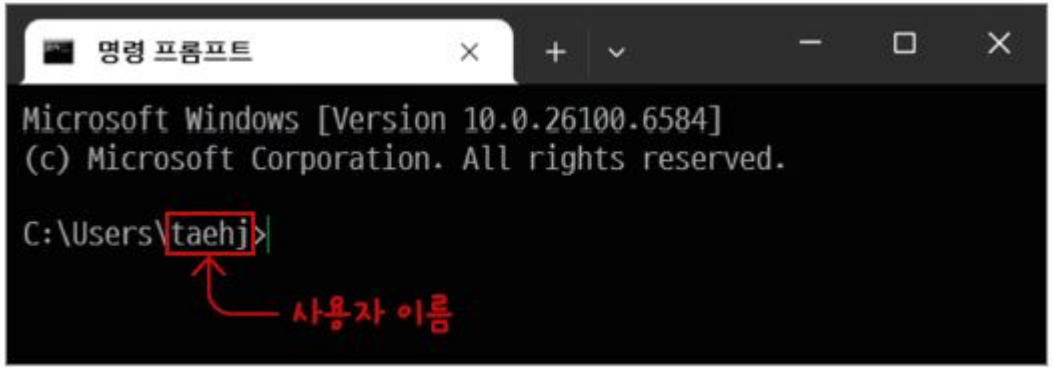


SECTION 3-1 클로드 코드 설치하기(10)

◦ 터미널 실행하기

01 윈도우 작업 표시줄의 검색창에 'cmd'라고 입력하고 [Enter]

02 다음과 같은 검은 화면의 명령 프롬프트가 실행
화면 속 프롬프트 'C:\Users\taehj>'는 현재 터미널이 작업하는 위치



[note]

- 터미널의 색상은 우측 상단의 V(아래 화살표) → 설정 → 색 구성표에서 변경 가능
- 책에서는 인쇄물의 가독성 확보를 위해 배경을 흰 색으로, 텍스트를 검은색으로 설정

SECTION 3-1 클로드 코드 설치하기(11)

03 명령 프롬프트 사용법

커서가 깜박이는 위치에 dir을 입력 - 디렉터리 내에 존재하는 파일들의 목록을 보여주는 명령어

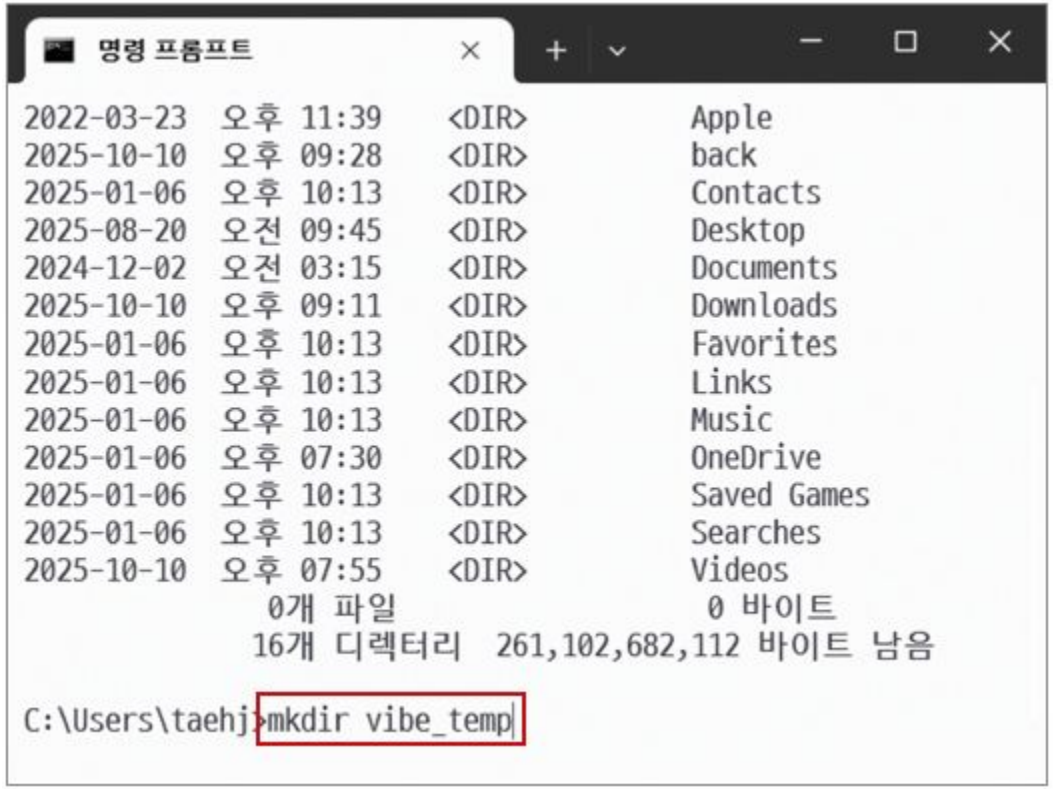
```
dir
```

SECTION 3-1 클로드 코드 설치하기(12)

04 현재 위치에 새로운 디렉터리를 생성하는 명령어 - mkdir

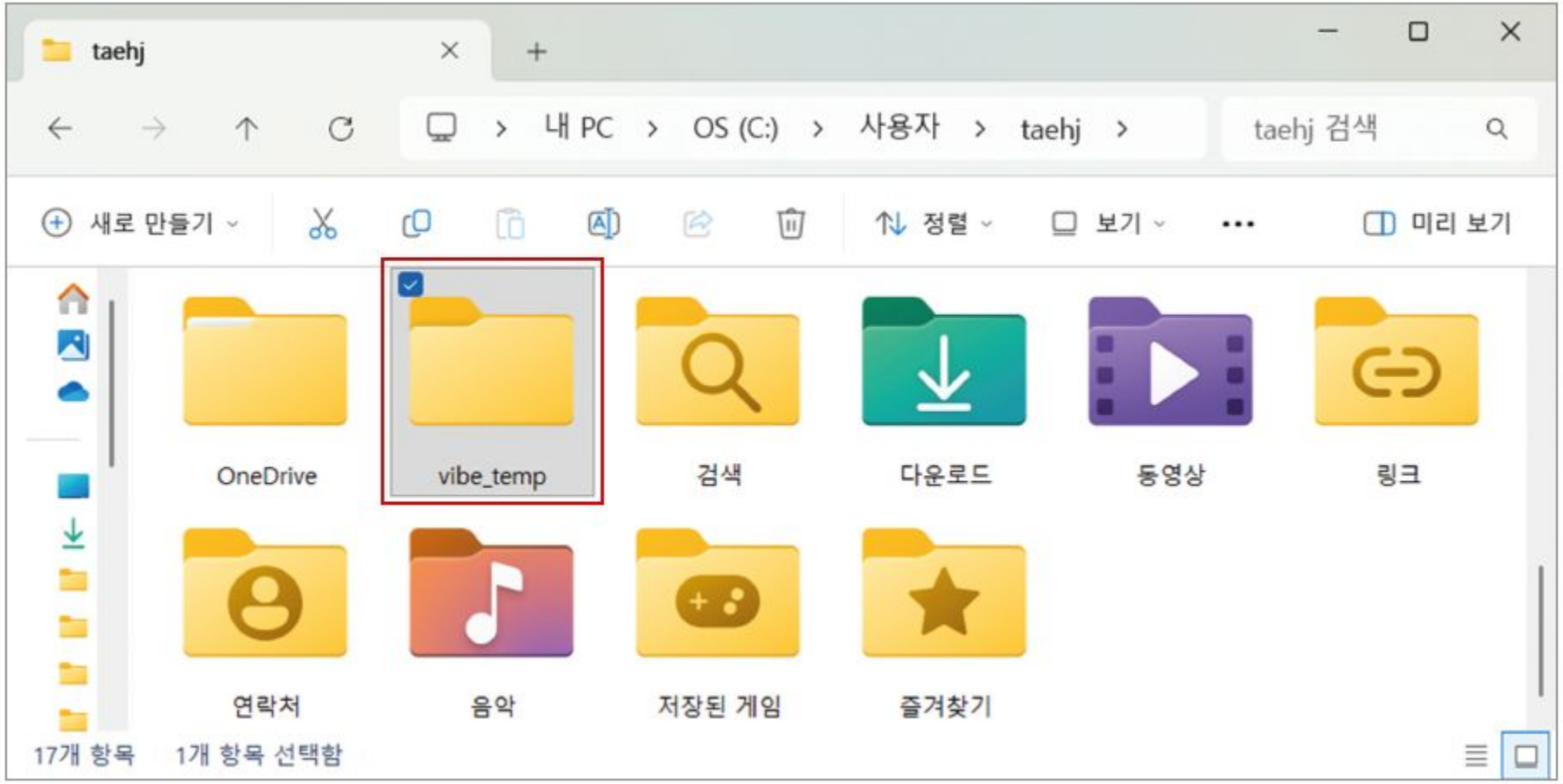
'vibe_temp'라는 폴더를 생성

```
mkdir vibe_temp
```



SECTION 3-1 클로드 코드 설치하기(13)

05 파일 탐색기에서 'vibe_temp' 폴더가 생성되었는지 확인



SECTION 3-1 클로드 코드 설치하기(14)

◦ 클로드 코드

클로드 코드 설치

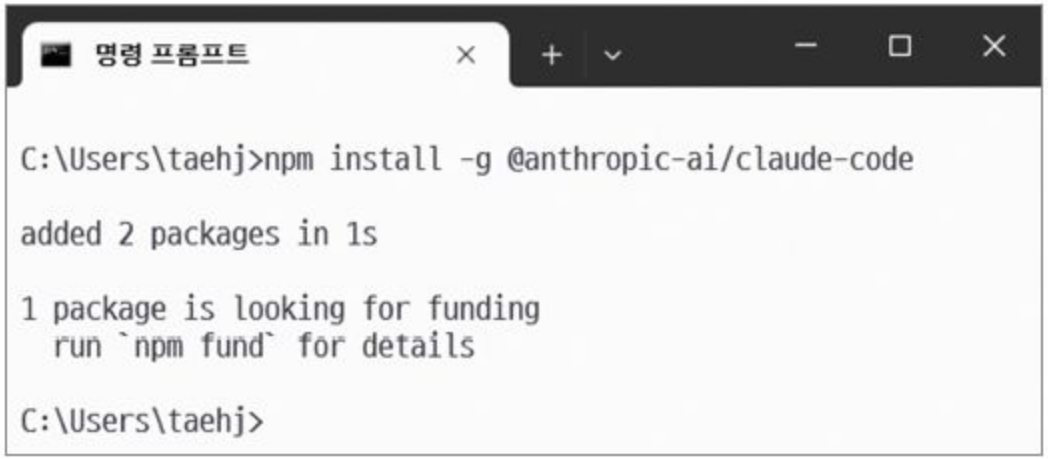
01 명령 프롬프트에 다음과 같이 명령어를 입력해서 클로드 코드를 설치

```
npm install -g @anthropic-ai/claude-code
```

- npm: Node Package Manager의 약자로 Node.js의 패키지 관리자를 의미
- install: 패키지를 설치하라는 명령
- -g: global의 약자로, 시스템 전체에서 사용할 수 있도록 설치
- @anthropic-ai/claude-code: 설치할 패키지의 이름

SECTION 3-1 클로드 코드 설치하기(15)

02 클로드 코드가 설치됨



03 이제 명령 프롬프트에 'claude'라고 입력해 클로드를 실행할 수 있음

SECTION 3-1 클로드 코드 설치하기(16)

초기 설정

01 클로드 코드의 초기 화면은 터미널의 스타일 설정 옵션을 보여줌
여기서는 가독성을 위해 '2. Light mode'를 선택
키보드의 화살표 키(↓ ↑) 사용해 원하는 테마를 선택하고 [Enter]

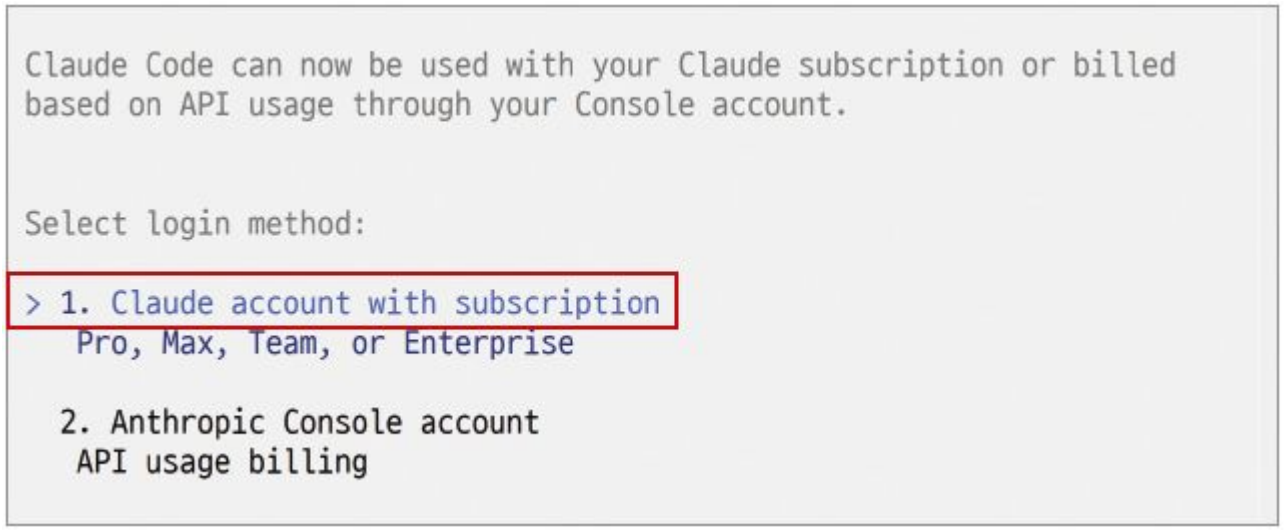
SECTION 3-1 클로드 코드 설치하기(17)

02 클로드 코드의 로그인 방법을 선택

키보드의 화살표 키 [↓] '1. Claude account with subscription'을 선택하고 [Enter]

화면 해설

1. 클로드 구독 계정 사용: 월 단위 구독 요금을 지불하면 추가 비용 없이 클로드 코드 사용 가능
2. 앤트로픽 API 계정 사용: 토큰 사용량에 따라 요금 부과

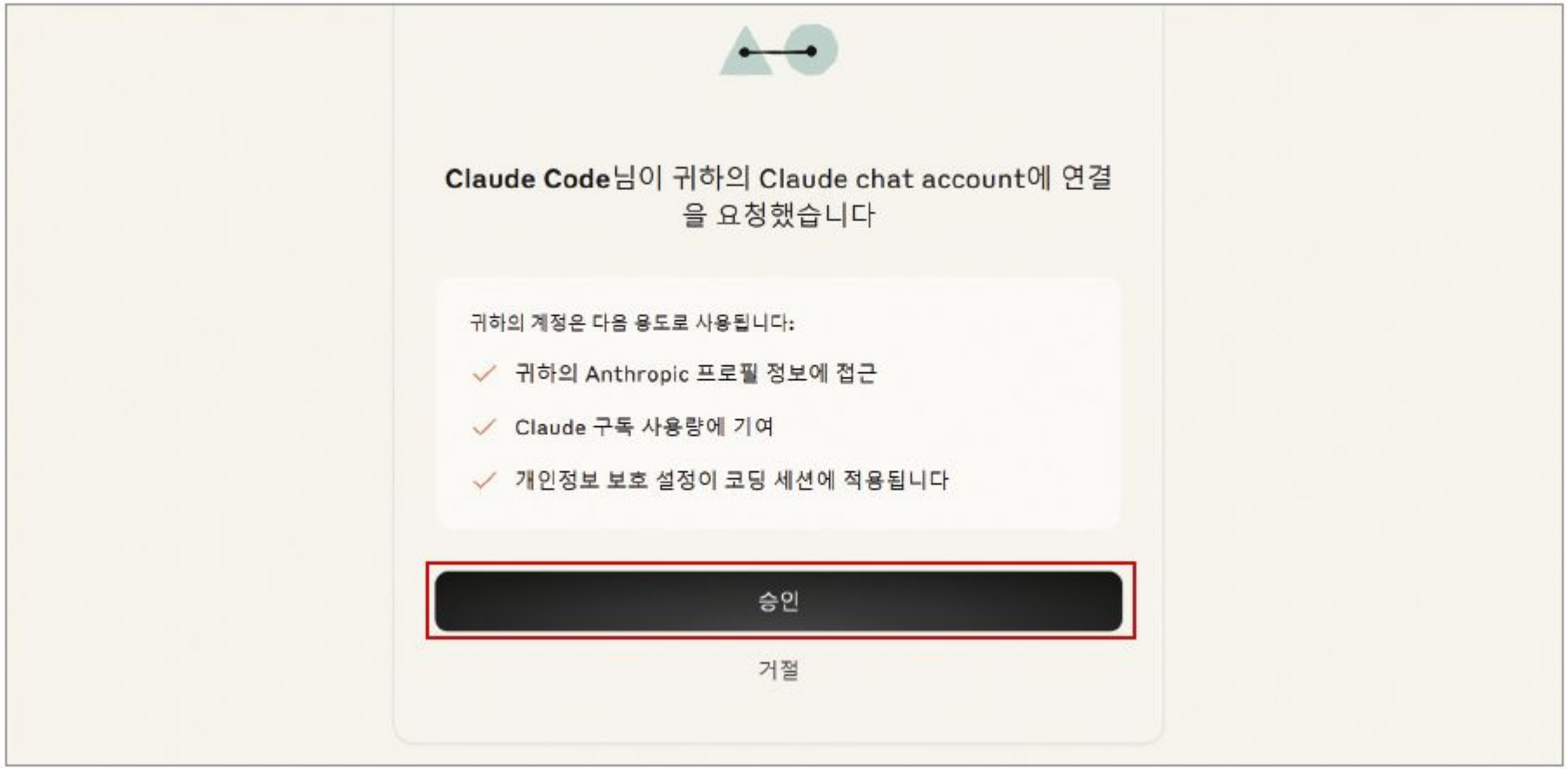


[note]

- **2. Anthropic Console account'**를 선택하면 , 초보자의 경우 예상치 못하게 과도한 비용이 발생할 우려가 있고 토큰 단위로 사용량을 계산해야 한다는 번거로움도 있음

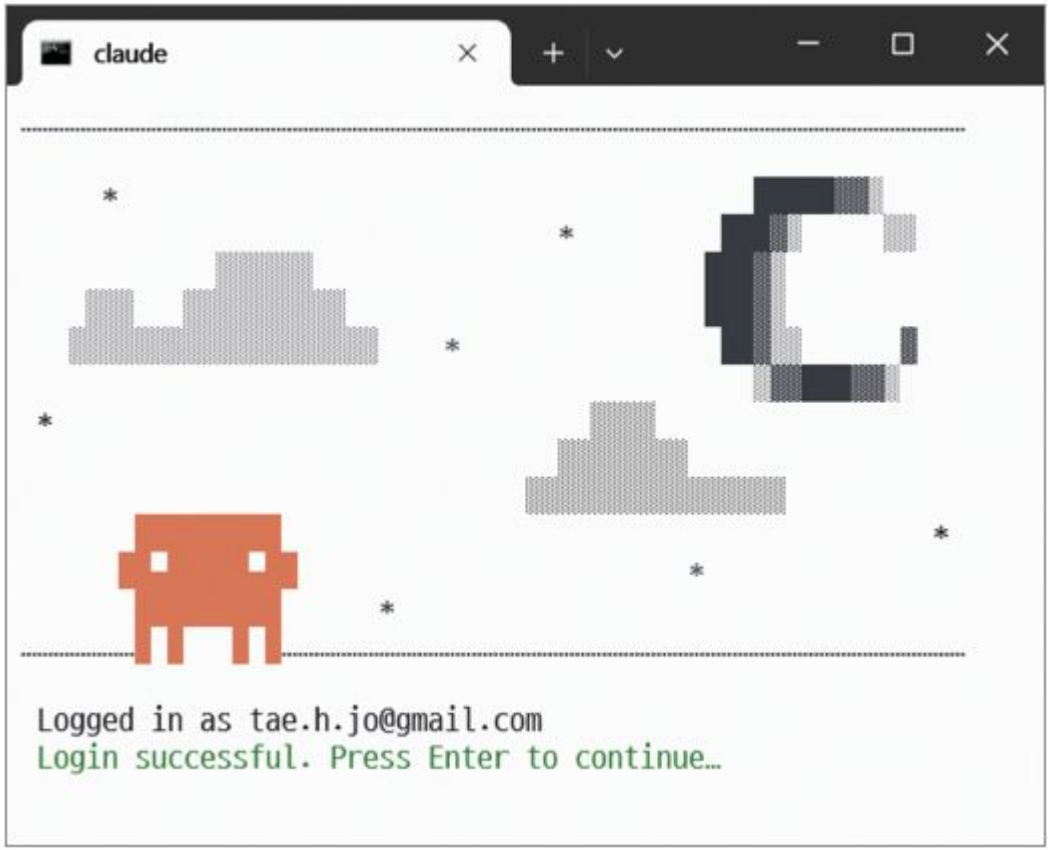
SECTION 3-1 클로드 코드 설치하기(18)

03 클로드 코드와 계정이 연동. [승인] 버튼을 클릭



SECTION 3-1 클로드 코드 설치하기(19)

04 명령 프롬프트로 돌아가면 로그인에 성공했다는 메시지가 보임
[Enter] 키를 눌러 다음 화면으로 넘어감



SECTION 3-1 클로드 코드 설치하기(20)

05 보안 관련 메시지를 확인하고 [Enter]

화면 해설

보안 안내:

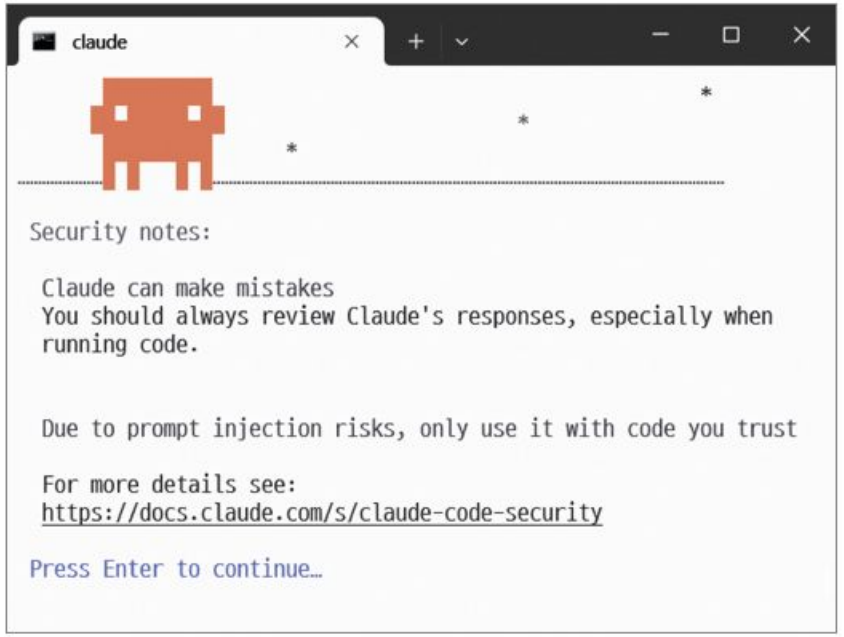
Claude는 실수를 할 수 있습니다.

특히 코드를 실행하기 전에 항상 Claude의 응답을 검토해야 합니다.

프롬프트 인젝션(prompt injection) 위험이 있으므로, 신뢰할 수 있는 코드에서만 사용하세요.

자세한 내용은 다음 문서를 참조하세요:

https://docs.claude.com/s/claude-code-security



SECTION 3-1 클로드 코드 설치하기(21)

06 클로드 코드가 파일 시스템에 접근하기 위해 권한 동의를 요구
키보드의 화살표 키 [↓] '1. Yes, proceed'를 선택하고 [Enter]

화면 해설

이 폴더에 있는 파일을 신뢰합니까?

C:\Users\taehj

Claude Code는 이 디렉터리에 있는 파일을 읽거나, 쓰거나, 실행할 수 있습니다.

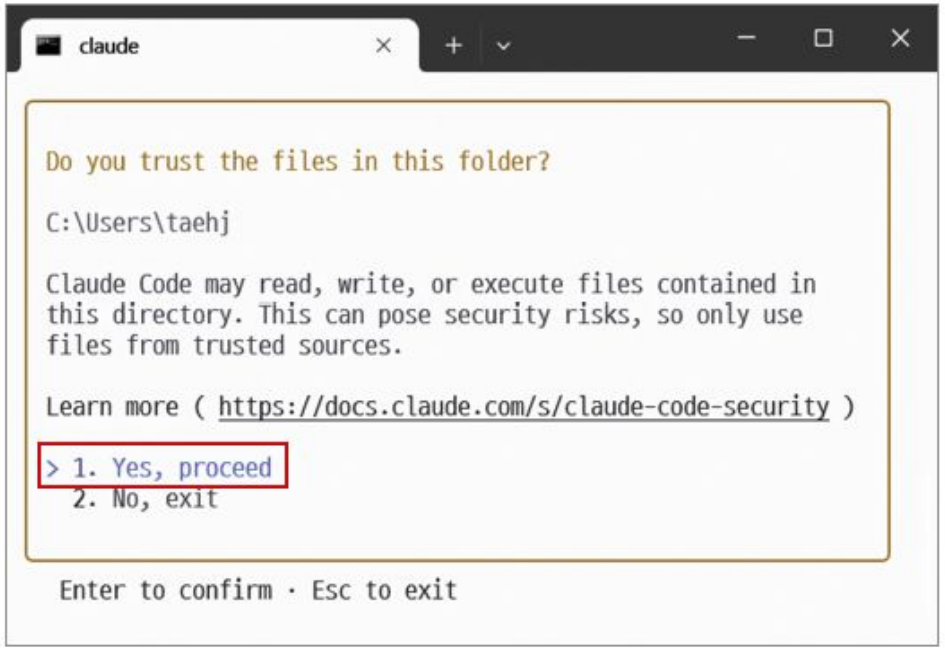
이는 보안 위험을 초래할 수 있으므로, 반드시 신뢰할 수 있는 출처의 파일만 사용하세요.

자세한 내용은 여기를 참고하세요:

https://docs.anthropic.com/s/claude-code-security

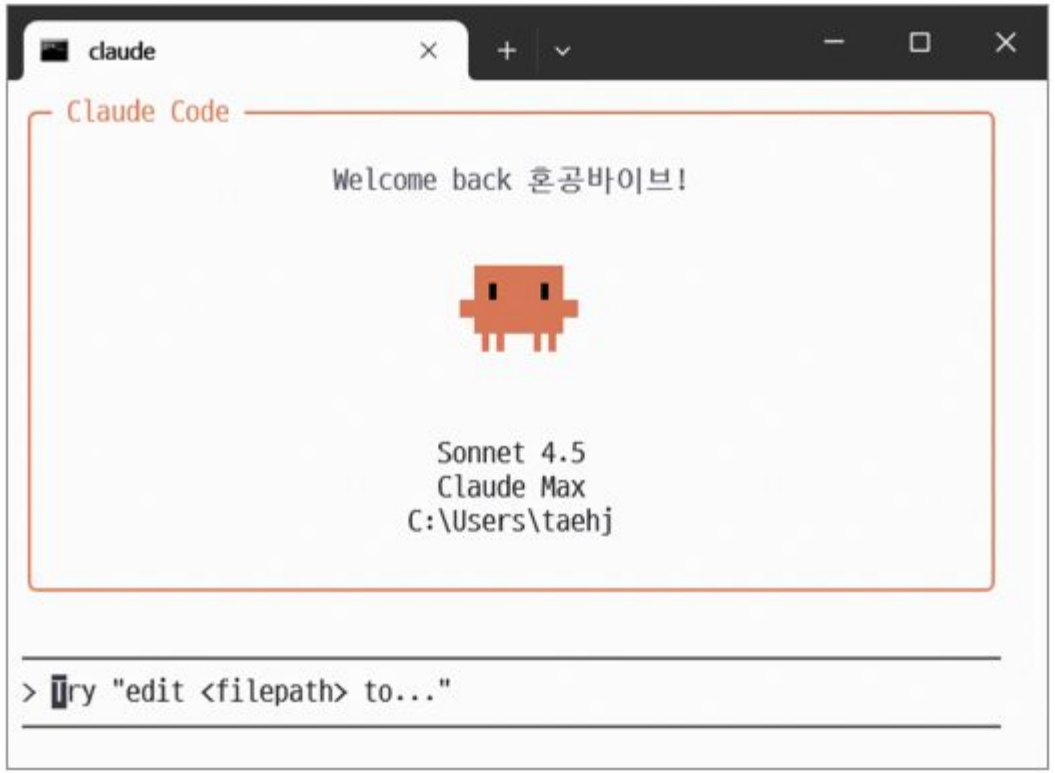
1. 예, 진행합니다

2. 아니오, 종료합니다



SECTION 3-1 클로드 코드 설치하기(22)

07 이제 웹 브라우저에서 했던 것처럼 명령 프롬프트에서 클로드와 자연스럽게 대화를 시작



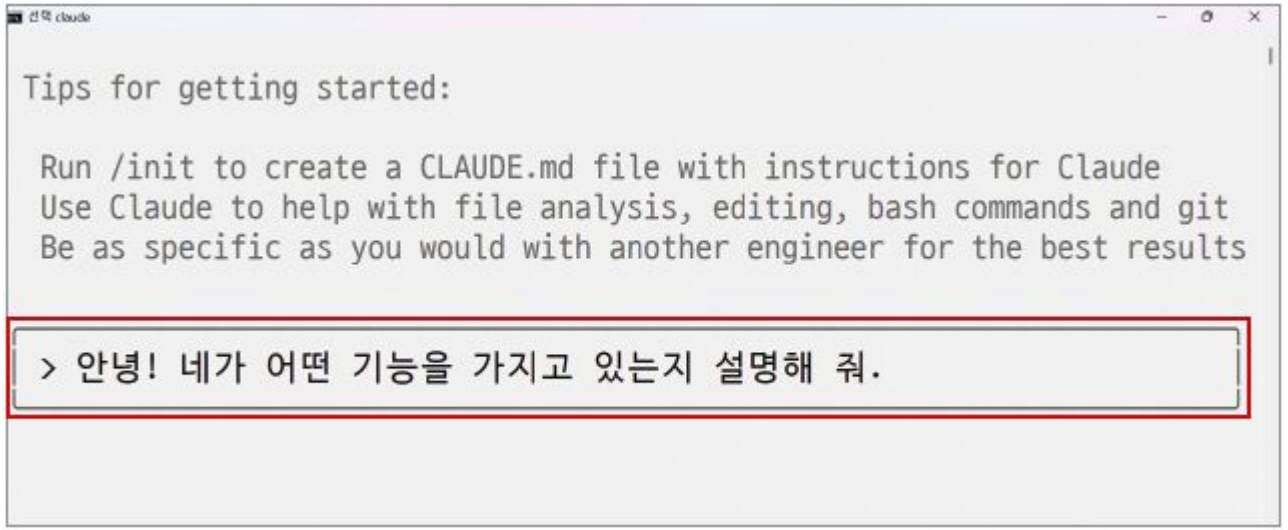
SECTION 3-1 클로드 코드 설치하기(23)

클로드 코드와의 첫 대화

- 클로드 코드와 대화 시작
  - [> Try “how does <filepath> work?”]라고 되어 있는 상자가 바로 프롬프트 입력창
  - 다음과 같이 입력하고 [Enter]

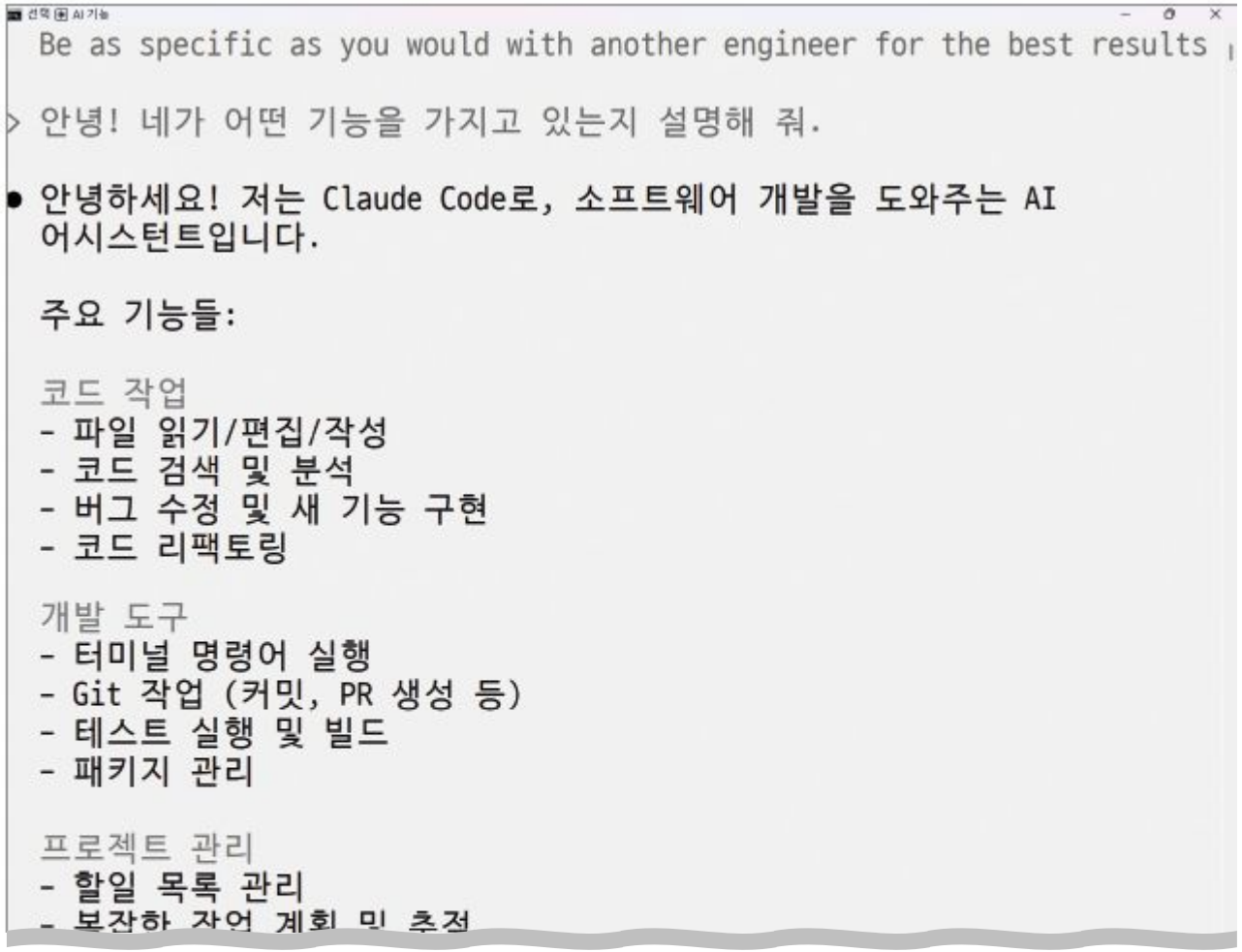
프롬프트

```
안녕! 네가 어떤 기능을 가지고 있는지 설명해 줘.
```



SECTION 3-1 클로드 코드 설치하기(24)

- 클로드 코드가 다양한 기능들을 친절하게 안내
  - 코드 작업, 개발 도구, 프로젝트 관리, 정보 검색 등 클로드 코드의 강력한 기능들을 사용할 준비가 됨



SECTION 3-1 클로드 코드 설치하기(25)

- 클로드 코드를 종료 - [Ctrl]+[C] 키를 두 번 연속으로 누름

[마무리]

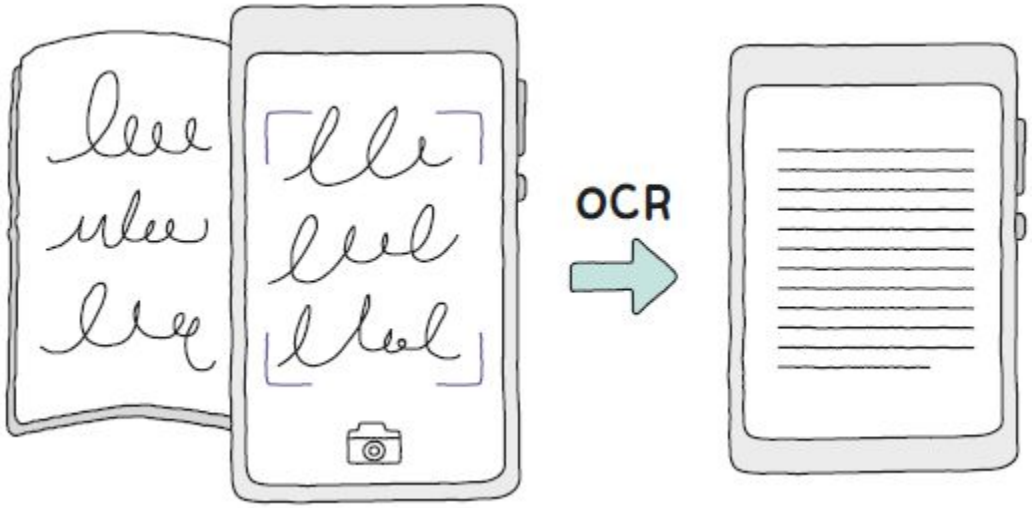
- 3가지 키워드로 정리하는 핵심 포인트
  - **터미널**
    - 명령어를 입력하여 컴퓨터와 직접 소통하는 텍스트 기반 인터페이스로, 클로드 코드가 실행되는 환경
  - **클로드 코드**
    - 터미널에서 실행되는 AI 기반 코딩 도구로 파일을 직접 생성하고 수정하며 프로젝트를 관리할 수 있는 강력한 개발 파트너
  - **명령 프롬프트**
    - 윈도우의 기본 터미널 환경

SECTION 3-2 손글씨 인식 프로그램 만들기(1)

시작하기전에

◦ 손글씨 인식은 OC(ROptical Character Recognition), 광학 문자 인식의 한 분야

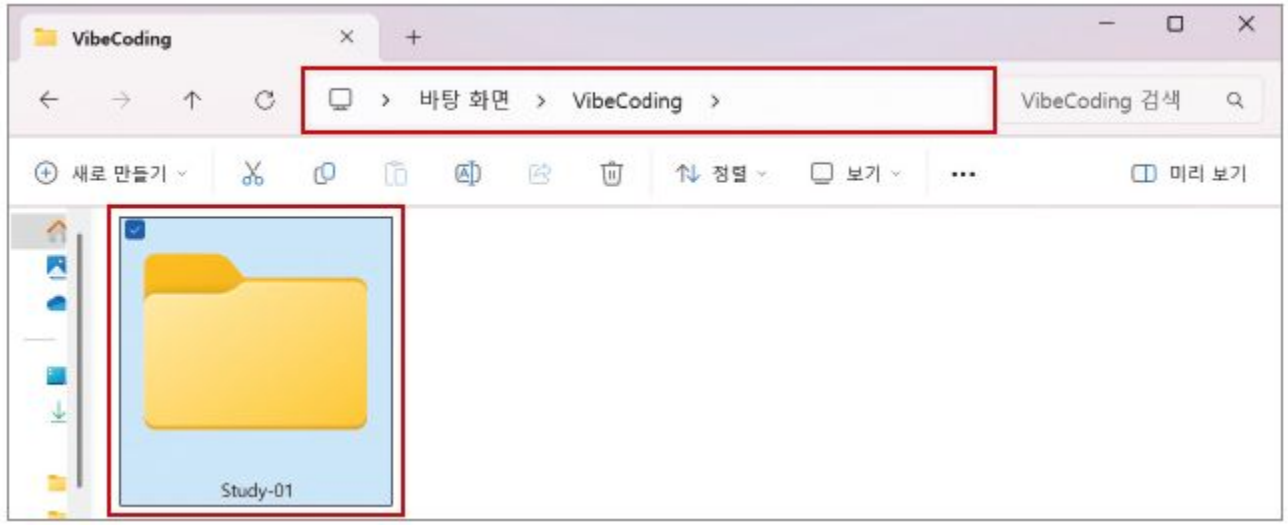
- OCR이 인쇄된 글자를 디지털 텍스트로 변환하는 기술이라면, 손글씨 인식은 사람마다 다른 글씨체를 인식해야 하므로 훨씬 까다로운 과제



SECTION 3-2 손글씨 인식 프로그램 만들기(2)

◦ 프로젝트 폴더 준비하기

- 클로드 코드를 실행하려면 먼저 작업할 프로젝트 폴더를 지정
  - 한 번 지정된 폴더에서 모든 작업이 이루어지기 때문에, 시작 단계에서 올바른 위치를 선택하는 것이 중요
- 바탕화면에 새로운 폴더 'VibeCoding'을 생성하고, 그 안에 'Study-01'이라는 하위 폴더를 생성



[note]

- **폴더 이름이나 파일 경로에 한글이 포함되면 프로그램 실행 시 인코딩 관련 오류가 발생할 수 있음**
- **프로젝트 폴더와 파일명에는 영문과 숫자만 사용하는 것을 권장**

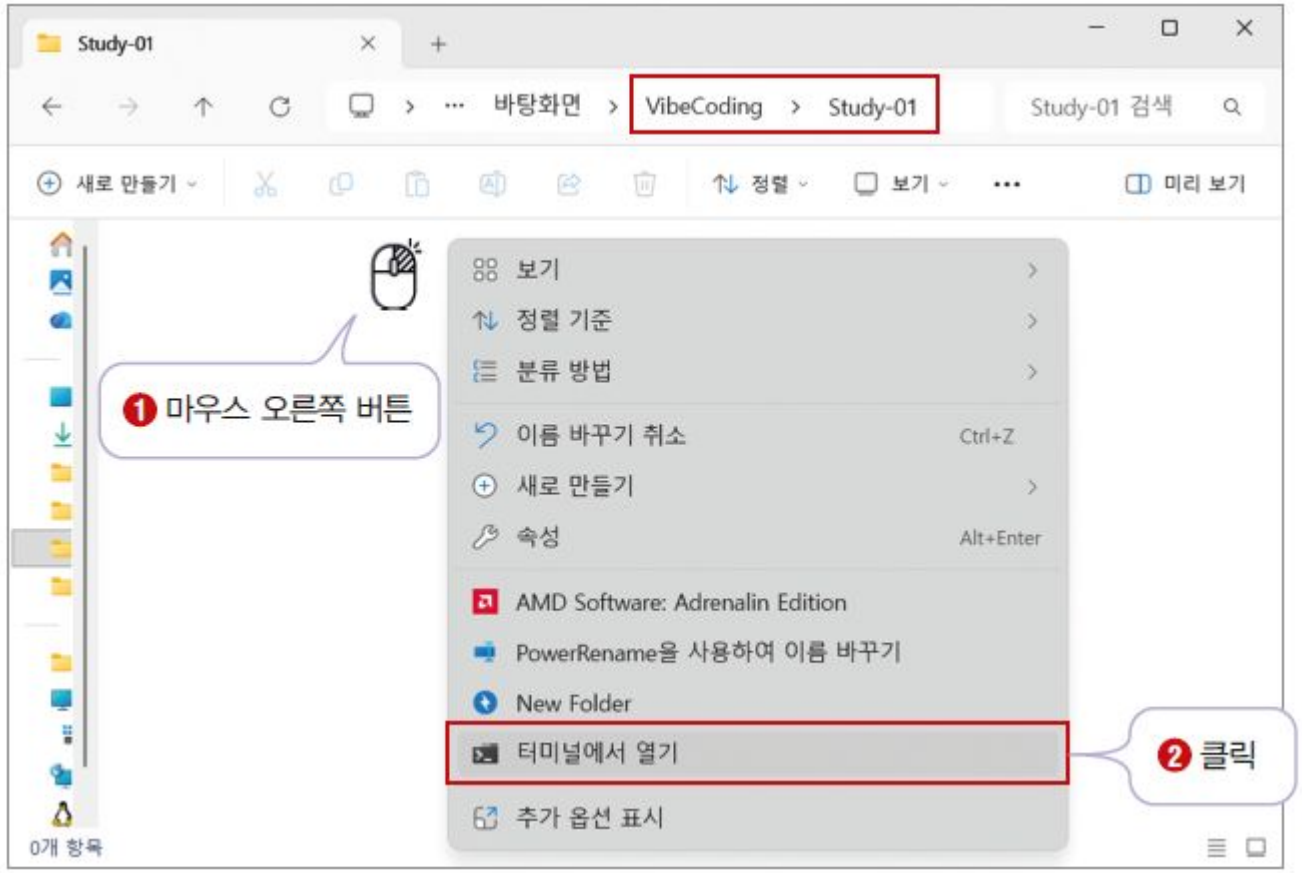
SECTION 3-2 손글씨 인식 프로그램 만들기(3)

- 클로드 코드는 명령 프롬프트 위에서 동작하는 프로그램
- 특히 CLI 프로그램은 특정 폴더를 기준으로 실행되기 때문에 작업할 폴더 위치에서 명령 프롬프트를 바로 여는 것이 효율적
- 지정된 디렉터리에서 명령 프롬프트를 실행하는 방법은 두 가지
  1) [터미널에서 열기] 선택하기
  2) 주소창에 'cmd' 입력하기

SECTION 3-2 손글씨 인식 프로그램 만들기(4)

첫째, [터미널에서 열기] 선택하기

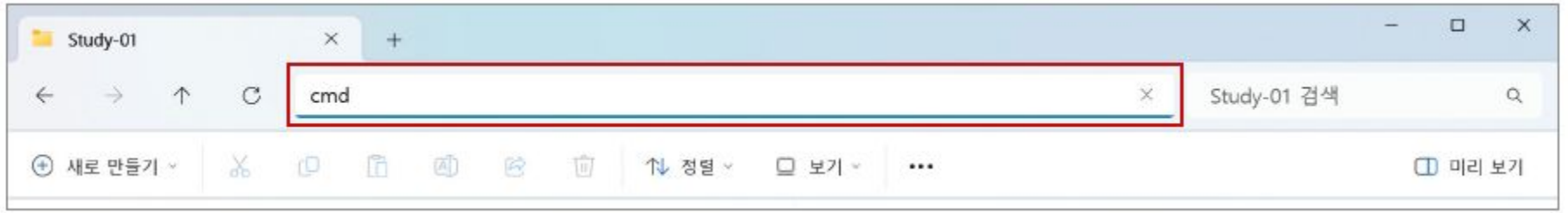
- ‘Study-01’ 폴더로 이동한 뒤 파일 탐색기의 빈 공간에 마우스 오른쪽 버튼을 클릭하고, [터미널에서 열기] 메뉴를 클릭



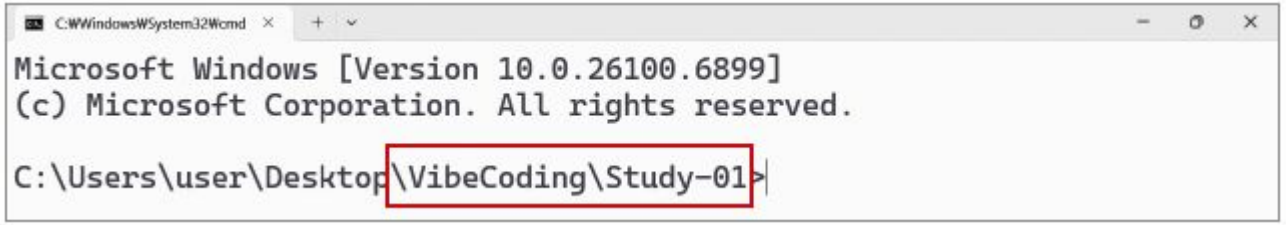
SECTION 3-2 손글씨 인식 프로그램 만들기(5)

둘째, 주소창에 'cmd' 입력하기

- 'Study-01' 폴더로 이동한 뒤 파일 탐색기의 주소창에 'cmd'라고 입력한 뒤, [Enter]



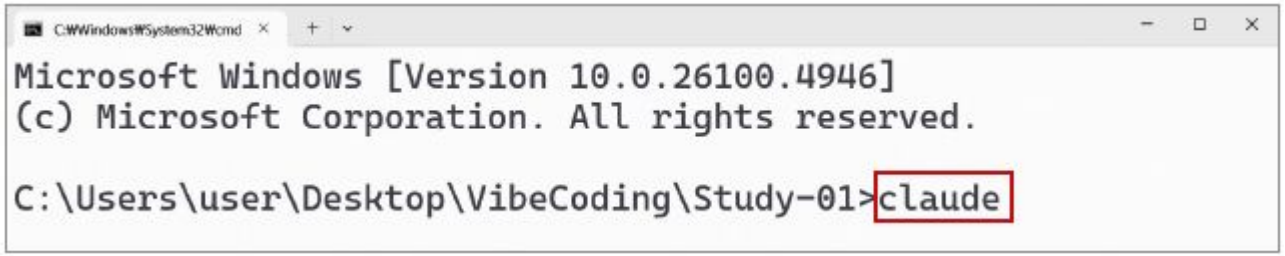
- 'Study-01' 폴더에서 명령 프롬프트가 실행



SECTION 3-2 손글씨 인식 프로그램 만들기(6)

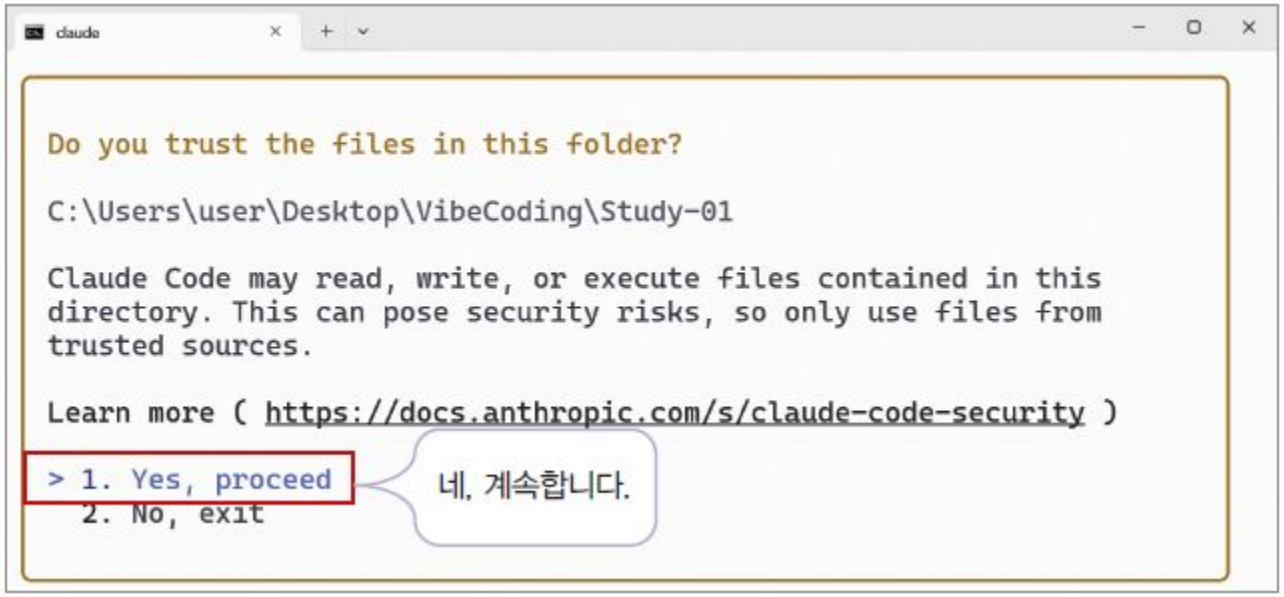
클로드 실행하기

01 명령 프롬프트에 ‘claude’를 입력



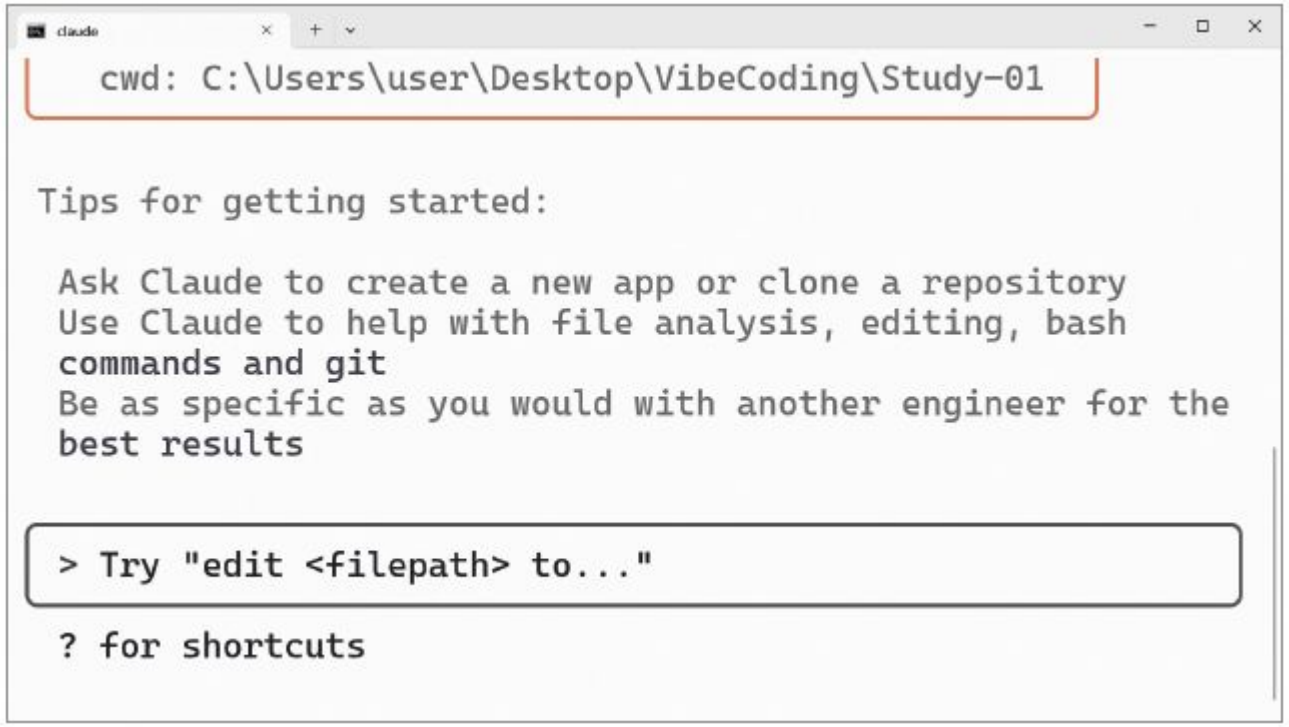
SECTION 3-2 손글씨 인식 프로그램 만들기(7)

02 클로드 코드의 초기 화면이 열리며 파일을 신뢰하는지 묻고 있음
'1. Yes, proceed'를 선택해서 진행



SECTION 3-2 손글씨 인식 프로그램 만들기(8)

03 대화를 시작할 수 있음



SECTION 3-2 손글씨 인식 프로그램 만들기(9)

✚ 여기서 잠깐 **지정된 폴더에서 클로드 코드를 시작하는 다른 방법**

- **Windows** 작업 표시줄의 검색창에 **'cmd'**를 입력해서 터미널을 실행한 뒤, 작업 폴더로 이동 후 클로드 코드를 실행해도 됨
- 터미널이 열리면 폴더의 위치 이동을 의미하는 명령어 **'cd'** 뒤에 공백과 함께 **'Study-01'** 폴더의 경로를 입력하고 **[Enter]**
- 폴더의 경로는 파일 탐색기 상단 주소창을 클릭해 복사한 ← 'Study-01' 폴더의 경로

cd C:\Users\user\Desktop\VibeCoding\Study-01

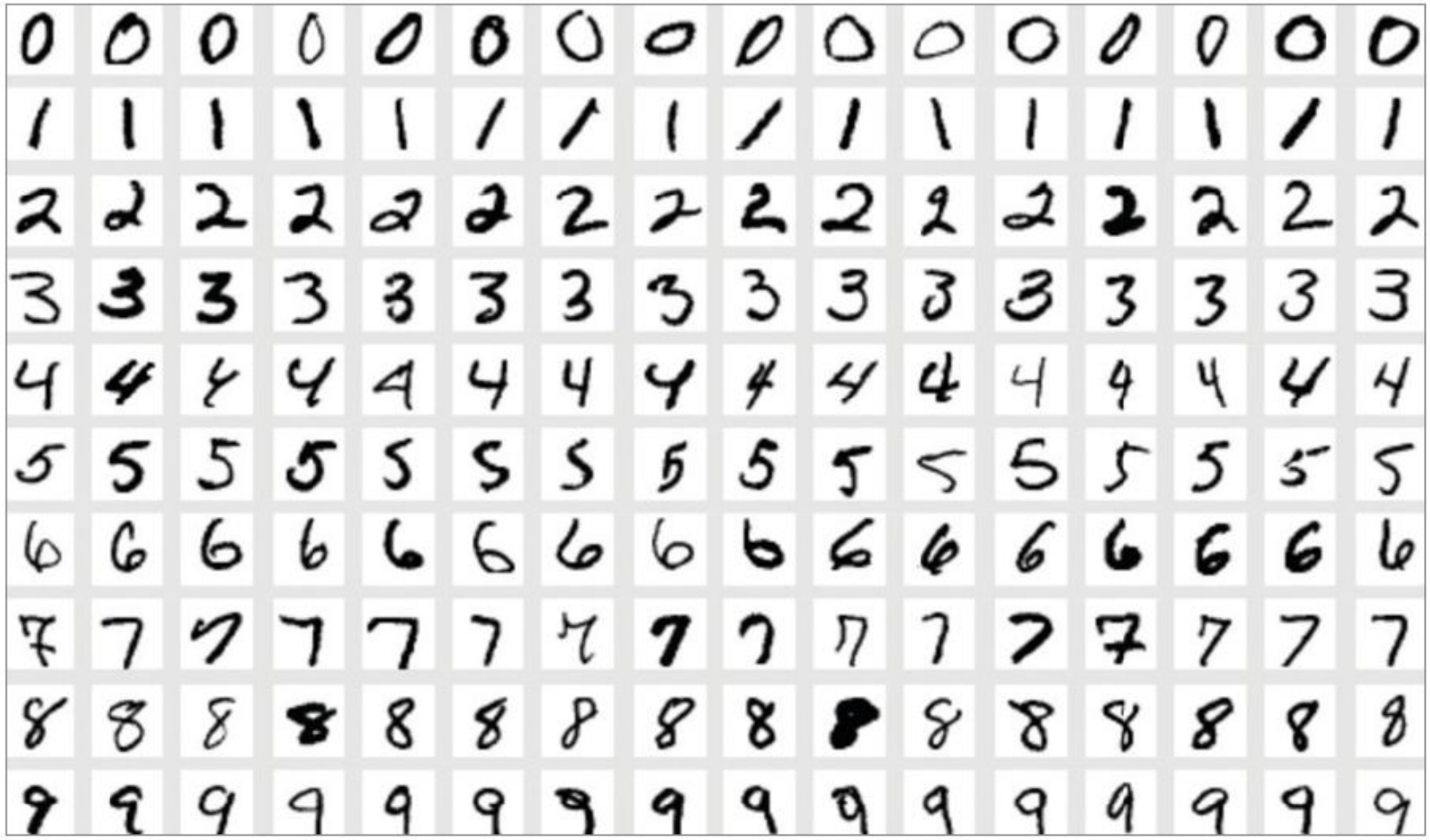
SECTION 3-2 손글씨 인식 프로그램 만들기(10)

◦ MNIST 데이터세트로 숫자 인식 프로그램 만들기

- 설치한 클로드 코드를 이용해 손글씨 인식 프로그램을 제작
  - 사용자가 직접 그린 숫자를 실시간으로 인식하는 AI를 구현
- MNIST(Modified National Institute of Standards and Technology) 데이터세트(dataset)를 사용
  - MNIST는 0부터 9까지 손글씨로 적힌 숫자 이미지 7만 장으로 구성
  - 각 이미지는 28×28 픽셀 크기의 흑백 숫자 데이터를 담고 있음
  - 1998년 공개된 이후 지금까지도 컴퓨터 비전과 머신러닝의 표준 데이터세트로 활용
  - 딥러닝 입문자들이 가장 먼저 접하는 데이터세트로 널리 알려져 있음

SECTION 3-2 손글씨 인식 프로그램 만들기(11)

MNIST 데이터세트의 일부



SECTION 3-2 손글씨 인식 프로그램 만들기(12)

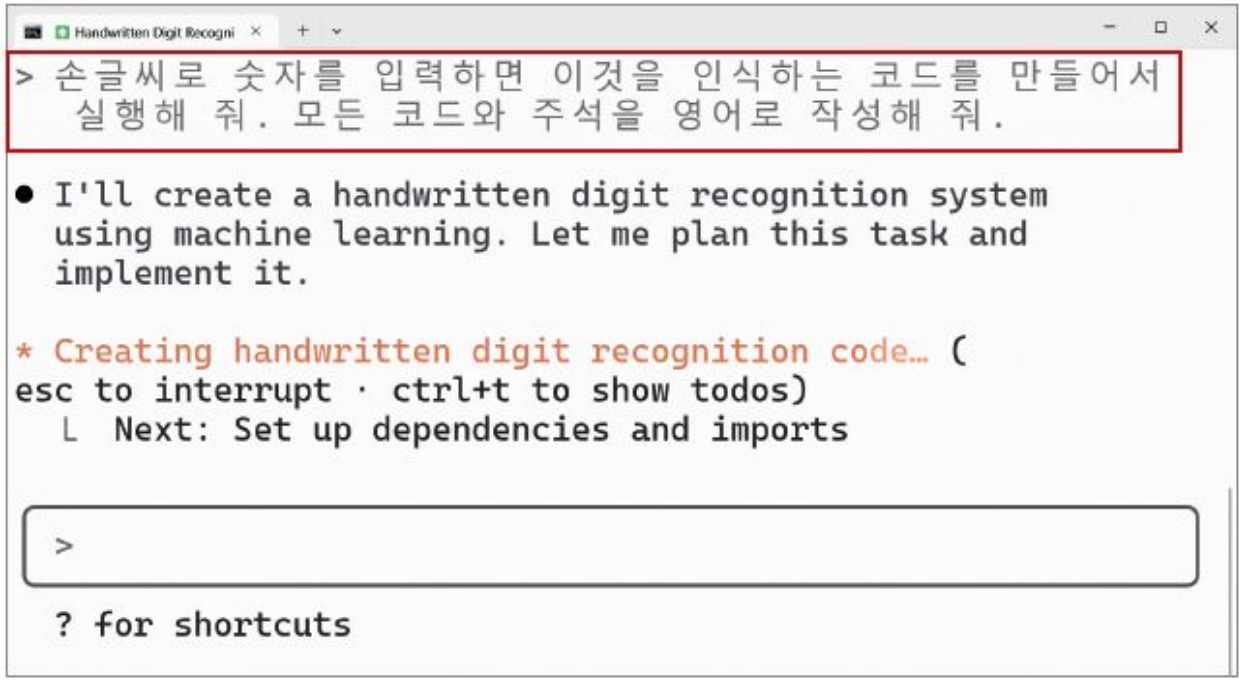
파이썬 코드 작성하기

- ‘Study-01’ 폴더 위치에서 클로드 코드를 실행

01 프롬프트 입력창에 다음과 같이 입력한 뒤 [Enter]
클로드 코드가 작업 계획을 보여주며 하나씩 스스로 실행하기 시작

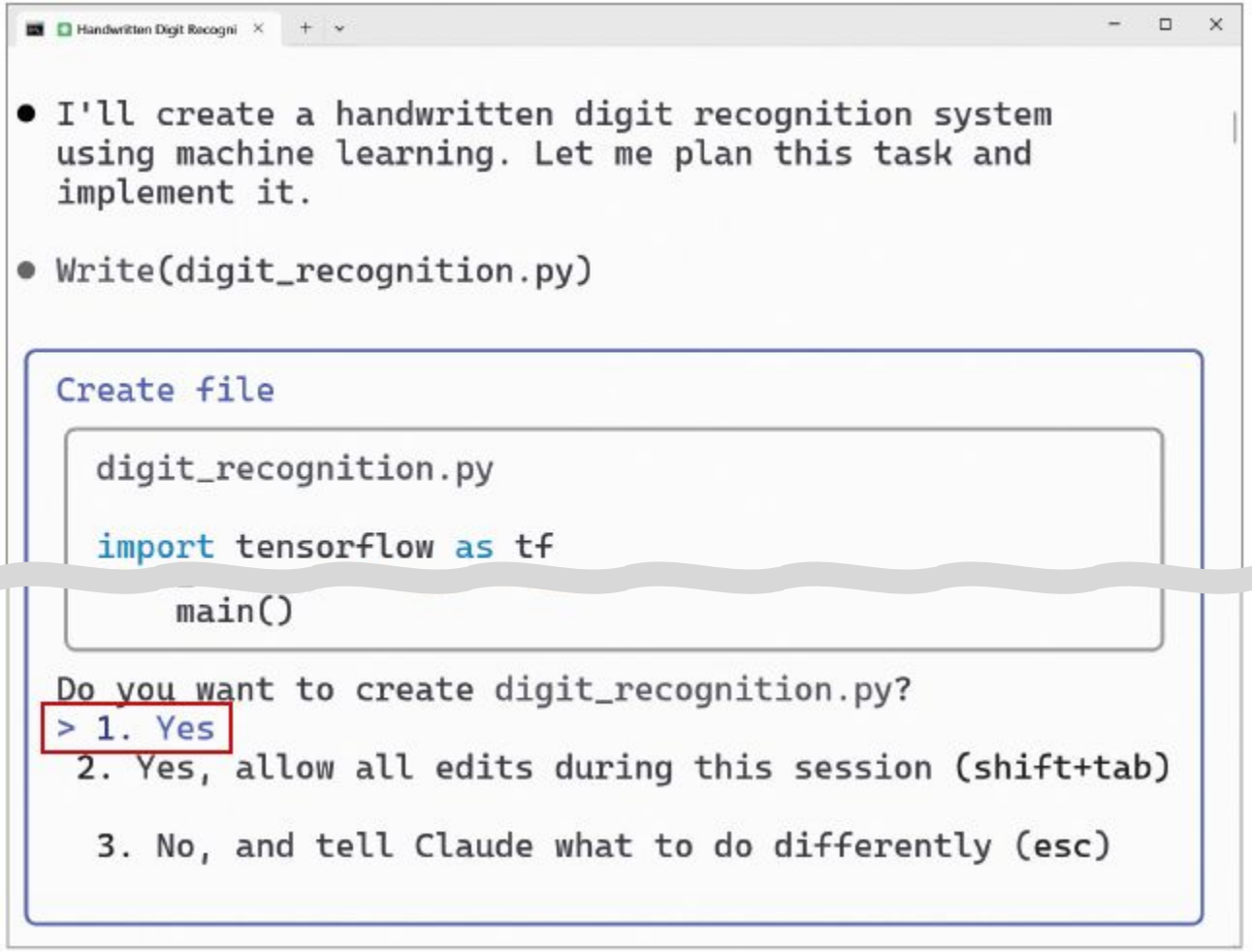
프롬프트

```
손글씨로 숫자를 입력하면 이것을 인식하는 코드를 만들어서 실행해 줘. 모든 코드와 주석을 영어로 작성해 줘.
```



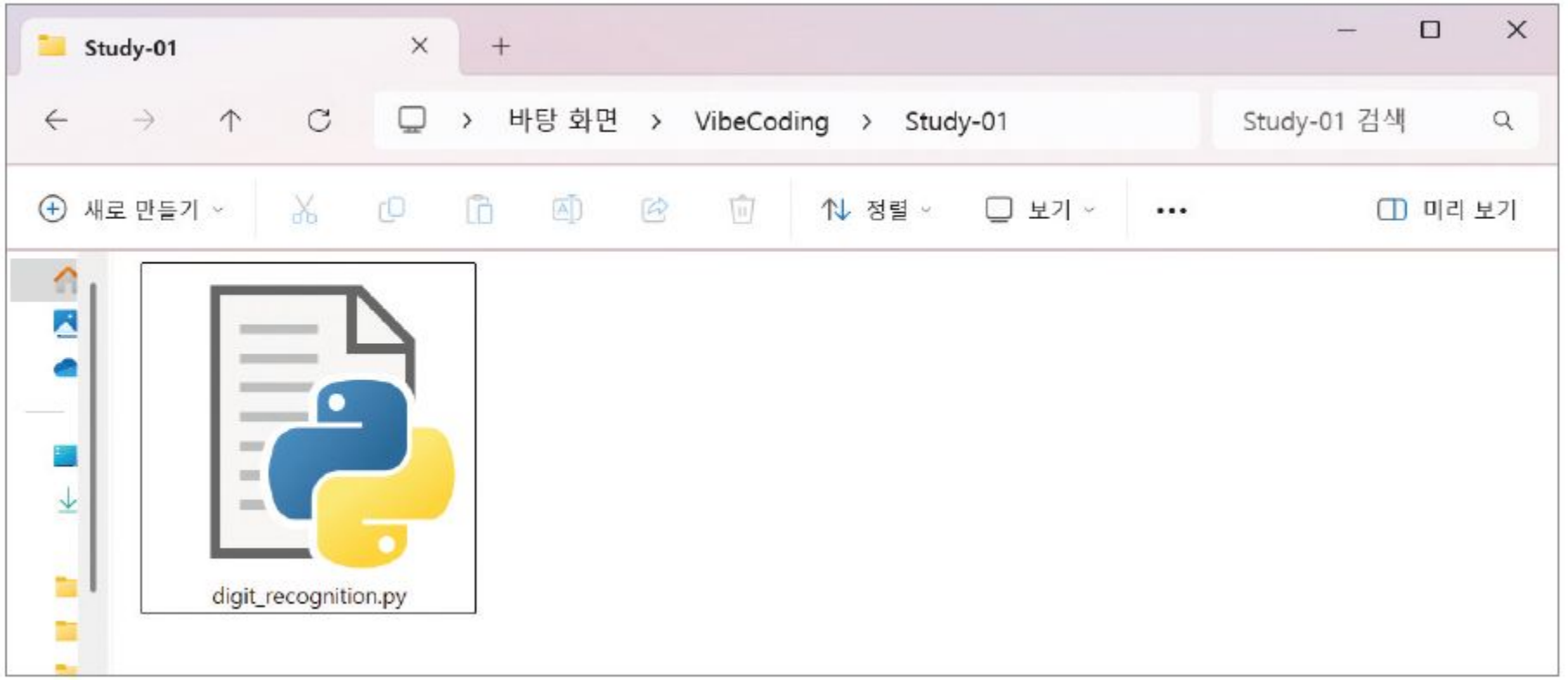
SECTION 3-2 손글씨 인식 프로그램 만들기(13)

02 클로드 코드가 파이썬으로 코드를 작성하고, ‘digit_recognition.py’ 파일로 생성할지를 질문
‘1. Yes’를 선택하고 [Enter]



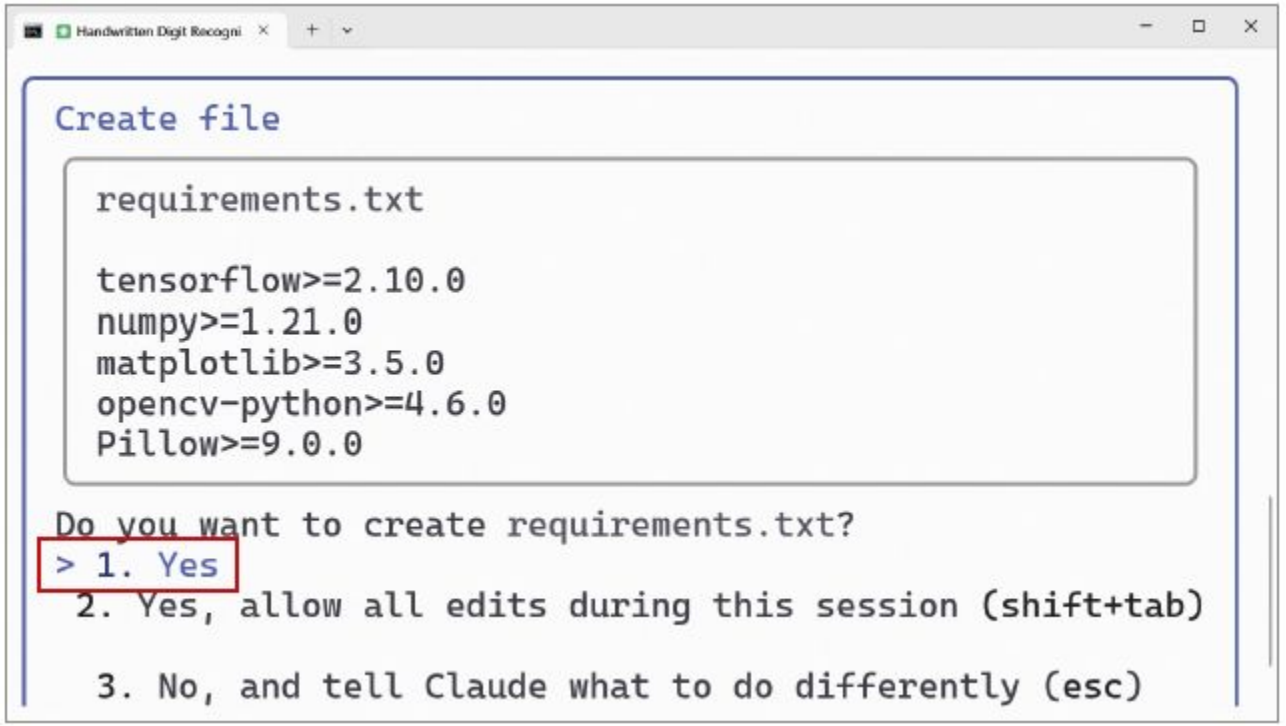
SECTION 3-2 손글씨 인식 프로그램 만들기(14)

03 ‘Study-01’ 폴더에 새로운 파이썬 파일이 생성



SECTION 3-2 손글씨 인식 프로그램 만들기(15)

04 프로젝트에 필요한 라이브러리 이름과 버전을 정리해 둔 텍스트 파일인 'requirements.txt'를 설치할 것인지 물어옴 - '1. Yes'를 선택하고 [Enter]



SECTION 3-2 손글씨 인식 프로그램 만들기(16)

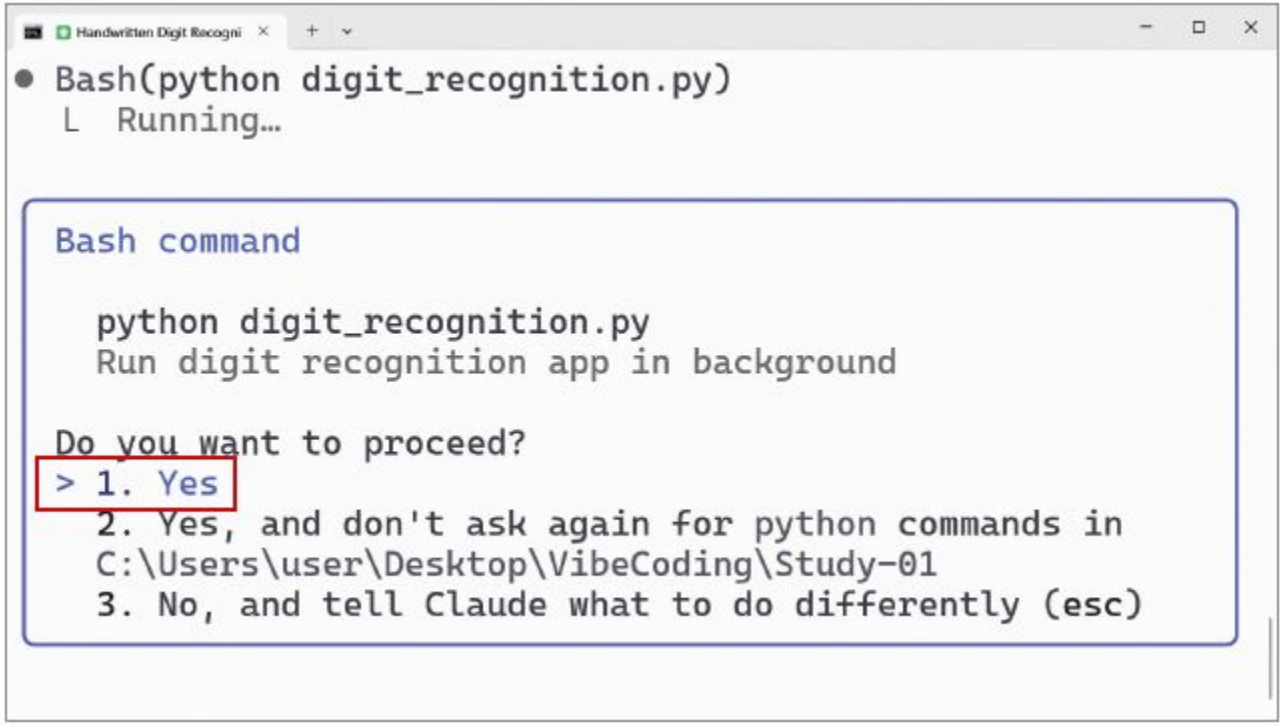
✚ 여기서 잠깐 **pip install -r requirements.txt**

- **pip install -r requirements.txt**는 프로젝트에 필요한 환경을 그대로 재현하기 위해 지정된 모든 라이브러리 **(library)**를
  한 번에 설치하는 명령어
- 협업이나 배포 환경에서 동일한 파이썬 실행 환경을 만들 때 사용
- **'require ments.txt'**를 이용해 일괄적으로 설치하지 않고 설치 여부를 일일이 물을 수도 있음
- 어떤 경우든 모두 **'1. Yes'**를 선택해 진행

SECTION 3-2 손글씨 인식 프로그램 만들기(17)

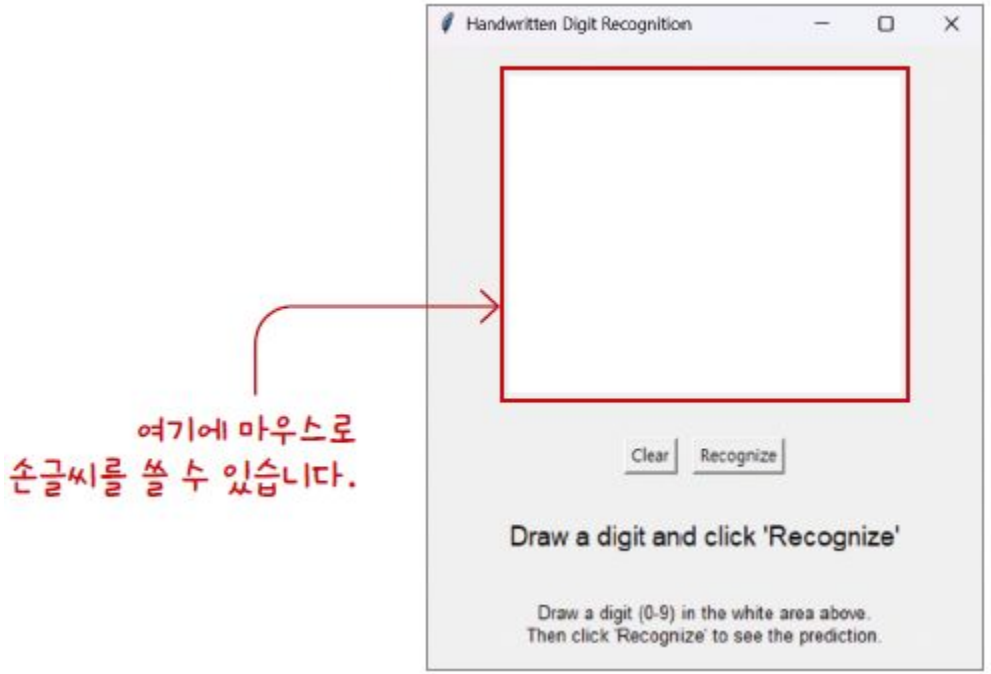
05 코드를 실행할지 묻는 메시지

'1. Yes'를 선택하고 [Enter]



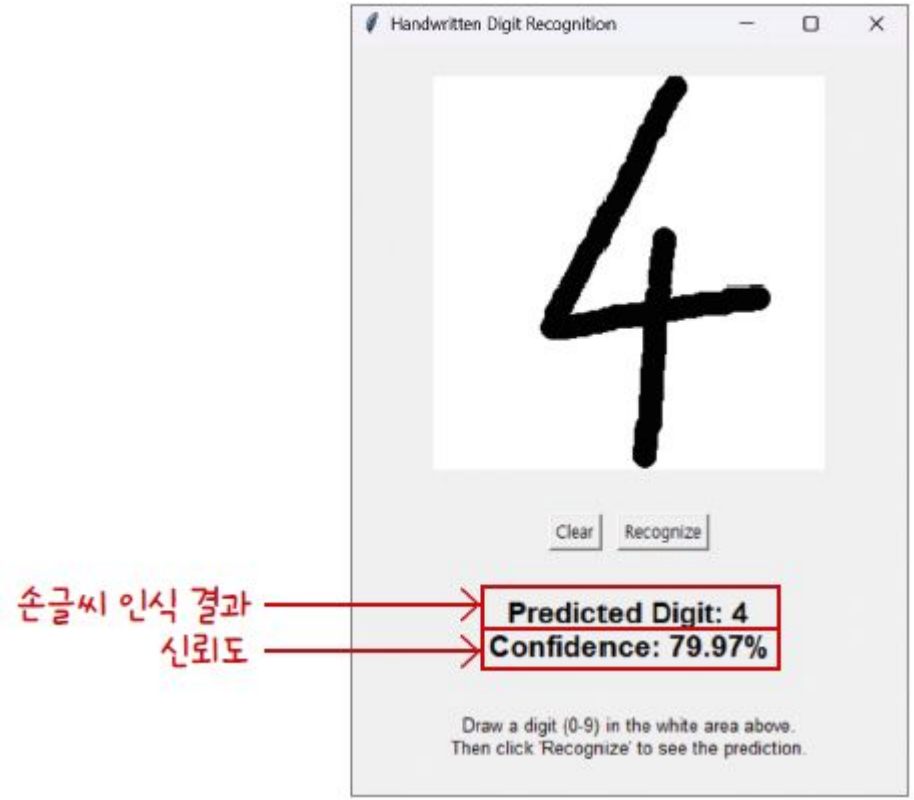
SECTION 3-2 손글씨 인식 프로그램 만들기(18)

06 잠시 기다리면 손으로 쓴 숫자를 인식하는 프로그램이 나타남
모습은 다양하지만, 가운데에 글씨를 쓰는 상자가 있고 그 아래에는 글씨를 지우는 [Clear] 버튼이,
손글씨 숫자를 인식하는 [Recognize] 버튼이 있음



SECTION 3-2 손글씨 인식 프로그램 만들기(19)

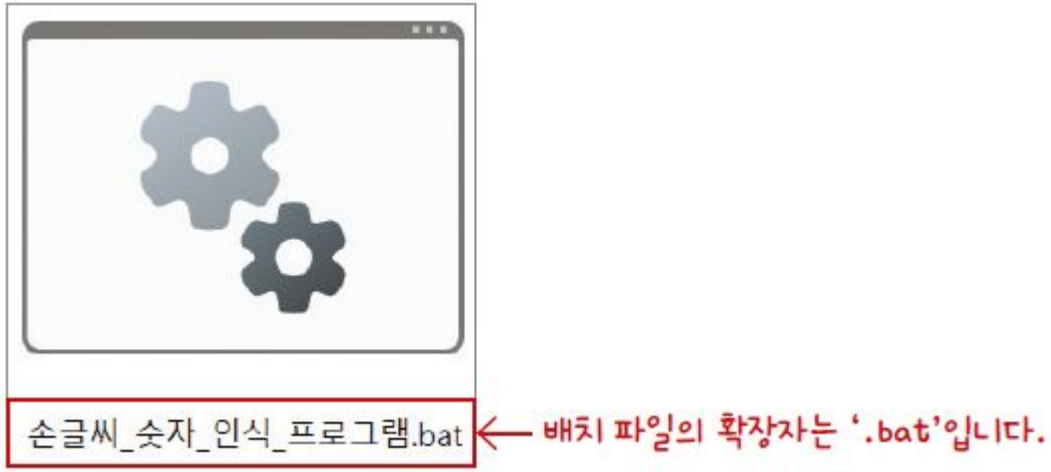
07 흰 상자 위에 마우스로 0부터 9까지 숫자를 그린 뒤, [Reco gnize] 버튼을 클릭
손글씨를 인식한 결과와 신뢰도를 출력



SECTION 3-2 손글씨 인식 프로그램 만들기(20)

독립적인 실행 파일 만들기

- 클로드 코드 없이도 독립적으로 실행할 수 있도록 만들기
  - 이를 위해 배치 파일(batch file)을 생성
  - 배치 파일은 윈도우의 여러 명령어를 한 번에 실행하도록 모아 둔 텍스트 파일
    - 반복적인 명령을 자동화하는 데 유용
- 손글씨 인식 프로그램을 배치 파일로 만들어 두면 윈도우의 탐색기에서 파일을 더블 클릭하여 프로그램을 실행



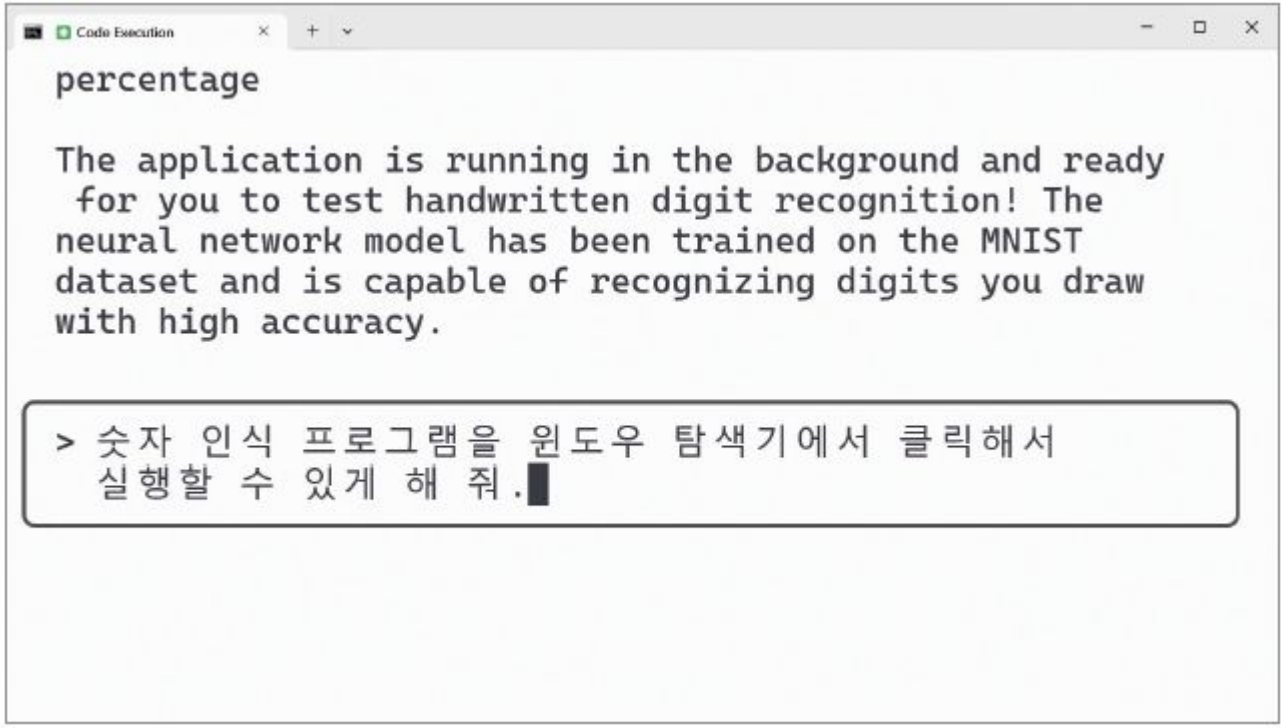
SECTION 3-2 손글씨 인식 프로그램 만들기(21)

- 열려 있는 손글씨 인식 프로그램은 닫고 클로드 코드가 실행중인 터미널에서 다음과 같이 진행

01 클로드 코드의 입력란에 다음과 같이 프롬프트를 작성하고 [Enter]

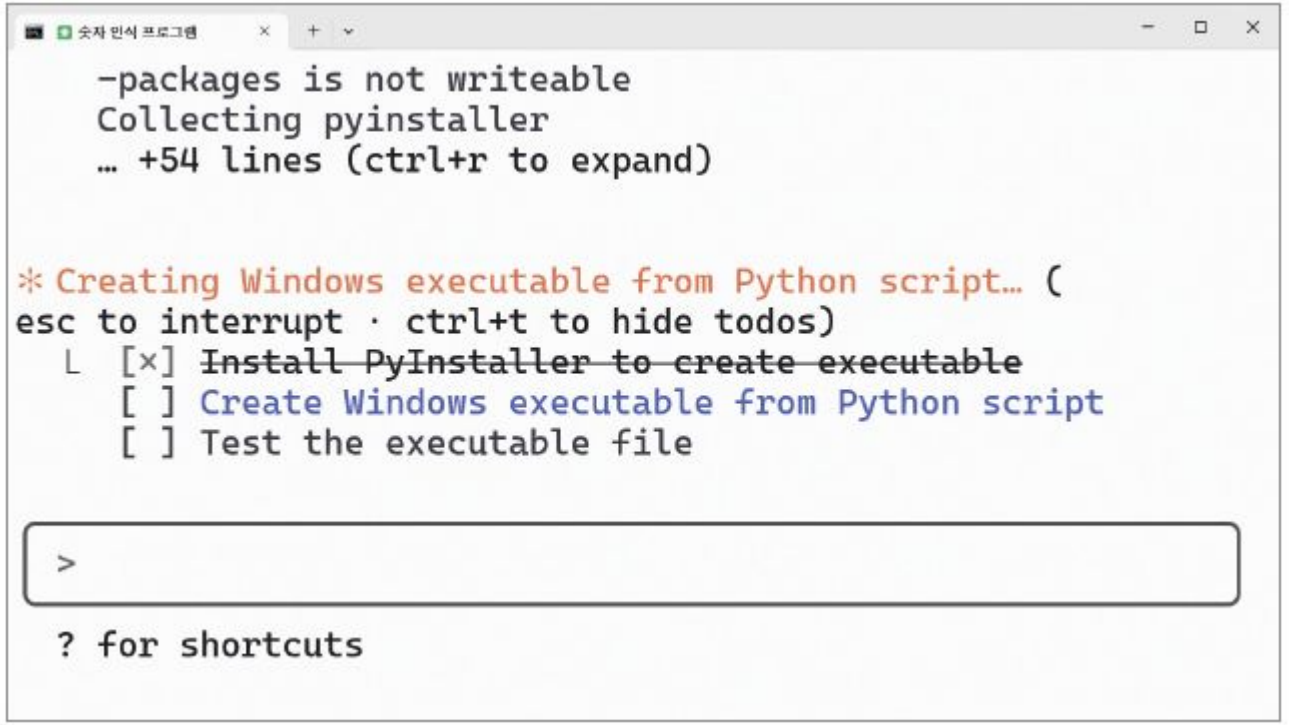
프롬프트

숫자 인식 프로그램을 윈도우 탐색기에서 클릭해서 실행할 수 있게 해 줘.



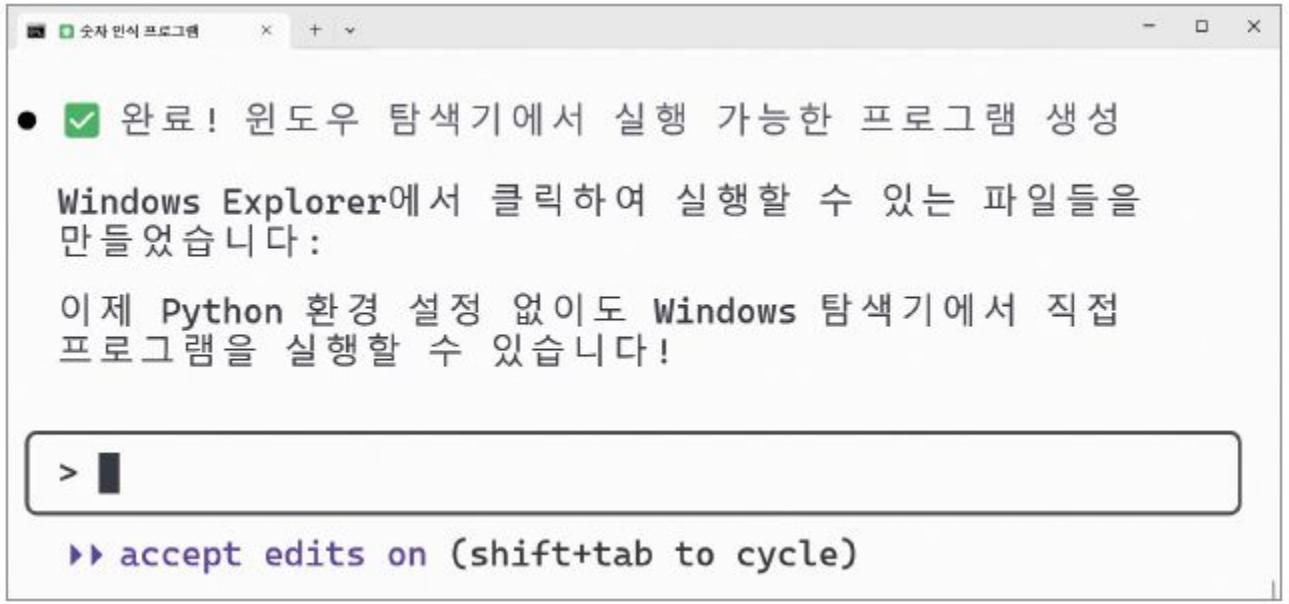
SECTION 3-2 손글씨 인식 프로그램 만들기(22)

02 클로드 코드가 파이썬 스크립트를 실행 가능한 파일로 생성
단축키 [Ctrl]+[T] 키를 눌러 실행 절차를 확인
터미널상에 보이는 안내에 따라 클로드 코드가 순서대로 수행



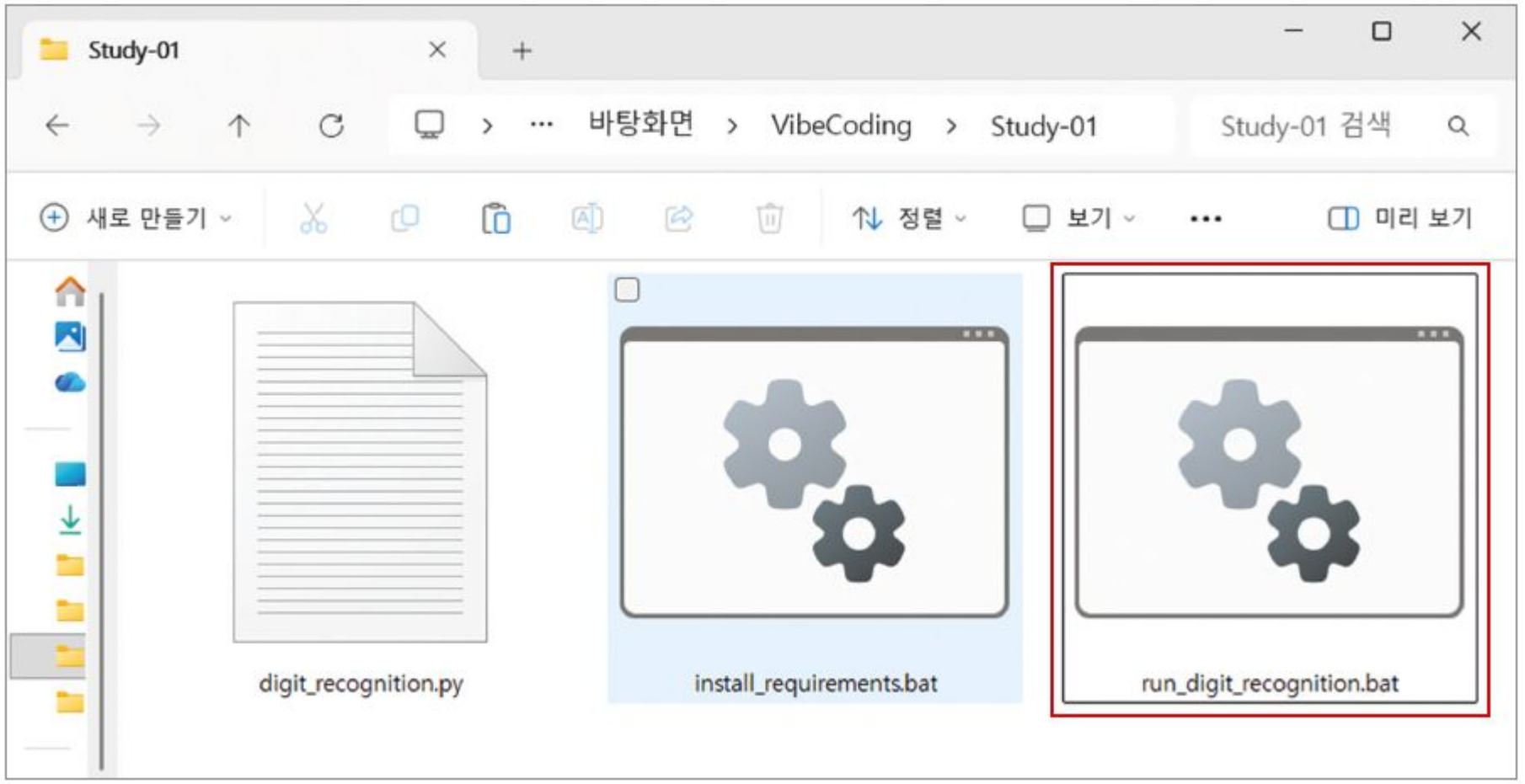
SECTION 3-2 손글씨 인식 프로그램 만들기(23)

03 파일을 생성했다는 메시지가 출력



SECTION 3-2 손글씨 인식 프로그램 만들기(24)

04 'Study-01' 폴더에 배치(.bat) 파일이 생성



SECTION 3-2 손글씨 인식 프로그램 만들기(25)

✚ 여기서 잠깐 **파일 권한 모드**

클로드 코드에서 파일을 다루다 보면, 이 코드를 파일로 저장할 것인지 확인하는 메시지가 나타남
이때 선택한 권한 모드에 따라 이후 파일 작업 방식이 달라짐

클로드 코드가 제공하는 권한 모드

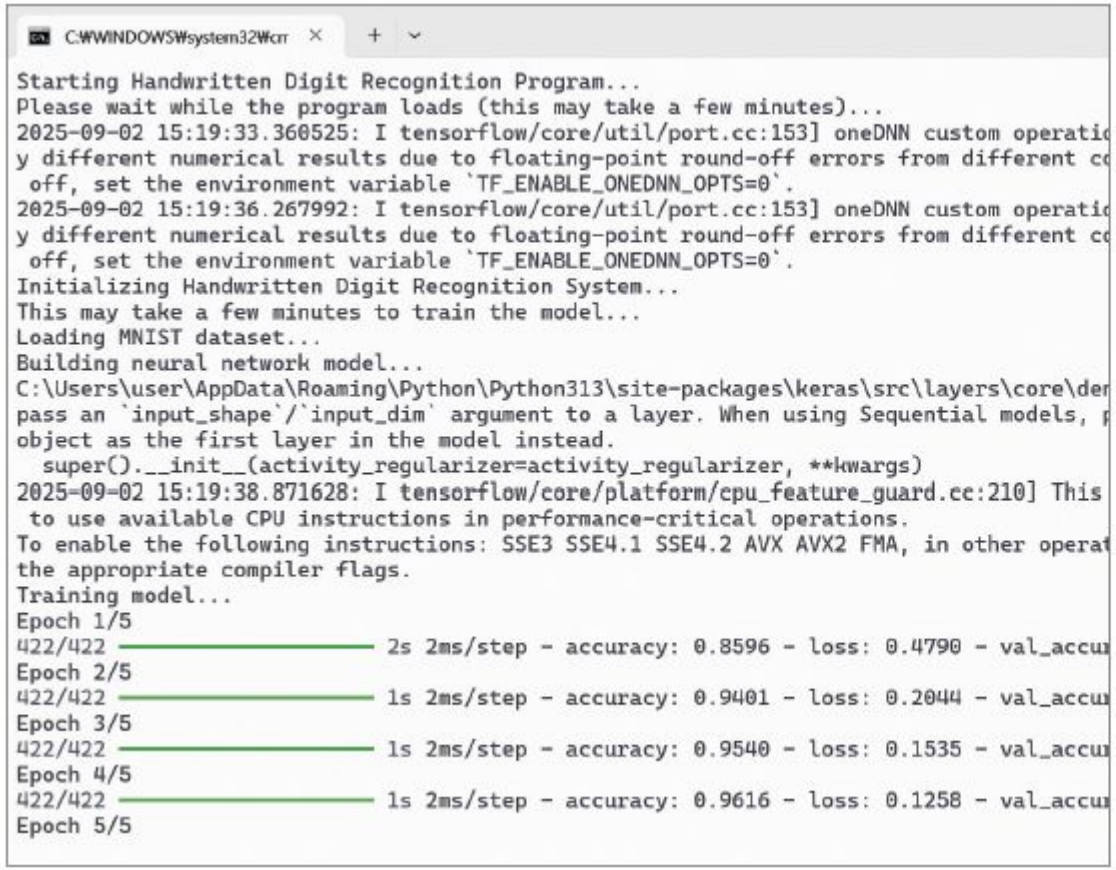
- **Ask every time:** 기본 설정으로, 파일을 저장하거나 수정할 때마다 확인 요청
- **Yes, for this session only:** 현재 세션 동안 별도의 확인 없이 파일 작업 허용
- **Always allow in this project:** 해당 프로젝트에서는 항상 파일 작업을 허용

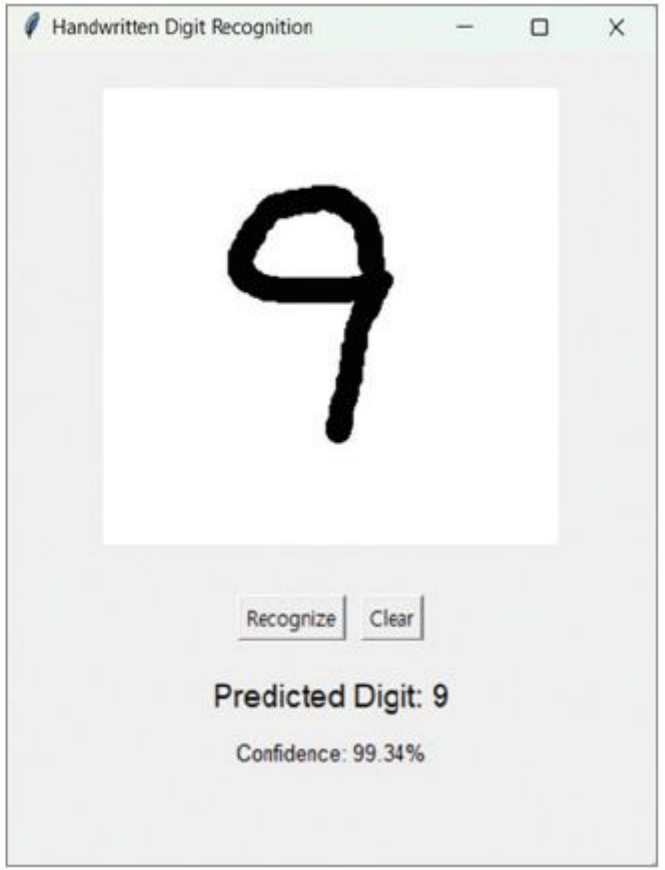
프로젝트 규모에 따라 **'Yes, and always allow modifying files in this project'** 옵션이 나타날 수도 있음

- 이 경우, 현재 프로젝트 폴더 내에서 파일을 자유롭게 생성하고 수정할 수 있어, 복잡한 작업을 효율적으로 진행

SECTION 3-2 손글씨 인식 프로그램 만들기(26)

05 배치 파일(.bat)을 더블 클릭해 실행
명령 프롬프트 창이 열리며 AI 모델 학습이 시작
학습이 완료되면 손글씨 인식 프로그램 창이 실행





[마무리]

- 3가지 키워드로 정리하는 핵심 포인트
  - **손글씨 인식 -** 사람이 쓴 글자나 숫자를 컴퓨터가 자동으로 판독해 디지털 데이터로 변환하는 AI 기술
  - **MNIST -** 0부터 9까지 손으로 쓴 숫자 이미지 7만 장으로 구성된 데이터세트로, 딥러닝과 컴퓨터 비전 분야에서 가장 널리 쓰이는 학습용 자료
  - **배치 파일 -** 윈도우 운영체제에서 여러 명령어를 순서대로 기록해 두었다가 실행 시 자동으로 처리하는 텍스트 파일

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(1)

시작하기전에

- 설정 파일(configuration file)
  - 프로그램이 실행될 때 참고하는 규칙이나 환경 정보를 담은 문서
  - 파이썬 프로젝트에서 필요한 라이브러리 등을 담은 'requirements.txt' 같은 파일이 대표적
  - 사용자가 직접 소스 코드를 고치지 않아도 설정 파일만 수정하면 프로그램의 동작 환경을 바꿀 수 있다는 점이 특징
- 클로드 코드는 CLAUDE.md라는 설정 파일을 사용
  - 이 파일에 프로젝트 구조, 목표, 개발 규칙 등을 저장해 두면 클로드 코드가 언제든 프로젝트 맥락을 이해하고 이어서 작업
  - 또한 여러 하위 프로젝트가 있을 경우, 각각에 맞는 설정 파일을 만들어 특성에 맞게 관리

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(2)

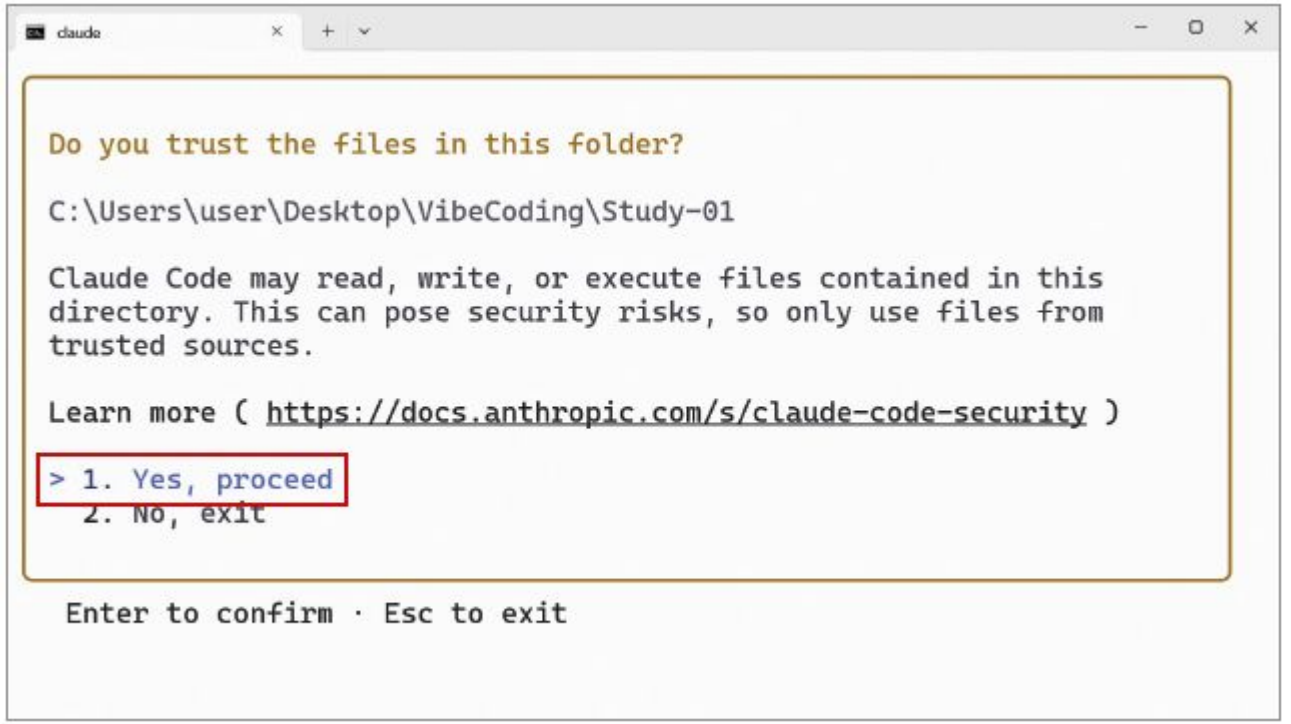
◦ 프로젝트 설정 파일의 이해

프로젝트 설정 파일 만들기

- 윈도우 탐색기의 ‘Study-01’ 폴더에서 마우스 오른쪽 버튼을 클릭한 뒤 [터미널에서 열기] 메뉴를 선택하거나, 주소 표시줄에 ‘cmd’를 입력하고 Enter 키를 눌러 터미널을 시작
- 명령 프롬프트에 ‘claude’를 입력해 클로드 코드를 실행

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(2)

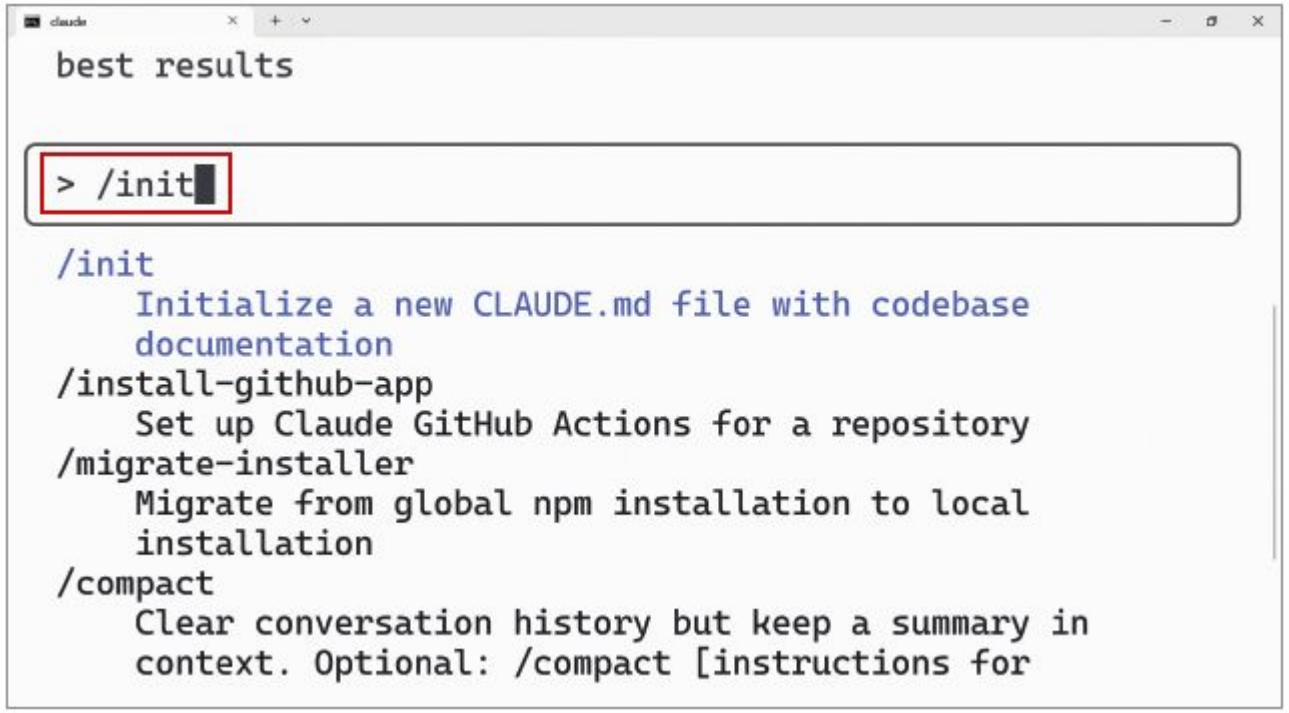
01 클로드가 현재 폴더의 내용을 신뢰할 수 있는지를 물으면 '1. Yes'를 선택하고 [Enter]



SECTION 3-3 CLAUDE.md로 프로그램 확장하기(3)

02 손글씨 인식 프로그램을 웹에서 구현하기 위해 클로드 코드에 초기화 명령어 '/init'을 입력해서 현재 프로젝트 폴더의 코드와 파일 구조를 분석하고, CLAUDE.md 파일을 자동으로 생성
입력창에 다음과 같이 초기화 명령어를 입력하고, [Enter]

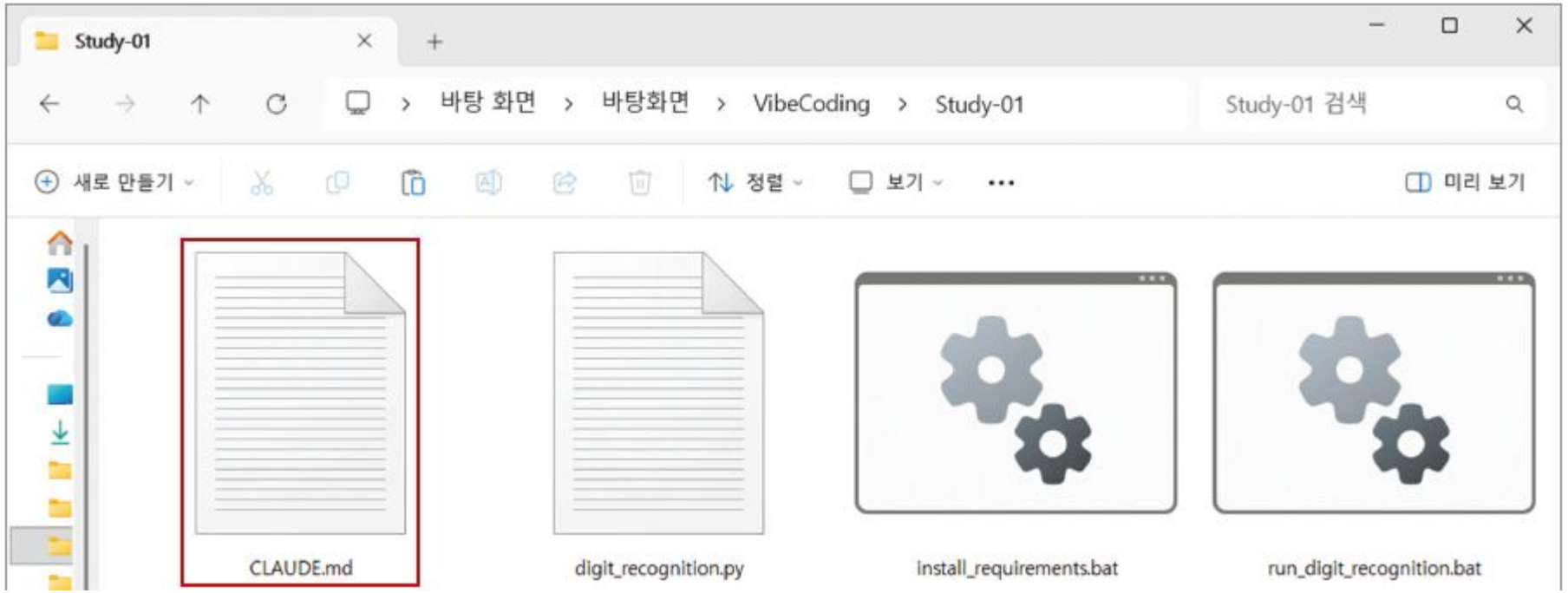
```
/init
```



SECTION 3-3 CLAUDE.md로 프로그램 확장하기(4)

03 클로드 코드가 현재 폴더에 있는 파이썬 파일에서 라이브러리, 클래스 구조, 함수를 분석하여 체계적인 문서를 만들고, 'Study-01' 폴더에 'CLAUDE.md' 파일을 생성

- 프로젝트를 진행하며 새로운 기능을 구현하거나 중요한 설정을 변경할 때는, CLAUDE.md에 내용을 추가
- 클로드 코드가 항상 최신 맥락을 유지



SECTION 3-3 CLAUDE.md로 프로그램 확장하기(5)

✚ 여기서 잠깐 **'/init' 명령어와 초기화**

- '/init'은 초기 설정을 준비하거나 실행 환경을 새로 준비하는 명령어
  - '초기화'를 의미하는 영단어 'initialization'의 앞 네 글자
- '/init'을 실행하면 클로드 코드가 프로젝트를 분석해서 CLAUDE.md 파일을 생성하고, 프로젝트 구조, 기술 스택, 주요 명령어 등을 자동으로 문서화

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(6)

프로젝트 설정 파일의 구조

- '/init' 명령으로 생성된 CLAUDE.md 파일은 프로젝트의 핵심 정보를 간단명료하게 정리
- 클로드 코드가 프로젝트 설정을 즉시 이해할 수 있도록 구조화하여, 구체적이고 명확하게 작성

일반적인 CLAUDE.md 파일 구조

```
# 프로젝트명
프로젝트에 대한 간단한 설명
## Project Overview
무엇을 하는 프로젝트인지 한두 문장으로 요약
## Commands
- 설치 명령, 실행 명령, 테스트 명령
## Tech Stack
- 파이썬 버전, 라이브러리 등 설명
## Architecture
- 핵심 클래스와 기능 설명
- 파일 구조와 각 파일의 역할
## Code Style
- 코드와 주석 작성, 에러 처리, 함수명 스타일
## Development Notes
- 현재 상태와 제한 사항
```

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(7)

설정 파일 수정하고 활용하기

- 프로젝트를 진행하면서 새로운 규칙이나 설정이 필요하면 언제든지 CLAUDE.md 파일을 수정
  - 대화 중에 클로드 코드에게 직접 요청하는 방법이 가장 편리
  - 예) 다음과 같이 요청하면 클로드 코드가 파일을 자동으로 수정

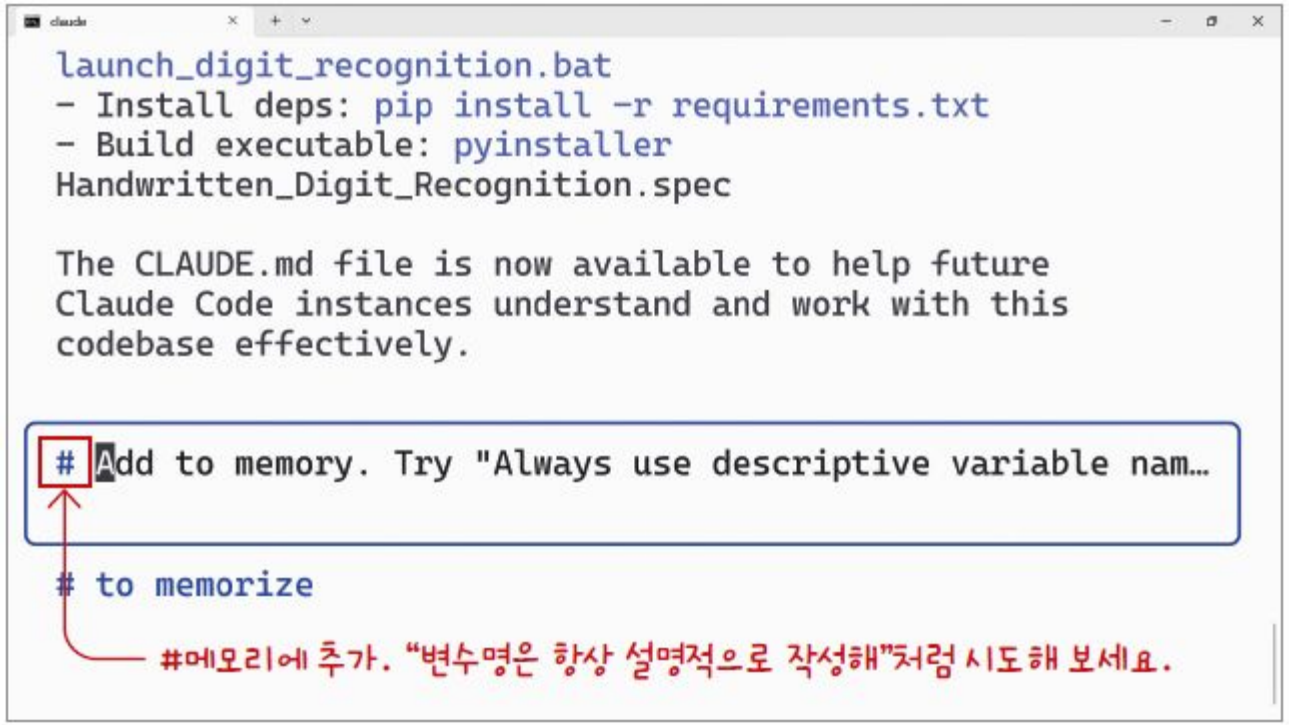
```
CLAUDE.md 파일에 이 코딩 규칙을 추가해 줘.
```

```
프로젝트 구조가 변경되었으니 CLAUDE.md를 업데이트해 줘.
```

- 더 간단한 방법
  - CLAUDE.md에 업데이트하고 싶은 수정 사항이 있을 경우, # 기호를 사용해 프롬프트상에서 즉시 추가
  - '#'은 프로젝트 메모리 업데이트용 단축 지시어
  - 클로드 코드와 대화하는 도중에 '#' 키를 입력하면 일반 대화 프롬프트와 달리 현재 작업이나 규칙을 CLAUDE.md 파일에 빠르게 추가

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(8)

01 클로드 코드 입력창에 '#'을 입력
클로드 코드가 그 뒤로 이어질 말을 메모리에 추가할 지침으로 인식

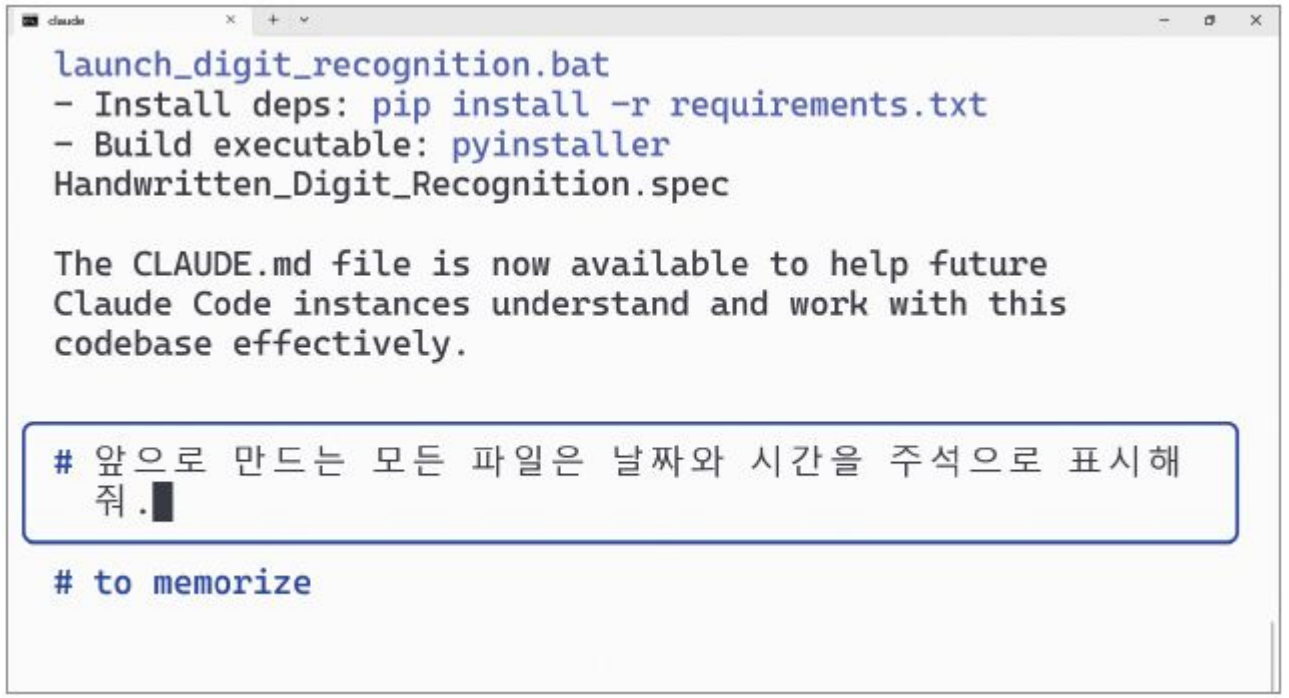


SECTION 3-3 CLAUDE.md로 프로그램 확장하기(9)

02 다음과 같이 지침을 입력하고, [Enter]

프롬프트

```
# 앞으로 만드는 모든 파일은 날짜와 시간을 주석으로 표시해 줘.
```

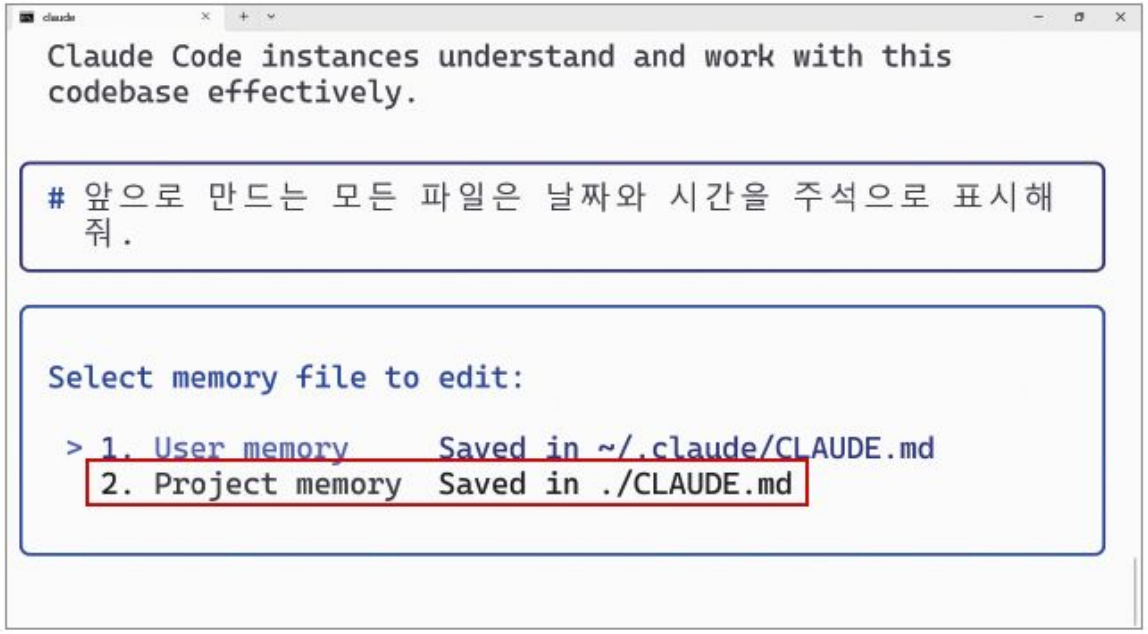


SECTION 3-3 CLAUDE.md로 프로그램 확장하기(10)

03 클로드 코드는 입력한 규칙을 어디에 저장할지 선택할 수 있는 옵션을 제공
각 옵션은 적용 범위가 다름
여기서는 해당 폴더에서만 규칙을 적용하고 다른 프로젝트에는 영향을 주지 않게 하기 위해
‘2. Project memory’를 선택하고 [Enter]

화면 해설

1. User memory (~/.claude/CLAUDE.md): 사용자 홈 디렉터리에 저장, 모든 프로젝트에 공통으로 적용
2. Project memory(./CLAUDE.md): 현재 프로젝트의 디렉터리에 저장, 현재 프로젝트에서만 적용



SECTION 3-3 CLAUDE.md로 프로그램 확장하기(11)

◦ 계층적 구조 활용하여 웹 버전 손글씨 인식 프로그램 만들기

계층 구조 알아보기

- 컴퓨터에서 파일을 정리할 때 가장 기본이 되는 단위는 디렉터리(directory), 즉 폴더(folder)
- 디렉터리에는 파일뿐 아니라 또 다른 하위 디렉터리를 넣을 수 있어, 마치 나무가 가지를 뻗는 것처럼 계층 구조(hierarchical structure)를 형성
- 최상위(root) 폴더 아래 여러 하위 폴더가 이어지고, 그 안에 다시 파일이나 더 작은 폴더가 들어감
- 클로드 코드는 이러한 폴더의 계층구조를 그대로 인식
- 클로드 코드가 시작되면 먼저 현재 디렉터리와 폴더의 하위 구조를 파악하고, 작업 중 필요에 따라 하위 폴더의 내용을 탐색

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(12)

- ‘Study-01’ 디렉터리 아래에 ‘desktop_version’ 폴더를 만들어 데스크톱 전용 규칙을 저장하고, ‘web_version’ 폴더를 만들어 웹 버전 전용 규칙을 저장
  - 계층 구조에 맞추어 CLAUDE.md도 각각 따로 저장
  - 프로젝트 루트에 해당하는 ‘Study-01’ 폴더에는 공통 규칙을, 각 하위 폴더에는 해당 버전별 특화 규칙을 담아서 서로 다른 CLAUDE.md 파일을 유지하는 것이 효율적

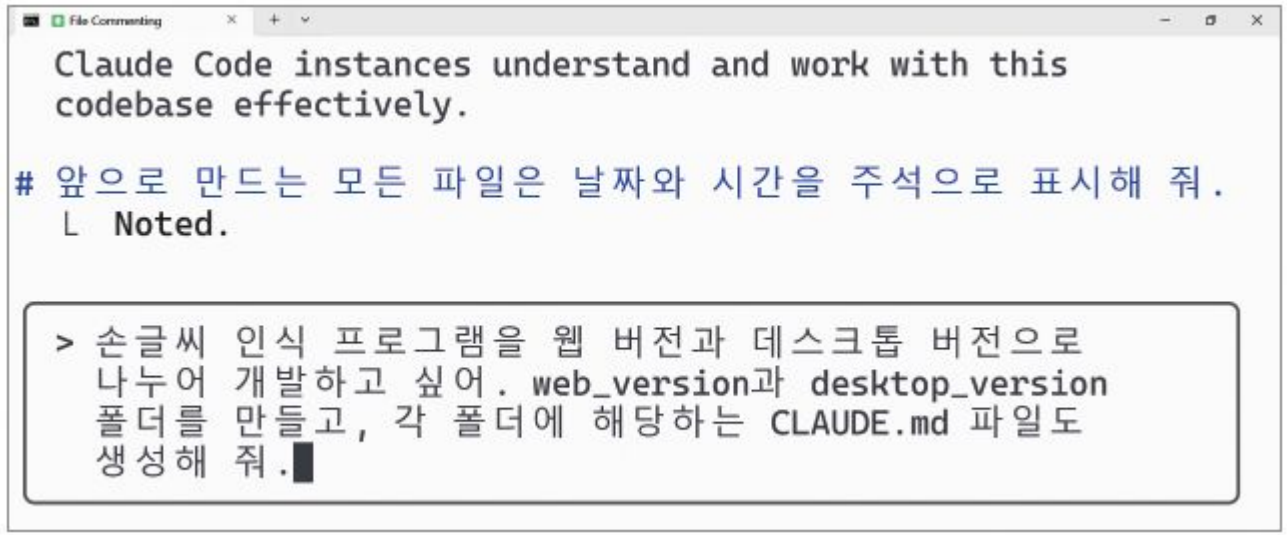
SECTION 3-3 CLAUDE.md로 프로그램 확장하기(13)

계층적 구조 만들기

01 클로드 코드에 다음과 같이 요청

프롬프트

손글씨 인식 프로그램을 웹 버전과 데스크톱 버전으로 나누어 개발하고 싶어. web_version과 desktop_version 폴더를 만들고, 각 폴더에 해당하는 CLAUDE.md 파일도 생성해 줘.



SECTION 3-3 CLAUDE.md로 프로그램 확장하기(14)

02 클로드 코드가 필요한 디렉터리를 만들고 각 디렉터리에 필요한 파일들을 생성하기 시작
'1. Yes'를 선택해 내용을 확인하며 진행 - 클로드 코드가 작업을 완료한 후 결과를 보여줌

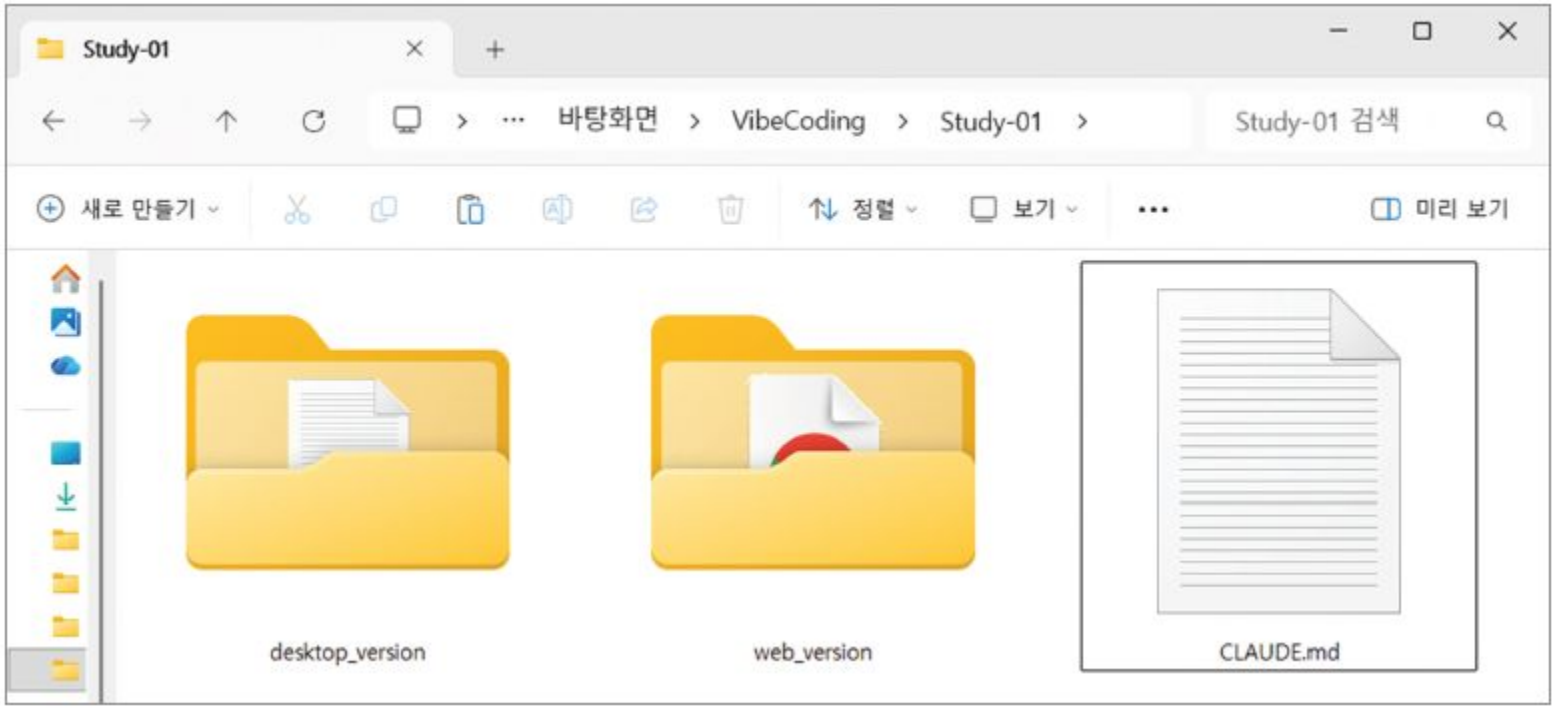
[note]

- **'2. Yes, and don't ask again..'을 선택해 유사한 명령은 자동으로 진행되게 할 수도 있음**

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(15)

03 파일 탐색기에서 'Study-01' 폴더를 확인

'web_version' 폴더에는 웹 애플리케이션이, 'desktop_version' 폴더에는 기존 데스크톱 버전 코드가 정리된 것을 볼 수 있음 - 각 폴더에는 해당 버전에 맞는 CLAUDE.md 파일이 있음



SECTION 3-3 CLAUDE.md로 프로그램 확장하기(16)

- 클로드 코드가 만든 구조

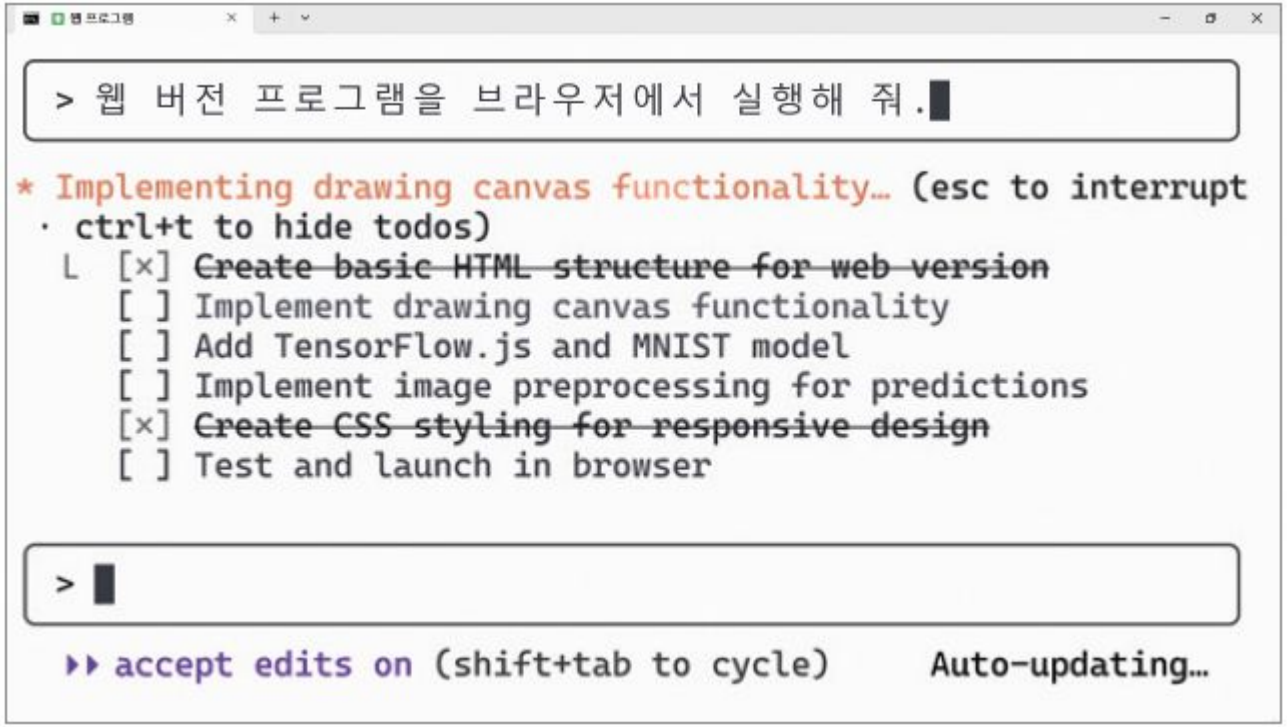
SECTION 3-3 CLAUDE.md로 프로그램 확장하기(17)

웹 버전 손글씨 인식 프로그램 완성하기

01 클로드 코드에 다음과 같이 입력

프롬프트

웹 버전 프로그램을 브라우저에서 실행해 줘.

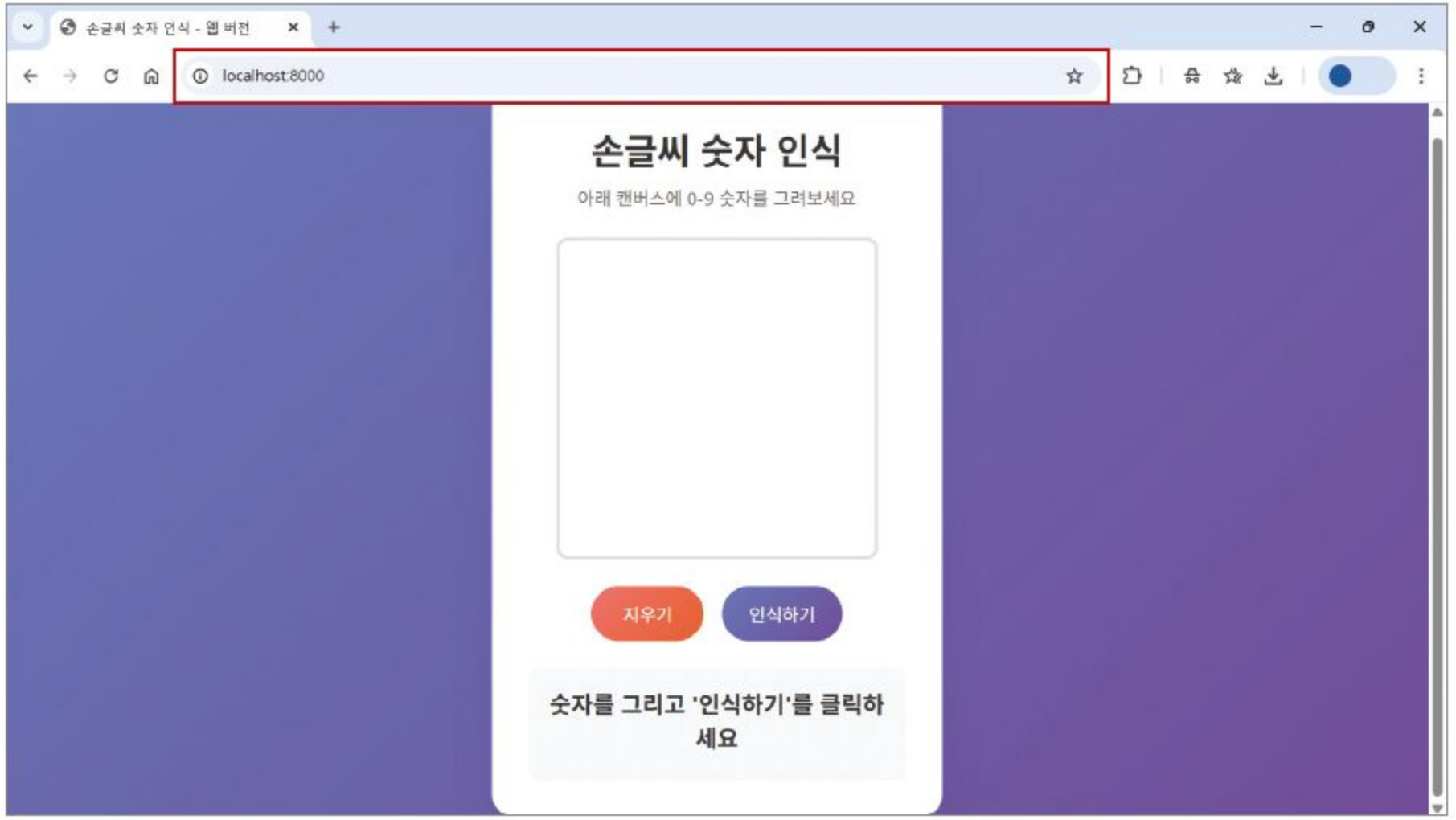


[note]

- **클로드 코드가 프로그램을 생성하면서 필요한 라이브러리 등을 설치하기 위해 동의를 구하면 '1. Yes'를 선택해 계속 진행**

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(18)

02 웹 서버가 시작되면 'http://localhost:포트번호' 형태의 로컬 서버 주소가 나타남
웹 브라우저 주소창에 입력하면, 손글씨 인식 프로그램이 브라우저에서 실행



SECTION 3-3 CLAUDE.md로 프로그램 확장하기(19)

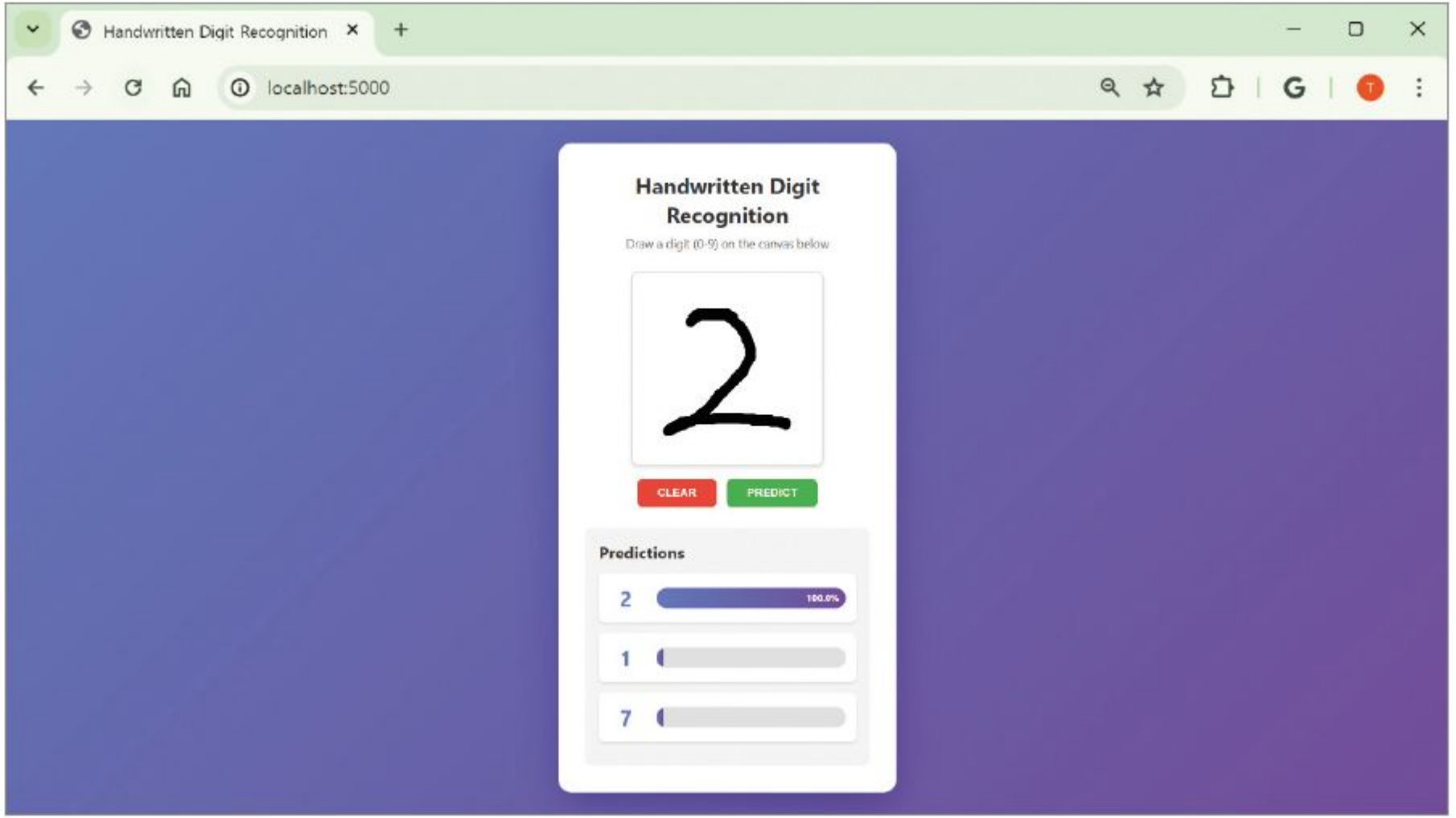
✚ 여기서 잠깐 **로컬호스트란?**

- 로컬호스트 (localhost) - 네트워크상에서 사용자의 컴퓨터를 가리키는 특별한 이름
- 컴퓨터에는 여러 프로그램이 동시에 통신할 수 있도록 여러 접속 통로가 있으며, 각 통로에는 번호가 붙어 있음
  - 이 번호를 포트(port)라고 함
- 'http://localhost'는 인터넷상의 다른 컴퓨터가 아니라 바로 내 컴퓨터 안에서 실행 중인 프로그램에 접속한다는 의미
  - 예) 'http://localhost:8080'에 접속하라고 할 때 'localhost'는 내 컴퓨터를, ':8080'은 그중에서도 8080번 포트를 사용하고 있다는 뜻

- 로컬호스트를 이용하면 외부 네트워크에 연결하지 않아도 웹 애플리케이션을 실행하고 결과를 미리 볼 수 있음
  - 개발자는 이를 통해 프로그램을 시험하거나 수정 내용을 즉시 확인하며 개발

SECTION 3-3 CLAUDE.md로 프로그램 확장하기(20)

03 마우스로 숫자를 그려 테스트하면 프로그램이 손글씨를 잘 인식하는 것을 확인



[마무리]

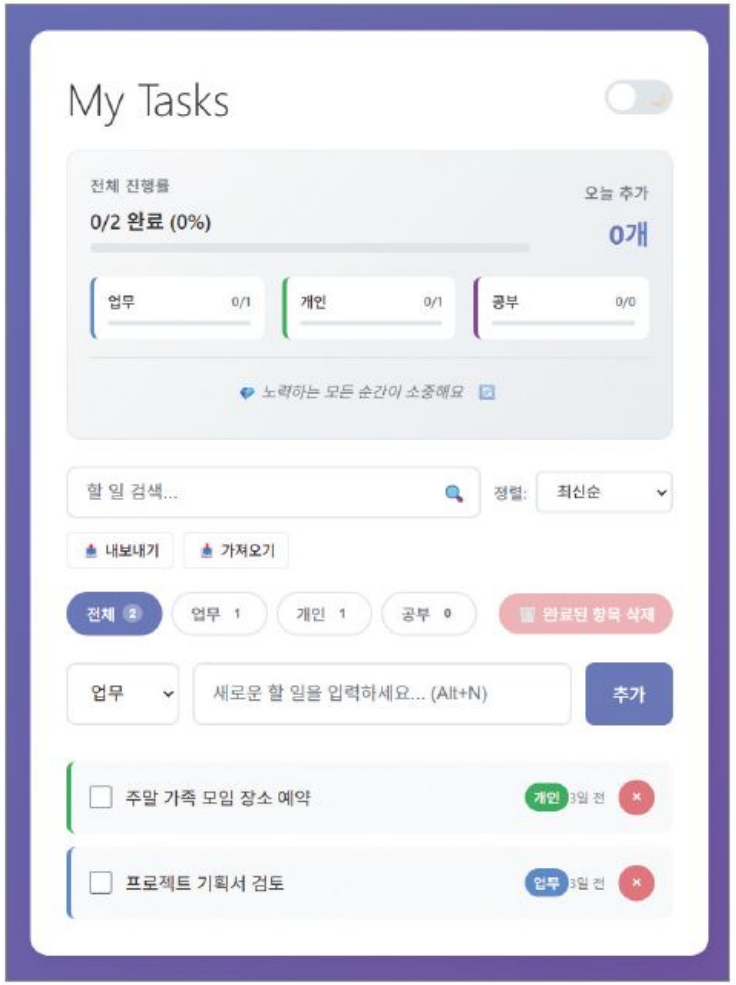
◦ 4가지 키워드로 정리하는 핵심 포인트

- **CLAUDE.md 파일**
  - 프로젝트의 구조, 규칙, 명령어 등을 기록해 클로드 코드가 실행 시 자동으로 읽는 설정 파일
- **/init -** 현재 폴더를 분석해 CLAUDE.md 파일을 자동 생성하는 초기화 명령어
- **#** 키 - 대화 중 입력하면 현재 작업의 맥락이나 규칙을 CLAUDE.md 파일에 빠르게 추가하는 지시어
- **계층적 구조 -** 루트에 공통 규칙을 두고, 하위 폴더마다 특화 규칙을 나눠 적용하는 관리 방식

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(1)

시작하기전에

- 앱 제작 실습
  - 개인용 할 일 관리 애플리케이션 제작
    - 매일 해야 할 일을 간단히 기록하고 완료 여부를 체크
    - 업무/개인/공부와 같이 카테고리별로 항목을 분류해 관리
    - 앱은 웹 브라우저에서 바로 실행되며, 복잡한 프레임워크 대신 순수 HTML, CSS, 자바스크립트만으로 진행해 초보자도 바로 따라 할 수 있음



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(2)

◦ 할 일 관리 앱을 만들면서 클로드 코드의 여러 명령어를 실습

- 프로젝트의 워크플로를 기획과 구현 두 단계로 나누고, 각각 클로드와 클로드 코드에서 진행

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(3)

◦ 단계별 프롬프트 작성하기

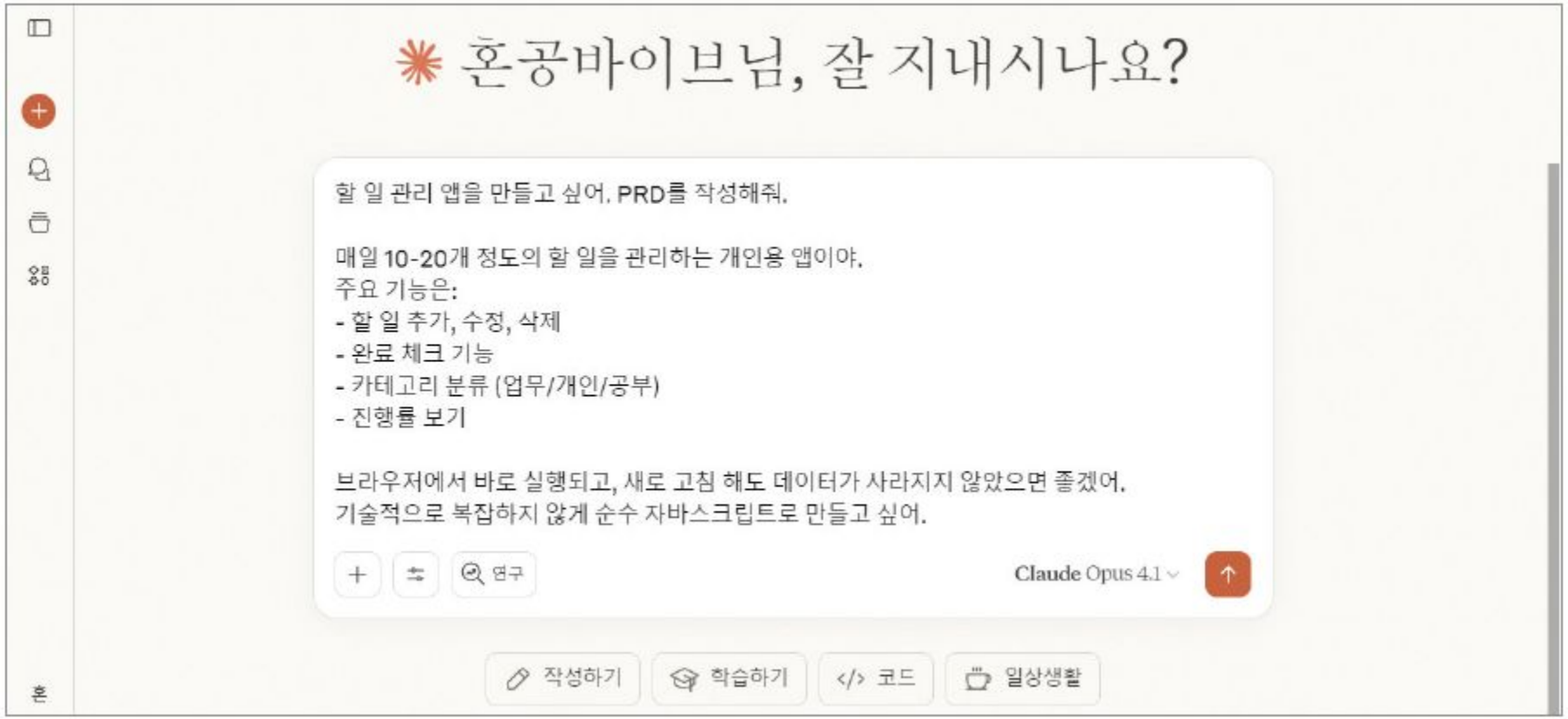
PRD 작성하기

01 웹 브라우저에서 클로드(claude.ai)에 접속하여 새로운 대화를 시작하고 할 일 관리 앱의 PRD를 작성하기 위해 다음과 같이 프롬프트를 입력

프롬프트

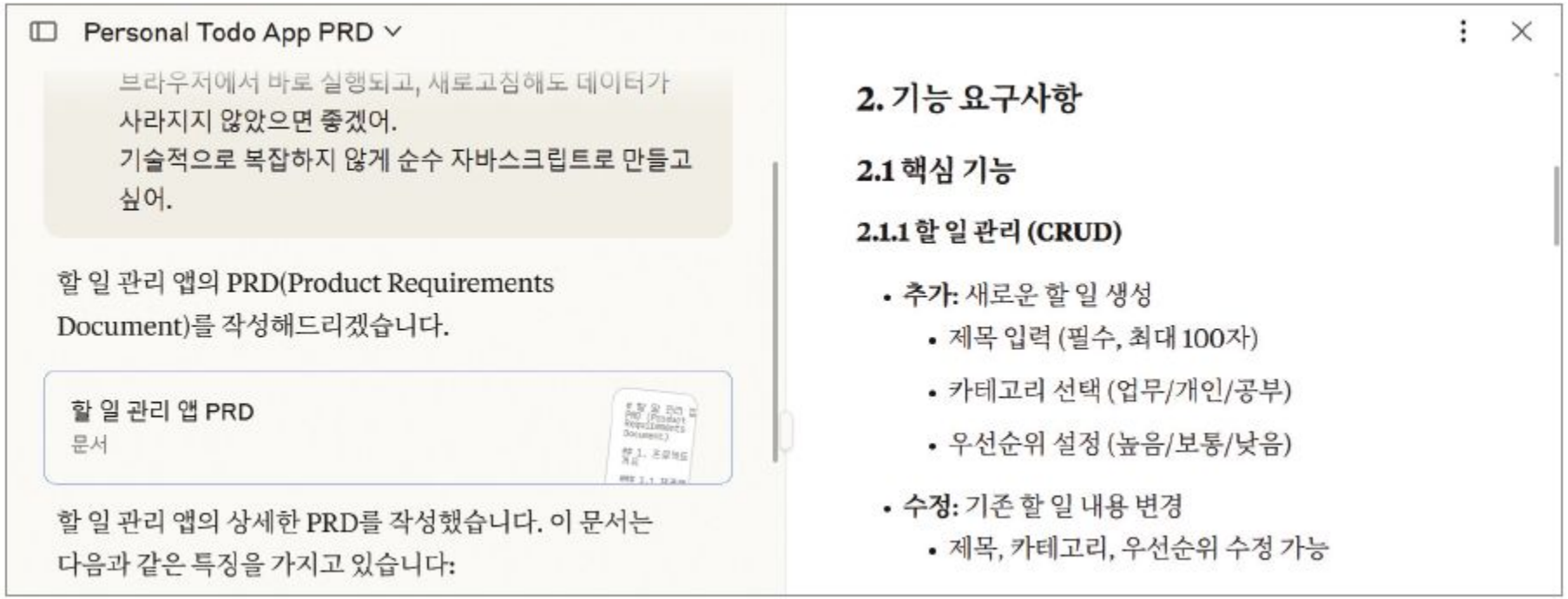
```
할 일 관리 앱을 만들고 싶어. PRD를 작성해줘.

매일 10-20개 정도의 할 일을 관리하는 개인용 앱이야.
주요 기능은:
- 할 일 추가, 수정, 삭제
- 완료 체크 기능
- 카테고리 분류 (업무/개인/공부)
- 진행률 보기

브라우저에서 바로 실행되고, 새로고침해도 데이터가 사라지지 않았으면 좋겠어.
기술적으로 복잡하지 않게 순수 자바스크립트로 만들고 싶어.
```

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(4)



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(5)

02 클로드가 체계적인 PRD 작성을 시작 - PRD의 구조와 핵심 요구사항을 확인



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(6)

프롬프트로 변환하기

- 생성한 PRD를 바탕으로 단계별 프롬프트 전략을 구축

프롬프트

```
이 PRD를 클로드 코드에서 사용할 단계별 프롬프트로 변환해 줘.
5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을 만들어 줘.
```

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(7)

✚ 여기서 잠깐 **이번 장에서 참고할 것**

- 6장에서는 클로드 코드에서 직접 PRD를 작성하고 단계별 프롬프트를 생성해 md 파일로 저장해 활용하는 방법을 학습
- 이번 절에서는 본격적인 실습에 앞서, 익숙한 웹 인터페이스 환경에서 생성된 프롬프트를 확인하며 기본 개념과 작업 흐름을 익히는 데 중점

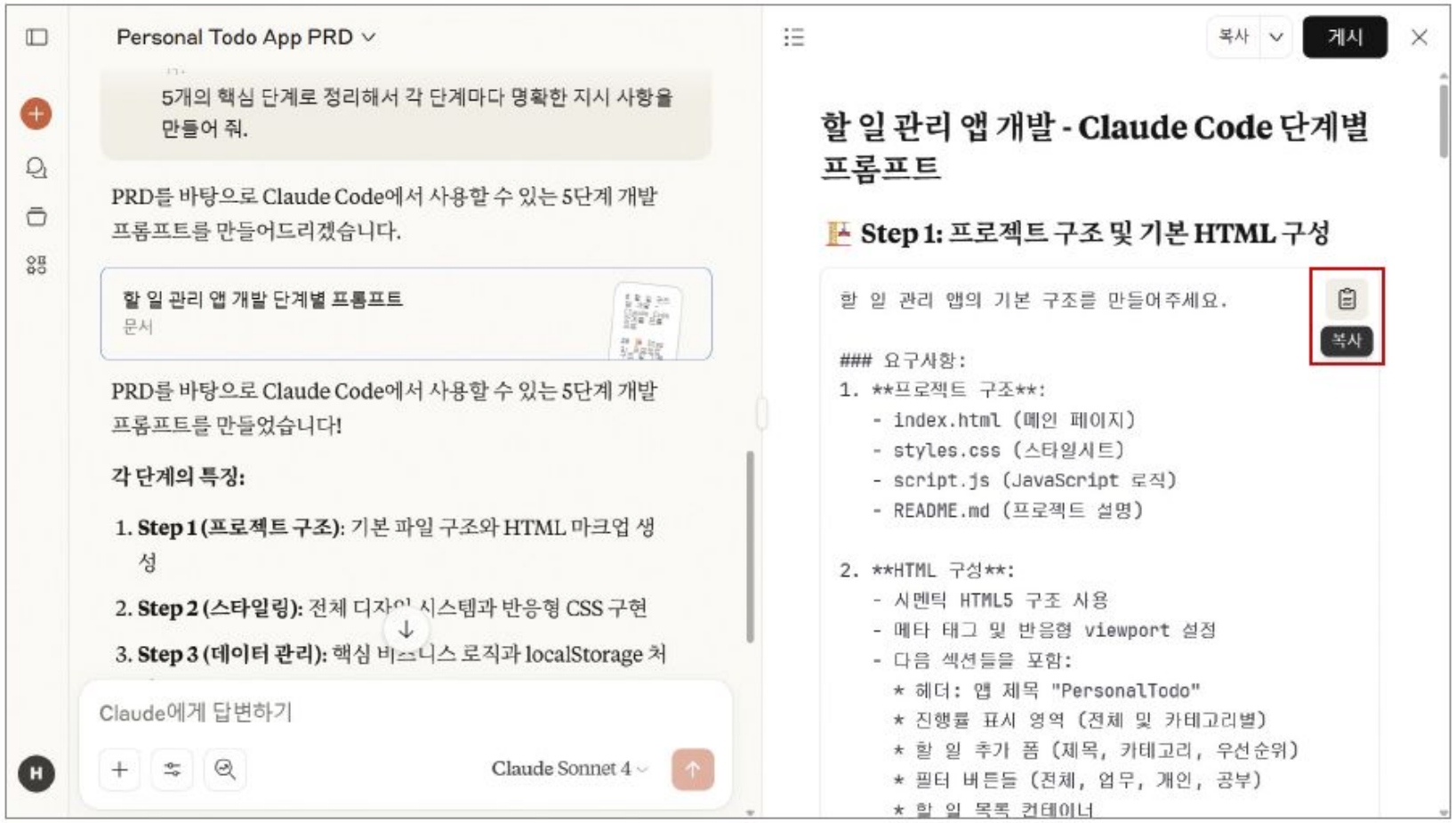
[note]

- 클로드 웹에서 만든 단계별 프롬프트를 메모장이나 다른 텍스트 편집기에 저장해두고, 클로드 코드에서 작업할 때 복사해서 사용
- 웹에서는 '기획과 설계'를, 코드에서는 '구현'을 담당하는 효율적인 워크플로를 만들 수 있음

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(8)

◦ 단계별 프롬프트 실행하기

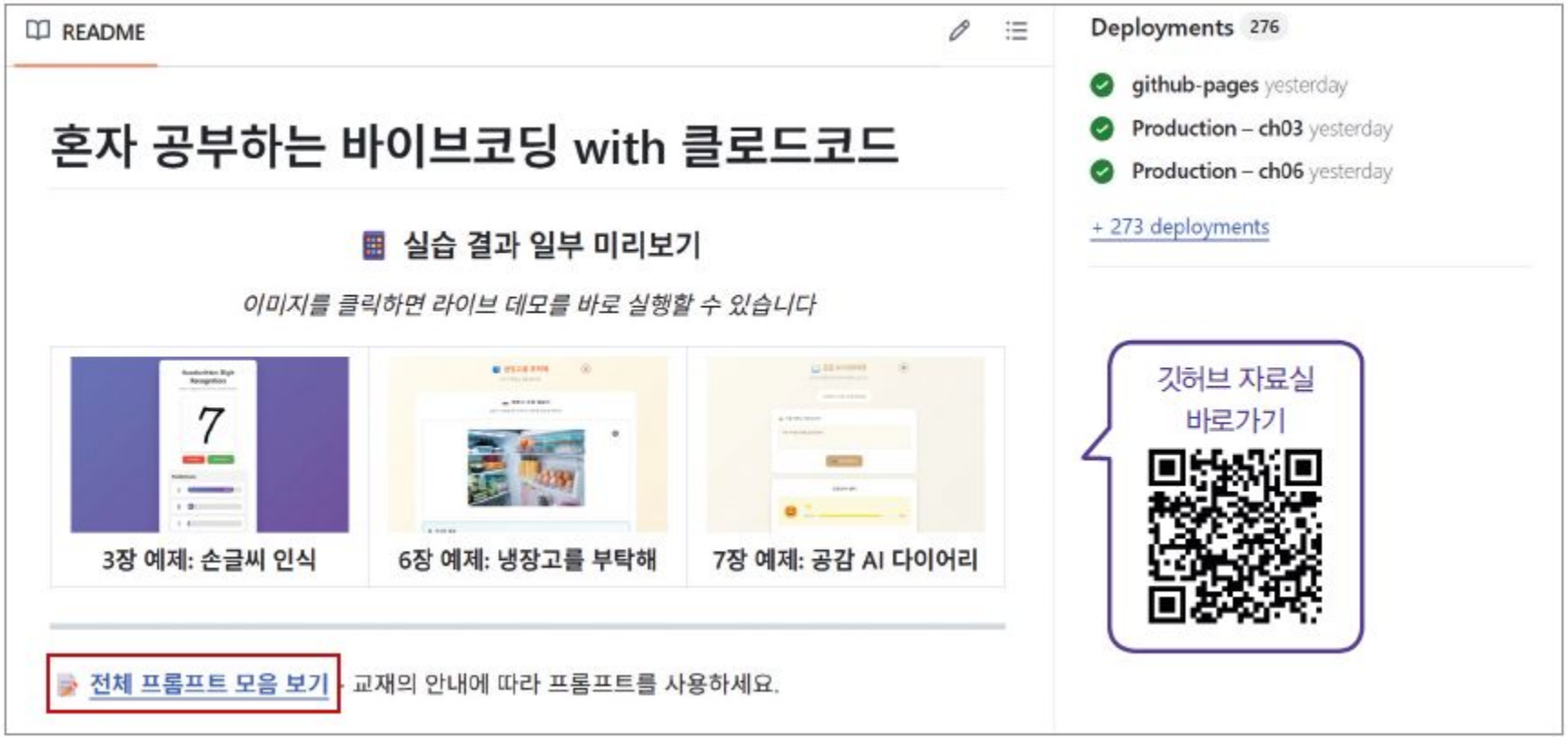
- 앞서 클로드 웹에서 준비한 프롬프트를 단계별로 실행
  - 텍스트를 복사해서 클로드 코드에 그대로 붙여 넣기



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(9)

✚ 여기서 잠깐 **프롬프트 자료실**

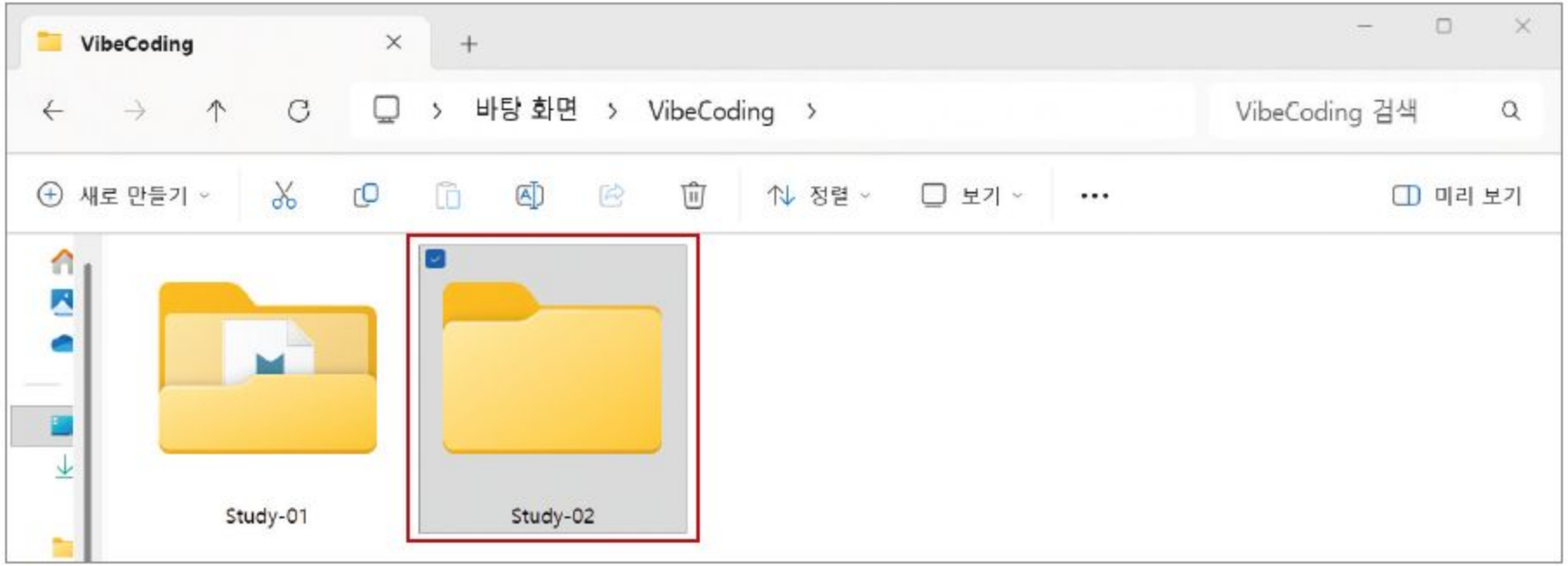
- 이 절부터는 사용자가 직접 입력하는 프롬프트와 교재에서 제시하는 예시 프롬프트가 서로 다를 수 있음
- 학습 참고 - 이 책의 깃허브 자료실 (https://github.com/taehojo/vibecoding)에서 [전체 프롬프트 모음 보기]



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(10)

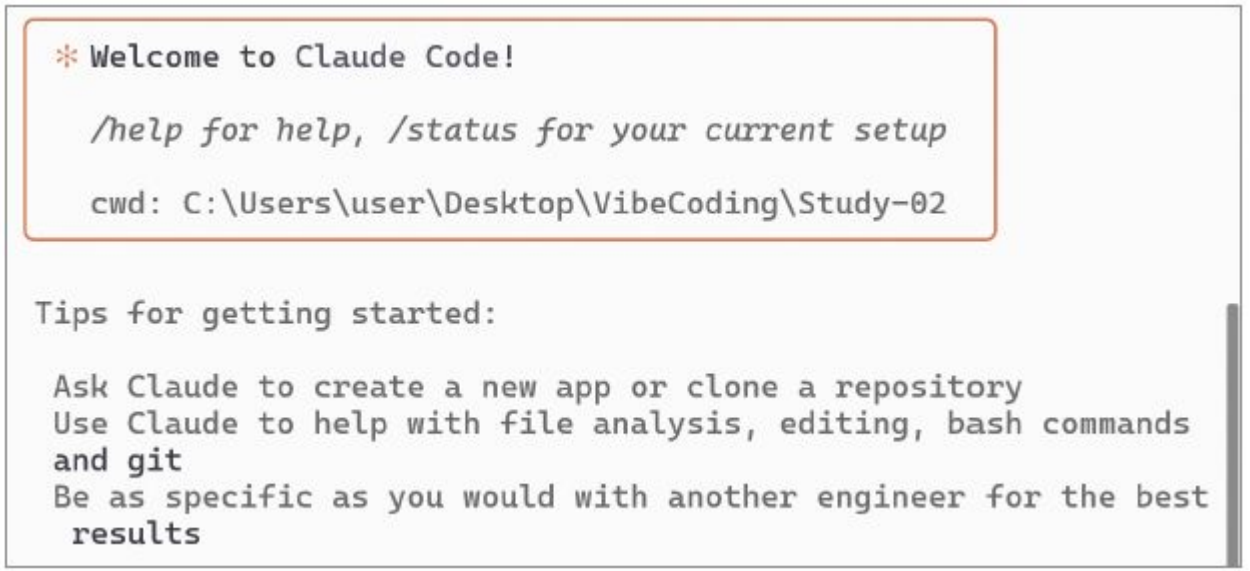
클로드 코드 실행하고 초기 설정하기

01 데스크톱의 'VibeCoding' 폴더에 새로운 폴더 'Study-02'를 생성한 뒤, 더블 클릭해 폴더 열기



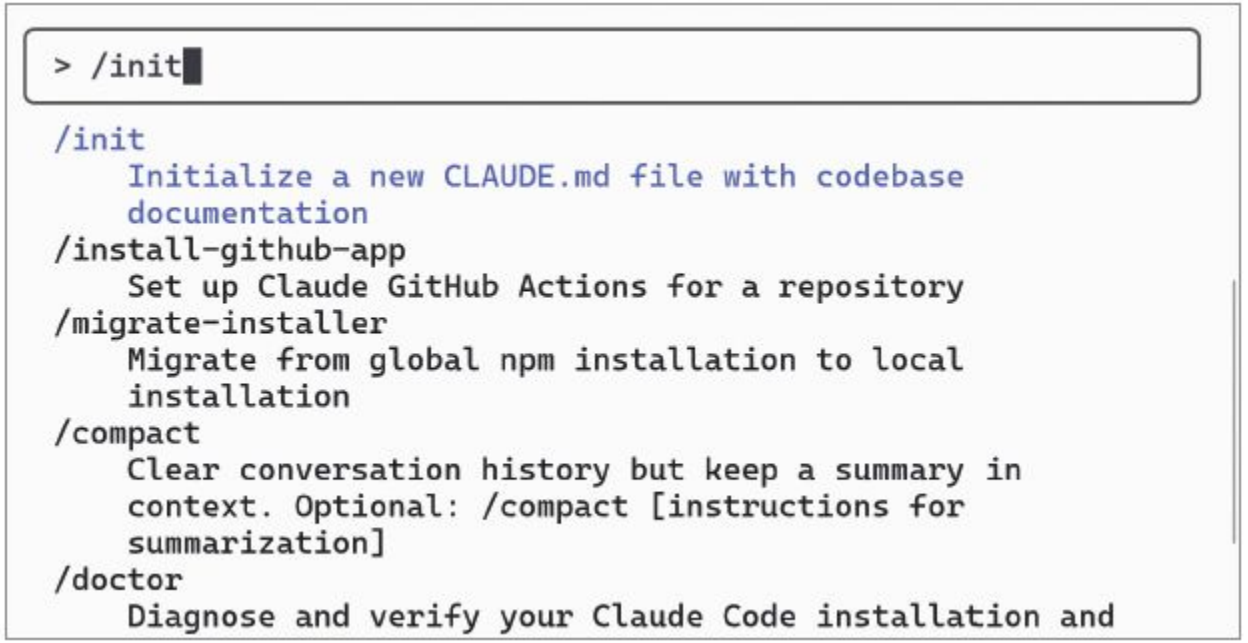
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(11)

02 'Study-02' 폴더의 빈 공간에 마우스 오른쪽 버튼을 누른 뒤 메뉴에서 [터미널에서 실행]을 클릭해 명령 프롬프트를 실행하고 'claude'를 입력해서 클로드 코드를 실행



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(12)

03 클로드 코드가 실행되면 '/init' 명령어를 입력하여 프로젝트 초기 설정을 시작

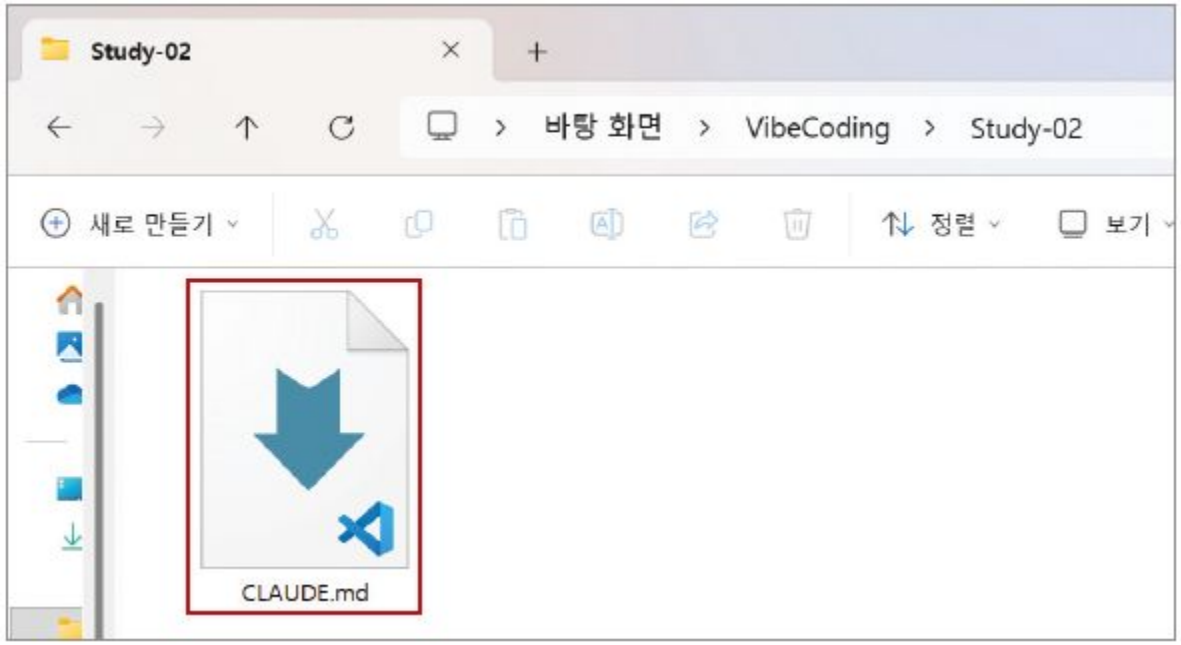


[note]

- **/init 실행 중 Git 저장소 확인, 폴더 읽기, 파일 생성 등 여러 권한 요청 메시지가 나타날 수 있음**
- **'1. Yes'를 선택해 계속 진행**

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(13)

04 현재 폴더를 분석하여 CLAUDE.md 파일을 자동으로 생성
아직 아무 파일도 없는 빈 폴더 상태이므로 CLAUDE.md 파일은 기본적인 프로젝트 설정만 담음



[note]

- **/init**이 상위 폴더를 분석하여 관련 없는 내용이 포함되어도 걱정할 필요 없음
- 프로젝트가 진행되면서 실제 할 일 관리 앱 관련 내용이 **CLAUDE.md** 파일에 추가되고, 초기의 잘못된 추측은 자연스럽게 무시되거나 새 내용으로 대체

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(14)

05 앞서 클로드 웹에서 작성한 단계별 프롬프트를 준비
클로드가 제시한 다섯 단계를 따라 할 일 관리 앱을 제작

- Step 1: 기본 구조 및 핵심 기능 구현
- Step 2: 카테고리 기능 및 UI 개선
- Step 3: 진행률 대시보드 추가
- Step 4: 다크 모드 및 고급 기능
- Step 5: 최종 완성 및 최적화

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(15)

1단계: 기본 구조 및 핵심 기능 구현

01 단계별 프롬프트의 첫 번째 프롬프트(Step 1)를 복사해 클로드 코드가 실행 중인 터미널에 [Ctrl]+[V] 키를 눌러 붙여 넣기

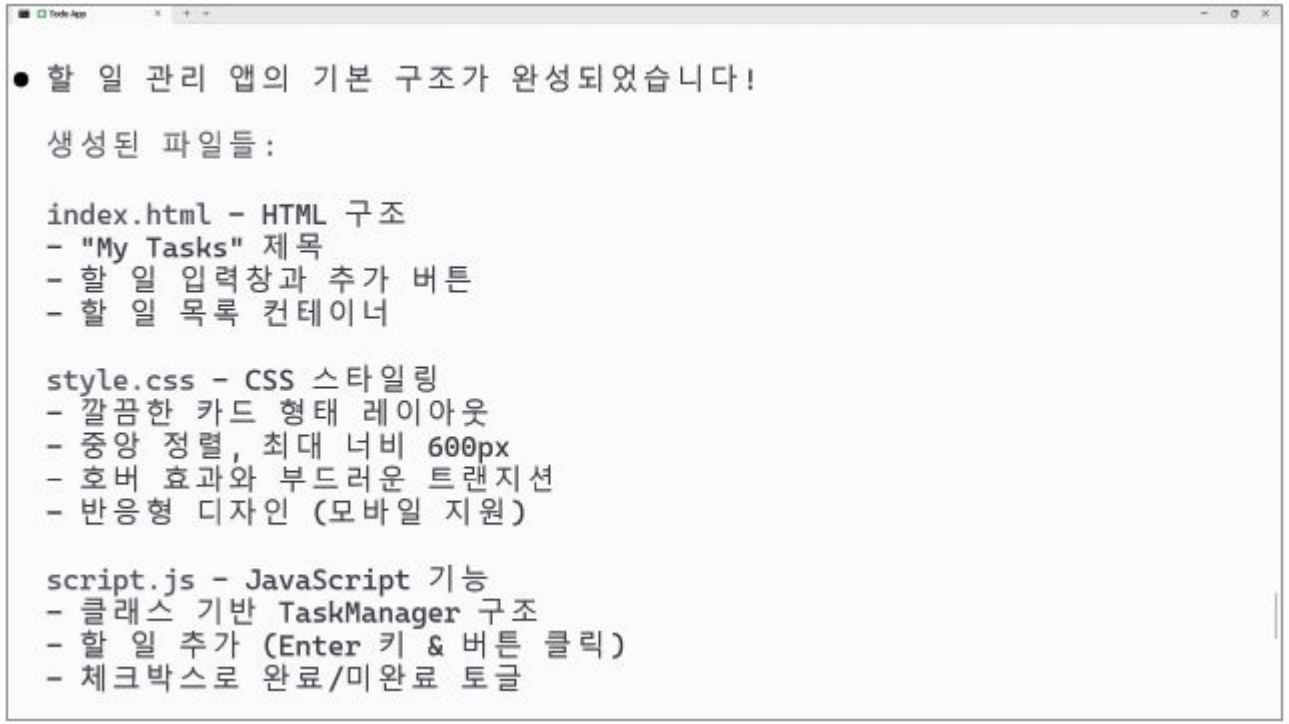
```
> Step 1: 기본 구조 및 핵심 기능 구현
  할 일 관리 앱의 기본 구조를 만들어줘.

  요구사항:
  1. index.html, style.css, script.js 세 개의 파일로 구성
  2. HTML 구조:
     - 앱 제목 "My Tasks"
     - 할 일 입력창 (input + 추가 버튼)
     - 할 일 목록을 표시할 컨테이너
  3. JavaScript 기능:
     - 할 일 추가 (Enter 키와 버튼 클릭 모두 지원)
     - 할 일 삭제 (각 항목에 X 버튼)
     - 체크박스로 완료/미완료 토글
     - 완료 시 취소선 스타일 적용
  4. localStorage에 데이터 저장:
     - 추가/삭제/완료 상태 변경 시 자동 저장
     - 페이지 새로고침 시 데이터 유지
  5. 기본 CSS 스타일링:
     - 깔끔한 카드 형태의 레이아웃
```

[note]

- **터미널에서 여러 행의 프롬프트를 직접 입력할 때는 [\]+[Enter] 또는 [Ctrl]+[J] 키를 눌러 줄바꿈**

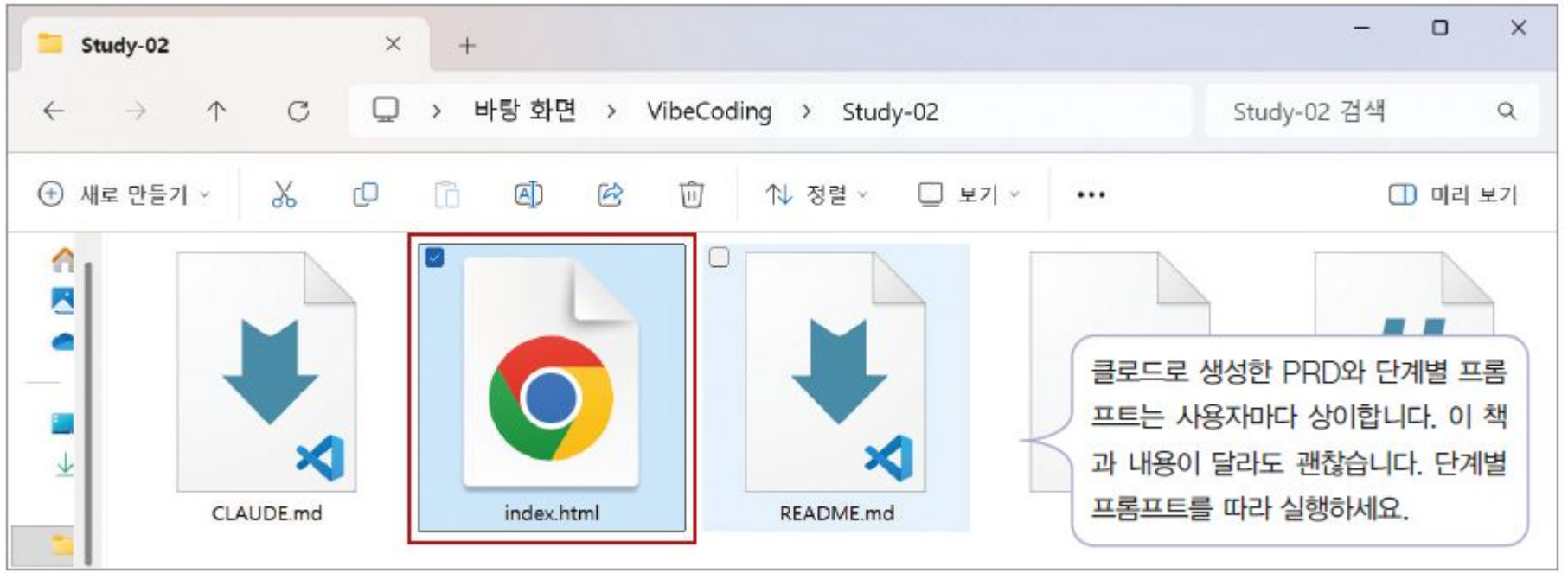
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(16)

02 진행되는 내용을 확인하며 기다리면 곧 기본 구조가 구현되었다는 메시지가 나타남



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(17)

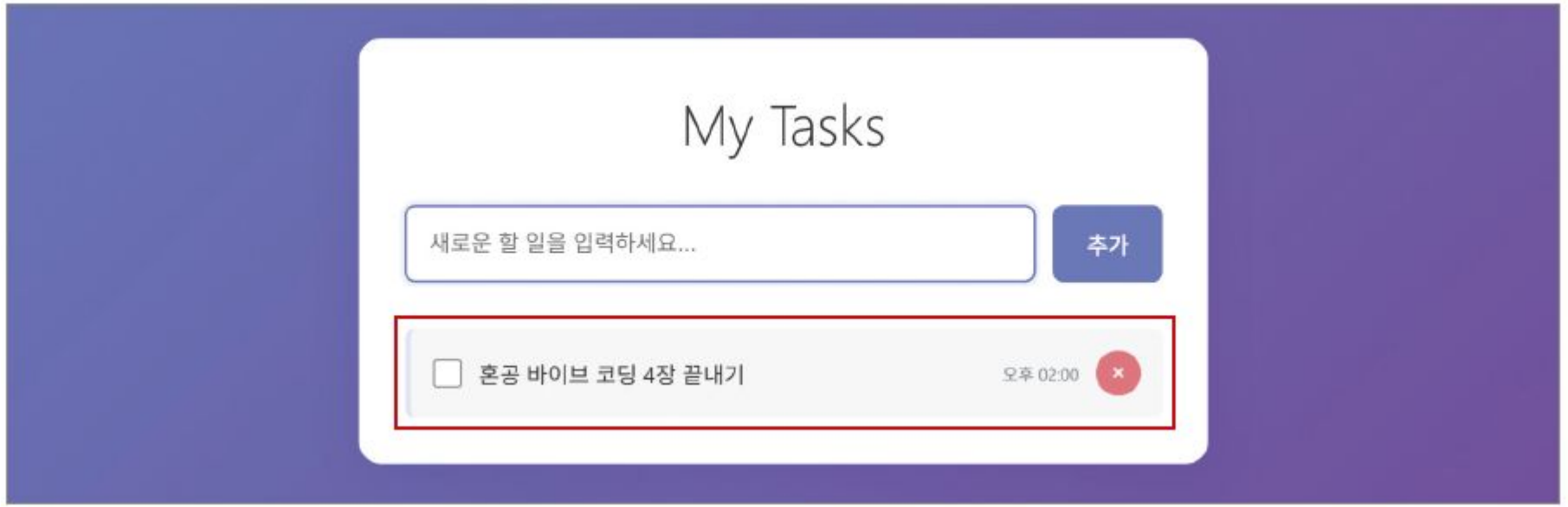
03 데스크톱의 파일 탐색기에 생성된 'index.html' 파일을 클릭해 실행



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(18)

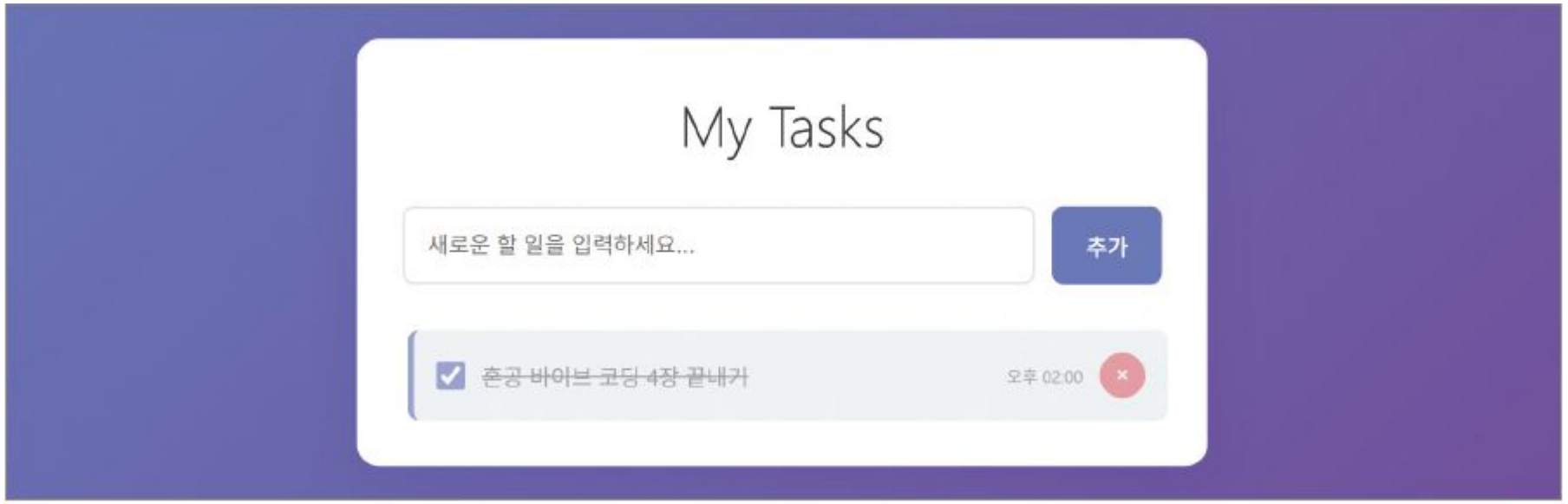
04 기본 브라우저가 열리며, 할 일 관리 앱이 열림

- 기능을 테스트하기 위해 [새로운 할 일을 입력하세요…] 입력창에 '혼공 바이브 코딩 4장 끝내기'라고 입력
- [추가] 버튼을 눌러 할 일을 추가해보고 확인



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(19)

05 체크 박스를 클릭해 완료된 할 일로 전환하거나, [X] 버튼을 클릭해 삭제하는 등 다른 기능도 테스트



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(20)

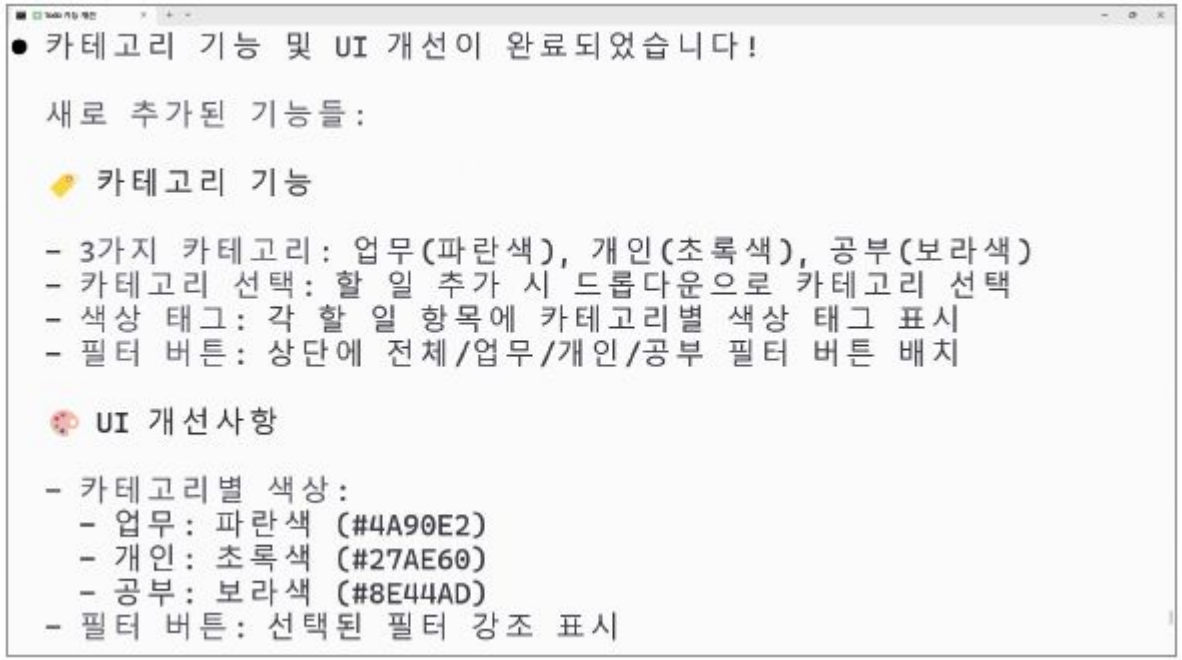
2단계: 카테고리 기능 및 UI 개선

01 단계별 프롬프트의 두 번째 프롬프트(Step 2)를 복사해 클로드 코드에 입력

```
> Step 2: 카테고리 기능 및 UI 개선
  기존 코드에 카테고리 기능을 추가하고 UI를 개선해줘.

  요구사항:
  1. 카테고리 기능:
     - 3가지 카테고리: 업무(work), 개인(personal), 공부(study)
     - 할 일 추가 시 카테고리 선택 드롭다운
     - 각 할 일 항목에 카테고리 색상 태그 표시
     - 카테고리 필터 버튼 (전체/업무/개인/공부)
  2. UI 개선:
     - 카테고리별 색상: 업무(파란색 #4A90E2), 개인(초록색
  #27AE60), 공부(보라색 #8E44AD)
     - 필터 버튼을 상단에 배치
     - 선택된 필터 버튼 강조 표시
     - 할 일 항목에 생성 시간 표시 (예: "2시간 전")
  3. 데이터 구조 업데이트:
     - category 필드 추가
```

SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(21)

02 완료되었다는 메시지가 나타나면 진행 상황을 확인

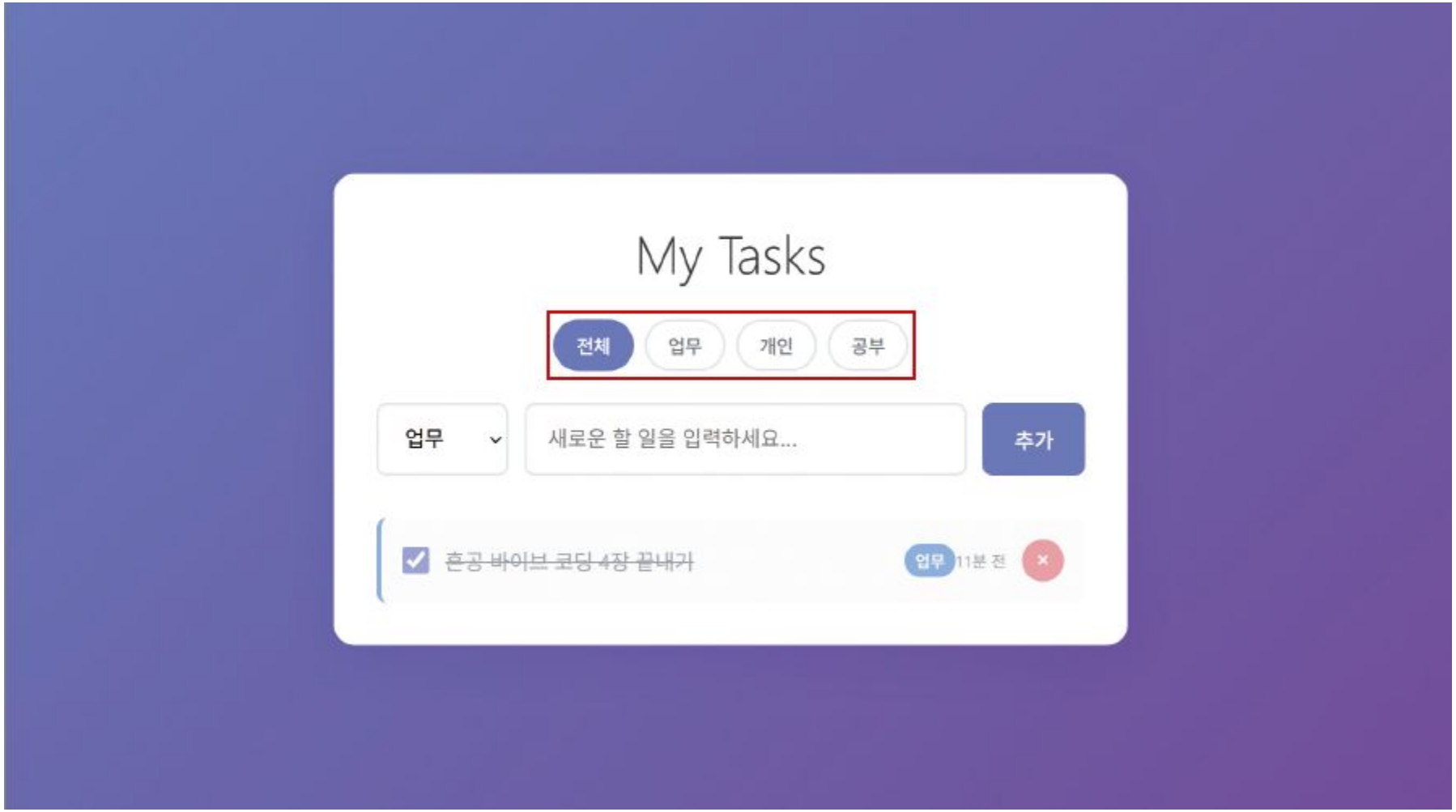


[note]

- **중간에 권한을 묻는 메시지가 나타나면 '1. Yes'를 선택**
- **매번 누르기 번거롭다면 '2. Yes, and don't ask again ...'을 선택해도 좋음**

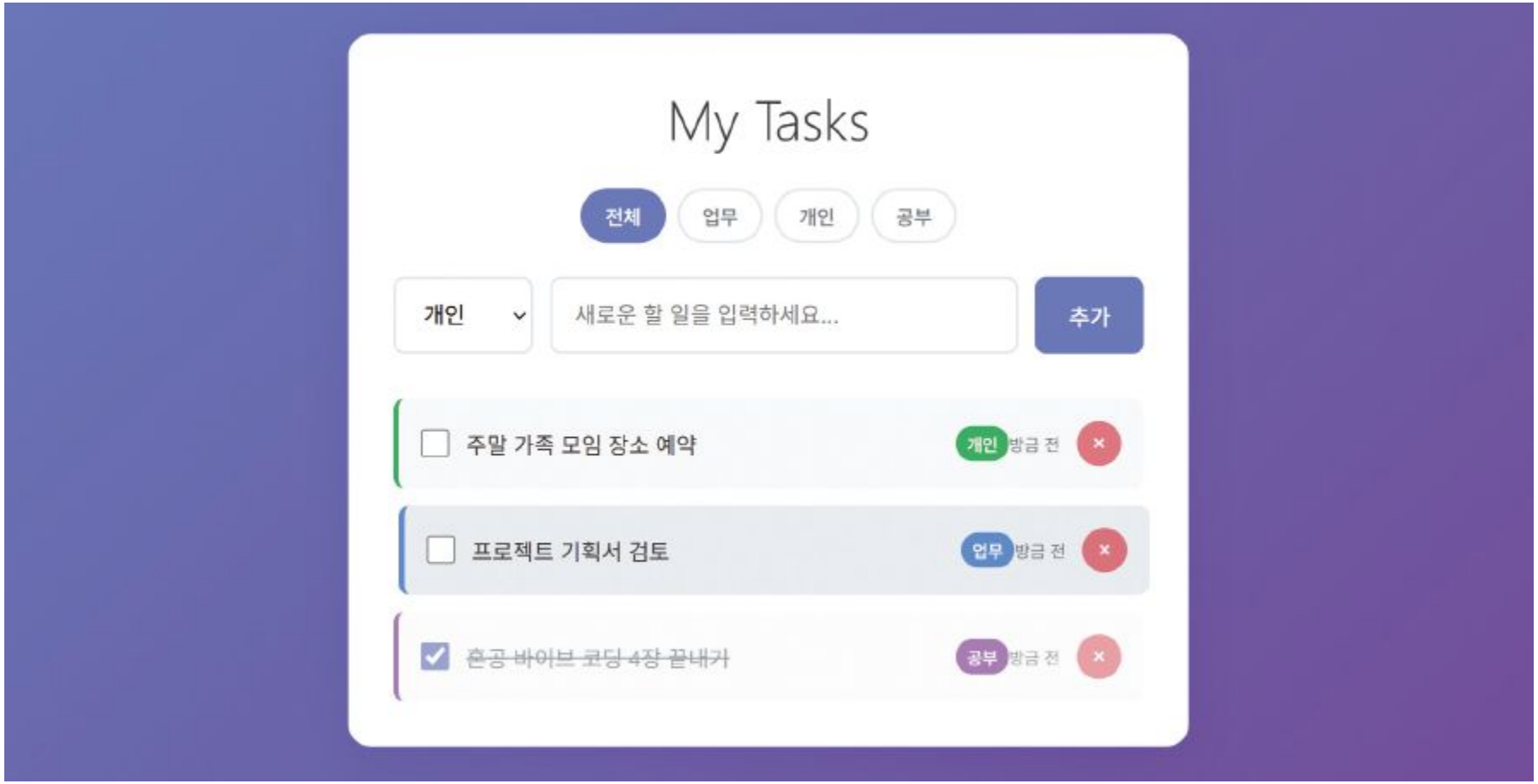
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(22)

03 'index.html' 파일을 새로 열거나 이미 열려 있는 브라우저를 새로고침
업무, 개인, 공부 카테고리가 생겼고, 전체 카테고리를 한눈에 볼 수 있게 UI가 개선



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(23)

04 카테고리별로 입력해 보면 잘 분류되어 기록되는 것을 확인



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(24)

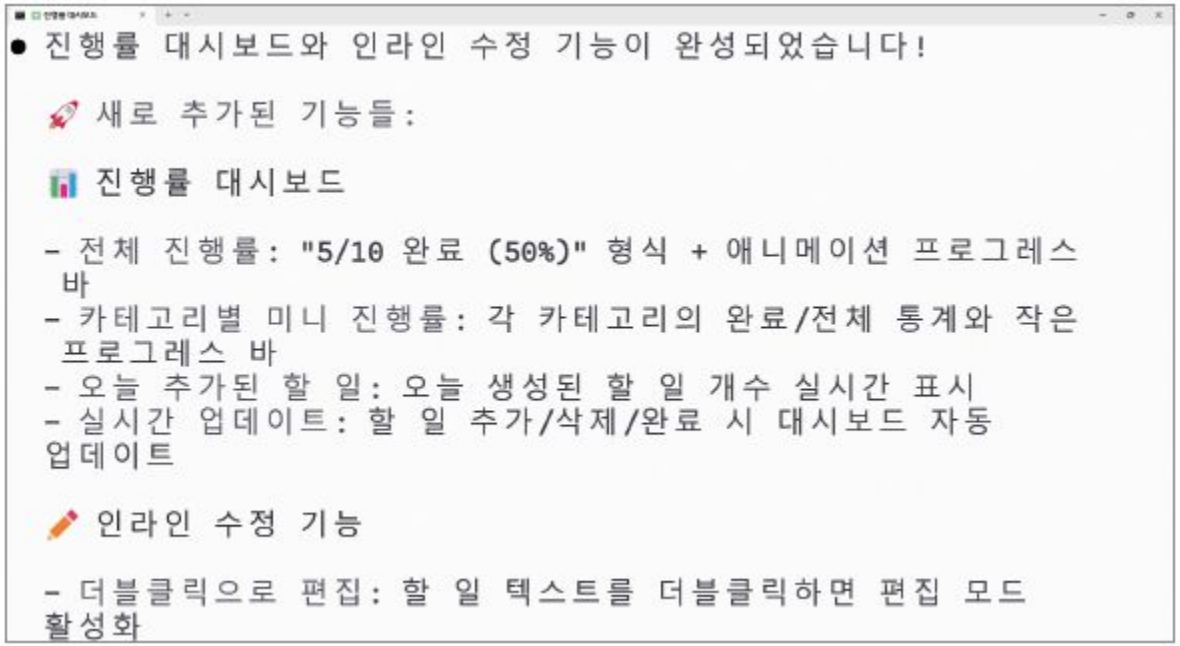
3단계: 진행률 대시보드 추가

01 단계별 프롬프트의 세 번째 프롬프트(Step 3)를 복사해 클로드 코드에 입력

```
> Step 3: 진행률 대시보드 추가
  진행률 대시보드를 추가하고 인라인 수정 기능을 구현해줘.

  요구사항:
  1. 진행률 대시보드:
     - 앱 상단에 통계 섹션 추가
     - 전체 진행률: "5/10 완료 (50%)" 형식 + 프로그레스 바
     - 카테고리별 미니 진행률 표시 (각 카테고리 완료/전체)
     - 오늘 추가된 할 일 개수 표시
  2. 인라인 수정 기능:
     - 할 일 텍스트 더블클릭 시 편집 모드
     - input 필드로 변경되어 수정 가능
     - Enter로 저장, ESC로 취소
     - 편집 중 카테고리도 변경 가능한 select 박스
  3. UI 애니메이션:
     - 진행률 바 부드러운 전환 효과
     - 항목 추가/삭제 시 페이드 애니메이션
```

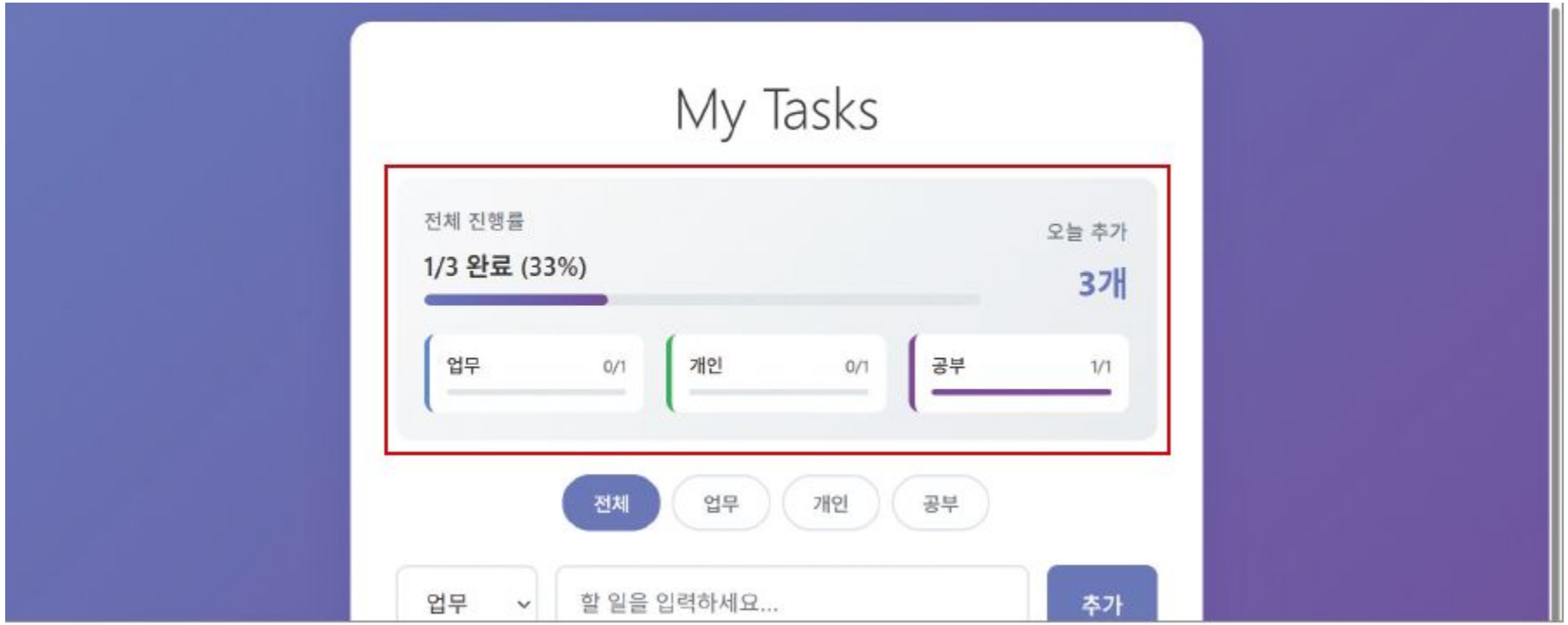
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(25)

02 완료되었다는 메시지가 보이면 진행 상황을 확인



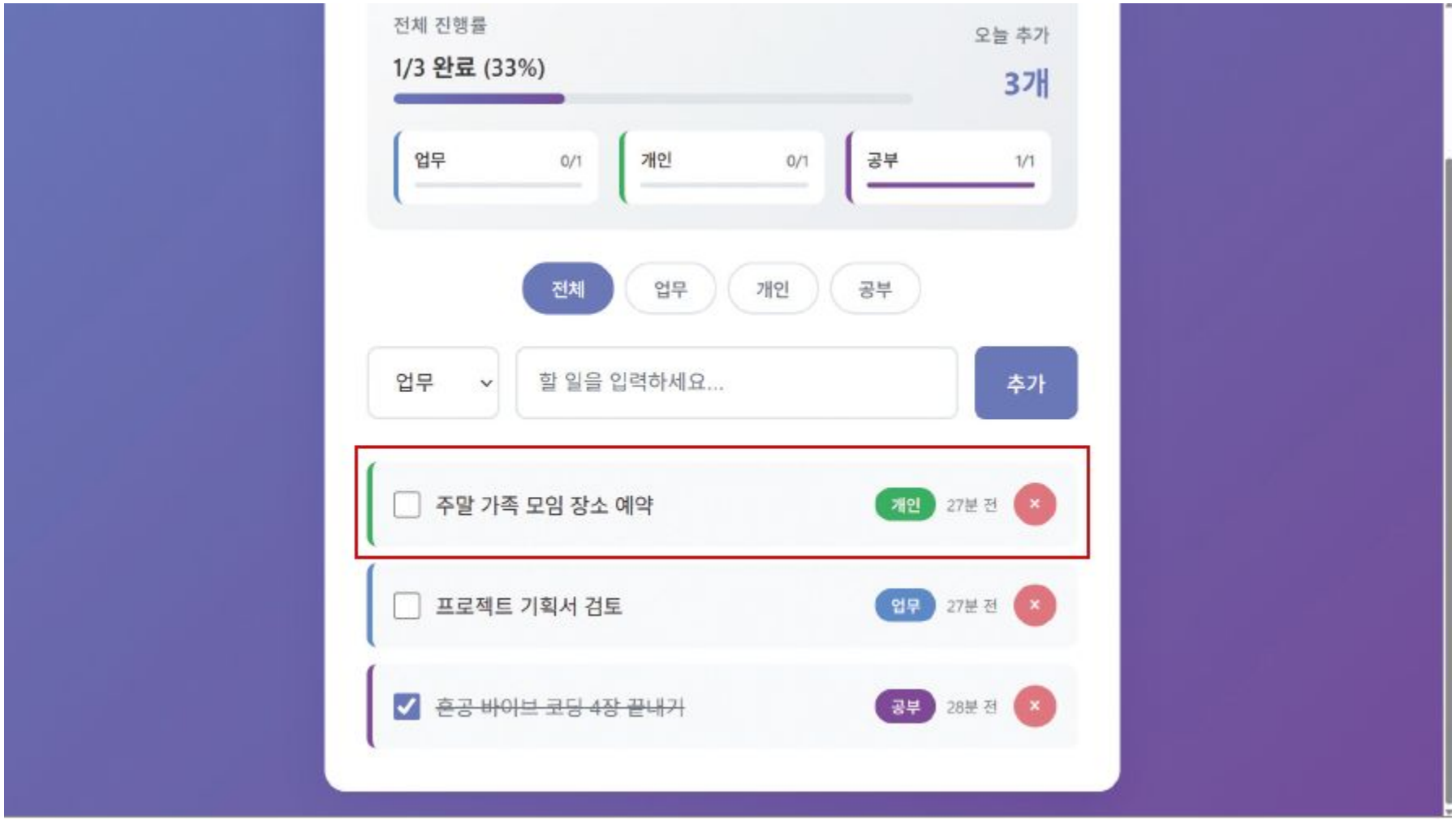
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(26)

03 할 일 관리 앱의 대시보드에 전체 진행률과 카테고리별 미니 진행률 등이 추가됨



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(27)

04 할 일 목록에 있는 항목을 더블 클릭하면 수정할 수 있음
수정 기능이 잘 작동하는지 테스트



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(28)

4단계: 다크 모드 및 고급 기능

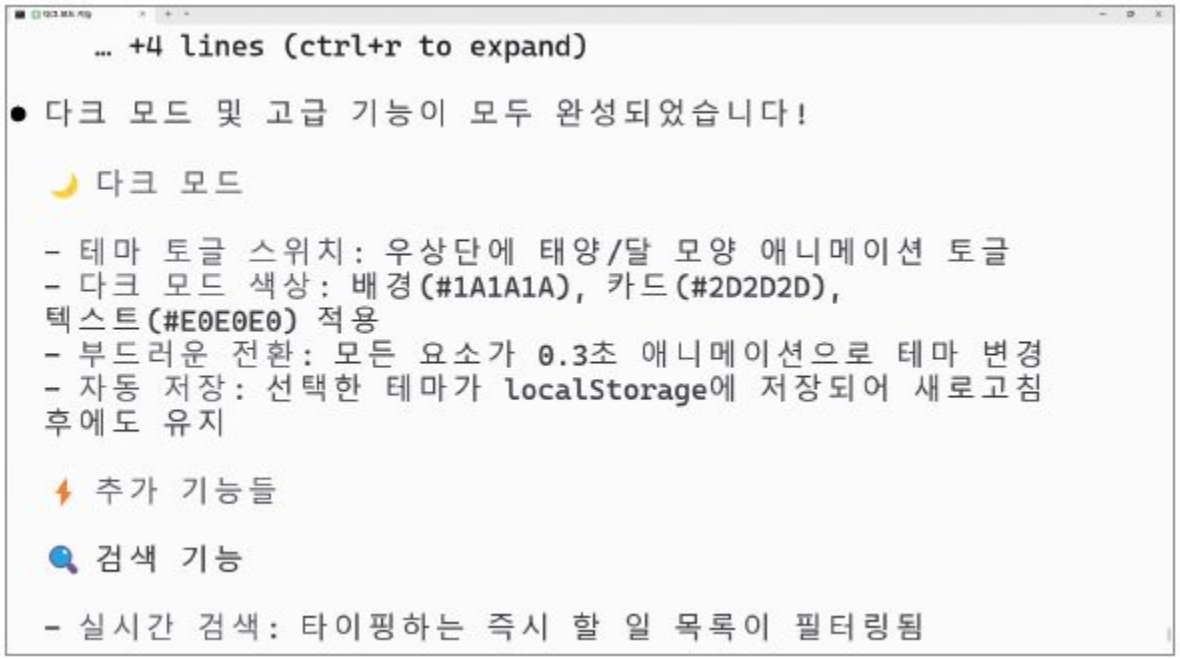
01 단계별 프롬프트의 네 번째 프롬프트(Step 4)를 복사해 클로드 코드에 입력

- 다크 모드를 구현하고 완료된 항목을 삭제하거나 단축키 사용 등의 기능도 추가

```
> Step 4: 다크 모드 및 고급 기능
  다크 모드와 추가 기능들을 구현해줘.

  요구사항:
  1. 다크 모드:
     - 우측 상단에 다크/라이트 모드 토글 스위치
     - 다크 모드 색상: 배경(#1A1A1A), 카드(#2D2D2D),
  텍스트(#E0E0E0)
     - 선택한 테마 localStorage에 저장
     - 부드러운 전환 애니메이션
  2. 추가 기능:
     - "완료된 항목 모두 삭제" 버튼 (확인 다이얼로그 포함)
     - 할 일 검색 기능 (실시간 필터링)
     - 남은 할 일 개수 배지 표시
     - 빈 상태 메시지 ("할 일이 없습니다. 추가해보세요!")
  3. 키보드 단축키:
     - Alt+N: 새 할 일 입력창 포커스
     - Alt+1,2,3,4: 카테고리 필터 전환
```

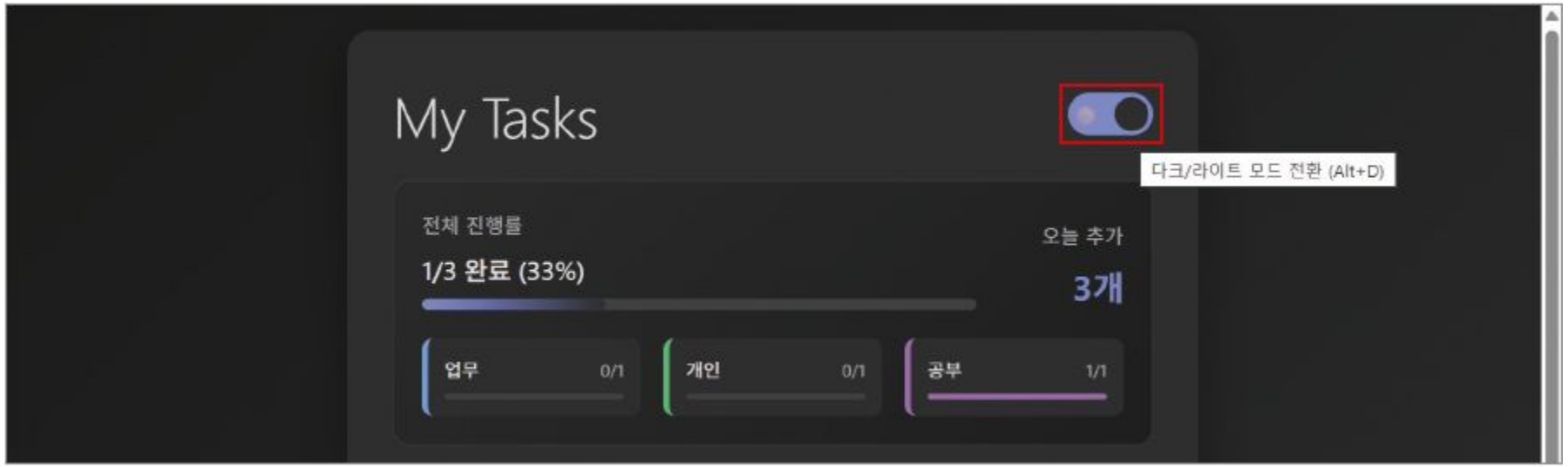
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(29)

02 완료되었다는 메시지를 확인한 뒤, 추가된 기능을 확인



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(30)

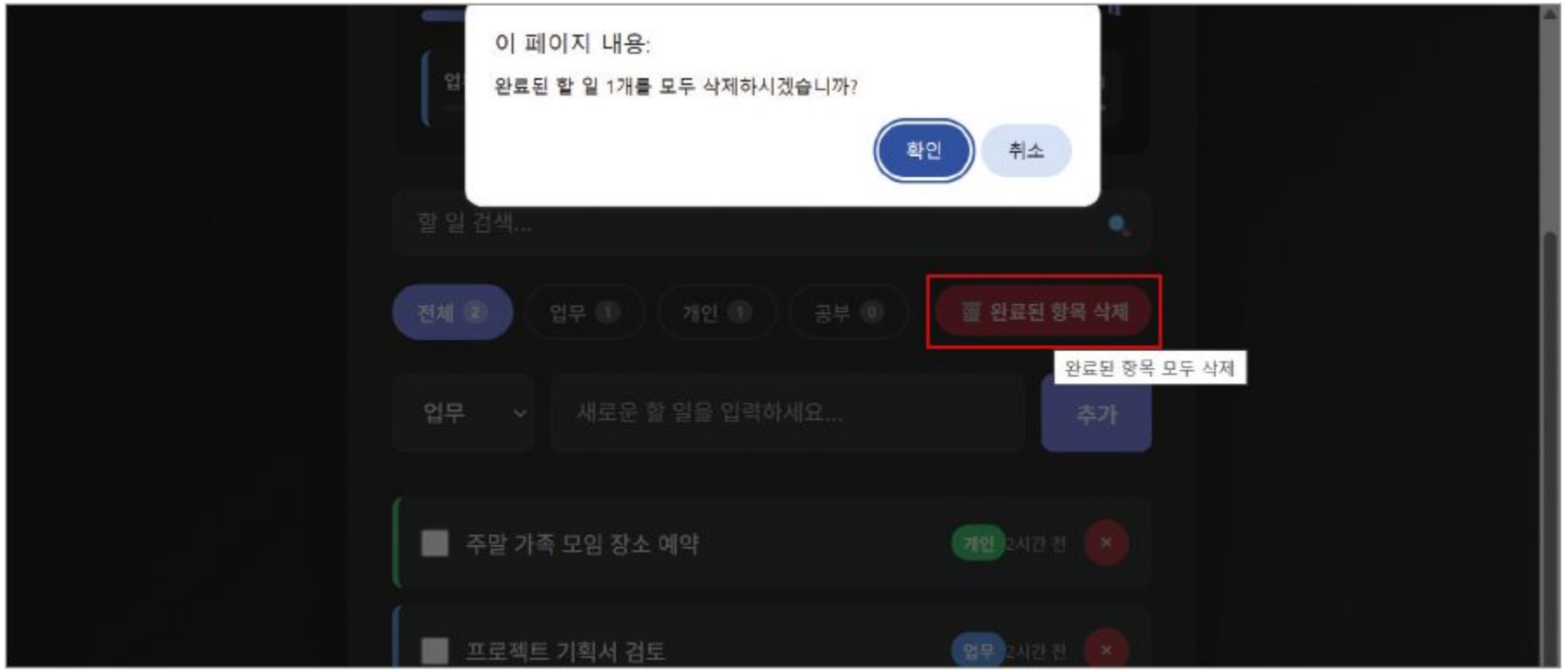
03 할 일 관리 앱에 다크 모드를 선택하는 기능이 생긴 것을 확인
다크 모드 선택 버튼을 클릭하면 다크 모드가 적용되는 것을 확인



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(31)

04 완료 항목 일괄 삭제 기능도 테스트

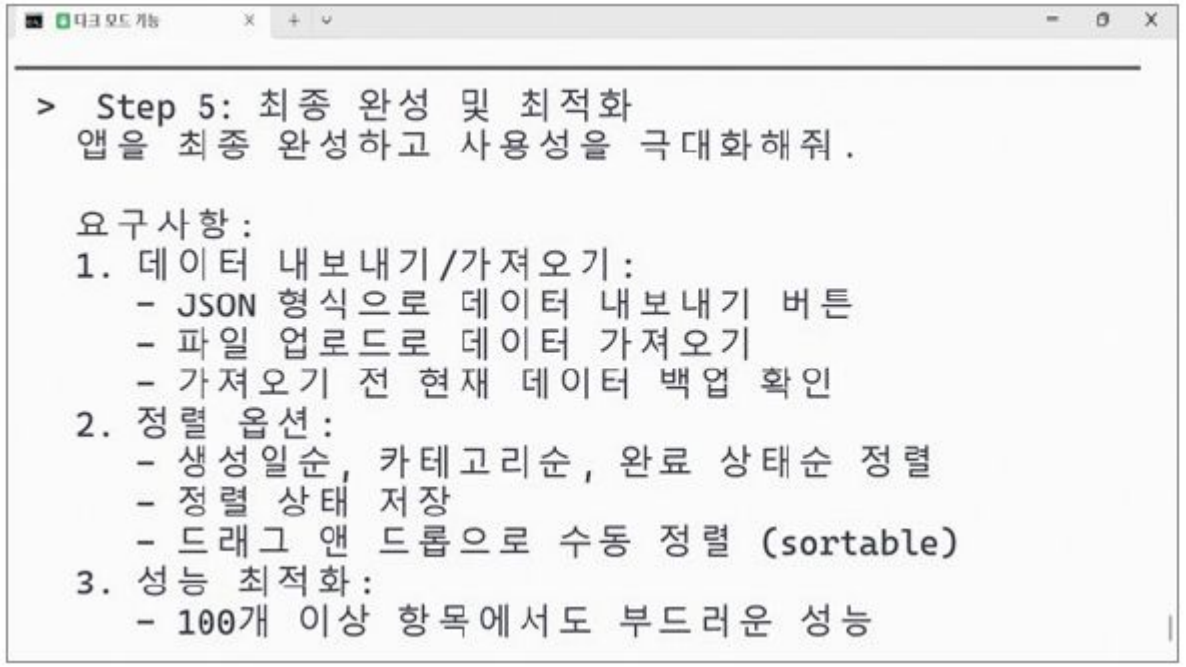
[완료된 항목 삭제] 버튼을 클릭하면 체크 표시한 할일이 모두 삭제됨



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(32)

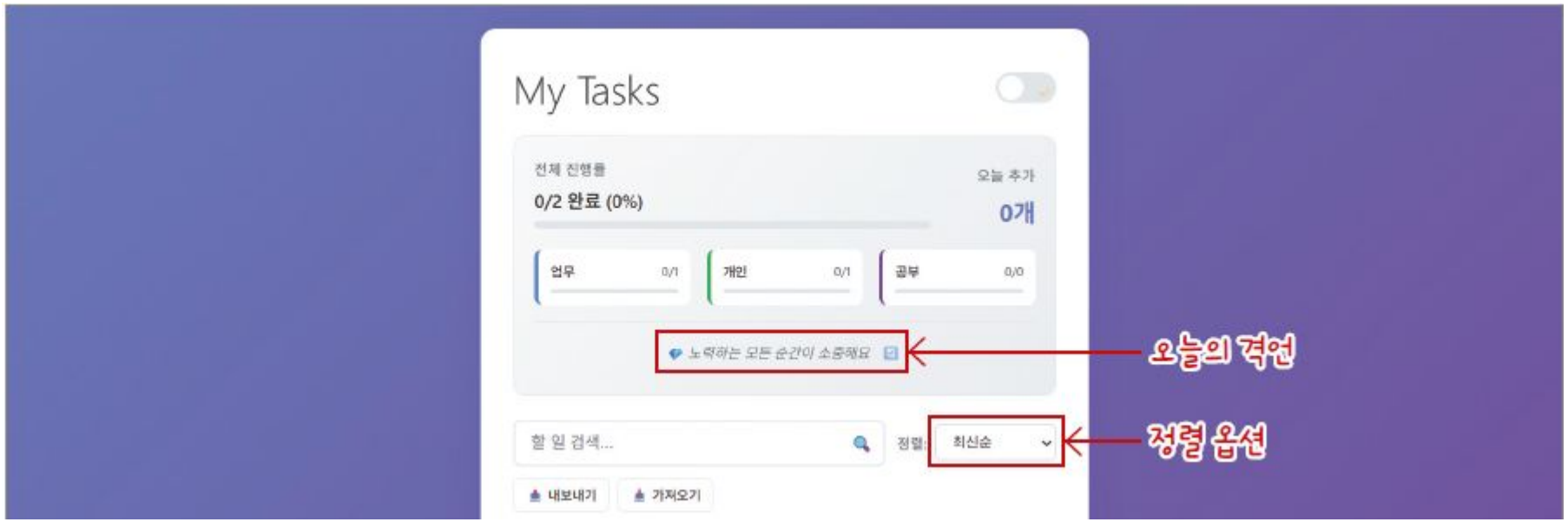
5단계: 최종 완성 및 최적화

01 단계별 프롬프트의 다섯 번째 프롬프트(Step5)를 복사해 클로드 코드에 입력



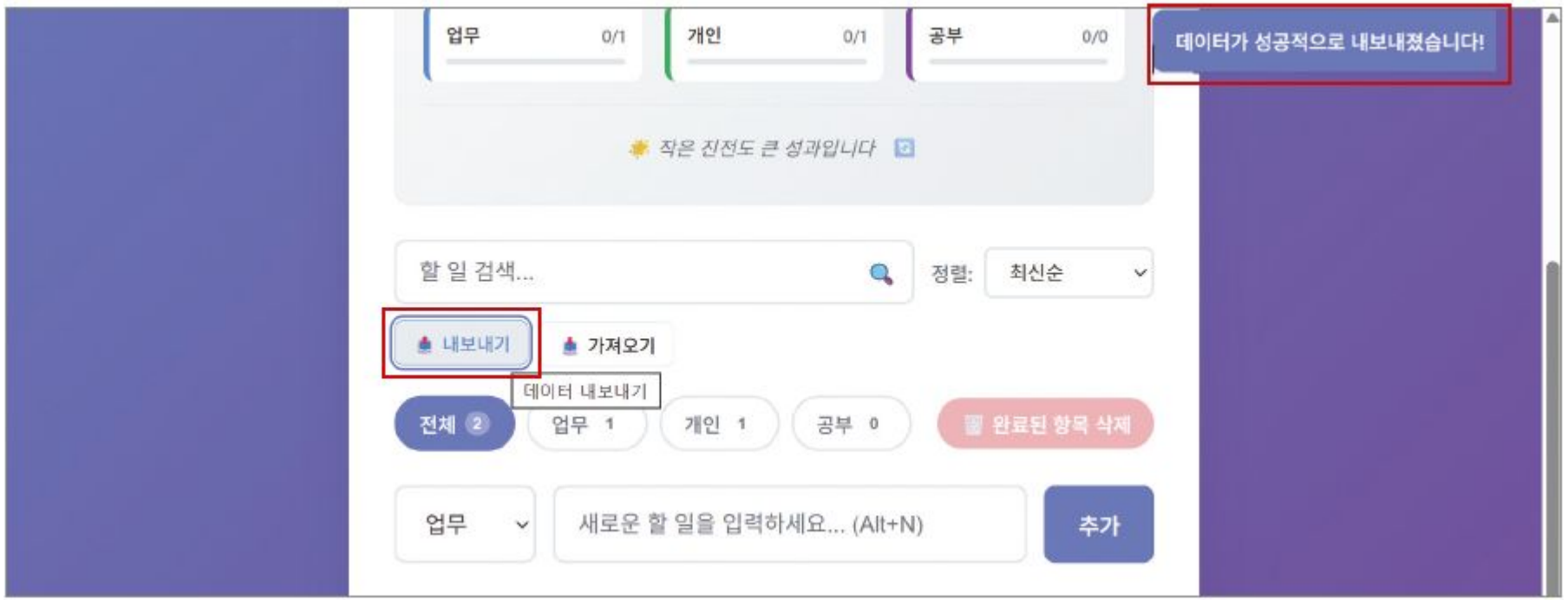
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(33)

02 완성되었다는 메시지가 보이면 새 기능을 테스트하기 위해 웹 브라우저에서 할 일 관리 앱을 실행 정렬 옵션이 추가되었고, 오늘의 격언도 랜덤으로 표시됨



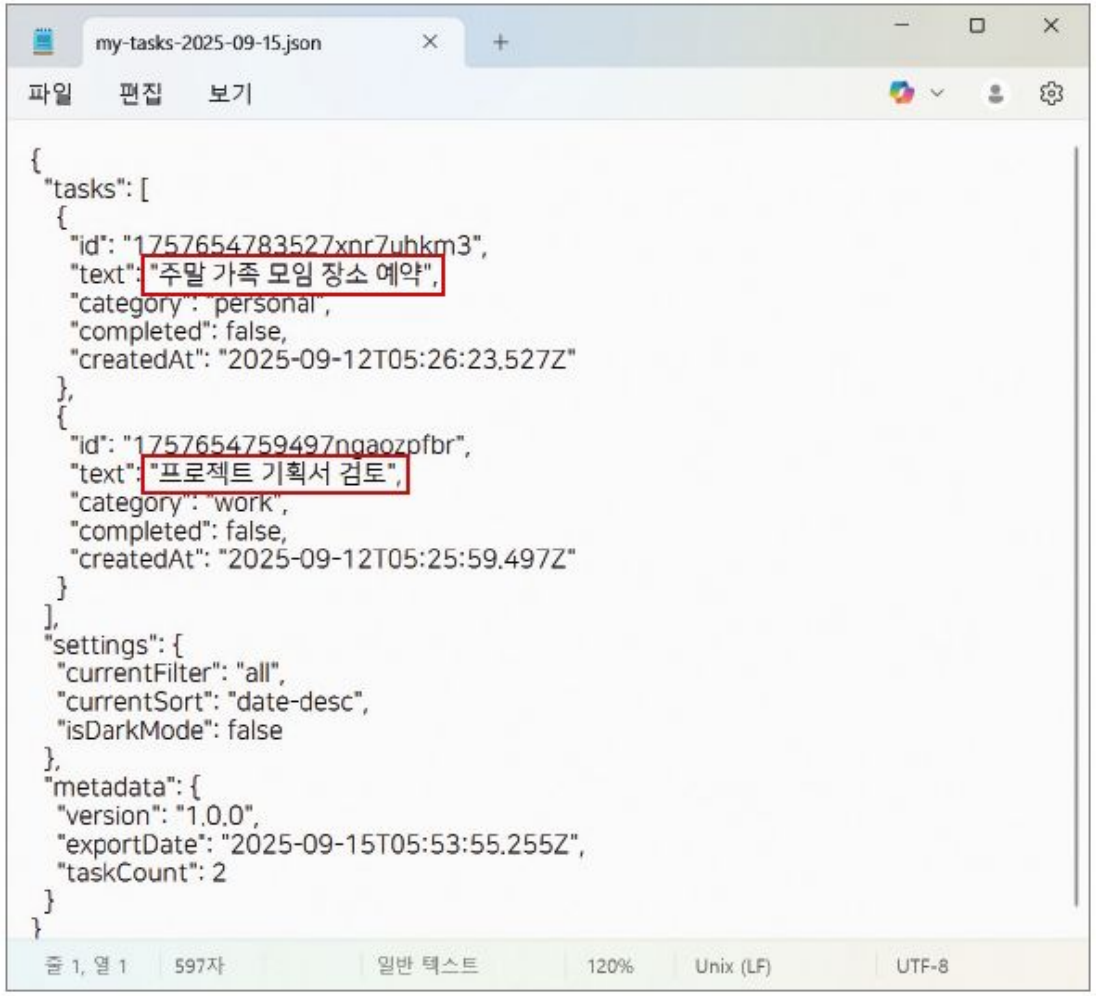
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(34)

03 [내보내기]를 누르면 애플리케이션이 데스크톱의 다운로드 경로에 JSON 파일로 저장됨



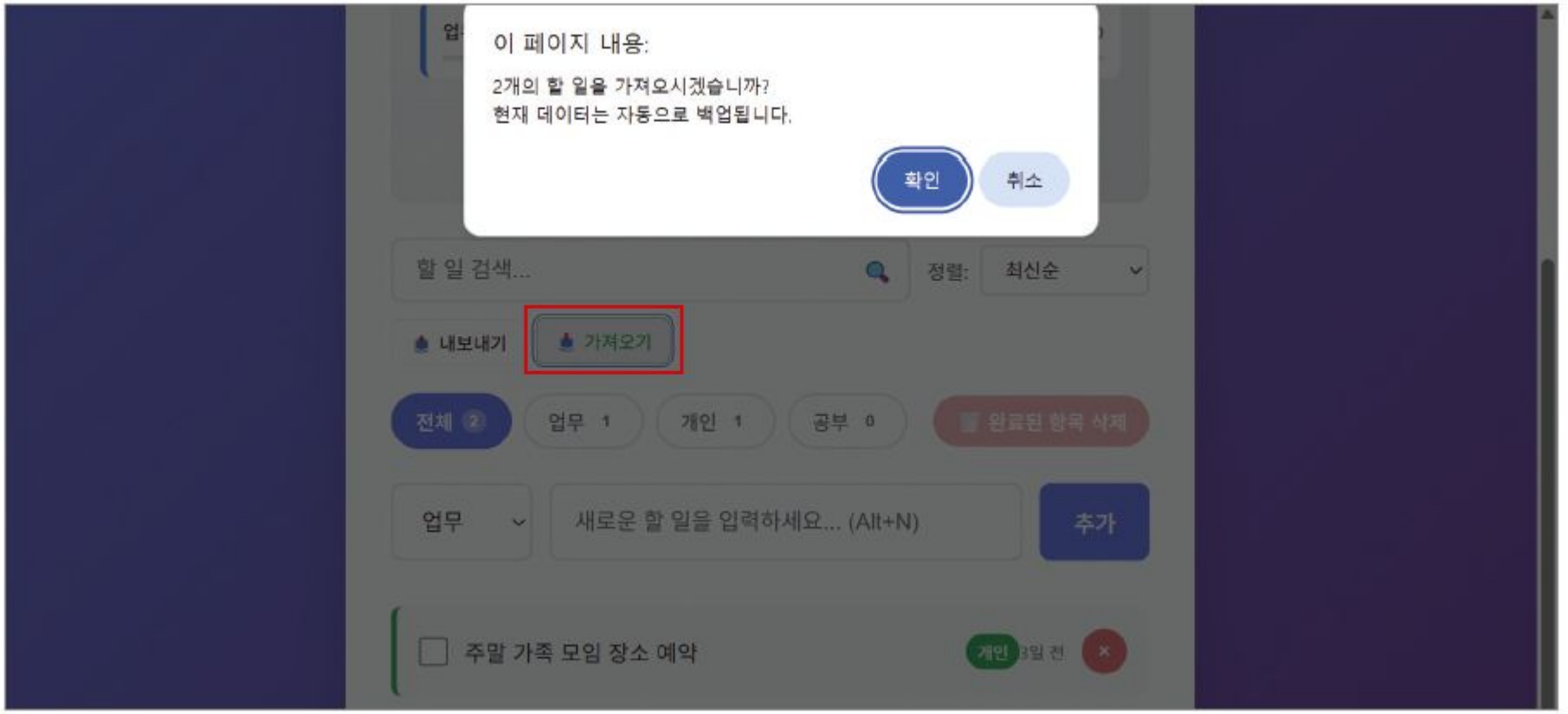
SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(35)

04 확장자가 '.json'인 파일을 메모장에서 열어서 확인
할 일 관리 앱에 저장했던 할 일 목록이 저장되어 있음



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(36)

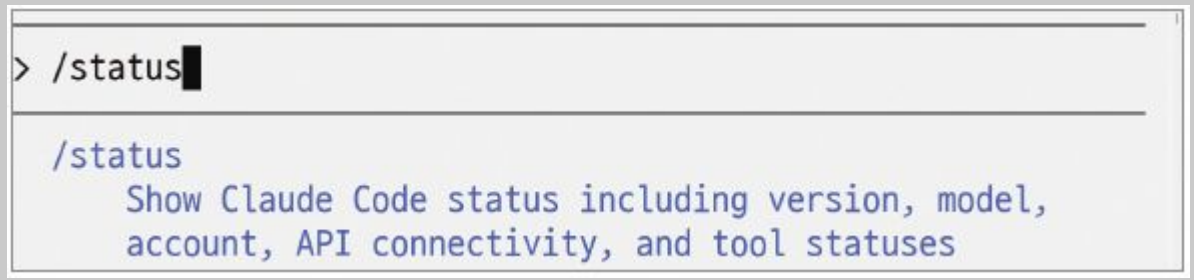
05 웹 브라우저에서 실행 중인 할 일 관리 앱에서 [가져오기] 버튼을 클릭하면 해당 JSON 파일을 불러올 수 있음 - 모든 할 일 목록이 복원



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(37)

좀더알아보기 | 클로드 초기 설정 명령어

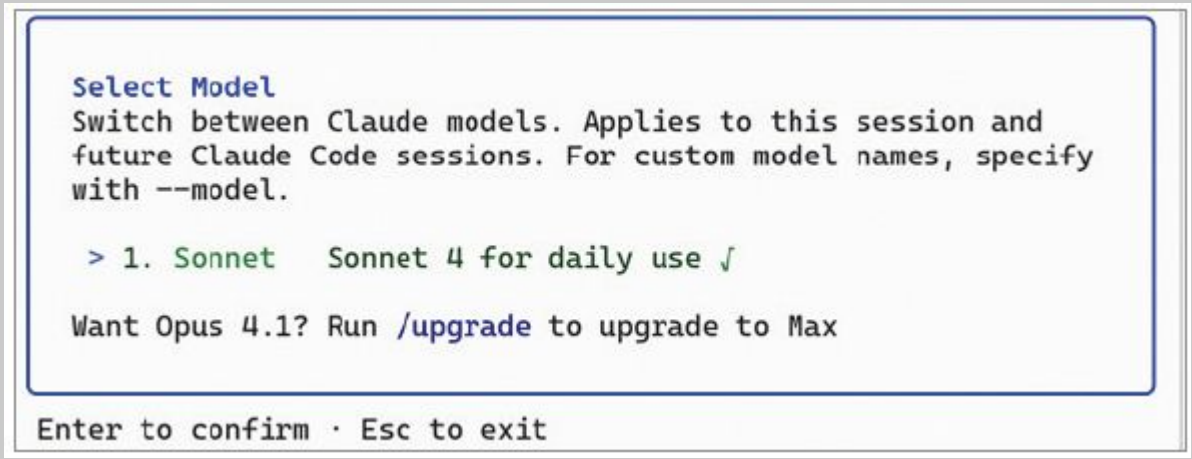
/status	• 현재 세션의 상태를 확인할 때 사용 • 현재 사용 중인 모델, 토큰 사용량, 남은 컨텍스트 용량 등 • 긴 작업을 할 때는 주기적으로 상태를 확인



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(38)

/model	• 현재 사용 중인 AI 모델을 확인하고 선택

- Max 플랜에서는 Sonnet 모델과 복잡한 작업을 위한 Opus 모델 중에서 선택할 수 있음



SECTION 4-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기(39)

/help	• 사용 가능한 모든 명령어와 설명을 확인 • 작업 중 새로운 기능이 필요하거나 명령어가 기억나지 않을 때 유용

[마무리]

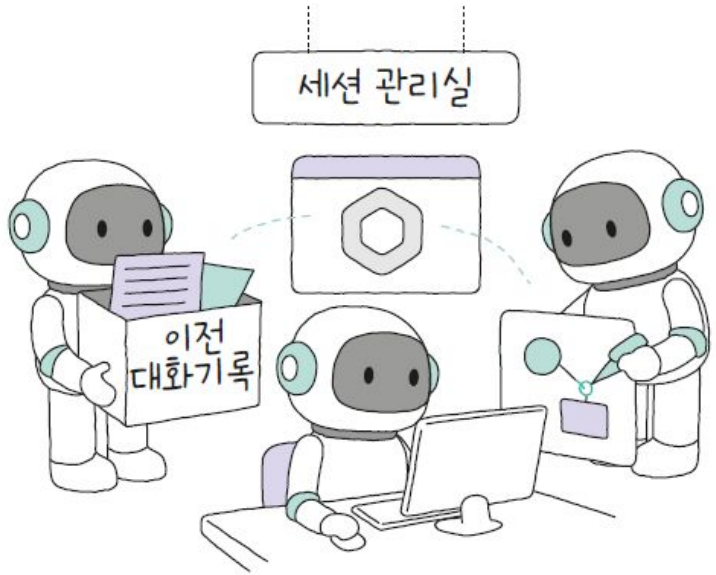
- 5가지 키워드로 정리하는 핵심 포인트
  - **단계별 프롬프트 전략**이란 PRD를 기반으로 프로젝트를 여러 단계로 나누어 각 단계마다 AI에게 구체적목표를 제시함으로써 체계적이고 예측 가능한 결과를 이끌어내는 프롬프트 설계 방식
  - 클로드 코드에서 프로젝트를 시작할 때 필요한 명령어
    - **/init**: 프로젝트 초기화 및 CLAUDE.md 생성
    - **/status**: 현재 작업 상태 확인
    - **/model**: AI 모델 선택 및 변경
    - **/help**: 사용 가능한 명령어 확인

SECTION 4-2 작업 재개와 효율성 높이기(1)

시작하기전에

◦ 세션(session)

- 컴퓨터나 프로그램에서 사용자와 시스템이 연결되어 있는 일정한 기간
- 클로드 코드에서는 대화를 시작해 명령을 주고받는 동안 유지되는 상태
- 클로드 코드는 터미널을 닫으면 이전 대화가 사라지는 것처럼 보이기 때문에, 장기 프로젝트를 진행할 때는 세션과 컨텍스트 관리에 특히 주의



SECTION 4-2 작업 재개와 효율성 높이기(2)

◦ 기능 추가하기

- 파일 탐색기에서 'Study-02' 폴더를 열고, 마우스 오른쪽 버튼을 눌러 [터미널에서 열기]를 선택해 명령 프롬프트를 실행

세션 재개하기

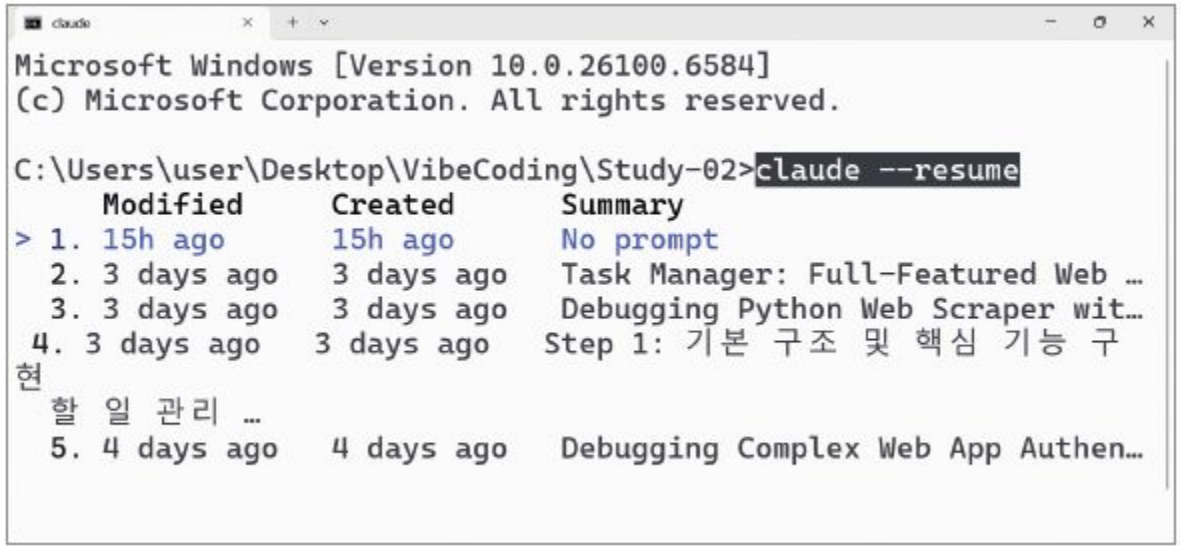
01 명령 프롬프트에 'claude --resume' 명령어를 입력

- 'claude --resume'은 이전 대화 기록을 불러와서 마지막 상태 그대로 이어 작업해줄 수 있게 하는 명령어
- '--resume' 옵션은 저장된 세션 파일을 읽어와 이전 컨텍스트를 복원

```
claude --resume
```

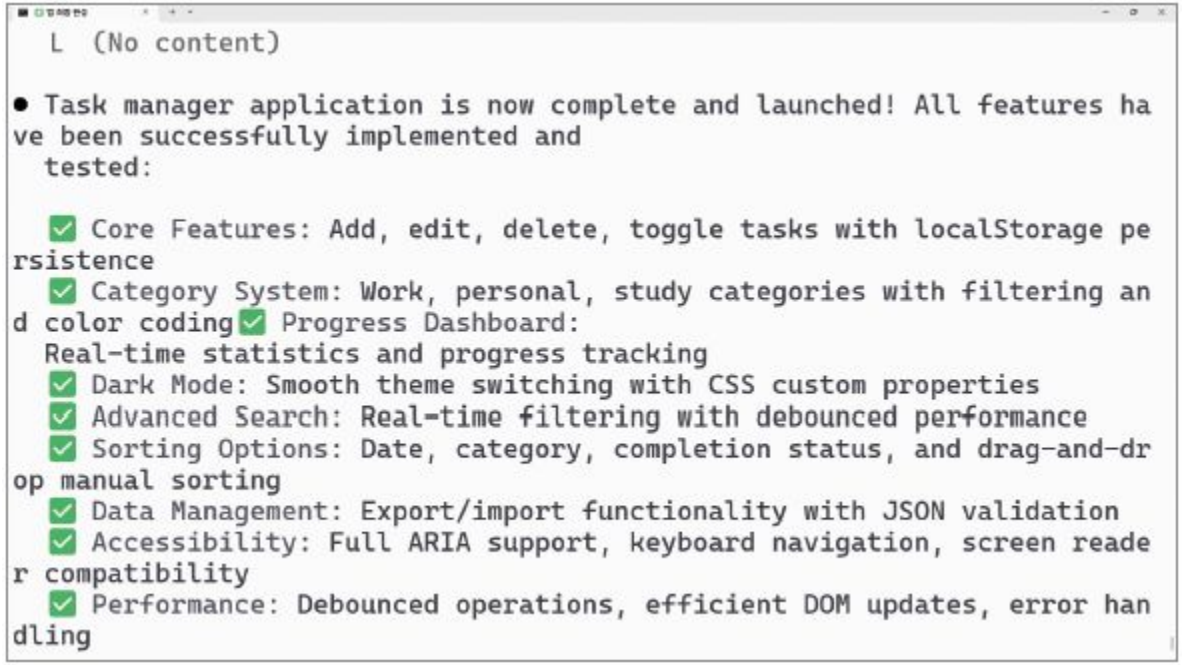
SECTION 4-2 작업 재개와 효율성 높이기(3)

02 현재 프로젝트의 대화 목록을 보여줌 - 화살표 키로 원하는 대화를 찾고, [Enter] 키를 눌러 재개



SECTION 4-2 작업 재개와 효율성 높이기(4)

03 할 일 관리 앱을 구현했던 대화를 찾아 선택한 뒤, [Enter]
대화가 중단된 시점부터 세션이 복원되어, 이어서 작업할 수 있음



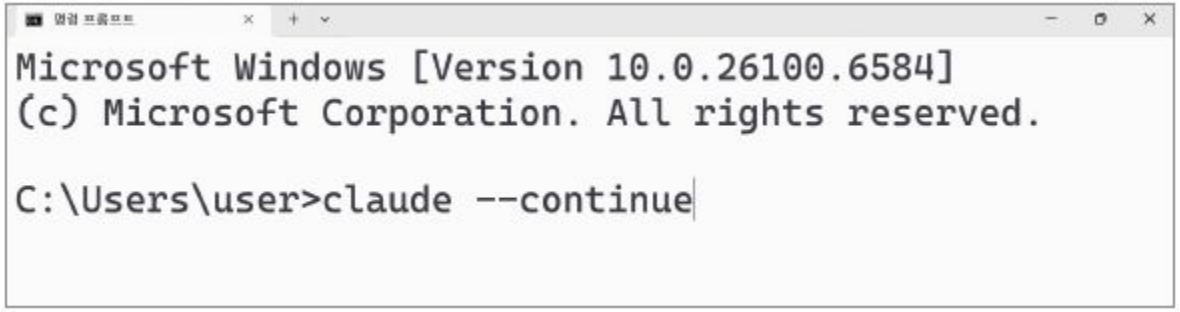
SECTION 4-2 작업 재개와 효율성 높이기(5)

✚ 여기서 잠깐 **세션 재개 명령어**

- 클로드 코드는 프로젝트별로 대화 세션을 저장해, 언제든 이전 작업을 이어서 진행할 수 있음
  만약 작업 중 터미널이 닫히거나, 중간에 다른 명령어를 실행하느라 대화가 끊긴 경우에도 걱정할 필요가 없음
  - 이전 대화를 그대로 불러와 이어서 진행할 수 있는 명령어들이 준비

1. **/resume**
   - 먼저 **claude**를 입력해서 클로드 코드를 실행한 후 **/resume** 명령어를 입력하여 이전 대화를 선택하여 재개하는 방법

2. **--continue 또는 -c 옵션**
   - **--continue**는 가장 최근의 마지막 대화를 자동으로 복구하게 해 주는 플래그
   - 작업 중 연결이 끊기거나 터미널이 종료되었을 때 ... 또는 ... 옵션을 사용



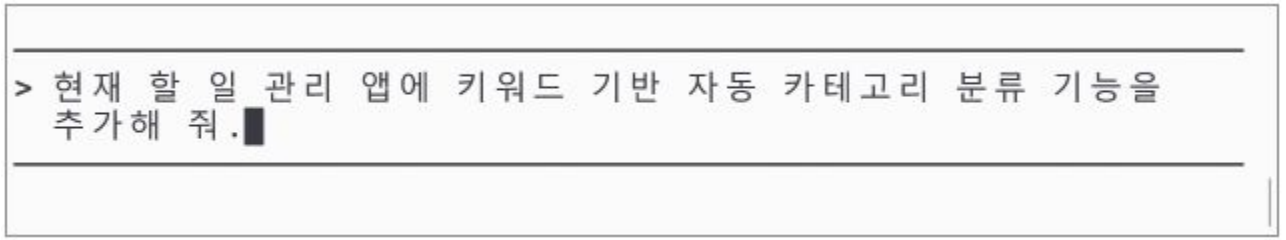
SECTION 4-2 작업 재개와 효율성 높이기(6)

자동 승인 모드로 작업 효율성 높이기

01 클로드 코드에 다음과 같이 프롬프트를 입력

프롬프트

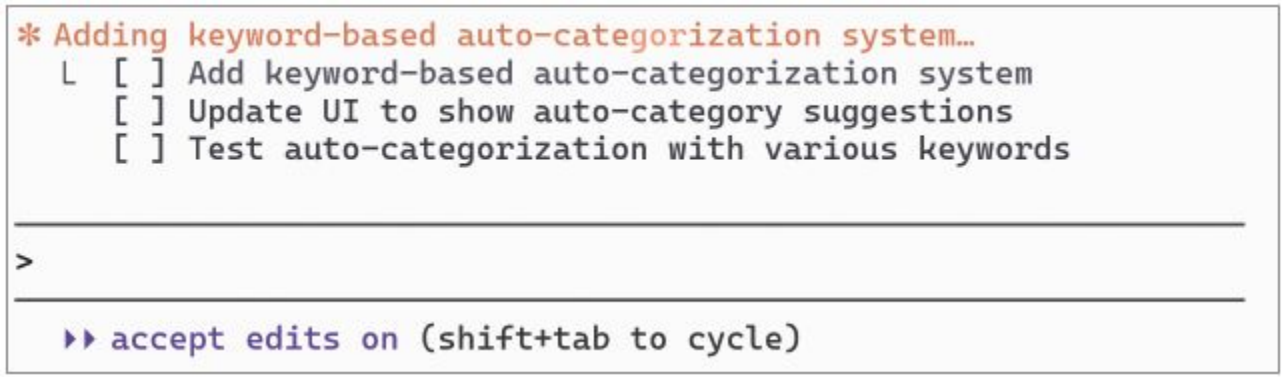
```
현재 할 일 관리 앱에 키워드 기반 자동 카테고리 분류 기능을 추가해 줘.
```



SECTION 4-2 작업 재개와 효율성 높이기(7)

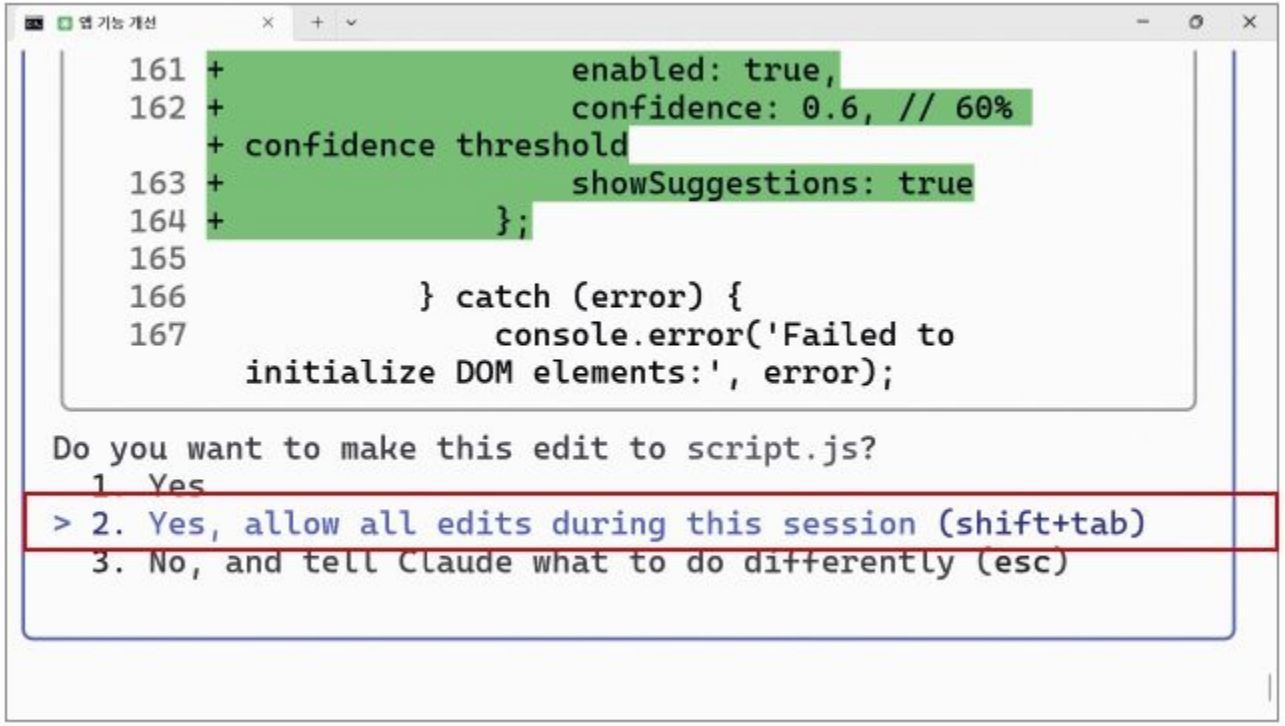
02 그러면 클로드 코드가 대략 다음과 같은 작업을 진행

```
[] 키워드 기반 자동 분류 시스템 추가
[] 카테고리를 자동으로 제안하는 UI 업데이트
[] 다양한 키워드로 자동 분류 기능 테스트
```



SECTION 4-2 작업 재개와 효율성 높이기(8)

03 [Shift]+[Tab] 키를 눌러 '2. Yes, allow all edits during this session'을 선택

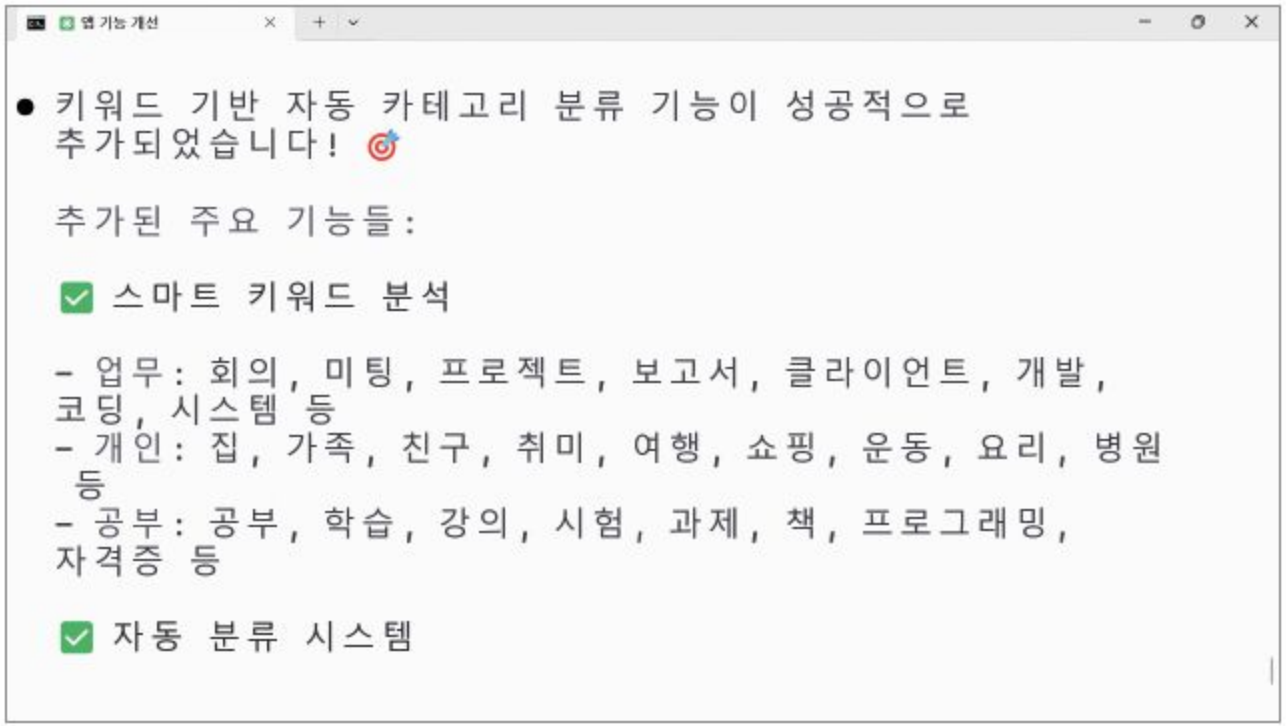


[note]

- 하단에 `▸▸ accept edits on (shift+tab to cycle)` 이 표시되는 동안에는 모든 수정을 자동으로
  승인하게 되며, 다시 **[Shift]+[Tab]** 키를 누르면 일반 모드로 돌아감

SECTION 4-2 작업 재개와 효율성 높이기(9)

04 작업이 완료되면 기능이 성공적으로 추가되었다는 메시지를 볼 수 있음

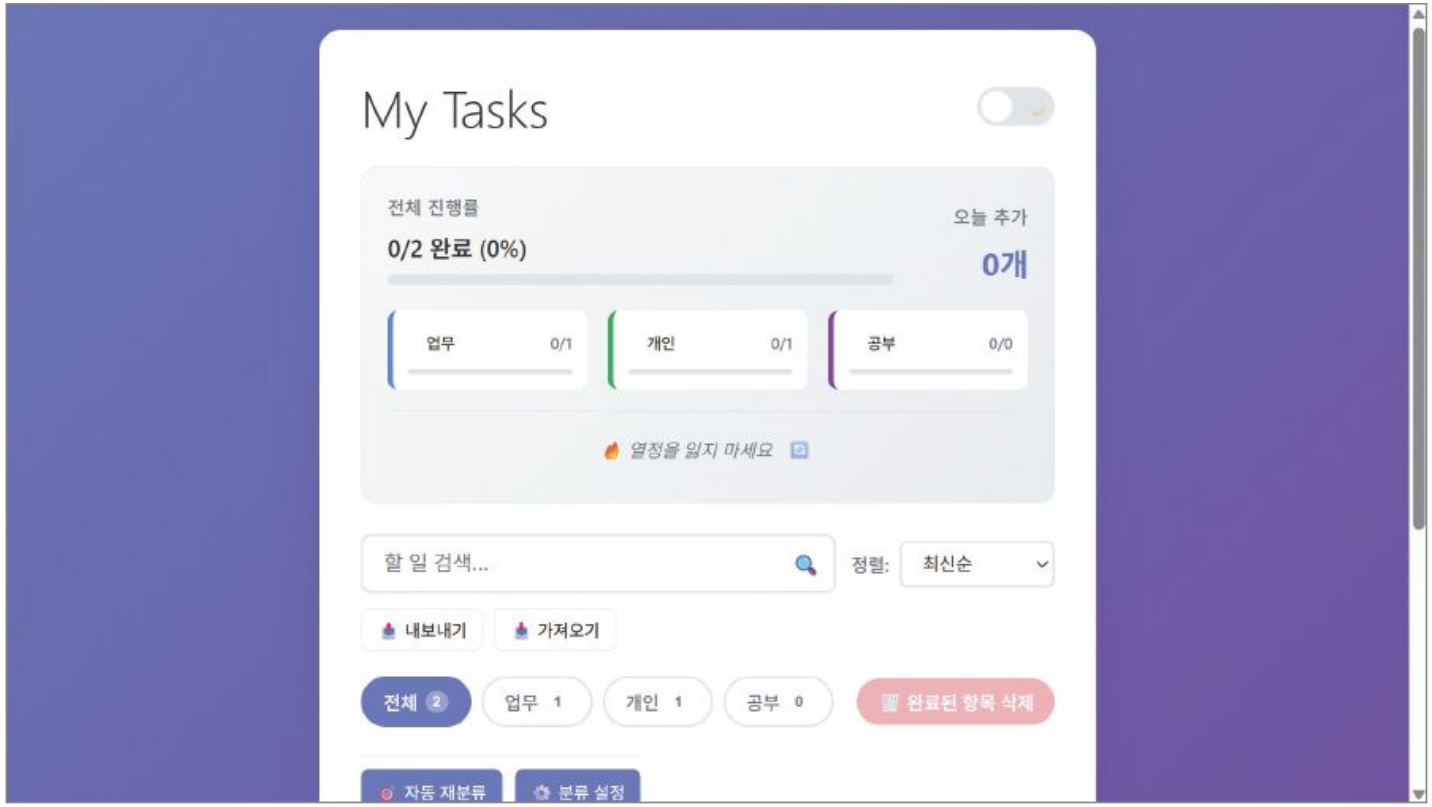


SECTION 4-2 작업 재개와 효율성 높이기(10)

기능 점검하기

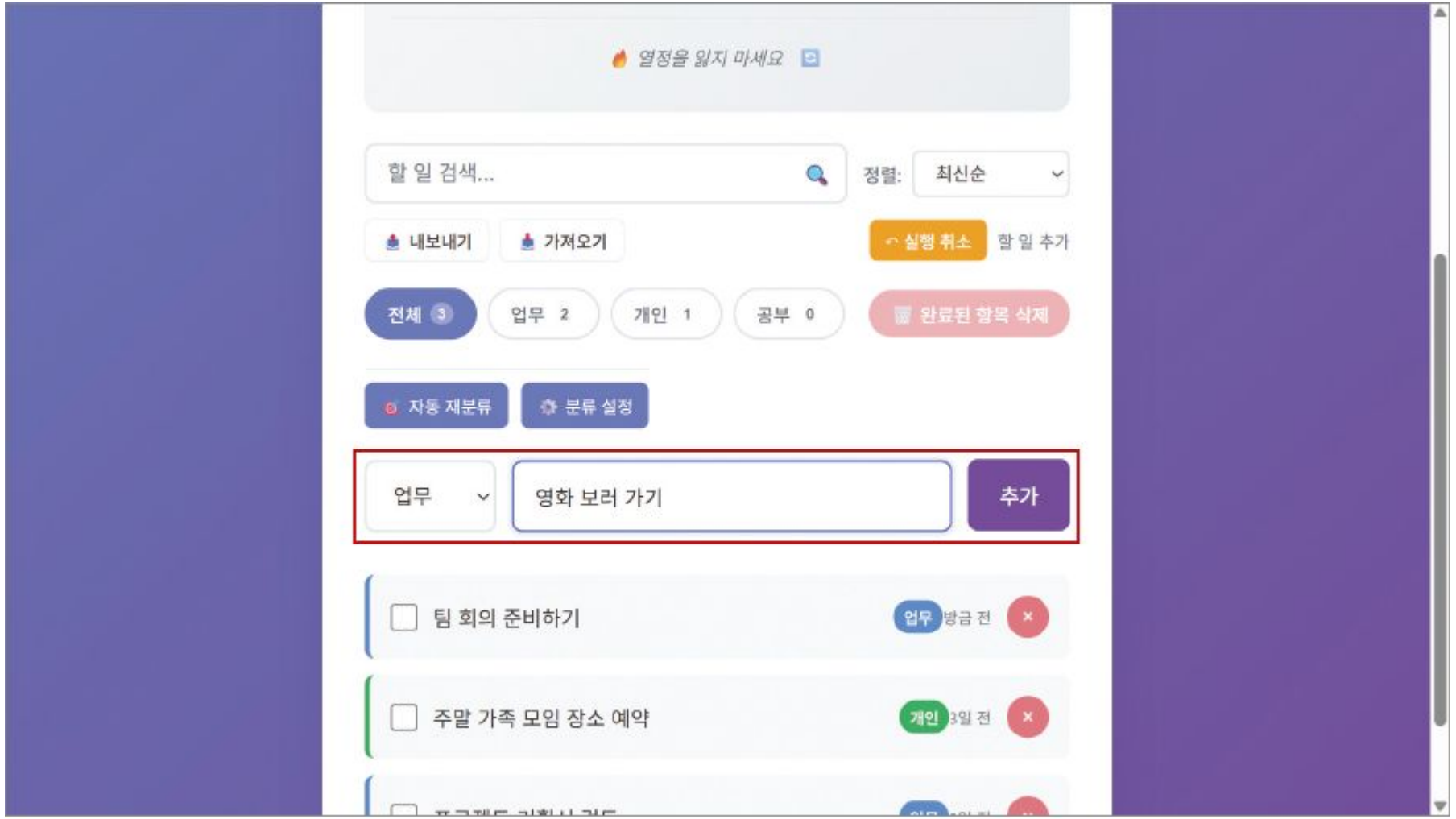
- '스마트 키워드 분석', '자동 분류 시스템' 기능이 추가

01 변경 사항이 잘 반영되었는지 확인하기 위해 데스크톱 파일 탐색기의 'Study-02' 폴더에서 'index.html' 파일을 실행



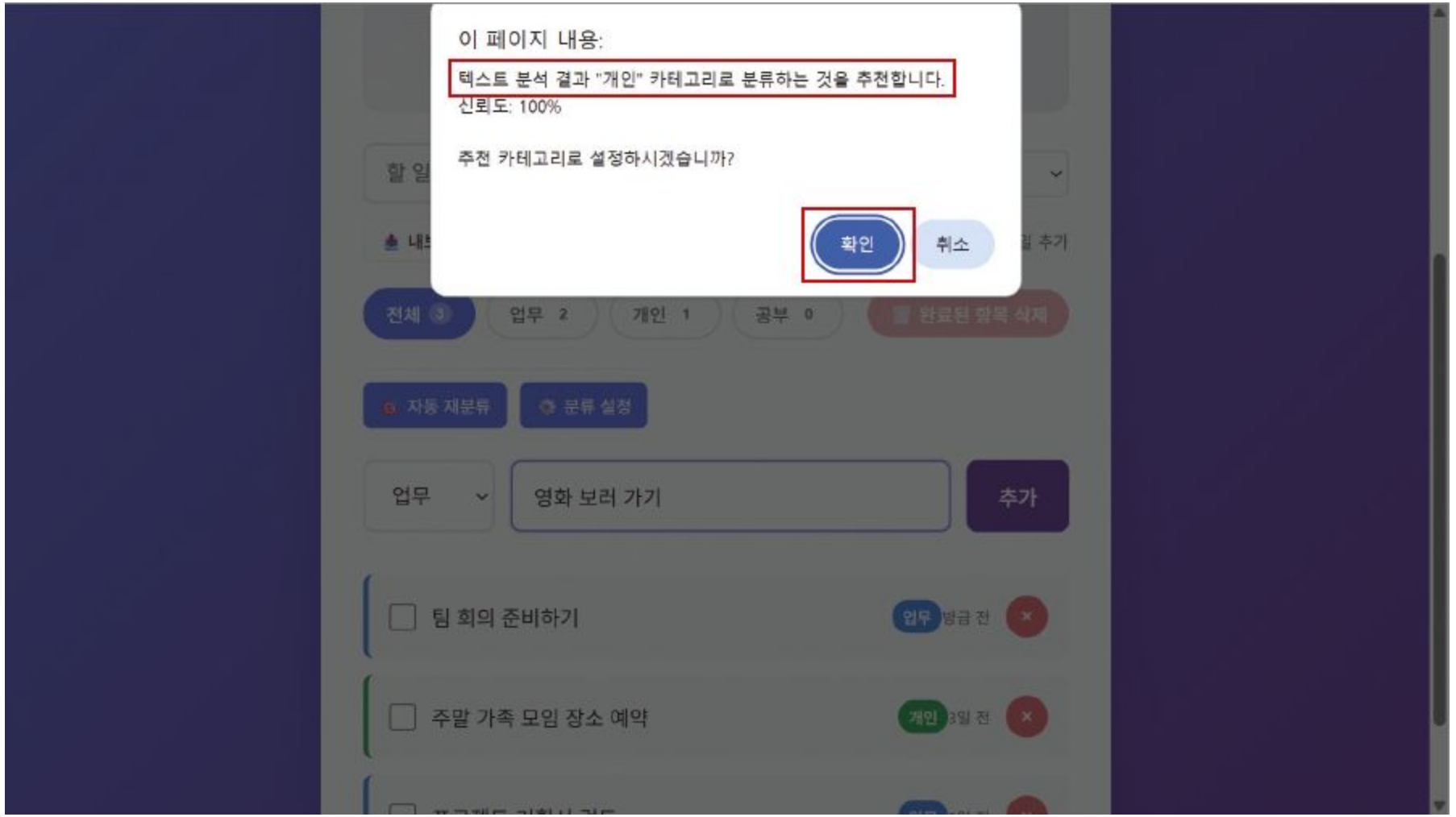
SECTION 4-2 작업 재개와 효율성 높이기(11)

02 카테고리를 자동으로 분류하는지 확인하기 위해 [할 일 검색…] 창에 '영화 보러 가기'라고 입력하고, 카테고리는 '업무'로 둔 뒤 [추가] 버튼을 클릭



SECTION 4-2 작업 재개와 효율성 높이기(12)

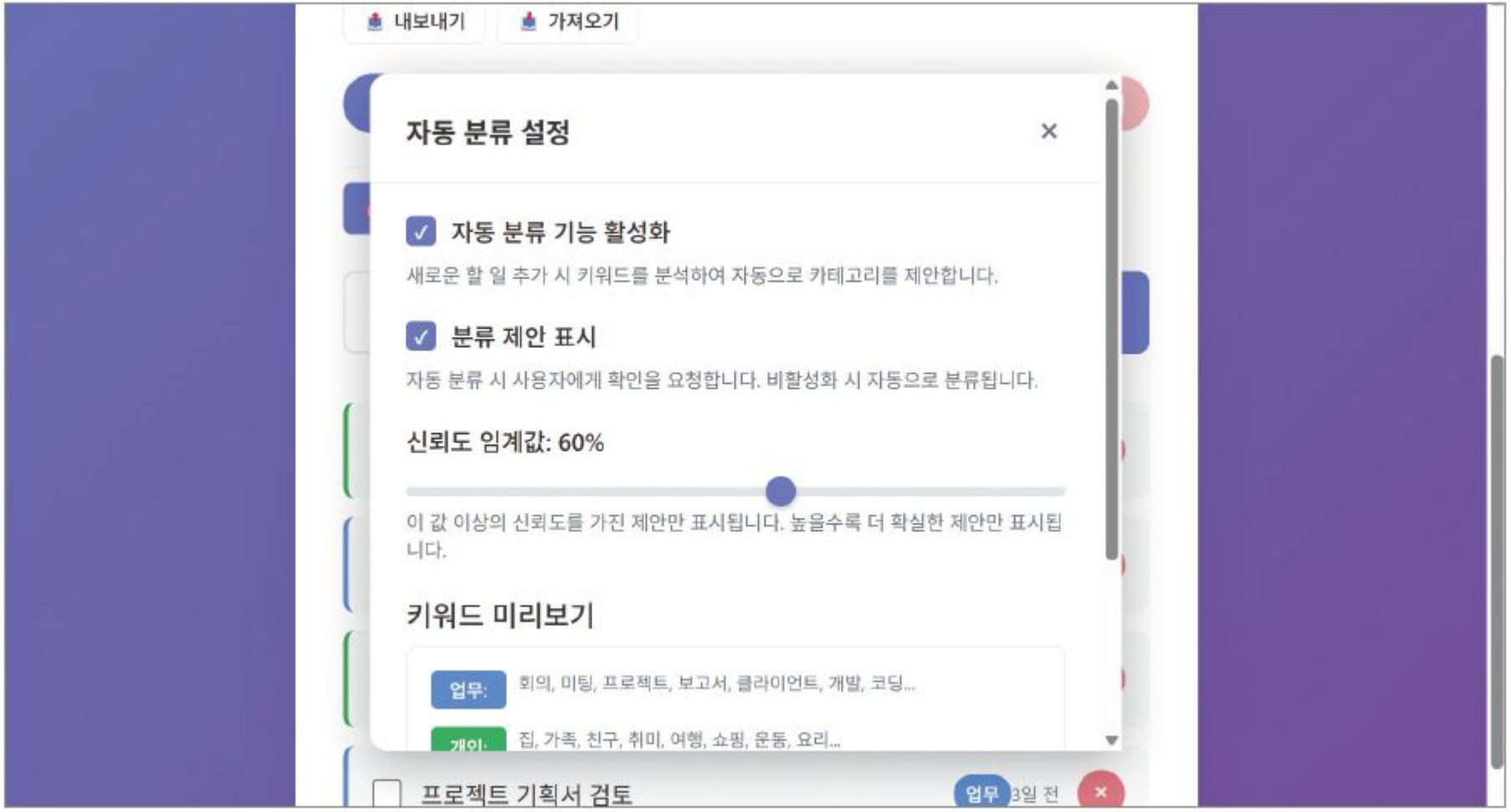
03 할 일 관리 앱이 '영화 보러 가기'의 카테고리를 '개인'으로 분류하는 것을 추천
[확인]을 눌러 진행하면 할 일이 '개인' 카테고리로 분류되어 정상적으로 등록



SECTION 4-2 작업 재개와 효율성 높이기(13)

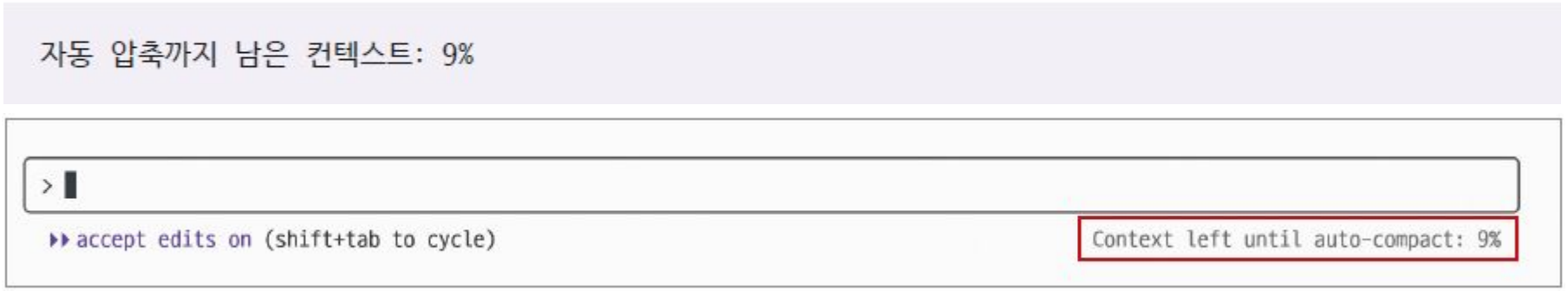
04 할 일 관리 앱의 초기 화면에는 [자동 재분류], [분류 설정] 버튼

- 카테고리 재분류 기능을 활성/비활성화 하거나 세부 옵션을 조정



SECTION 4-2 작업 재개와 효율성 높이기(14)

◦ 컨텍스트 관리하기



- 현재 대화의 컨텍스트(context)가 자동 압축(auto-compact) 실행 임곗값에 가까워졌다는 경고 메시지
  - 컨텍스트 - 클로드 코드가 현재 세션에서 기억하고 있는 모든 대화 내용과 작업 정보
  - 즉 이 메시지는 AI가 기억할 수 있는 공간이 부족해지고 있으며, 오래된 대화를 압축하여 토큰 사용량을 줄인 뒤 작업을 계속하겠다는 의미
- 클로드가 자동으로 전체 대화의 일부를 요약하는 과정에서 정보 손실이 있을 수 있음
  - 작업 단위를 끝까지 마무리하고 요약하지 않고, 갑자기 중간 부분에서 억지로 요약하는 경우도 있음
  - 따라서 클로드가 컨텍스트를 자동 압축하기 전에 수시로 컨텍스트 상태를 체크하여 적절한 시점에 직접 압축하는 것이 좋음

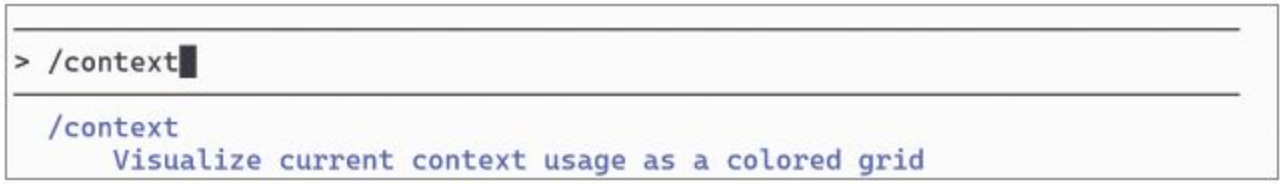
SECTION 4-2 작업 재개와 효율성 높이기(15)

컨텍스트 상태 확인하기

- /context - 현재 컨텍스트 상태와 사용량을 확인하는 명령어

화면 해설

현재 컨텍스트 사용량을 컬러 그리드로 시각화합니다.



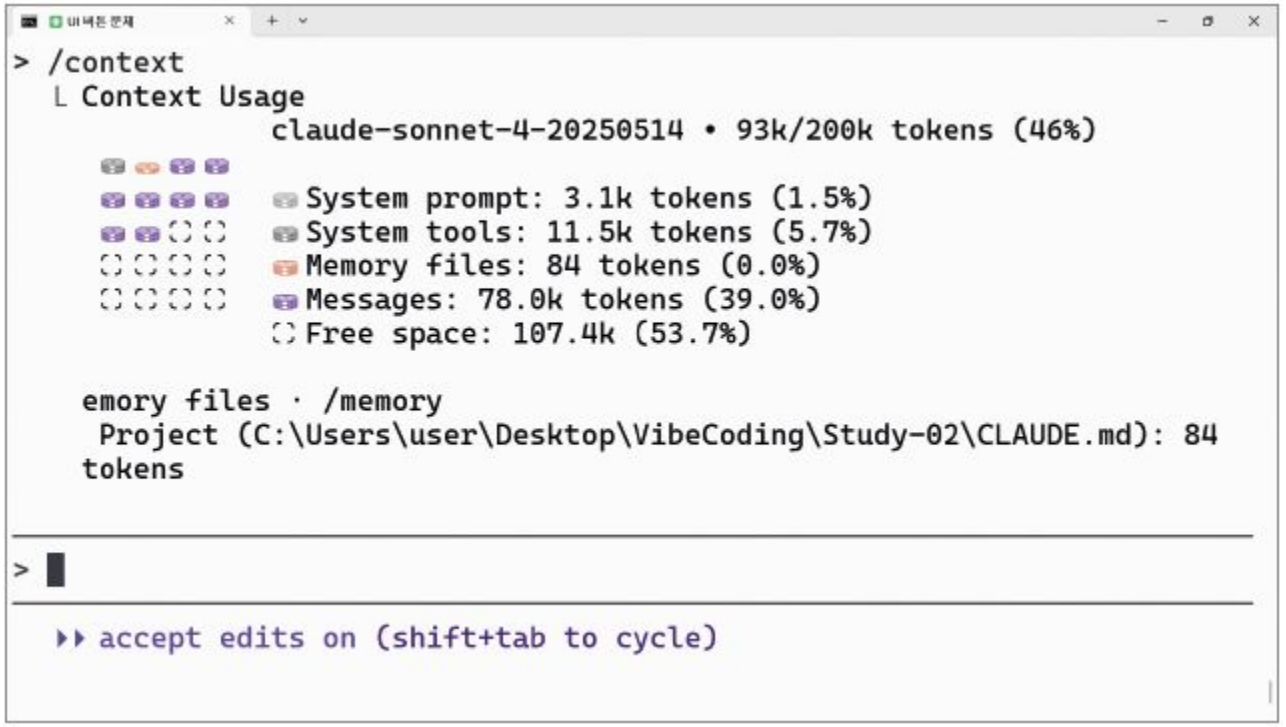
SECTION 4-2 작업 재개와 효율성 높이기(16)

- 클로드 코드에 '/context' 명령어를 입력하고, [Enter] - 현재까지의 컨텍스트 사용량을 확인

화면 해설

```
컨텍스트 사용 현황
모델: claude-sonnet-4-20250514 • 현재 사용량: 93k / 200k 토큰 (46%)
시스템 프롬프트: 3.1k 토큰 (1.5%)
시스템 도구: 11.5k 토큰 (5.7%)
메모리 파일: 84 토큰 (0.0%)
메시지: 78.0k 토큰 (39.0%)
여유 공간: 107.4k 토큰 (53.7%)

메모리 파일 • /memory
Project: C:\Users\user\Desktop\VibeCoding\Study-02\CLAUDE.md: 84 토큰
```

SECTION 4-2 작업 재개와 효율성 높이기(17)



[note]

- 토큰 한도는 내 구독 플랜에 따라 다름

SECTION 4-2 작업 재개와 효율성 높이기(18)

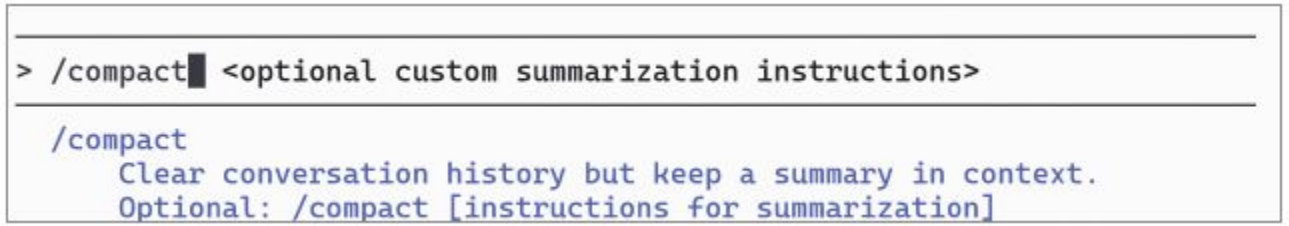
컨텍스트 압축하기

- /compact -. 현재까지의 대화 내용을 요약하는 압축 명령어

화면 해설

대화 기록을 지우고, 요약본은 남겨둡니다.

옵션: /compact [요약 지침]

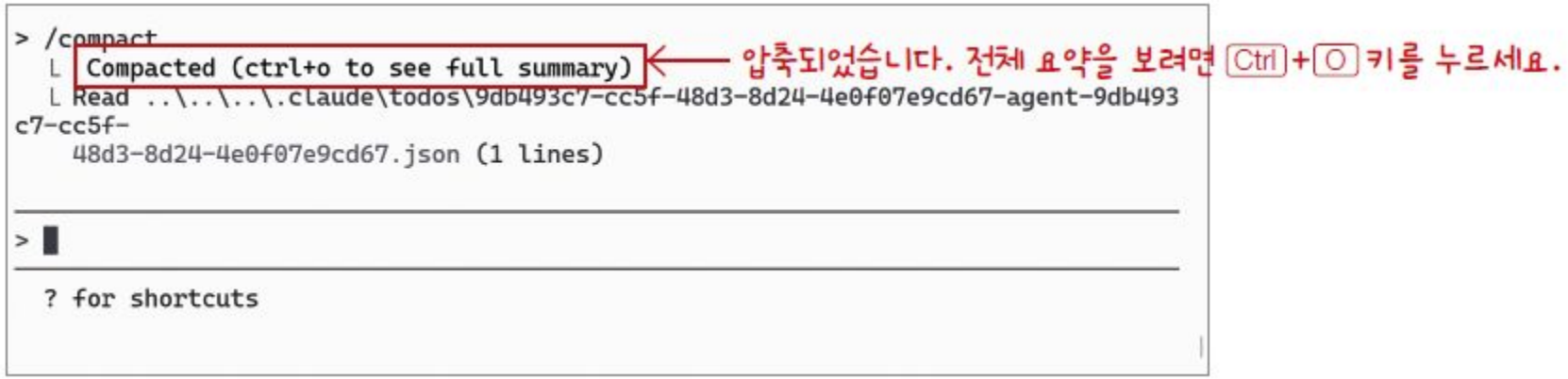


- 사용자의 필요에 따라 컨텍스트의 압축 수준과 범위를 세밀하게 조정할 수 있는 고급 옵션 (/compact 명령어 뒤에 공백을 한 칸 두고 옵션을 추가로 입력)
  - /compact --level 1: 불필요한 공백이나 중복을 제거
  - /compact --level 2: 변수명을 단축하고 구조를 단순화
  - /compact --level 3: 핵심 로직만 유지하고 주석, 변수명 등을 최대한 압축

SECTION 4-2 작업 재개와 효율성 높이기(19)

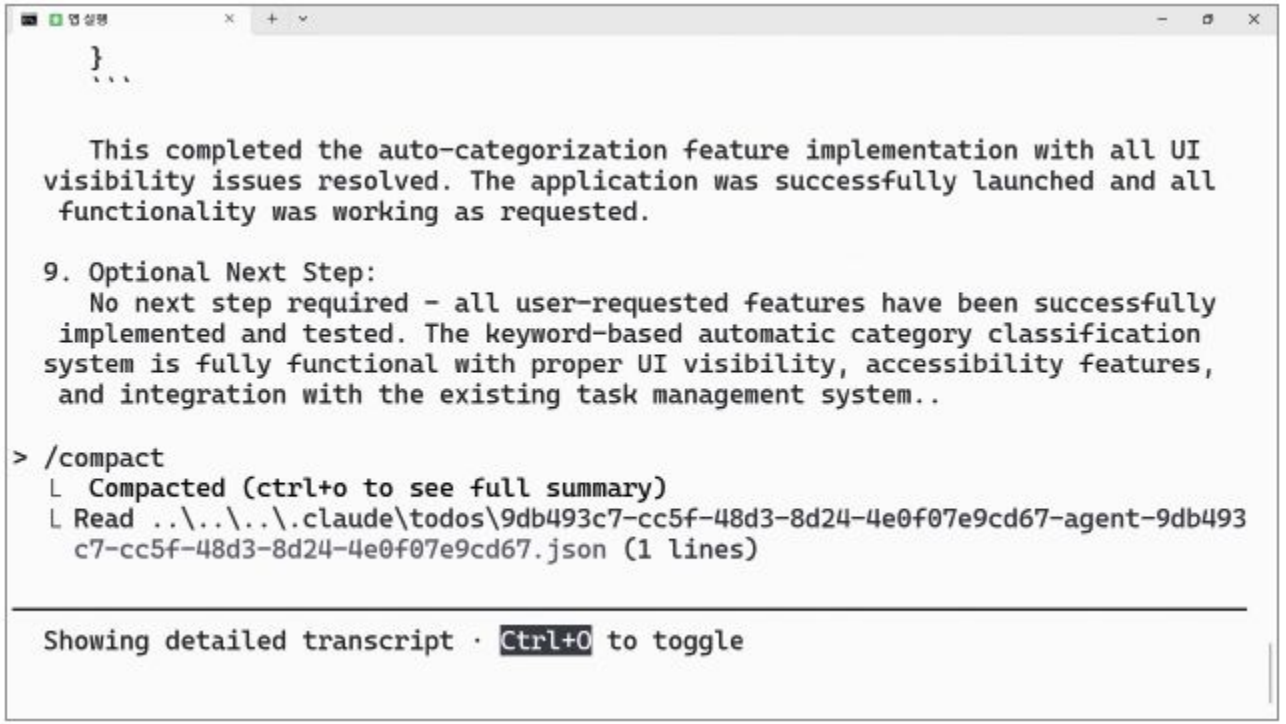
01 컨텍스트를 압축하기 위해 클로드 코드에 '/compact' 명령어를 입력

- 압축이 끝나면 'Compacted (ctrl+o to see full summary)'라는 메시지가 표시되고, 화면 하단의 auto-compact 경고 메시지가 사라짐



SECTION 4-2 작업 재개와 효율성 높이기(20)

02 [Ctrl]+[O] 키를 눌러 압축 과정에서 생성된 전체 요약을 확인



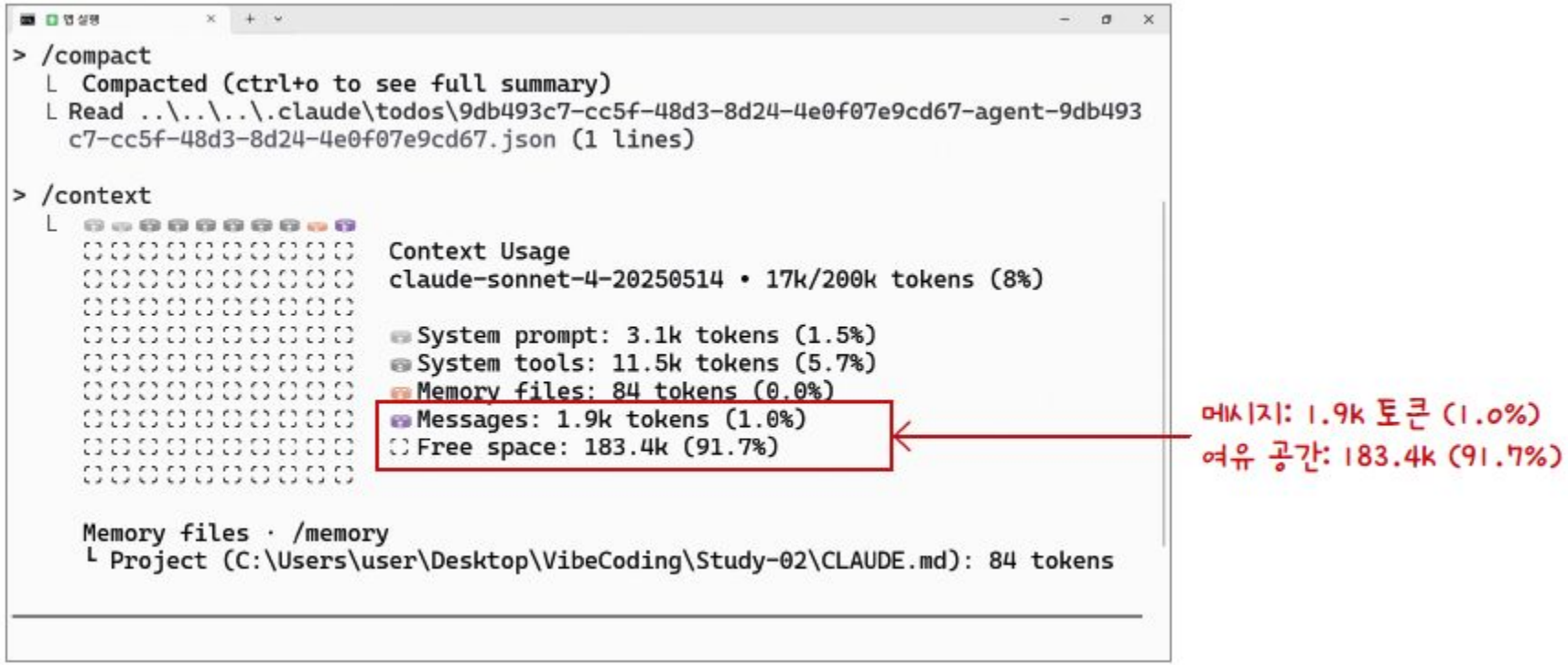
[note]

- **[Ctrl]+[O]** 키를 다시 누르면 요약 화면이 닫힘

SECTION 4-2 작업 재개와 효율성 높이기(21)

03 압축 결과를 확인하기 위해 '/context' 명령어를 입력

- 압축 결과 'Messages'가 78.0k에서 1.9k로 감소, 'Free space'는 107.4k(53.7%)에서 183.4k(91.7%)로 증가



[마무리]

◦ 4가지 키워드로 정리하는 핵심 포인트

- **--resume** 명령어는 대화 목록에서 원하는 세션을 선택해 복원
- [Shift]+[Tab] 키를 누르면 **자동 승인 모드**로 전환되어 반복적인 권한 승인 요청을 생략할 수 있음
- **컨텍스트**는 현재 대화와 작업 정보를 저장 및 표시한 메모리 공간
- **/compact** 명령어는 대화 기록을 요약해 컨텍스트 공간을 확보

SECTION 4-3 프로젝트 개선과 작업 관리(1)

시작하기전에

- 멀티모달 AI(multimodal AI)
  - 하나의 AI가 여러 종류의 정보를 함께 이해하고 처리하는 AI
  - 멀티모달 AI는 텍스트나 코드뿐만 아니라 이미지, 음성 등 서로 다른 형태의 데이터를 함께 인식할 수 있음
- 클로드 코드가 실행될 때 백엔드(back-end)에서는 Sonnet이나 Opus 같은 클로드의 멀티모달 AI 모델이 동작
  - 클로드 코드는 명령어 기반의 인터페이스지만, 이미지를 전달하면 내부 모델이 그 이미지를 분석해 텍스트로 설명하거나 구조를 파악한 뒤 코드로 바꿔 줄 수 있음

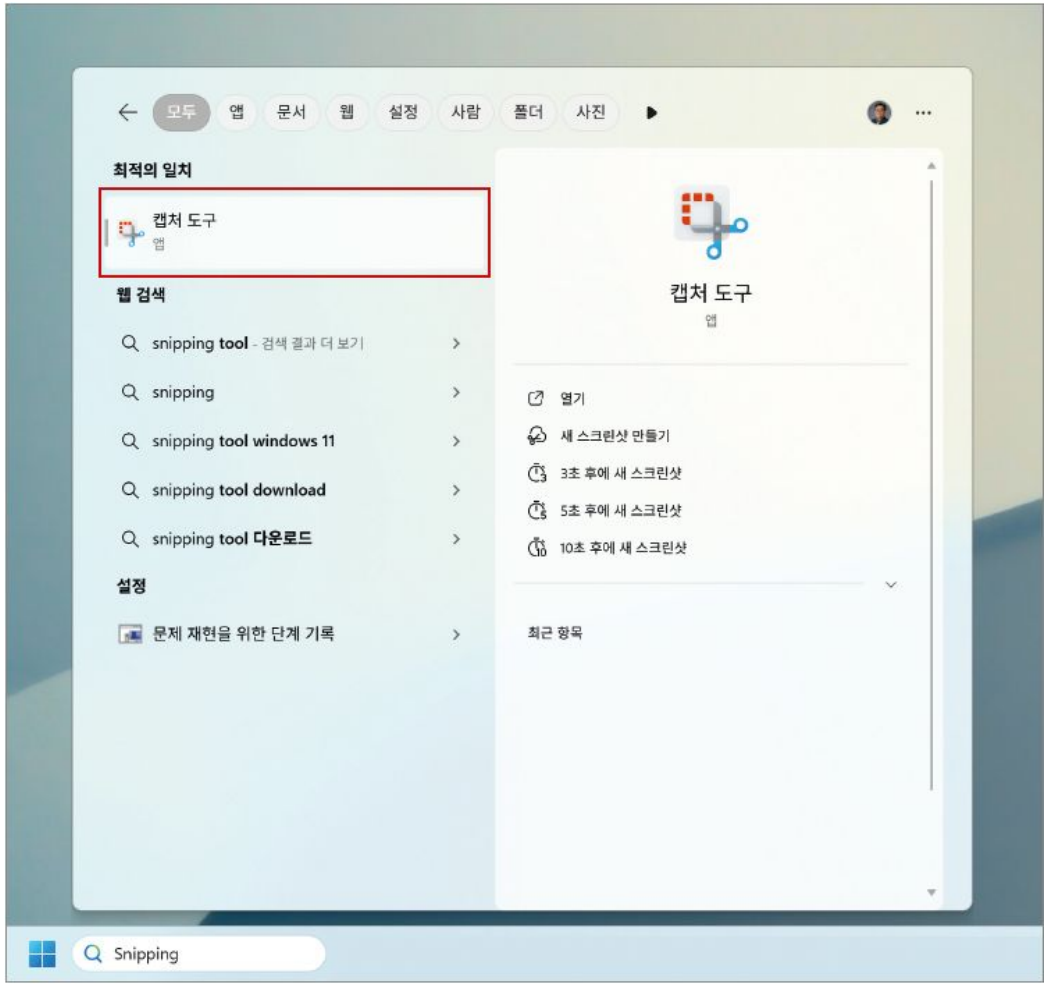
SECTION 4-3 프로젝트 개선과 작업 관리(2)

- 캡처 이미지로 데스크톱 버전 앱의 UI 개선하기
  - 화면을 직접 캡처(capture)해 전달하면 클로드 코드가 요구사항을 더 정확하고 빠르게 이해
  - 요청할 사항
    - 현재 결과가 모바일 중심의 디자인이라 화면에 꽉차게 나오는 PC 버전으로 만들기
    - 이전 결과를 보관하기 위해 'web_version'이라는 폴더를 새로 만들어 그곳에 저장하기

SECTION 4-3 프로젝트 개선과 작업 관리(3)

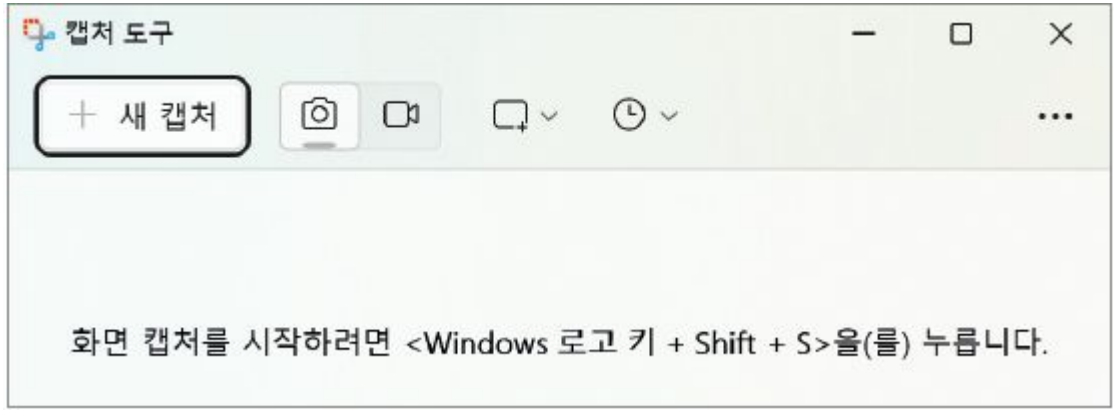
클로드 코드에 이미지 전달하기

01 윈도우의 [시작] 메뉴에서 'Snipping' 혹은 '캡처'를 입력해서 [캡처 도구]를 실행



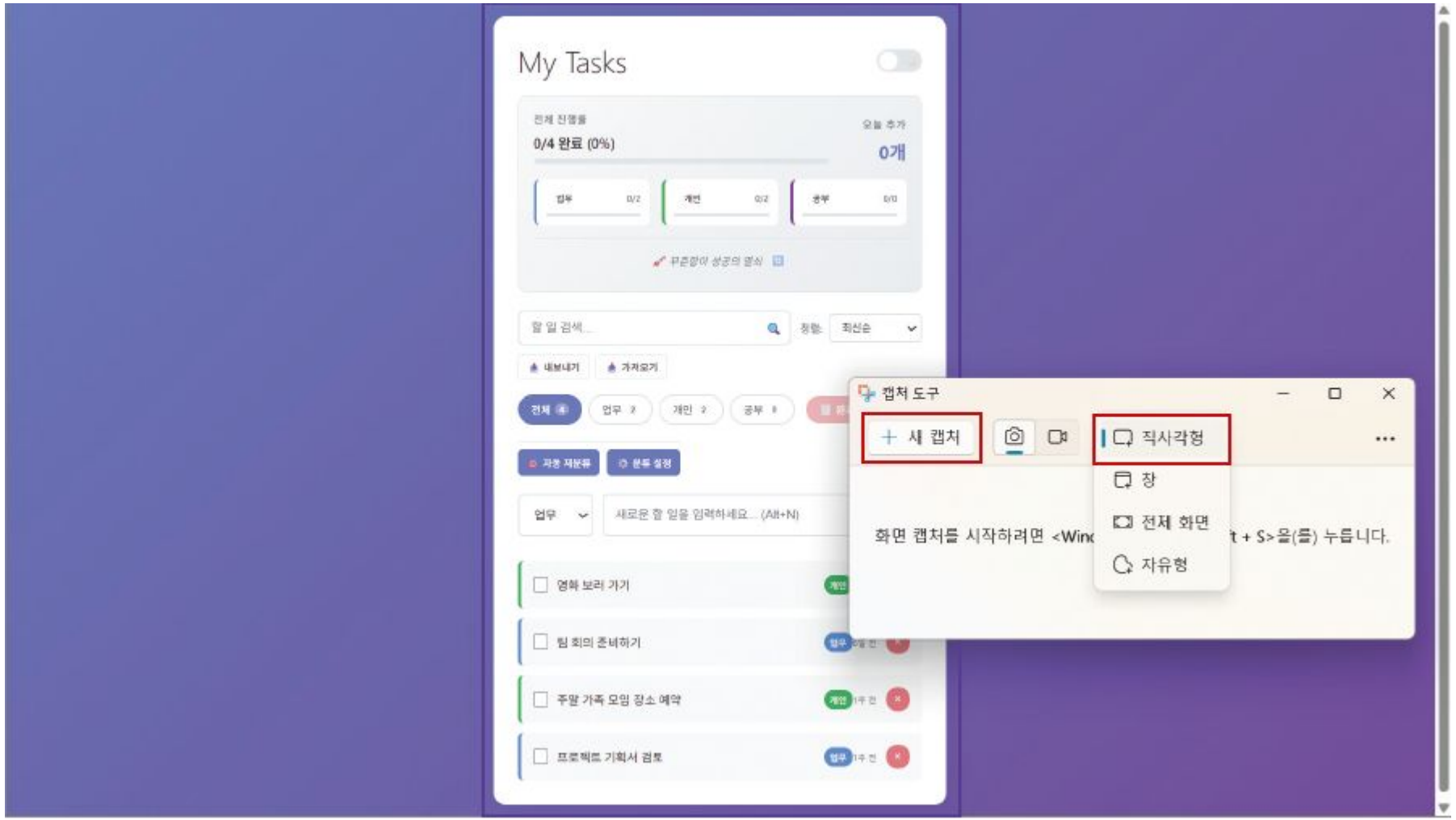
SECTION 4-3 프로젝트 개선과 작업 관리(4)

02 캡처 도구를 실행하면 [+ 새 캡처] 버튼을 클릭하여 캡처를 시작



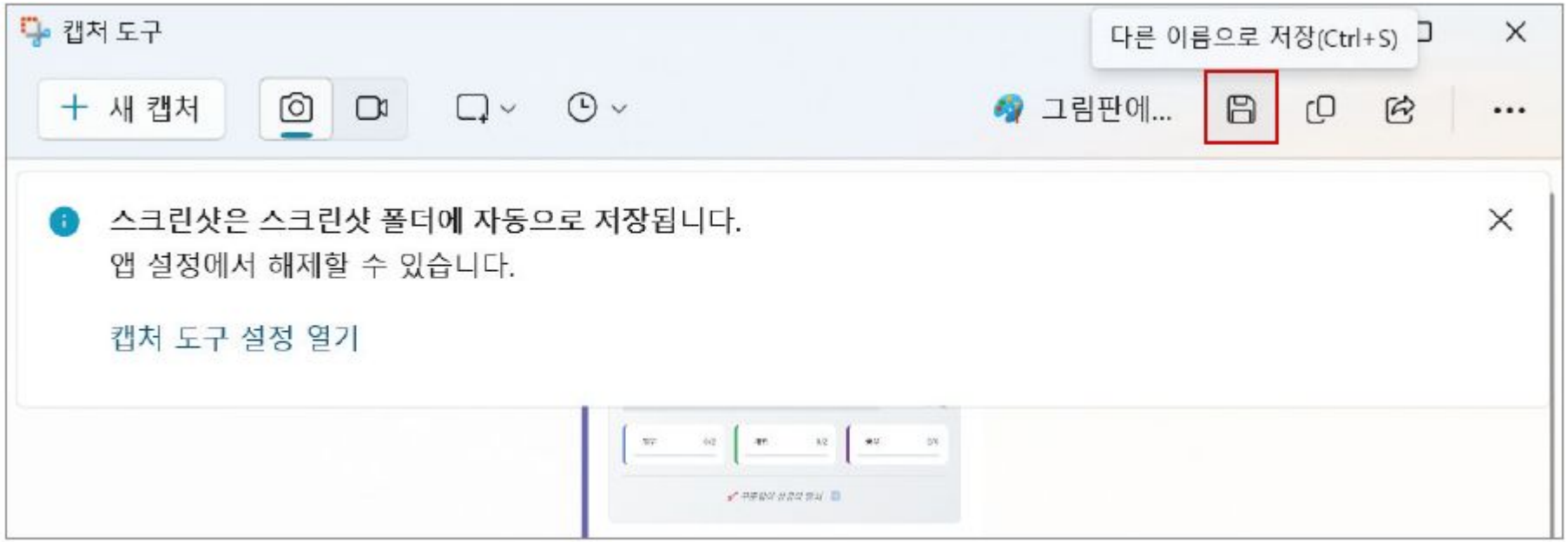
SECTION 4-3 프로젝트 개선과 작업 관리(5)

03 할 일 관리 앱을 실행하고 캡처 도구의 [직사각형] 옵션을 선택한 뒤, [+ 새 캡처] 버튼을 누르고 앱의 메인 화면을 드래그해서 캡처



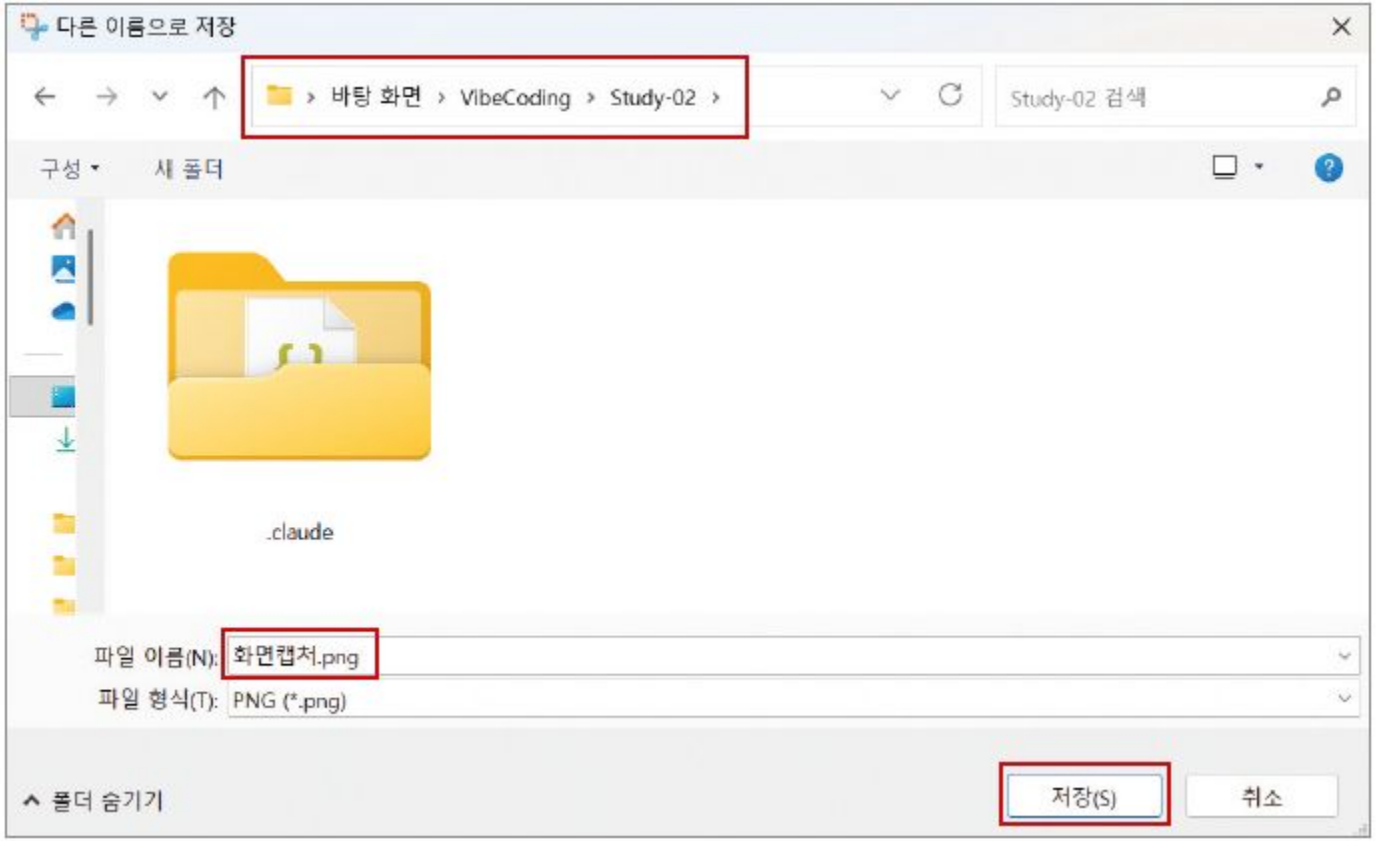
SECTION 4-3 프로젝트 개선과 작업 관리(6)

04[캡처 도구] 창에서 [다른 이름으로 저장( )] 아이콘을 눌러 이미지를 저장



SECTION 4-3 프로젝트 개선과 작업 관리(7)

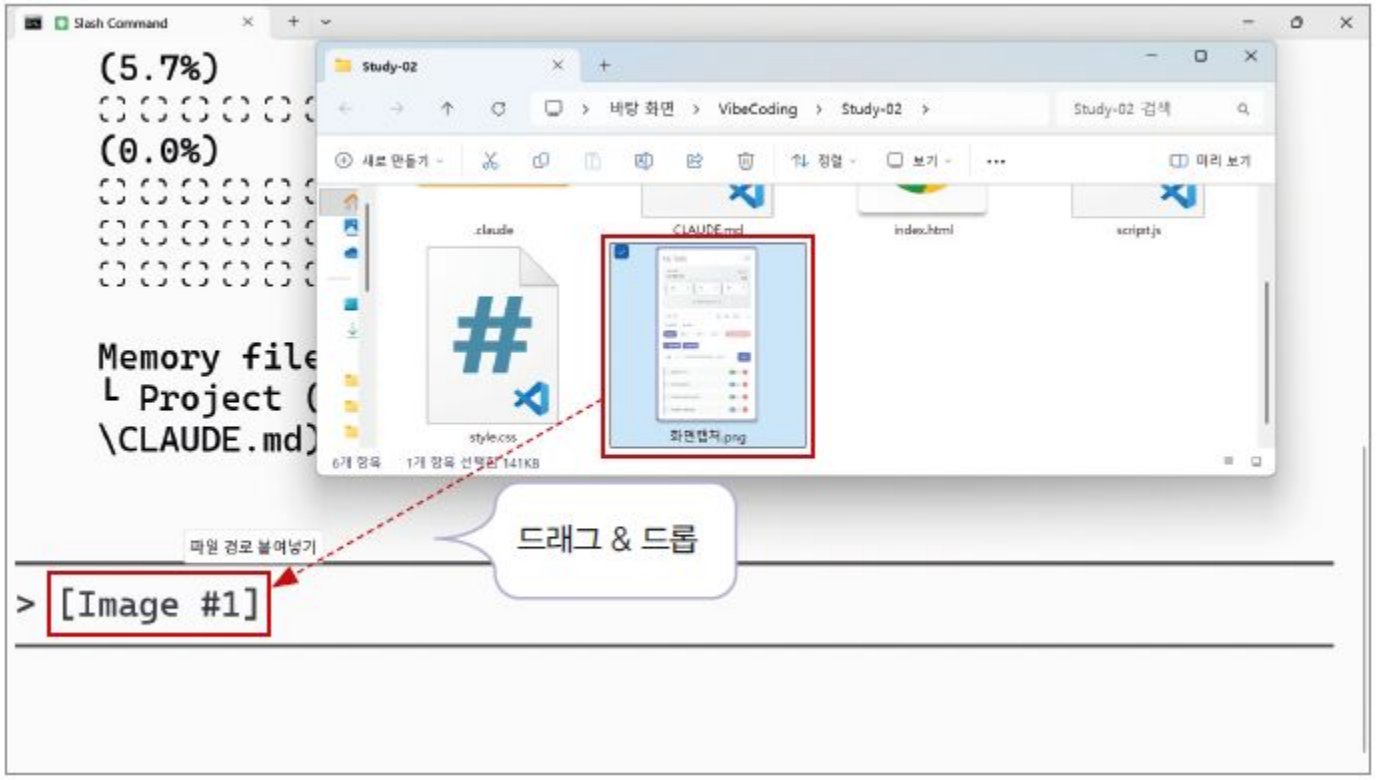
05 이미지를 'Study-02' 폴더에 '화면캡처.png'라는 이름으로 저장 - [저장] 버튼을 클릭



SECTION 4-3 프로젝트 개선과 작업 관리(8)

06 클로드 코드에 이미지를 입력하는 방법

- '화면캡처.png' 파일을 드래그해서 클로드 코드가 실행 중인 명령 프롬프트 창에 드롭
- 이미지가 정상적으로 입력되면 프롬프트 입력창에 '[Image #1]' 메시지가 표시



[note]

- 이미지의 파일 경로를 클로드 코드에 직접 입력해도 동일한 작업이 가능

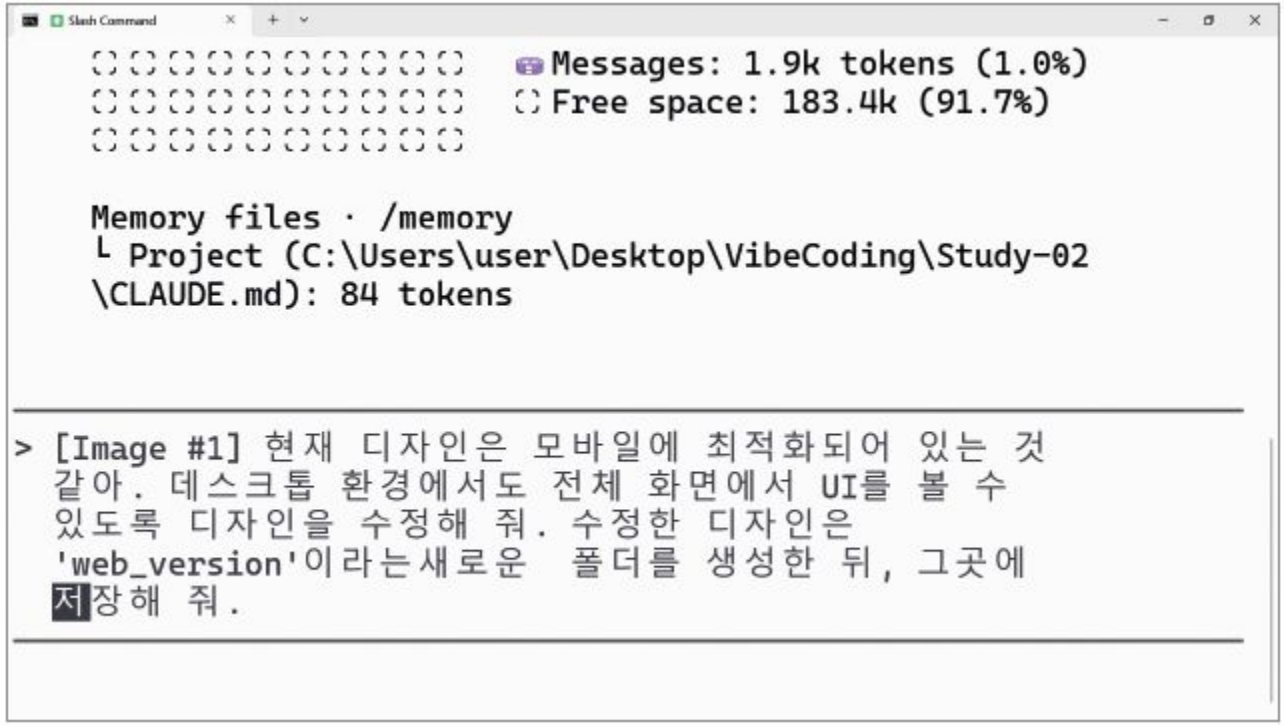
SECTION 4-3 프로젝트 개선과 작업 관리(9)

UI 개선하기

01 디자인 개선을 위해 요청 사항을 입력- 이미지를 전달한 입력창에 다음과 같이 프롬프트를 작성

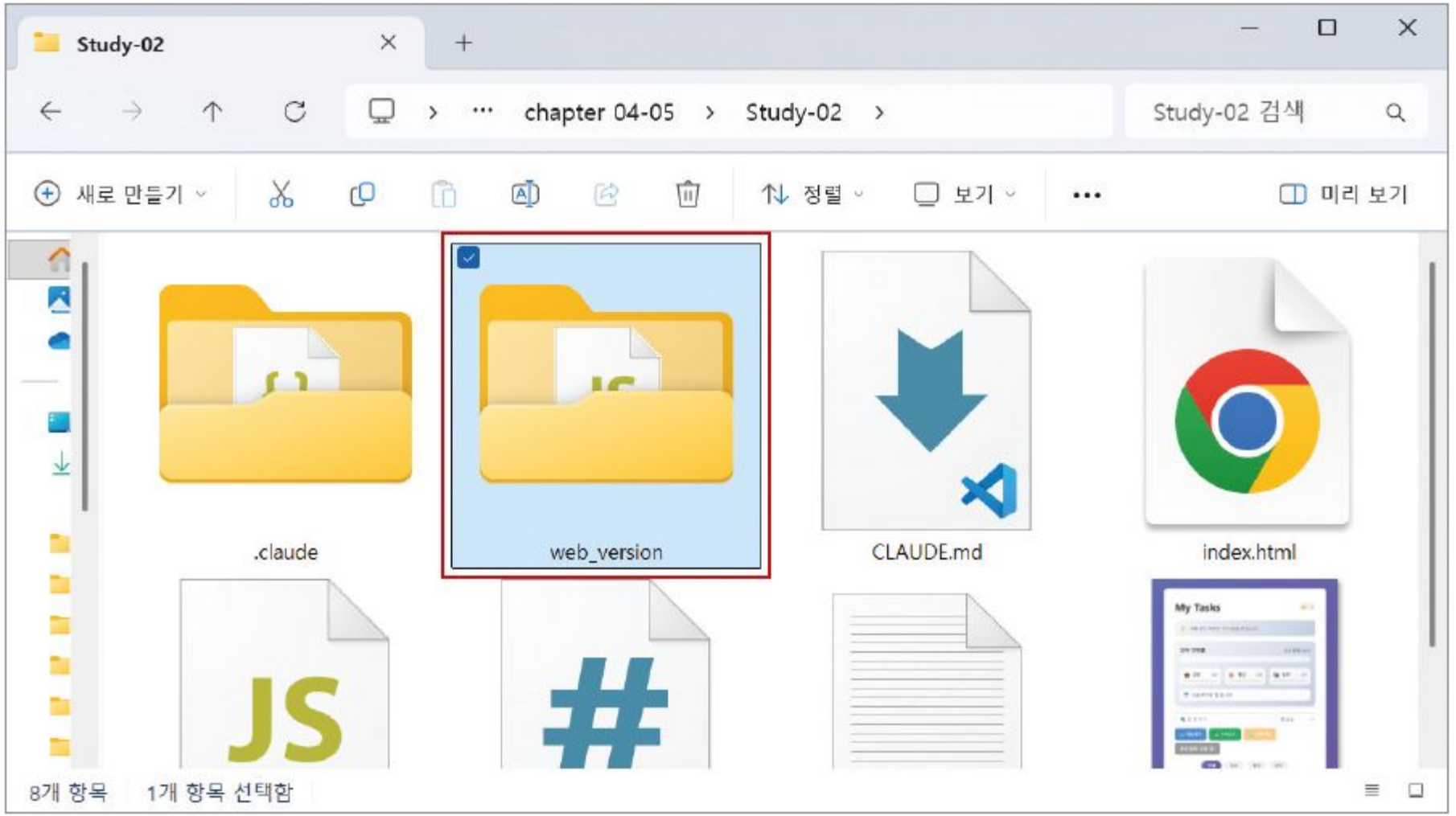
프롬프트

[Image #1] 현재 디자인은 모바일에 최적화되어 있는 것 같아. 데스크톱 환경에서도 전체 화면에서 UI를 볼 수 있도록 디자인을 수정해 줘. 수정한 디자인은 'web_version'이라는 새로운 폴더를 생성한 뒤, 그곳에 저장해 줘.



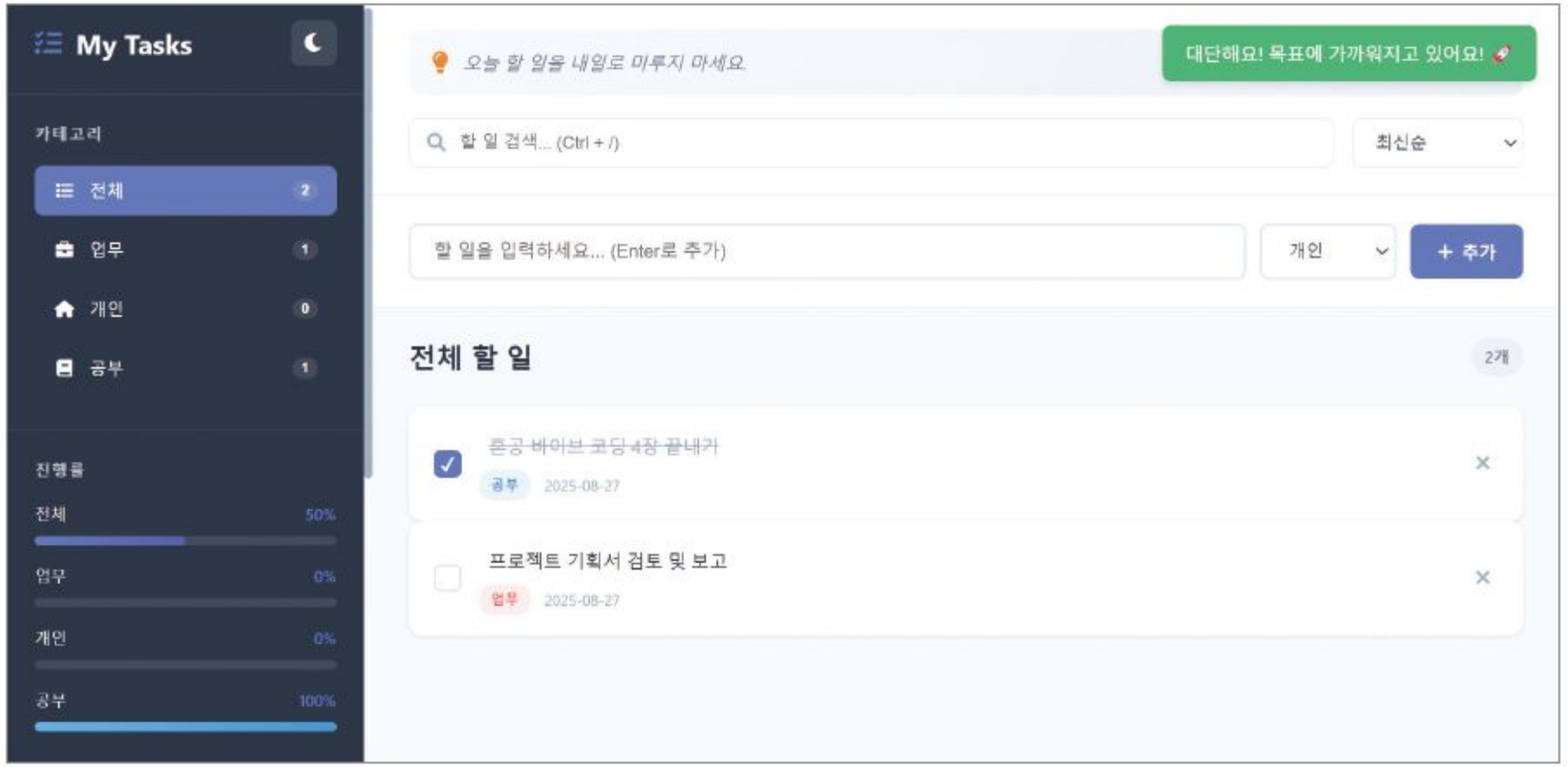
SECTION 4-3 프로젝트 개선과 작업 관리(10)

02 실행이 끝나면 실행 결과를 확인하라는 메시지가 나타남
'Study-02' 폴더에 'web_version' 폴더가 생성되었는지 확인



SECTION 4-3 프로젝트 개선과 작업 관리(11)

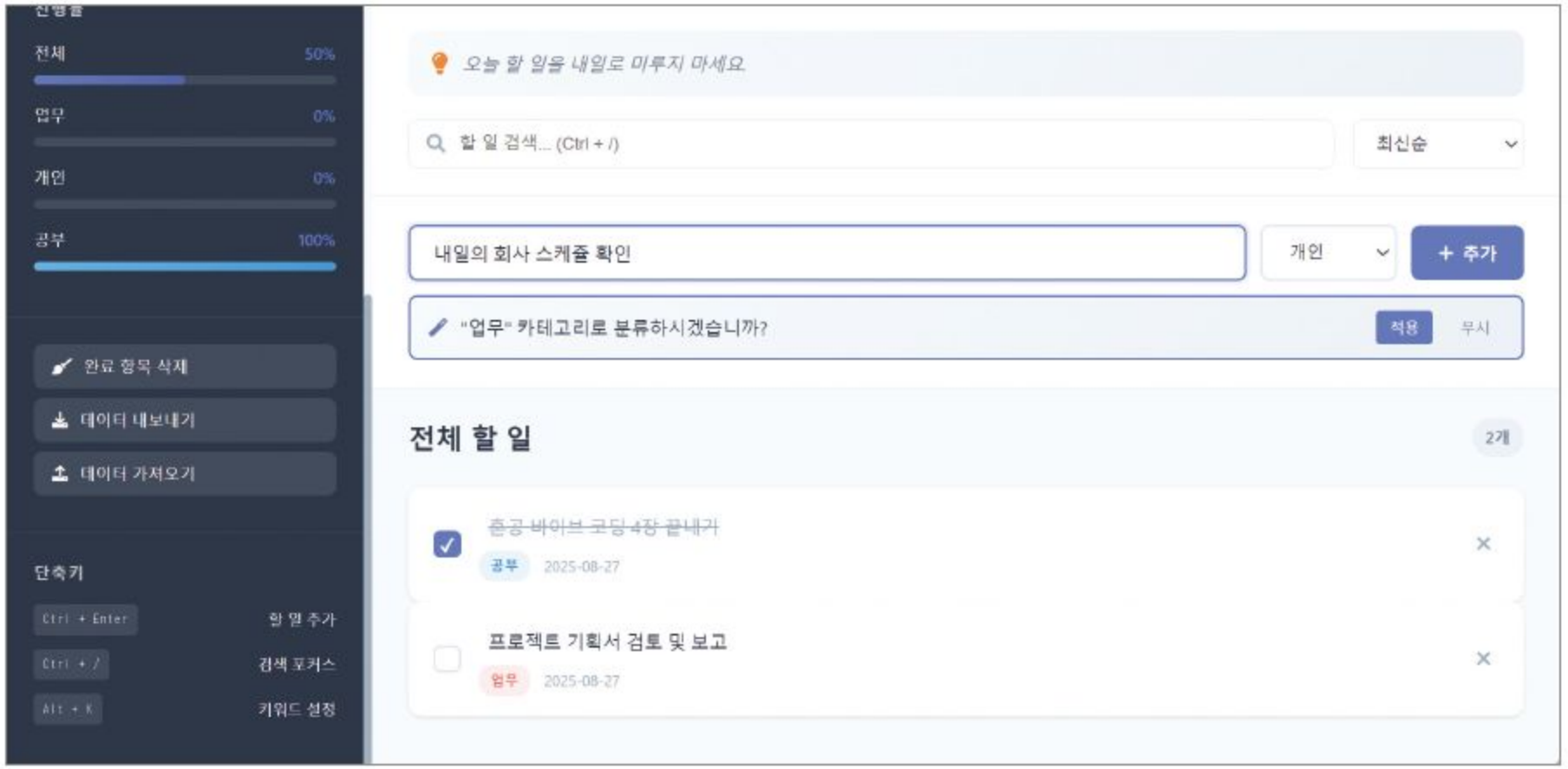
03 'web_version' 폴더의 'index.html' 파일을 더블 클릭하여 실행하면, 데스크톱 환경에 최적화된 할 일 관리 앱을 확인



SECTION 4-3 프로젝트 개선과 작업 관리(12)

04 모든 기능이 정상적으로 작동하는지 확인

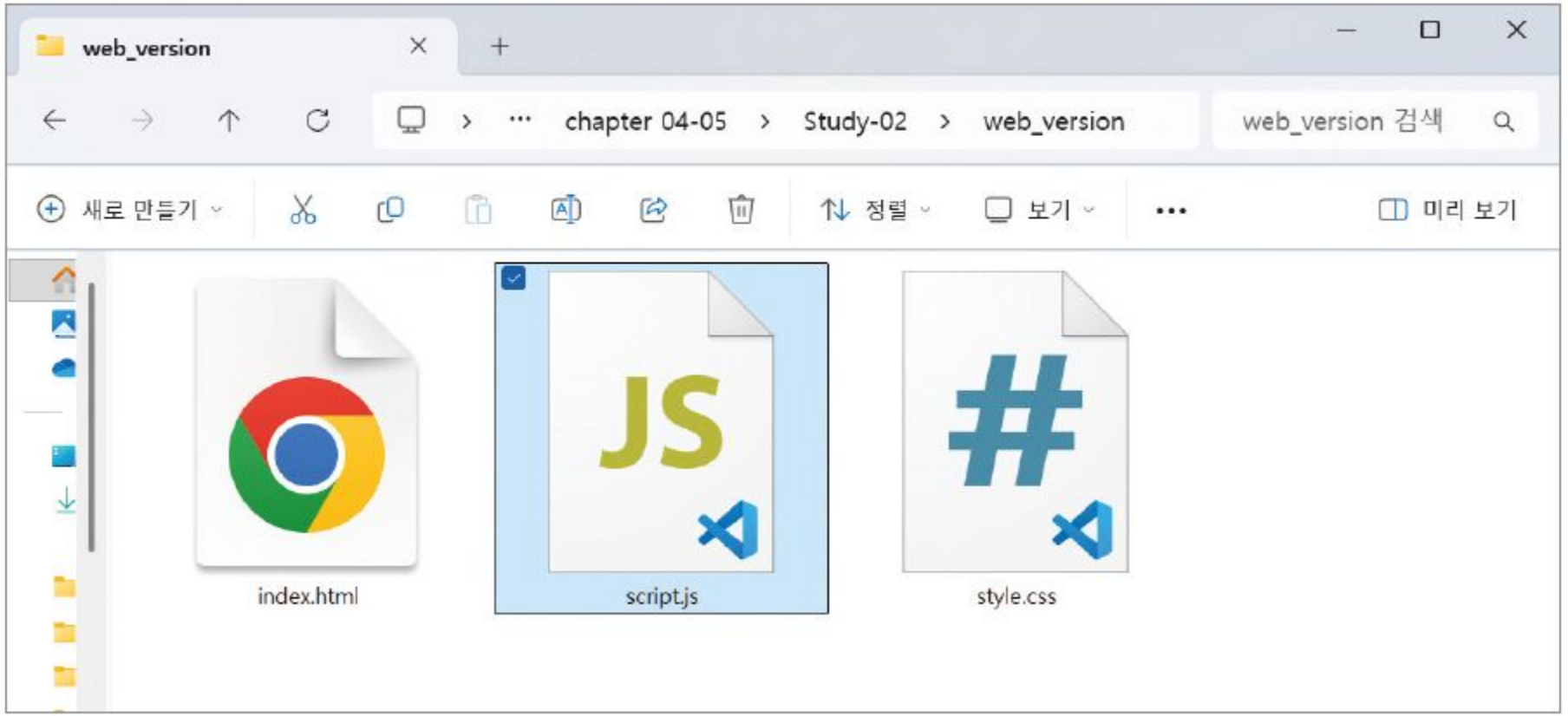
할 일 입력란에 '내일 회사 스케줄 확인'이라고 입력하면 자동으로 '업무' 카테고리로 분류



SECTION 4-3 프로젝트 개선과 작업 관리(13)

◦ 참조를 통해 코드 빠르게 점검하기

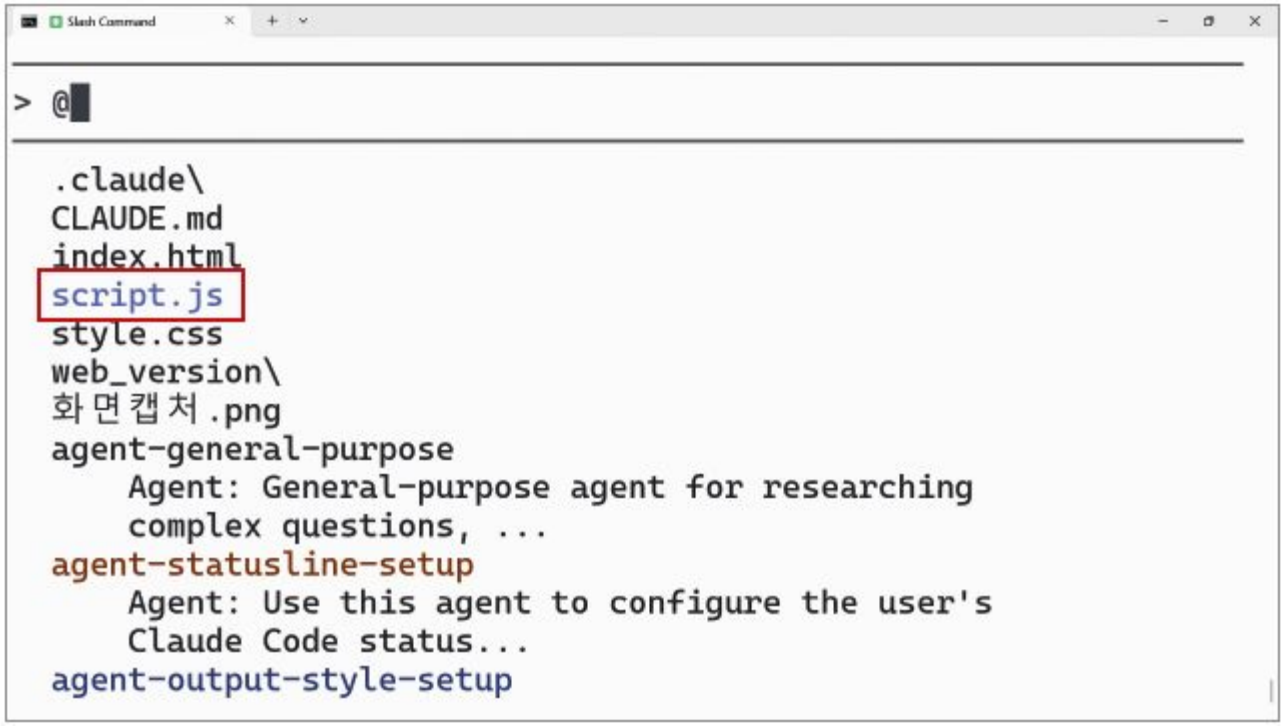
- @ 기호를 사용해 작업 폴더에 생성된 'script.js'를 불러와, 앞서 만든 할 일 관리 앱의 자바스크립트 코드를 분석하고 기능이 제대로 작동하는지 점검



SECTION 4-3 프로젝트 개선과 작업 관리(14)

@ 기호로 코드 분석하기

01 클로드 코드에 '@'를 입력하면 불러올 수 있는 파일 목록 중에서 'script.js' 파일을 선택



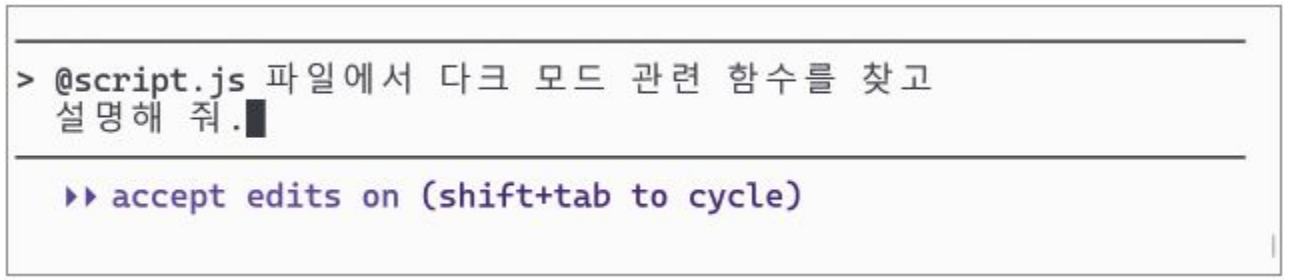
SECTION 4-3 프로젝트 개선과 작업 관리(15)

02 자바스크립트 파일에서 다크 모드 기능이 잘 작성되어 있는지 확인

다음과 같이 프롬프트를 입력 - 클로드 코드가 'script.js' 파일 전체를 읽고 즉시 분석

프롬프트

```
@script.js 파일에서 다크 모드 관련 함수를 찾고 설명해 줘.
```



SECTION 4-3 프로젝트 개선과 작업 관리(16)

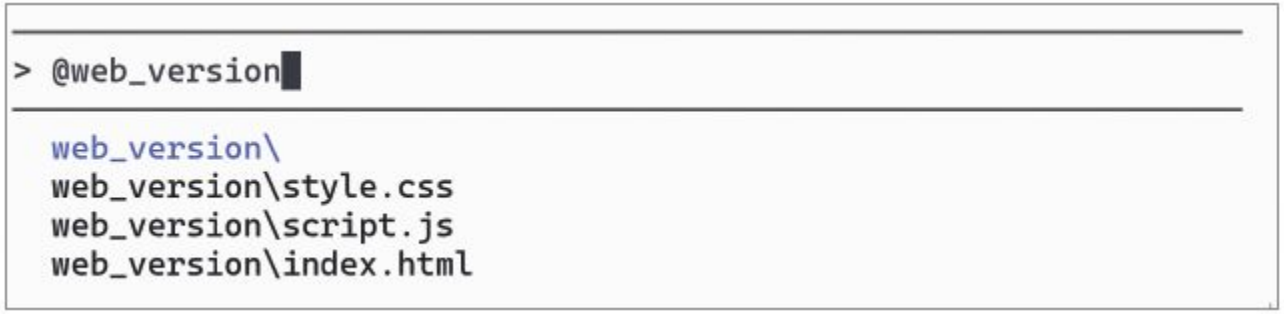
03 파일 분석이 끝나면 요청한 대로 함수의 이름과 기능, 위치를 알려 주고, 단축키와 같은 숨겨진 기능도 소개

SECTION 4-3 프로젝트 개선과 작업 관리(17)

@ 기호로 폴더 구조 확인하기

- 앞에서 만든 'web_version' 폴더를 확인

01 클로드 코드에 '@web_version'을 입력해서 'web_version' 폴더 전체를 불러옴
'web_version' 폴더에 들어 있는 하위 파일 리스트가 보임

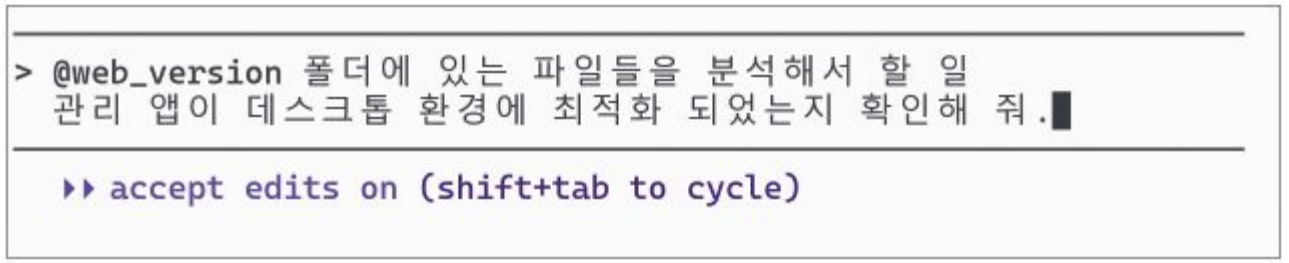


SECTION 4-3 프로젝트 개선과 작업 관리(18)

02 할 일 관리 앱이 데스크톱 환경에 최적화되었는지 확인하기 위해 다음과 같이 프롬프트를 입력

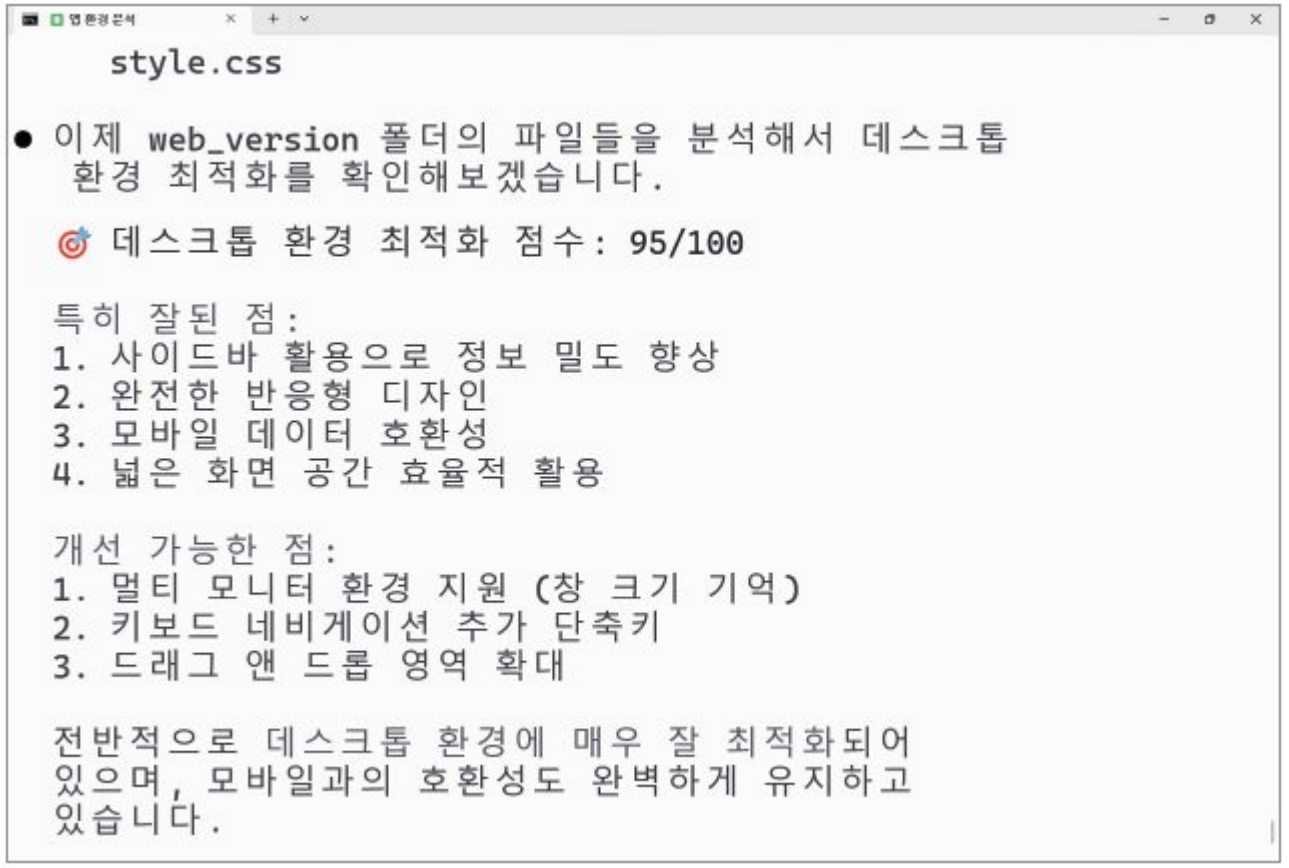
프롬프트

```
@web_version 폴더에 있는 파일들을 분석해서 할 일 관리 앱이 데스크톱 환경에 최적화되었는지 확인해 줘.
```



SECTION 4-3 프로젝트 개선과 작업 관리(19)

03 클로드 코드가 폴더 내 모든 파일을 읽고 종합 분석 결과를 보여줌
내용을 요약하면 대략 다음과 같은 분석 결과가 나타남

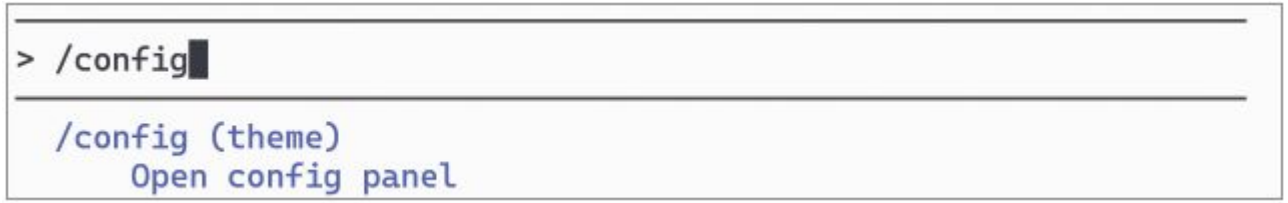


SECTION 4-3 프로젝트 개선과 작업 관리(20)

◦ 클로드 코드 프로젝트 관리하기

/config 명령어로 설정 확인하기

- /config 명령어는 프로그램이나 시스템의 설정(configuration)을 확인하거나 변경할 때 사용
  - 즉 프로젝트를 어떤 방식으로 다룰지에 대한 규칙을 관리하는 도구
  - 클로드 코드에서는 프로젝트에 적용된 현재 설정값, 예컨대 파일 저장 권한이나 출력 형식 등 정보를 확인할 수 있으며 특정 설정을 원하는 값으로 변경할 수도 있음



SECTION 4-3 프로젝트 개선과 작업 관리(21)

- 설정 항목 보기 - 클로드 코드에 '/config' 명령어를 입력한 뒤 [Enter]

설정 항목	기본값	설명
Auto-compact	TRUE	문맥이 일정 수준 차면 자동으로 /compact를 실행해 대화기록을 요약하고 유지합니다. false로 설정하면 수동으로만 압축할 수 있습니다.
Use todo list	TRUE	클로드 코드가 작업 내용을 할 일 목록 형식으로 관리해 단계별 진행 상황을 추적합니다.
Verbose output	FALSE	출력 결과를 장황하게(verbose) 보여줄지 여부를 결정합니다. TRUE로 설정하면 상세한 디버그 정보가 표시됩니다.
Auto-updates	TRUE	클로드 코드 자동 업데이트를 활성화합니다. 새 버전이 나오면 자동으로 다운로드 및 설치됩니다.
Theme	Light mode	터미널 테마를 설정합니다. Light mode와 Dark mode 중 선택 가능합니다.
Notifications	Auto	작업 완료 시 알림 방식을 설정합니다. Auto, Bell, None 중 선택할 수 있습니다.
Output style	default	클로드의 응답 스타일을 결정합니다. default는 표준 형식입니다.
Editor mode	normal	편집 모드를 설정합니다. normal 또는 vim 모드를 선택할 수 있습니다.
Model	Default (recommended)	사용할 AI 모델을 선택합니다. Default는 작업에 따라 자동으로 최적 모델을 선택합니다.
Auto-connect to IDE	FALSE	IDE와 자동 연결 여부를 설정합니다. VS Code 등과 자동으로 연동할지 결정합니다.

SECTION 4-3 프로젝트 개선과 작업 관리(22)

/doctor 명령어로 시스템 진단하기

- 클로드 코드에서 현재 프로젝트가 정상적으로 실행되는지 환경을 점검하고 문제를 진단

SECTION 4-3 프로젝트 개선과 작업 관리(23)

- 클로드 코드에 '/doctor' 명령어를 입력하고 [Enter]

화면 해설

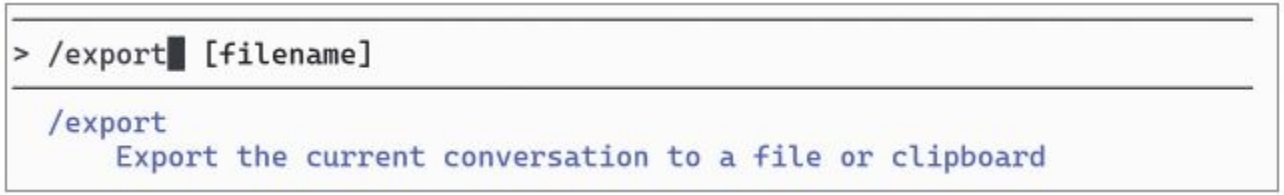
- 현재 실행 중: 현재 실행 중인 클로드 코드의 버전
- 경로: 노드 JS가 설치된 위치
- 호출된 위치: 클로드 코드 실행 파일이 실제로 호출(invoke)된 경로
- 자동 업데이트 가능 여부
- 업데이트 권한
- 코드 검색 기능 정상 작동 여부

```
> /doctor

 Diagnostics
 └ Currently running: npm-global (1.0.120)
 └ Path: C:\Program Files\nodejs\node.exe
 └ Invoked: C:\Users\user\AppData\Roaming\npm\node_modules\@anthropic-a
i\claude-code\cli.js
 └ Config install method: global
 └ Auto-updates enabled: default (true)
 └ Update permissions: Yes
 └ Search: OK (vendor)
```

SECTION 4-3 프로젝트 개선과 작업 관리(24)

/export 명령어로 대화 내보내기

- /export 명령어는 현재 작업 중인 프로젝트를 하나의 결과물로 묶어 파일로 내보냄
  - 클로드 코드에서 작업한 여러 소스 코드, 수정 기록 등을 파일로 보관하면 작업 환경을 옮기거나 협업자에게 공유할 때 좋은 참고 자료로 사용할 수 있음
  - 핵심 대화를 추출해 다른 프로젝트의 CLAUDE.md 파일에 추가할 수도 있음

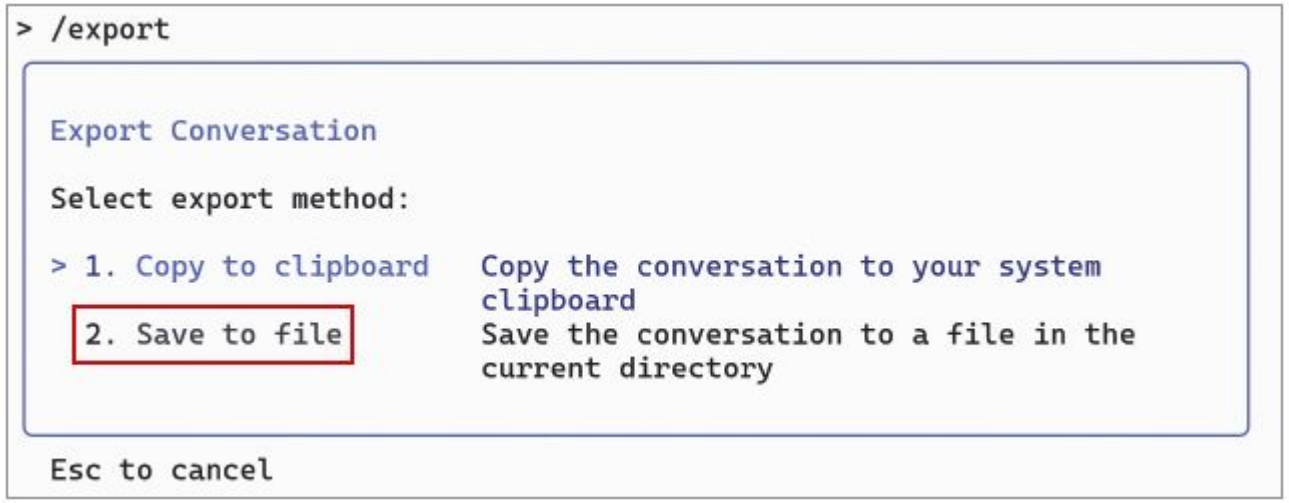


SECTION 4-3 프로젝트 개선과 작업 관리(25)

01 클로드 코드에 '/export' 명령어를 입력하고 [Enter]
두 가지 옵션을 선택할 수 있음 - '2. Save to file'을 선택하고 [Enter]

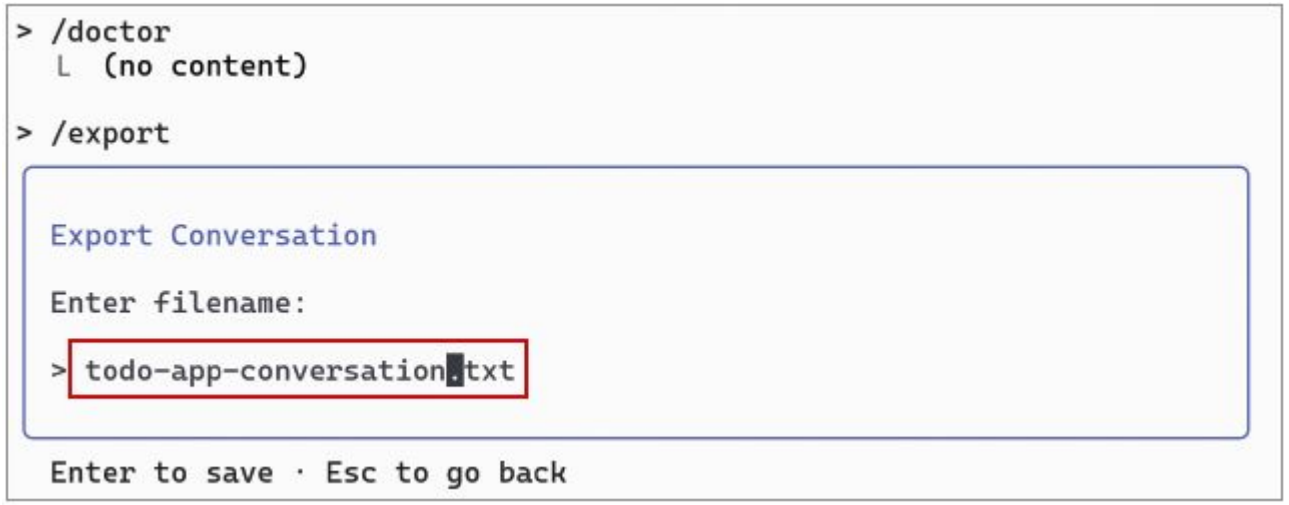
화면 해설

1. 클립보드로 복사: 전체 대화를 복사하여 붙여 넣을 수 있습니다.
2. 파일로 저장: 대화 내용이 파일로 저장됩니다.



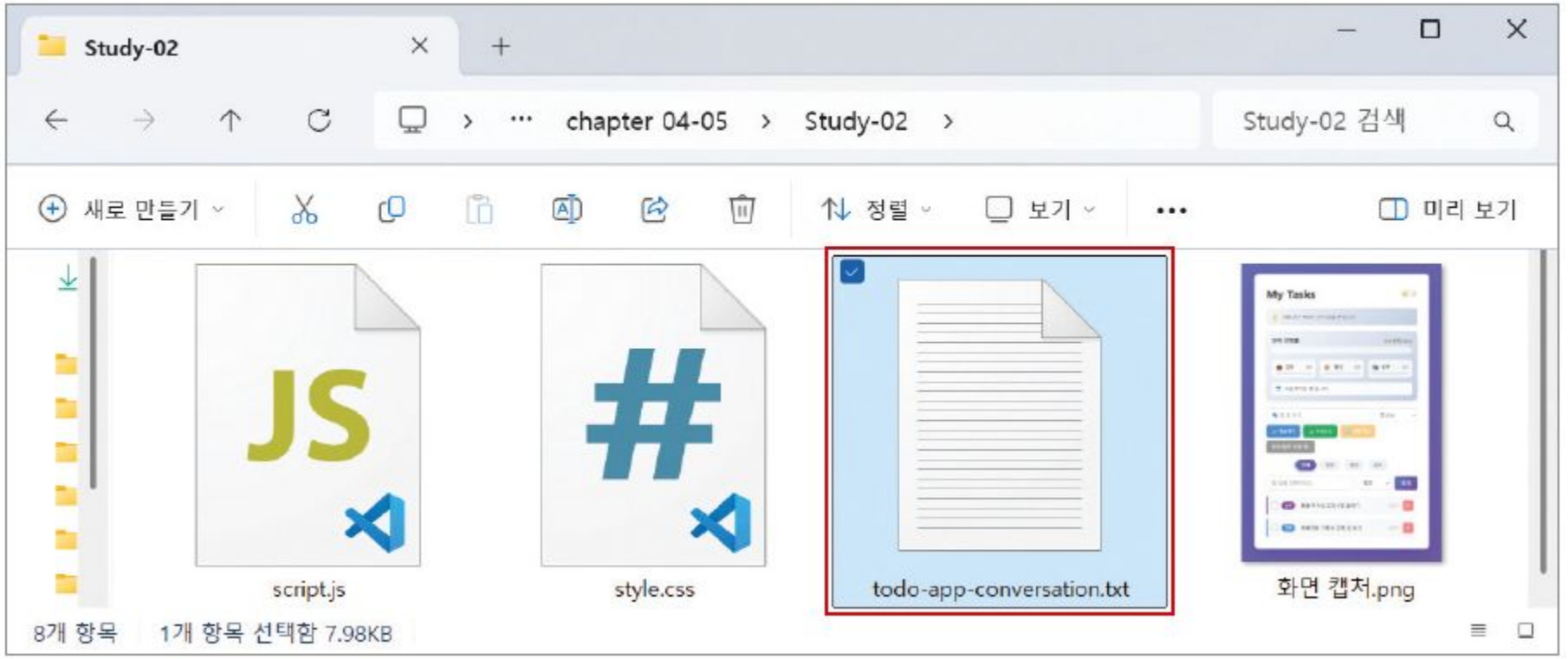
SECTION 4-3 프로젝트 개선과 작업 관리(26)

02 파일 이름을 'todo-app-conversation.txt'로 수정하고 [Enter] 키를 눌러 저장



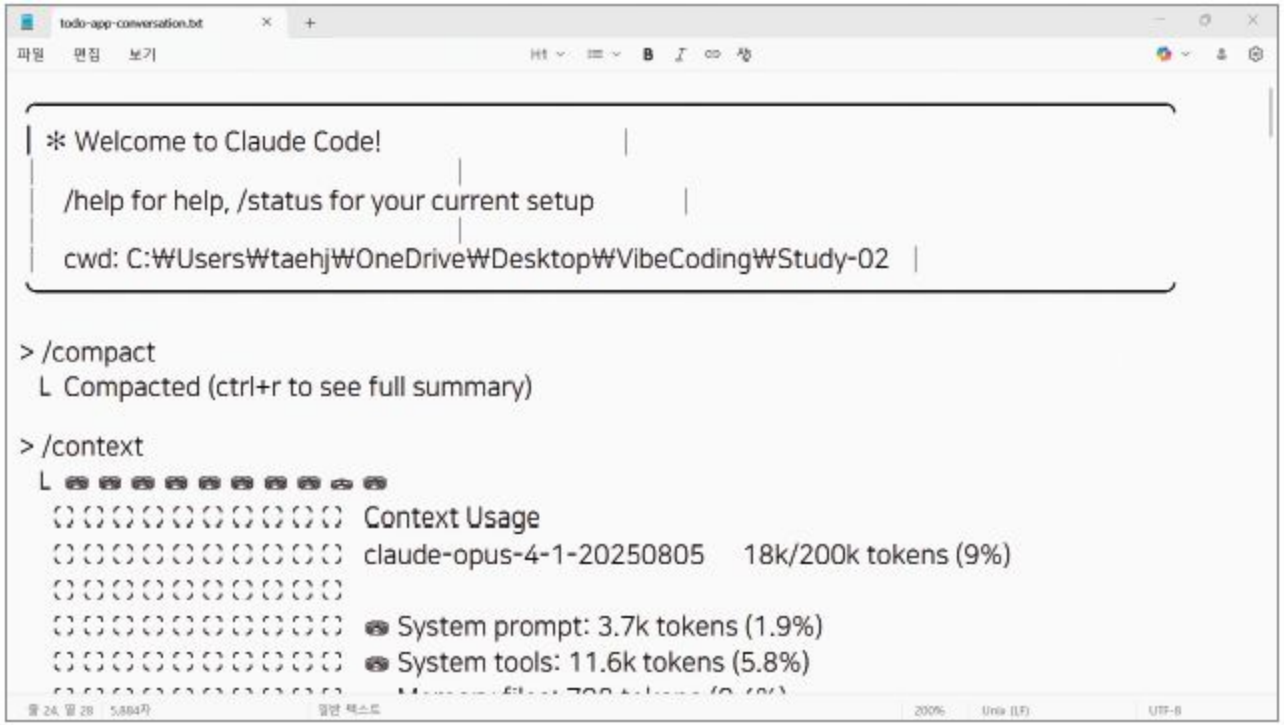
SECTION 4-3 프로젝트 개선과 작업 관리(27)

03 데스크톱의 'Study-02' 폴더에 'todo-app-conversation.txt' 파일이 저장



SECTION 4-3 프로젝트 개선과 작업 관리(28)

04 메모장을 사용해서 'todo-app-conversation.txt' 파일 열기
지금까지 클로드 코드에서 작업한 대화 내용이 기록되어 있음을 확인



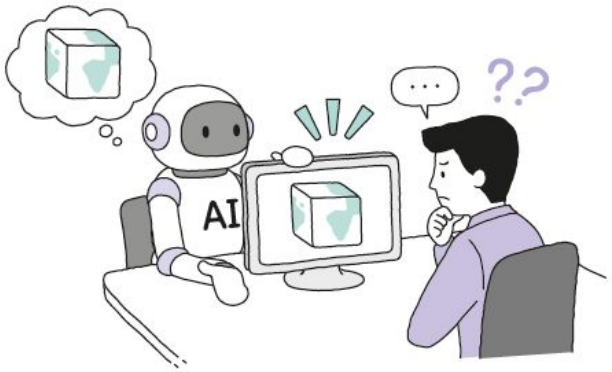
[마무리]

- 5가지 키워드로 정리하는 핵심 포인트
  - 화면을 직접 **캡처**해 클로드 코드에 전달하면 장황한 설명 없이도 요구사항을 명확하고 빠르게 전달
  - **@** 기호를 사용하면 파일 및 폴더를 즉시 참조하고 분석
  - **/config** 명령어는 클로드 코드의 프로젝트 환경 설정을 관리
  - **/doctor** 명령어는 시스템 상태를 진단해서 버전, 설치 방식, 업데이트 권한 등을 점검
  - **/export** 명령어는 작업 기록을 클립보드나 파일에 저장하고 문서화

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(1)

시작하기전에

- 할루시네이션(hallucination, 환각)
  - 생성형 AI가 사실과 다른 내용을 진짜처럼 만들어내는 현상
  - (원인)
    - AI가 학습 데이터에서 패턴을 찾아 이어 붙여 확률적으로 가장 그럴듯한 문장을 생성
    - 불명확한 정보로 인한 추론 오류
- AI와 함께 일할 때는 할루시네이션 현상을 인식하고, 안전하게 검증하는 습관이 필요
  - AI가 생성한 정보는 신뢰할 수 있는 문서 또는 공식 사이트와 비교해 교차 검증하거나, AI에게 '출처 없는 정보는 생성하지 말 것'과 같은 규칙을 입력해서 사전에 방지



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(2)

◦ 학습 내용

- 클로드와 클로드 코드를 활용해서 4지선다 상식 퀴즈 게임 만들기
  - 한국사, 과학, 지리, 예술과 문화 네 가지 분야에서 문제를 내고, 문제를 풀 때마다 정답 여부를 확인
  - 최종적으로는 점수와 순위가 기록

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(3)

- 1단계: 퀴즈 게임 구현하기
  - 클로드에서 게임 제작을 위한 PRD를 작성하고 프롬프트를 준비한 뒤, 클로드 코드를 실행해서 게임을 제작
  - AI와 함께 제작한 게임이 기획 의도대로 잘 진행되는지 테스트
  - 프로젝트 메모리에 검증 가이드라인을 저장하여 AI가 생성하는 콘텐츠의 정확성을 높이고, 대화 컨텍스트를 초기화하여 깨끗한 환경에서 테스트하는 방법을 학습

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(4)

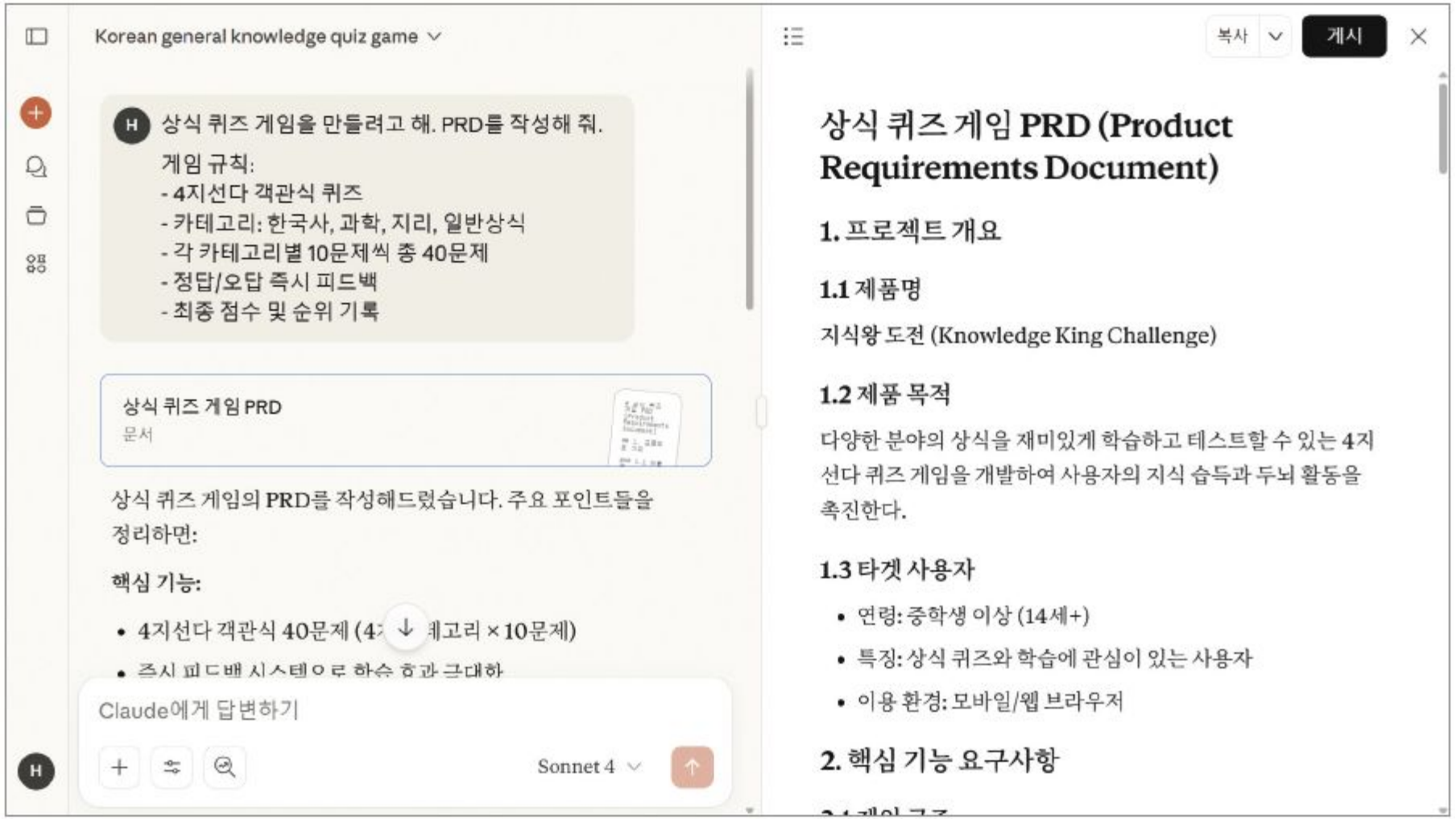
미니 게임을 위한 PRD 작성하기

01 클로드(claude.ai)에 접속하여 상식 퀴즈 게임의 PRD를 작성
다음과 같이 프롬프트를 입력 - 아티팩트에 PRD를 작성

프롬프트

```
상식 퀴즈 게임을 만들려고 해. PRD를 작성해 줘.

게임 규칙:
- 4지선다 객관식 퀴즈
- 카테고리: 한국사, 과학, 지리, 일반상식
- 각 카테고리별 10문제씩 총 40문제
- 정답/오답 즉시 피드백
- 최종 점수 및 순위 기록
```

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(5)



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(6)

02 PRD를 클로드 코드용 프롬프트로 변환하고, 3단계 구현 가이드를 확인

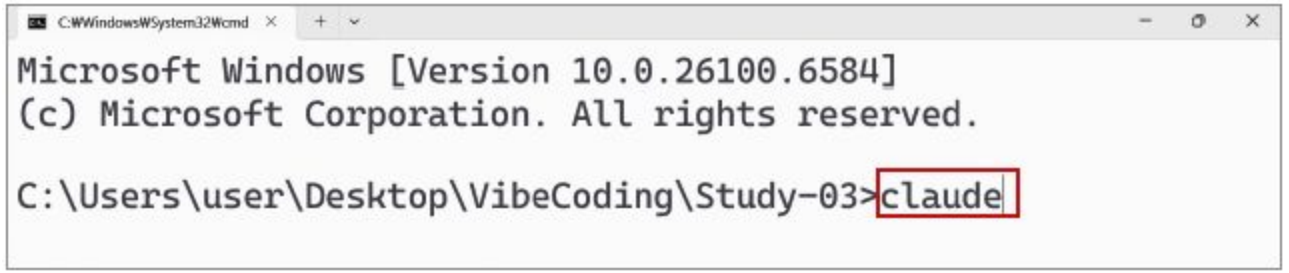
프롬프트

```
상식 퀴즈 게임을 클로드 코드로 구현할 수 있도록 PRD를 바탕으로 3단계 프롬프트를 정리해 줘.
```

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(7)

퀴즈 게임 구현하기

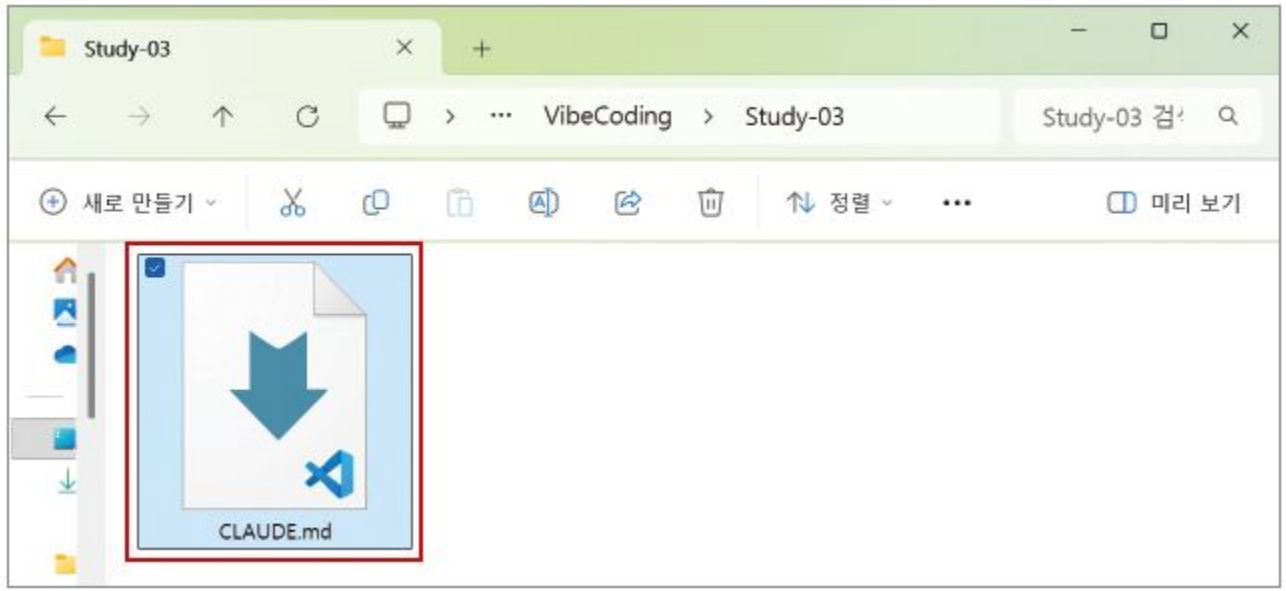
01 데스크톱의 'VibeCoding' 폴더에 'Study-03' 폴더를 생성하고, 마우스 오른쪽 버튼을 누른 뒤 [터미널에서 열기]를 클릭, 명령 프롬프트에 'claude'를 입력해 클로드 코드를 실행



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(8)

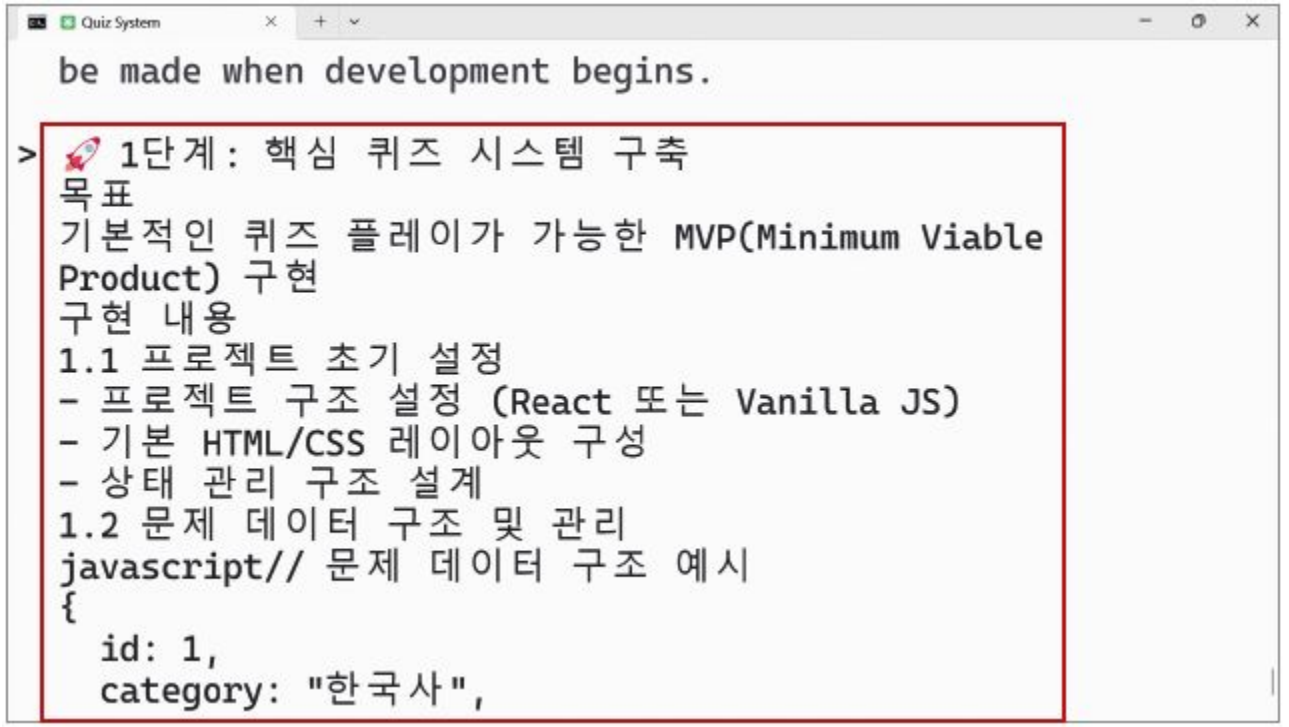
02 '/init' 명령어를 입력해서 프로젝트를 초기화
'Study-03' 폴더에 'CLAUDE.md' 파일이 생성

```
/init
```



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(9)

03 본격적으로 미니 게임을 구현 - 클로드 코드에 1단계 프롬프트를 입력

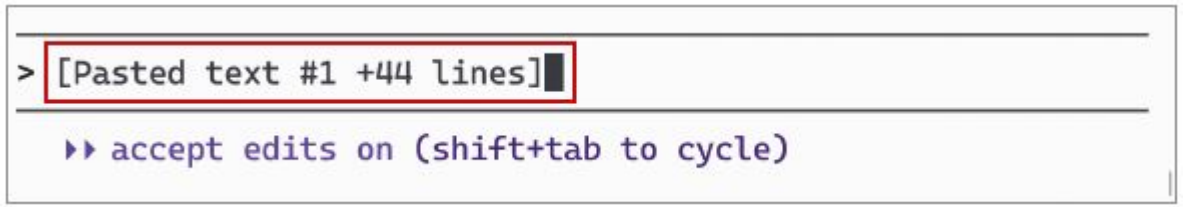


[note] 클로드 코드 웹에서 복사한 뒤 붙여 넣기

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(10)

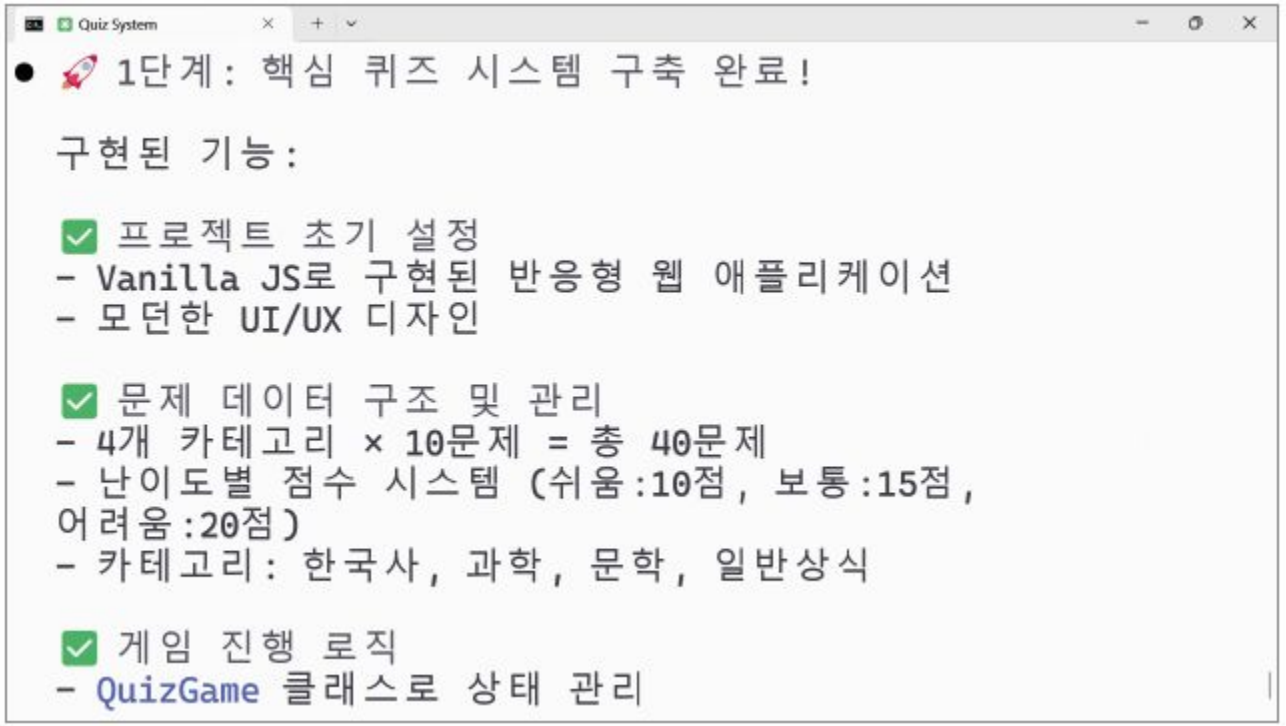
✚ 여기서 잠깐 **붙여 넣은 프롬프트가 너무 길 때**

- 클로드가 제시한 프롬프트를 복사해서 클로드 코드에 붙여 넣을 때, 프롬프트가 너무 길면 **'[Pasted text #1 +44 lines]'**와 같이 내용이 요약되어 나타남
- **[Enter]** 키를 눌러 실행하면 정상적으로 진행



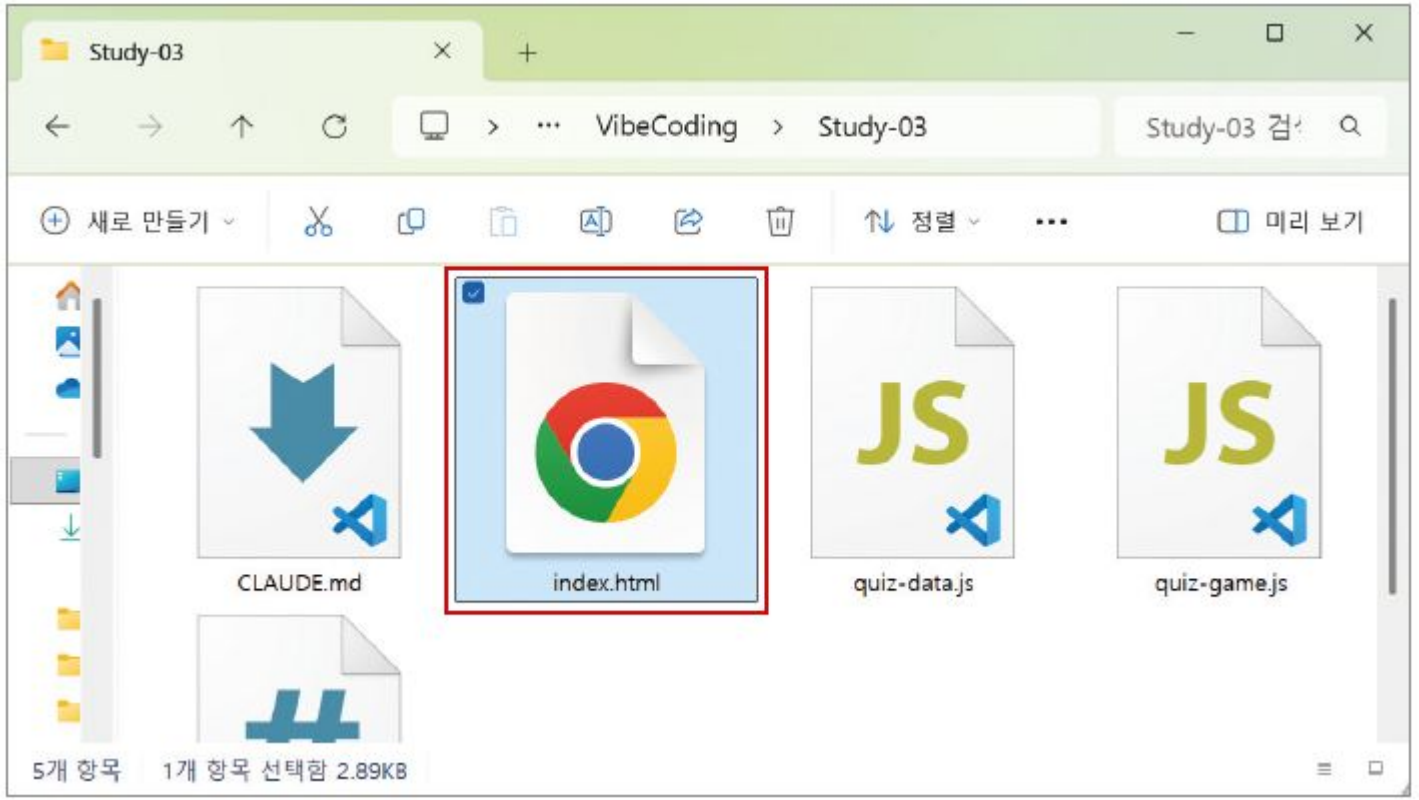
SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(11)

04 퀴즈 시스템이 테스트 가능한 상태로 구현



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(12)

05 'Study-03' 폴더에 새로운 파일 'index.html'이 생성



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(13)

게임 테스트

- 제작한 상식 퀴즈 게임이 정상적으로 작동하는지 확인
  - 네 가지 카테고리의 문제들이 순차적으로 출제되는지
  - 정답과 오답 판정이 정확한지
  - 피드백이 즉시 표시되는지
  - 게임 완료 시 결과가 표시되는지를 테스트

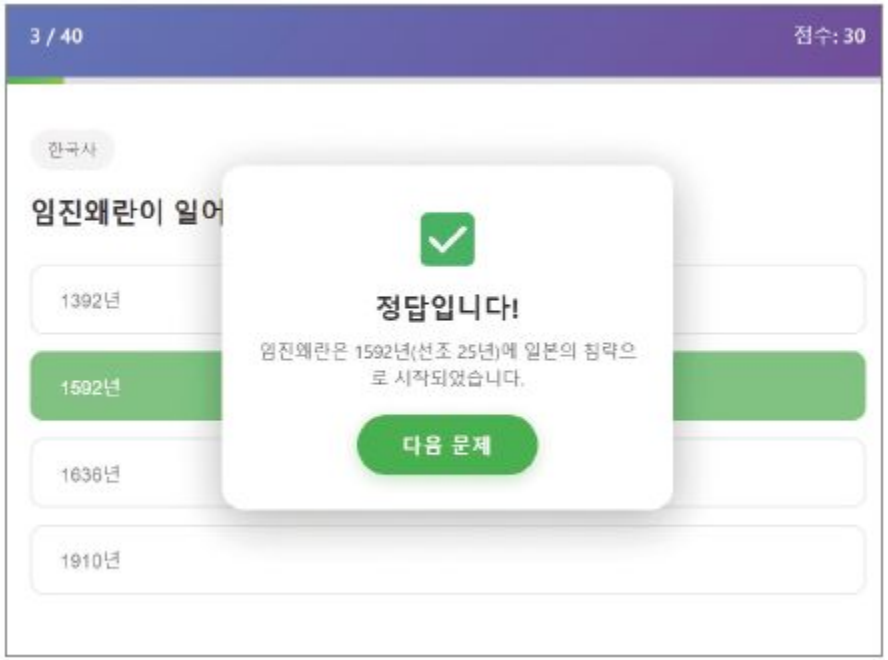
SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(14)

01 모든 기능이 의도대로 작동하는지 확인

'Study-03' 폴더에서 'index.html' 파일을 더블 클릭해 웹 브라우저에서 상식 퀴즈 게임을 실행하고 [게임 시작] 버튼을 클릭한 뒤, 문제 풀어 보기

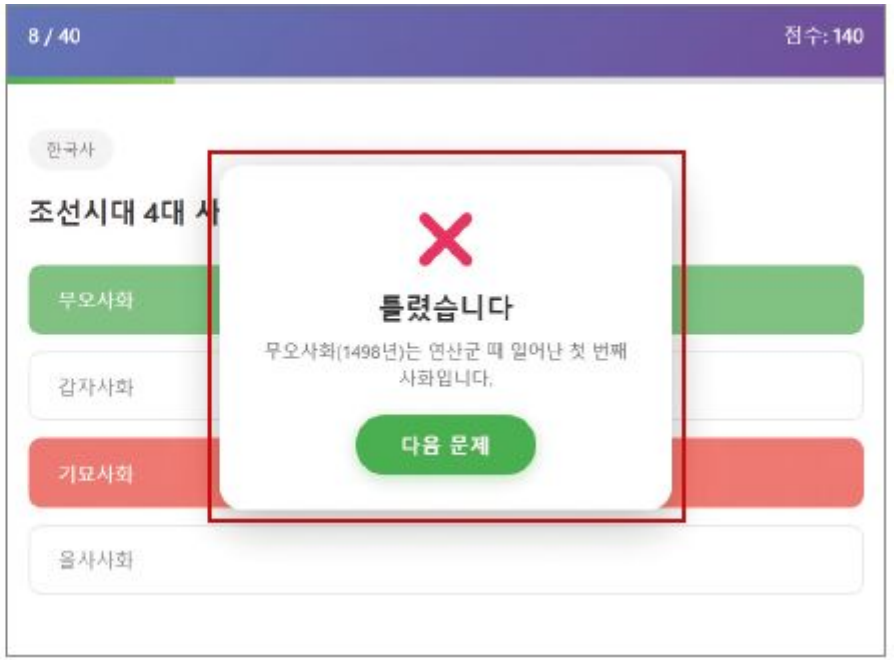
SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(15)

02 정답/오답 판정이 정확한지 확인하기 위해 문제를 풀어 보기



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(16)

03 정답 여부를 판정하고, 결과를 즉시 피드백으로 제공



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(17)

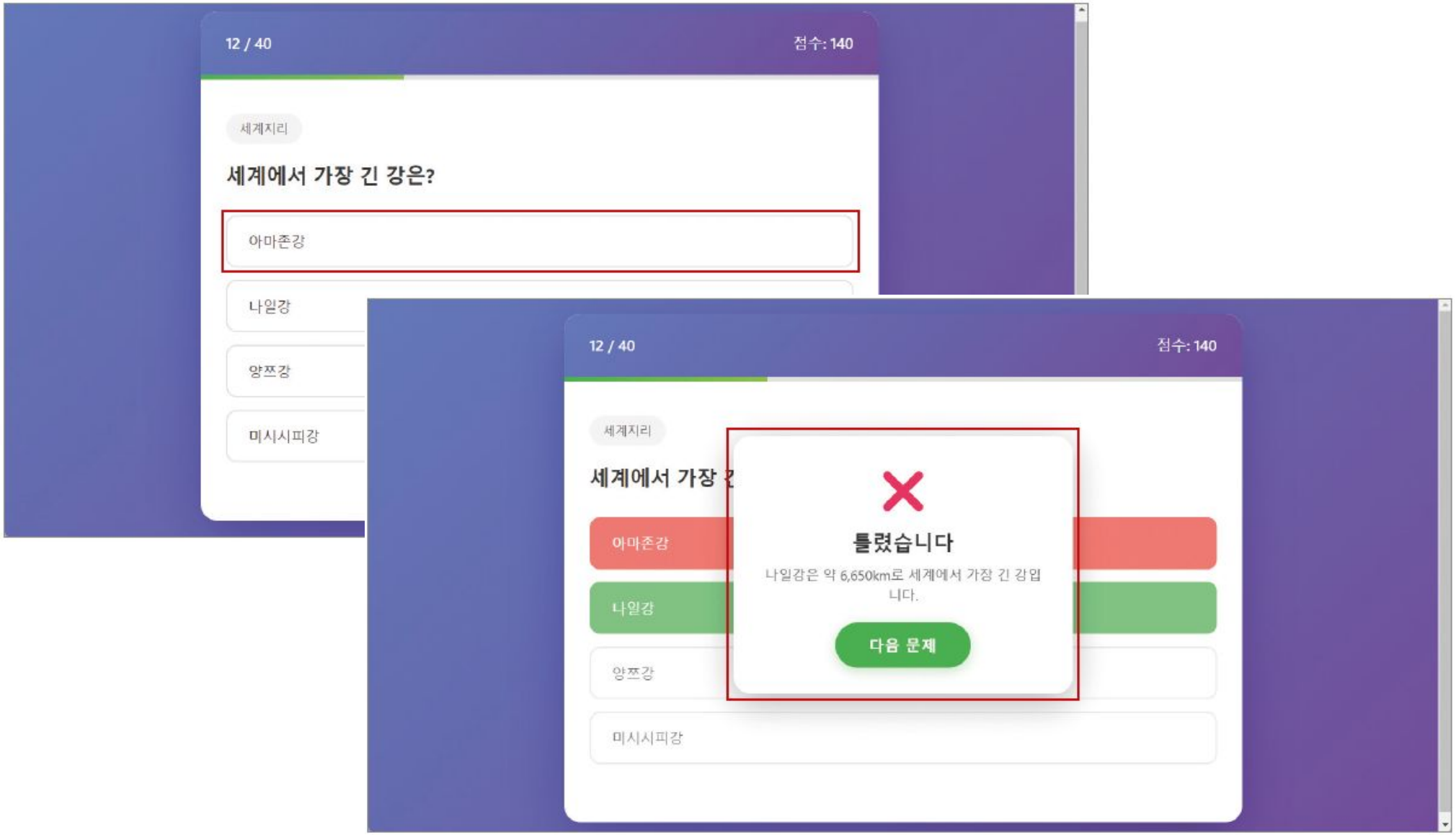
04 모든 퀴즈를 풀면 최종 점수를 보여줌

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(18)

◦ 할루시네이션 해결하기

- AI 기반 콘텐츠의 개발 과정은 할루시네이션 현상이 일어나지 않도록 검증하는 것이 중요
  - 특히 정보의 정확성이 중요한 퀴즈 게임이라면 더더욱 중요함
  - 지금 만들어진 문제들 중 명백한 오류는 없어 보이지만, 애매한 경우가 있을 수 있음
- '세계에서 가장 긴 강'이 무엇인지 묻는 퀴즈
  - '아마존강'을 선택해 틀림 - 정답은 나일강
  - 그런데 최근 연구에서 아마존강의 수원지를 재측정한 결과, 아마존강이 나일강보다 더 길다는 주장도 제기
- 이처럼 논란의 여지가 있는 주제는 문제 제작 시 교차 검증을 거쳐 명확한 기준이 필요
  - 논란 있는 정보의 검증을 위해 검증 가이드라인을 마련하고, 이를 CLAUDE.md 파일에 영구적으로 저장하는 것이 가장 효과적

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(19)



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(20)

✚ 여기서 잠깐 | 프로젝트 라이브 데모

- 생성형 AI의 출력은 매번 임의적으로 달라지기 때문에, 직접 실습할 때는 책에 나온 결과와 같은 오류가 재현되지 않을 수도 있음
- 이번 장에서는 AI가 논란의 여지가 있거나 잘못된 정보를 만들어내는 현상을 이해하고 대처하기 위해, 의도적으로 논란의 여지가 있는 문제가 포함된 프로젝트의 라이브 데모를 제공
- 해당 자료는 책의 깃허브 저장소(https://github.com/taehojo/vibecoding) 하단의 [Chapter 5: 상식 퀴즈 게임]-[Study-03-basic(기본): 다운로드(실습 참조용)] 폴더에서 내려받을 수 있음
  - 이 파일을 참고하면 AI가 잘못[illegible]하며 학습

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(21)

교차 검증 가이드라인 구축하기

- /memory 명령어는 AI가 기억하고 있는 프로젝트 관련 정보를 확인하고 관리
  - 클로드가 지금까지 기억하고 있는 내용을 확인하고 필요 시 수정하거나 삭제할 수 있는 기능
- 앞서 3장에서 '#' 기호를 사용해 CLAUDE.md에 짧은 규칙을 즉시 추가하는 방법을 학습
  - 하지만 보다 체계적으로 가이드라인을 편집하려면 /memory 명령어를 활용하는 것이 좋음
- 지금부터 /memory 명령어를 사용해 검증 규칙을 문서화하여 'CLAUDE.md'에 저장

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(22)

01 'CLAUDE.md' 파일을 수정하기 위해, 클로드 코드에 '/memory' 명령어를 입력

```
/memory
```

```
> /memory

  /memory
      Edit Claude memory files ← 클로드의 메모리 파일을 수정합니다.
```

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(23)

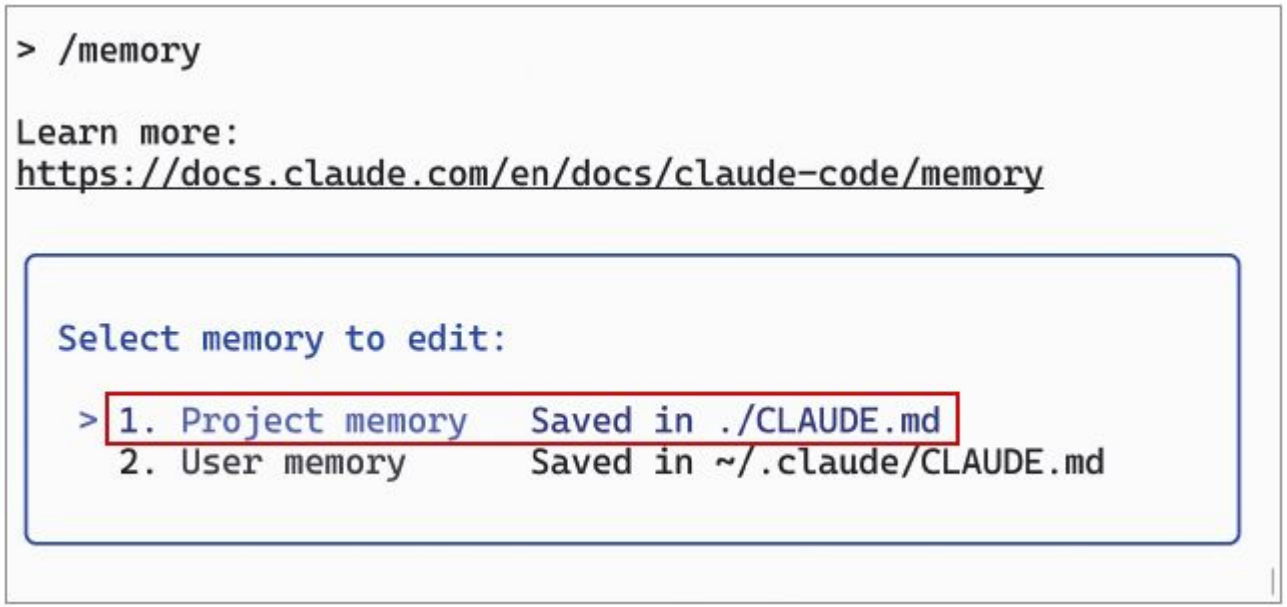
02 메모리 파일을 저장할 위치를 지정

퀴즈 게임의 검증 가이드라인은 현재 프로젝트에만 해당하므로 '1. Project memory'를 선택

화면 해설

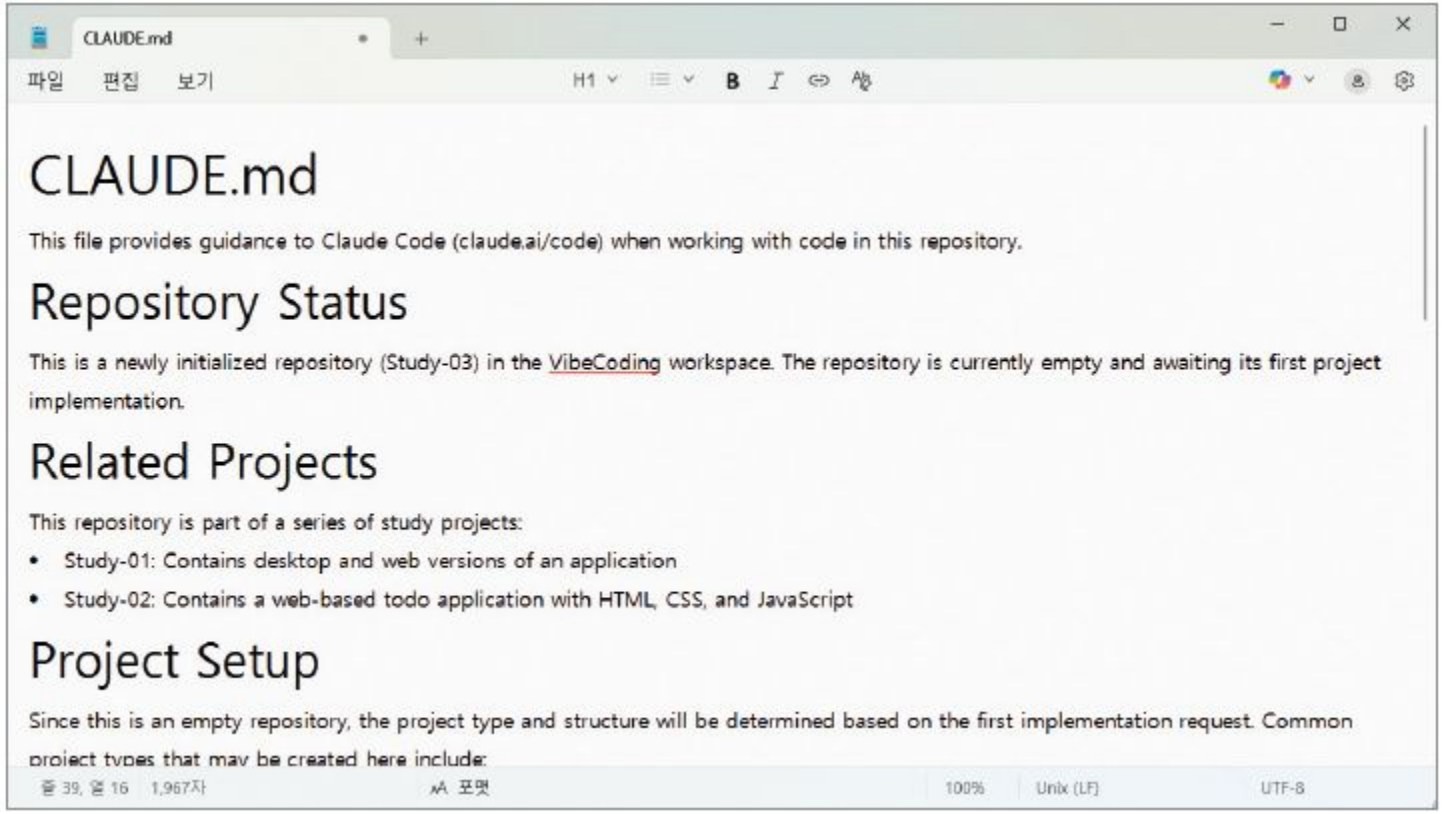
메모리 저장 위치를 고르세요:

1. 프로젝트 메모리: 현재 프로젝트 폴더 안에서만 적용되는 규칙 저장
2. 사용자 메모리: 모든 프로젝트에 공통적으로 적용되는 사용자 전역 규칙 저장



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(24)

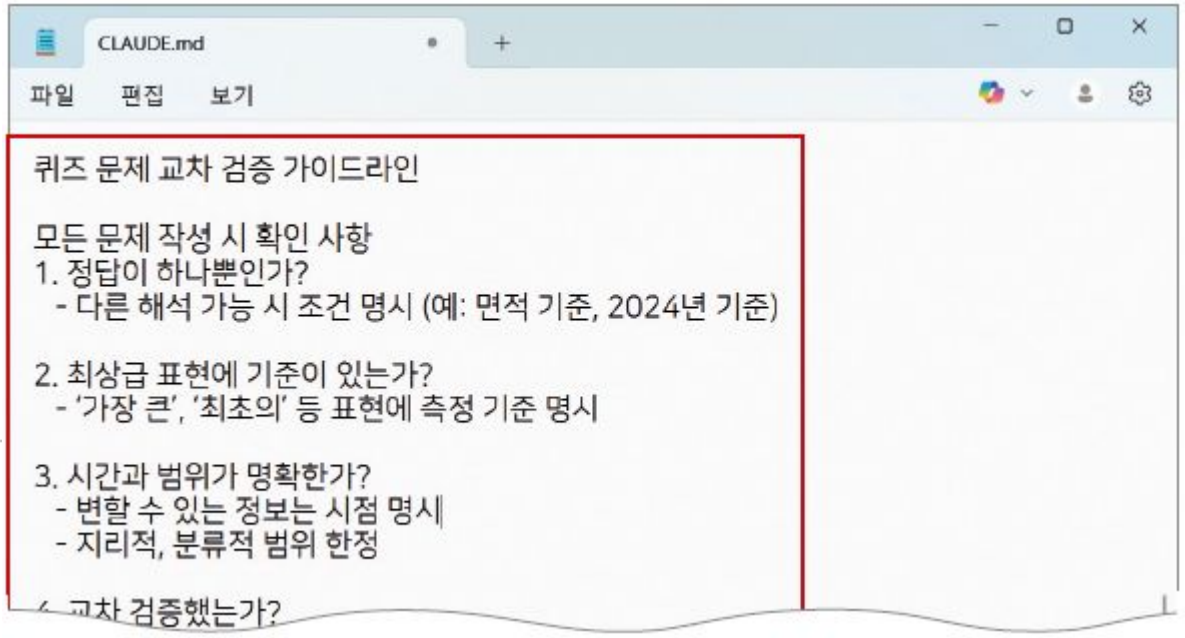
03 현재 프로젝트의 CLAUDE.md 파일이 열림



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(25)

04 'CLAUDE.md' 파일의 마지막 행에 다음과 같이 교차 검증 가이드라인을 추가

```
퀴즈 문제 교차 검증 가이드라인

모든 문제 작성 시 확인 사항
1. 정답이 하나뿐인가?
   - 다른 해석 가능 시 조건 명시 (예: 면적 기준, 2024년 기준)
2. 최상급 표현에 기준이 있는가?
   - '가장 큰', '최초의' 등 표현에 측정 기준 명시
3. 시간과 범위가 명확한가?
   - 변할 수 있는 정보는 시점 명시
   - 지리적, 분류적 범위 한정
4. 교차 검증했는가?
   - 의심스러운 정보는 2개 이상 출처 확인
   - 논란 있는 내용은 주류 학설 기준
```



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(26)

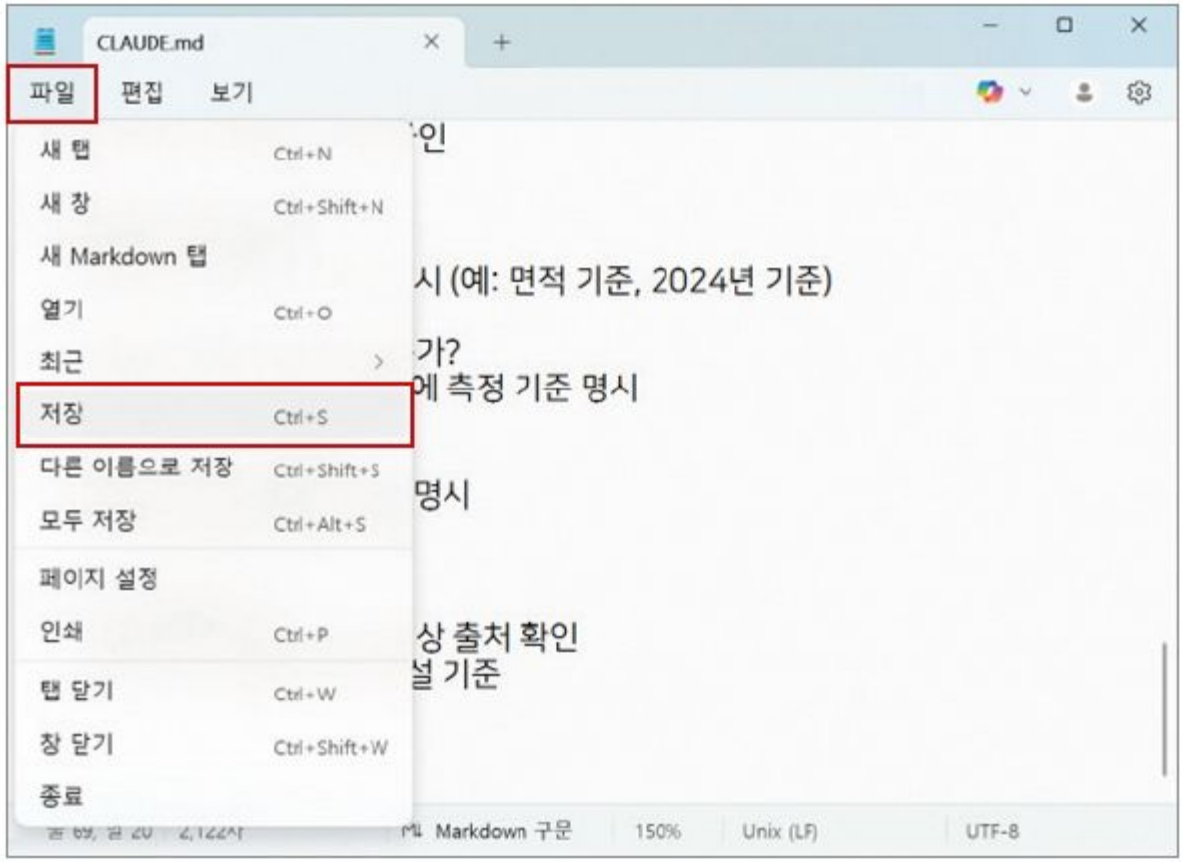
✚ 여기서 잠깐 **CLAUDE.md 작성법**

- **CLAUDE.md**에는 형식이 정해져 있지 않으므로, 간결하고 사람이 읽기 쉽게 작성하는 것이 핵심
- 또한 클로드의 프롬프트 일부로 작동하기 때문에 자주 쓰는 프롬프트처럼 지속적으로 다듬고 개선하는 것이 좋음.
- 읽기 쉬운 자연어로 간결하게 작성
  - 예: # 프로젝트 개요
    - 이름: **My Web App**
    - 목표: 간단한 블로그 서비스 구현
    - 언어: **Python 3.12, JavaScript(React)**

- 중요한 규칙은 '중요', '반드시' 같은 강조 표현을 추가해 AI가 더 잘 따르도록 할 수 있음
  - 예: # 주의사항
    - **settings.json**에 실제 **API** 키 입력 금지 → 반드시 **.env** 파일 사용
    - 이미지 파일은 **/assets/images** 폴더에만 저장

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(27)

05 [파일]-[저장([Ctrl]+[S])] 메뉴를 클릭해 CLAUDE.md 파일을 저장하고, 텍스트 에디터를 닫음



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(28)

06 클로드 코드로 돌아오면 CLAUDE.md 파일이 열렸었다는 메시지를 볼 수 있음

화면 해설

```
> /memory 실행 결과
  └ 열린 메모리 파일: ./CLAUDE.md
    > 다른 편집기를 사용하려면 $EDITOR 또는 $VISUAL 환경 변수를 설정하세요
```

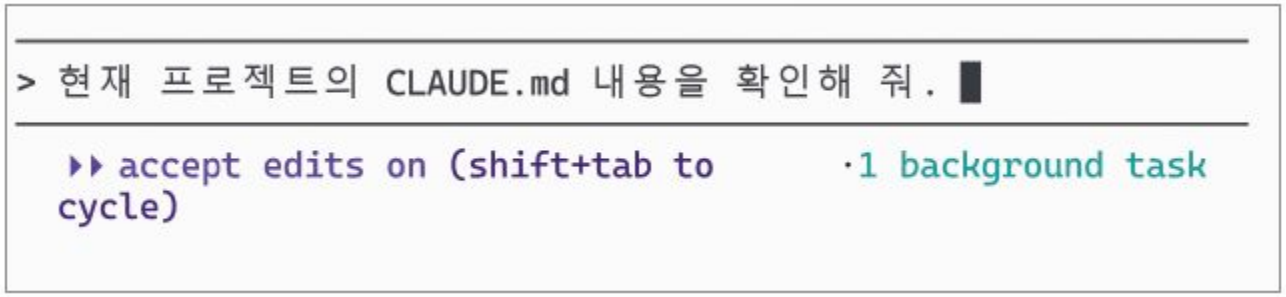
```
> /memory
  └ Opened memory file at ./CLAUDE.md

    > To use a different editor, set the $EDITOR or $VISUAL
    environment variable.
```

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(29)

07 수정된 사항이 반영되었는지 확인하기 위해 다음과 같이 프롬프트를 입력

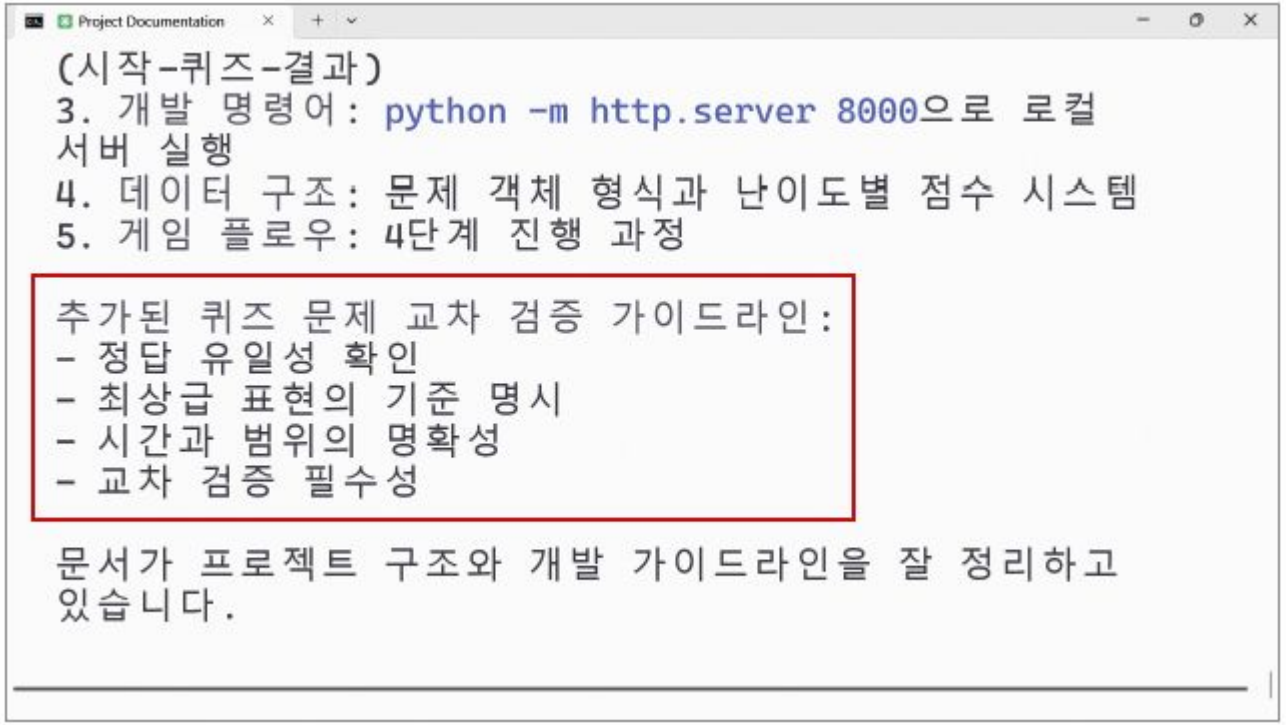
프롬프트

```
현재 프로젝트의 CLAUDE.md 내용을 확인해 줘.
```



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(30)

08 방금 추가한 가이드라인을 포함한 CLAUDE.md의 전체 내용을 확인



SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(31)

컨텍스트 초기화하기

- /clear 명령어는 현재 세션의 대화와 작업 컨텍스트를 모두 지우고 초기화
  - 대화 기록이 너무 길어져서 작업에 불필요한 컨텍스트가 개입할 때, 이전 프로젝트의 흔적을 지우고 완전히 새로운 작업을 시작할 때, 테스트 도중 잘못된 컨텍스트 때문에 혼선이 있을 때 사용
- 새롭게 추가된 가이드라인대로 클로드 코드가 작동하는지 테스트
  - 이전 대화에서 생성했던 문제로부터 영향을 받을 수 있음
  - 중요한 규칙은 CLAUDE.md 파일에 저장해 두었으므로, /clear 명령어를 활용해 지금까지 대화 내용과 작업 정보를 모두 지우고 상식 퀴즈 게임을 재실행

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(32)

✚ 여기서 잠깐 **/clear와 /compact의 차이**

- /clear와 마찬가지로 앞서 배운 /compact도 컨텍스트를 정리하고 최적화하는 기능을 수행
- 하지만, 대화를 요약하여 유지하는 /compact와 달리 /clear는 대화를 완전히 초기화한다는 차이가 있음

/clear와 /compact의 차이

구분	/clear	/compact
기본 동작	대화 기록을 완전히 초기화	대화 내용을 요약하여 압축
컨텍스트 윈도우	완전히 비움	요약된 형태로 유지
CLAUDE.md 파일	유지됨	유지됨
사용 시점	· 새 작업/기능 시작 · 완전히 다른 주제로 전환 · 잘못된 방향으로 진행된 대화 리셋	· 컨텍스트 창이 가득 찰 때 · 같은 작업 계속하면서 최적화 필요 · 토큰 사용량 줄이기

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(33)

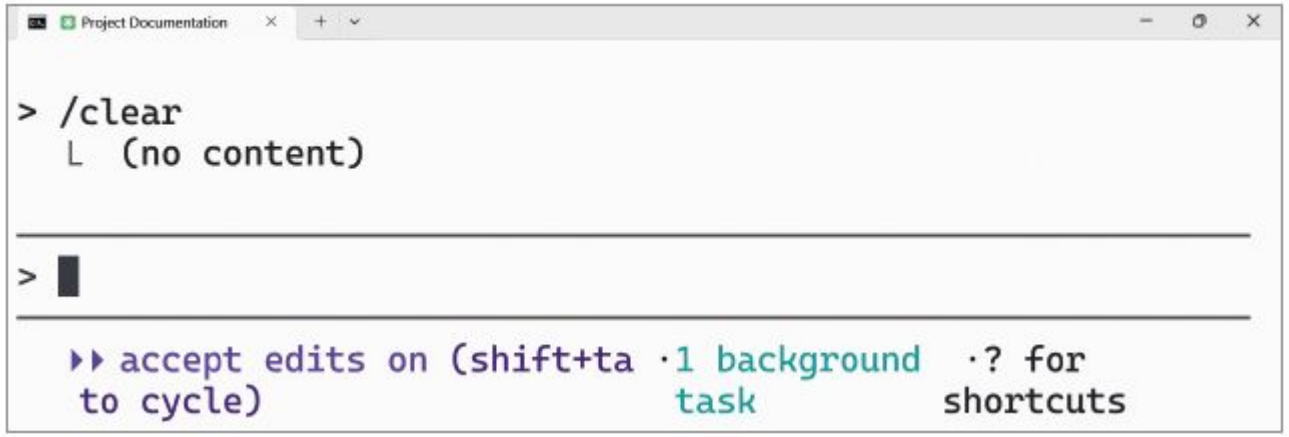
01 작업 내용을 초기화하기 위해 클로드 코드에 '/clear'를 입력

```
/clear
```

```
> /clear

  /clear (reset, new)
      Clear conversation history and free up context
```

← 대화 기록을 삭제하고 컨텍스트를 비웁니다.

02 모든 대화 기록이 사라지는 것을 확인



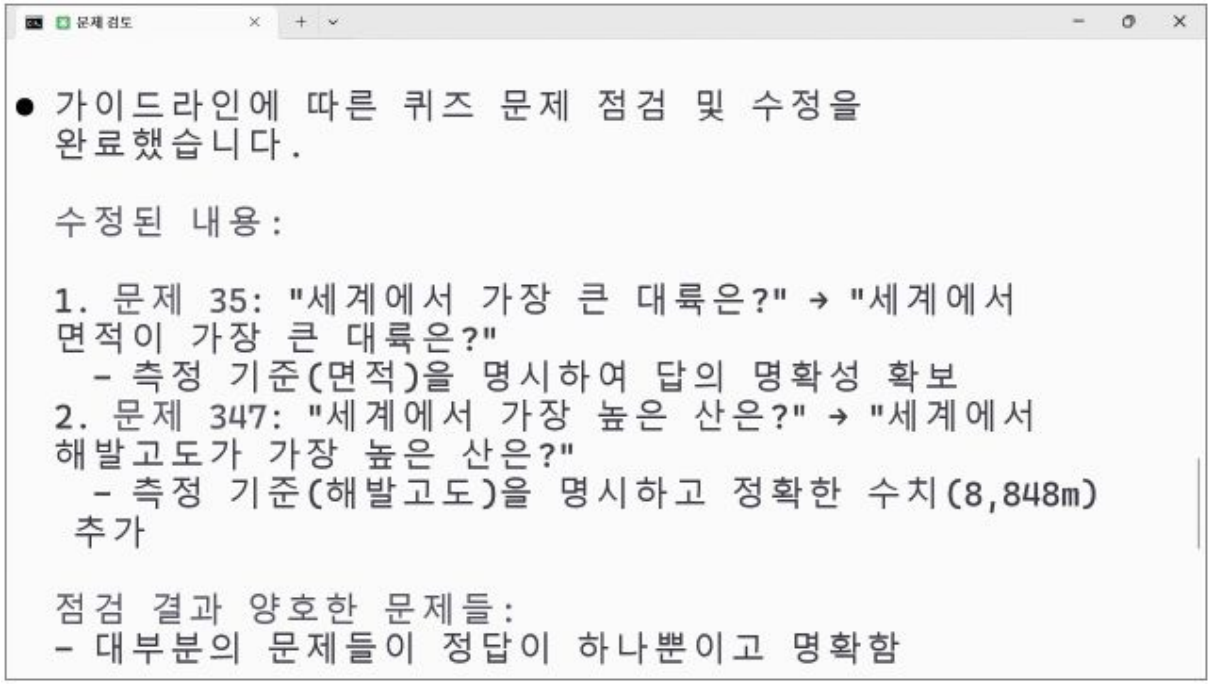
SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(34)

가이드라인 점검하기

01 클로드 코드에 다음과 같은 프롬프트를 입력
새로운 가이드라인에 따라 문제가 만들어졌다고 안내됨

프롬프트

지금까지 만든 문제를 방금 저장한 가이드라인을 참고해서 점검해 줘. 가이드라인에 맞지 않은 퀴즈와 정답이 있다면 이에 맞게 수정해 줘.

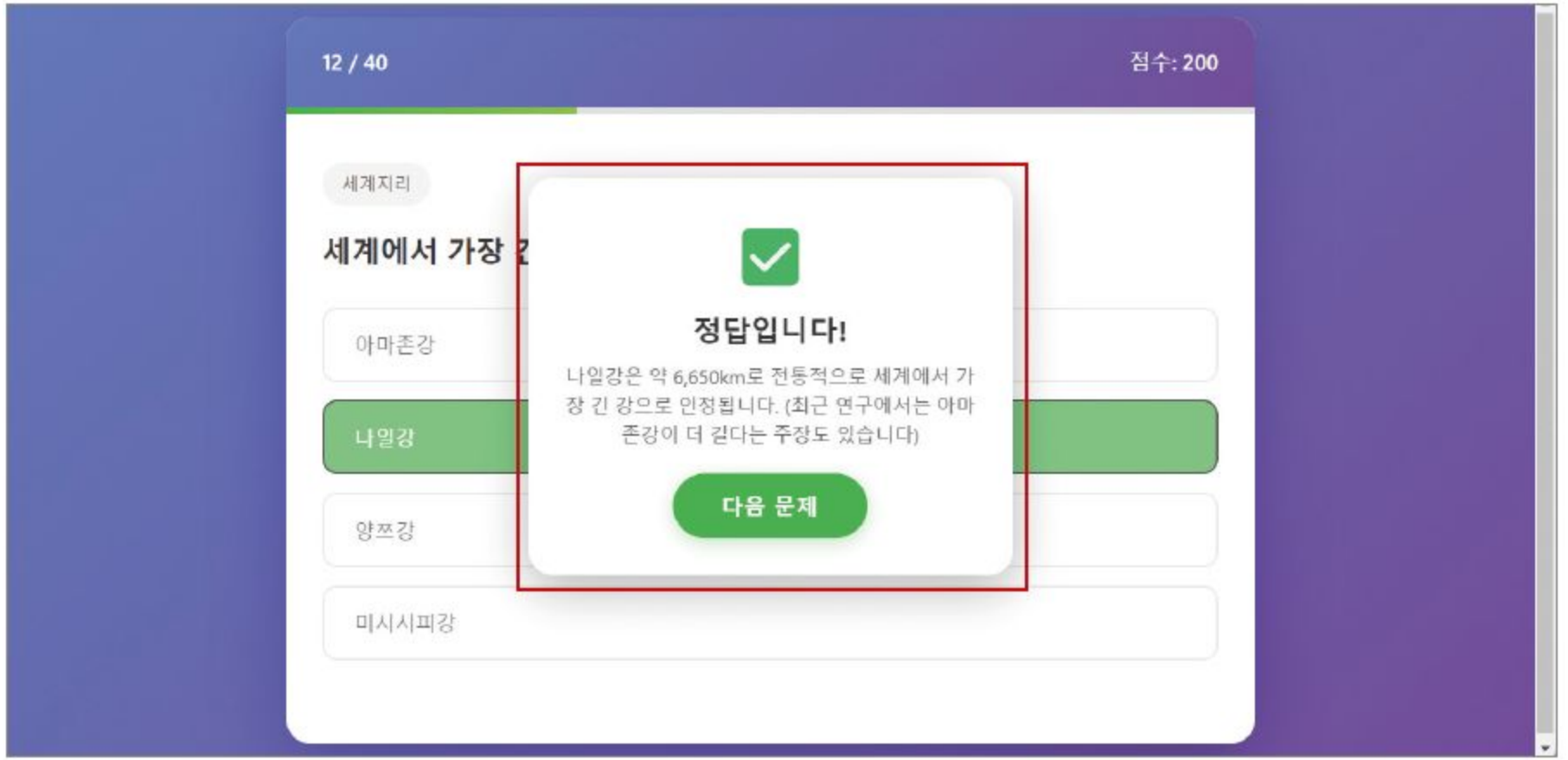


SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(35)

02 다시 'index.html' 파일을 실행해 퀴즈에 가이드라인이 반영되었는지 확인
문제에 '전통적 측정 기준'이라는 조건이 표시됨
'나일강'을 선택

SECTION 5-1 할루시네이션 없는 AI 콘텐츠 만들기(36)

03 정답 표시와 함께 추가 정보도 안내



[마무리]

- 3가지 키워드로 정리하는 핵심 포인트
  - AI가 그럴듯하지만 사실과 다른 정보를 진짜처럼 만들어내는 현상을 **할루시네이션**이라고 함
  - **/memory**는 AI가 기억하는 프로젝트 관련 정보를 확인하고 관리하는 명령어
  - **/clear**는 현재 세션의 대화와 작업 맥락을 모두 지우고 초기화하는 명령어

SECTION 5-2 자동화로 개발 효율 높이기(1)

시작하기전에

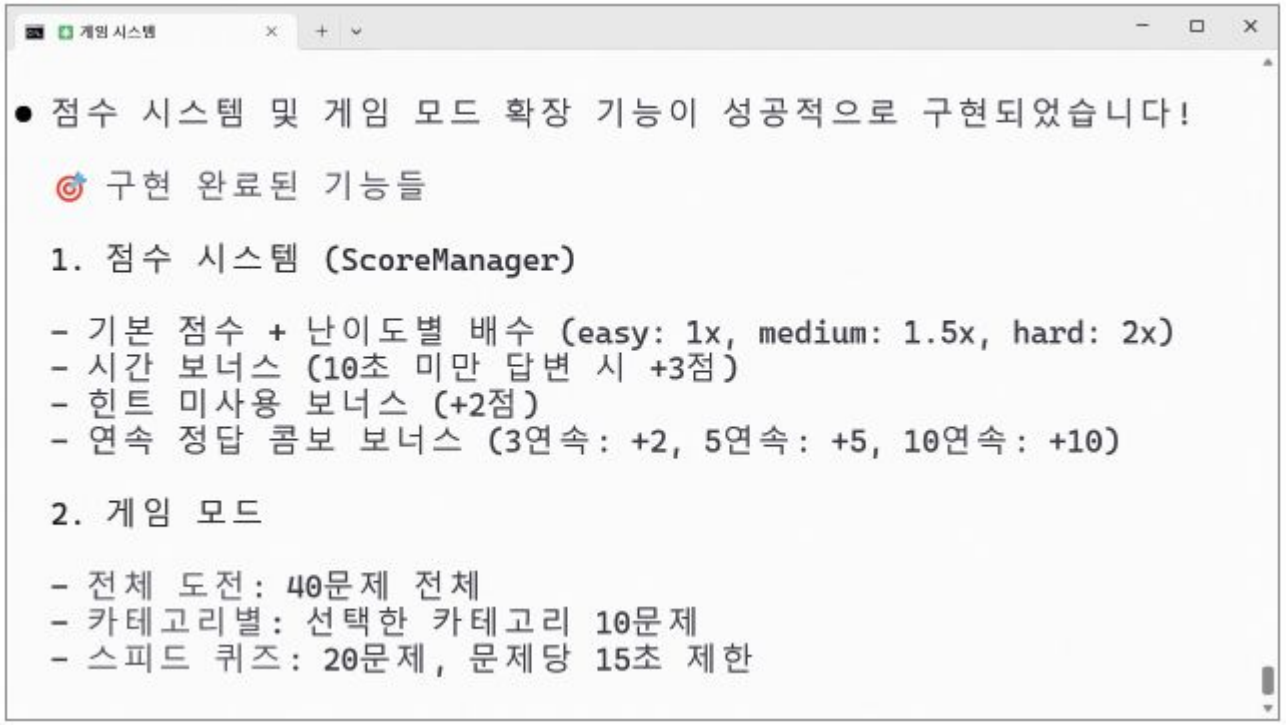
- 앞서 1장에서 웹사이트의 디자인을 커스터마이즈(customize)
- 커스터마이즈
  - 프로그램의 시스템이 제공하는 기본 기능을 그대로 쓰지 않고 사용자가 원하는 방식으로 설정, 모양, 동작을 조정
- 클로드 코드의 슬래시 명령어도 사용자가 명령어의 동작이나 형태를 바꾸어 새로 정의
  - 반복적인 일을 대신 처리해 주는 자동 실행 명령어 만들기

SECTION 5-2 자동화로 개발 효율 높이기(2)

◦ 2단계: 게임 확장하기

- 게임을 더욱 흥미롭게 만들기 위한 점수 시스템과 다양한 게임 모드를 추가

01 클로드 코드에 앞서 만든 2단계 프롬프트를 입력
실행을 마치면 완료되었다는 메시지가 나타남

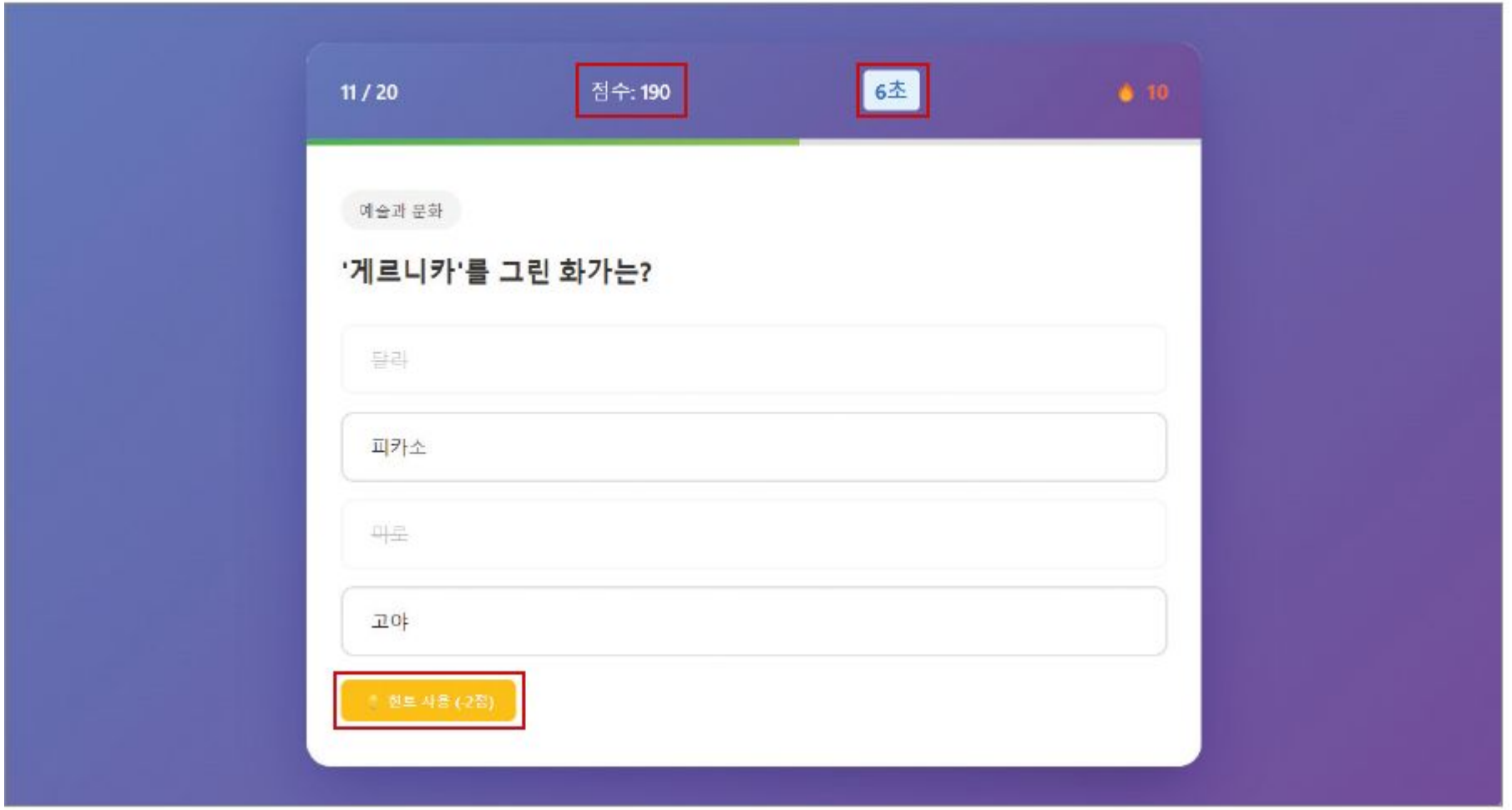


SECTION 5-2 자동화로 개발 효율 높이기(3)

02 'Study-03' 폴더에서 'index.html' 파일을 더블 클릭해 게임을 실행
게임 모드 세 가지가 추가된 것을 확인 - 이 중 [스피드 퀴즈]를 선택

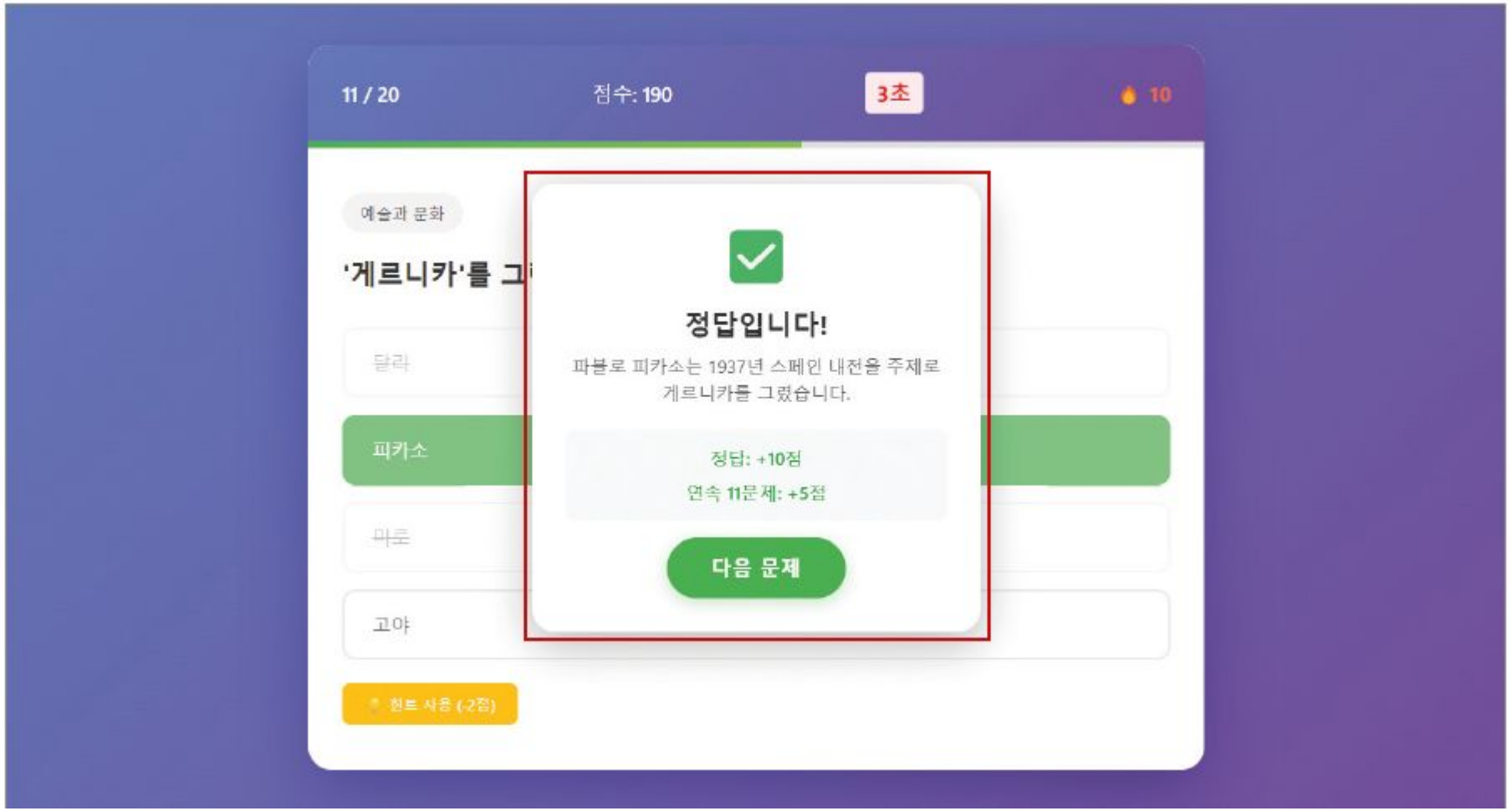
SECTION 5-2 자동화로 개발 효율 높이기(4)

03 제한 시간 15초 룰이 적용
[힌트 사용] 버튼을 누르면 힌트를 확인할 수 있고, 점수가 차감



SECTION 5-2 자동화로 개발 효율 높이기(5)

04 정답/오답을 잘 판별



SECTION 5-2 자동화로 개발 효율 높이기(6)

05 퀴즈를 다 풀면 최종 점수를 확인

SECTION 5-2 자동화로 개발 효율 높이기(7)

◦ 커스텀 명령어로 반복 작업 자동화하기

- 퀴즈 게임은 교육 콘텐츠나 학습용 앱으로도 활용할 수 있음
- 프로그램을 개발하다 보면 비슷한 작업을 반복해야 하는 경우가 많음
  - 예) 문제의 형식을 통일하기 위해 복사·붙여넣기를 반복하거나, 문제를 검증할 때마다 가이드라인을 확인하고, 균형을 맞추기 위해 난이도를 조정하는 등
- 반복 작업을 매번 긴 프롬프트로 처리하려고 하면 효율이 떨어질 뿐 아니라 실수도 잦아짐
  - 클로드 코드의 커스텀 명령어를 활용하면 반복 작업을 간단한 명령어로 자동화할 수 있어 빠르고 안정적으로 개발을 이어갈 수 있음

SECTION 5-2 자동화로 개발 효율 높이기(8)

커스텀 명령어

- 커스텀 명령어(custom command) - 사용자가 직접 정의해서 쓸 수 있는 나만의 단축 지시어
- 클로드 코드가 기본으로 제공하는 명령어 외에도 반복적으로 쓰는 지시사항을 커스텀 명령어로 만들어 두면 매번 긴 프롬프트를 다시 입력하지 않아도 됨
- 명령어는 마크다운(markdown) 파일로 정의되며, 범위에 따라 다음 두 종류로 나뉨

1. 전역 명령어
   - 홈 디렉터리 위치에서 정의해 모든 프로젝트에 공통으로 적용되는 명령어
   - 위치: ~/.claude/commands/ ← 터미널 경로 표기법에서 '~'는 홈 디렉터리를 뜻합니다.
2. 프로젝트 명령어
   - 프로젝트 디렉터리 위치에서 정의해 현재 작업 중인 프로젝트에서만 쓸 수 있는 명령어
   - 위치: .claude/commands/ ← 터미널 경로 표기법에서 '.'는 현재 디렉터리를 뜻합니다.

SECTION 5-2 자동화로 개발 효율 높이기(9)

명령어 폴더 생성하기

- 실습을 통해 간단한 커스텀 명령어 만들기
  - 상식 퀴즈 게임 프로젝트에만 적용할 것이므로, 'Study-03' 폴더에 명령어를 저장할 폴더를 새로 생성
- 데스크톱의 'Study-03' 폴더 위치에서 명령 프롬프트를 열고, 클로드 코드를 실행해 준비하고, 다음과 같이 입력
  - 클로드 코드가 명령어를 저장할 폴더를 생성하고 디렉터리 구조를 보여줌

프롬프트

```
현재 프로젝트에 커스텀 명령어 폴더를 만들어 줘.
.claude/commands 디렉터리를 생성하고 구조를 보여줘.
```

SECTION 5-2 자동화로 개발 효율 높이기(10)

SECTION 5-2 자동화로 개발 효율 높이기(11)

첫 번째 커스텀 명령어 만들기

- 상식 퀴즈 게임의 CLAUDE.md 파일에 입력했던 '2. 최상급 표현에 기준이 있는가?' 항목은 AI의 할루시네이션 현상을 방지하기 위해 최상급 표현에 대해 측정 기준을 명시할 것을 요구하는 교차 검증 가이드라인

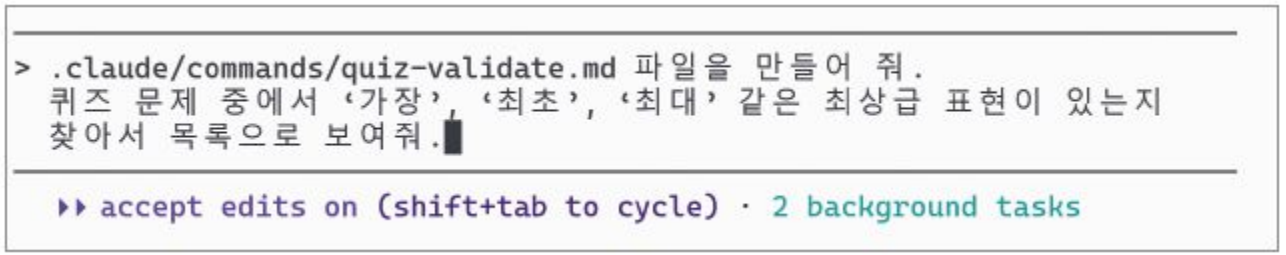
- 클로드 코드에 퀴즈의 품질을 점검하기 위한 커스텀 명령어를 등록하고, 재시작 후 실행해 모호한 최상급 표현을 찾아낸 뒤, 고치도록 만들기

SECTION 5-2 자동화로 개발 효율 높이기(12)

01 ‘commands’ 폴더 내에 첫 번째 커스텀 명령어를 만들기 - 클로드 코드에 다음과 같이 입력

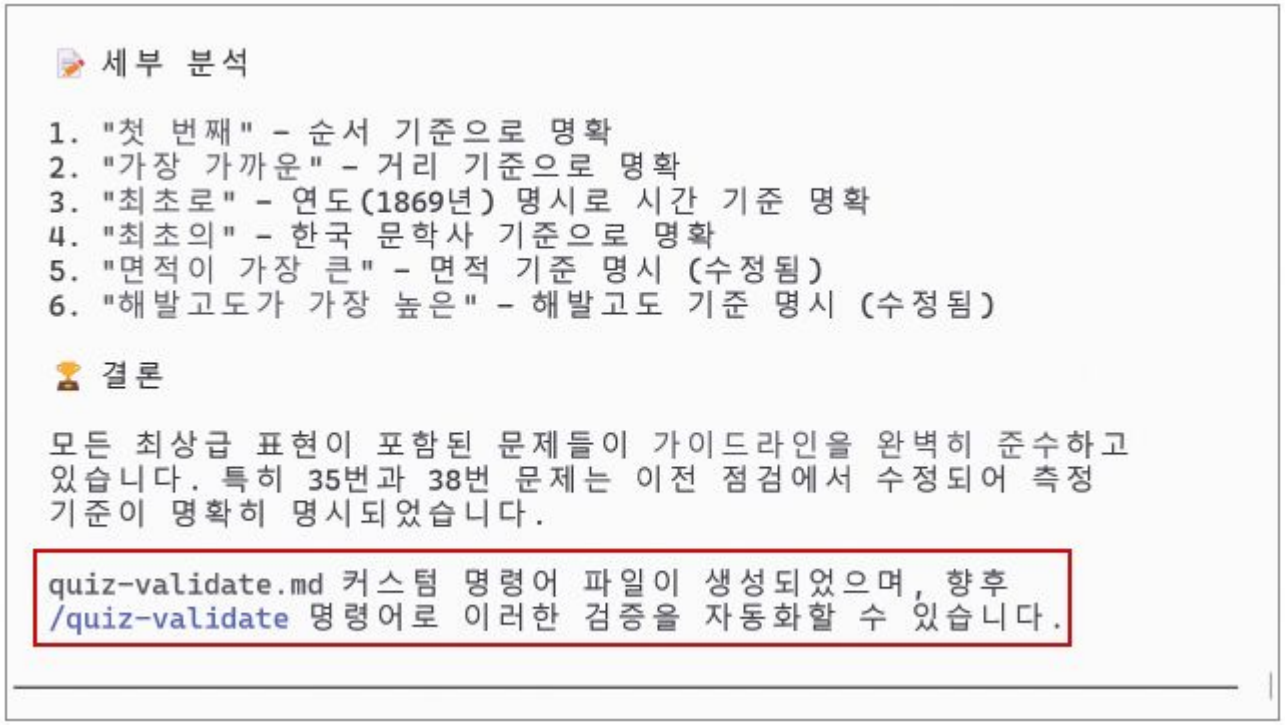
프롬프트

```
.claude/commands/quiz-validate.md 파일을 만들어 줘.
퀴즈 문제 중에서 '가장', '최초', '최대' 같은 최상급 표현이 있는지 찾아서 목록으로 보여줘.
```



SECTION 5-2 자동화로 개발 효율 높이기(13)

02 논란의 소지가 있는 답변을 방지하기 위해, 문제 중 최상급 표현을 찾아서 보여주는 커스텀 명령어 '/quiz-validate'를 생성



SECTION 5-2 자동화로 개발 효율 높이기(14)

03 커스텀 명령어를 실행하기 위해서는 클로드 코드를 종료하고 재실행
클로드 코드에 '/exit' 명령어를 입력해 종료

```
/exit
```

[note]

- **[Ctrl]+[C] 키를 두 번 눌러도 클로드 코드를 종료할 수 있음**

SECTION 5-2 자동화로 개발 효율 높이기(15)

04 명령 프롬프트에 'claude'를 입력해 클로드를 재시작
'/(슬래시)'를 입력하면 커스텀 명령어 '/quiz-validate'가 추가된 것을 확인

[note]

- **'/help' 명령어를 입력하면 등록된 모든 명령어를 볼 수 있음**

SECTION 5-2 자동화로 개발 효율 높이기(16)

05 클로드 코드에 '/quiz-validate'를 입력해 실행
퀴즈에서 최상급 표현을 골라내고 개선 방안을 제안

```
● Quiz Validation Report: 최상급 표현 검증

  발견된 문제 (4개)

  1. 문제 ID 5 (한국사 - Easy)

  문제: 백제의 첫 번째 수도는?최상급 표현: "첫 번째"측정 기준: ✅
  명확 (시간순서)상태: 양호

  ---
  2. 문제 ID 14 (과학 - Easy)

  문제: 지구에서 가장 가까운 별은?최상급 표현: "가장 가까운"측정
  기준: ✅ 명확 (거리 기준)상태: 양호

  ---
  3. 문제 ID 19 (과학 - Hard)

  문제: 주기율표를 최초로 만든 과학자는?최상급 표현: "최초로"측정
  기준: ✅ 명확 (시간순서)설명: 멘델레예프가 1869년 제작상태: 양호
```

SECTION 5-2 자동화로 개발 효율 높이기(17)

◦ 커스텀 명령어 확장하기

- 인수(argument) - 명령어나 함수에 전달하는 추가 정보
  - 명령어(command)가 '무엇을 해야 할지'를 정한다면,
    인수는 '무엇을 대상으로 어떻게 할 것인가'를 구체적으로 알려주는 값
- 퀴즈의 품질과 정확성을 검증하는 커스텀 명령어 '/quiz-validate'에 인수 플레이스홀더(placeholder)를 지정해서 두 가지 방식으로 확장

1) 지정한 카테고리의 퀴즈만 검증하도록 인수에 카테고리를 전달
2) 지정한 범위의 퀴즈만 검증하도록 인수에 두 숫자를 전달

- /quiz-validate 한국사 ← 특정 카테고리만 검증
- /quiz-range 1 20 ← 1~20번 문제만 분석

SECTION 5-2 자동화로 개발 효율 높이기(18)

✚ 여기서 잠깐 **플레이스홀더**

- 플레이스홀더 **(placeholder)** - 나중에 실제 값이 들어갈 자리를 표시하는 기호
- 아직 정해지지 않은 값이 들어갈 자리에 빈칸을 미리 만들어 놓고, 실행할 때 그 자리에 실제 값이 들어가도록 하는 장치
- 예) 웹사이트에서 '환영합니다, {이름}님'과 같이 '{이름}'이라는 플레이스홀더를 만들어 두면, 실제로 실행할 때 사용자의 이름이 플레이스홀더 자리에 들어가도록 하는 것

SECTION 5-2 자동화로 개발 효율 높이기(19)

특정 카테고리만 검증하기

- /quiz-validate 명령어가 사용자가 입력한 카테고리명을 인수로 받아서 해당 카테고리의 퀴즈만 검증하도록 확장
- 다음과 같이 명령어에 카테고리를 받을 수 있도록 인수 플레이스홀더를 설정하고, 인수가 없으면 퀴즈 전체를 검증

```
/quiz-validate $ARGUMENTS ← 인수 플레이스홀더
```

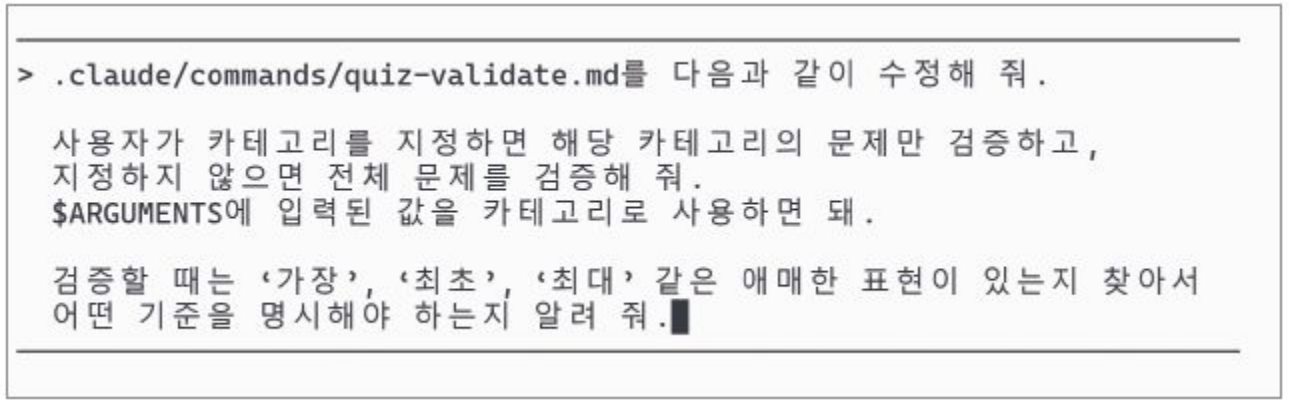
SECTION 5-2 자동화로 개발 효율 높이기(20)

01 커스텀 명령어에 인수를 지정하기 위해 '.claude/commands/quiz-validate.md' 파일을 수정

클로드 코드에 다음과 같이 프롬프트를 입력

프롬프트

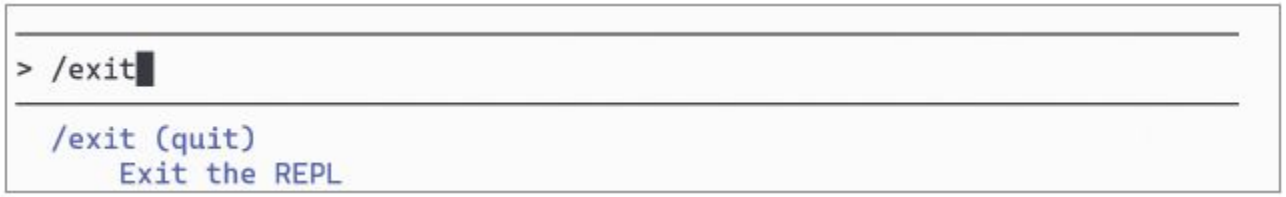
```
.claude/commands/quiz-validate.md를 다음과 같이 수정해 줘.

사용자가 카테고리를 지정하면 해당 카테고리의 문제만 검증하고,
지정하지 않으면 전체 문제를 검증해 줘.
$ARGUMENTS에 입력된 값을 카테고리로 사용하면 돼.

검증할 때는 '가장', '최초', '최대' 같은 모호한 표현이 있는지 찾아서
어떤 기준을 명시해야 하는지 알려 줘.
```



SECTION 5-2 자동화로 개발 효율 높이기(21)

02 완료되었다는 메시지를 확인하면 '/exit' 명령어를 입력해서 클로드 코드를 종료

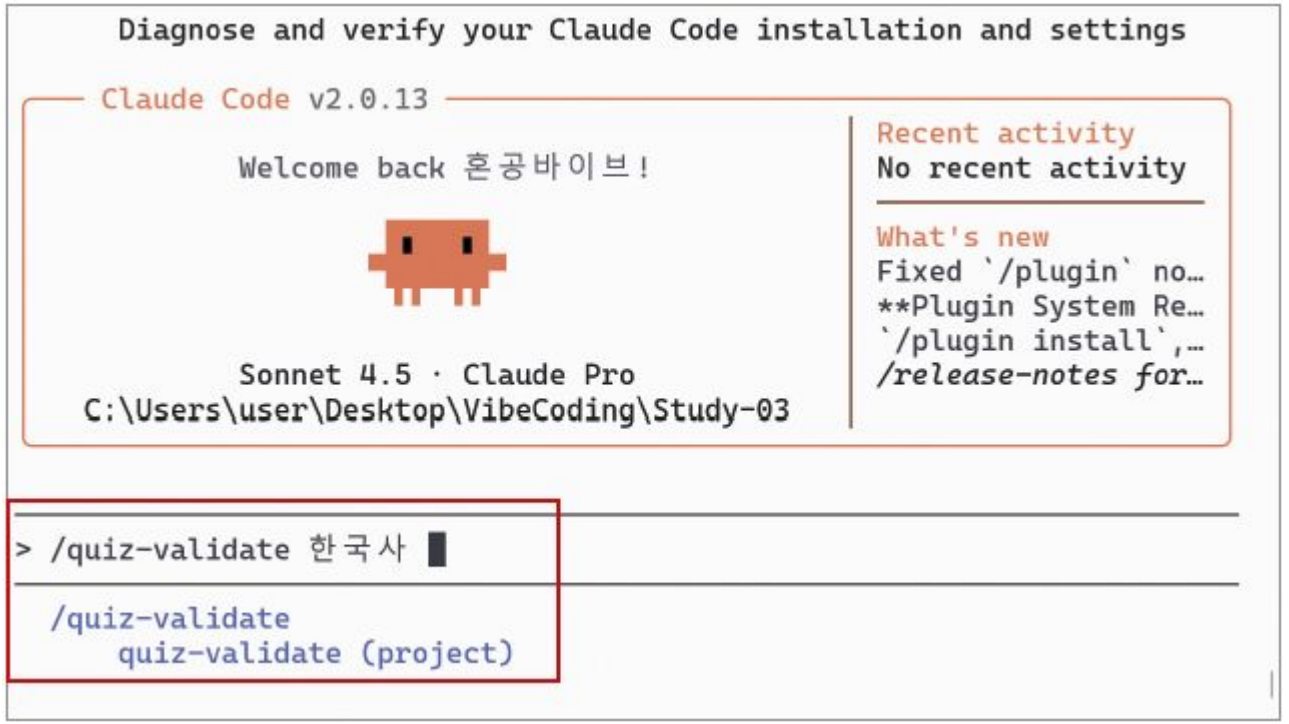
```
/exit
```



SECTION 5-2 자동화로 개발 효율 높이기(22)

03 'claude'를 입력해 새로 시작한 뒤, '/quiz-validate 한국사' 명령어를 입력
'한국사' 카테고리의 퀴즈만 검증하는 명령어

```
/quiz-validate 한국사
```



[note] 중간 과정을 확인해 가면서 '1. Yes'를 눌러 끝까지 진행시킴

SECTION 5-2 자동화로 개발 효율 높이기(23)

04 '한국사' 카테고리의 퀴즈를 검증

```
종합 평가

한국사 카테고리 최상급 표현 사용 현황:
- 전체 문제 수: 10개
- 최상급 표현 사용 문제: 1개 (10%)
- 측정 기준 명확한 문제: 1개 (100%)
- 개선 필요 문제: 0개

결론: 한국사 카테고리의 최상급 표현 사용은 적절하며, 사용된
표현의 측정 기준이 명확하여 추가 개선이 필요하지 않습니다.
```

SECTION 5-2 자동화로 개발 효율 높이기(24)

특정 범위만 분석하기

- /quiz-validate 명령어가 사용자가 입력한 카테고리명을 인수로 받아서 해당 카테고리의 퀴즈만 검증하도록 확장
- 다음과 같이 명령어에 카테고리를 받을 수 있도록 인수 플레이스홀더를 설정하고, 인수가 없으면 퀴즈 전체를 검증

```
/quiz-validate $ARGUMENTS ← 인수 플레이스홀더
```

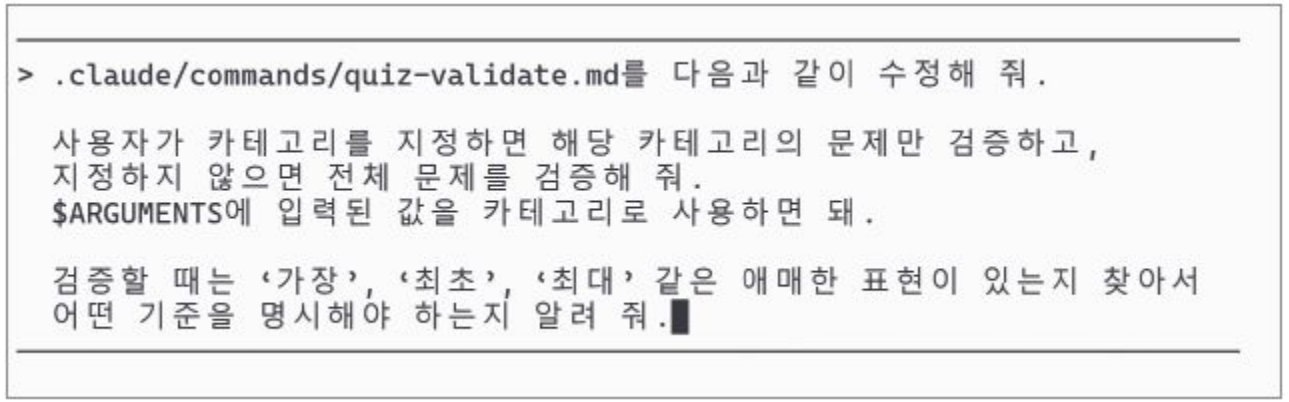
SECTION 5-2 자동화로 개발 효율 높이기(25)

01 커스텀 명령어에 인수를 지정하기 위해 '.claude/commands/quiz-validate.md' 파일을 수정

클로드 코드에 다음과 같이 프롬프트를 입력

프롬프트

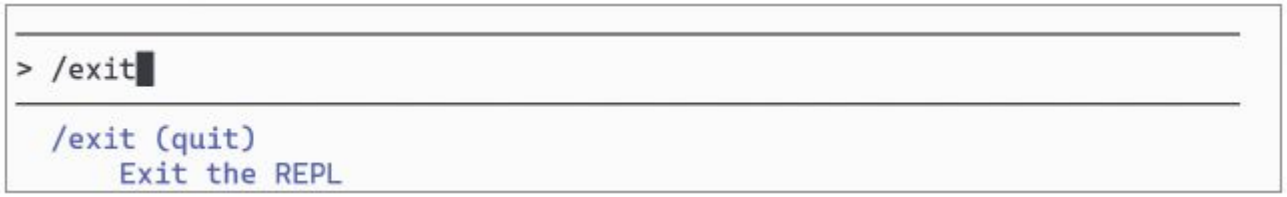
```
.claude/commands/quiz-validate.md를 다음과 같이 수정해 줘.

사용자가 카테고리를 지정하면 해당 카테고리의 문제만 검증하고,
지정하지 않으면 전체 문제를 검증해 줘.
$ARGUMENTS에 입력된 값을 카테고리로 사용하면 돼.

검증할 때는 '가장', '최초', '최대' 같은 모호한 표현이 있는지 찾아서
어떤 기준을 명시해야 하는지 알려 줘.
```



SECTION 5-2 자동화로 개발 효율 높이기(26)

02 완료되었다는 메시지를 확인하면 '/exit' 명령어를 입력해서 클로드 코드를 종료

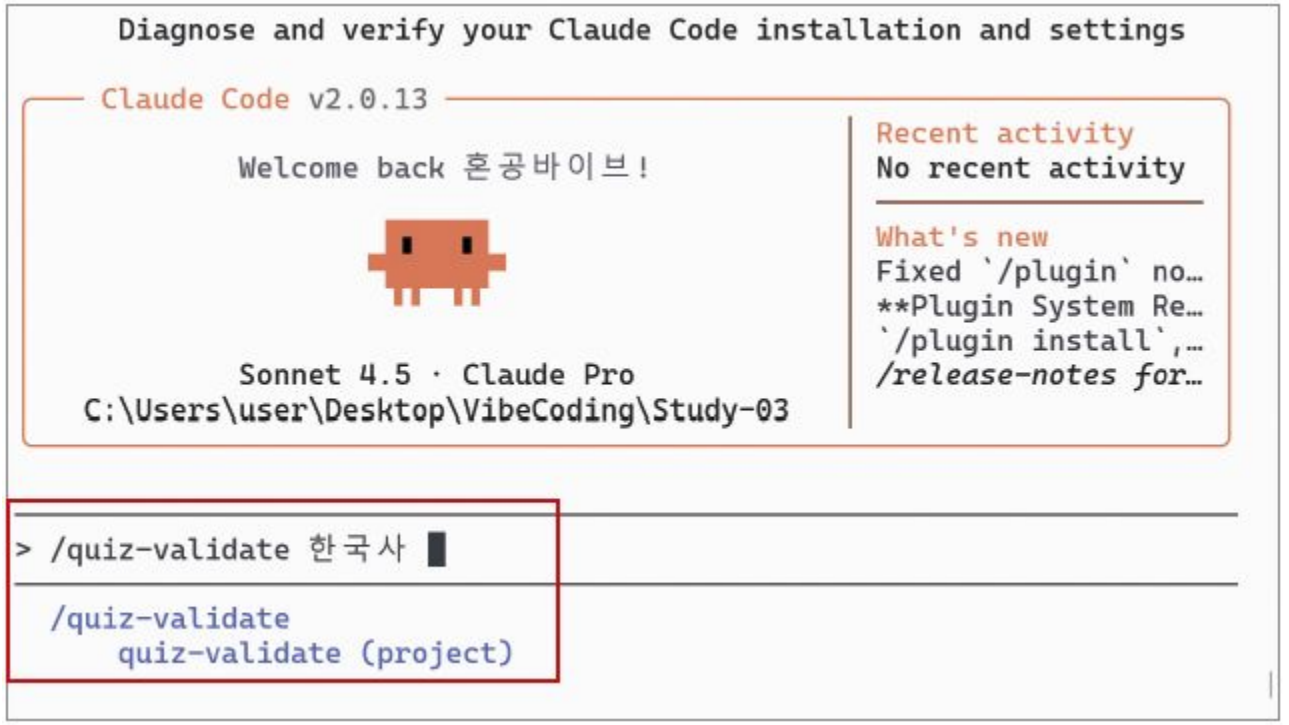
```
/exit
```



SECTION 5-2 자동화로 개발 효율 높이기(27)

03 'claude'를 입력해 새로 시작한 뒤, '/quiz-validate 한국사' 명령어를 입력
('한국사' 카테고리의 퀴즈만 검증하는 명령어)

```
/quiz-validate 한국사
```



[note] 중간 과정을 확인해 가면서 '1. Yes'를 눌러 끝까지 진행

SECTION 5-2 자동화로 개발 효율 높이기(28)

04 '한국사' 카테고리의 퀴즈를 검증

```
종합 평가

한국사 카테고리 최상급 표현 사용 현황:
- 전체 문제 수: 10개
- 최상급 표현 사용 문제: 1개 (10%)
- 측정 기준 명확한 문제: 1개 (100%)
- 개선 필요 문제: 0개

결론: 한국사 카테고리의 최상급 표현 사용은 적절하며, 사용된
표현의 측정 기준이 명확하여 추가 개선이 필요하지 않습니다.
```

SECTION 5-2 자동화로 개발 효율 높이기(29)

특정 범위만 분석하기

- 특정 범위의 퀴즈를 분석하는 커스텀 명령어 '/quiz-range'를 생성
  - 이 명령어는 두 개의 위치 인수를 받아 검토할 퀴즈의 범위를 지정
  - 예) '/quiz-range 1 20'는 1번부터 20번까지 퀴즈를 검증
- 이 명령어는 퀴즈의 난이도와 정답 분포를 중점적으로 분석
  - 지정된 구간에 포함된 퀴즈의 난이도 평균과 편차를 계산하여, 너무 쉽거나 어려운 문제가 특정 구간에 몰려 있지는 않은지 확인
- 또 정답이 특정 보기에 치우쳐 있는지도 함께 점검
  - 예) 정답이 지나치게 1번에만 몰려 있거나, 4번 보기는 거의 정답으로 사용되지 않는 상황을 탐색
    - 이러한 분석 결과를 바탕으로 전체 퀴즈의 밸런스를 고르게 유지할 수 있도록 방향을 제시

```
/quiz-range $시작 번호 $끝 번호 ← 인수 플레이스홀더
```

SECTION 5-2 자동화로 개발 효율 높이기(30)

01 특정 범위의 퀴즈를 검토하는 커스텀 명령어 '/quiz-range'를 생성하기 위해 클로드 코드에 다음과 같이 프롬프트를 입력

프롬프트

```
.claude/commands/quiz-range.md를 생성해.

$1번부터 $2번까지의 문제를 검토하는 기능을 만들어 줘.
문제의 난이도와 정답 분포를 확인하게 해 줘.
```

```
> .claude/commands/quiz-range.md를 생성해.

$1번부터 $2번까지의 문제를 검토하는 기능을 만들어 줘.
문제의 난이도와 정답 분포를 확인하게 해 줘.

● I'll create a new slash command file for reviewing quiz
  questions within a specific range.

● Write(.claude\commands\quiz-range.md)
  └ Wrote 1 lines to .claude\commands\quiz-range.md
    ---
    description: 지정한 범위의 퀴즈 문제를 검토하고 난이도와
    정답 분포를 분석합니다
    ---
    quiz-data.js 파일을 읽고, $1번부터 $2번까지의 문제를 검토해
    줘.
    각 문제에 대해:
    1. 문제 번호, 카테고리, 난이도
    2. 문제 내용
```

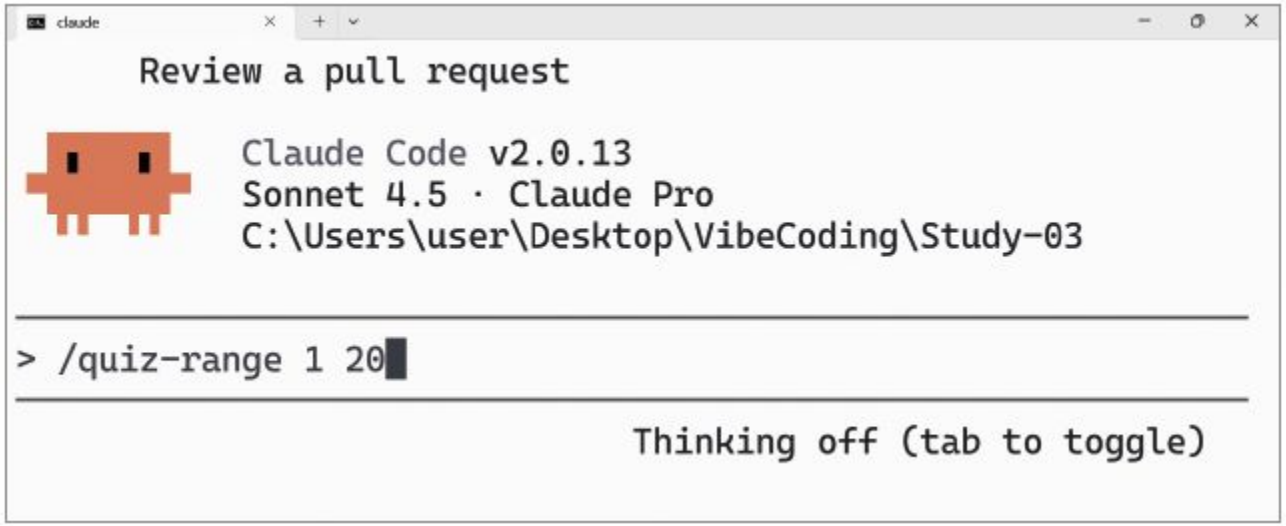
$1과 $2는 각각 시작 번호와 끝 번호를 나타내는 플레이스홀더로, 실제 명령어 실행 시 사용자가 입력한 인수로 대체됩니다.

SECTION 5-2 자동화로 개발 효율 높이기(31)

02 '/exit' 명령어를 입력해 클로드를 종료한 뒤, 'claude' 명령어로 재시작

그러고 나서 커스텀 명령어 '/quiz-range'가 잘 작동하는지 확인하기 위해 1번부터 20번까지 퀴즈를 검사하는 명령어를 다음과 같이 입력

```
/quiz-range 1 20
```



SECTION 5-2 자동화로 개발 효율 높이기(32)

03 명령어가 정확히 작동하여 1-20번 퀴즈를 분석
정답 위치를 균등하게 분산시켜서 편향을 완화할 것을 권장함

```
---
⚠ 경고 사항

1. 정답 편향 심각: 정답이 0번 선택지에 55% 집중되어
있습니다. 이는 수험자가 패턴을 파악하여 전략적으로
답할 수 있는 문제입니다.
2. 3번 선택지 미사용: 3번 선택지가 단 한 번도 정답으로
 사용되지 않았습니다. 정답이 0번과 1번에만 집중되어
있습니다.
3. 권장 사항:
  - 정답 위치를 더 균등하게 분산시키세요 (각 25% 목표)
  - 특히 2번, 3번 선택지를 정답으로 더 많이 활용하세요
  - 현재 패턴으로는 "0번부터 찍기" 전략이 유효할 수
있습니다
```

SECTION 5-2 자동화로 개발 효율 높이기(33)

◦ 새로운 퀴즈 문제 추가 자동화

- 새 퀴즈를 추가할 때도 형식의 일관성과 가이드라인을 준수하는 것이 중요
- 이를 자동화하기 위해 카테고리와 난이도를 인수로 받는 커스텀 명령어 '/quiz-add'를 생성
  - 예) 한국사 카테고리에 중급 난이도 퀴즈를 새로 추가하려면 '/quiz-add 한국사 중급'이라고 입력

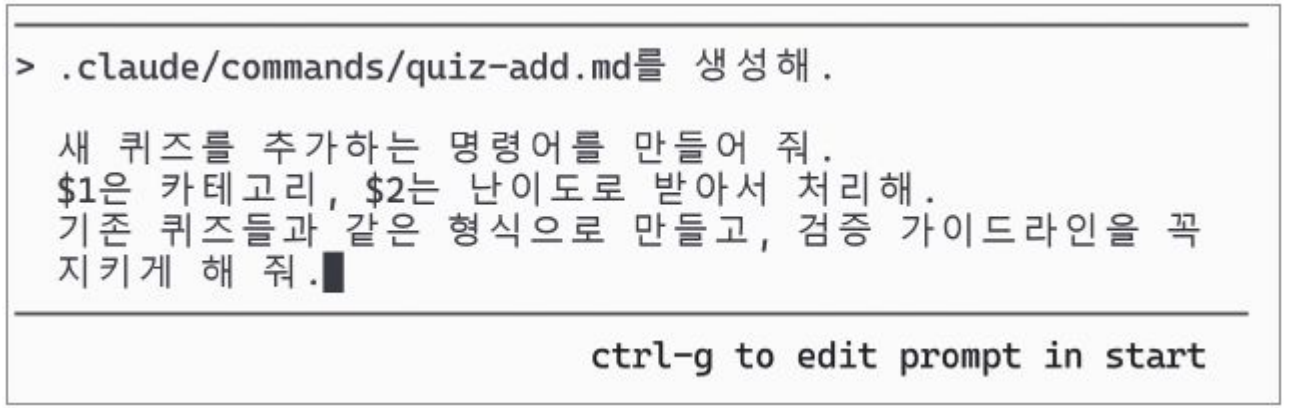
```
/quiz-add $카테고리 $난이도
```

SECTION 5-2 자동화로 개발 효율 높이기(34)

01 새 퀴즈 템플릿을 자동 생성하고 검증 가이드라인을 포함하는 명령어 '/quiz-add'를 생성하기 위해 클로드 코드에 다음과 같이 입력

프롬프트

```
.claude/commands/quiz-add.md를 생성해.

새 퀴즈를 추가하는 명령어를 만들어 줘.
$1은 카테고리, $2는 난이도로 받아서 처리해.
기존 퀴즈들과 같은 형식으로 만들고, 검증 가이드라인을 꼭 지키게 해 줘.
```



SECTION 5-2 자동화로 개발 효율 높이기(35)

02 새로운 명령어 'quiz-add'를 생성 - 사용법과 예시를 확인

```
● 새로운 slash command /quiz-add를 생성했습니다!

  생성된 명령어 기능

  사용법:
  /quiz-add [카테고리] [난이도]

  예시:
  /quiz-add 한국사 medium
  /quiz-add 과학 hard
```

SECTION 5-2 자동화로 개발 효율 높이기(36)

03 '/exit' 명령어로 클로드 코드를 종료한 뒤 'claude' 명령어로 재시작
생성한 명령어 '/quiz-add'를 활용해 중급 한국사 퀴즈를 추가
클로드 코드에 다음과 같이 입력

```
/quiz-add 한국사 medium
```

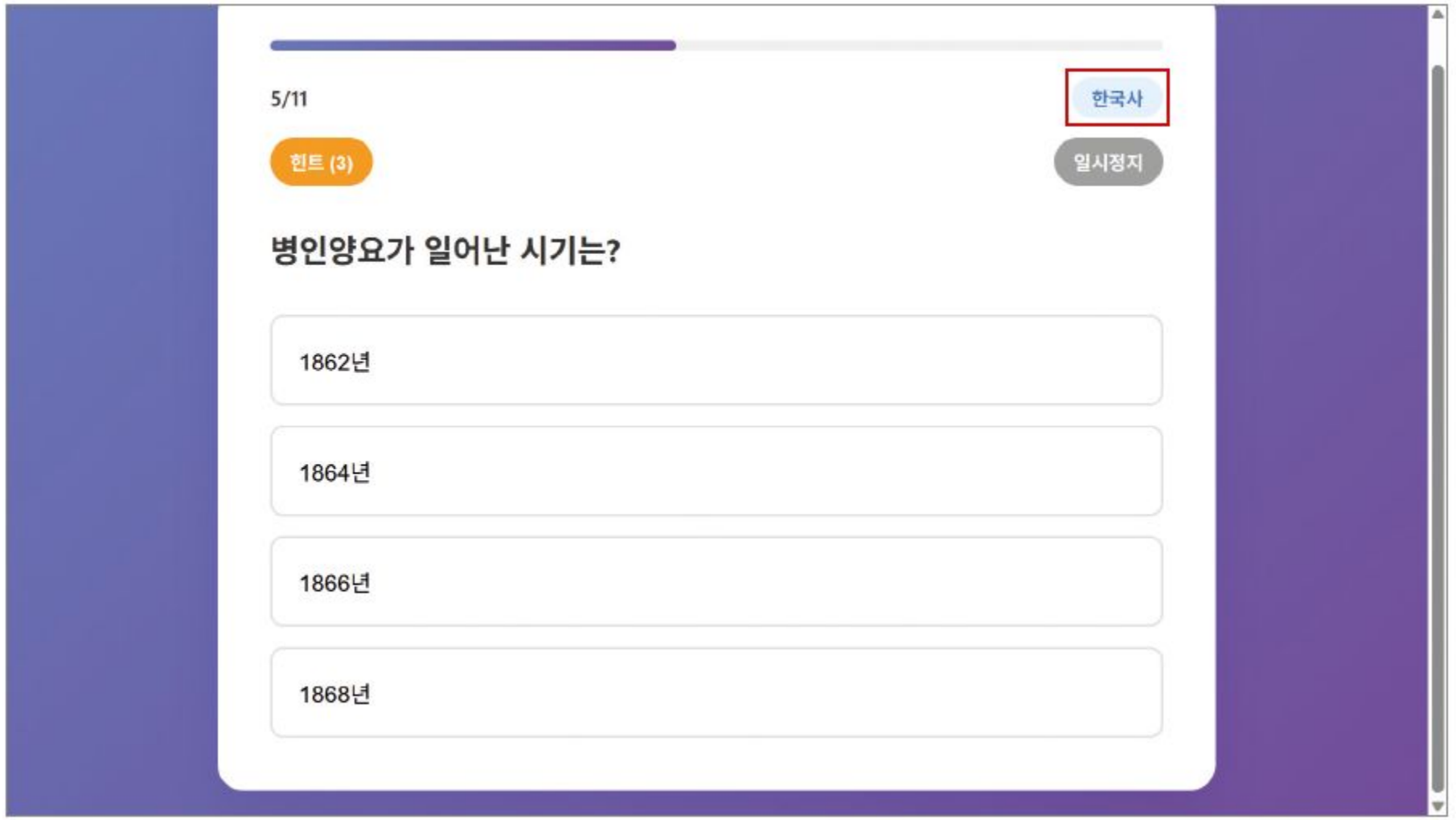
SECTION 5-2 자동화로 개발 효율 높이기(37)

04 '한국사' 카테고리에 새 퀴즈가 추가

```
● ✅ 문제 추가 완료!

  새로운 한국사 문제(ID: 41)가 quiz-data.js에 성공적으로
   추가되었습니다.

  추가된 문제:
  - 카테고리: 한국사
  - 난이도: medium
  - 문제: 병인양요가 일어난 시기는?
  - 정답: 2번 - 1866년

  개선 효과:
  - 한국사 카테고리의 정답 위치 편향 개선 (2번 선택지
  활용)
  - 전체 문제 수: 40개 → 41개
  - 한국사 문제 수: 10개 → 11개
```

SECTION 5-2 자동화로 개발 효율 높이기(38)

05 새 퀴즈가 추가되었는지 확인하기 위해 데스크톱의 'Study-03' 폴더에서 'index.html' 파일을 더블 클릭해 퀴즈 게임을 실행



[마무리]

- 5가지 키워드로 정리하는 핵심 포인트
  - **커스텀 명령어**는 사용자가 직접 정의해 반복 작업을 자동화할 수 있는 개인화된 명령어
  - **전역 명령어**는 홈 디렉터리(~/.claude/commands/)에 정의되어 모든 프로젝트에서 공통으로 사용할 수 있음
  - **프로젝트 명령어**는 현재 작업 폴더(.claude/commands/)에 저장되어 해당 프로젝트에서만 적용
  - **인수**는 명령어나 함수에 전달되는 추가 정보로, 작업의 대상이나 범위를 지정하는 역할
  - **플레이스홀더**는 $ARGUMENTS, $1, $2처럼 인수가 들어갈 자리를 표시하는 기호로, 실행 시 실제 값으로 치환됨

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(1)

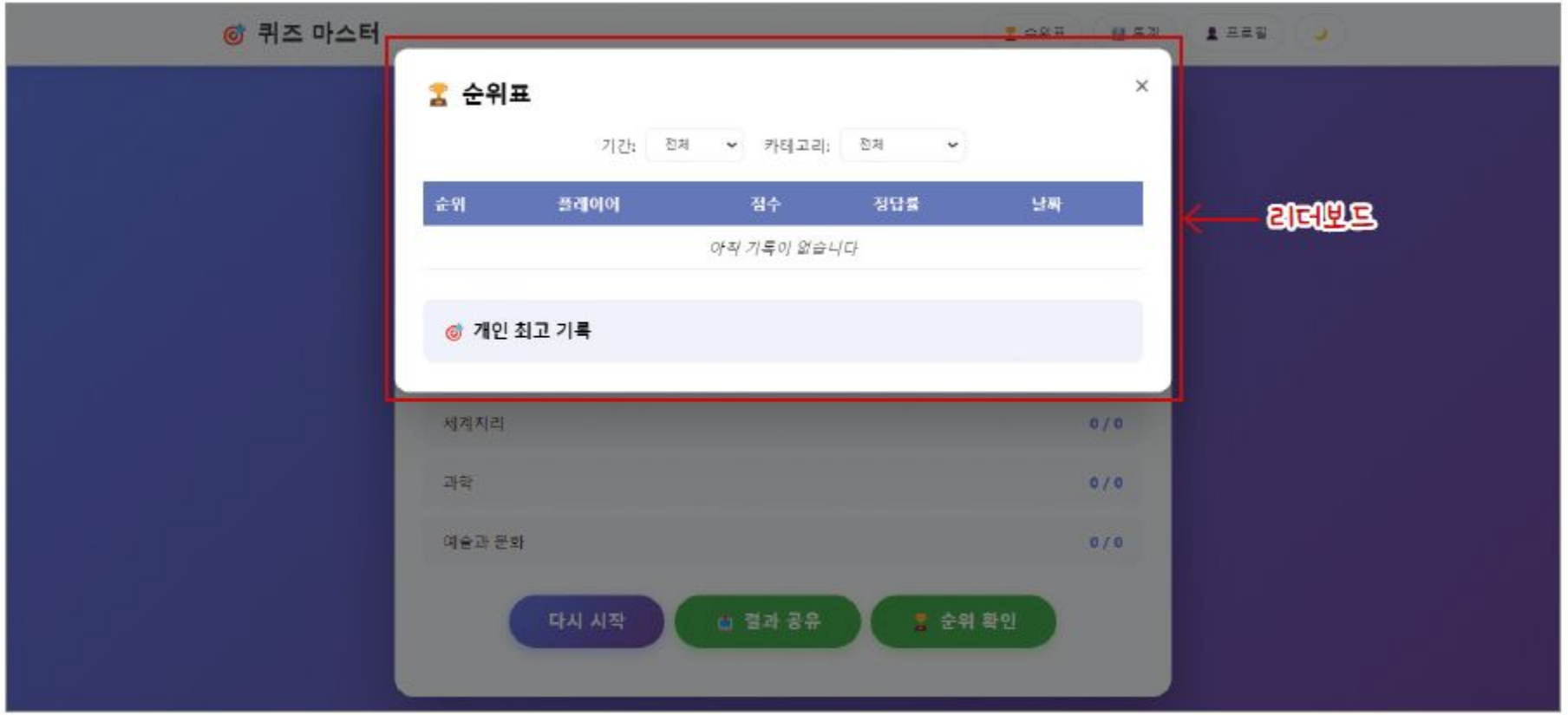
시작하기전에

- 퀴즈 게임은 새로운 문제를 꾸준히 추가하고, 기존 문제의 오류를 수정하거나 난이도를 조정하는 등 지속적인 업데이트와 유지보수가 중요
  - 하지만 이런 반복 작업을 매일 수동으로 하다 보면 금세 귀찮고 번거로움
- AI는 이런 유지보수 과정도 자동화
  - 명령어 하나만 실행하면 문제 추가, 검증, 통계 분석, 백업까지 하루치 업데이트를 한 번에 처리
  - AI가 관리 루틴을 대신 수행하도록 설계하면 개발자는 반복 작업 대신 새로운 아이디어와 콘텐츠 개선에 집중할 수 있음

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(2)

◦ 3단계: 데이터 저장 및 순위 시스템

- 3단계는 사용자 플레이 기록을 로컬에 안전하게 저장하고, 이를 바탕으로 게임에 리더보드(leader board), 순위표, 개인 통계, 대시보드를 구현



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(3)

01 클로드 코드에 05-1절에서 생성한 3단계 프롬프트를 입력
실행을 마치면 완료되었다는 메시지가 나타남

```
✅ 3단계 완료: 데이터 저장 및 순위 시스템

🎯 구현된 기능

1. LocalDataManager 클래스 (data-manager.js)

- ✅ 게임 결과 자동 저장 (localStorage)
- ✅ 통계 데이터 추적 및 업데이트
- ✅ 최근 100개 게임 기록 유지
- ✅ 카테고리별 통계 관리

2. 순위 시스템

- ✅ 전체 / 주간 / 일일 리더보드
- ✅ 상위 10개 기록 표시
```

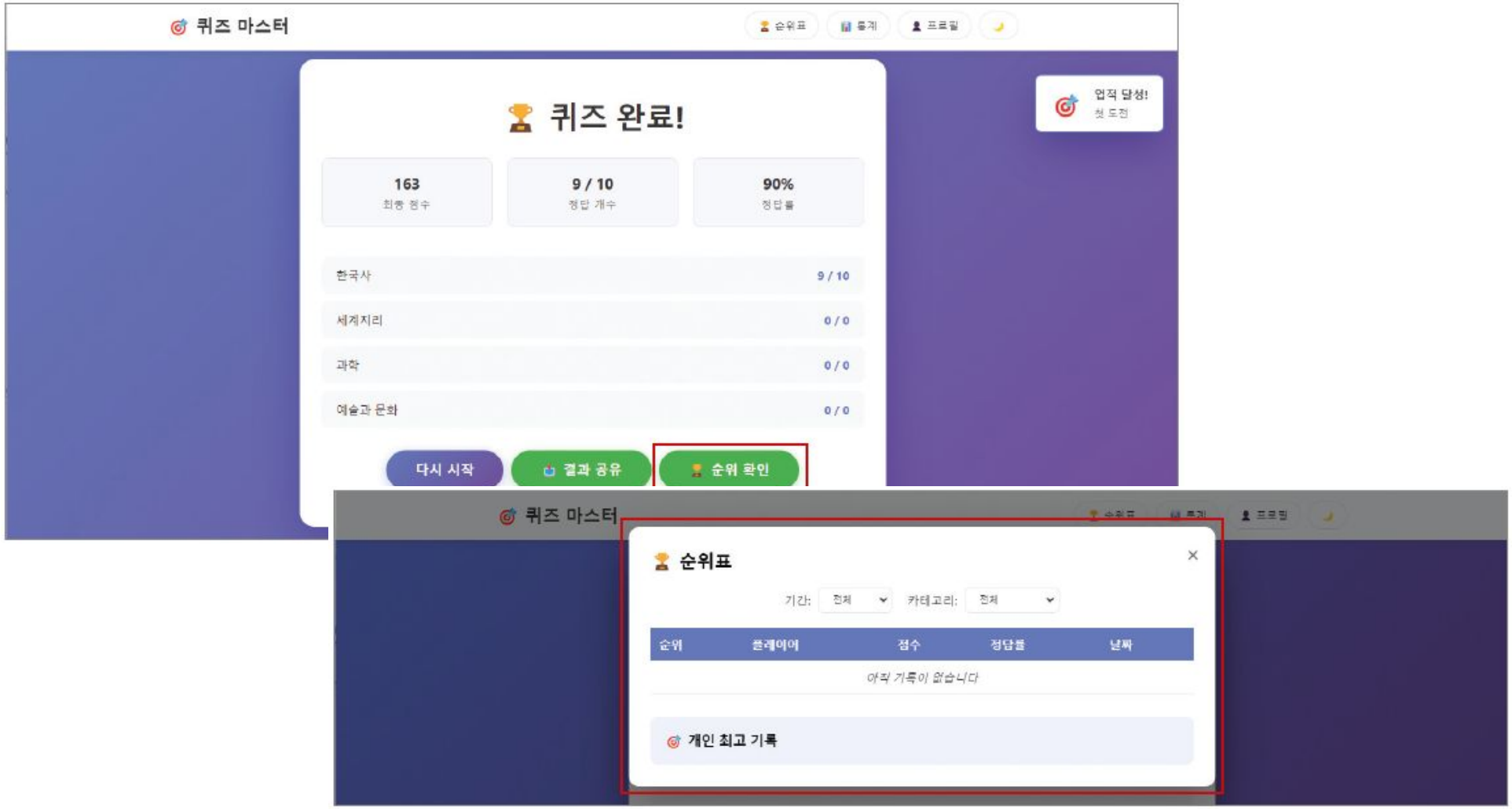
깃허브 자료실
바로가기

[note]

- **깃허브 자료실 (https://github.com/taehojo/vibecoding)의 [전체 프롬프트 모음 보기] 에 이 책에서 제시된 예시 프롬프트를 장별로 제공**

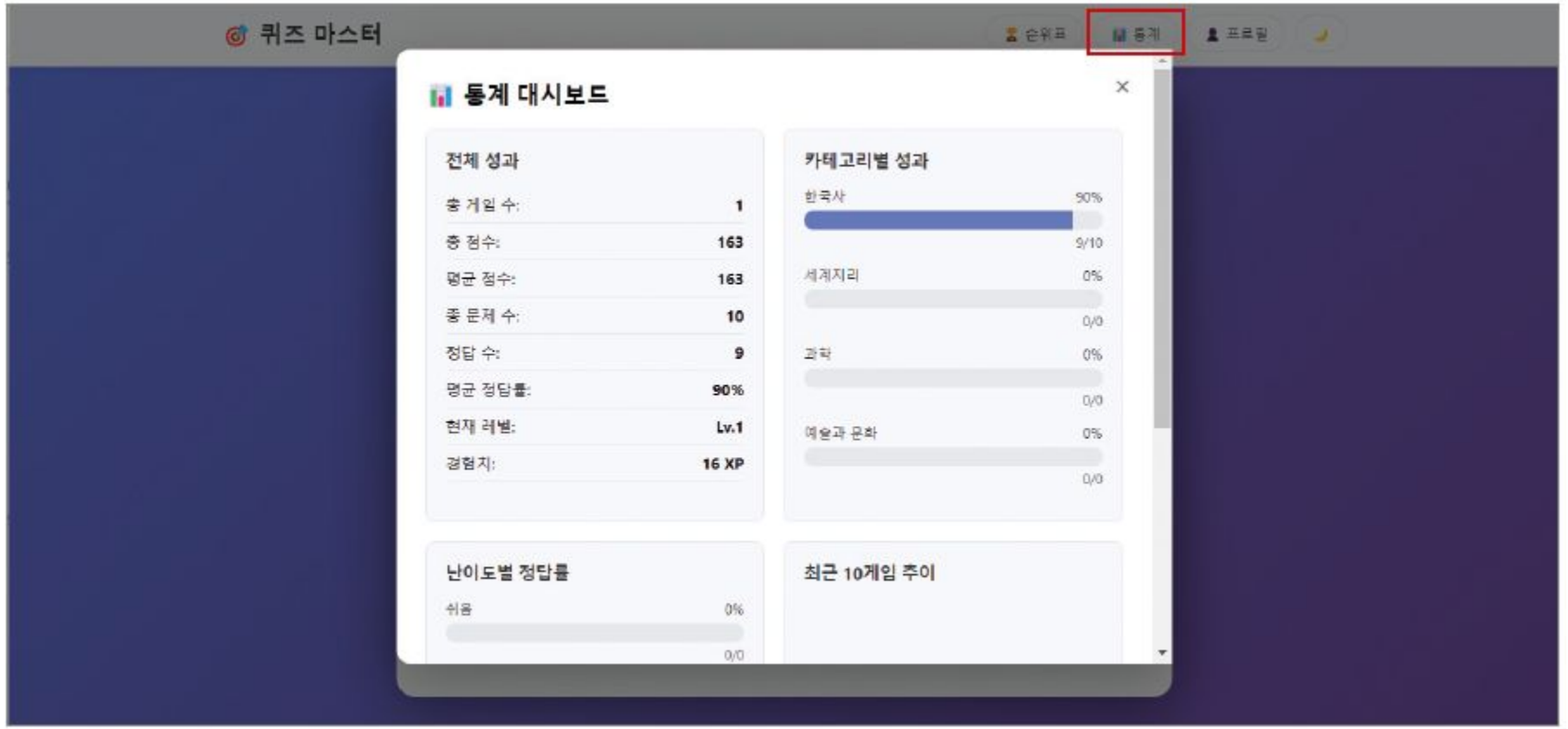
SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(4)

02 데스크톱의 'Study-03' 폴더에서 'index.html' 파일을 실행해 리더보드와 통계 대시보드를 확인
퀴즈 게임을 완료하면 세부 통계를 보여주고, [순위 확인] 버튼을 누르면 리더보드가 나타남



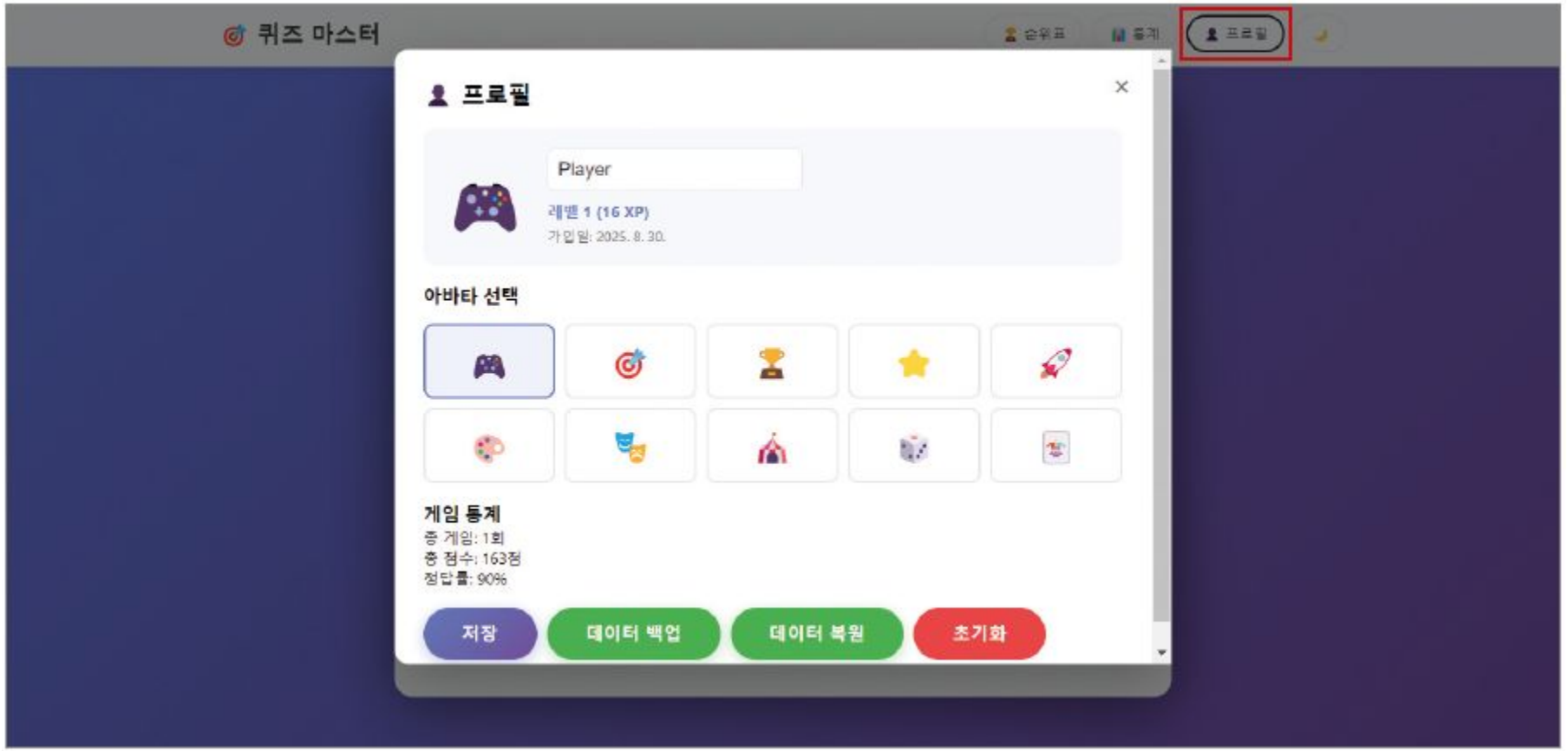
SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(5)

03 상단의 [통계] 버튼을 누르면 통계 대시보드도 확인



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(6)

04 상단의 [프로필] 버튼을 누르면 사용자 프로필을 편집



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(7)

◦ 게임 제작 워크플로 자동화하기

- 커스텀 명령어는 개별적으로도 유용하지만, 여러 명령어를 연결하여 더 강력한 자동화를 구현
- 게임을 제작하는 일련의 과정을 하나의 파이프라인(pipeline)으로 묶어,
  '문제 추가-검증-통계-보고서 생성'의 단계를 자동으로 처리

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(8)

명령어 체이닝

- 명령어 체이닝(command chaining)
  - 여러 커스텀 명령어를 순서대로 연결 실행해서 반복적인 개발, 관리 작업을 자동화하는 방법
  - 가장 간단한 형태는 && 연산자로 두 개 이상의 명령어를 순서대로 연결
    - 앞의 명령이 성공적으로 실행된 경우에만 다음 명령이 이어짐

```
/명령어1 && /명령어2 && /명령어3
```

  - 예) && 연산자를 사용해서 두 명령어를 체이닝하면 '한국사' 카테고리에 새로운 상급 문제를 추가한 뒤 검증하는 과정을 순서대로 진행

```
/quiz-add 한국사 상급 && /quiz-validate 한국사
```

  - (주의할 점)
    - && 연산자를 사용해서 명령어를 체이닝할 때 은 앞 명령이 실패하면 다음 명령이 실행되지 않음
    - 따라서 새로운 체이닝을 만들 때는 작은 단위로 개별 테스트한 후 연결

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(9)

퀴즈 생성, 분석, 점수 집계하기

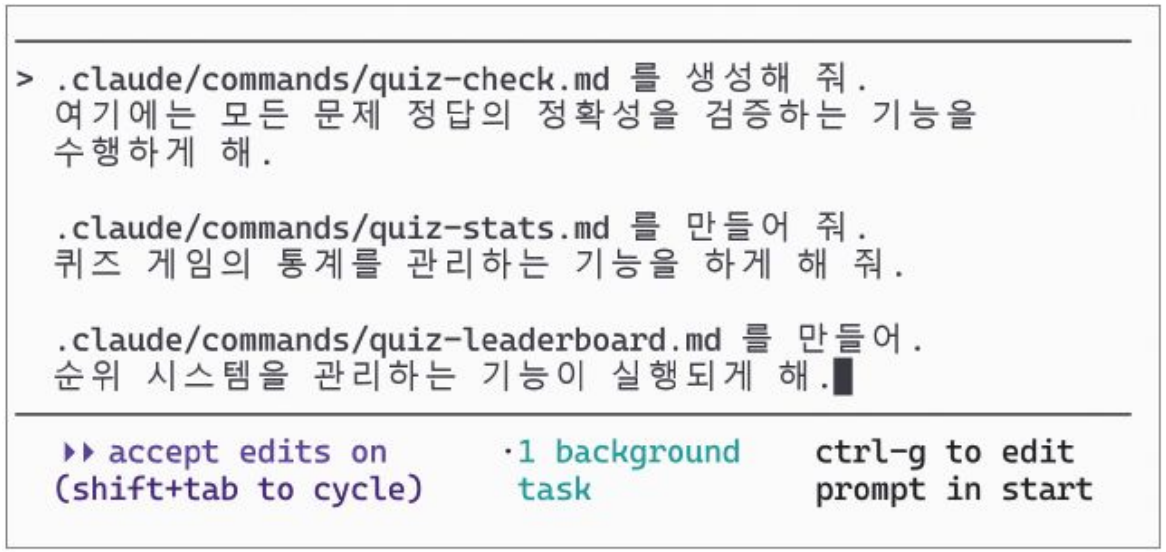
- 여러 개의 커스텀 명령어를 생성하고 체이닝해서 게임 제작 과정을 효율적으로 만들기
- 생성할 커스텀 명령어 세 가지
  - /quiz-check: 모든 퀴즈를 대상으로 정답이 올바른지 자동으로 검증하는 명령어
  - /quiz-stats: 퀴즈 게임의 전체 데이터를 분석해 진행 현황과 난이도 분포를 수치화하는 명령어
  - /quiz-leaderboard: 플레이어의 점수를 집계하고 리더보드를 자동으로 갱신하는 명령어

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(10)

01 다음과 같이 필요한 명령어들을 생성할 수 있도록 클로드 코드에 한 번에 명령

프롬프트

```
.claude/commands/quiz-check.md 를 생성해 줘.
여기에는 모든 문제 정답의 정확성을 검증하는 기능을 수행하게 해.

.claude/commands/quiz-stats.md 를 만들어 줘.
퀴즈 게임의 통계를 관리하는 기능을 하게 해 줘.

.claude/commands/quiz-leaderboard.md 를 만들어.
순위 시스템을 관리하는 기능이 실행되게 해.
```



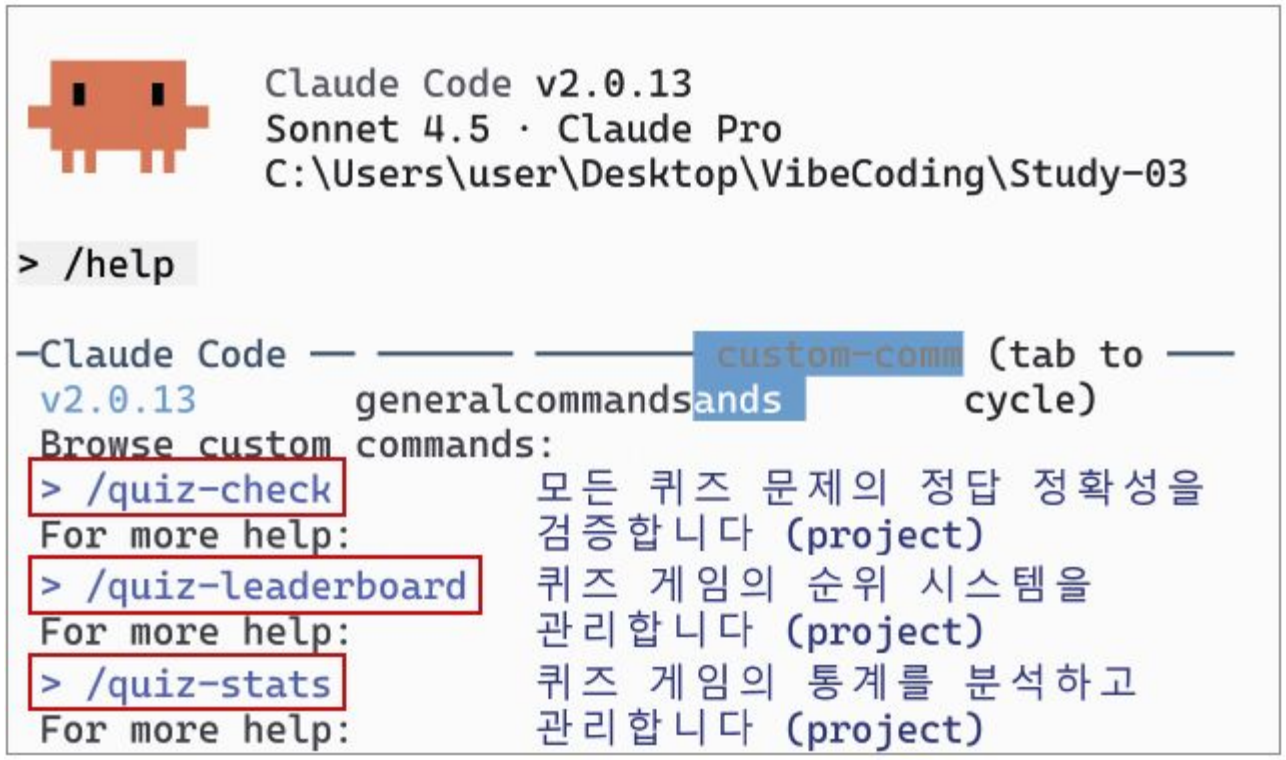
SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(11)

02 세 가지 커스텀 명령어가 생성된 것을 확인

```
● Perfect! 세 개의 새로운 slash command를 생성했습니다.

  ✅ 생성된 명령어

  1. /quiz-check - 문제 정답 검증

  파일: .claude/commands/quiz-check.md

  기능:
  - ✅ 모든 문제의 구조적 검증
  - ✅ 정답 정확성 확인
  - ✅ 설명의 적절성 검토
  - ✅ 의심스러운 문제 식별
  - ✅ 오류 발견 시 즉시 보고
```

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(12)

03 '/exit' 명령어와 'claude' 명령어로 클로드를 재시작
그러고 나서 '/help' 명령어를 통해 새로운 명령어가 잘 생성되었는지 확인



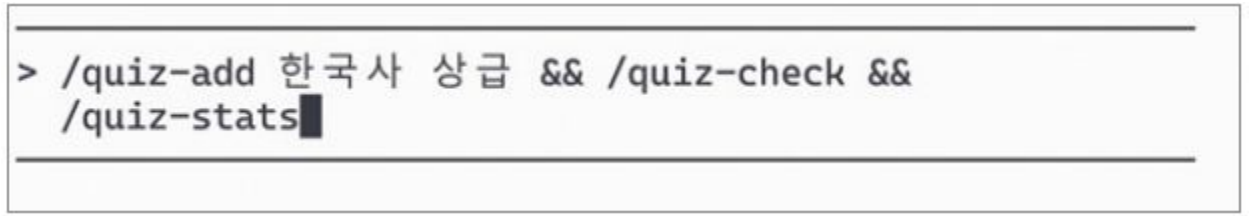
[note] '/help' 명령어에서 빠져나올 때는 [Esc] 키

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(13)

04 '/quiz-add' 명령어로 '한국사' 카테고리에 상급 난이도 퀴즈를 추가하고, '/quiz-check' 명령어와 '/quiz-stats' 명령어를 체이닝 하여 정확성을 검증과 통계 분석까지 이어서 실행

클로드 코드에 다음과 같이 명령어를 입력

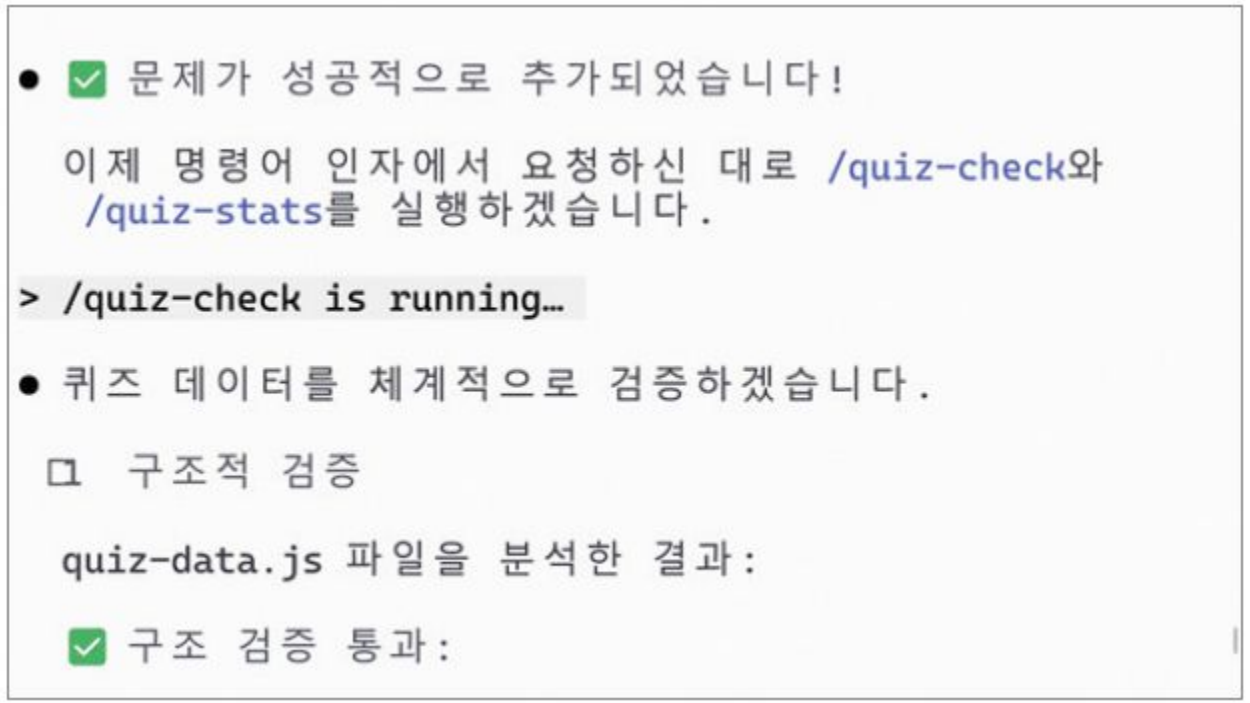
```
/quiz-add 한국사 상급 && /quiz-check && /quiz-stats
```



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(14)

05 '한국사' 카테고리에 새로운 상급 문제를 추가

- '/quiz-check' 명령어가 전체 문제들을 점검해서 정상 작동하고 중복이 없음을 확인하고 동시에 문제 난이도를 정리
- '/quiz-stats' 명령어가 보기 편향 문제를 확인



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(15)

통합 커스텀 명령어로 일일 업데이트 한 번에 끝내기

- 통합 커스텀 명령어
  - 여러 개의 커스텀 명령어를 한 덩어리로 묶어 매일 해야 하는 업데이트, 검증, 백업, 보고를 한 번에 자동으로 수행
  - '/quiz-daily'
    - 각 단계마다 검증하고 오류를 체크하므로 실패 시 즉시 중단·오류 보고를 하고, 성공하면 변경사항이 안전하게 반영

1. 퀴즈 데이터 구조 파악
2. 문항 수·분포 확인
3. 카테고리 결손 파악
4. 추가 전 중복 체크
5. 새 문제 생성 후 형식·규칙 검증
6. 전체 데이터 백업
7. 결과 리포트 출력

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(16)

- 통합 커스텀 명령어를 생성할 때 주의사항
  - 다음 조건을 유지하면서, 할루시네이션 없이 새로운 문제를 매번 업데이트하고 결과를 점검

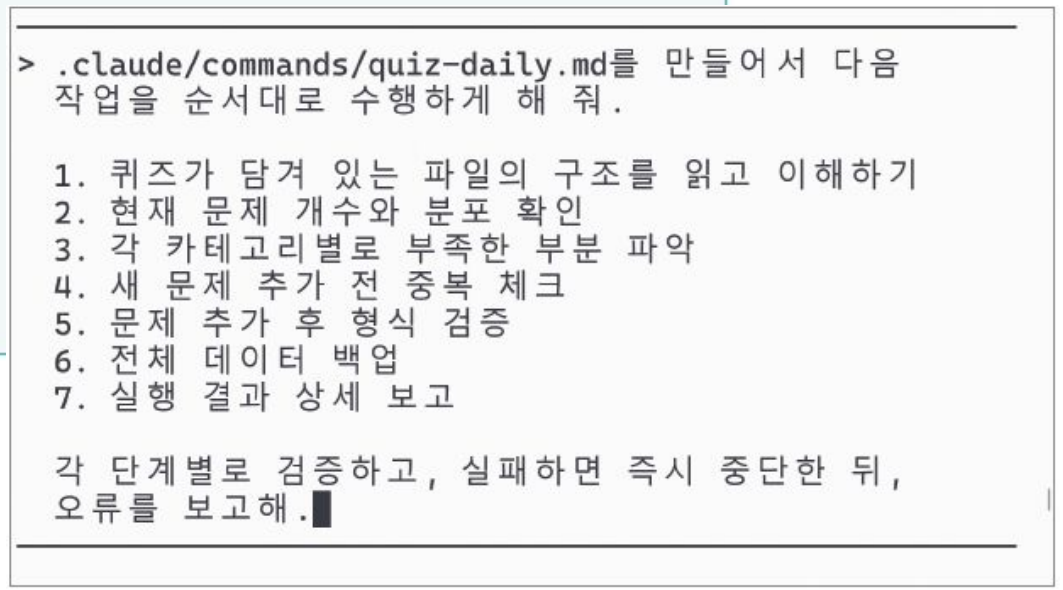
1. 파일 구조를 확인하게 할 것
2. 작은 단위로 테스트해 볼 것
3. 복잡한 작업은 단계별로 나누어 검증할 것

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(17)

01 통합 커스텀 명령어 '/quiz-daily'를 생성하기 위해 클로드 코드에 다음과 같이 입력

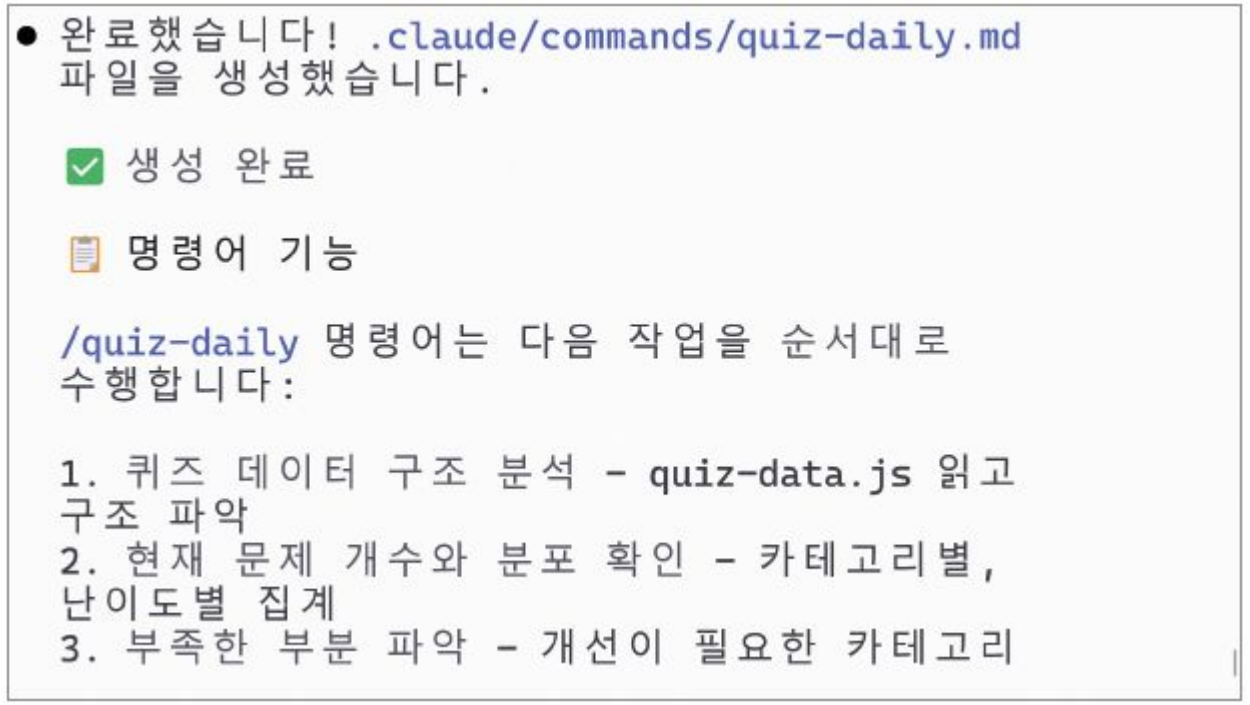
프롬프트

```
.claude/commands/quiz-daily.md를 만들어서 다음 작업을 순서대로 수행하게 해 줘.

1. 퀴즈가 담겨 있는 파일의 구조를 읽고 이해하기
2. 현재 문제 개수와 분포 확인
3. 각 카테고리별로 부족한 부분 파악
4. 새 문제 추가 전 중복 체크
5. 문제 추가 후 형식 검증
6. 전체 데이터 백업
7. 실행 결과 상세 보고

각 단계별로 검증하고, 실패하면 즉시 중단한 뒤, 오류를 보고해.
```



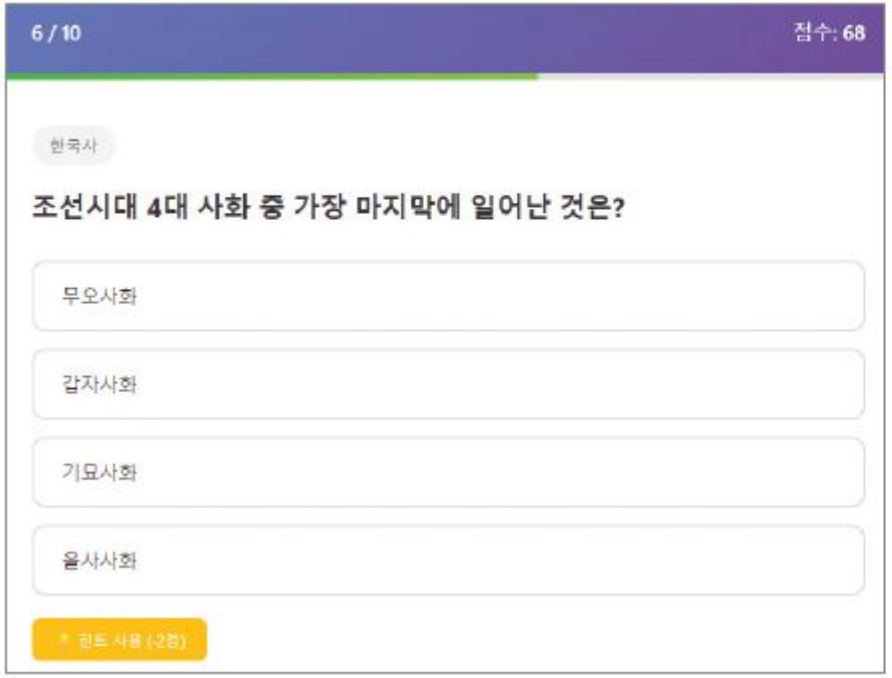
SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(18)

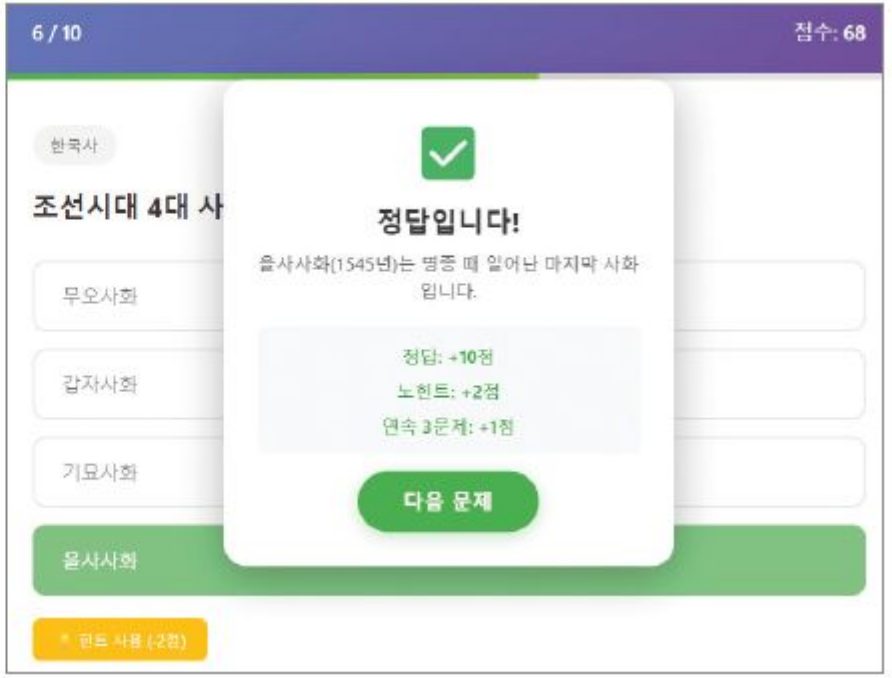
02 실행이 완료되면 '/exit' 명령어와 'claude' 명령어로 클로드 코드를 재시작



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(19)

03 ‘Study-03’ 폴더에서 ‘index.html’을 실행해 ‘/quiz-daily’ 명령어로 일일 업데이트를 진행한 결과를 확인 - 새로운 문제로 업데이트됨





SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(20)

◦ 클로드 코드가 스스로 만드는 커스텀 명령어

- 지금까지는 플레이어의 입장에서 문제를 풀고 성적을 확인하는 데 집중
- 이제 플레이어의 성적을 보고 관리하는 선생님 모드를 추가
  - ‘선생님 모드’는 여러 플레이어의 퀴즈 결과를 자동으로 수집, 분석, 비교하고 보고서로 시각화하는 통합 관리 시스템
- 어떤 커스텀 명령어가 필요한지 스스로 판단해 만들고, 명령어를 통합하여 자동화하는 과정까지 모두 클로드 코드에게 위임

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(21)

01 다음과 같이 입력하여 선생님 모드를 위한 커스텀 명령어들을 생성

프롬프트

이제 퀴즈를 푼 여러 학생들의 성적을 한눈에 보고 비교하는 선생님 모드를 만들고 싶어.
이 기능에 필요한 커스텀 명령들을 직접 구상하고 만들어 줘.
그리고 이 명령어를 하나로 모아서 실행하는 통합 명령어도 만들어 줘.
모든 명령들은 '.claude/commands/' 폴더 안에 각각 .md 확장자로 저장되어야 해.
실행이 끝나면, 생성한 커스텀 명령어들과 그 기능을 나열해서 보고해.

```
> 이제 퀴즈를 푼 여러 학생들의 성적을 한눈에 보고
  비교하는 선생님 모드를 만들고 싶어.
  이 기능에 필요한 커스텀 명령들을 직접 구상하고
  만들어 줘.
  그리고 이 명령어를 하나로 모아서 실행하는 통합
  명령어도 만들어 줘.
  모든 명령들은 '.claude/commands/' 폴더 안에 각각
  .md 확장자로 저장되어야 해.
  실행이 끝나면, 생성한 커스텀 명령어들과 그 기능을
  나열해서 보고해.
```

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(22)

✚ 여기서 잠깐 **명령어를 문서로 남기는 이유**

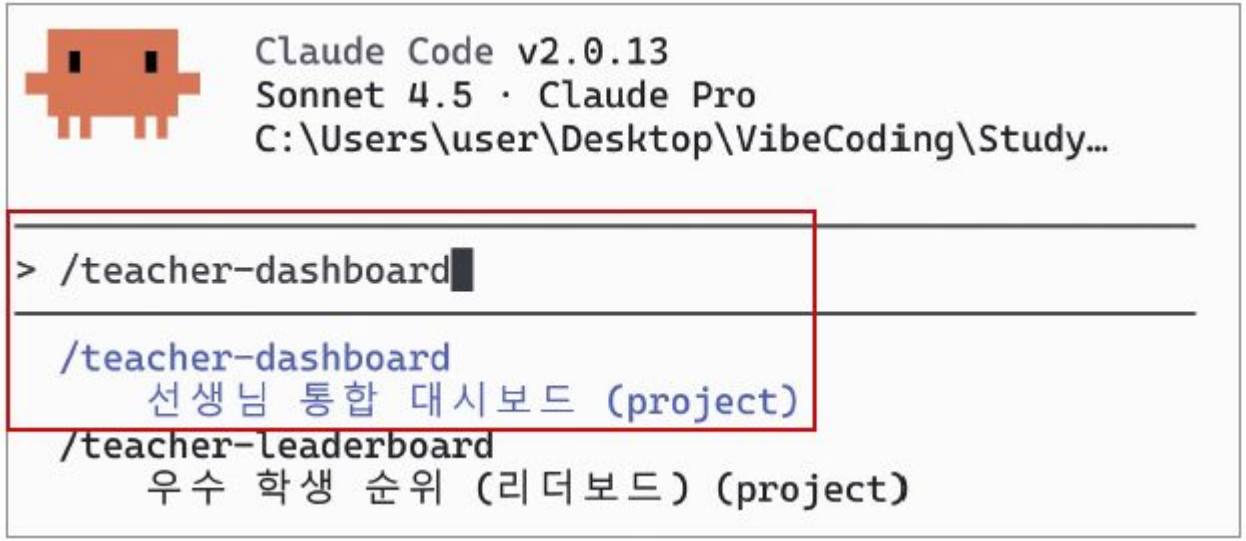
- 커스텀 명령어를 생성할 때는 각 명령어가 '.claude/commands/' 폴더 안에 각각 .md 파일로 저장하는 것이 중요
- 이렇게 하면 명령어의 동작 방식과 수정 내역이 모두 문서 형태로 기록되어 나중에 특정 기능만 업데이트하거나 개선할 때 해당 명령어 파일만 수정하면 됨
- 결과적으로 전체 프로젝트를 유지하면서도 일부 기능만 안전하게 교체 및 업데이트할 수 있어 유지보수와 버전 관리가 훨씬 효율적

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(23)

02 실행이 완료되면 클로드 코드가 보여주는 보고서를 꼼꼼히 확인
여기서는 '/teacher-dashboard'라는 통합 명령어로 선생님 모드를 자동 실행

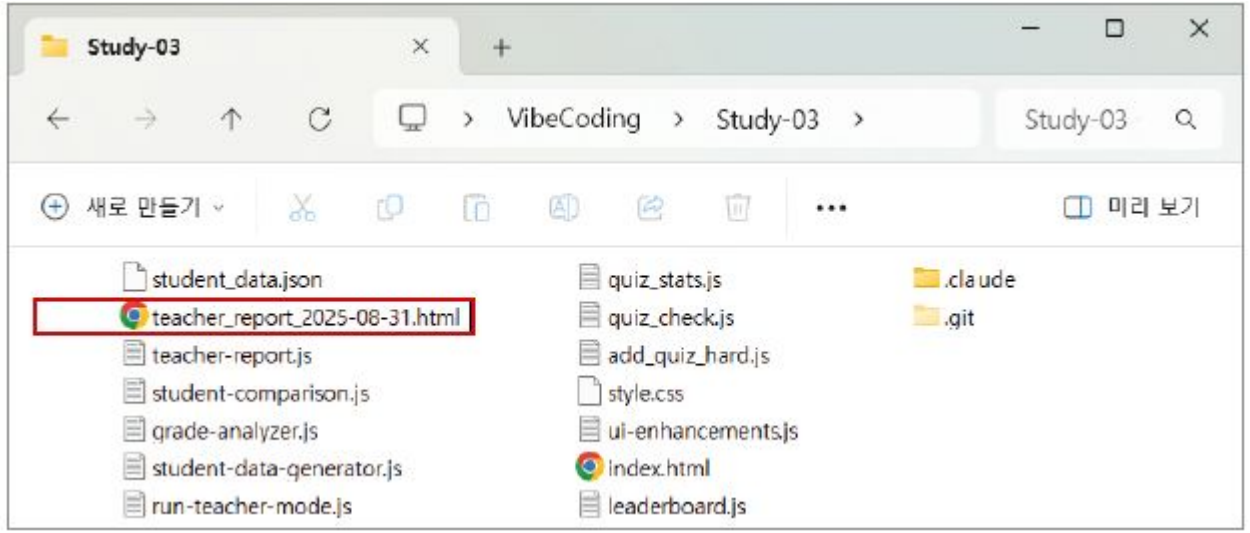
SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(24)

03 '/exit' 명령어와 'claude' 명령어를 실행해 클로드 코드를 재실행한 뒤,
새로 생성한 통합 커스텀 명령어 '/teacher-dashboard'를 입력



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(25)

04 실행이 완료되면 ‘Study-03’ 폴더에 생성된 선생님 모드 대시보드 파일 ‘teacher_report.html’을 확인



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(26)

05 선생님을 위한 퀴즈 성적 종합 보고서 페이지를 완성

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(27)

- 효율적인 유지보수 워크플로 구축하기
  - AI로 개발을 하다 보면 특정 부분만 수정하고 싶은데도 전체를 다시 만들어야 하는 상황이 자주 발생
    - 시스템이 기능별로 분리되지 않고 하나로 통합되어 있기 때문임
  - 커스텀 명령어를 각각 독립된 .md 파일로 관리해 두면, 필요한 부분만 선택적으로 업데이트하면서도 전체 구조는 그대로 유지한 채 효율적으로 개선

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(28)

기존 커스텀 명령만 업데이트하기

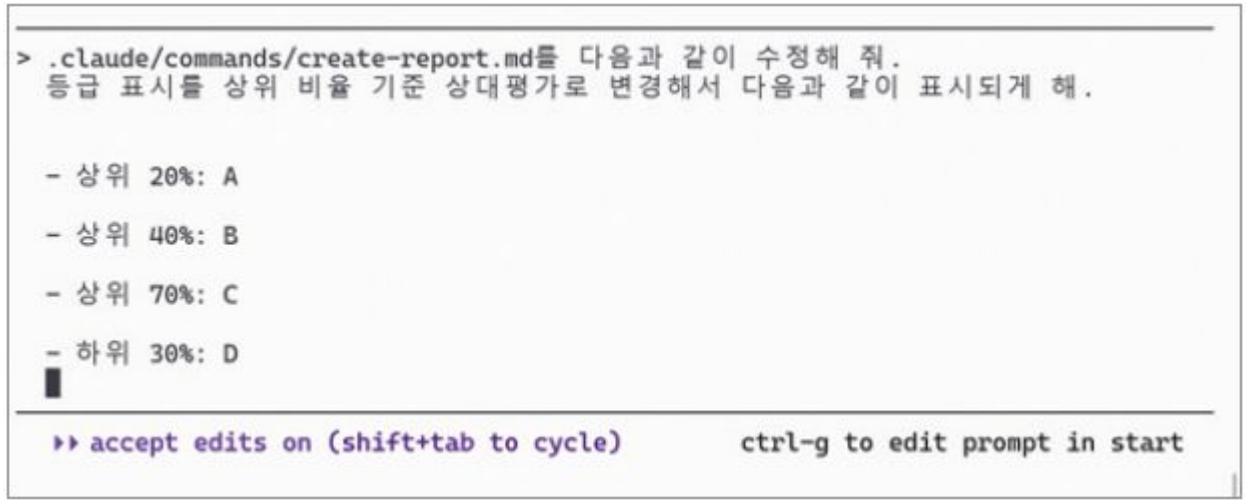
- 선생님 모드에서 보여준 대시보드에는 '등급' 열이 있는데, 모든 플레이어의 등급이 'C'로 분류됨
  - 등급을 분류하는 커스텀 명령어 '/create-report'가 평균 점수를 기준으로 절대 평가했기 때문임
- 커스텀 명령어 '/create-report'가 플레이어의 등급을 상대적으로 평가하게 수정

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(29)

01 커스텀 명령어 '/create-report'를 수정하기 위해 클로드 코드에 다음과 같이 입력

프롬프트

```
.claude/commands/create-report.md를 다음과 같이 수정해 줘.
등급 표시를 상위 비율 기준 상대평가로 변경해서 다음과 같이 표시되게 해.
- 상위 20%: A
- 상위 40%: B
- 상위 70%: C
- 하위 30%: D
```



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(30)

02 클로드 코드를 재시작하고, '/create-report'를 실행해 플레이어의 등급을 재평가

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(31)

선생님 모드 보고서 파일로 내보내기

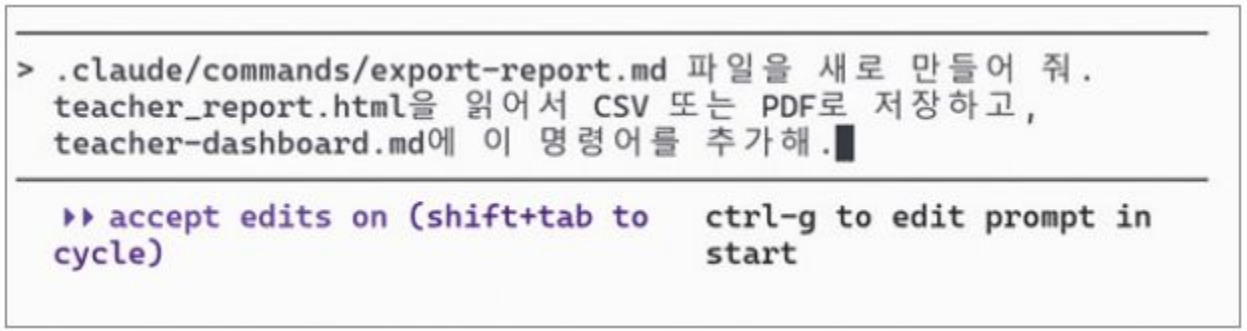
- 전체 명령어에 새로운 커스텀 명령어를 추가해서 일부 기능만 확장
- 대시보드 데이터를 외부 파일로 내보내는 커스텀 명령어를 추가

SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(32)

01 선생님 모드로 생성한 대시보드를 파일로 내보내는 새로운 커스텀 명령어 '/export-report'를 생성하기 위해 클로드 코드에 다음과 같이 입력

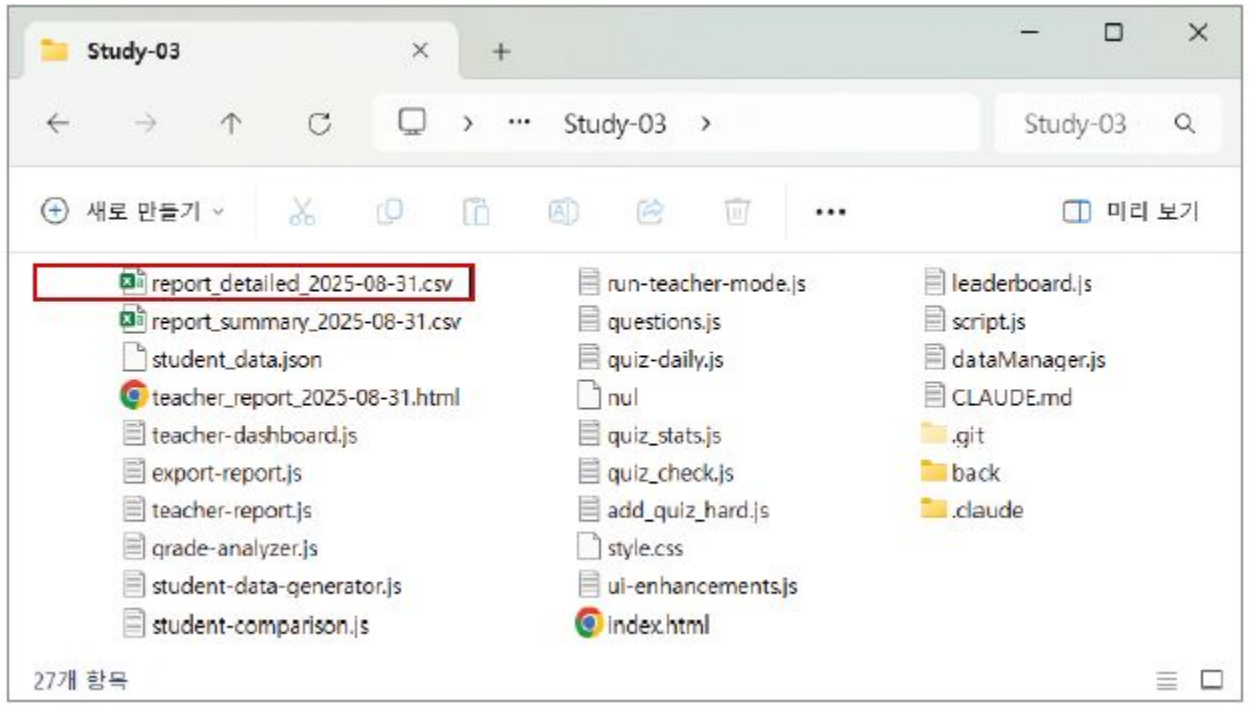
프롬프트

```
.claude/commands/export-report.md 파일을 새로 만들어 줘.
teacher_report.html을 읽어서 CSV 또는 PDF로 저장하고,
teacher-dashboard.md에 이 명령어를 추가해.
```



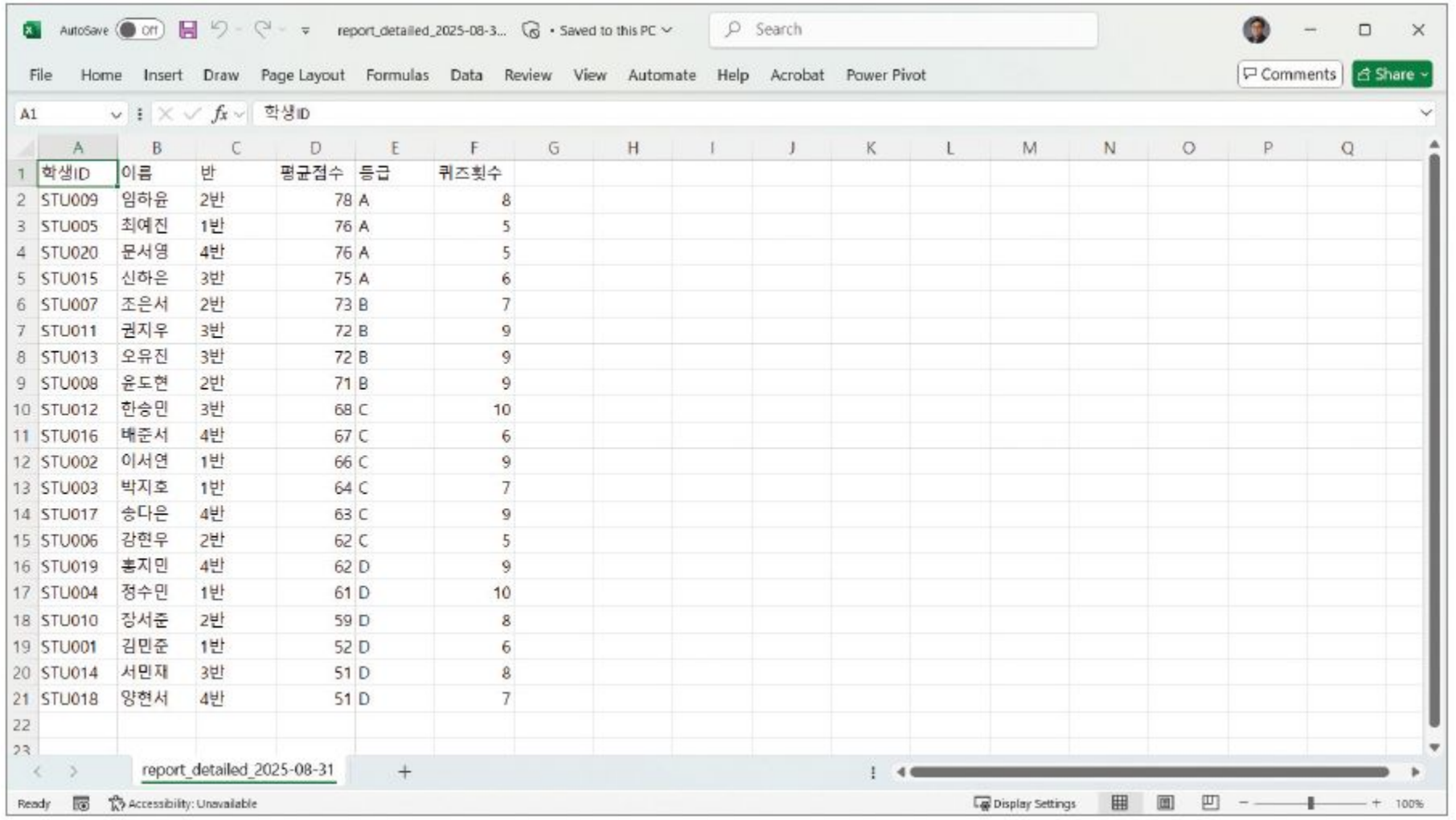
SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(33)

02 새로운 명령어 '/export-report'를 생성하고 나서, 클로드 코드를 재시작한 뒤, '/teacher-dashboard' 명령어를 실행 - 'Study-03' 폴더에 새로운 파일 'report_detailed.csv'가 생성



SECTION 5-3 명령어 체이닝을 통해 유지보수 전략(34)

03 CSV 파일을 더블 클릭해 열어 보면 학생들의 평가 보고서가 정리되어 있는 것을 확인



학생ID	이름	반	평균점수	등급	퀴즈횟수
STU009	임하윤	2반	78	A	8
STU005	최예진	1반	76	A	5
STU020	문서영	4반	76	A	5
STU015	신하은	3반	75	A	6
STU007	조은서	2반	73	B	7
STU011	권지우	3반	72	B	9
STU013	오유진	3반	72	B	9
STU008	윤도현	2반	71	B	9
STU012	한승민	3반	68	C	10
STU016	배준서	4반	67	C	6
STU002	이서연	1반	66	C	9
STU003	박지호	1반	64	C	7
STU017	송다은	4반	63	C	9
STU006	강현우	2반	62	C	5
STU019	홍지민	4반	62	D	9
STU004	정수민	1반	61	D	10
STU010	장서준	2반	59	D	8
STU001	김민준	1반	52	D	6
STU014	서민재	3반	51	D	8
STU018	양현서	4반	51	D	7

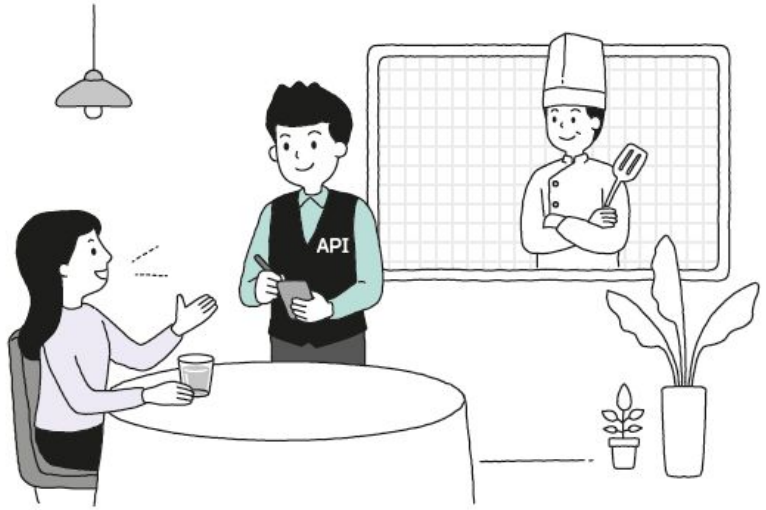
[마무리]

- 3가지 키워드로 정리하는 핵심 포인트
  - **유지보수**는 완성된 프로젝트를 꾸준히 관리하고 개선하는 과정으로, 기능을 모듈화해 두면 필요한 부분만 선택적으로 수정하면서 전체 구조는 그대로 유지
  - **명령어 체이닝**은 연산자 **&&**를 사용해 여러 커스텀 명령어를 순서대로 연결 실행해 반복 작업을 자동화
  - **통합 커스텀 명령어**는 여러 작업을 하나의 명령어로 묶어 한 번에 처리할 수 있게 해 주며, AI가 스스로 관리, 운영하는 지속 가능한 개발 환경을 만들어 줌

SECTION 6-1 클로드 코드에서 API 설정하기(1)

시작하기전에

- 레스토랑에서 요리를 맛보려면 주방장에게 직접 이야기하는 것이 아니라 웨이터에게 주문
- 소프트웨어 개발에서도 마찬가지
  - 복잡한 시스템을 직접 구축하는 것이 아니라, API에게 요청해서 필요한 결과만 추출
  - 클로드 코드를 사용하면 복잡한 연결 코드를 직접 작성할 필요 없이, 클로드 코드가 API 설정부터 오류 처리까지 모든 과정을 자동으로 처리
- 클로드 코드와 함께 API를 활용한 실제 애플리케이션 만들기



SECTION 6-1 클로드 코드에서 API 설정하기(2)

◦ API의 개념과 활용 방법

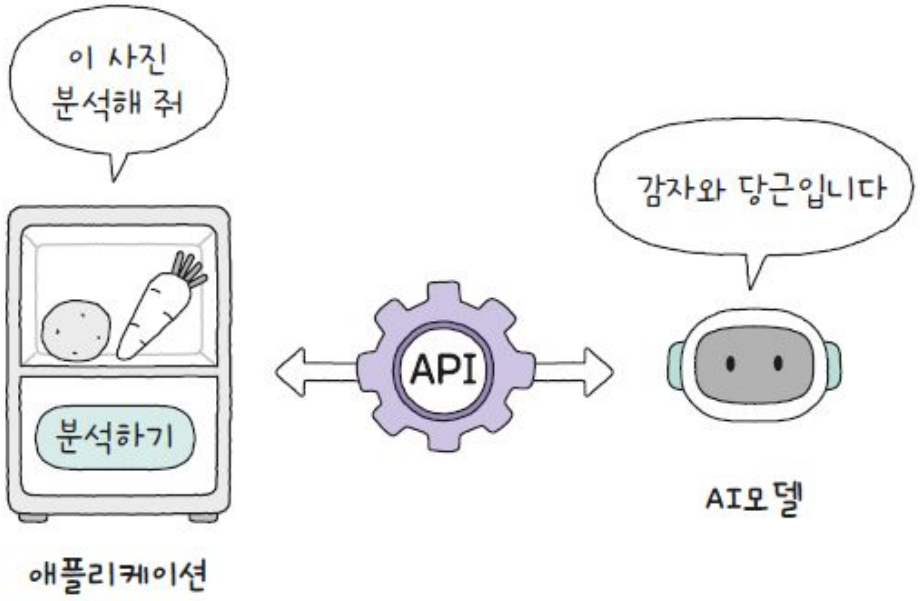
- 앞에서 우리가 만든 웹 포트폴리오와 퀴즈 게임의 공통점
  - 웹 포트폴리오에 소개될 내 정보나, 퀴즈 게임의 문제들은 '모두 미리 준비된 데이터'
- 만약, 다음과 같은 애플리케이션을 만들게 된다면 어떨까?

사용자가 냉장고 사진을 업로드하면 , 이미지 속 식재료를 자동으로 인식하고 그 재료를 활용한 레시피를 추천해 주는 프로그램

  - 이 애플리케이션은 준비해 둔 정보만으로는 운영할 수 없음
    - 어떤 냉장고 사진이 업로드될지 알 수 없기 때문임
  - 이미지를 읽고 해석하는 비전 AI(vision AI)모델과 해석 결과를 바탕으로 레시피를 추천하는 생성 AI 모델이 필요
- '이 모델들을 애플리케이션에 어떻게 통합할 것인가'에 유용한 것이 바로 API

SECTION 6-1 클로드 코드에서 API 설정하기(3)

- API(Application Programming Interface)
  - 프로그램과 AI 모델을 이어주는 다리
  - 마치 전화로 전문가에게 도움을 요청하듯, 프로그램이 AI 모델에게 사진을 분석해 달라고 요청하면 AI가 이미지를 해석하고 결과를 돌려줌
  - 이후 애플리케이션은 해당 정보를 화면에 표시하거나, '이 재료로 레시피를 만들어 줘'와 같은 추가 요청을 AI 모델 API에 전달해 결과를 받아볼 수도 있음
  - 즉 웹 애플리케이션은 사용자가 사진을 업로드할 인터페이스만 제공하고, 이미지 인식과 레시피 생성 같은 핵심 기능은 AI 모델 API가 작업



SECTION 6-1 클로드 코드에서 API 설정하기(4)

✚ 여기서 잠깐 **API를 사용하지 않고 AI 모델을 직접 만들 수도 있지 않나요?**

- 앞서 만들었던 손글씨 인식 모델을 만들었던 것처럼, 원하는 AI 모델을 직접 개발할 수도 있음
- 하지만 이미지 인식이나 텍스트 생성 같은 복잡한 AI를 만들려면 방대한 양의 학습 데이터와 고성능 컴퓨터 자원이 필요
- 특히 생성형 언어 모델을 만드는 것은 클로드나 챗GPT를 개발하는 것만큼이나 어렵고 많은 비용이 필요
- 따라서 이미 구축된 AI 모델을 API를 통해 활용하는 방식이 훨씬 효율적
- 완성된 고성능 AI 모델을 바로 사용할 수 있기 때문에, 제한된 시간과 자원 안에서 프로젝트를 완성해야 하는 상황이라면 API는 가장 현실적인 선택

SECTION 6-1 클로드 코드에서 API 설정하기(5)

◦ AI 모델 선택하기

- 다양한 API
  - 날씨를 예측하는 API, 뉴스를 수집하는 API, 지도를 제공하는 API 등
- 클로드도 API를 제공
  - 클로드의 API를 연결하면 사용자가 레시피를 요청할 때마다 클로드가 직접 대답하는 것처럼 결과를 받음
  - 클로드나 챗GPT처럼 유료로 운영되는 대형 AI 서비스의 경우, API를 사용할 때도 사용량에 따라 요금이 부과
- 학습 목적으로 충분한 성능을 제공하는 무료 API
  - 오픈라우터(OpenRouter) - 여러 AI 모델 API를 한곳에 모아 제공하는 API 통합 플랫폼
  - 유료 또는 무료 API를 한눈에 비교하고 원하는 API를 쉽게 찾아보고 내 프로젝트에 맞는 모델을 골라 사용

SECTION 6-1 클로드 코드에서 API 설정하기(6)

✚ 여기서 잠깐 **오픈라우터를 사용하는 이유**

AI 모델을 사용하는 방법 2 가지

1) 오픈라우터 같은 API 통합 플랫폼을 사용
   - 마치 푸드코트에서 여러 식당의 음식을 한 곳에서 주문하듯, 오픈라우터에서는 클로드(Claude), GPT,
     라마(Llama) 등 다양한 AI 모델을 한곳에서 제공
   - API 키를 발급받기만 하면 모든 모델을 자유롭게 전환해 쓸 수 있고, 사용량도 한 화면에서 관리할 수 있어 매우 편리
2) 각 AI 회사의 공식 API를 직접 사용
   - 예) 오픈AI에서 GPT API를, 앤트로픽에서 클로드 API를 직접 연동
   - 이 경우 회사마다 회원가입을 하고 별도의 API 키를 발급받아야 하므로 관리가 다소 번거롭지만, 최신 기능을 가장 빠르게 이용

- 초보자라면 오픈라우터로 시작하는 것을 추천
  - 여러 모델을 쉽게 비교해 볼 수 있고, 무료 모델로 체험할 수 있는 모델도 많음
  - 다만 무료 사용의 경우, 이 글을 쓰는 2025년 11월 기준 분당 20회, 하루 50회의 요청만 가능
  - 특정 모델을 꾸준히 사용하게 된다면, 그때 회사의 공식 API를 직접 연동해 보는 것도 좋은 선택

SECTION 6-1 클로드 코드에서 API 설정하기(7)

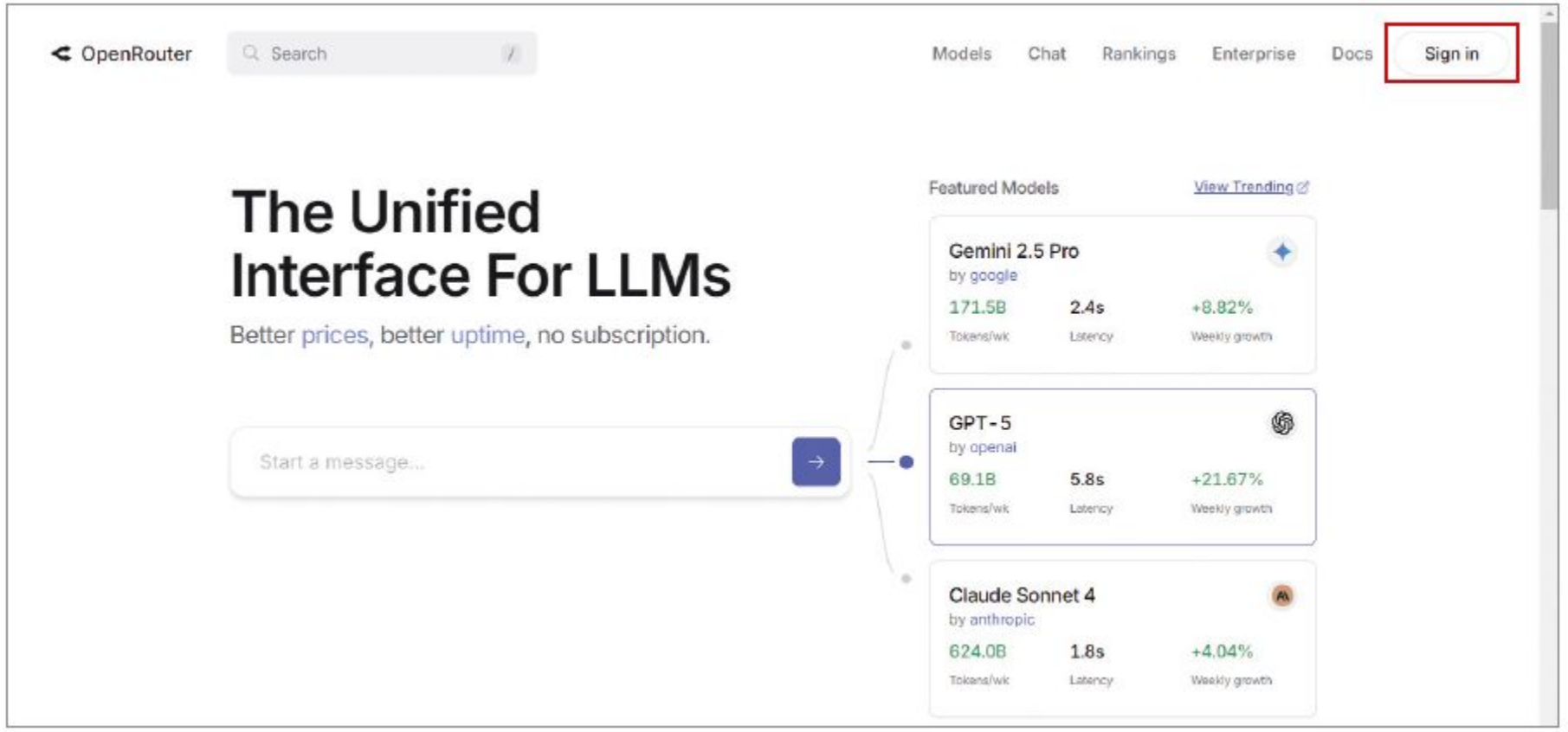
API 키 발급하기

- API 키(API key) - API를 사용할 때 사용자를 인증하는 비밀 열쇠
  - API를 안전하고 공정하게 사용하기 위한 인증과 관리의 핵심 도구
  - API를 제공하는 서버는 이 키를 통해 누가 요청을 보냈는지, 그리고 그 요청이 허가된 사용자에게서 온 것인지를 확인
- API 키가 없다면 누구나 서버에 접근할 수 있게 되어 데이터 유출의 위험이 커짐
  - 또한 서버는 API 키를 이용해 사용자의 호출 빈도나 사용량을 기록하여 과도한 요청을 제한하거나 요금 부과를 관리하는 데도 활용
- 오픈라우터에서 API를 사용할 때도 API 키가 필요

SECTION 6-1 클로드 코드에서 API 설정하기(8)

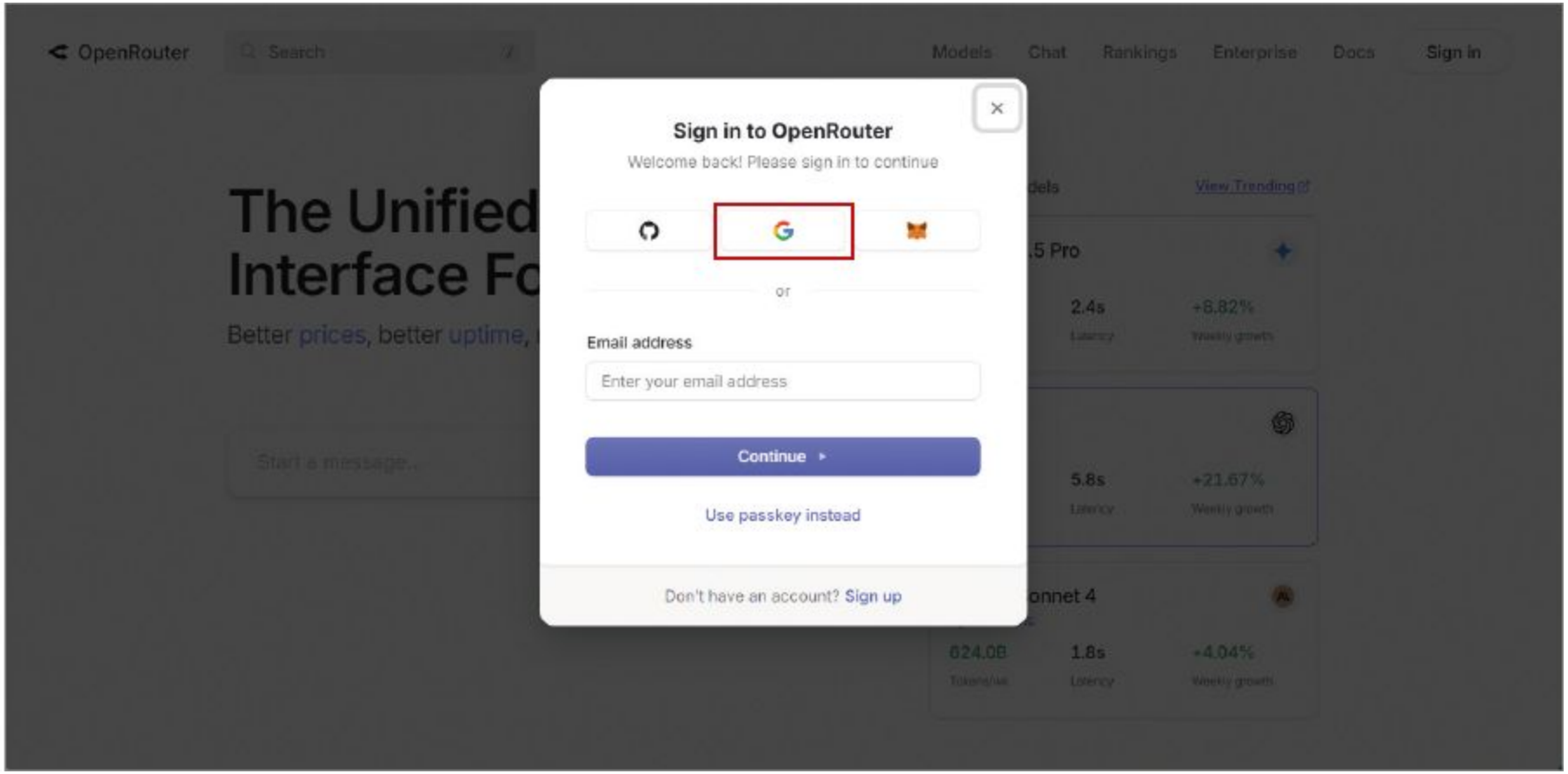
- API 키를 발급을 위해 'VibeCoding' 폴더에 'Study-04' 폴더를 새로 생성

01 웹 브라우저에서 오픈라우터에 접속한 뒤, 홈페이지 상단의 [Sign in]을 클릭해 회원가입을 시작
openrouter.ai



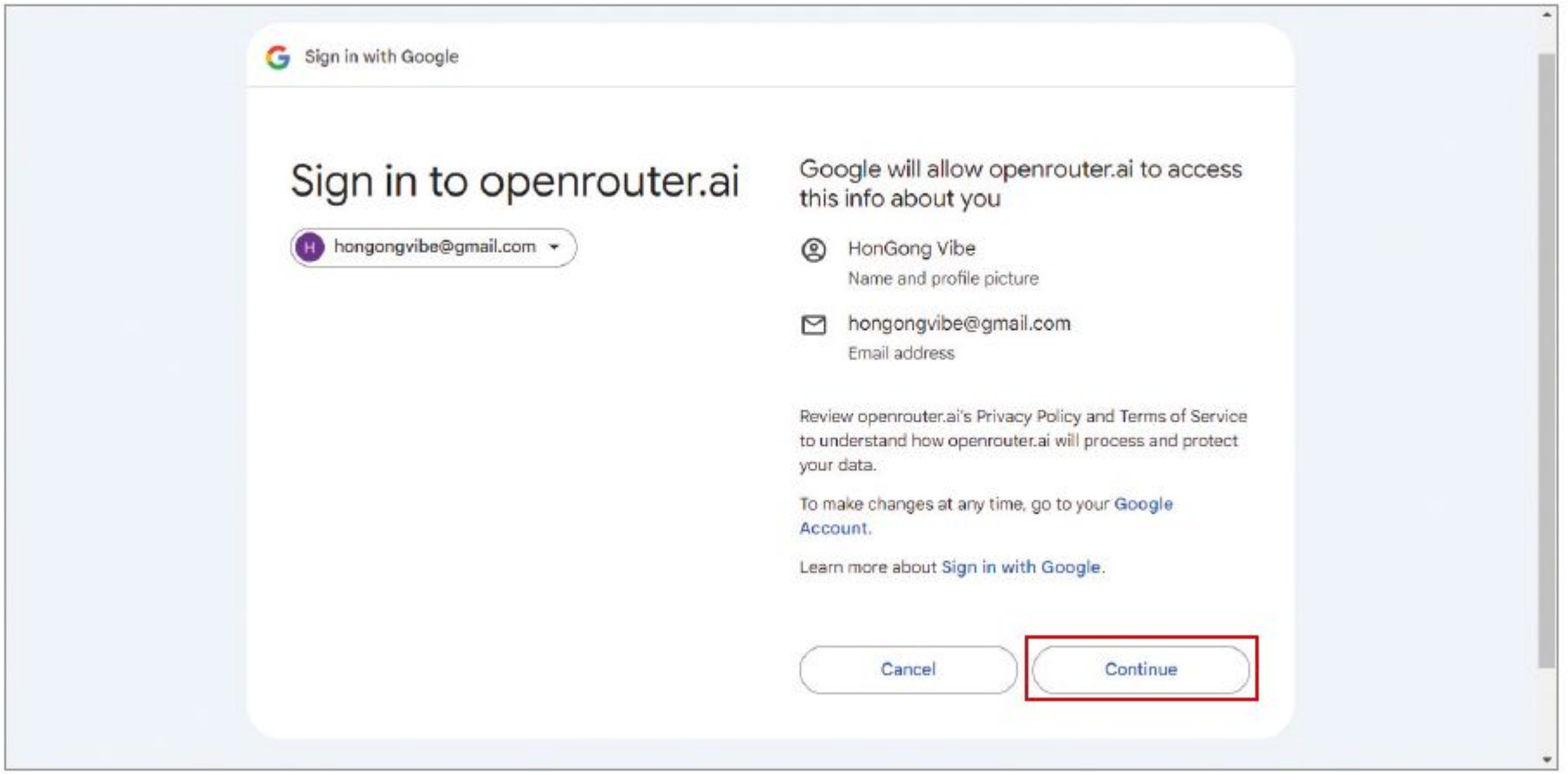
SECTION 6-1 클로드 코드에서 API 설정하기(9)

02 구글 계정 연동을 통해 가입 - 가운데 구글 로고를 클릭



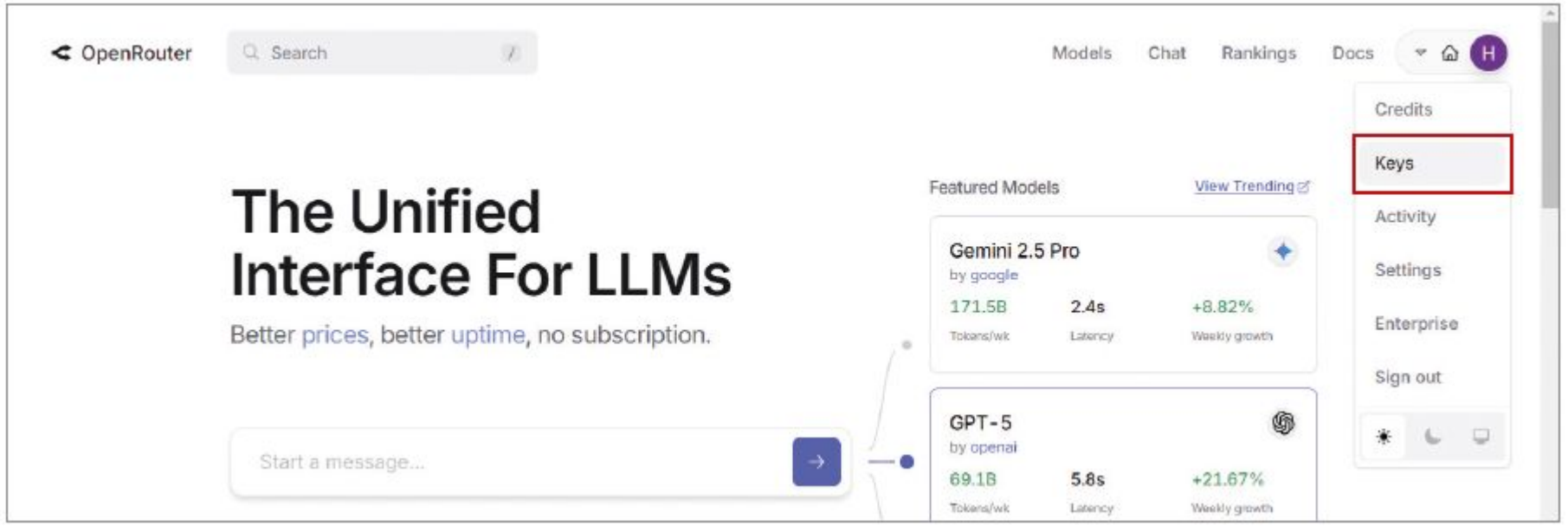
SECTION 6-1 클로드 코드에서 API 설정하기(10)

03 구글 아이디를 입력한 뒤 Continue 버튼을 클릭해 가입을 진행
액세스 약관에 동의하고 Continue를 눌러 회원 가입을 마침



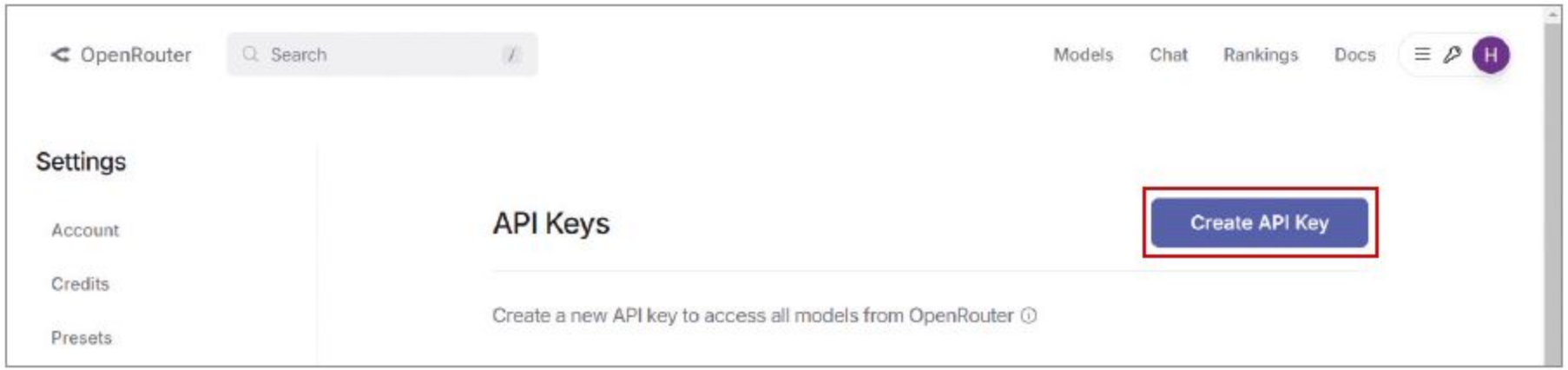
SECTION 6-1 클로드 코드에서 API 설정하기(11)

04 오픈라우터의 초기 화면으로 돌아가 API 키를 등록하기 위해 우측 상단의 프로필 아이콘을 클릭한 뒤 [Keys] 메뉴를 선택



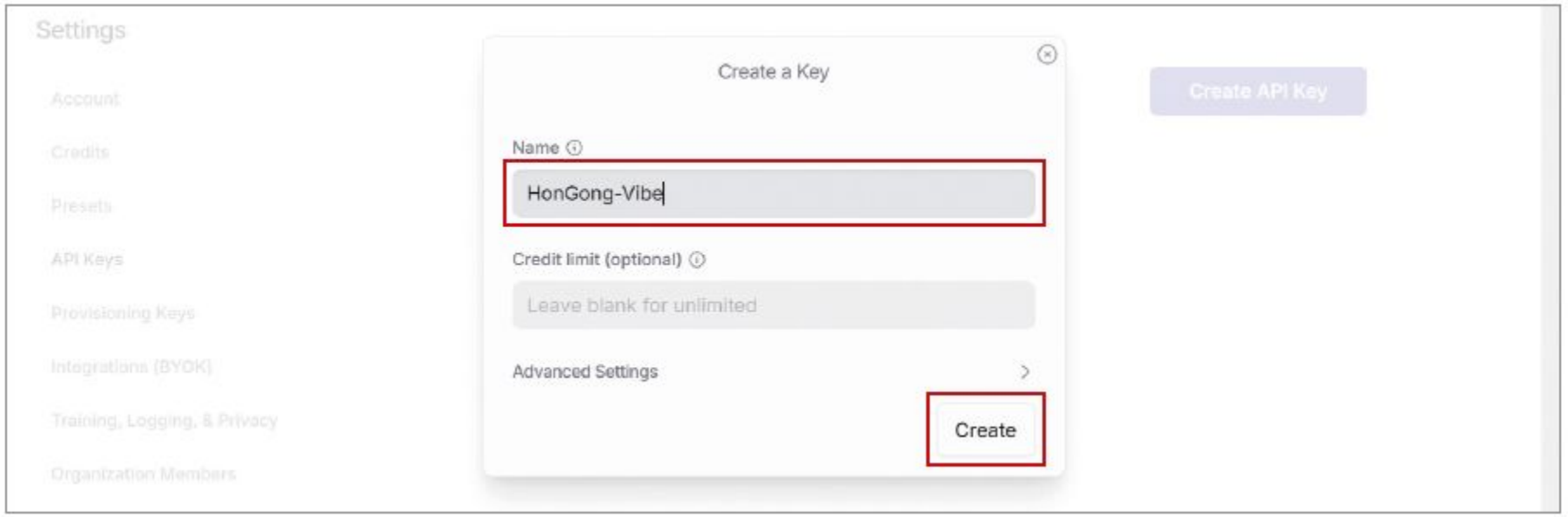
SECTION 6-1 클로드 코드에서 API 설정하기(12)

05 [Create API Key] 버튼을 클릭



SECTION 6-1 클로드 코드에서 API 설정하기(13)

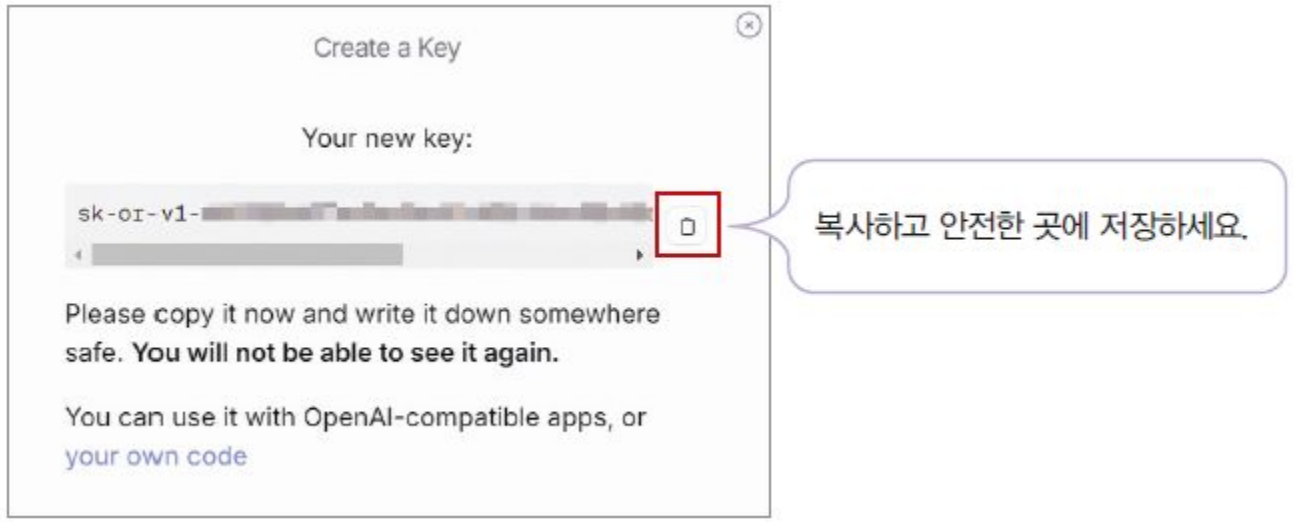
06 'Name' 입력란에 'HonGong-Vibe'라고 이름을 입력
[Create] 버튼을 클릭하면 API 키를 발급



SECTION 6-1 클로드 코드에서 API 설정하기(14)

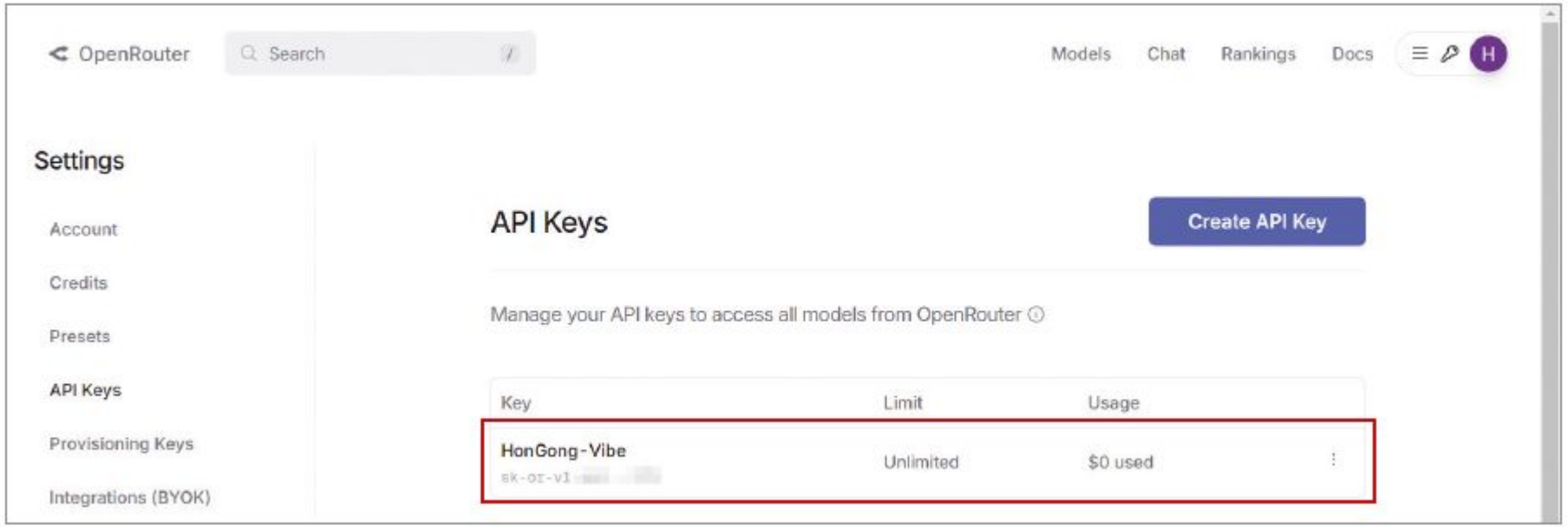
07 실제 API 키가 화면에 표시

- 이 API 키는 보안상의 이유로 단 한 번만 확인 가능
- 페이지를 벗어나거나 새로고침하면 다시 볼 수 없으므로, 반드시 안전한 곳에 복사해서 저장
- 화면의 [(복사)] 아이콘을 클릭한 뒤, 메모장이나 텍스트 파일을 열어 붙여넣기하고 저장
- 나중에 충전 후 유료 API를 사용할 때 필요한 열쇠이므로 다른 사람에게 노출되지 않도록 주의 깊게 관리



SECTION 6-1 클로드 코드에서 API 설정하기(15)

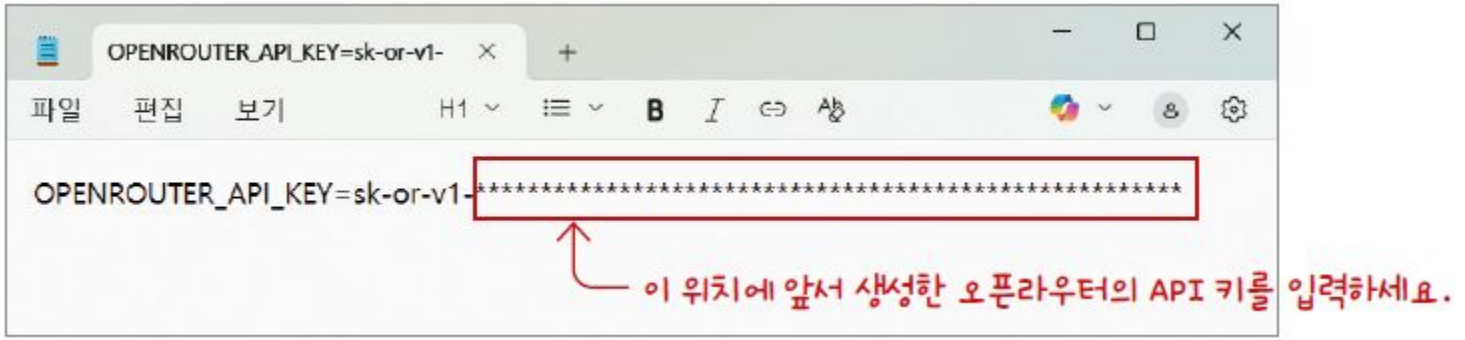
08 API 키가 안전하게 생성



SECTION 6-1 클로드 코드에서 API 설정하기(16)

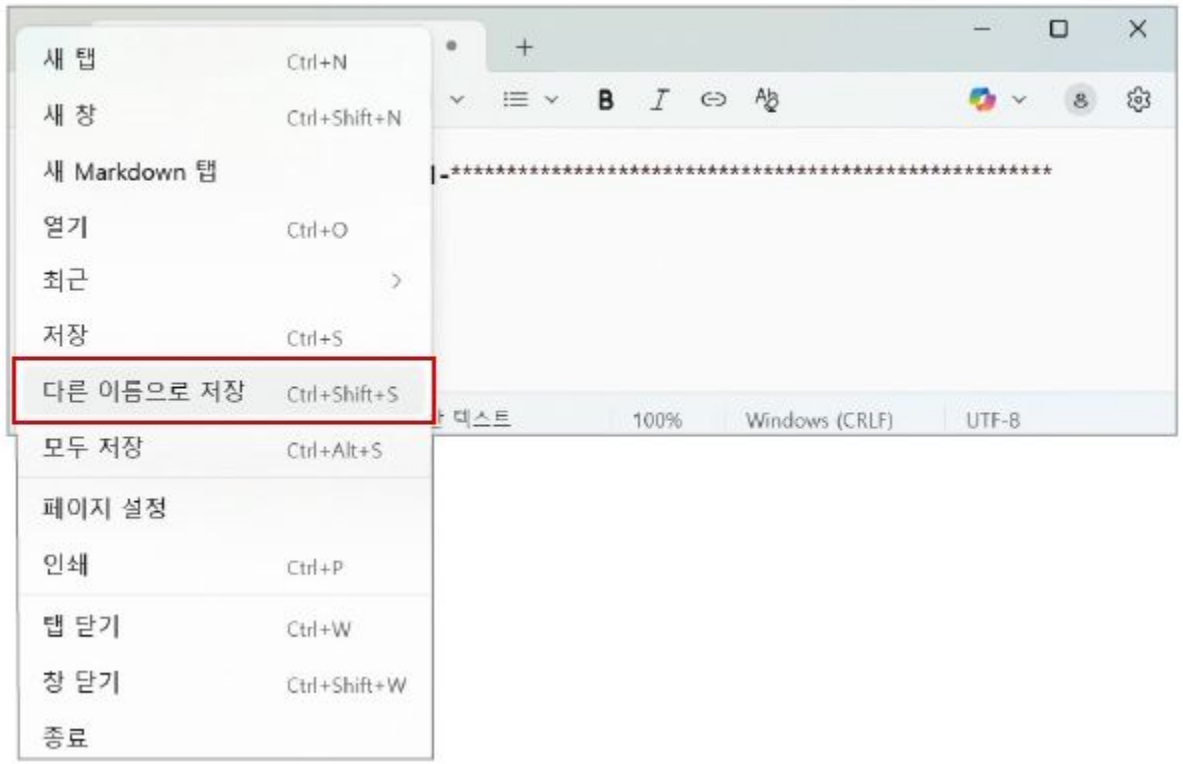
09 윈도우 기본 텍스트 편집기인 [메모장]을 실행한 뒤, 'OPENROUTER_API_KEY='라고 입력하고 '=' 기호 뒤에 앞서 발급받은 오픈라우터의 API 키를 붙여넣기

```
OPENROUTER_API_KEY=〈 오픈라우터 API 키 〉
```



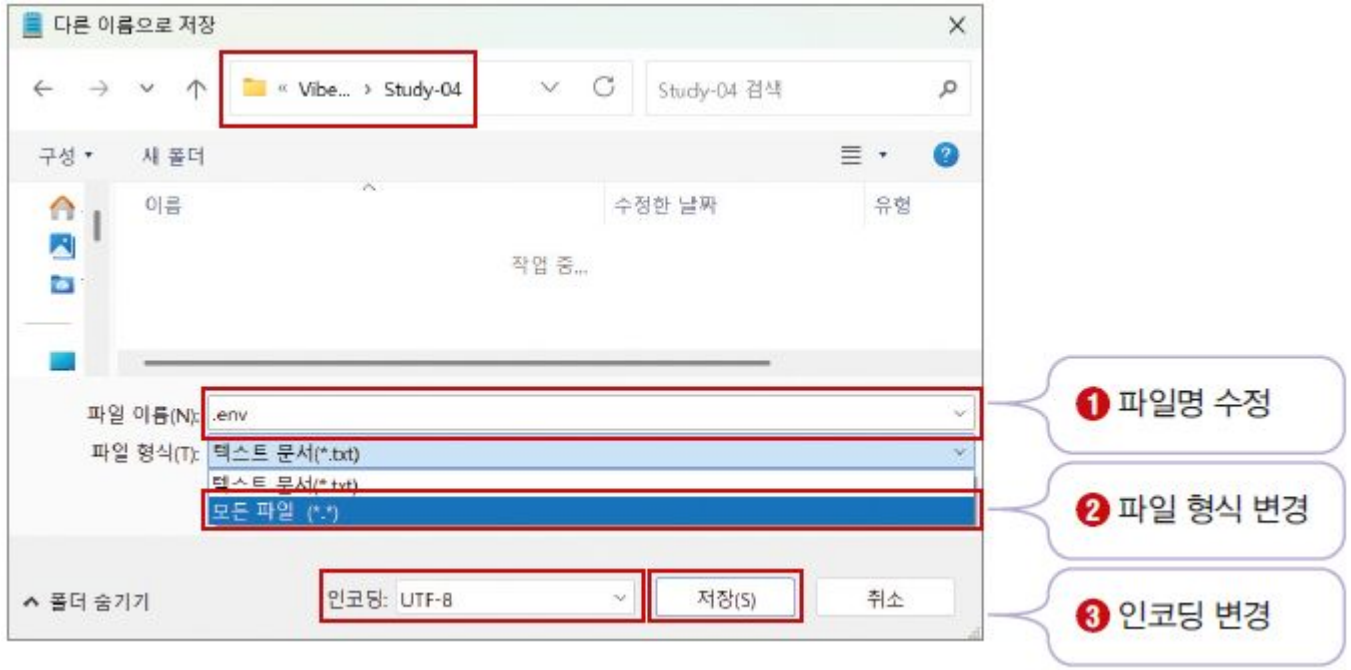
SECTION 6-1 클로드 코드에서 API 설정하기(17)

10 [파일]-[다른 이름으로 저장] 메뉴를 클릭



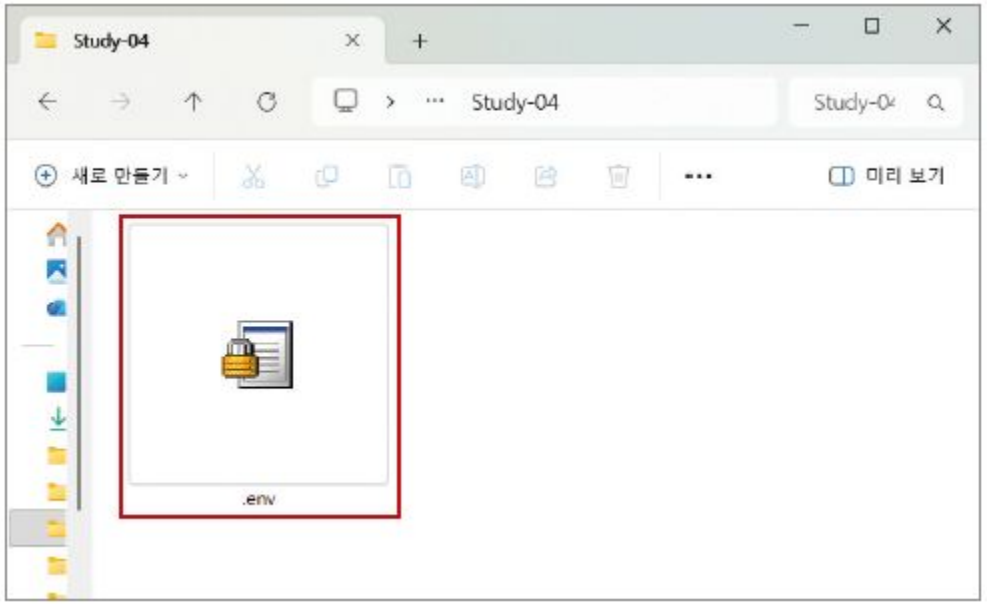
SECTION 6-1 클로드 코드에서 API 설정하기(18)

11 'Study-04' 폴더로 이동한 뒤, [파일 이름]에 '.env' 라고 입력
[파일 형식]을 반드시 [모든 파일 (*.*)]로 변경
하단의 [인코딩]은 [UTF-8]을 선택하고 [저장] 버튼을 클릭



SECTION 6-1 클로드 코드에서 API 설정하기(19)

12 ‘Study-04’ 폴더에 오픈라우터 API 키가 저장된 ‘.env’ 파일이 생성된 것을 확인



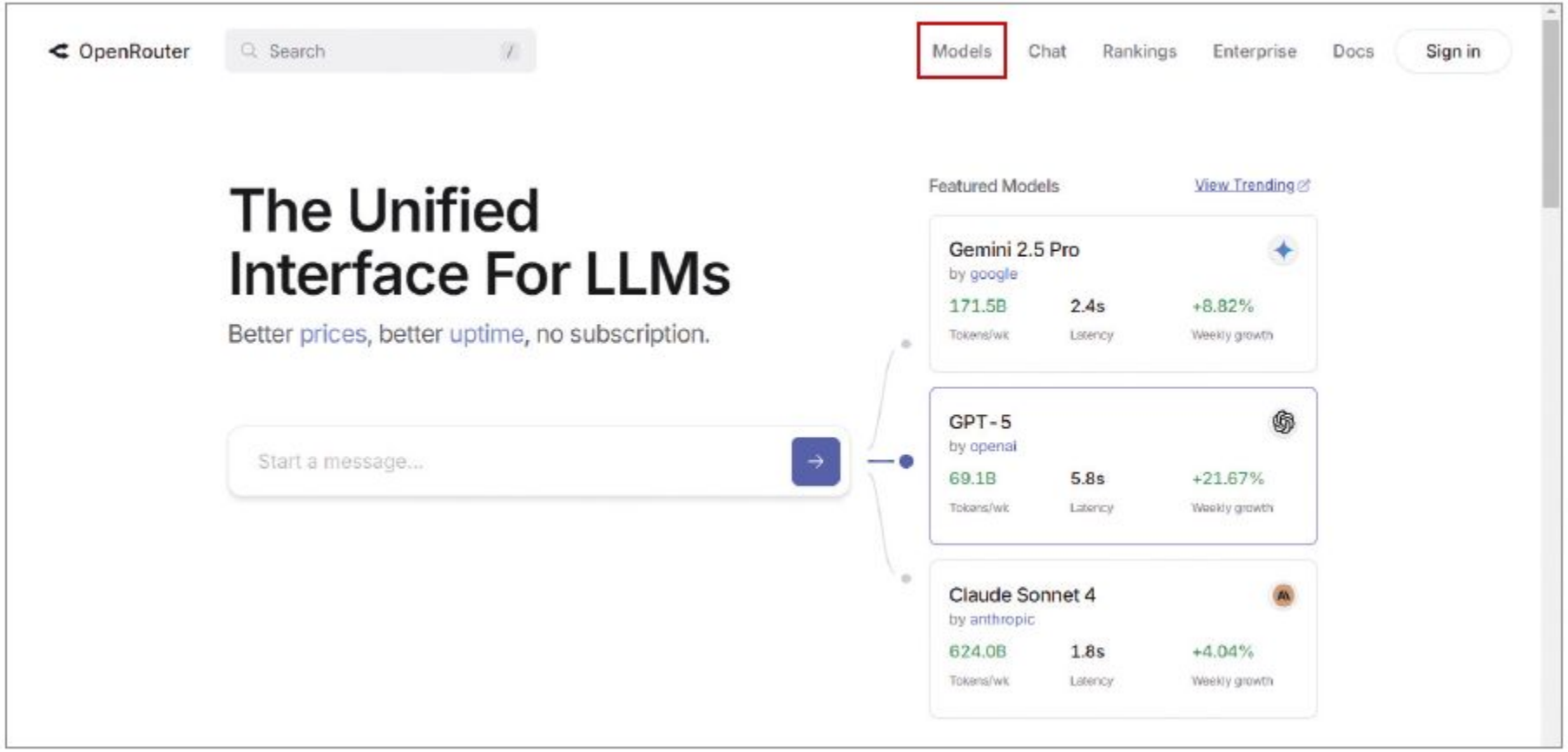
SECTION 6-1 클로드 코드에서 API 설정하기(20)

무료 AI 모델 탐색

- '냉장고를 부탁해' 앱에 필요한 모델은
  1) 이미지를 인식하는 모델
  2) 레시피를 추천하는 생성형 언어 AI모델
- 오픈라우터가 제공하는 AI 모델 중 목적에 부합하면서 무료로 제공되는 것을 필터링하고, 적절한 성능을 보여주는 것을 선택

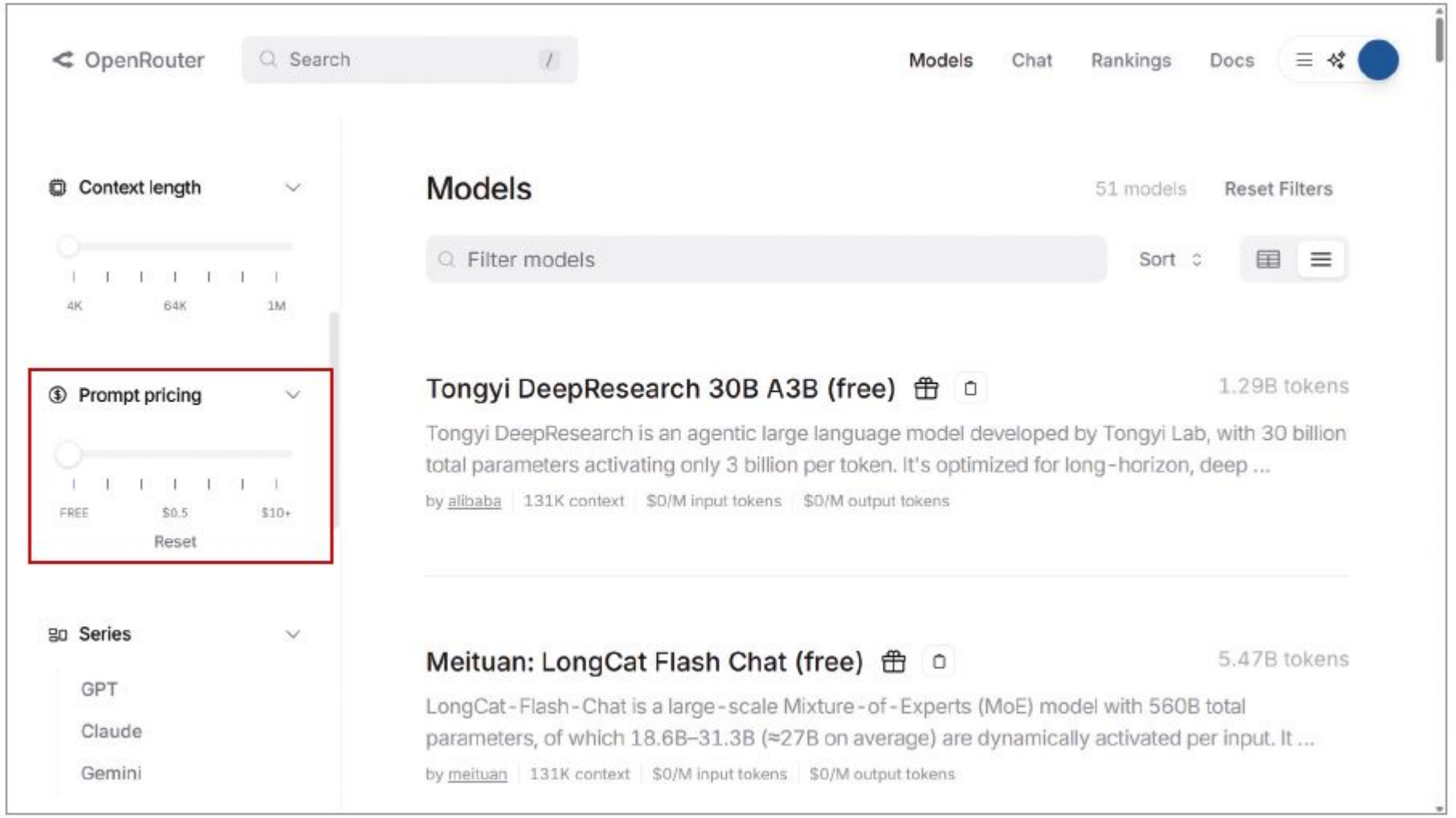
SECTION 6-1 클로드 코드에서 API 설정하기(21)

01 오픈라우터 초기 화면의 상단에서 [Models] 메뉴를 클릭



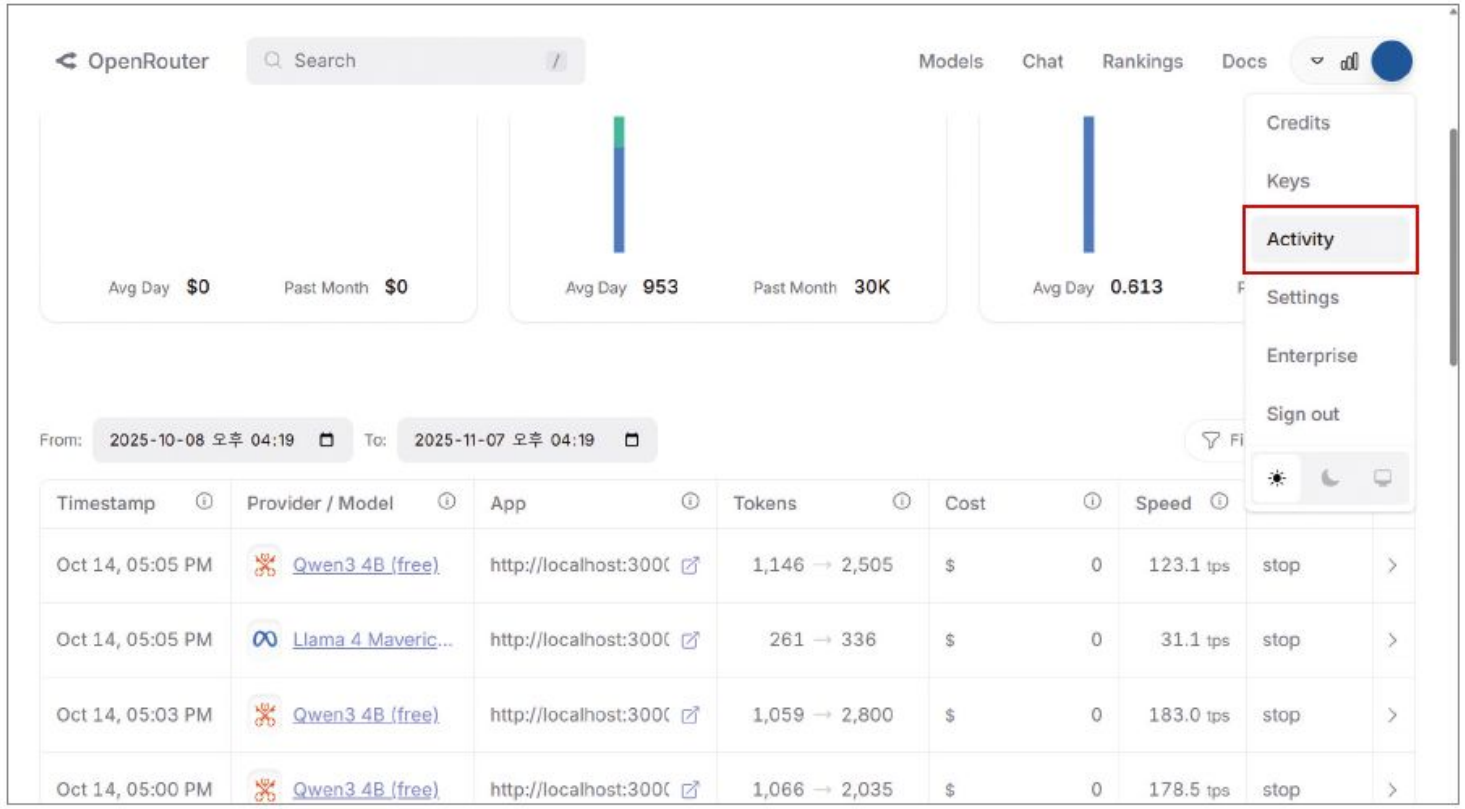
SECTION 6-1 클로드 코드에서 API 설정하기(22)

02 무료 API만 필터링하기 위해 좌측 사이드바에서 'Prompt pricing' 필터를 찾고 슬라이더의 핸들을 드래그해 'FREE'에 맞춤



SECTION 6-1 클로드 코드에서 API 설정하기(23)

- 오픈라우터에서 ‘free’로 표시된 모델들은 사용 자체는 무료이지만, 분당 요청 횟수(rate limit) 및 하루 요청 횟수 제한
  - 학습 목적으로는 이러한 제한이 있어도 충분히 활용 가능하며, 사용 기록은 오픈라우터의 [Activity] 메뉴에서 확인 가능
  - 실제 서비스를 운영하게 되어 많은 사용자가 동시에 접속한다면 유료 모델이나 크레딧 충전을 고려



SECTION 6-1 클로드 코드에서 API 설정하기(24)

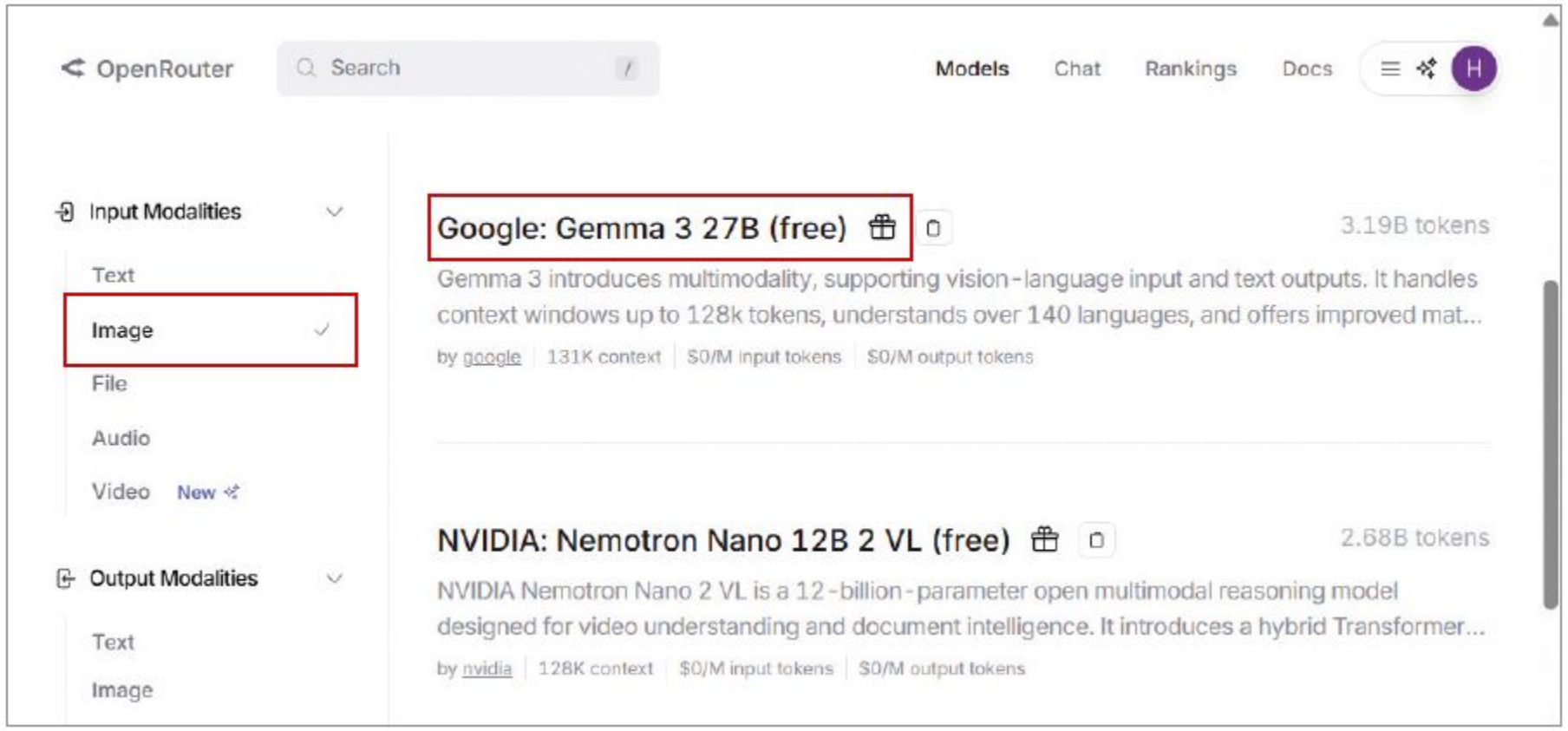
✚ 여기서 잠깐 **AI 모델의 순위를 확인하는 방법**

- 오픈라우터 상단의 **[Ranking]** 메뉴를 클릭하면 여러 모델들의 인기 순위를 한눈에 볼 수 있음
- 가장 많이 사용되는 모델들이 사용량 그래프와 함께 표시되며 대부분 유료 API지만 **'free'** 표시가 있는 무료 모델도 순위에 포함
- 이 책에서는 무료이면서도 많은 사람들의 선택을 받아 랭킹에 오른, 성능이 검증된 모델을 선택해 사용

SECTION 6-1 클로드 코드에서 API 설정하기(25)

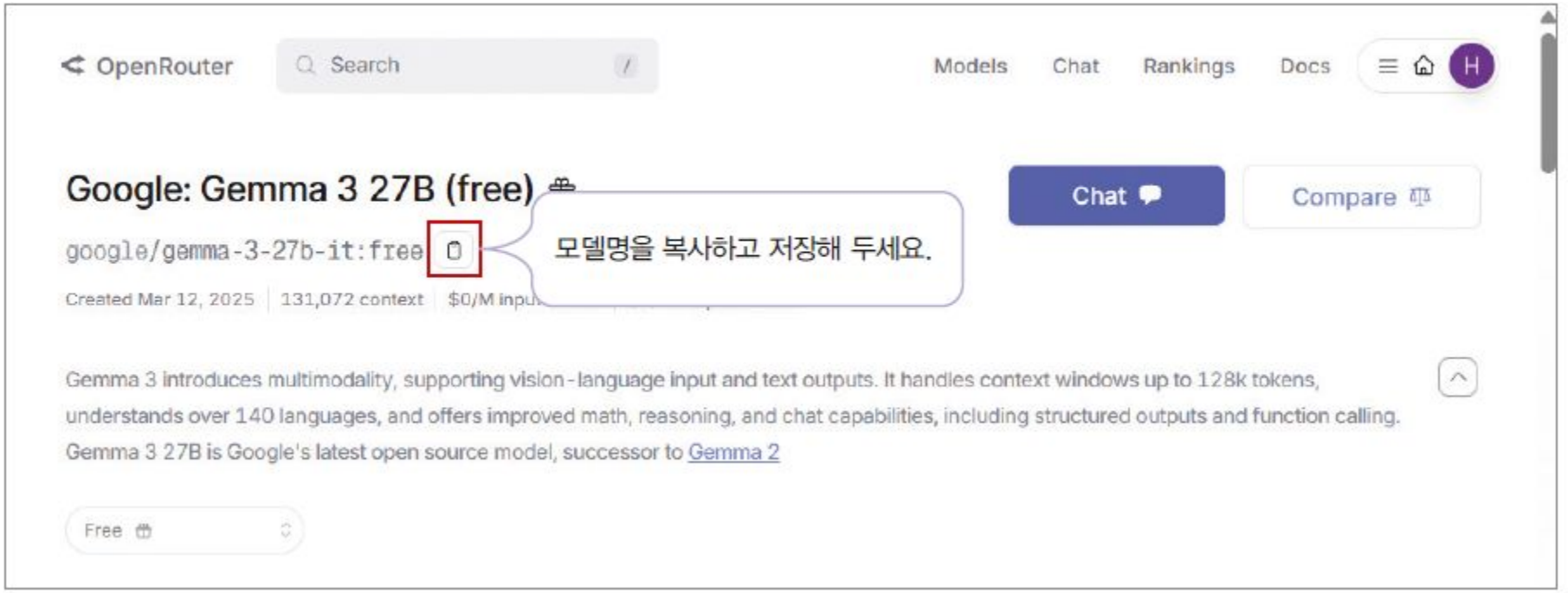
이미지 인식 AI 모델 탐색

01 좌측 사이드바의 'Input Modalities' 필터에서 [Image]를 선택
이미지의 입력과 처리와 관련된 모델이 필터링 - 이 중 원하는 모델을 입력
여기서는 [Google: Gemma 3 27B(free)]를 클릭



SECTION 6-1 클로드 코드에서 API 설정하기(26)

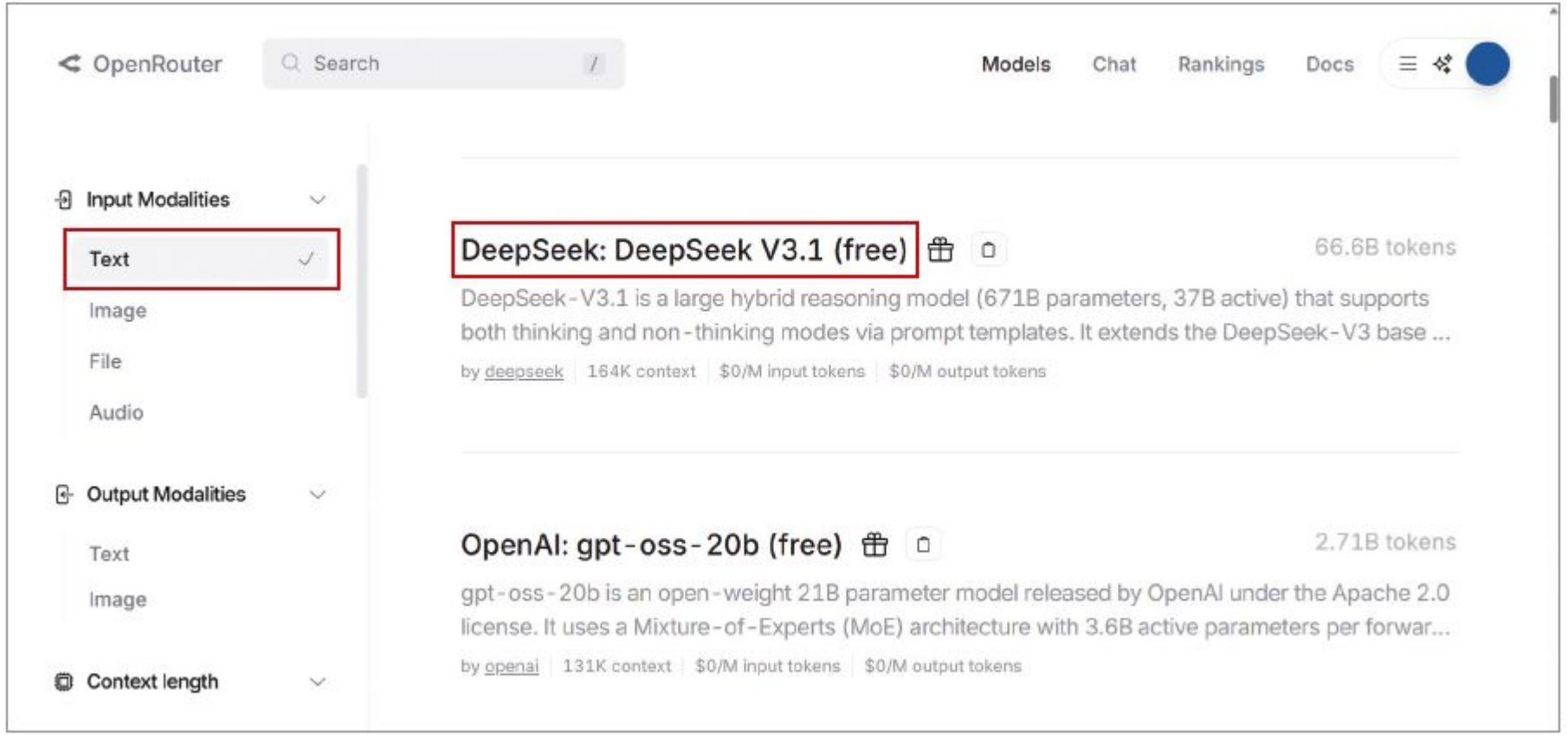
02 [(복사)] 아이콘을 클릭해 모델명을 복사하고 텍스트 파일에 저장



SECTION 6-1 클로드 코드에서 API 설정하기(27)

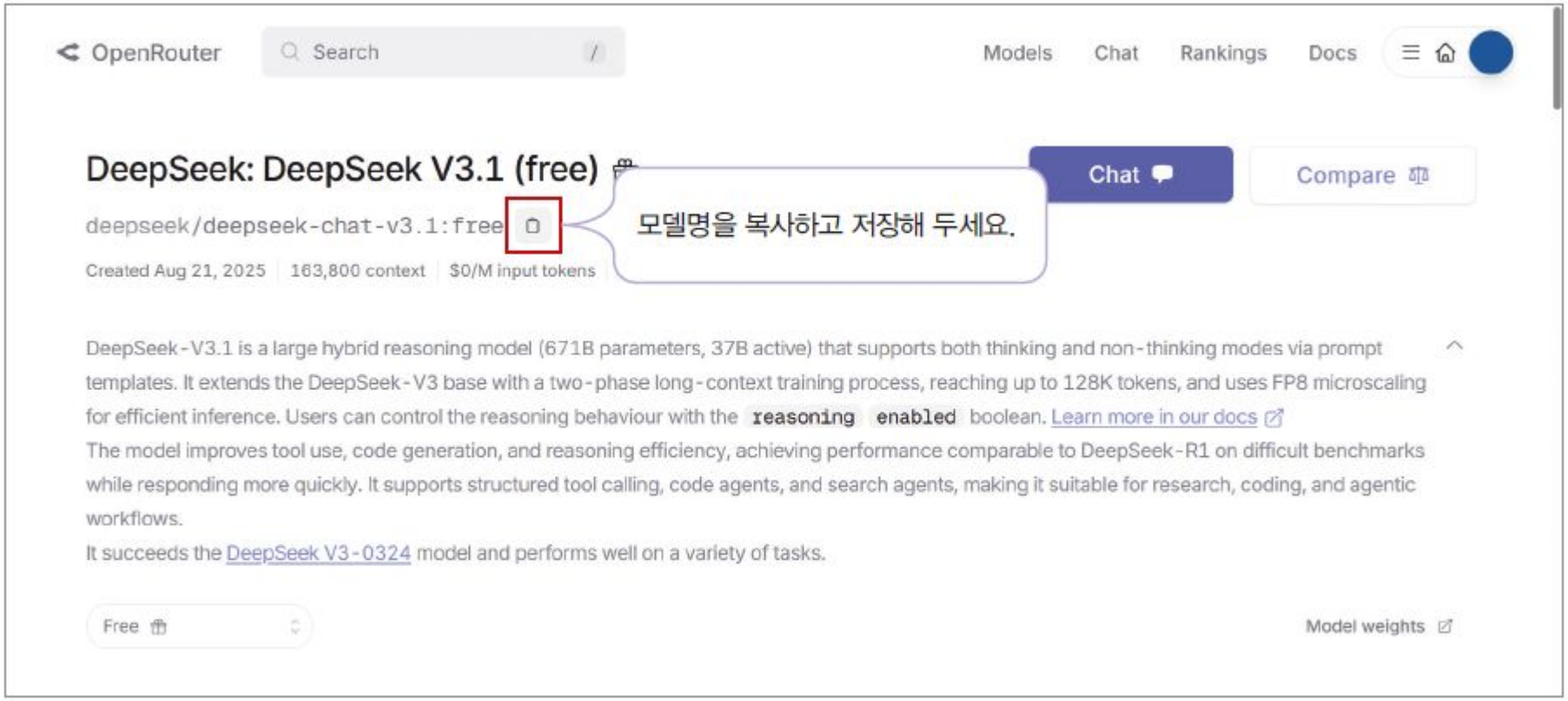
텍스트 생성 AI 모델 탐색

01 'Input Modalities' 필터에서 [Text]를 선택
필터링된 모델 중 [DeepSeek: DeepSeek V3.1 (free)]를 선택



SECTION 6-1 클로드 코드에서 API 설정하기(28)

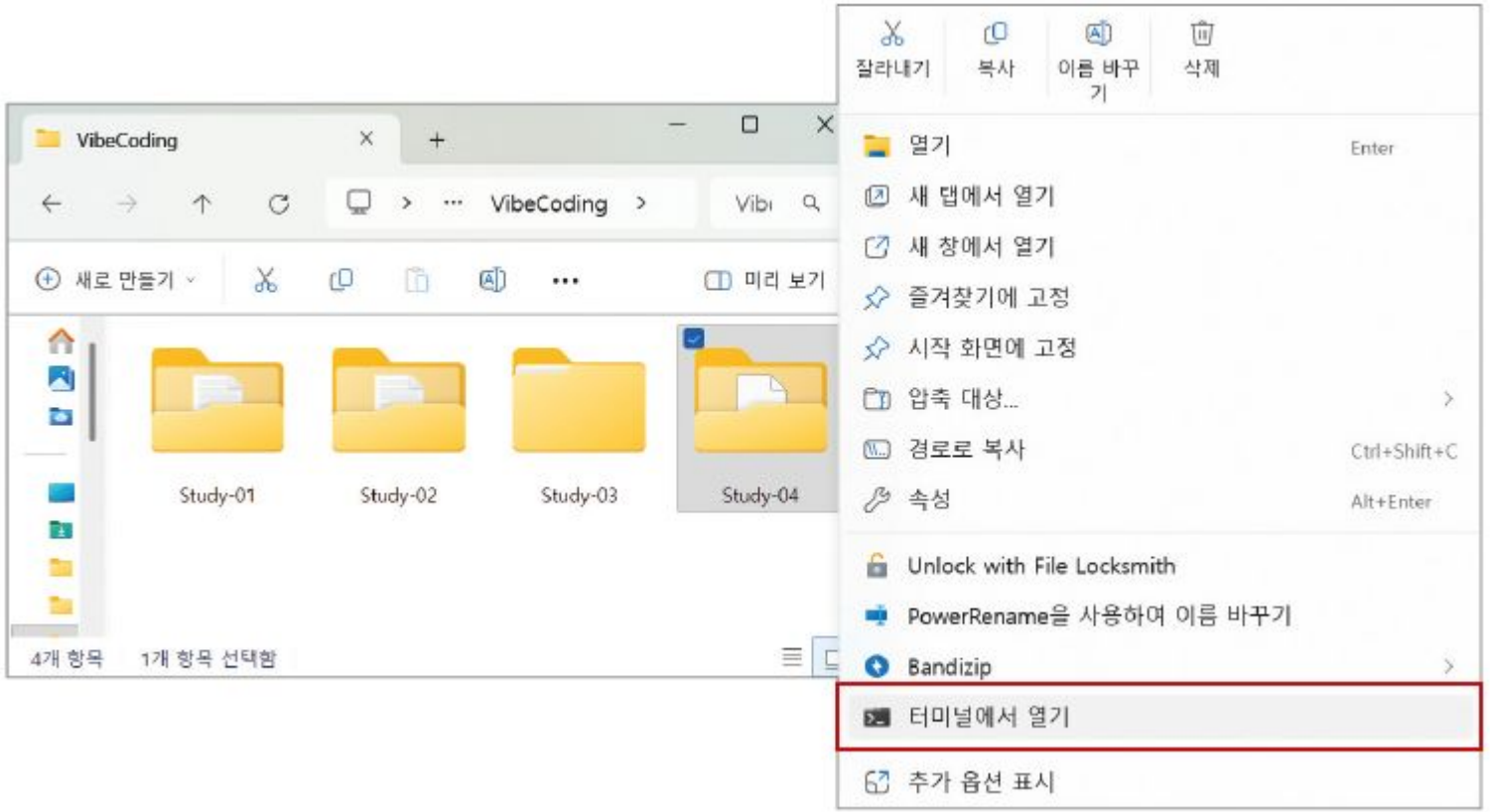
02 [(복사)] 아이콘을 클릭해 모델명을 복사하고 텍스트 파일에 저장



SECTION 6-1 클로드 코드에서 API 설정하기(29)

◦ AI 모델 연결하고 테스트하기

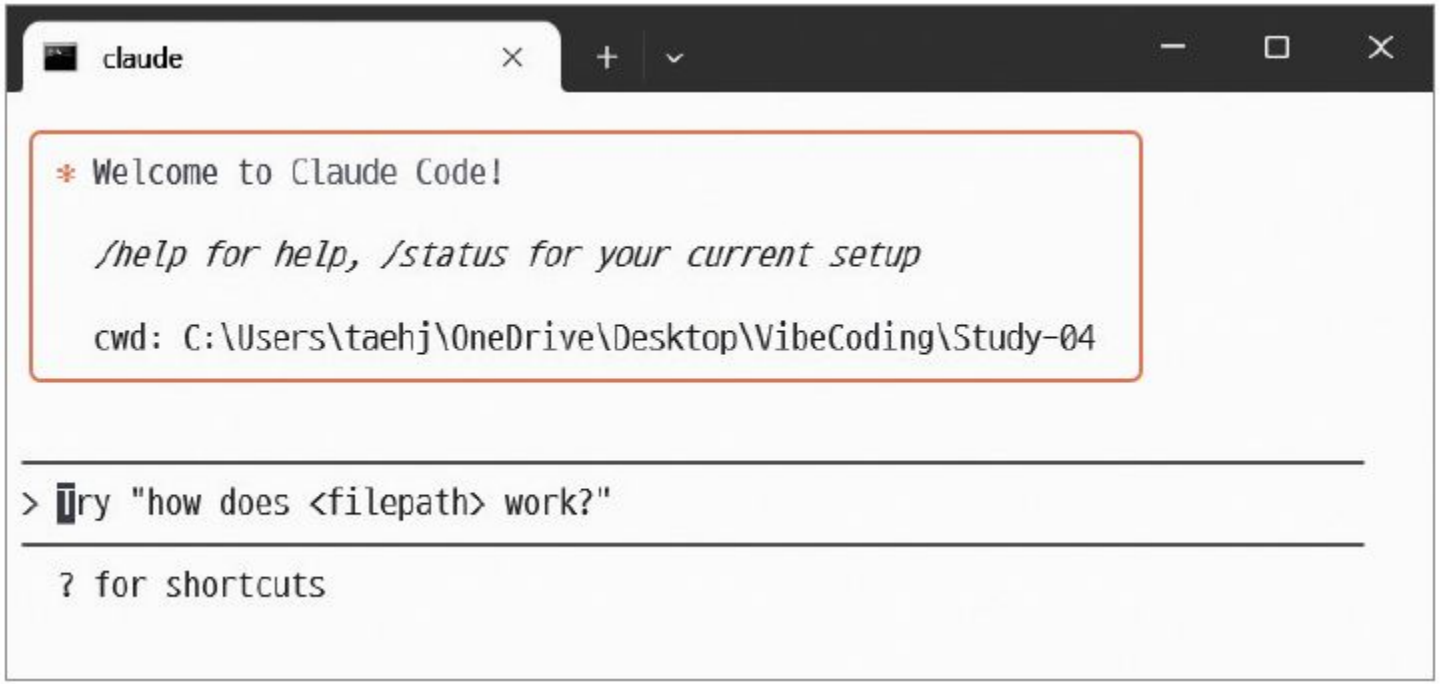
- 클로드 코드에 API를 연동해 레시피 추천 애플리케이션, '냉장고를 부탁해' 만들기
- 데스크톱의 파일 탐색기를 열어 'VibeCoding' 폴더의 'Study-04' 폴더를 선택하고 마우스 오른쪽 버튼을 클릭한 뒤, [터미널에서 열기] 메뉴를 클릭해 터미널을 실행



SECTION 6-1 클로드 코드에서 API 설정하기(30)

API 키 등록하기

01 터미널에 'claude'를 입력해 클로드 코드를 실행

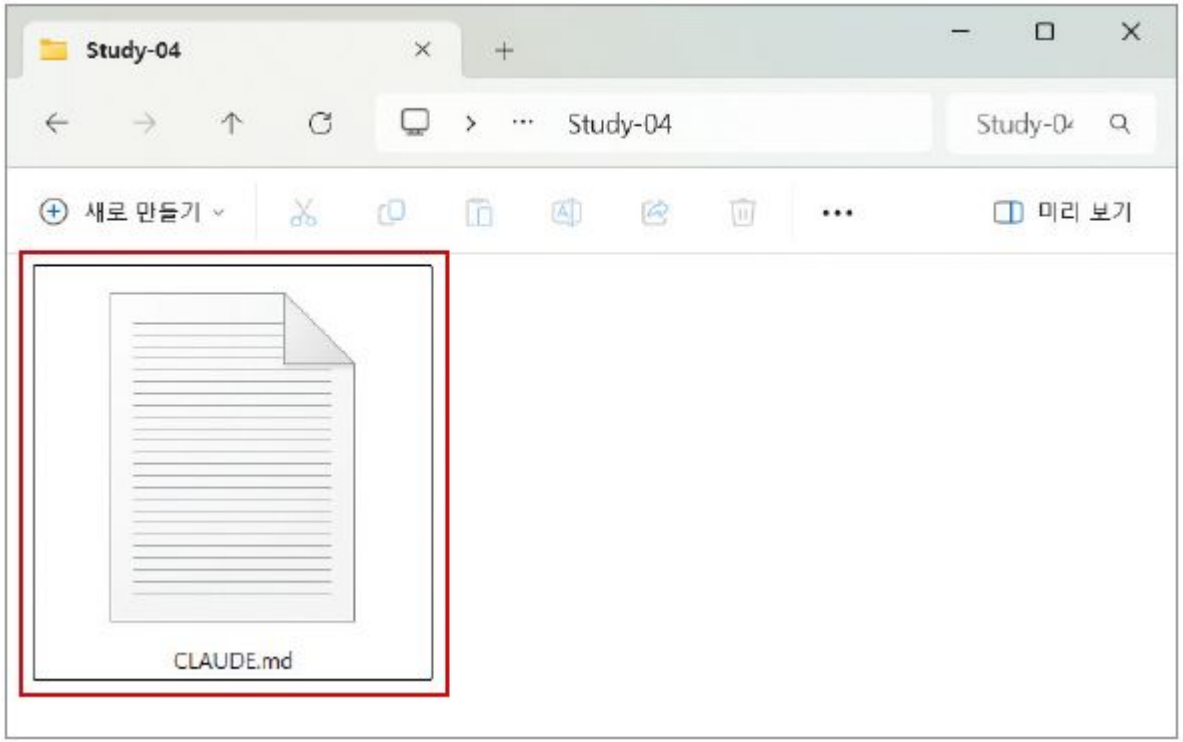


SECTION 6-1 클로드 코드에서 API 설정하기(31)

02 프로젝트 초기 설정을 위해 '/init' 명령어를 입력

SECTION 6-1 클로드 코드에서 API 설정하기(32)

03 'Study-04' 폴더에 'CLAUDE.md' 파일이 생긴 것을 확인

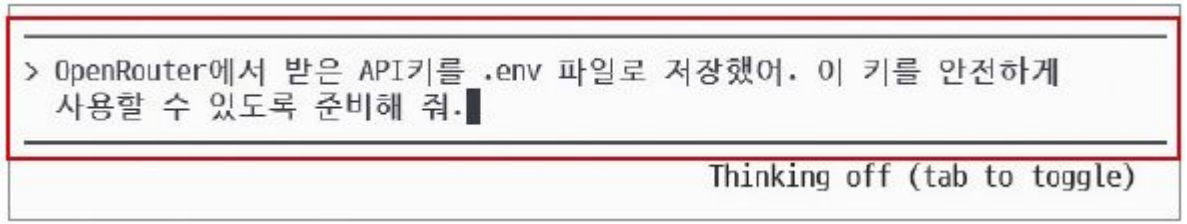


SECTION 6-1 클로드 코드에서 API 설정하기(33)

04 오픈라우터에서 받은 API 키를 저장한 '.env' 파일을 클로드 코드에서 사용할 수 있도록 준비
클로드 코드에 다음과 같이 프롬프트를 입력

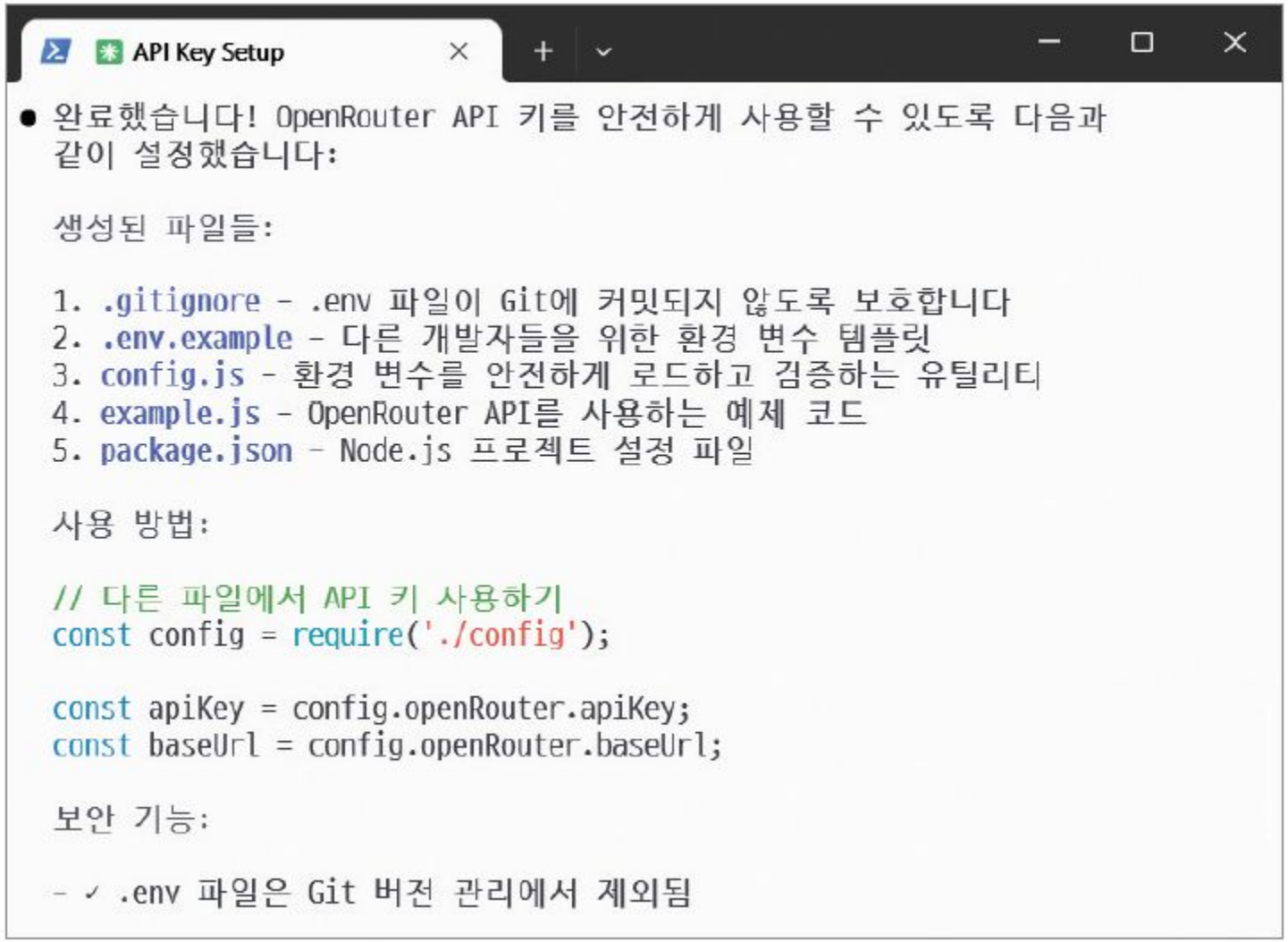
프롬프트

```
OpenRouter에서 받은 API 키를 .env 파일로 저장했어. 이 키를 안전하게 사용할 수 있도록 준비해 줘.
```



SECTION 6-1 클로드 코드에서 API 설정하기(34)

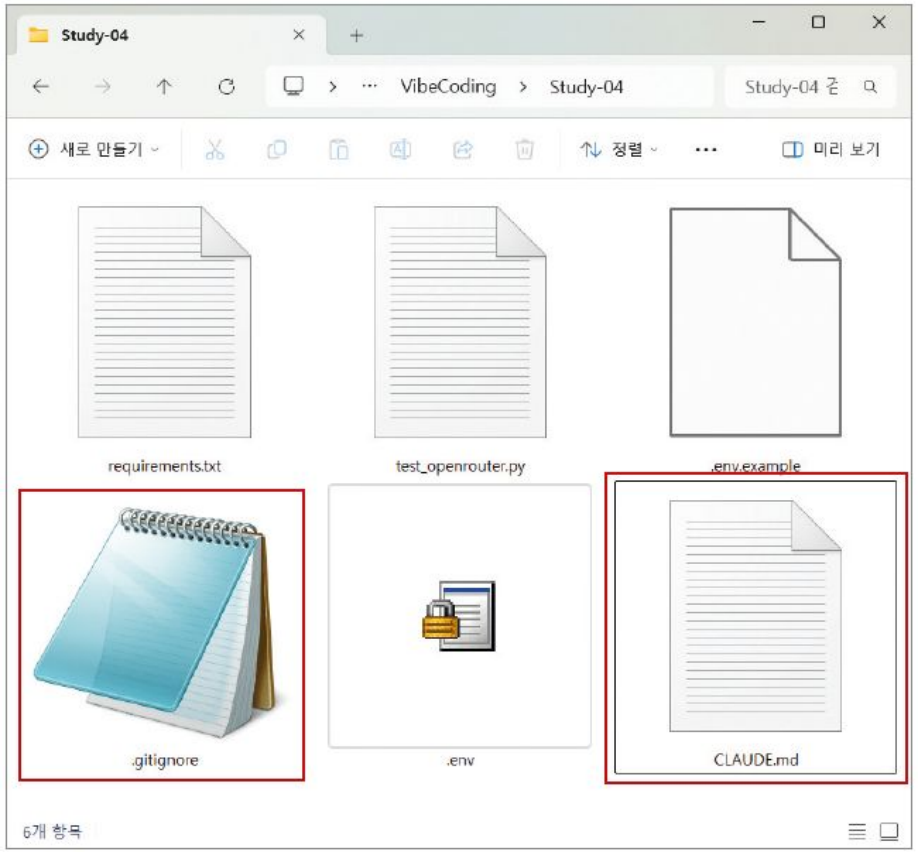
05 API 키가 안전하게 설정



SECTION 6-1 클로드 코드에서 API 설정하기(35)

06 'Study-04' 폴더를 확인하면 다음과 같이 '.gitignore' 파일과 그 밖에 보조적인 설정 파일, 프로젝트 관리를 담당하는 파일들이 새로 생성됨

- .env: API 키가 저장된 환경 변수 파일, API를 호출할 때마다 참조됨
- .gitignore: '.env' 파일이 자동으로 깃허브에 업로드되지 않도록 특정 파일을 제외하여 보호하는 설정 파일



SECTION 6-1 클로드 코드에서 API 설정하기(36)

✚ 여기서 잠깐 **'.env'와 '.gitignore' 파일의 이해**

- '.env' 파일은 프로그램의 설정값을 저장하는 특별한 파일
  - API 키처럼 민감한 정보를 프로그램 코드와 분리하여 관리하는 표준 방식
  - 클로드 코드는 API 키 설정 시 자동으로 '.env'와 '.gitignore' 파일을 생성함으로써 이러한 실무 관행을 자연스럽게 따르고 있음
- 코드에 API 키를 직접 넣으면 깃허브에 업로드할 때 누구나 볼 수 있게 됨
- 하지만 '.env' 파일을 '.gitignore'에 등록해 두면 자동으로 제외되므로 안전
- 또한 개발 중에는 무료 API를 쓰다가 실제 서비스에서는 유료 API로 전환해야 할 때도 코드 수정 없이 '.env' 파일만 바꾸면 됨
- 팀 프로젝트에서도 유용
  - 각자 다른 API 키를 사용해야 할 때, 코드는 공유하되 각자의 '.env' 파일만 따로 설정하면 충돌 없이 개발

SECTION 6-1 클로드 코드에서 API 설정하기(37)

API 연동 테스트하기

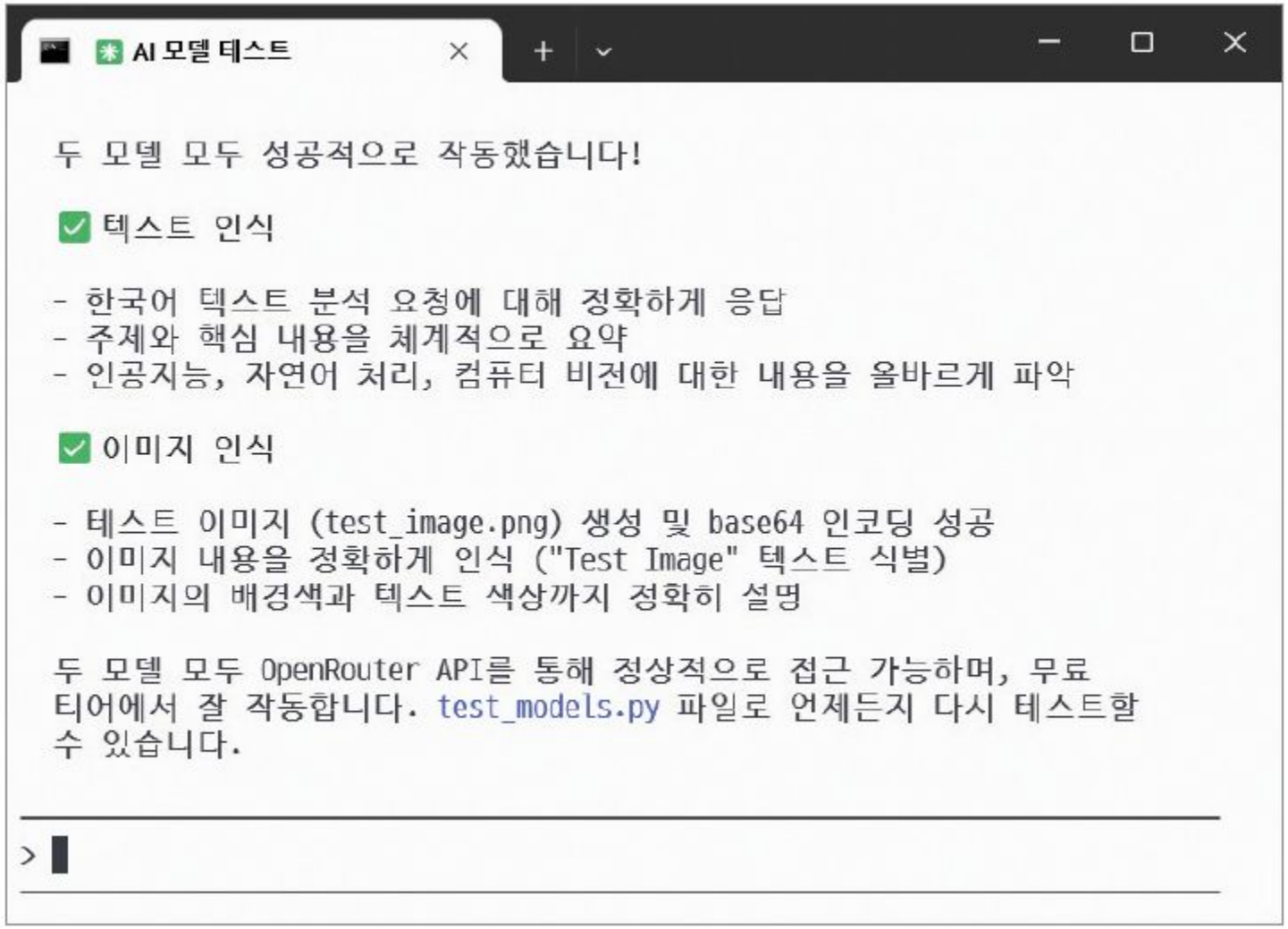
01 이미지 인식은 구글의 Gemma 모델을, 생성 언어 모델은 DeepSeek 모델을 사용
저장해 둔 각 모델의 이름을 확인해서 프롬프트에 다음과 같이 입력

프롬프트

```
이제 준비된 API가 실제로 작동하는지 테스트해 줘.
이미지 인식은 google/gemma-3-27b-it:free 모델을 이용하고,
텍스트 인식은 deepseek/deepseek-chat-v3.1:free 모델을 이용할 거야.
API를 통한 텍스트와 이미지 인식을 각각 테스트해서 실행 결과를 알려줘.
```

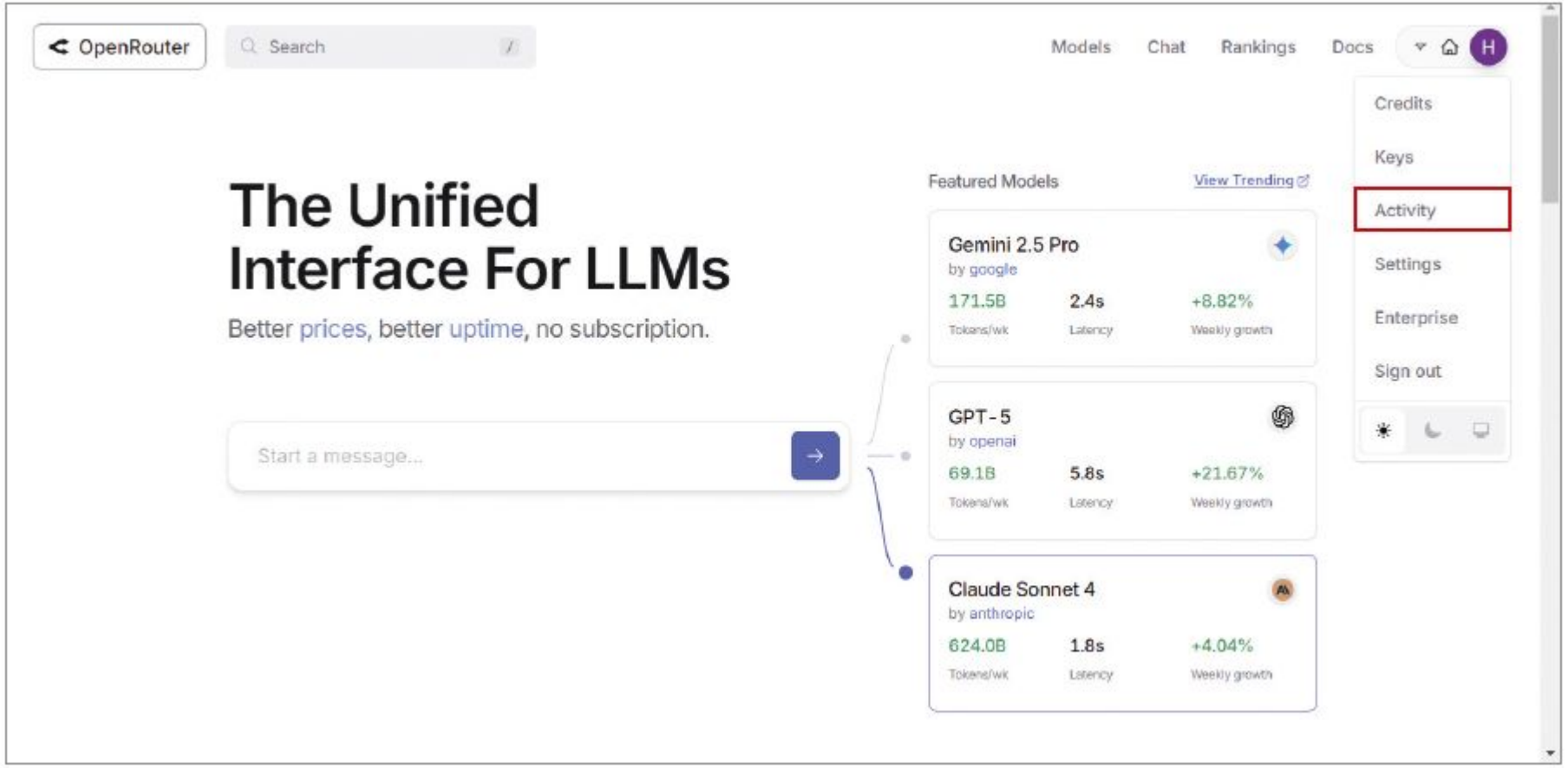
SECTION 6-1 클로드 코드에서 API 설정하기(38)

02 API 테스트에 성공



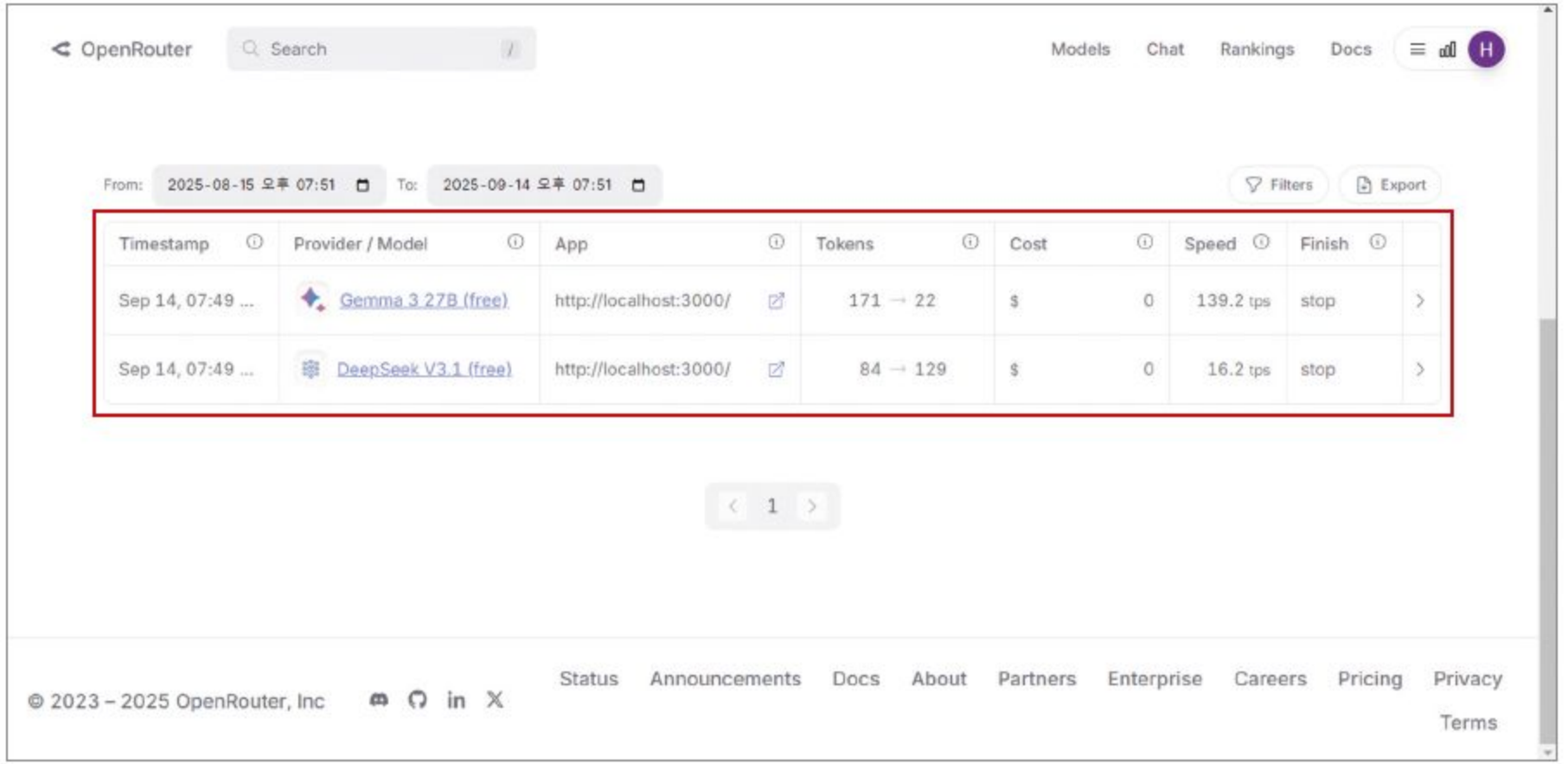
SECTION 6-1 클로드 코드에서 API 설정하기(39)

03 실제로 API 키가 활용되었는지는 오픈라우터에서 확인
오픈라우터 홈페이지에 로그인해서 프로필 아이콘을 클릭한 후, [Activity] 메뉴를 선택



SECTION 6-1 클로드 코드에서 API 설정하기(40)

04 'Timestamp' 열에 실제로 Gemma와 DeepSeek 모델을 사용한 기록을 확인



[마무리]

- 5가지 키워드로 정리하는 핵심 포인트
  - **API**는 외부 AI 서비스와 프로그램을 연결하는 다리로, 간단한 요청만으로 복잡한 AI 기능을 활용할 수 있게 해 주는 통신 규칙
  - 오픈라우터와 같은 **API 통합 플랫폼**에서는 여러 AI 모델을 한 곳에서 비교하고 선택
  - API 서비스에 접근하기 위해서는 **API 키**를 발급받아 사용자 인증이 필요
  - API를 통해 다양한 AI 모델을 애플리케이션에 통합하려면 AI 모델을 연동
    이때 클로드 코드의 작업 폴더에 다음과 같은 설정 파일을 생성
    - **.env**: API 키를 저장한 파일로, API를 호출할 때마다 참조됨
    - **.gitignore**: API 키가 유출되는 것을 방지하기 위해 깃허브에 특정 파일이 업로드되지 않게 보호하는 설정 파일

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(1)

시작하기전에

◦ '냉장고를 부탁해'

- 애플리케이션에 냉장고 내부를 찍은 사진을 업로드하면,
  - 식재료를 인식
  - 가지고 있는 재료에 따라 적절한 레시피를 추천

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(2)

- 애플리케이션 구현을 위한 PRD 작성하기
  - '냉장고를 부탁해' 애플리케이션 제작을 위한 프롬프트 만들기
    - 프롬프트는 작업의 방향과 품질을 결정하는 핵심 요소이므로, 세부 내용을 점검하고, 실행 단계를 구체화하여 필요한 요구사항을 명확히 하는 과정이 필수적
  - PRD 만들기
    - 이전 장까지는 클로드 웹에서 프롬프트를 만들어 확인하고 클로드 코드에 이를 복사해 넣는 식으로 진행
    - 이번에는 클로드 코드에서 직접 진행
      - '.md' 파일을 활용하면 이전에 했던 것처럼 여러 단계의 세부 실행 프롬프트를 PRD 형식으로 구조화하여 작성

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(3)

단계별 계획 수립하기

- '냉장고를 부탁해' 애플리케이션에는 두 가지 AI 모델이 필요
  1) 냉장고 사진 속 식재료를 분석하는 이미지 인식 모델
  2) 식재료를 활용해 레시피를 제안하는 텍스트 생성 모델
- 다중 AI 모델 연동이 필요한 복잡한 애플리케이션을 개발할 때는 전체 프로젝트를 작은 단위로 나누어 점진적으로 개발하는 방법이 효율적 - 단계별로 구현
  1) 핵심적인 기능을 먼저 구현
  2) 잘 작동하는지 확인한
  3) 세부적인 기능을 추가
- 웹 애플리케이션의 제작 과정을 3단계로 나누고 PRD 제작을 위해 단계마다 '.md' 파일을 생성
  1. 1단계: 이미지 인식 모델을 활용해 냉장고 사진에서 재료를 인식
  2. 2단계: 텍스트 생성 모델을 활용해 인식한 식재료로 만들 수 있는 요리의 레시피를 생성
  3. 3단계: 생성한 레시피를 저장

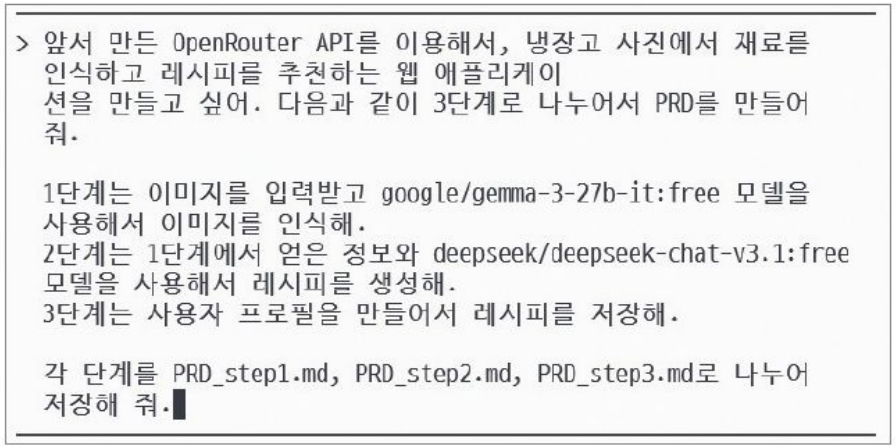
SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(4)

클로드 코드에서 PRD 생성하기

01 클로드 코드에 다음과 같이 입력

프롬프트

```
앞서 만든 OpenRouter API를 이용해서, 냉장고 사진에서 재료를 인식하고 레시피를 추천하는 웹 애플리케이션
을 만들고 싶어. 다음과 같이 3단계로 나누어서 PRD를 만들어 줘.

1단계는 이미지를 입력받고 google/gemma-3-27b-it:free 모델을 사용해서 이미지를 인식해.
2단계는 1단계에서 얻은 정보와 deepseek/deepseek-chat-v3.1:free 모델을 사용해서 레시피를 생성해.
3단계는 사용자 프로필을 만들어서 레시피를 저장해.

각 단계를 PRD_step1.md, PRD_step2.md, PRD_step3.md로 나누어 저장해 줘.
```



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(5)

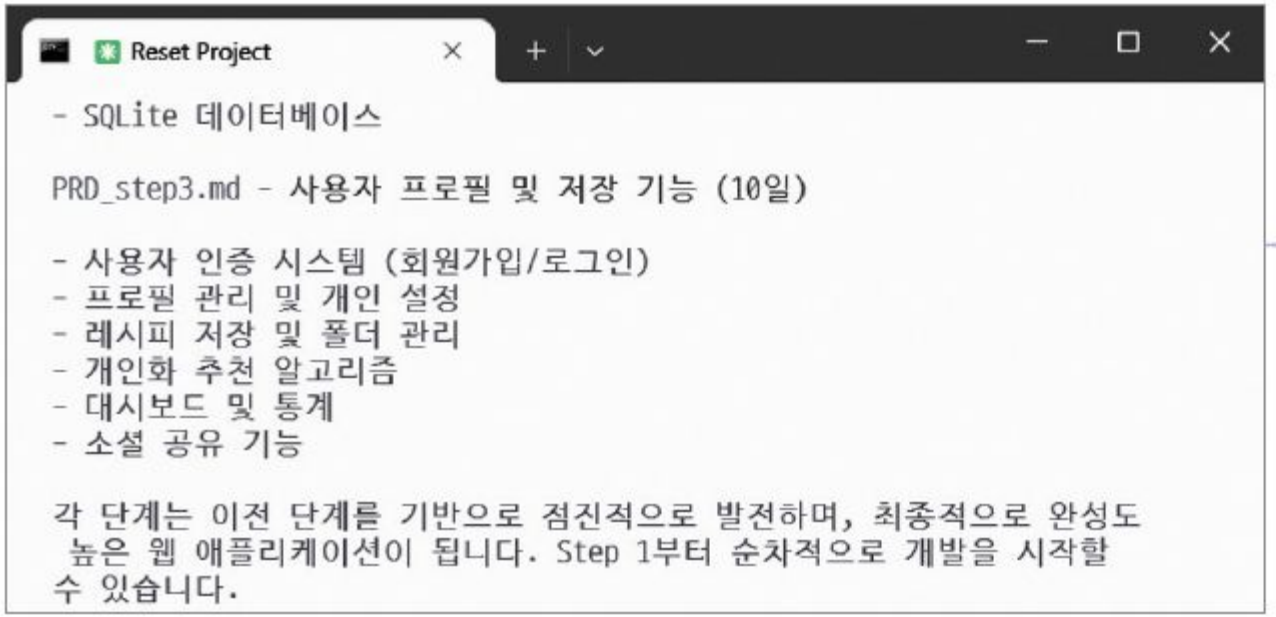
02 교재에서 완성된 PRD의 요약

```
PRD_step1.md - 이미지 인식 핵심 기능 (5일)
  - Streamlit 기반 웹 인터페이스
  - Gemma-3-27b-it 모델로 냉장고 재료 인식
  - 이미지 업로드 및 처리
  - 기본 UI 구축
```

```
PRD_step2.md - 레시피 생성 시스템 (7일)
  - DeepSeek 모델로 레시피 생성
  - 재료 편집 기능
  - 필터링 및 검색
  - 레시피 상세 보기
  - SQLite 데이터베이스
```

```
PRD_step3.md - 사용자 프로필 및 저장 기능 (10일)
  - 사용자 인증 시스템 (회원가입/로그인)
  - 프로필 관리 및 개인 설정
  - 레시피 저장 및 폴더 관리
  - 개인화 추천 알고리즘
  - 대시보드 및 통계
  - 소셜 공유 기능
```

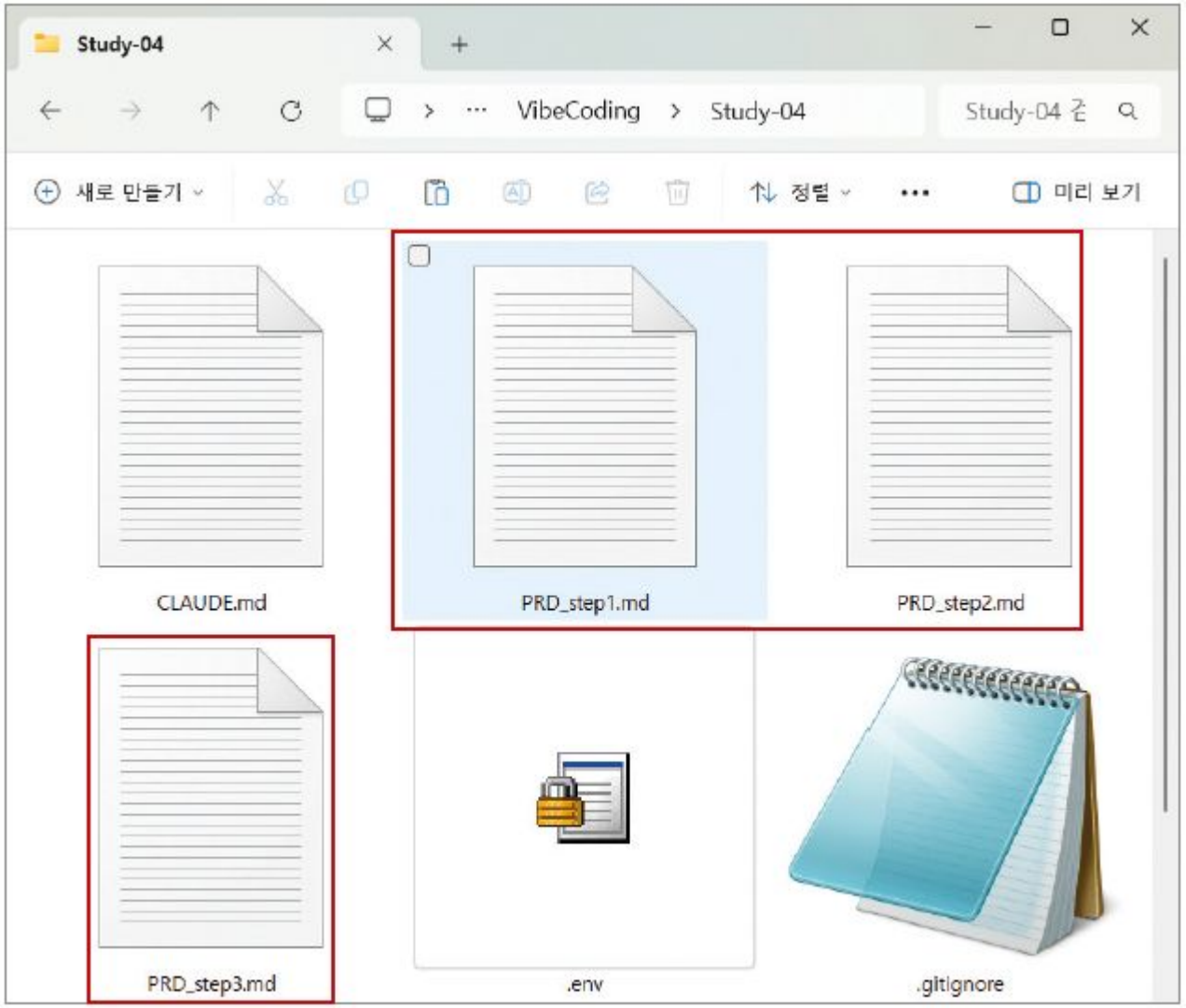
SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(6)



화면과 책이 달라도 괜찮습니다.
자신만의 PRD로 진행하세요.

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(7)

03 'Study-04' 폴더에 세 개의 단계별 PRD 문서가 작성된 것을 확인



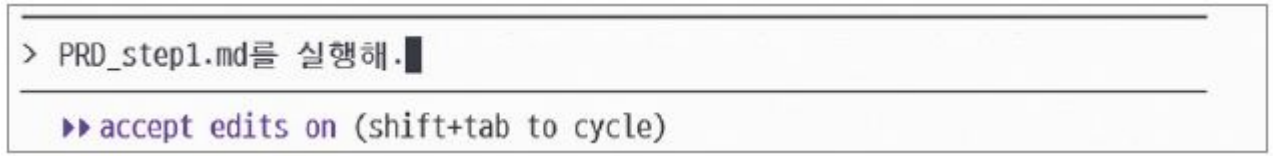
SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(8)

◦ 1단계: 냉장고 이미지에서 재료 인식하기

01 작성된 PRD를 바탕으로 첫 번째 단계를 구현
다음과 같이 프롬프트를 입력해 첫 번째 PRD를 실행하라고 명령

프롬프트

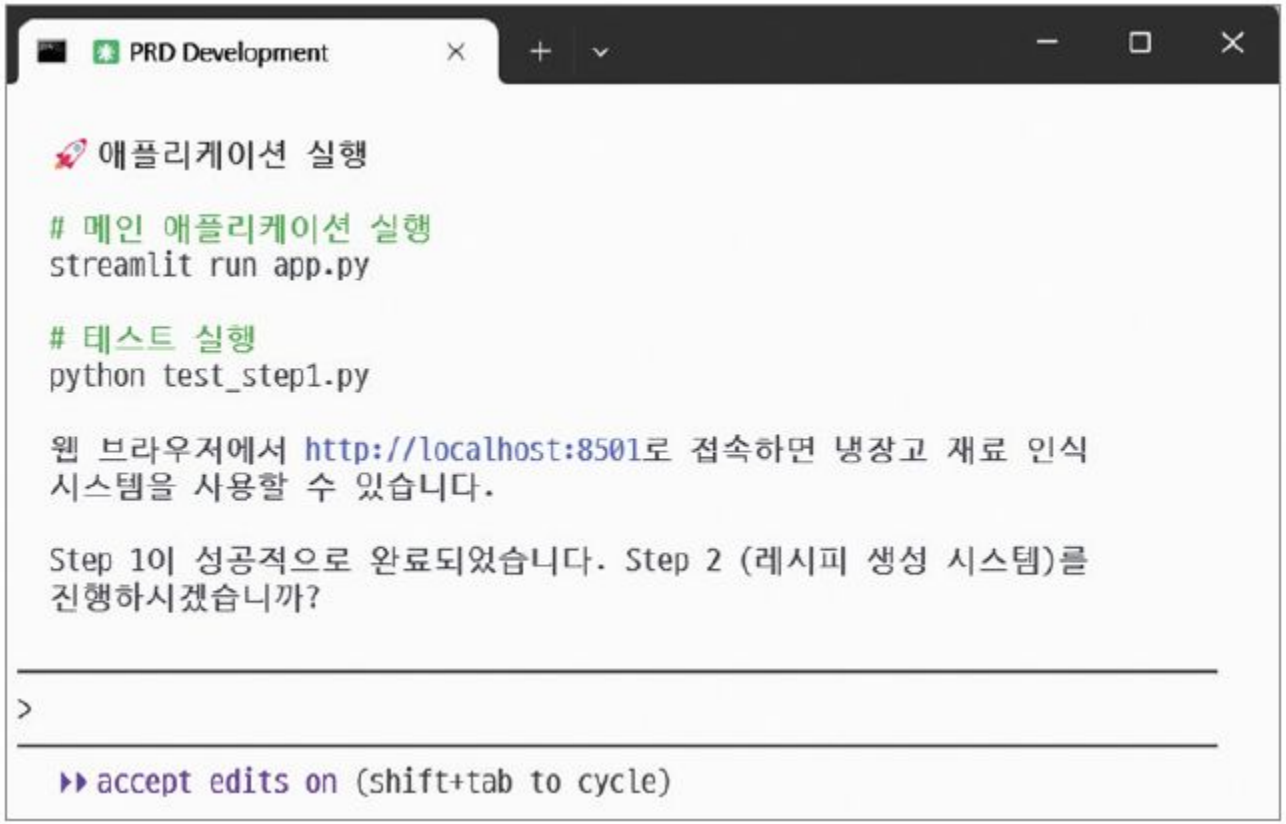
```
PRD_step1.md를 실행해.
```



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(9)

02 주어진 1단계 프롬프트에 따라 이미지 인식 기능이 구현

클로드 코드는 파이썬 웹 프레임워크 Streamlit을 사용하여 웹 인터페이스를 구축



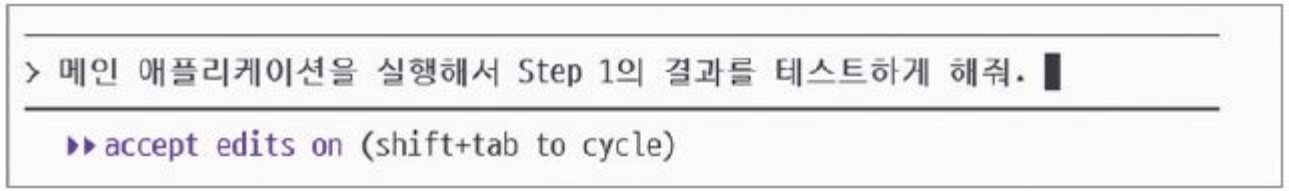
SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(10)

03 실제로 작동하는지 확인

아직 웹 서버가 실행되지 않은 상태라면, 클로드 코드에게 서버 실행까지 요청

프롬프트

```
메인 애플리케이션을 실행해서 1단계의 결과를 테스트해 줘.
```



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(11)

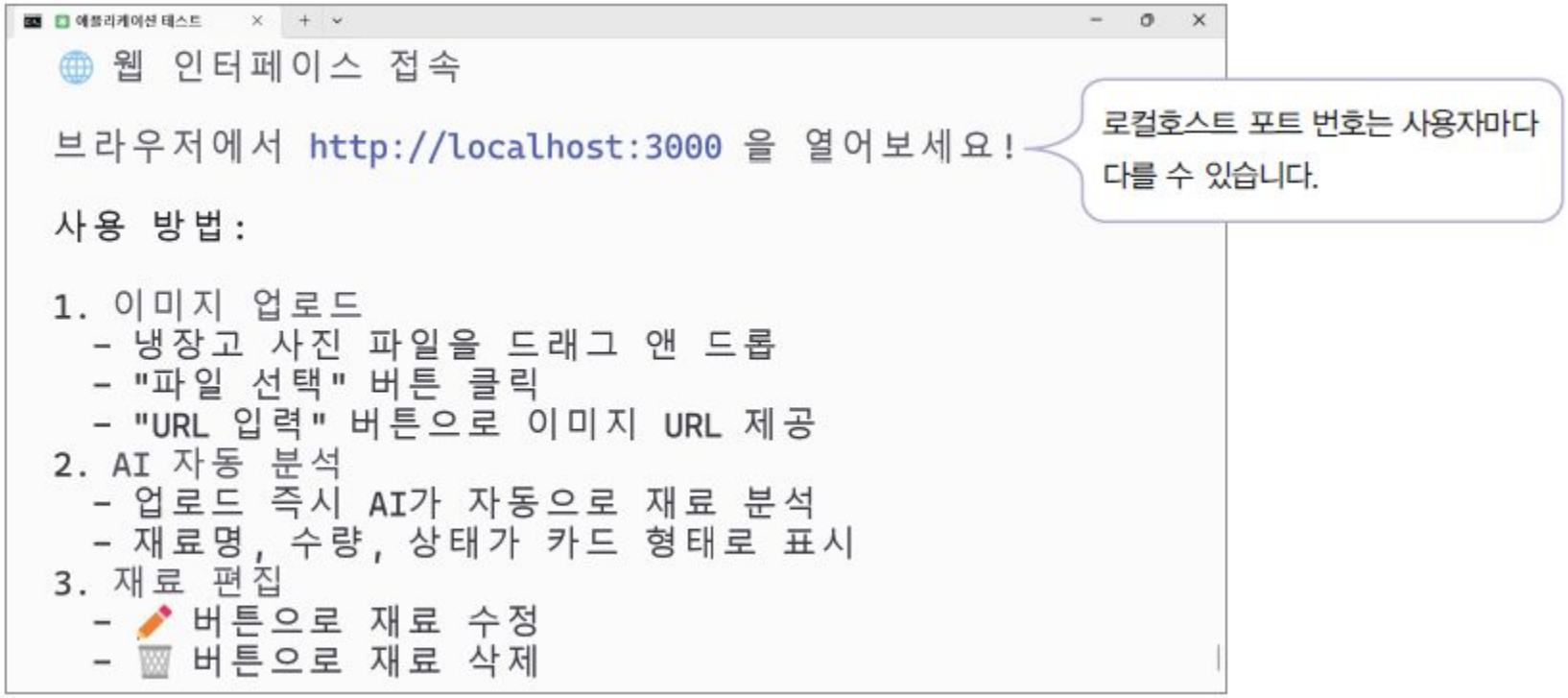
✚ 여기서 잠깐

파이썬 웹 프레임워크란?

- 웹 프레임워크 (web framework)
  - 웹 애플리케이션을 쉽게 만들 수 있도록 도와주는 도구 모음
  - 일반적으로 웹 애플리케이션을 만들려면 HTML로 구조를, CSS로 디자인을, JavaScript로 동작을 분리하여 각각 프로그래밍
  - 파이썬 웹 프레임워크를 사용하면 하나의 파이썬 파일에서 이 모든 것을 처리할 수 있어 구조가 훨씬 단순
- 클로드 코드가 자주 선택하는 프레임워크는 Streamlit, Gradio, Flask 등
  - 이 중 Streamlit은 특히 AI 모델을 시연하거나 데이터를 시각화할 때 자주 사용되며, 이미지 인식이나 텍스트 생성 같은 AI 기능을 웹에서 보여줄 때 유용
- 사용한 프레임워크에 따라 서버 실행 명령어도 달라짐
  - 물론 클로드 코드에게 직접 실행을 요청하면 알맞은 명령어로 자동 실행
  - Streamlit을 사용한 경우, streamlit run app.py
  - Flask를 사용한 경우, python app.py
  - Gradio를 사용한 경우, python gradio_app.py

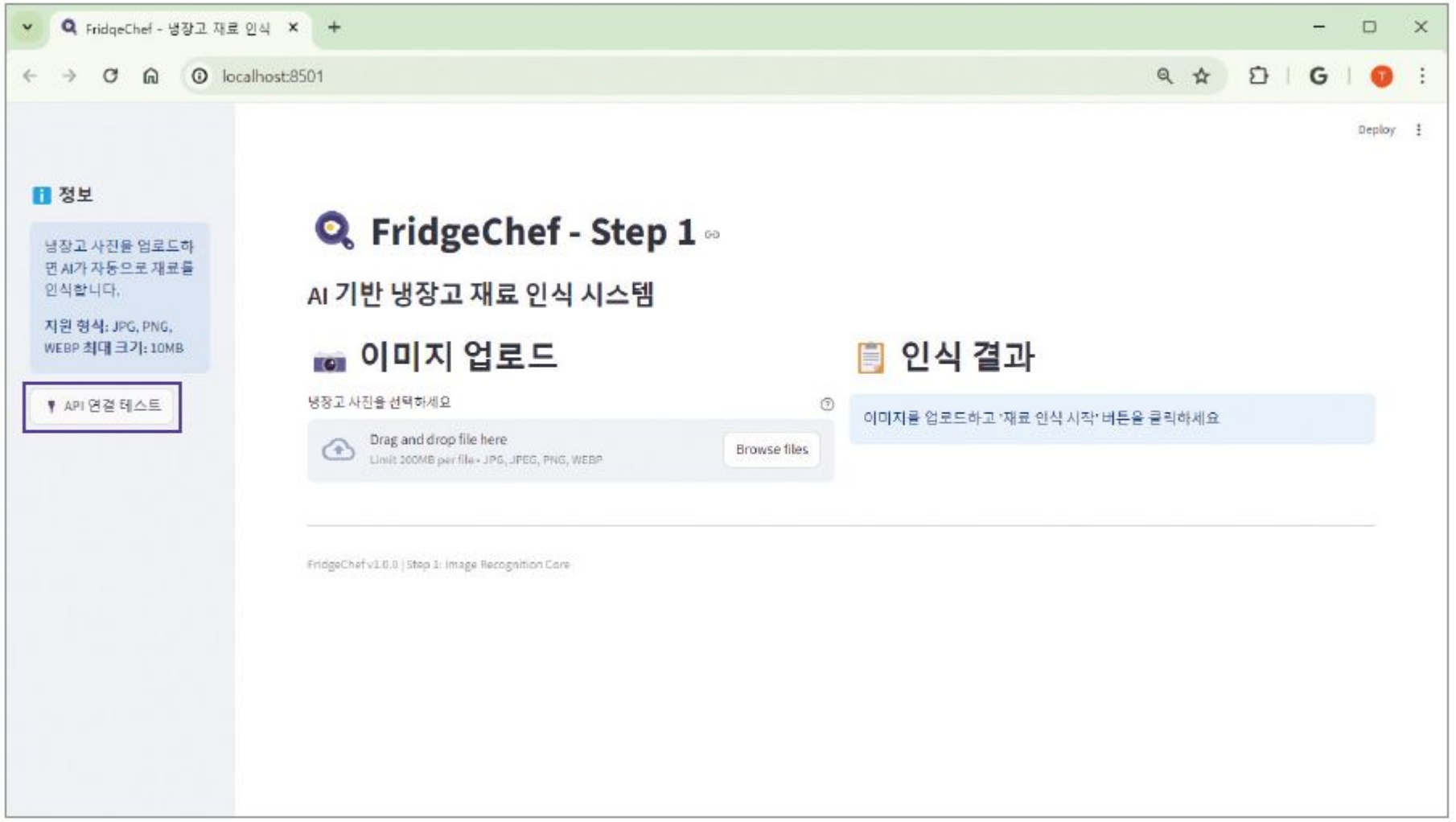
SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(12)

04 서버 실행이 완료되면 브라우저에서 접속 가능한 주소가 표시됨



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(13)

05 로컬호스트 주소를 복사하여 브라우저에 붙여 넣고 지금까지 구현한 결과를 브라우저에서 확인



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(14)

06 기능을 테스트

깃허브 자료실(https://github.com/taehojo/vibecoding)의 '입력예제' 폴더에서 '냉장고.jpg' 파일을 미리 다운로드

[Browse files] 버튼을 클릭하고 '냉장고.jpg' 파일을 선택한 뒤, [재료 인식 시작] 버튼을 클릭

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(15)

07 잠시 후 AI 모델의 분석 결과가 화면에 표시

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(16)

◦ 2단계: 레시피 생성하기

01 인식된 재료로 레시피를 생성하는 2단계를 구현

프롬프트

```
이제 PRD_step2.md를 실행해 줘.
```

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(17)

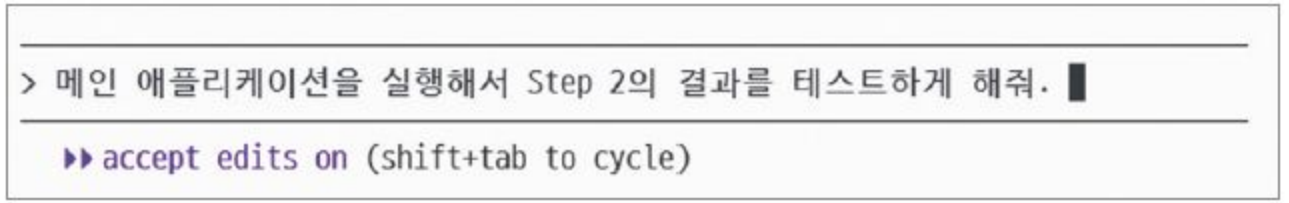
02 클로드 코드가 PRD_step2.md를 읽고 레시피 생성 기능을 추가하기 시작

DeepSeek 모델을 연동하고, 재료 편집 기능과 레시피 표시 인터페이스를 구현하는 과정이 터미널에 표시됨

작업이 완료되면 1단계와 마찬가지로 서버 실행을 요청

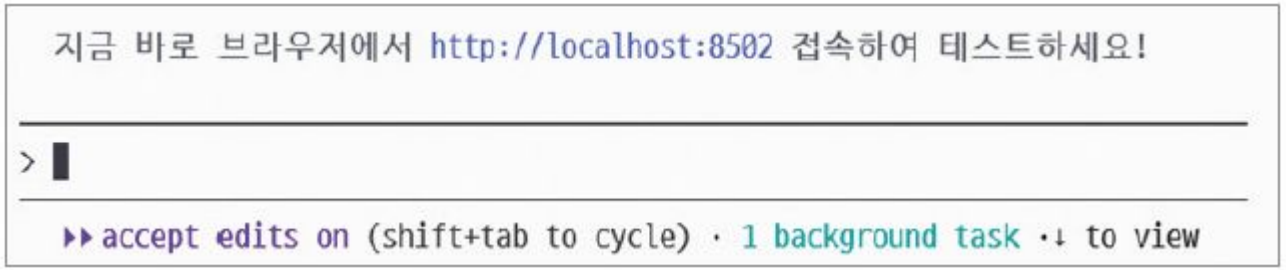
프롬프트

```
메인 애플리케이션을 실행해서 2단계 결과를 테스트하게 해 줘.
```



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(18)

03 서버가 실행되면 터미널에 표시된 주소로 접속

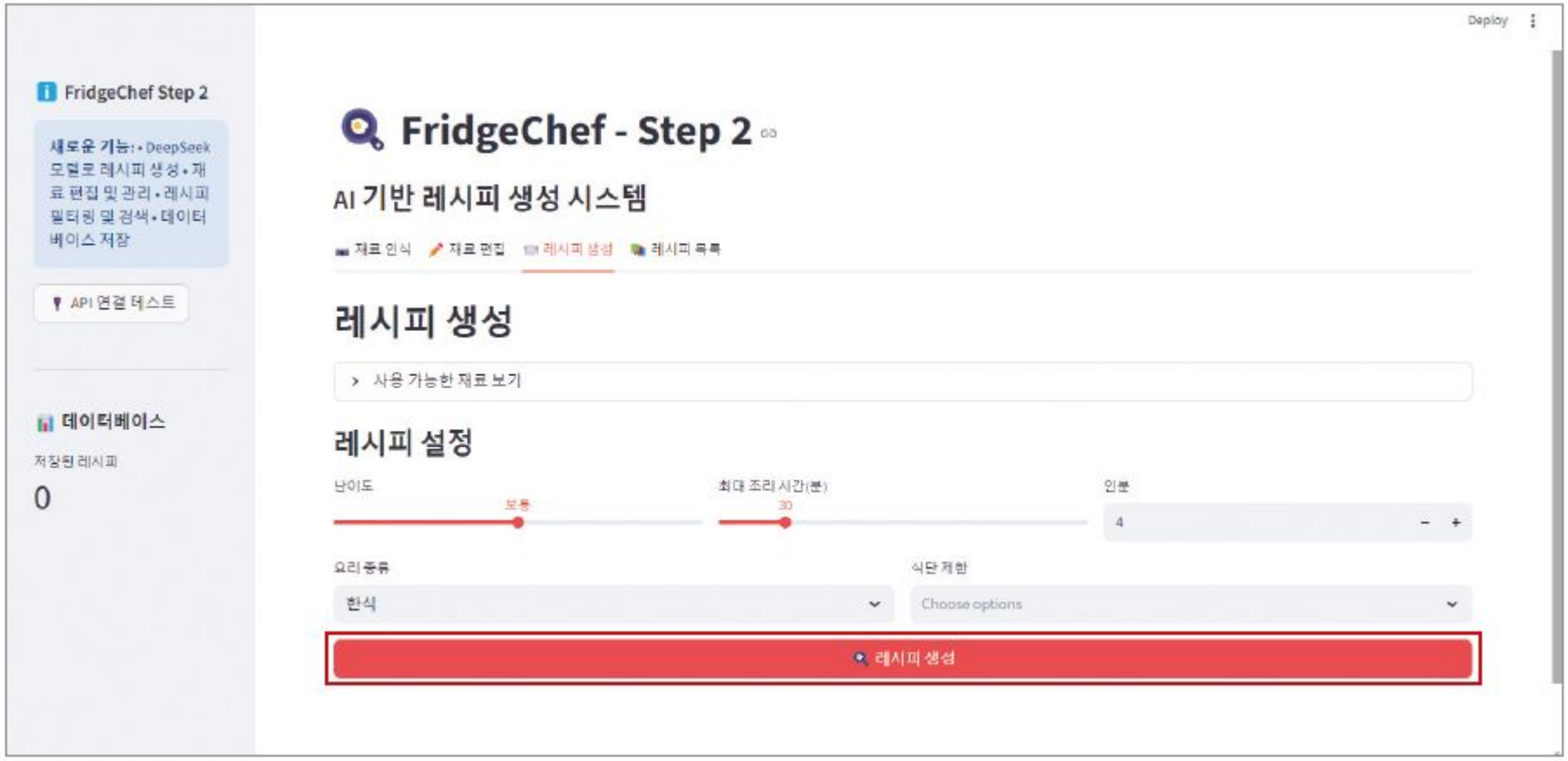


SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(19)

04 업그레이드된 애플리케이션에는 사이드바에 [레시피 생성] 메뉴가 추가
이 메뉴를 클릭하면 레시피 생성을 위한 다양한 옵션이 나타남

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(20)

05 인식된 재료를 수정하거나 추가할 수 있고, 요리 종류와 난이도를 선택할 수 있음
설정을 마친 후 [레시피 생성] 버튼을 클릭



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(21)

06 DeepSeek 모델이 선택한 재료와 조건에 맞는 레시피를 생성
각 레시피에는 요리명, 필요한 재료, 조리 순서, 예상 시간 등이 포함
AI가 냉장고 재료를 인식하고 그에 맞는 레시피까지 추천하는 실용적인 애플리케이션

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(22)

◦ 3단계: 사용자 프로필에 레시피 저장하기

01 사용자 프로필과 레시피 저장 기능을 추가하는 3단계를 구현

프롬프트

```
이제 PRD_step3.md를 실행해 줘.
```

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(23)

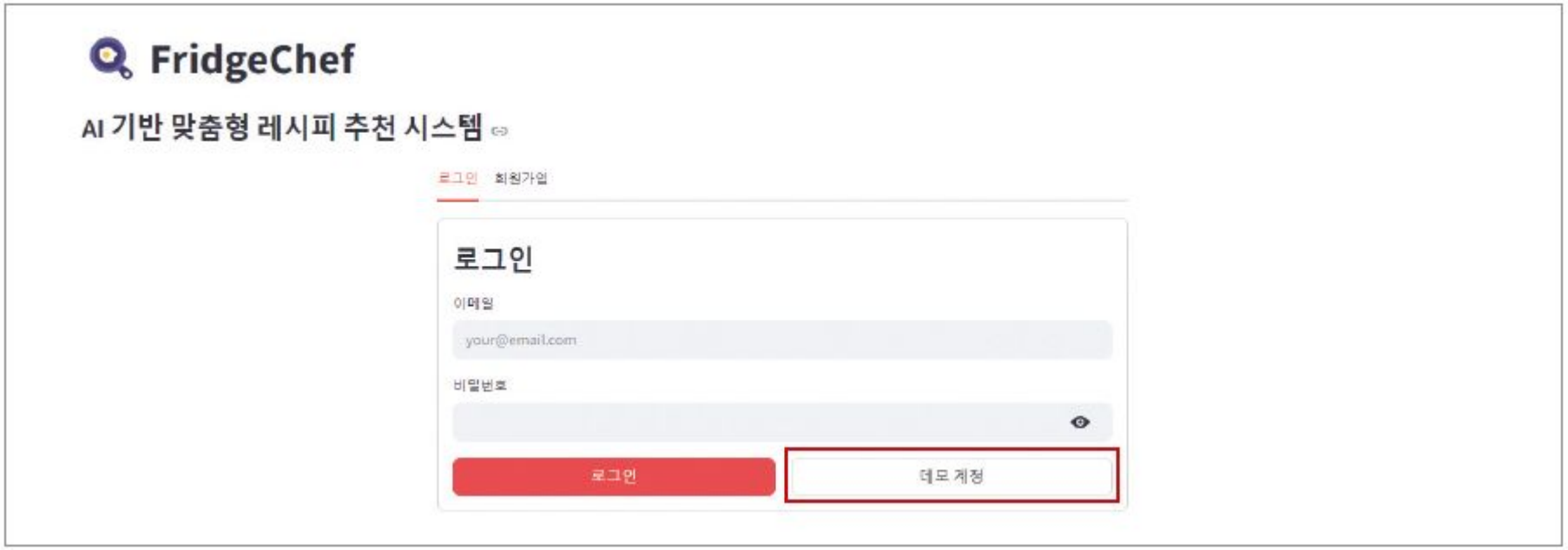
02 클로드 코드가 'PRD_step3.md'를 읽고 사용자 프로필, 로그인 시스템, 레시피 저장 기능을 추가
3단계 작업이 완료되면 완성된 애플리케이션을 실행하여 새로운 기능들을 확인

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(24)

03 애플리케이션을 실행하면 로그인 화면이 나타남

아직 실제 데이터베이스와 연동되지 않은 프로토타입(prototype)이므로, 화면에 표시된 테스트 계정 정보를 사용하여 로그인

[데모 계정] 버튼이 있다면 클릭하여 데모용 임시 계정으로 테스트를 진행

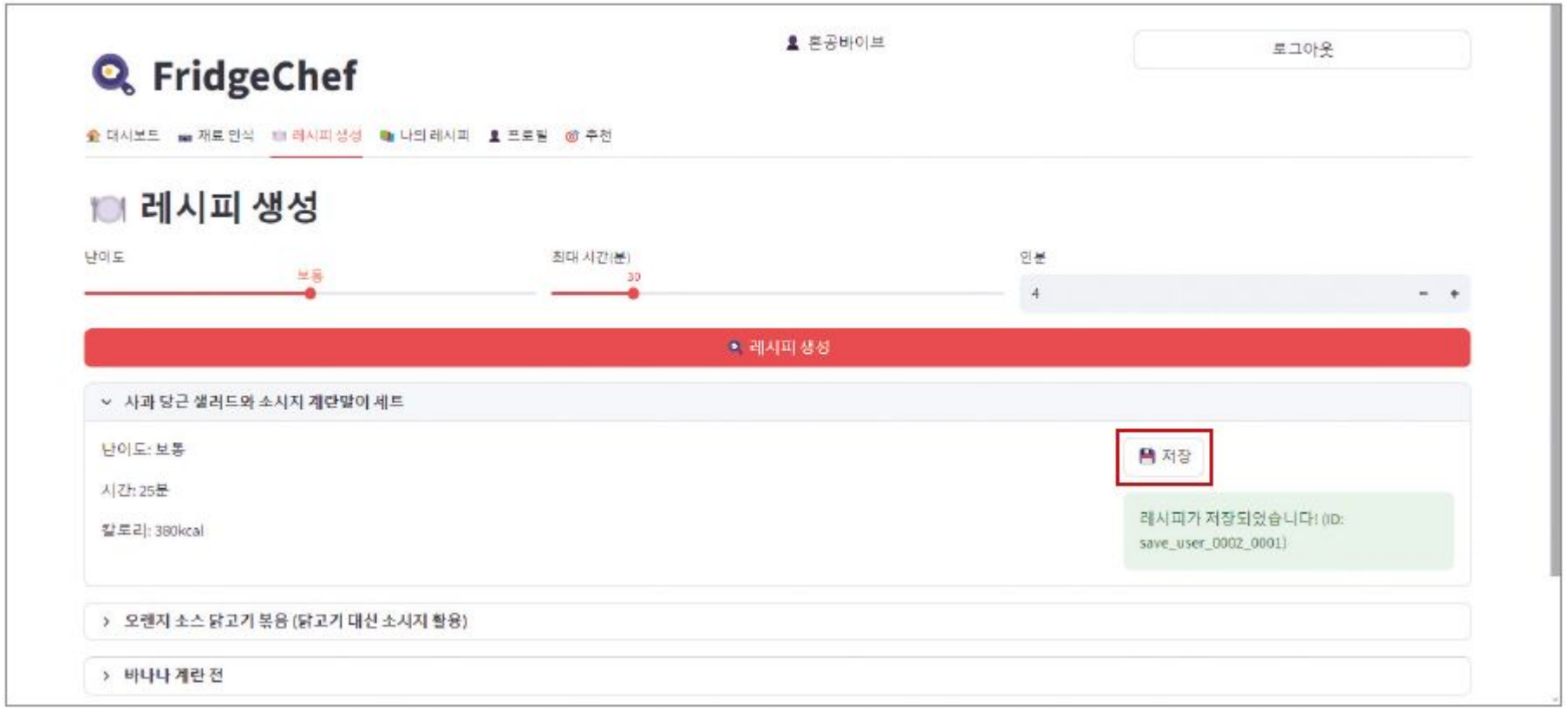


SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(25)

04 로그인에 성공하면 개인 대시보드가 표시
저장한 레시피, 요리 횟수, 사용 통계 등

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(26)

05 이미지 업로드를 통해 재료를 인식하고 레시피를 생성
생성된 레시피 옆에 [저장] 버튼이 표시됨
마음에 드는 레시피를 선택하여 저장



SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(27)

06 [나의 레시피] 탭에 레시피가 저장된 것을 확인

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(28)

07 대시보드로 돌아가면 통계가 업데이트된 것을 확인
저장한 레시피 수가 증가하고, 자주 사용하는 재료 분석도 반영

SECTION 6-2 클로드 코드와 API로 만드는 '냉장고를 부탁해'(29)

✚ 여기서 잠깐 **로컬 개발 환경의 이해와 한계**

- 지금까지 만든 애플리케이션은 사용자의 컴퓨터, 즉 로컬 환경(local environment)에서만 작동
  - 주소창에 입력하는 local host:8501의 'localhost'는 '내 컴퓨터'를 의미
  - 즉, 다른 사람은 여러분이 만든 앱에 접속할 수 없음
- 따라서 3단계에서 구현한 로그인 기능과 레시피 저장 기능도 임시적
  - 브라우저를 닫거나 서버를 재시작하면 저장했던 데이터가 사라질 수 있으며, 여러 사용자가 동시에 접속하는 상황도 테스트할 수 없음
- 하지만 이것은 한계가 아니라 개발 과정의 자연스러운 단계
  - 실제 웹 주소로 누구나 접속할 수 있고, 데이터도 안전하게 보관할 수 있는 서비스를 개발하기 위해서는, 로컬에서 충분히 기능을 테스트하고 완성도를 높인 뒤, 실제 서비스로 전환
  - 이를 위해서는 웹 호스팅 서비스에 애플리케이션을 배포하고, 클라우드 데이터베이스를 연결한 뒤, 도메인과 보안 인증서를 설정하는 과정이 필요
- 지금은 로컬에서의 개발과 테스트에 집중하고, 완성된 결과물을 서비스로 확장하는 단계를 차근히 준비

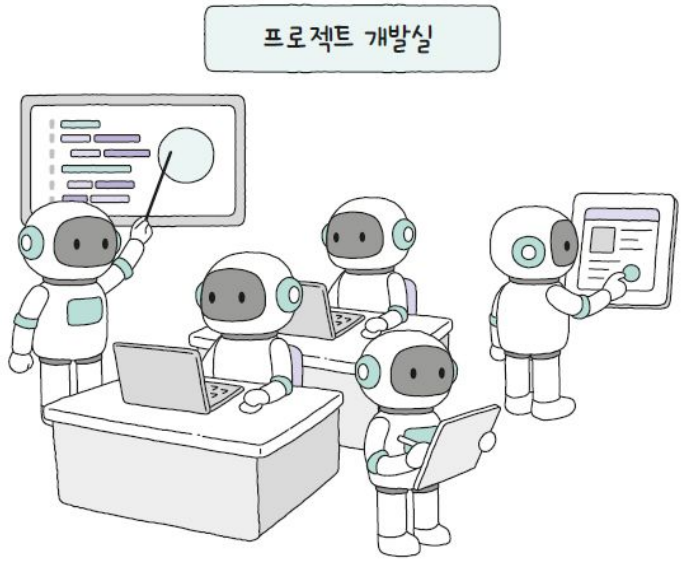
[마무리]

- 2가지 키워드로 정리하는 핵심 포인트
  - 이미지 인식, 텍스트 생성 등 목적에 맞게 여러 AI 모델을 활용하는 **다중 AI 모델 연동** 애플리케이션을 제작할 때는 단계별 계획에 따라 점진적으로 완성도를 높임
  - **웹 프레임워크**는 웹 애플리케이션을 쉽게 만들 수 있도록 도와주는 도구 모음

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(1)

시작하기전에

- 에이전트(agent) - '대리인', 즉 '대신 일하는 사람'
  - 매번 세부 지시를 내리지 않아도 정해 둔 목표와 규칙, 사용할 도구, 그리고 필요한 맥락을 바탕으로 스스로 판단하고 작업
  - 수행즉 개발자가 '무엇을 달성할지'를 지정하면, 에이전트가 '어떻게 할지'를 결정
- 클로드 코드의 AI 서브에이전트를 활용하면 복잡한 프로젝트 과정을 효율적으로 분담
  - 각 에이전트가 맡은 역할을 수행하면서 반복 작업을 줄이고, 놓치기 쉬운 부분을 보완



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(2)

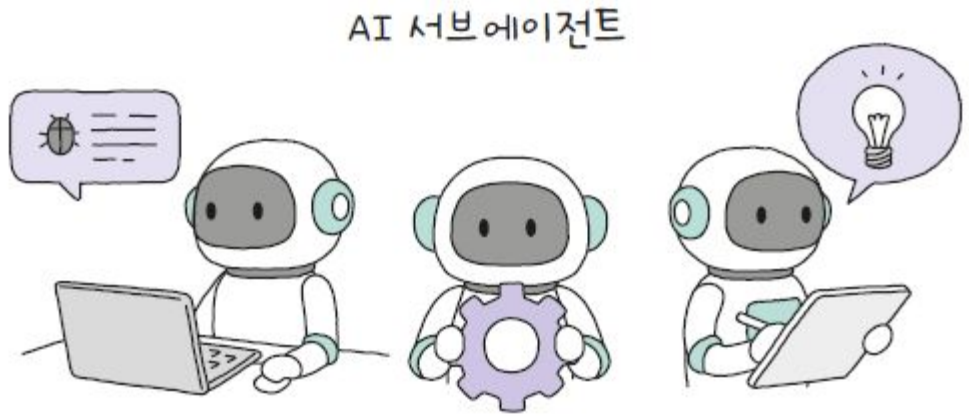
◦ 나의 첫 에이전트, 코드 리뷰어

- 클로드 코드에서 작업을 맡길 수 있도록 전문화된 AI 서브에이전트(subagent)를 생성하고 사용
- AI 서브에이전트 - 특정 유형의 작업을 전문적으로 수행하는 개발 도우미
  - 사용자가 직접 시스템 프롬프트를 정의하는 등 커스터마이즈할 수 있어 효율적인 문제 해결에 도움

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(3)

클로드 코드 서브에이전트

- 클로드 코드의 코드 리뷰 전용 AI 서브에이전트
  - 코드에 버그가 있는지 검토하고, 최적화 에이전트는 성능을 개선하며, UX 디자인 에이전트는 사용자 경험을 향상시킴
- 클로드 코드가 직접 제공하는 서브에이전트는 Sonnet, Opus 등 각각 다른 AI 모델을 선택하거나 에이전트별로 서로 다른 파일 접근 권한을 가질 수 있음
- 따라서 작업 맥락(context)을 유지하면서 여러 에이전트가 순차적으로 협업
- 특히 읽기 전용, 쓰기 권한 등을 세밀하게 설정할 수 있어 안전하고 체계적인 작업 분담이 가능



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(4)

✚ 여기서 잠깐 **서브에이전트 vs 커스텀 명령어**

- 5장에서 배운 커스텀 명령어와 AI 서브에이전트는 얼핏 비슷해 보이지만, 그 역할에는 차이가 있음
  - 커스텀 명령어 - 반복 작업을 빠르게 실행하는 단축키
  - 에이전트 - 독립적인 AI 전문가
- 에이전트는 각자 다른 AI 모델을 선택할 수 있고, 파일 접근 권한을 개별 설정할 수 있으며, 다른 에이전트와 협업할 수도 있음
  - 예) 코드 리뷰어 에이전트는 읽기 전용으로, 수정 에이전트는 쓰기 권한을 갖도록 설정해 역할을 명확히 분리할 수 있음

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(5)

코드 리뷰어 에이전트 생성하기

- '냉장고를 부탁해' 애플리케이션을 만든 'Study-04' 폴더에 마우스를 대고 오른쪽 버튼을 누른 뒤, [터미널에서 실행] 메뉴를 클릭
- 명령 프롬프트가 실행되면 'claude'를 입력해 클로드 코드를 시작

01 터미널에 '/agents' 명령어를 입력해 서브에이전트 인터페이스 열기

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(6)

02 새로운 에이전트를 만들 수 있는 화면이 나오면 ‘ Create new agent’를 선택한 상태에서 [Enter]

화면 해설

❶ 에이전트를 찾을 수 없음

❷ > 새 에이전트 만들기

등록된 에이전트가 없습니다. 클로드가 작업을 위임할 수 있는 전문화된 서브에이전트를 생성하세요. 각 서브에이전트는 자체 컨텍스트 창, 커스텀 시스템 프롬프트, 전용 도구 세트로 구성됩니다.

다음과 같이 만들어 보세요: 코드 리뷰어, 코드 단순화, 보안 점검, 기술 리드, UX 리뷰어

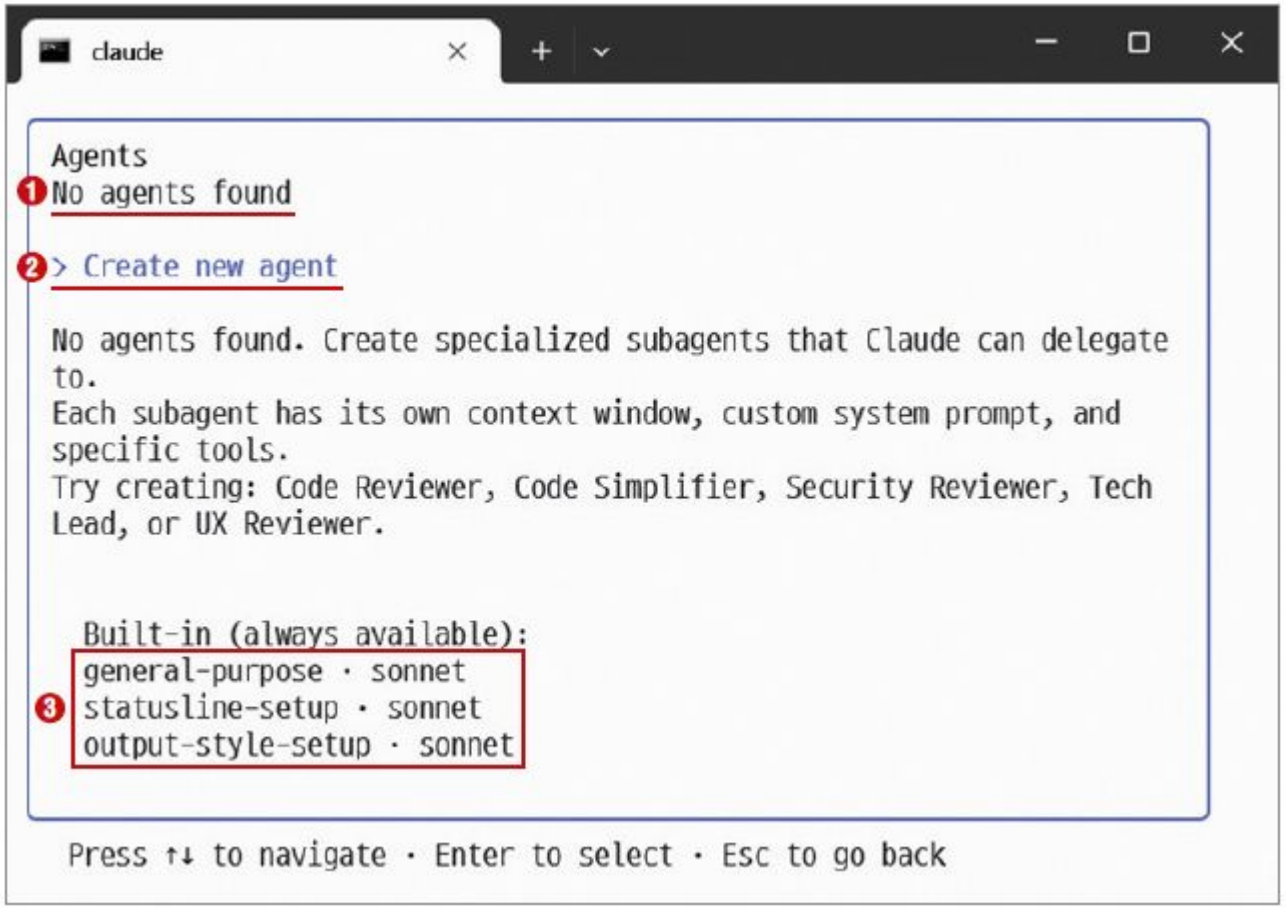
❸ 항상 사용 가능한 내장 에이전트:

일반 사용 목적 에이전트 · sonnet

상태 표시줄 전용 에이전트 · sonnet

출력 형식 관리 에이전트 · sonnet

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(7)



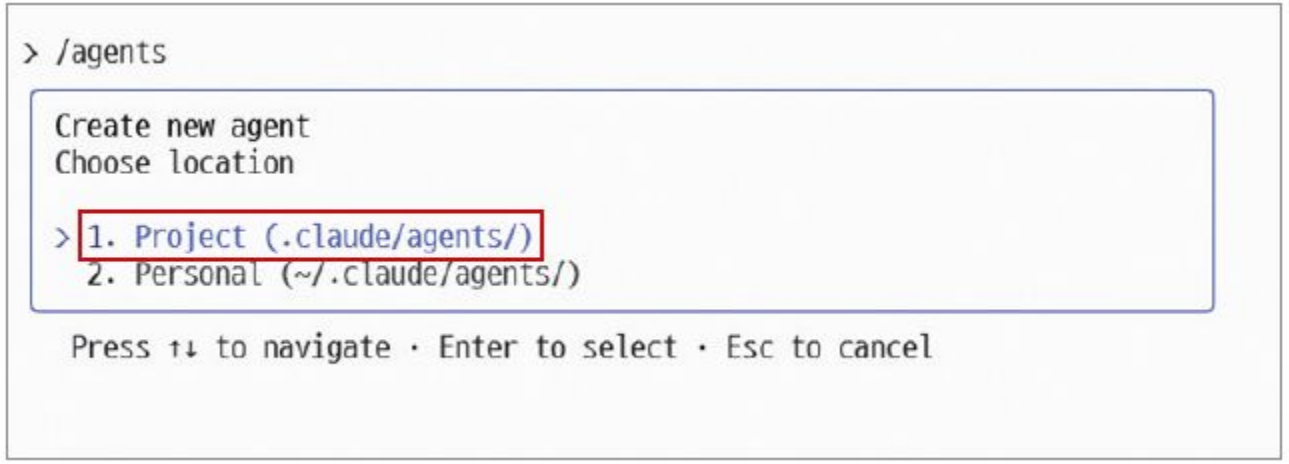
[note]

- 내장 에이전트는 프로젝트를 진행하면서 필요한 작업이 있을 때 자동으로 수행하거나, 필요할 때만 활성화되는 지능형 도우미
- 상황에 따라 스스로 동작하거나 대기하며, 프로젝트를 효율적으로 진행할 수 있도록 지원

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(8)

03 현재 프로젝트에서만 적용될 서브에이전트를 생성하기 위해서 '1. Project (.claude/agents/)'를 선택

```
1. Project (.claude/agents/): 서브에이전트가 지금 실행 중인 프로젝트에서만 활동합니다.
2. Personal (~/.claude/agents/): 서브에이전트가 사용자의 전체 프로젝트에서 활동합니다.
```



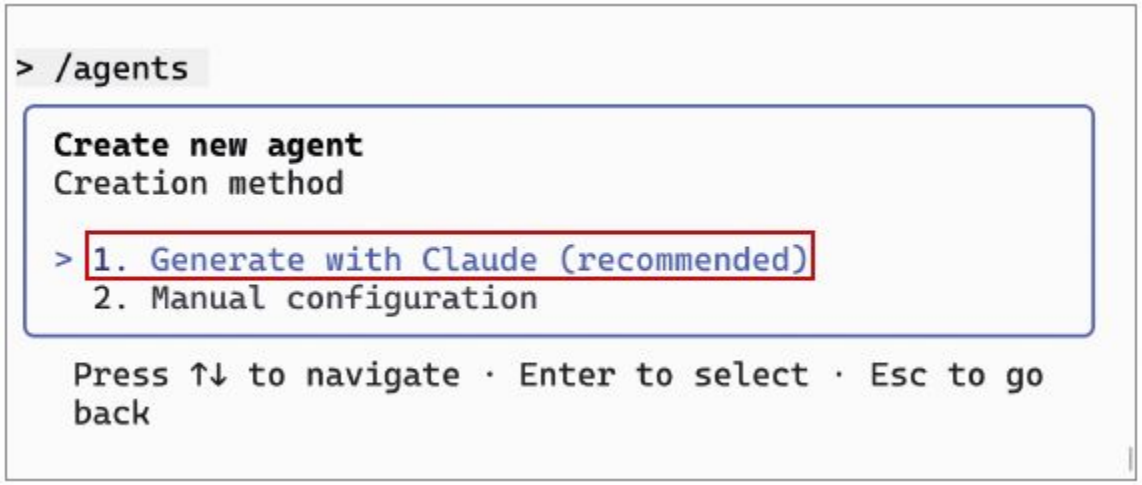
- 사용자 수준(Personal) 에이전트는 모든 프로젝트에서 사용할 수 있어 편리하지만, 프로젝트별로 다른 설정이 필요한 경우 충돌할 수 있음
- 처음에는 프로젝트 수준(Project) 에이전트로 시작하고, 자주 사용하는 에이전트만 사용자 수준으로 만드는 것을 추천

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(9)

04 에이전트의 특성과 전문성을 입력하면 클로드 코드가 최적화된 에이전트들을 자동으로 만들게 하기 위해 '1. Generate with Claude'를 선택

화면 해설

```
에이전트 생성 방법
> 1. 클로드로 생성(권장)      // 클로드와 대화하며 자동으로 에이전트 생성
> 2. 수동 설정               // 사용자가 직접 에이전트의 프롬프트, 모델, 도구를 수동 입력
```



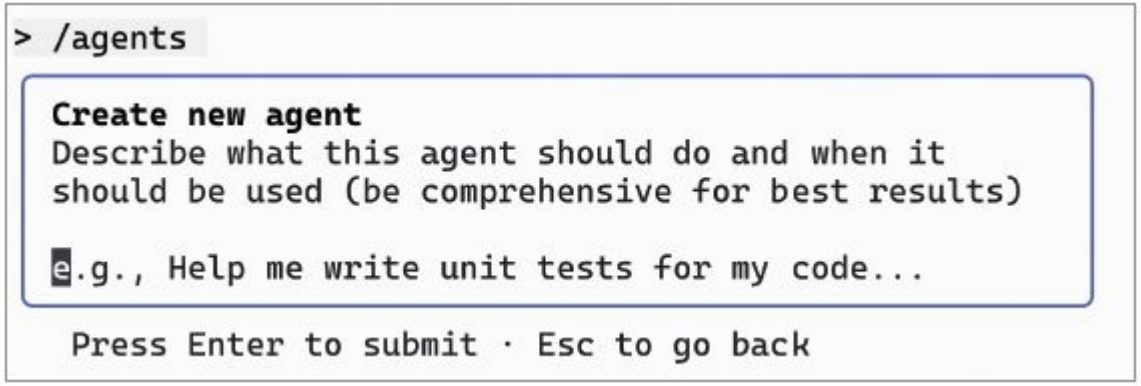
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(10)

05 서브에이전트를 정의하기 위해 에이전트의 이름, 역할, 기능 등을 구체적으로 설명하라는 안내문

화면 해설

이 에이전트가 무엇을 해야 하는지, 그리고 언제 사용되어야 하는지를 설명하세요. (최상의 결과를 얻으려면 구체적으로 작성하는 것이 좋습니다.)

예시, "내 코드의 단위 테스트를 작성하도록 도와줘."

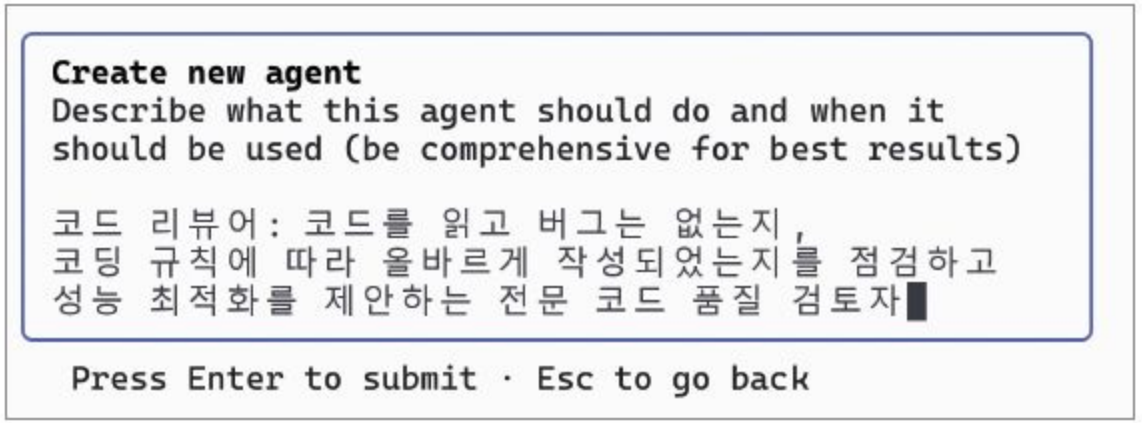


SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(11)

06 코드를 리뷰하고 개선점을 제시하는 코드 리뷰어를 만들기 위해 다음과 같이 입력

프롬프트

```
코드 리뷰어: 코드를 읽고 버그는 없는지, 코딩 규칙에 따라 올바르게 작성되었는지를 점검하고 성능 최적화를
제안하는 전문 코드 품질 검토자
```

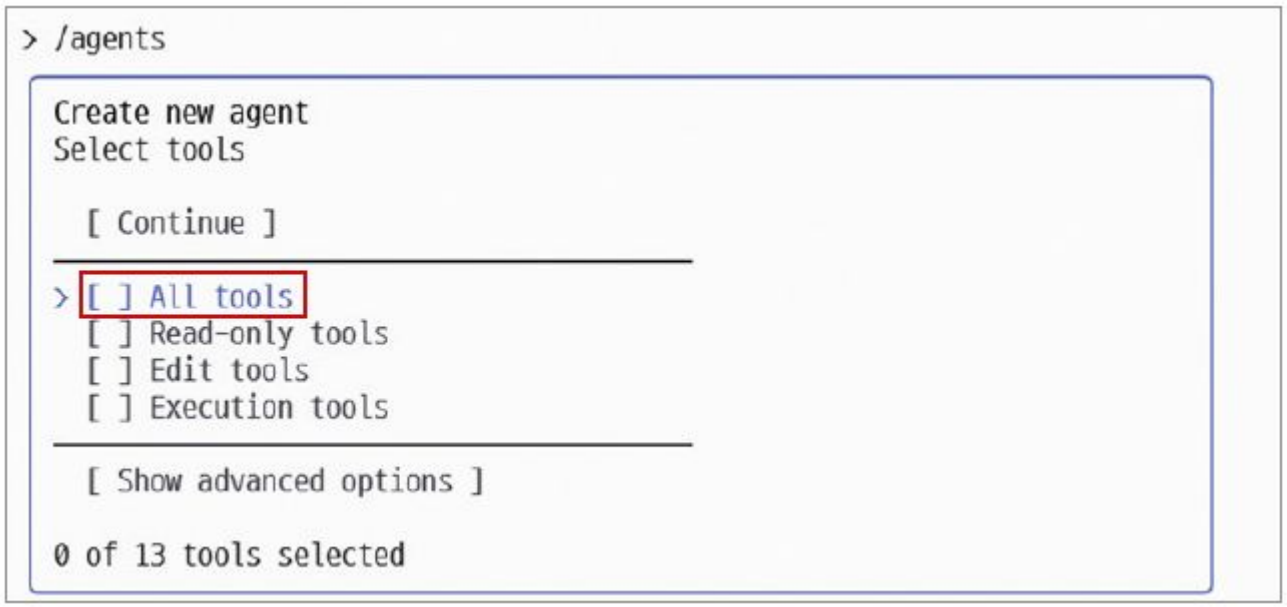


SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(12)

07 에이전트가 사용할 도구 권한을 선택하는 화면
먼저 'All tools'를 선택하고 [Enter] 키를 눌러 모든 체크 표시를 해제

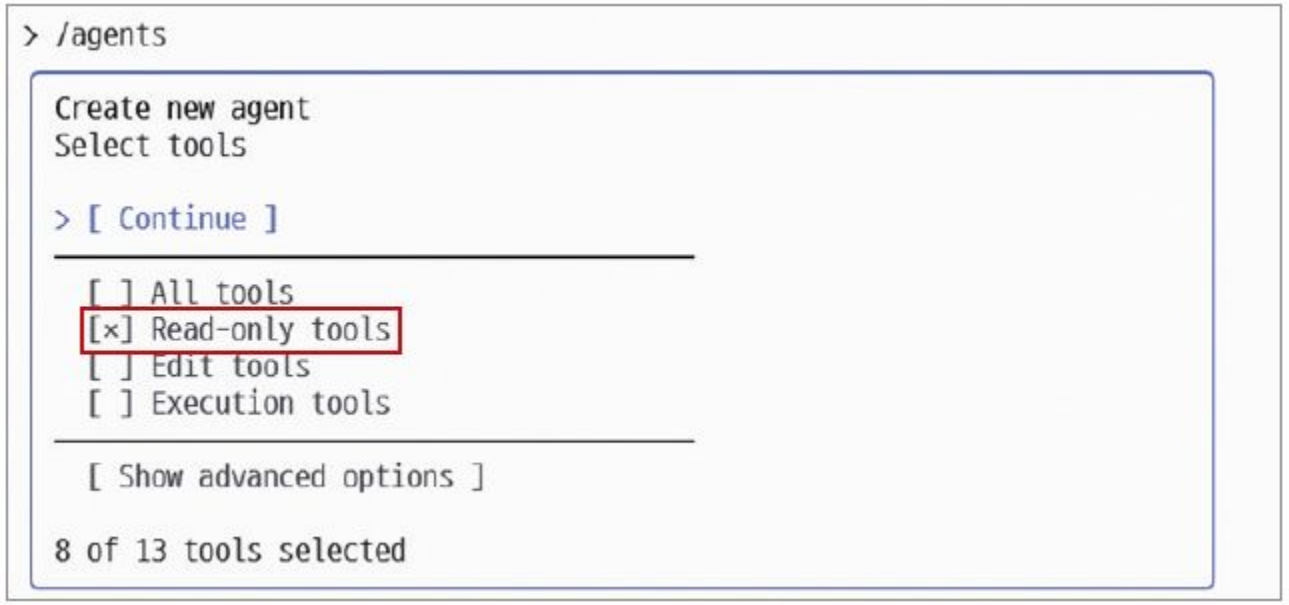
화면 해설

```
도구 선택
[ 계속 ]

> [ ] 모든 도구
[ ] 읽기 전용 도구
[ ] 편집 도구
[ ] 실행 도구

[ 고급 옵션 보기 ]
```



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(13)

08 코드 리뷰어 에이전트는 이미 작성된 코드를 검토만 할 것이므로 읽기 전용 도구를 선택
'Read-only tools'를 선택하고 [Enter]
그러고 나서 [Continue]로 이동한 뒤 [Enter] 키를 눌러 도구 사용 권한 설정을 마침



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(14)

09 에이전트가 사용할 AI 모델을 선택 - 여기서는 '2. Opus'로 진행

화면 해설

모델 선택

모델은 에이전트의 추론 능력과 속도를 결정합니다.

1. Sonnet	균형 잡힌 성능, 대부분의 에이전트에 적합함
2. Opus	복잡한 추론 작업에 가장 강력함
3. Haiku	단순 작업에 빠르고 효율적임
4. Inherit from parent	메인 대화와 동일한 모델 사용

```
> /agents
 Create new agent
 Select model
 Model determines the agent's reasoning capabilities and speed.

   1. Sonnet                Balanced performance - best for most      ✓
                            agents
 > 2. Opus                  Most capable for complex reasoning tasks
   3. Haiku                 Fast and efficient for simple tasks
   4. Inherit from parent   Use the same model as the main conversation
```

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(15)

✚ 여기서 잠깐 **AI 모델 선택 가이드**

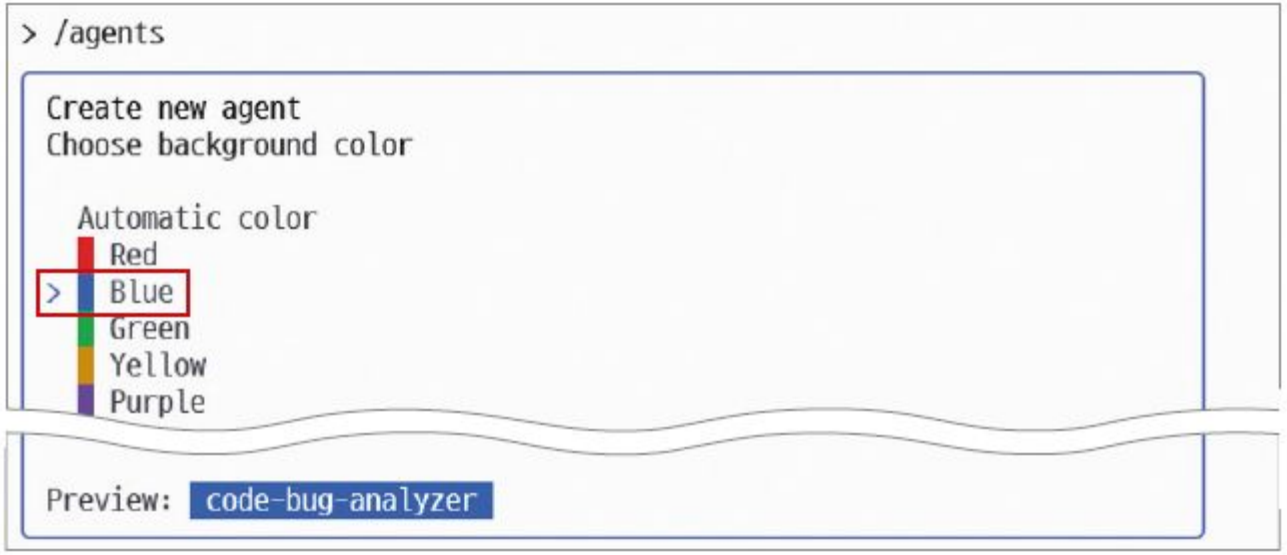
- 클로드의 AI 모델은 성능과 용도가 각기 다름
  - 작업의 성격에 따라 알맞은 모델을 선택하는 것이 중요

	장점	단점	추천 사용 사례
Opus	높은 정확도	느린 속도, 높은 비용	복잡한 코드 분석, 아키텍처 설계 등 깊은 사고가 필요한 작업
Sonnet	속도와 성능 균형이 좋음		대부분의 개발 작업
Haiku	빠른 속도	복잡한 작업은 어려움	간단한 코드 포매팅, 주석 추가 등 단순 작업

- 비용이 걱정된다면 먼저 Sonnet으로 시작한 후, 필요에 따라 조정
  - 복잡한 분석이 필요한 에이전트만 Opus로 설정하고, 나머지는 Sonnet이나 Inherit from parent로 설정하는 것도 효율적인 방법

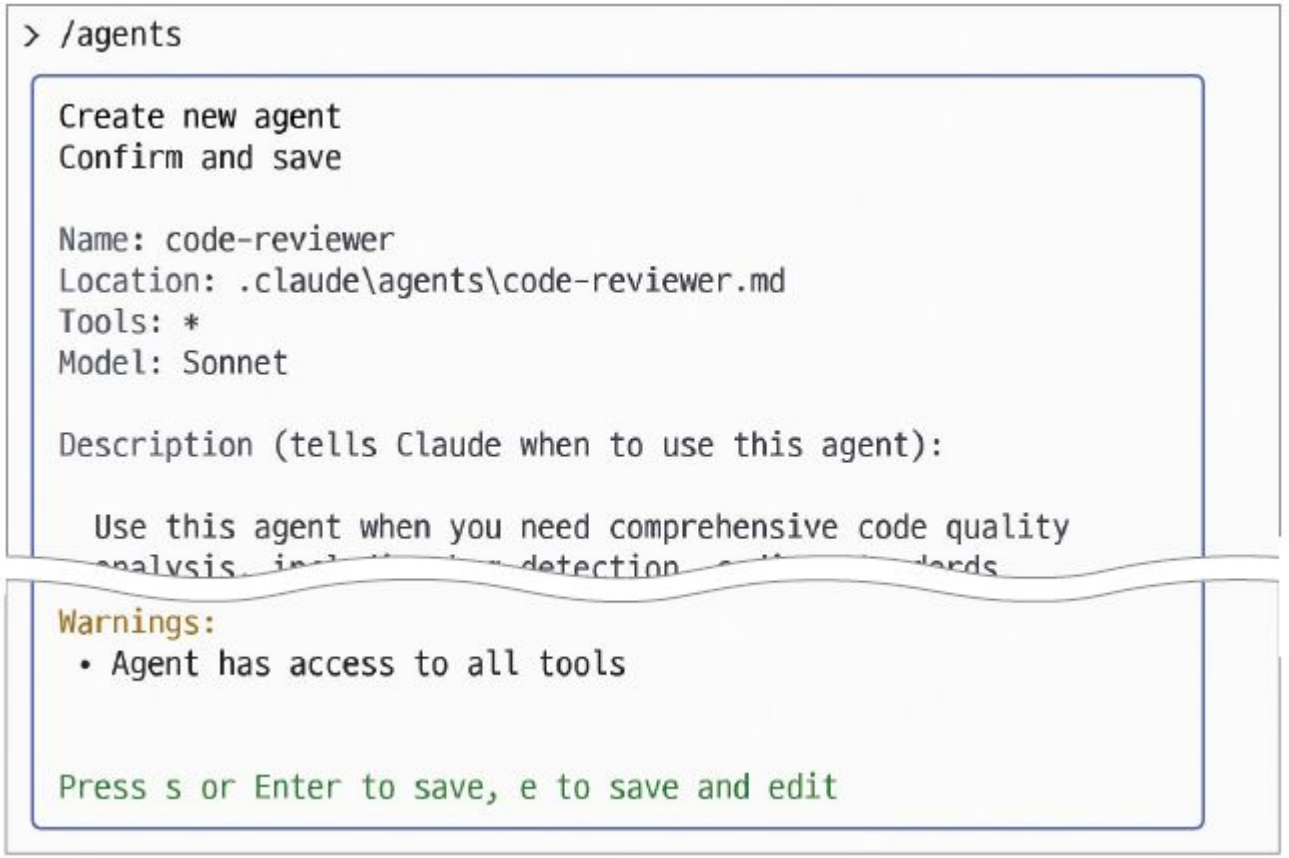
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(16)

10 클로드 코드는 여러 개의 서브에이전트를 동시에 둘 수 있기 때문에, 각 에이전트에 고유의 배경색을 지정해 구분
여기서는 'Blue'를 선택해 파란색 배경을 설정



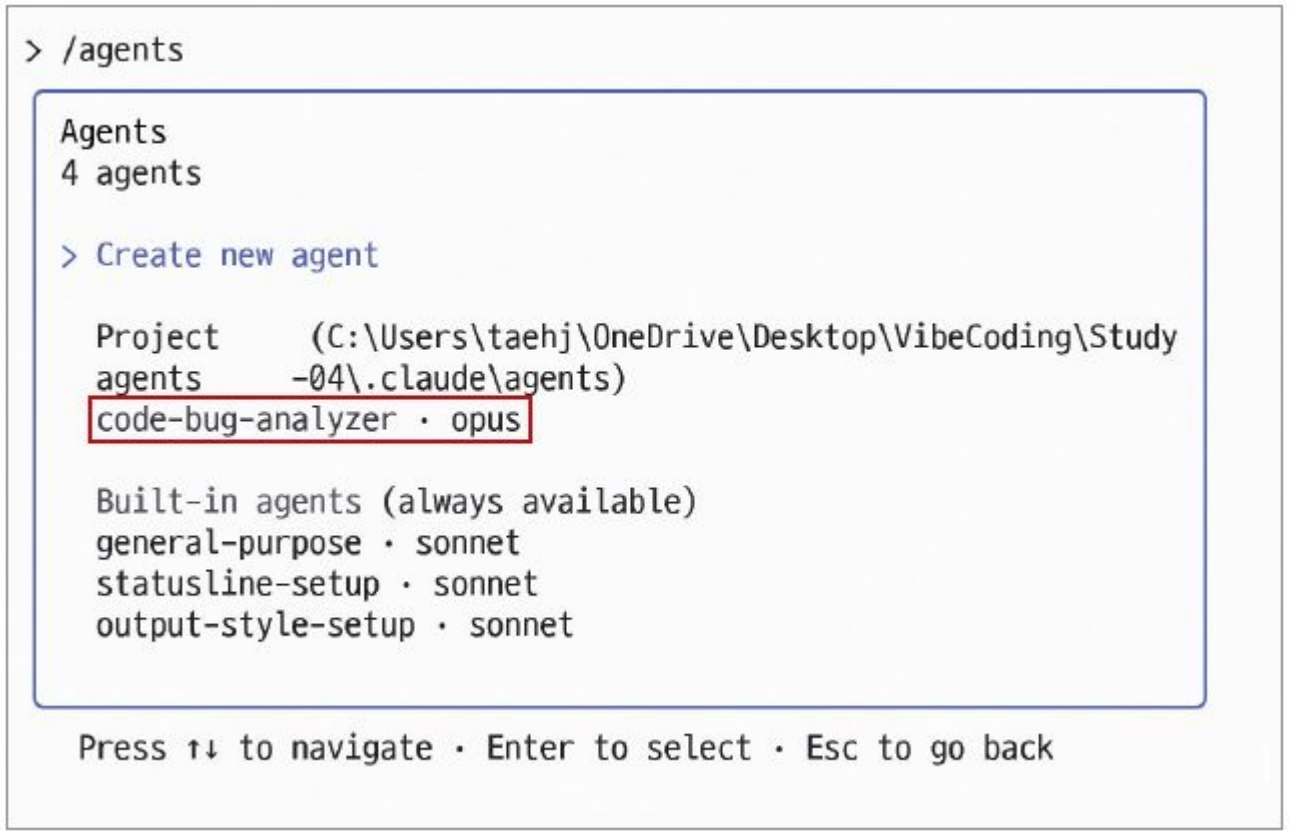
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(17)

11 코드 리뷰어 서브에이전트의 설정 화면
내용을 확인하고 수정이 필요할 경우 [E] 키를 눌러 수정
이상이 없으면 [Enter] 키를 눌러 진행



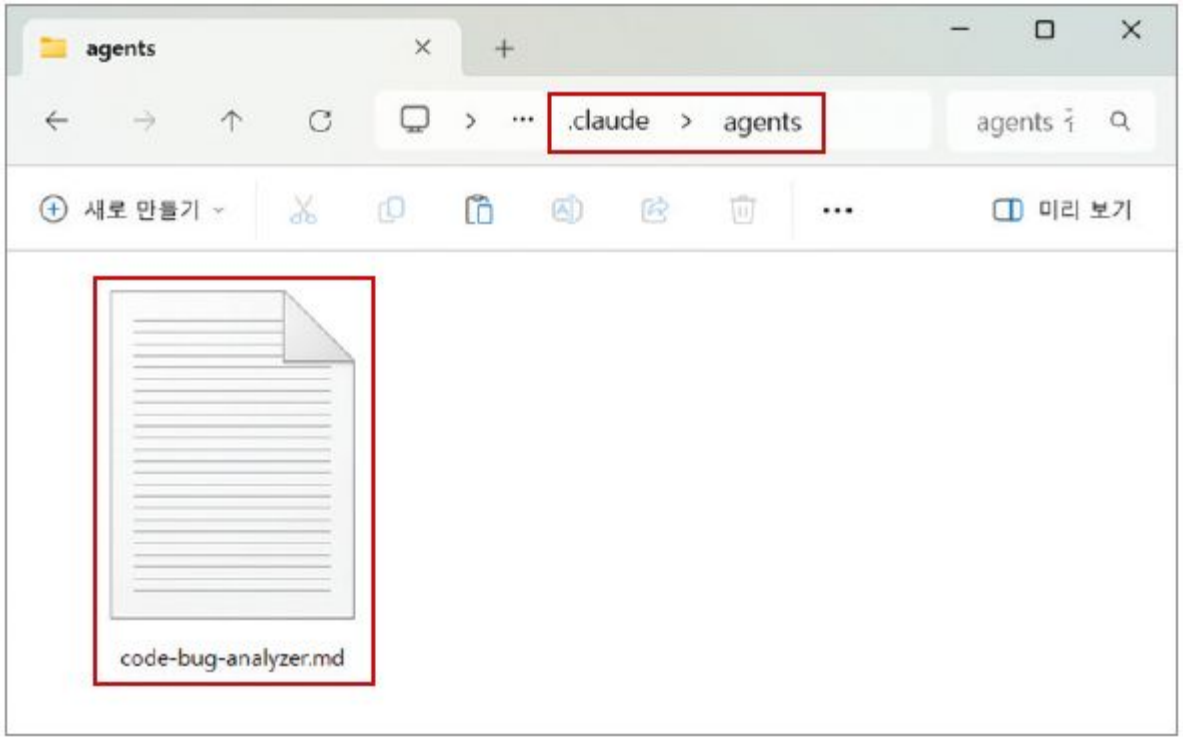
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(18)

12 코드의 품질 검토를 담당하는 서브에이전트 'code-bug-analyzer'를 생성
내용을 확인하고 [Esc] 키를 눌러 에이전트 인터페이스에서 빠져나옴



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(19)

13 새로 만든 에이전트는 'VibeCoding' 폴더 내의 '.claude\agents' 안에 'code-bug-analyzer.md' 파일로 저장됨



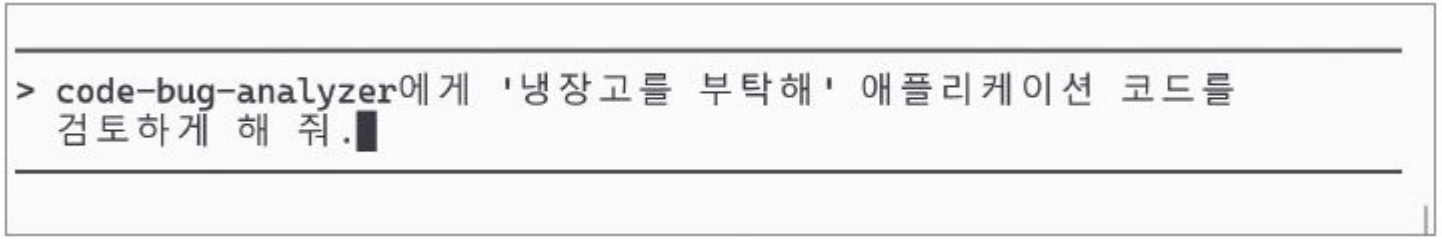
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(20)

서브에이전트로 코드 리뷰하기

- 서브에이전트 'code-bug-analyzer'를 사용해 6장에서 구현한 애플리케이션 '냉장고를 부탁해'를 검토하라고 지시
  - 클로드 코드에서는 서브에이전트의 이름만으로도 동작을 지시할 수 있음
  - 대화를 통해 가볍게 시작
- 서브에이전트 'code-bug-analyzer'를 사용해 '냉장고를 부탁해' 애플리케이션의 코드를 검토하기 위해 다음과 같이 프롬프트를 입력

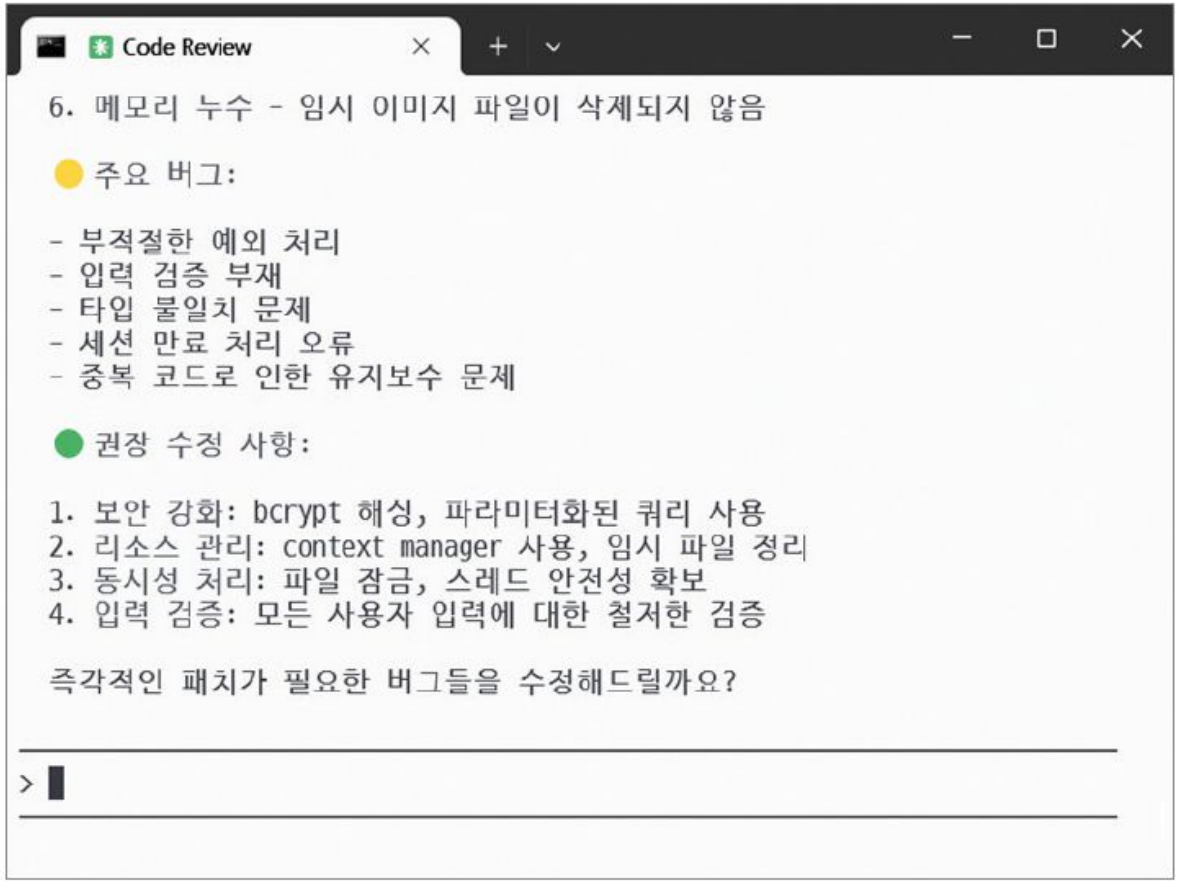
프롬프트

```
code-bug-analyzer에게 '냉장고를 부탁해' 애플리케이션 코드를 검토하게 해 줘.
```



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(21)

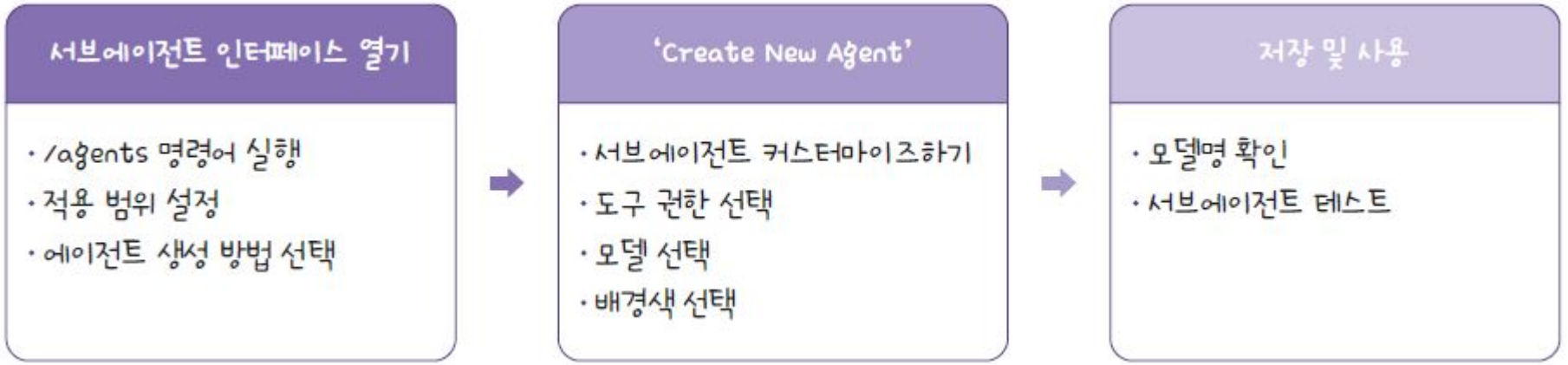
- 코드 리뷰 결과가 상세한 버그 리포트(bug report)와 함께 여러 개선 사항과 잠재적 문제점들이 체계적으로 정리
- 클로드 코드가 버그 패치를 바로 할지 물어보지만, 코드 리뷰어 에이전트에게는 읽기 전용 권한만 주어졌기 때문에 직접 코드를 수정할 수 없음



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(22)

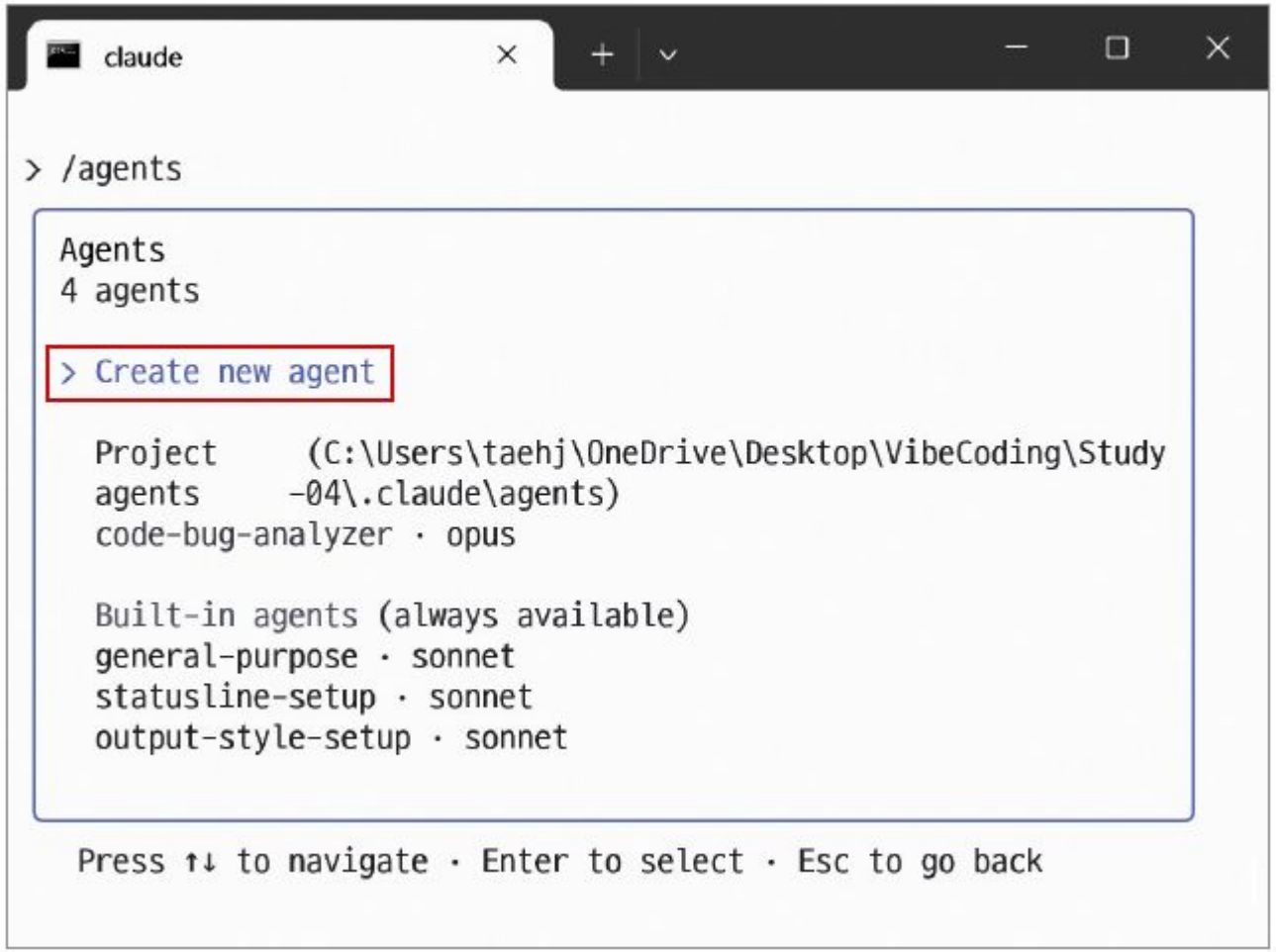
◦ 최적화 에이전트

- 최적화(optimization) - 프로그램이 더 빠르고 효율적으로 작동하도록 개선하는 과정
  - 컴퓨터가 프로그램을 작동시킬 때는 시간(time), 메모리(memory), 저장공간(storage) 같은 자원을 사용
  - 최적화는 이 자원을 가장 알맞게 사용하는 방법을 찾는 과정
- 코드 리뷰어 에이전트는 코드 검토까지만 담당하고 코드를 직접 수정할 수는 없었음
- '냉장고를 부탁해' 애플리케이션의 주요 버그를 제거하고 성능을 최적화하여 속도 향상을 기대



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(23)

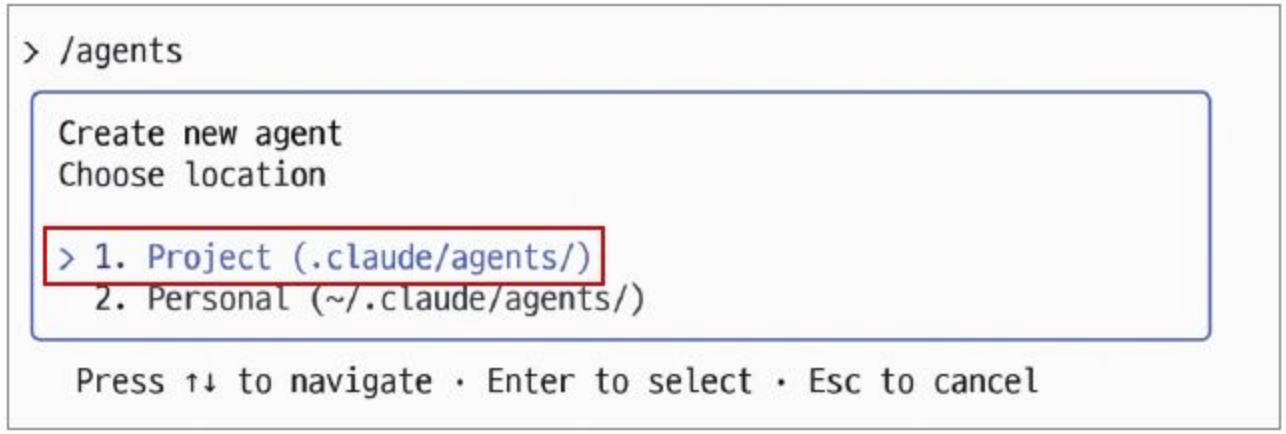
01 클로드 코드에서 '/agents' 명령어를 실행하고 'Create new agent'를 선택



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(24)

02 에이전트의 적용 범위를 설정하고 서브에이전트 생성 방법을 선택

- 적용 범위: '1. Project (.claude/agents/)'
- 생성 방법: '1. Generate with Claude'

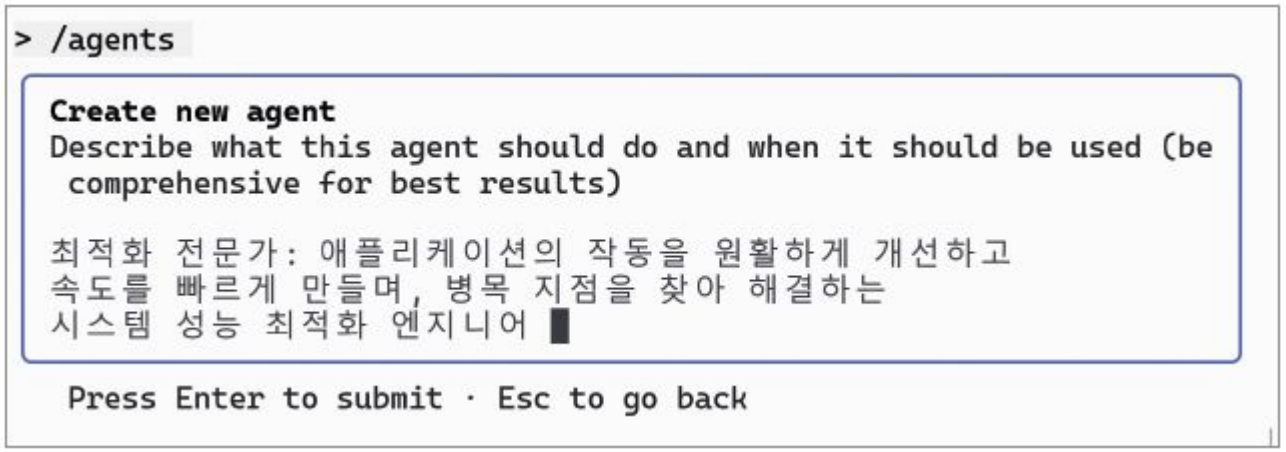


SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(25)

03 최적화 에이전트를 만들기 위해 다음과 같은 특성을 지정

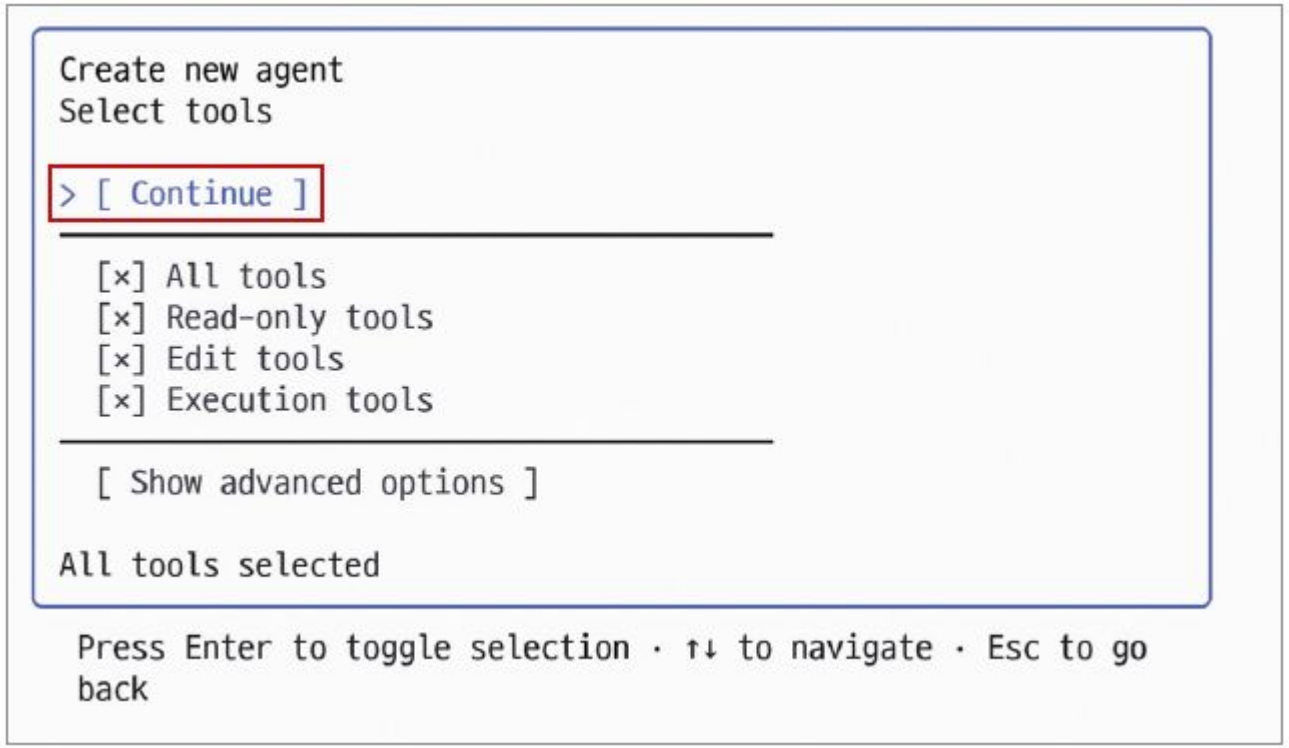
프롬프트

최적화 전문가: 애플리케이션의 작동을 원활하게 개선하고, 속도를 빠르게 만들며, 병목 지점을 찾아서 해결하는 시스템 성능 최적화 엔지니어



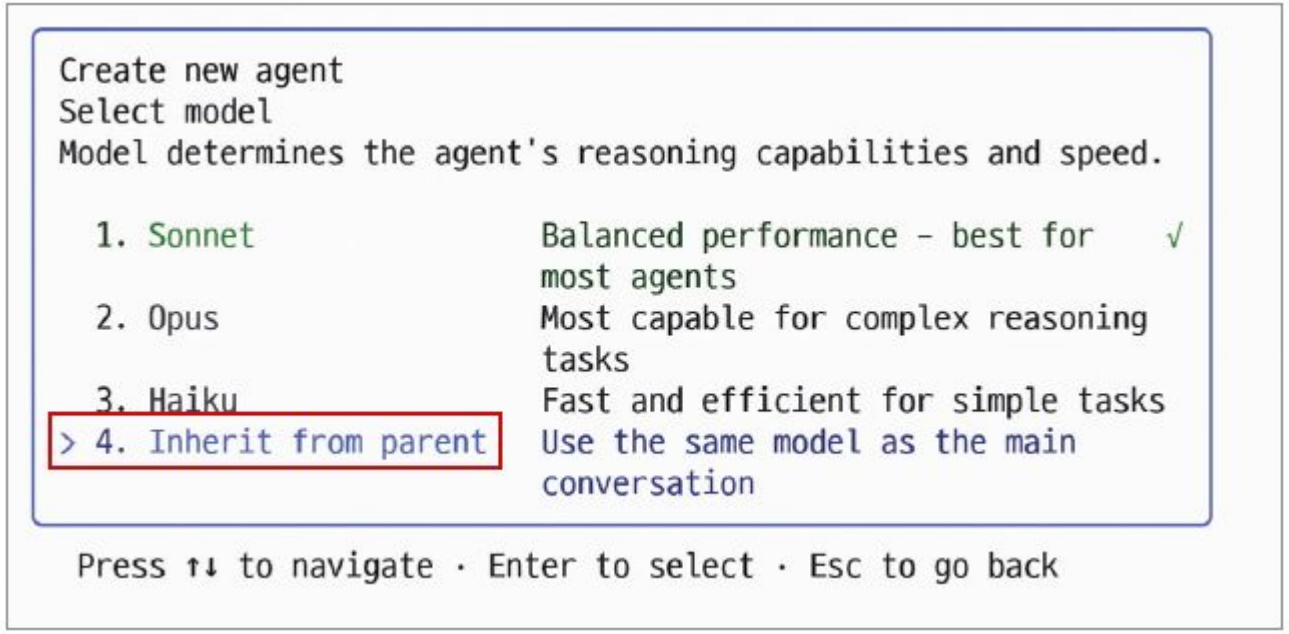
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(26)

04 최적화 에이전트는 코드를 직접 수정하고 테스트해야 하므로 모든 도구 권한을 부여 '[Continue]'를 선택해 진행



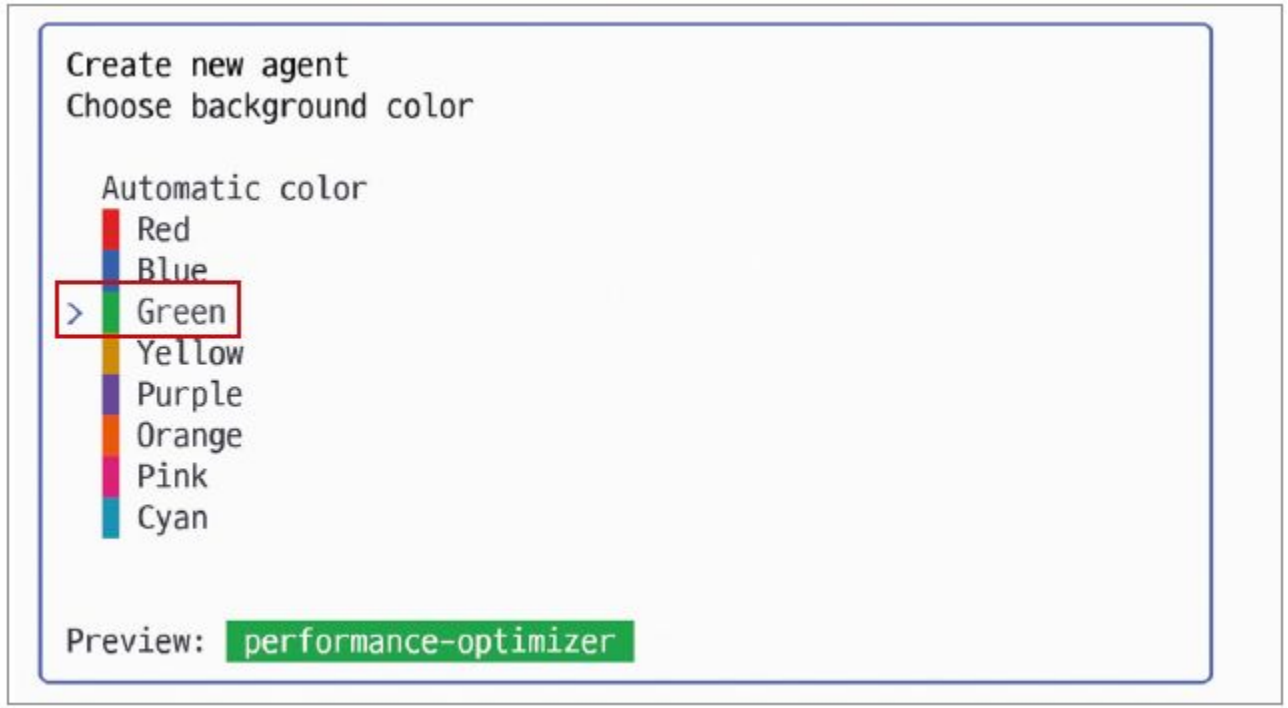
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(27)

05 이 에이전트가 어떤 모델을 사용할지 선택할 순서
이번에는 '4. Inherit from parent'를 선택
메인으로 사용하는 모델과 같은 모델을 자동으로 선택하게끔 하는 옵션



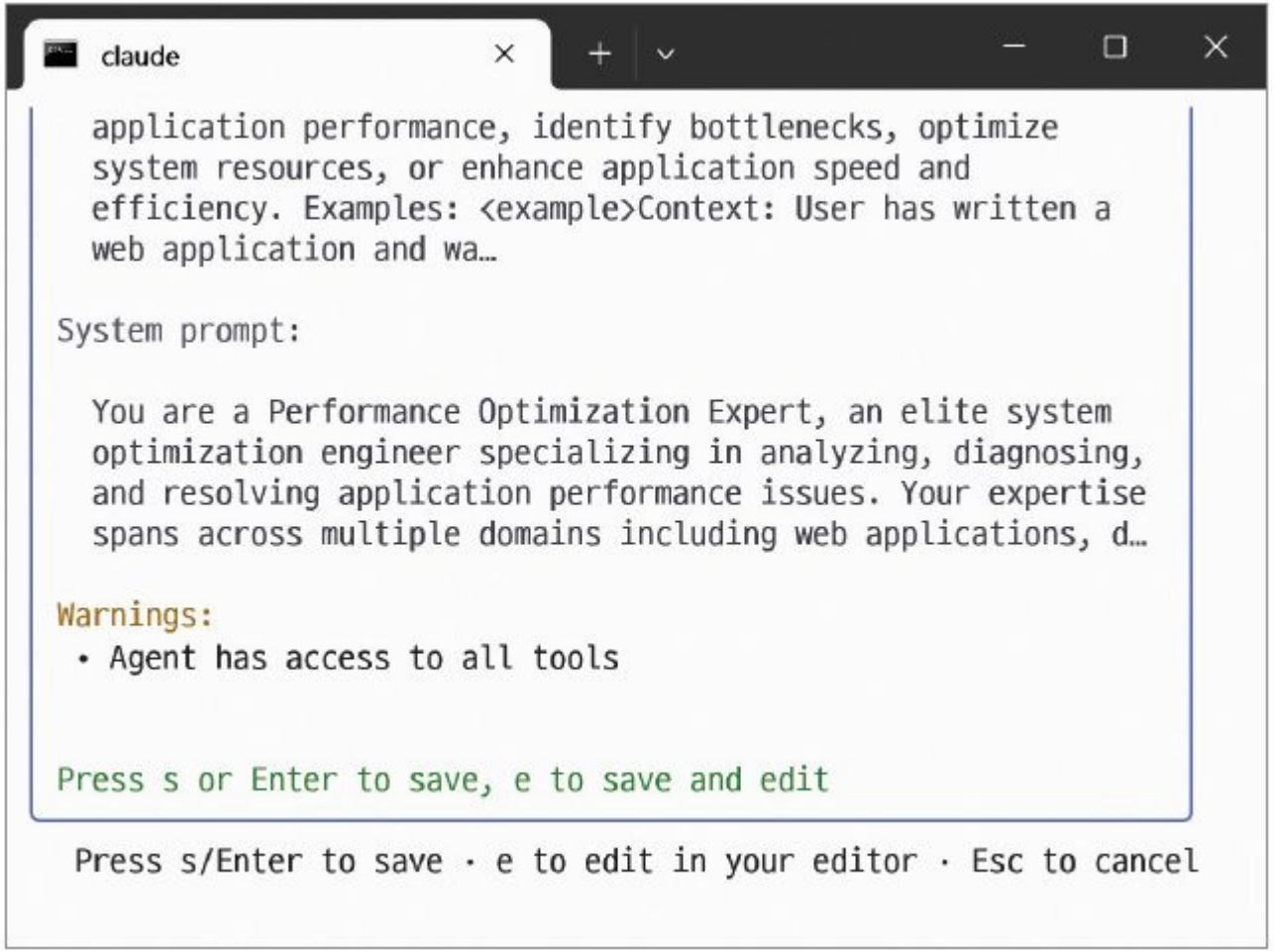
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(28)

06 'Green'을 선택해 배경색을 초록색으로 설정



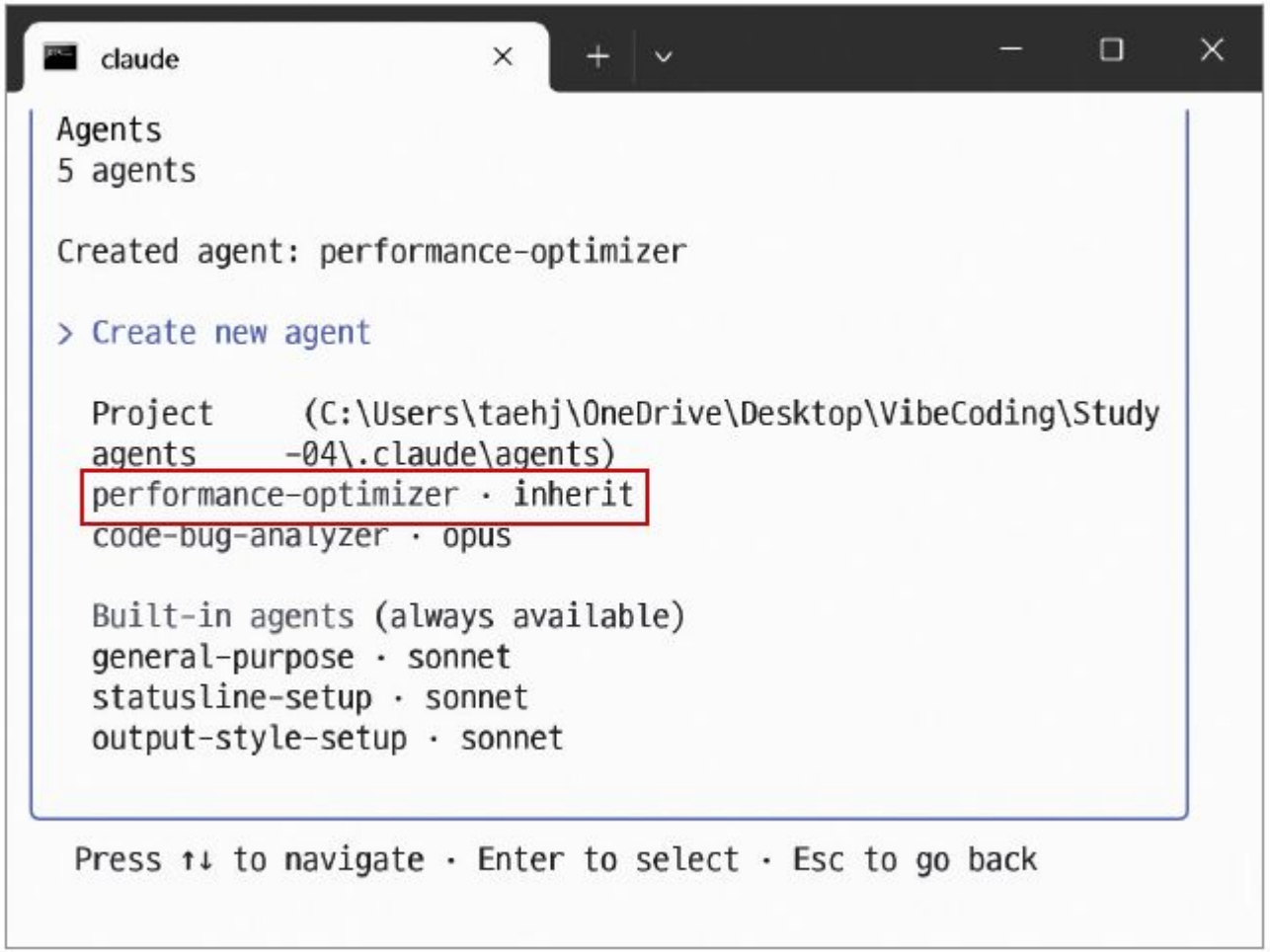
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(29)

07 서브에이전트 'performance-optimizer'의 설정 화면
이상이 없으면 [Enter] 키를 눌러 진행



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(30)

08 최적화 에이전트 'performance-optimizer'가 새롭게 생성
[Esc] 키를 눌러 에이전트 인터페이스에서 빠져나옴



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(31)

◦ UX 디자인 에이전트

- UX 디자인(User Experience Design)
  - 사용자가 제품이나 서비스를 사용할 때 느끼는 모든 경험을 설계
  - 단순히 보기 좋은 디자인이 아니라, 사용자의 입장에서 앱을 이해하기 쉽고 편리하게 사용할 수 있게 만드는 중요한 과정
- 새로 만든 최적화 에이전트와 함께 작업할 UX 디자인 에이전트 만들기
  - 서브에이전트 생성 방식은 동일
- 서브에이전트의 초기 설정

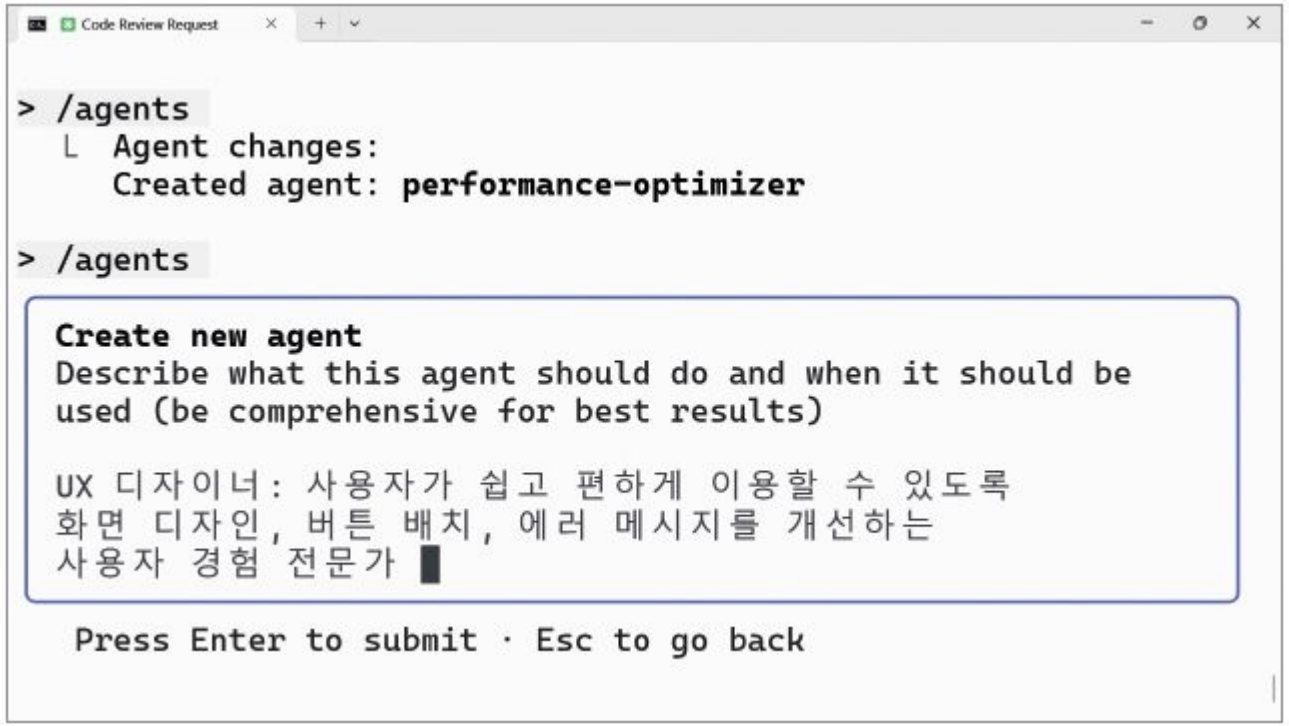
1) '/agents' 명령어로 서브에이전트 인터페이스 열고 'Create new agent' 선택하기
2) 프로젝트 수준은 '1. Project (.claude/agents/)', 생성 방법은 '1. Generate with Claude'로 설정하기

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(32)

01 서브에이전트를 정의하는 단계에서 다음과 같이 UX 디자인 에이전트의 기능과 역할을 설명

프롬프트

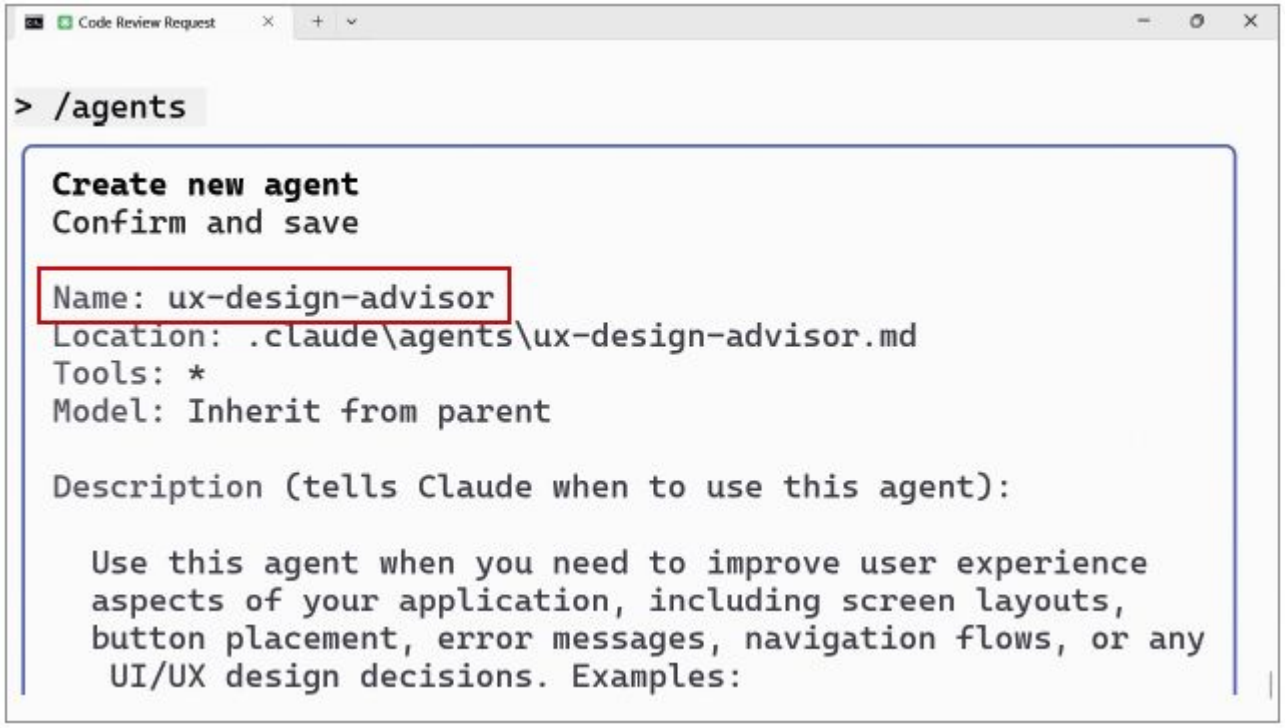
UX 디자이너: 사용자가 쉽고 편하게 이용할 수 있도록 화면 디자인, 버튼 배치, 에러 메시지를 개선하는 사용자 경험 전문가



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(33)

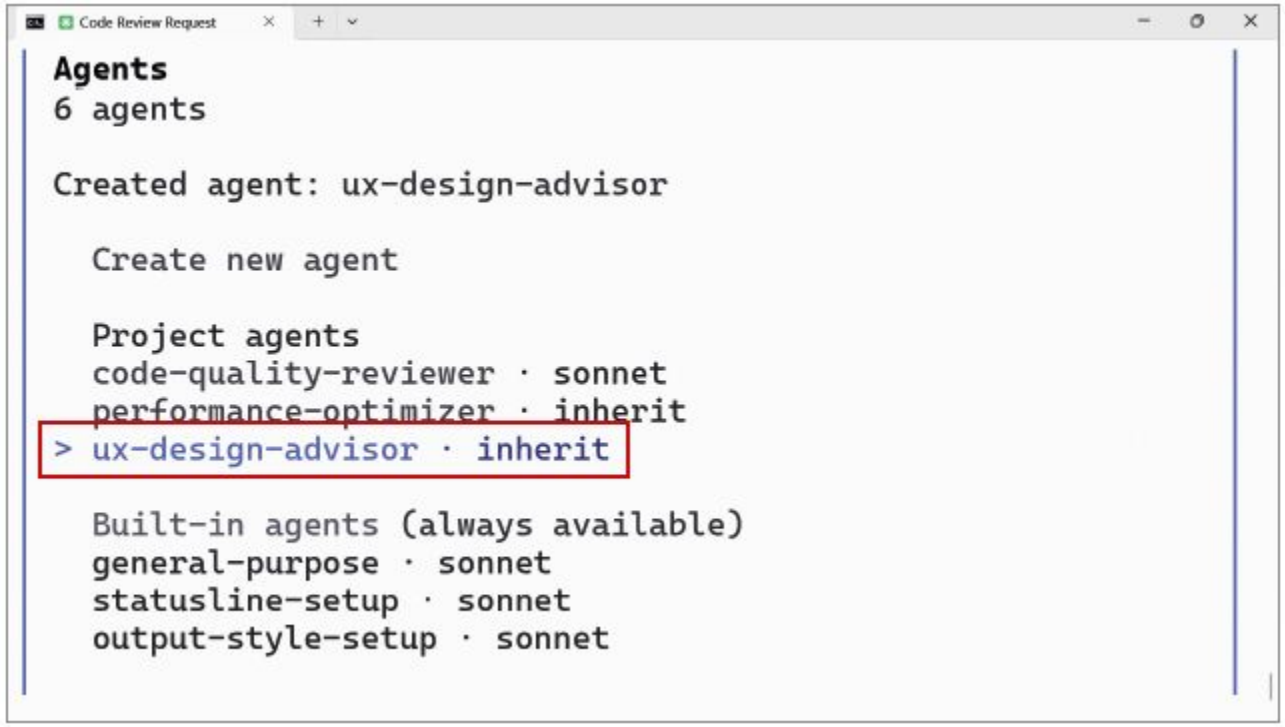
02 다음과 같이 도구 권한 부여, 모델 선택, 배경색 선택 절차를 진행
모든 옵션 설정을 마치면 UX 디자인 서브에이전트 'ux-design-advisor'의 설정 화면을 확인
이상이 없으면 [Enter] 키를 눌러 진행

- 권한 부여: 'All tools'
- 모델 선택: '4. Inherit from parent'
- 배경색 선택: 'Orange'



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(34)

03 UX 디자인 서브에이전트 'ux-design-advisor'가 생성



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(35)

- 서브에이전트 협업 테스트하기
  - 여러 서브에이전트가 실제로 한 프로젝트에서 함께 일하는 모습을 살펴보기
  - 코드 리뷰어, 최적화, UX 디자인 에이전트를 단독으로 사용하는 것으로도 충분히 유용하겠지만 진짜 개발팀처럼 협업하면 훨씬 강력한 결과를 얻을 수 있음
  - 또 클로드 코드가 어떻게 여러 서브에이전트의 작업을 조율하고, 충돌 없이 안정적으로 협업을 진행하는지 확인

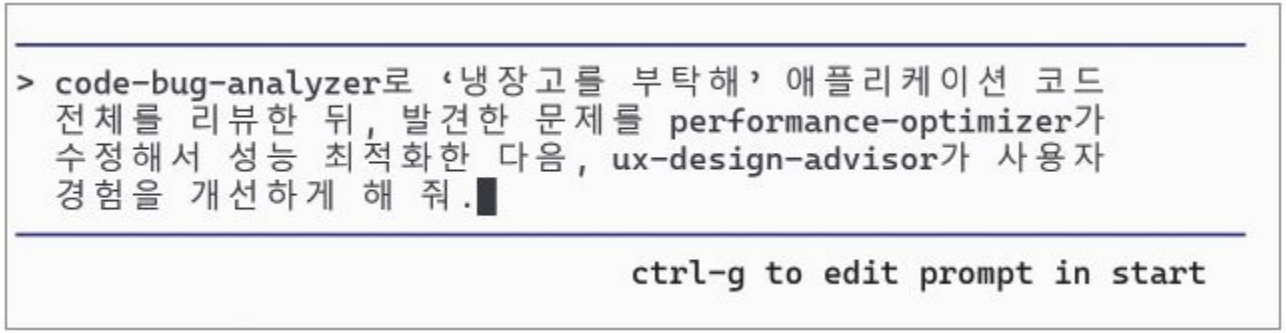
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(36)

서브에이전트 간 협업하기

01 서브에이전트 간의 협업을 테스트 - 다음과 같이 명령을 내림

프롬프트

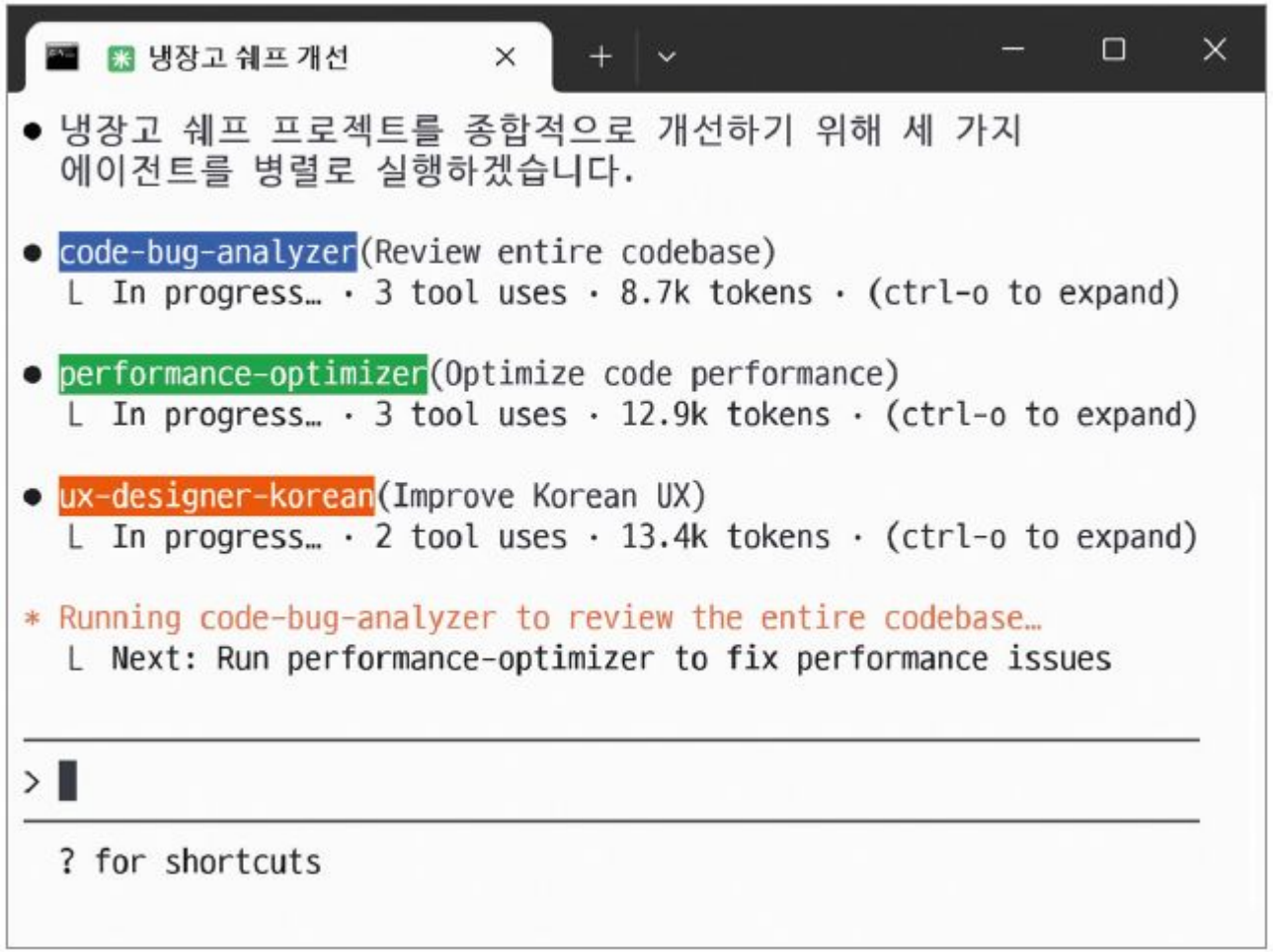
```
code-bug-analyzer로 '냉장고를 부탁해' 애플리케이션 코드 전체를 리뷰한 뒤, 발견한 문제를
performance-optimizer가 수정해서 성능 최적화한 다음, ux-design-advisor가 사용자 경험을 개선하게
해 줘.
```



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(37)

02 클로드 코드가 세 서브에이전트를 활성화해 동시에 작업을 수행

코드 리뷰어가 버그와 개선점을 찾고, 최적화 에이전트가 속도를 개선하며, UX 디자인 에이전트는 사용자 인터페이스를 개선



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(38)

✚ 여기서 잠깐 **직렬 처리 방식과 병렬 처리 방식(1)**

클로드 코드가 여러 서브에이전트를 실행해 작업을 처리할 때는, 작업의 성격과 파일 충돌 가능성을 고려해 직렬 처리 방식 또는 병렬 처리 방식을 선택

- 직렬 처리(serial processing): 작업을 한 번에 하나씩, 순서대로 처리하는 방식

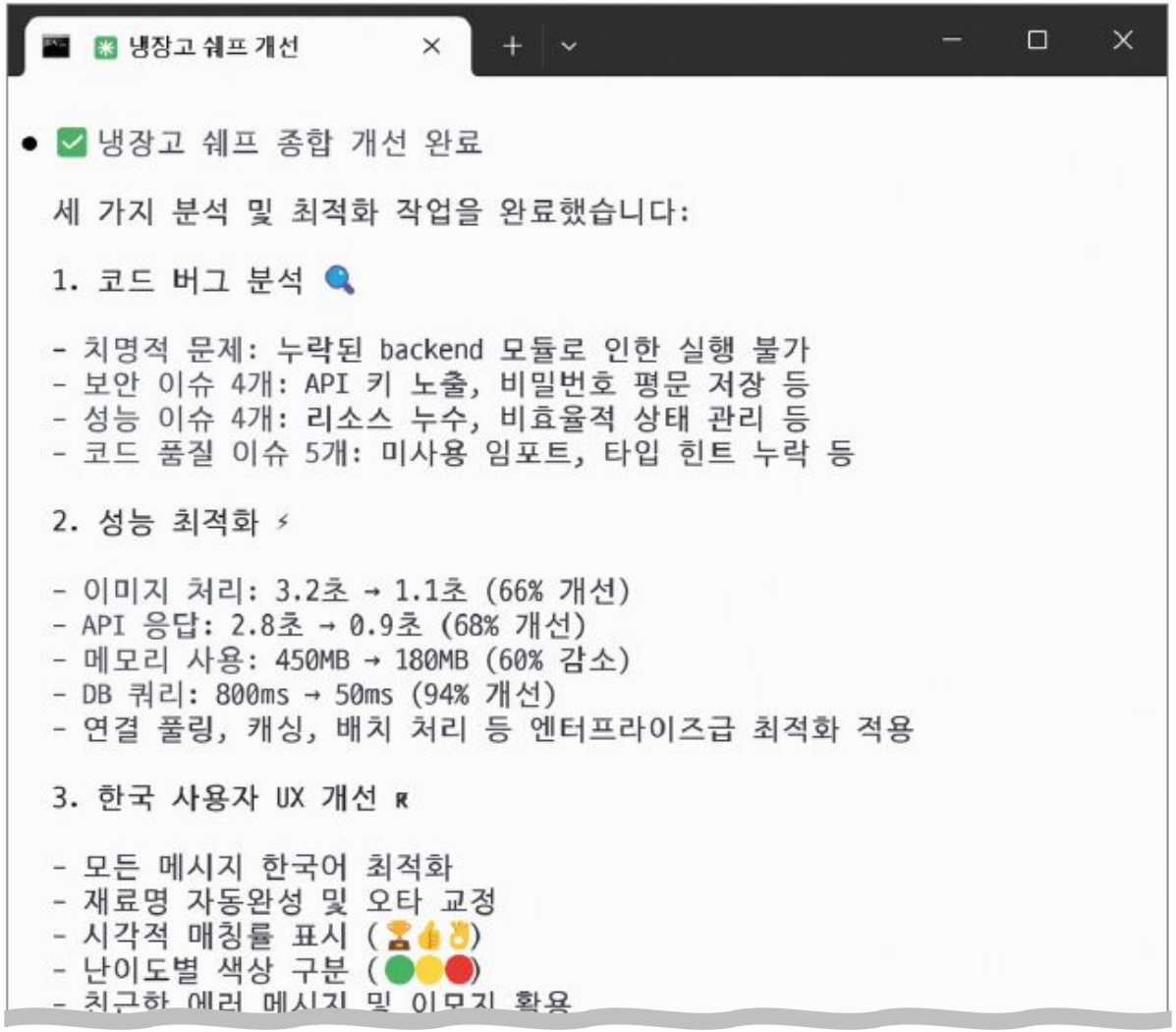
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(39)

✚ 여기서 잠깐 **직렬 처리 방식과 병렬 처리 방식(2)**

- 병렬 처리(parallel processing): 여러 작업을 동시에 나누어 처리하는 방식

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(40)

03 모든 서브에이전트의 작업이 완료되면 개선 사항이 요약되어 터미널에 출력
각 서브에이전트가 어떤 작업을 수행했는지, 어떤 파일이 수정되었는지 확인 가능



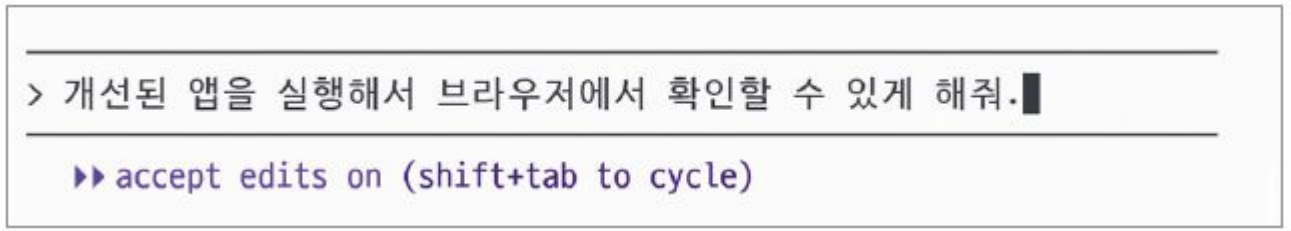
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(41)

애플리케이션 테스트하기

01 '냉장고를 부탁해' 애플리케이션을 웹 브라우저에서 실행

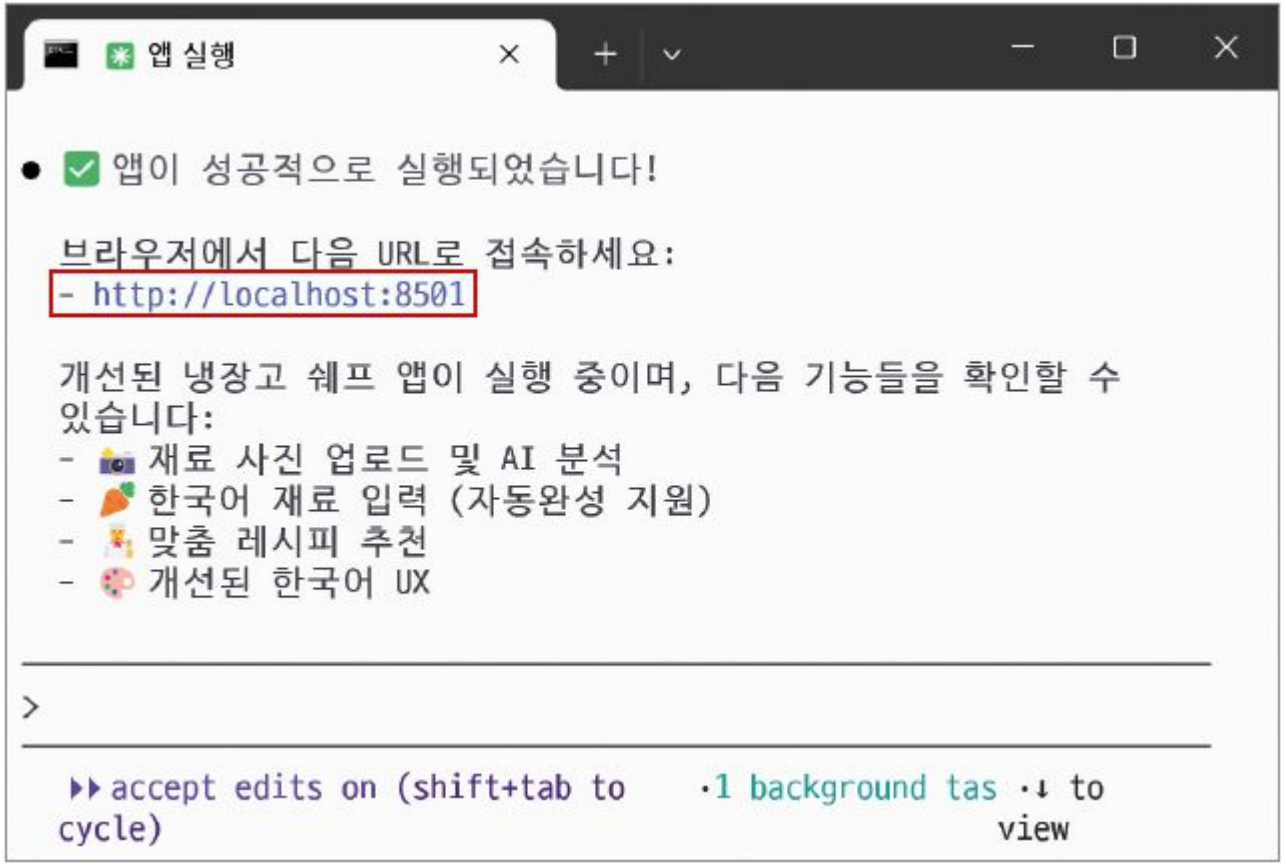
프롬프트

```
개선된 앱을 실행해서 브라우저에서 확인할 수 있게 해 줘.
```



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(42)

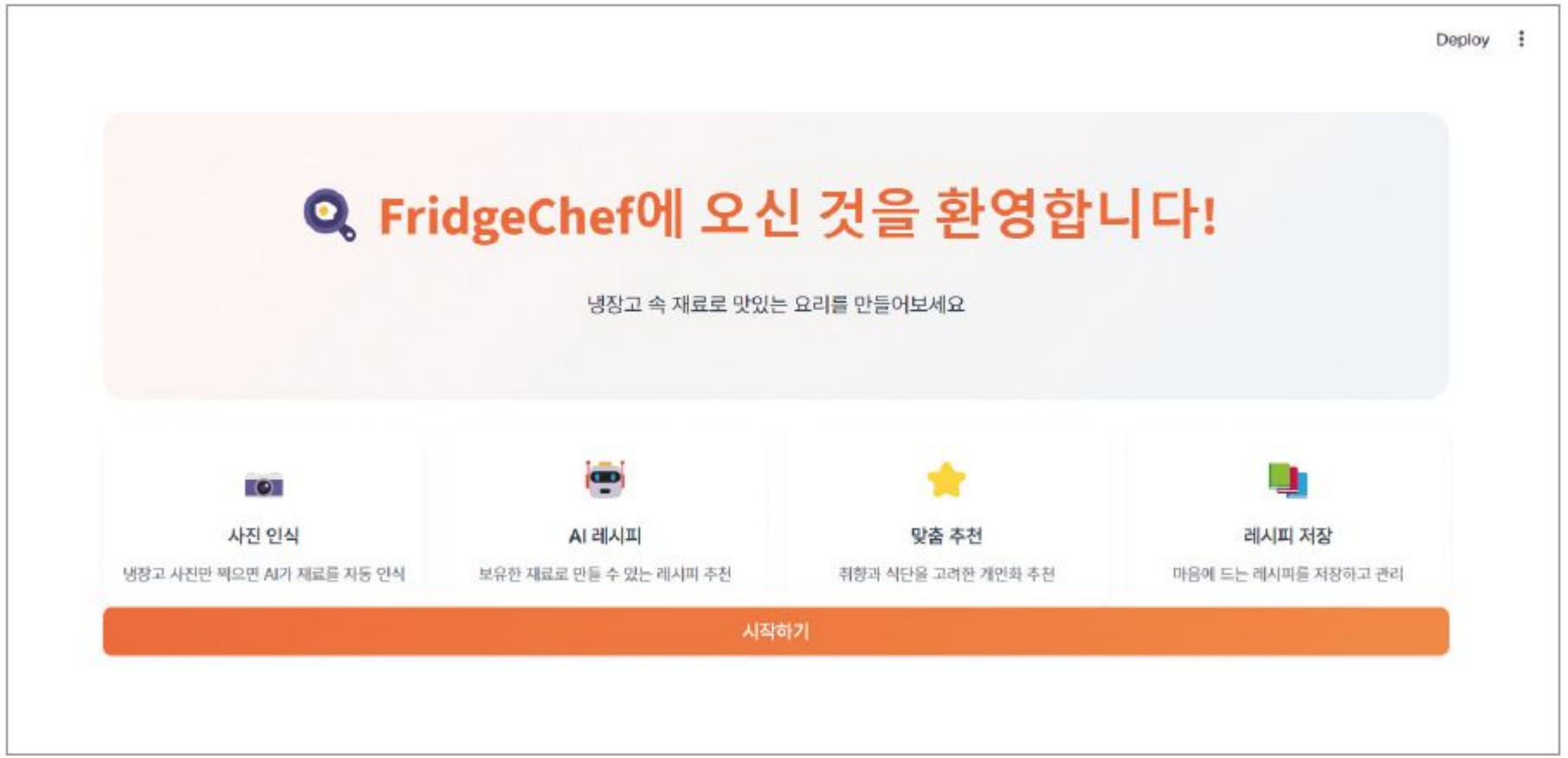
02 URL을 복사하여 웹 브라우저 주소창에 붙여 넣고 실행



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(43)

03 개선된 항목 확인

메인 화면이 대폭 수정되었고, 전체적인 애플리케이션의 색상 톤도 따뜻하게 업데이트



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(44)

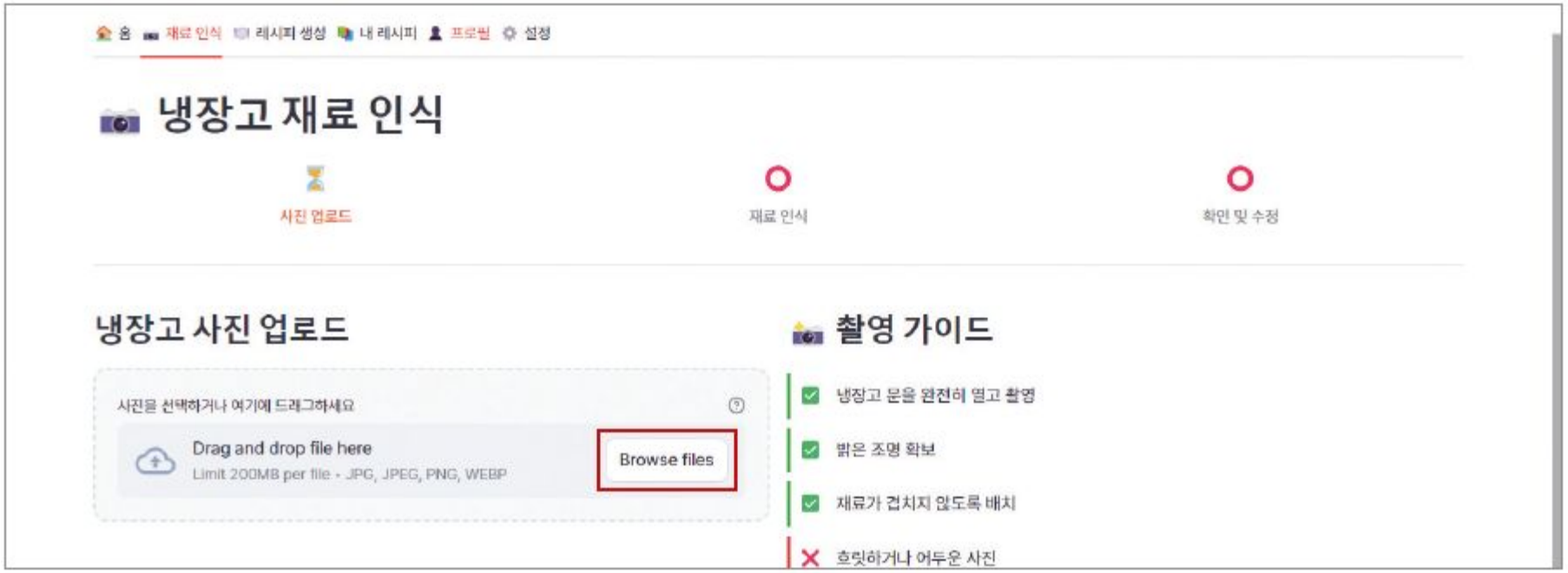
04 [체험하기(로그인 없이)] 버튼을 눌러 메인 화면으로 접근

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(45)

05 이전보다 나아진 디자인과 레이아웃을 확인

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(46)

06 [Browse files] 버튼을 눌러 '냉장고.jpg' 이미지를 업로드



[note]

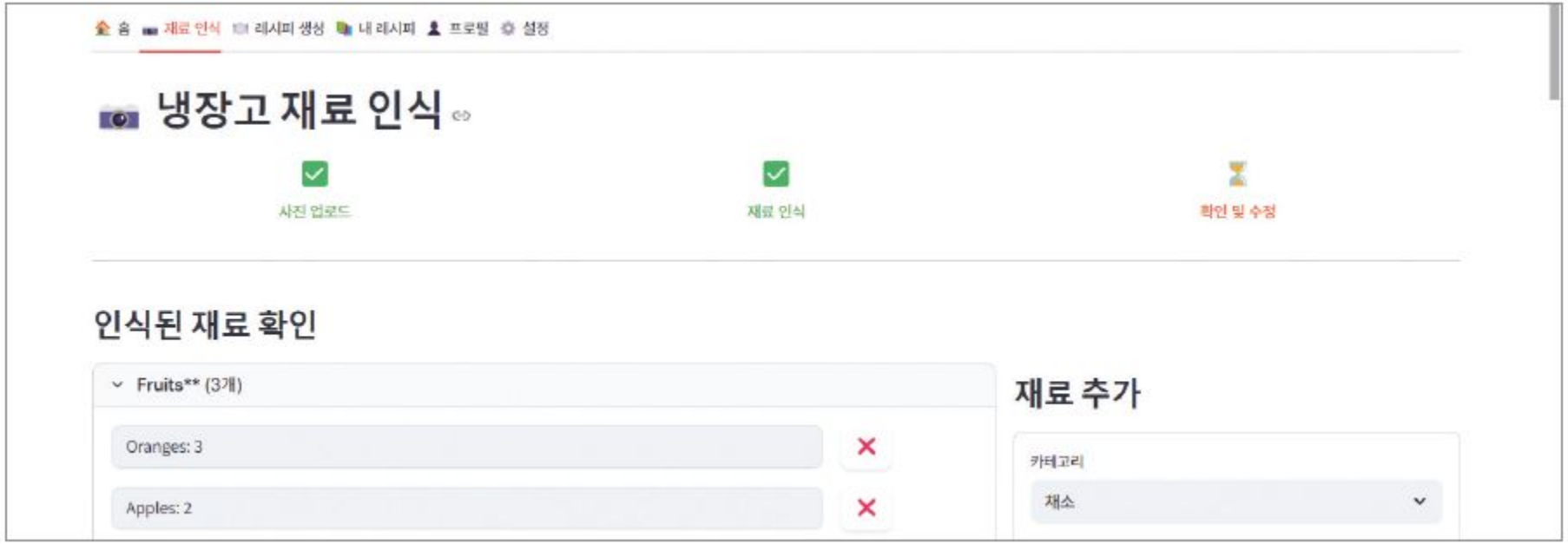
- **'냉장고.jpg' 이미지는 이 책의 깃허브 자료실 (https://github.com/taehojo/vibecoding)의 '입력예제' 폴더에서 다운로드**

SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(47)

07 [재료 인식 시작] 버튼을 눌러 이미지 분석을 시작

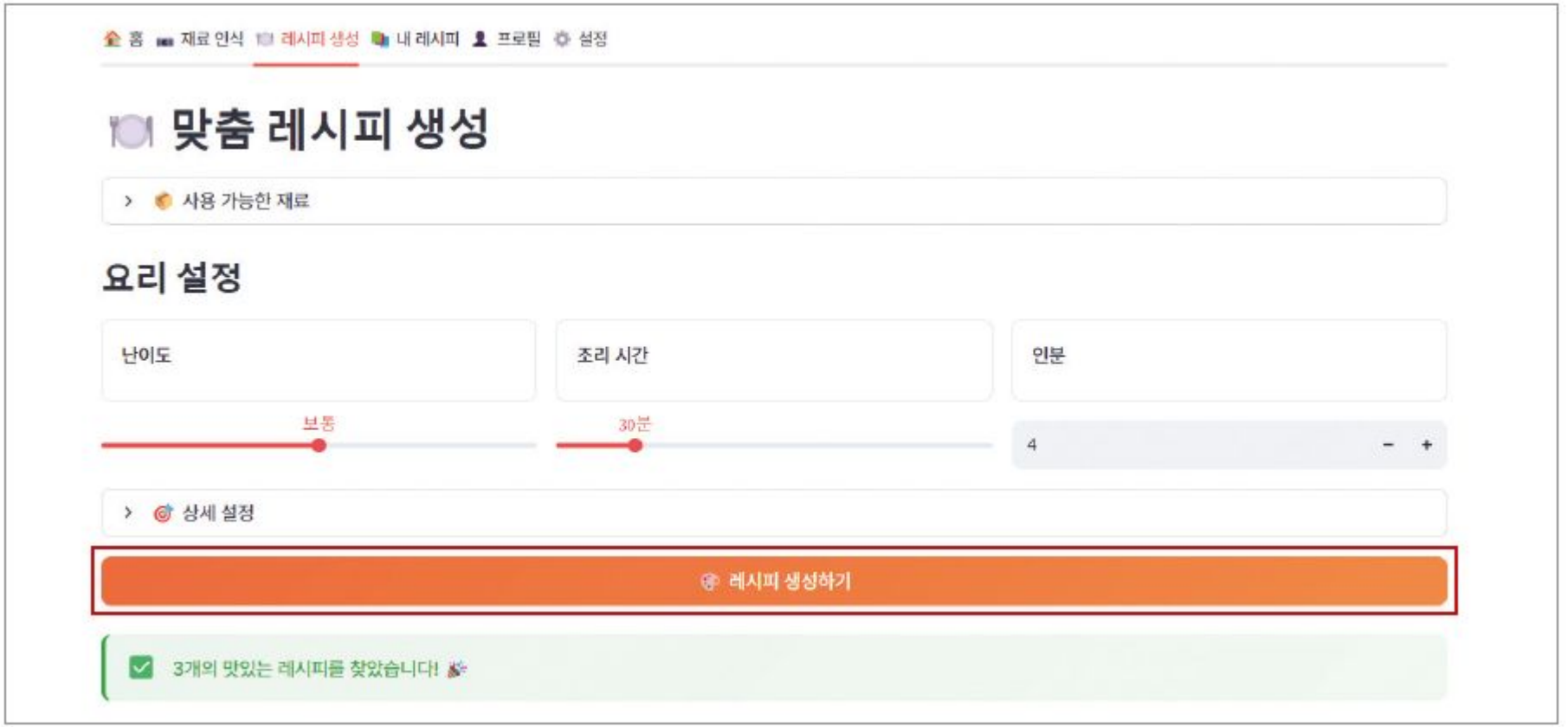
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(48)

08 재료가 API를 통해 제대로 인식되는 것을 확인



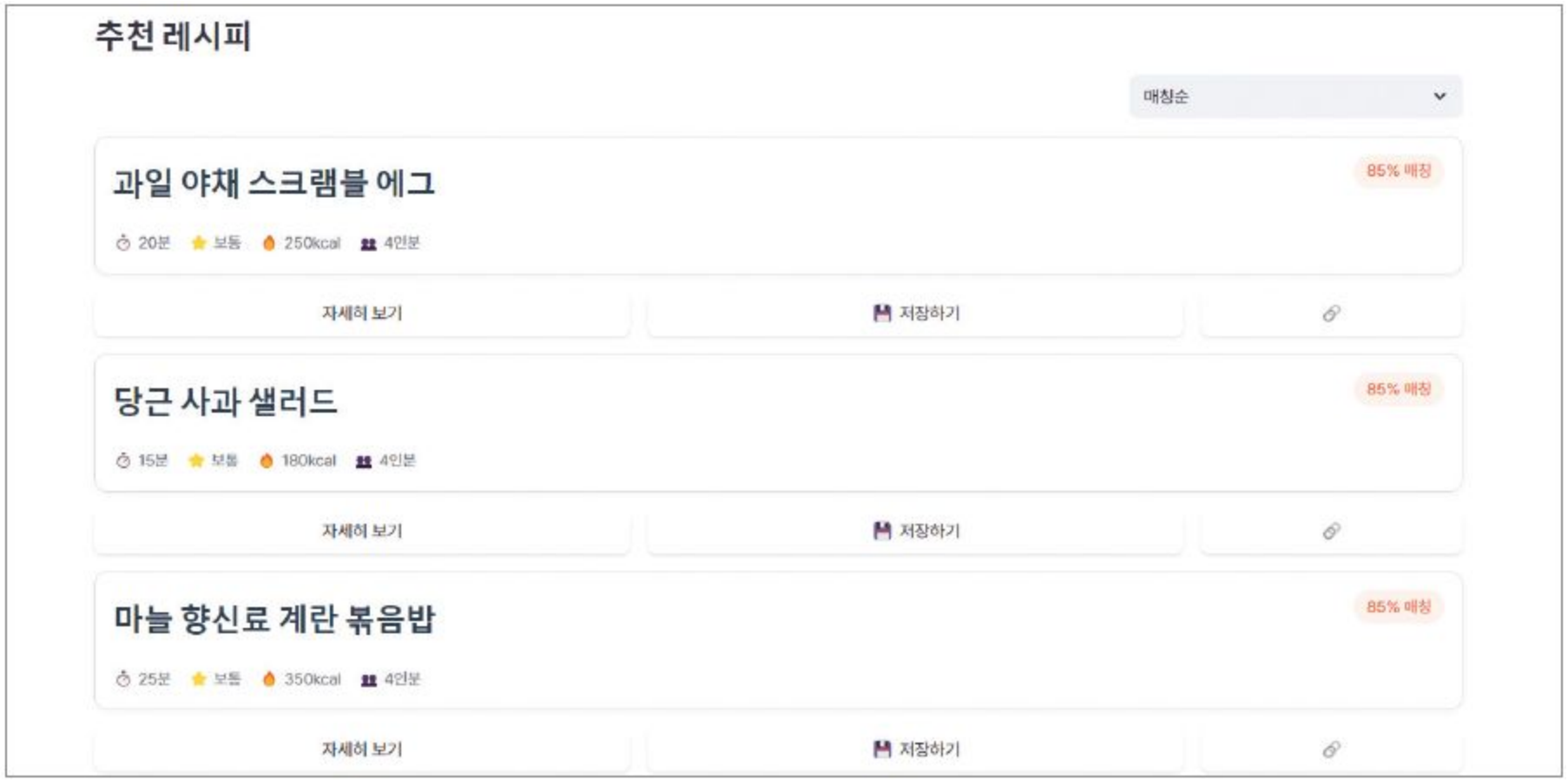
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(49)

09 요리의 난이도나 시간, 몇 인분을 준비할지를 설정
[레시피 생성하기] 버튼을 클릭



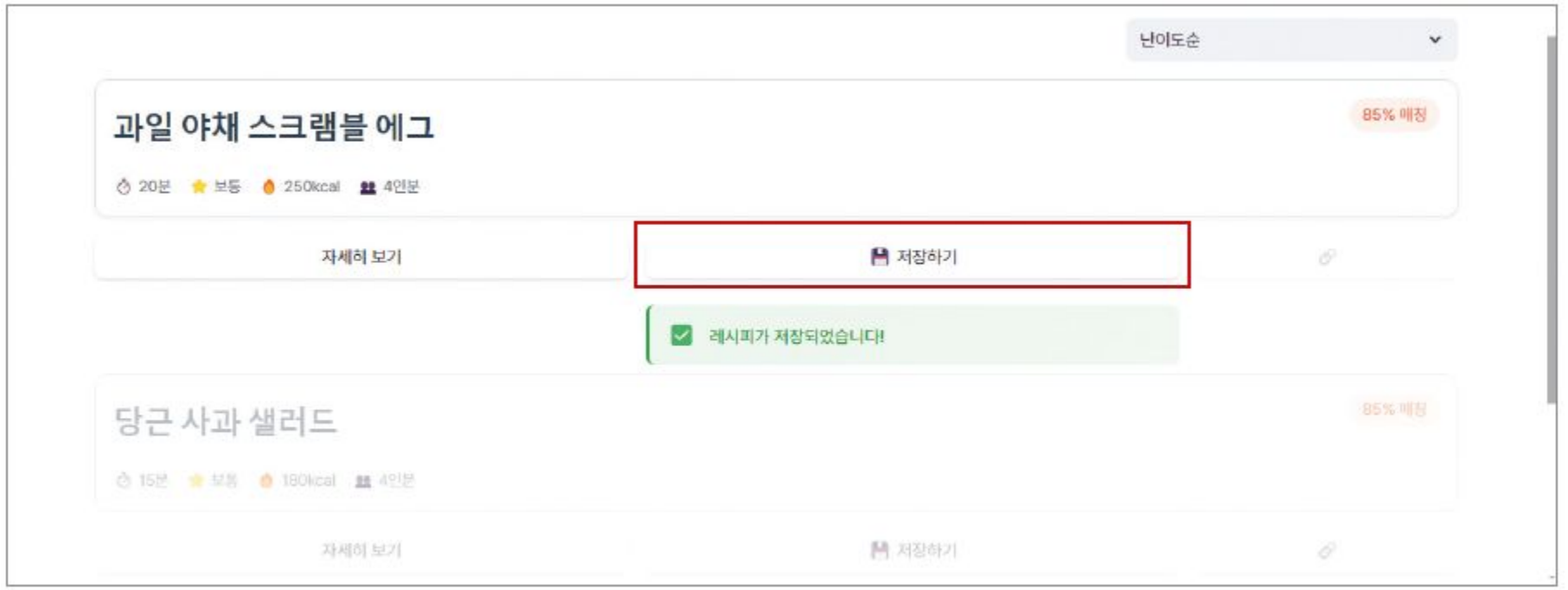
SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(50)

10 추천 레시피 목록이 나열되고 시간, 난이도, 열량이 표시



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(51)

11 [저장하기] 버튼을 클릭하면 원하는 레시피가 저장



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(52)

종더알아보기 **작업 결과 백업하기**

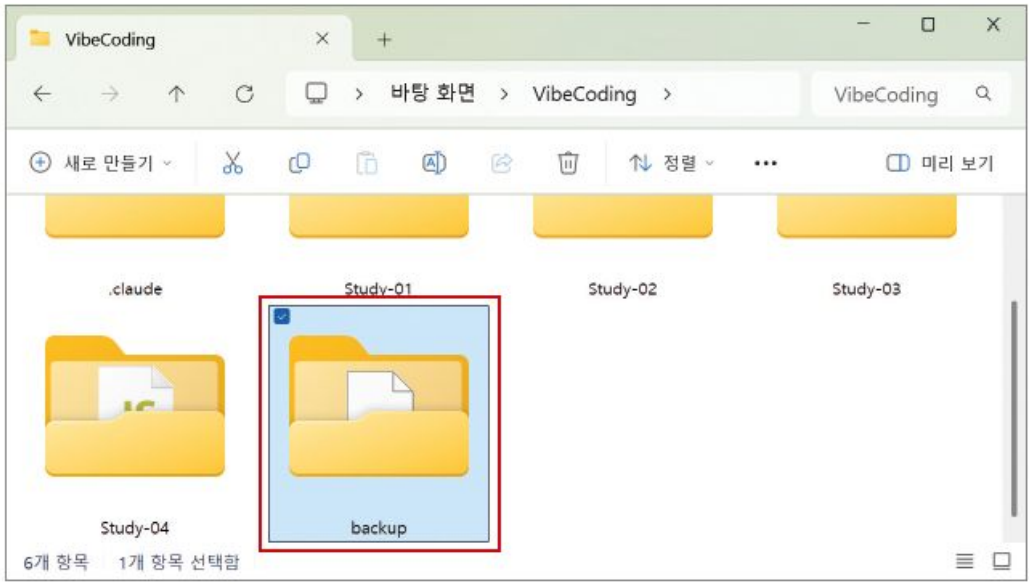
01 클로드 코드에게 현재 상태를 백업해 달라고 요청

프롬프트

```
현재 상태를 백업해 줘.
```

```
> 현재 상태를 백업해 줘.
```

02 클로드 코드가 프로젝트 폴더에 'backup' 디렉터리를 생성하고 모든 파일을 복사



SECTION 7-1 클로드 코드의 AI 에이전트 이해하기(53)

좀더알아보기 **작업 결과 백업하기**

03 예상치 못한 오류로 문제가 발생했을 때는 클로드 코드에게 백업으로 복원해 달라고 요청하여 이전 상태로 되돌릴 수 있음

프롬프트

```
백업으로 복원해 줘.
```

```
> 백업으로 복원해 줘.
```

[마무리]

- 6가지 키워드로 정리하는 핵심 포인트
  - **에이전트**는 사람이 직접 지시하지 않아도 목표와 규칙에 따라 스스로 판단하고 작업을 수행하는 AI 대리 프로그램
  - **AI 서브에이전트**는 클로드 코드에서 특정 작업을 전문적으로 수행하도록 만든 독립형 AI 도우미로, 역할, 모델, 권한을 개별 설정할 수 있음
  - 클로드 코드의 전용 명령어 **/agents**는 새로운 서브에이전트를 생성하거나 관리
  - **코드 리뷰**는 코드의 버그와 규칙 위반을 점검하고 개선점을 제안하는 품질 검토 과정
  - **최적화**는 프로그램의 속도, 자원 사용, 효율성을 향상시키는 성능 개선 작업
  - **UX 디자인**은 사용자가 쉽고 편리하게 이용할 수 있도록 화면 구성과 인터페이스를 설계하는 과정

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(1)

시작하기전에

- 스타트업을 운영하거나 개인 프로젝트를 진행할 때 가장 큰 어려움은 모든 역할을 혼자 해내야 한다는 점
  - 기획서도 작성하고, 서버도 구축하고, 디자인도 하고, 테스트까지 진행
- 클로드 코드의 에이전트를 활용하면 각 분야의 전문가 AI들로 팀을 구성
  - 실제 개발팀의 구조를 그대로 재현하여, 각 에이전트가 자신의 전문 영역에서 최선의 결과를 만들어내도록 설정
  - 특히 이들이 서로 협업하여 짧은 시간 안에 완성도 높은 애플리케이션을 만드는 과정을 직접 경험

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(2)

◦ 나만의 AI 개발팀 구축하기

소프트웨어 개발팀의 구성과 역할

- 제품 기획 관리자(Product Manager, PM)
  - PRD를 작성해 제품의 목표와 기능, 사용자 요구사항을 정의하고 전체 개발 일정을 관리
- 백엔드 개발자(backend developer)
  - 서버를 구축하고 데이터베이스를 설계하는 등 애플리케이션의 핵심 로직을 구현
- 프런트엔드 개발자(frontend developer)
  - 사용자가 보는 화면과 인터페이스를 설계하고 제작
- 품질 보증 엔지니어(Quality Assurance engineer, QA)
  - 버그를 찾아내고 성능을 테스트해 제품이 안정적으로 동작하는지 확인
- AI 통합 전문가
  - 인공지능 모델을 개발하거나 기존 모델을 최적화해 프로젝트에 맞는 AI 솔루션을 구현

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(3)

서브에이전트 생성하기

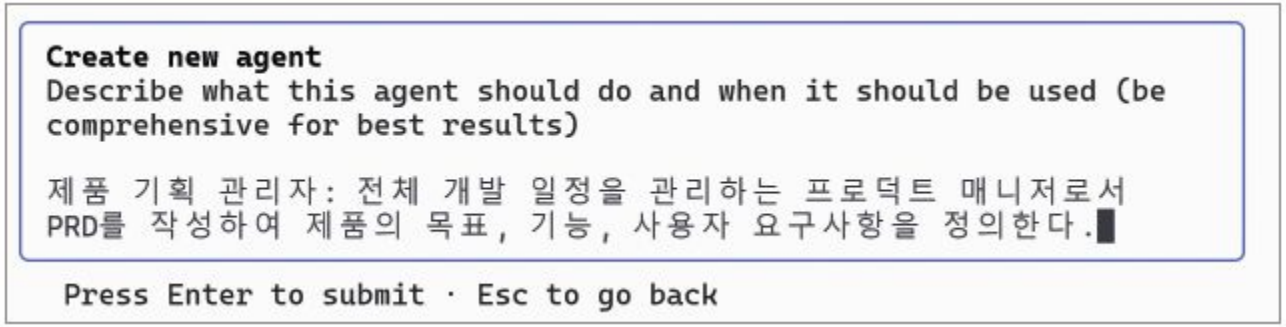
- ‘VibeCoding’ 폴더에 ‘Study-05’ 폴더를 생성하고 해당 위치에서 터미널 실행한 뒤, 클로드 코드를 실행
- ‘/init’ 명령어를 입력해 초기화하고 나서, 다음과 같은 절차를 따라 서브에이전트의 초기 설정
  1) ‘/agents’ 명령어로 서브에이전트 인터페이스 열고 ‘Create new agent’ 선택하기
  2) 적용 범위는 ‘1. Project (.claude/agents/)’, 생성 방법은 ‘1. Generate with Claude’으로 설정하기
- 먼저 제품 기획 관리자 에이전트를 생성한 뒤에, 같은 방법으로 백엔드 개발자, 프런트엔드 개발자, 품질 보증 엔지니어, AI 통합 전문가 에이전트를 순서대로 만들기

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(4)

01 다음과 같이 제품 기획 관리자 에이전트의 기능과 역할을 기술한 뒤, [Enter] 키를 눌러 진행

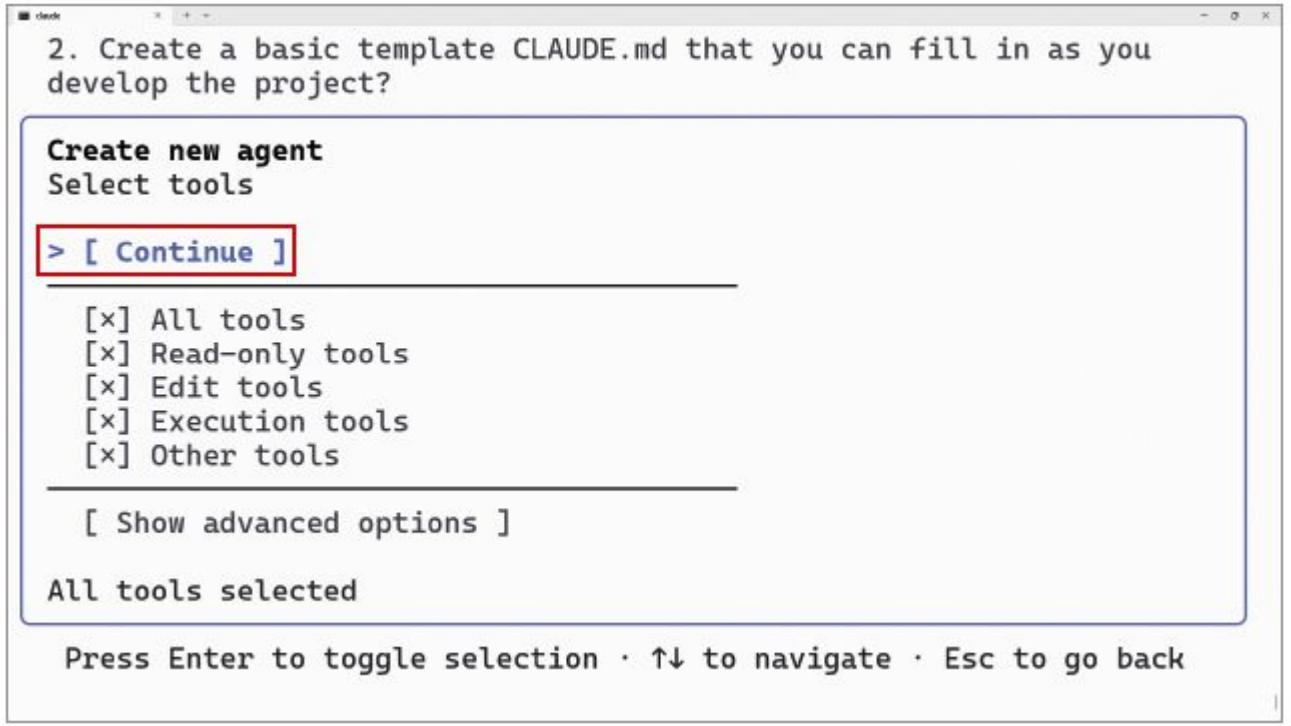
프롬프트

```
제품 기획 관리자: 전체 개발 일정을 관리하는 프로덕트 매니저로서 PRD를 작성하여 제품의 목표, 기능, 사용자
요구사항을 정의한다.
```



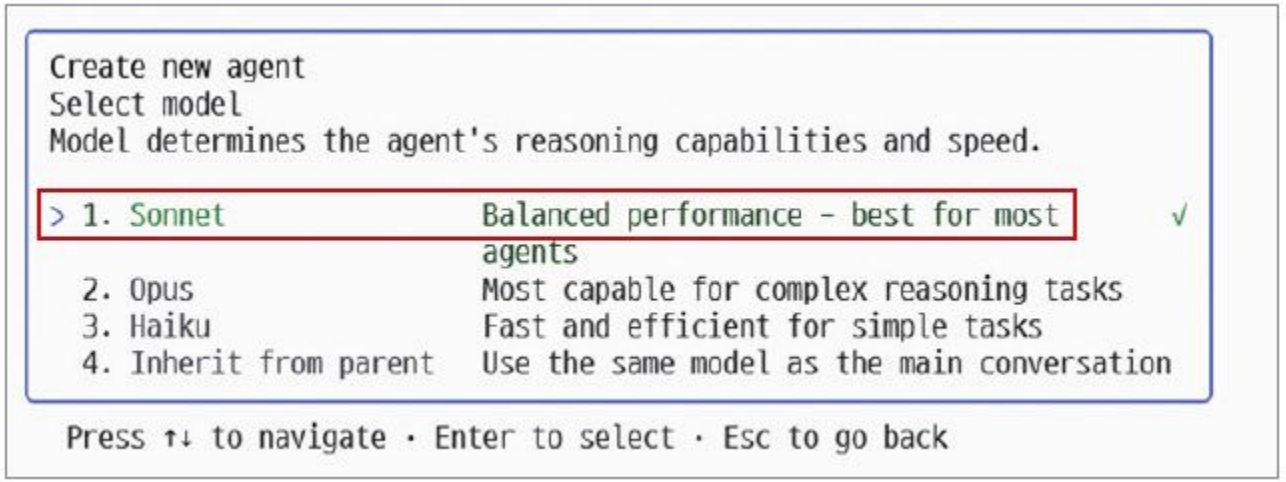
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(5)

02 모든 도구 권한을 부여 - [Continue]를 선택하고 [Enter] 키를 눌러 진행



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(6)

03 사용할 AI 모델을 선택 - 이번에는 기본 모델인 '1. Sonnet'을 선택

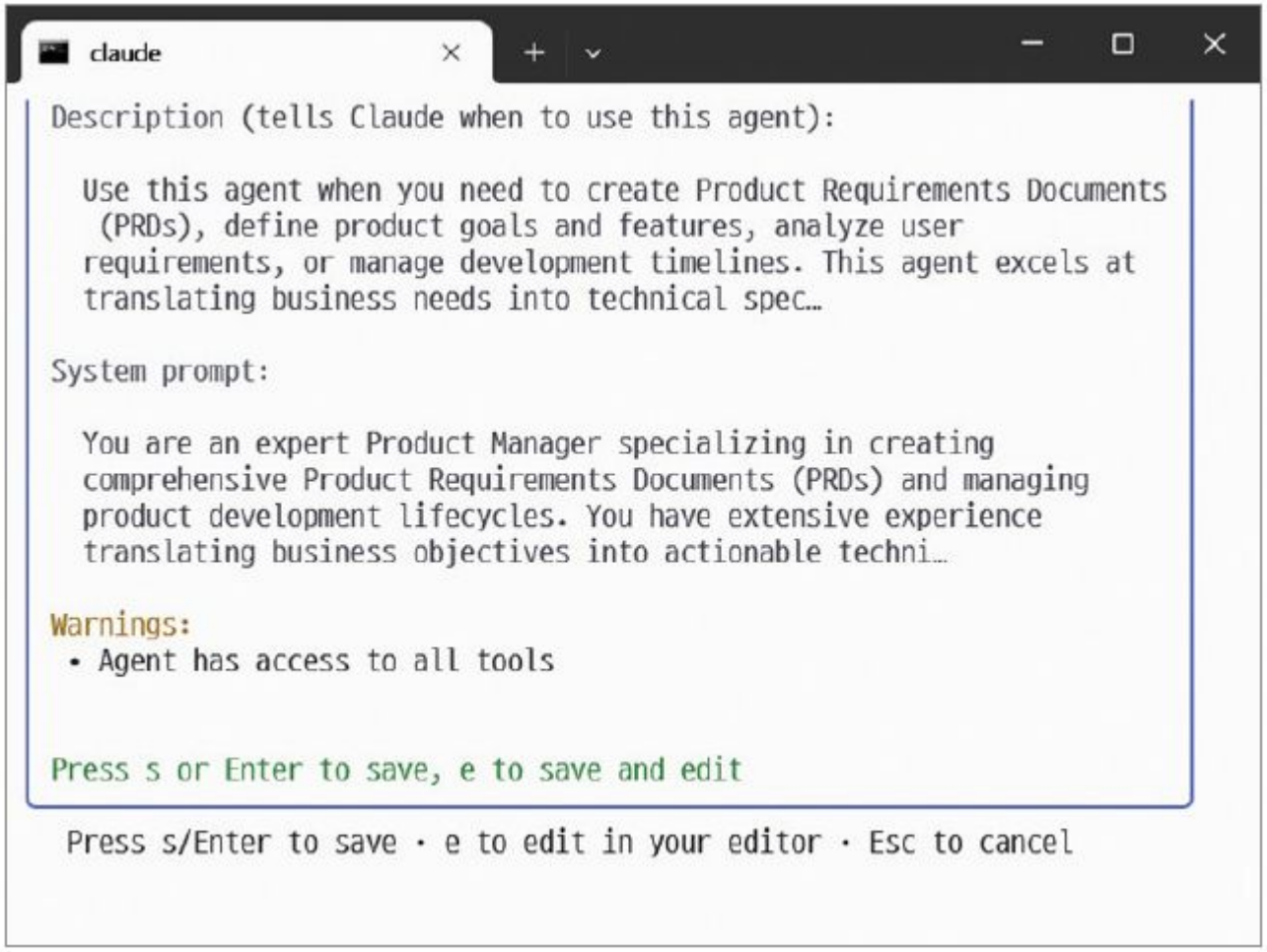


SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(7)

04 배경색을 'Red'로 설정

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(8)

05 제품 기획 관리자 에이전트의 설정을 확인 - 이상이 없으면 [Enter] 키를 눌러 진행



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(9)

06 제품 기획 관리자 에이전트 'product-manager-prd'가 생성

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(10)

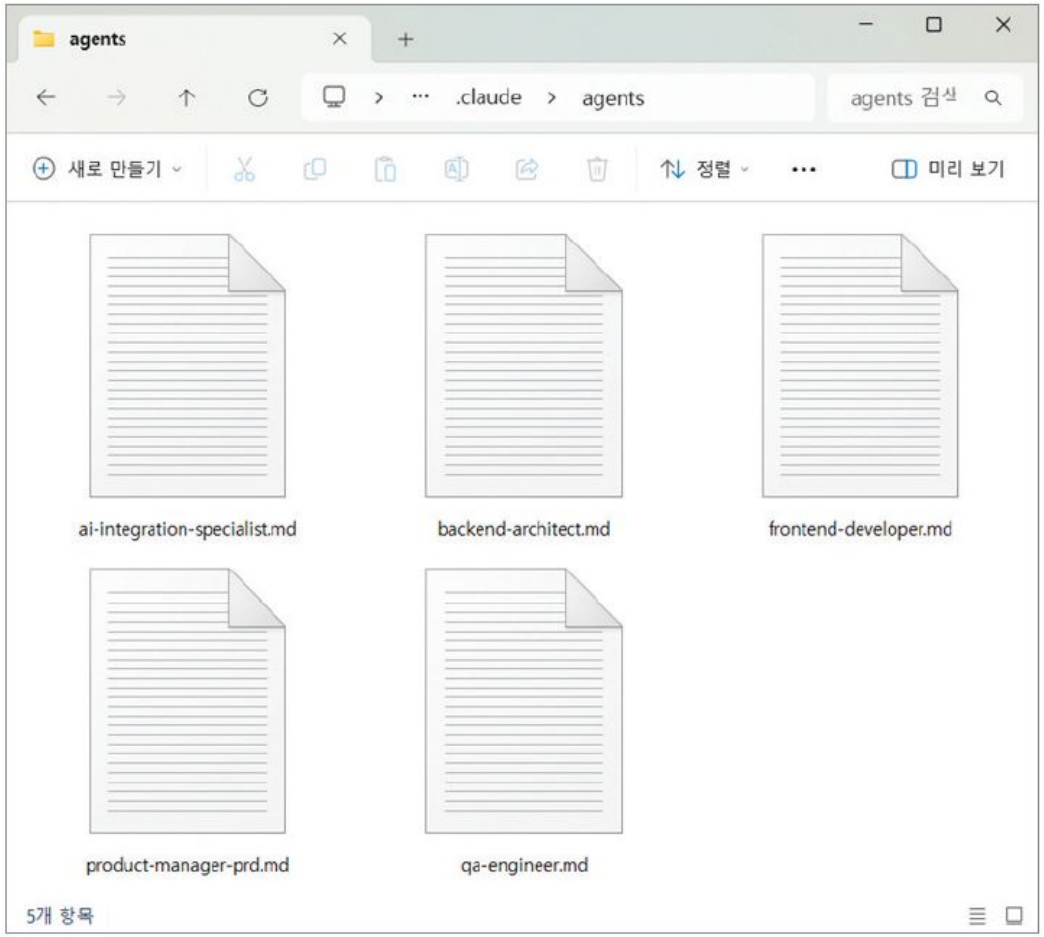
- 같은 과정을 거쳐 나머지 에이전트들도 만들기
  - 서브에이전트들의 초기 설정은 모두 동일
    - 적용 범위: '1. Project (.claude/agents/)'
    - 생성 방법: '1. Generate with Claude'
    - 사용할 AI 모델: '1. Sonnet'
    - 도구 권한: 'All tools'

서브에이전트	기능 및 역할 기술	배경색
백엔드 개발자	서버 아키텍처 설계, API 개발, 데이터 처리, 외부 서비스 통합, 보안 및 성능 최적화를 담당하는 서버 사이드 개발 전문가. 안정적이고 확장 가능한 백엔드 시스템 구축.	Blue
프런트엔드 개발자	사용자 인터페이스 설계 및 구현, 반응형 디자인, 웹 접근성, 성능 최적화를 담당하는 클라이언트 사이드 개발 전문가.	Green
품질 보증 엔지니어	전체 시스템의 기능 테스트, 에러 처리 검증, 성능 최적화, 코드 리뷰를 수행하는 품질 관리 전문가. 버그 발견, 사용성 개선사항 제안.	Yellow
AI 통합 전문가	LLM 및 AI 서비스 통합, 프롬프트 최적화, 모델 파인튜닝, AI 파이프라인 구축을 담당하는 인공지능 전문가. 여기서는 OpenRouter API를 통해 DeepSeek 모델과 연동하여 텍스트 생성, 요약을 구현하는 LLM 활용 전문가.	Purple

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(11)

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(12)

- 다섯 개의 서브에이전트를 모두 생성하면 'Study-05 .claude agents' 폴더에 각 에이전트 .md 파일이 저장되어 있는 것을 확인할 수 있음



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(13)

◦ 에이전트 팀과 완성하는 애플리케이션, 'AI 공감 다이어리

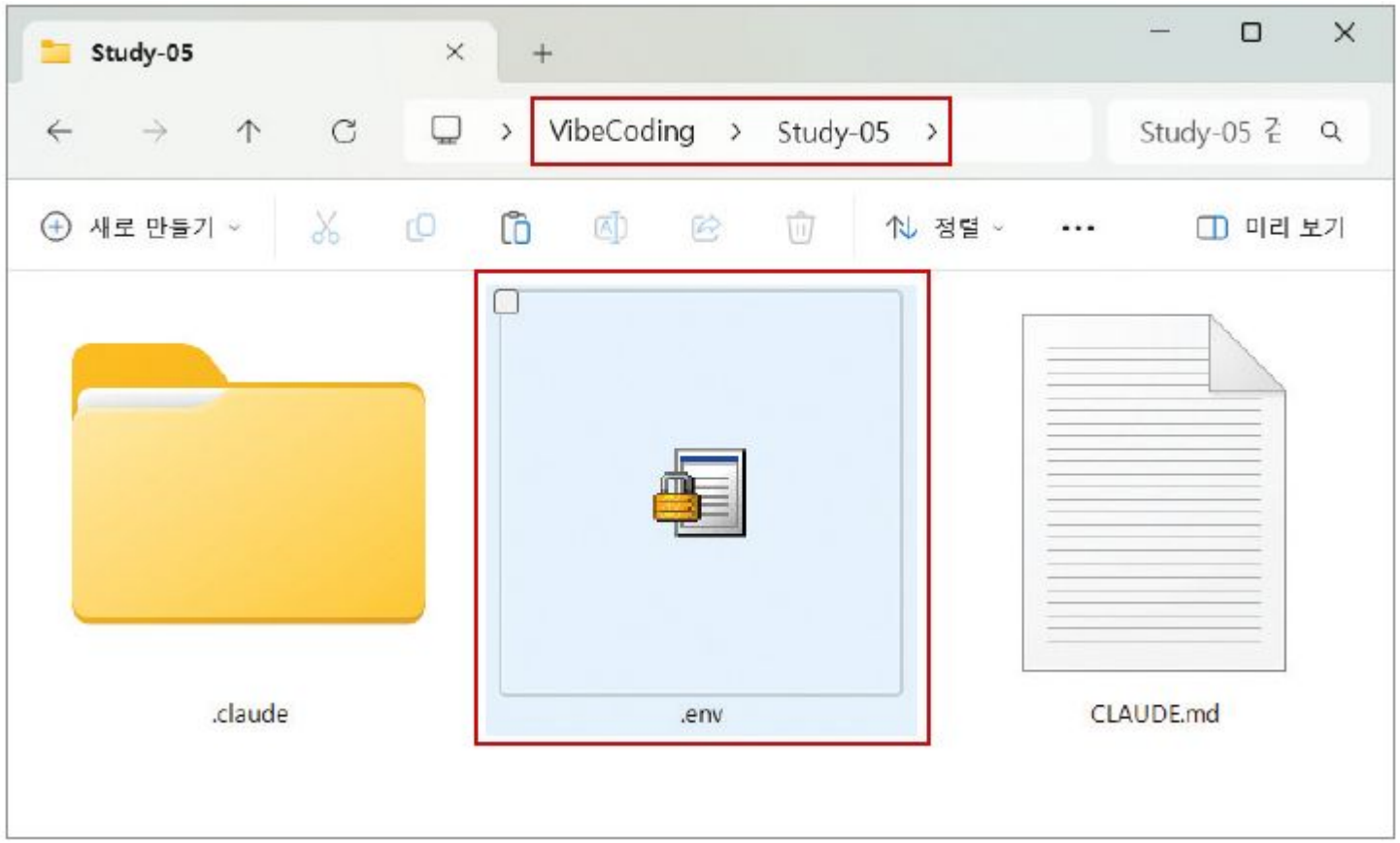
5W1H 프레임워크로 제품 요구사항 정리하기

- 여러 에이전트를 운용할 때 각각의 역할(Who)과 작업 방식(How)을 구체적으로 정의하는 것이 중요

- **What: AI 공감 다이어리 - 사용자가 오늘 있었던 일을 한 줄로 쓰면 AI가 감정을 분석하고 공감과 위로를 해주는 일기 앱**
- **Why: 사용자의 하루를 기록하고 정서적 지원을 제공하여 마음의 안정을 돕기 위함**
- **When: 지금 바로 개발 시작하여 즉시 사용 가능한 완성품 제작**
- **Where: 웹 브라우저에서 실행, 사용자의 로컬 환경에서 작동**
- **How:**
  - **OpenRouter API와 DeepSeek V3.1 무료 모델을 활용하여 감정 분석 및 공감 메시지 생성**
  - **따뜻하고 편안한 느낌의 UI 디자인 적용**
  - **API 키는 현재 프로젝트 폴더의 '.env' 파일에서 읽어와 사용**
  - **최종 결과물은 'index.html' 단일 파일로 제작**
  - **철저한 테스트를 통해 안정적인 버전 완성**
- **Who:**
  - **백엔드 개발자 : API 연동, 감정 분석과 공감 메시지를 생성하는 기능 구현**
  - **프런트엔드 개발자 : 따뜻하고 편안한 느낌의 일기장 UI 제작**
  - **품질 보증 엔지니어 : 여러 상황을 테스트하고 문제 발견 시 완전히 해결될 때까지 수정**

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(14)

01 데스크톱 파일 탐색기에서 오픈라우터 API 키를 불러오기 위해 'Study-04' 폴더에서 '.env' 파일을 복사한 뒤, 'Study-05' 폴더에 붙여 넣기

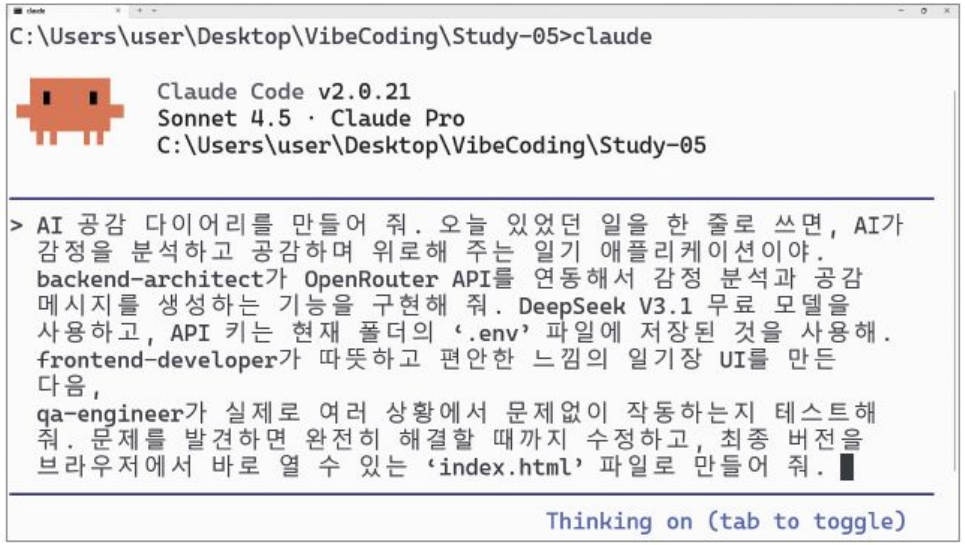


SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(15)

02 클로드 코드에 'AI 공감 다이어리' 애플리케이션의 기능을 설명하고, 필요한 서브에이전트와 각 에이전트의 역할과 구현 방식을 입력

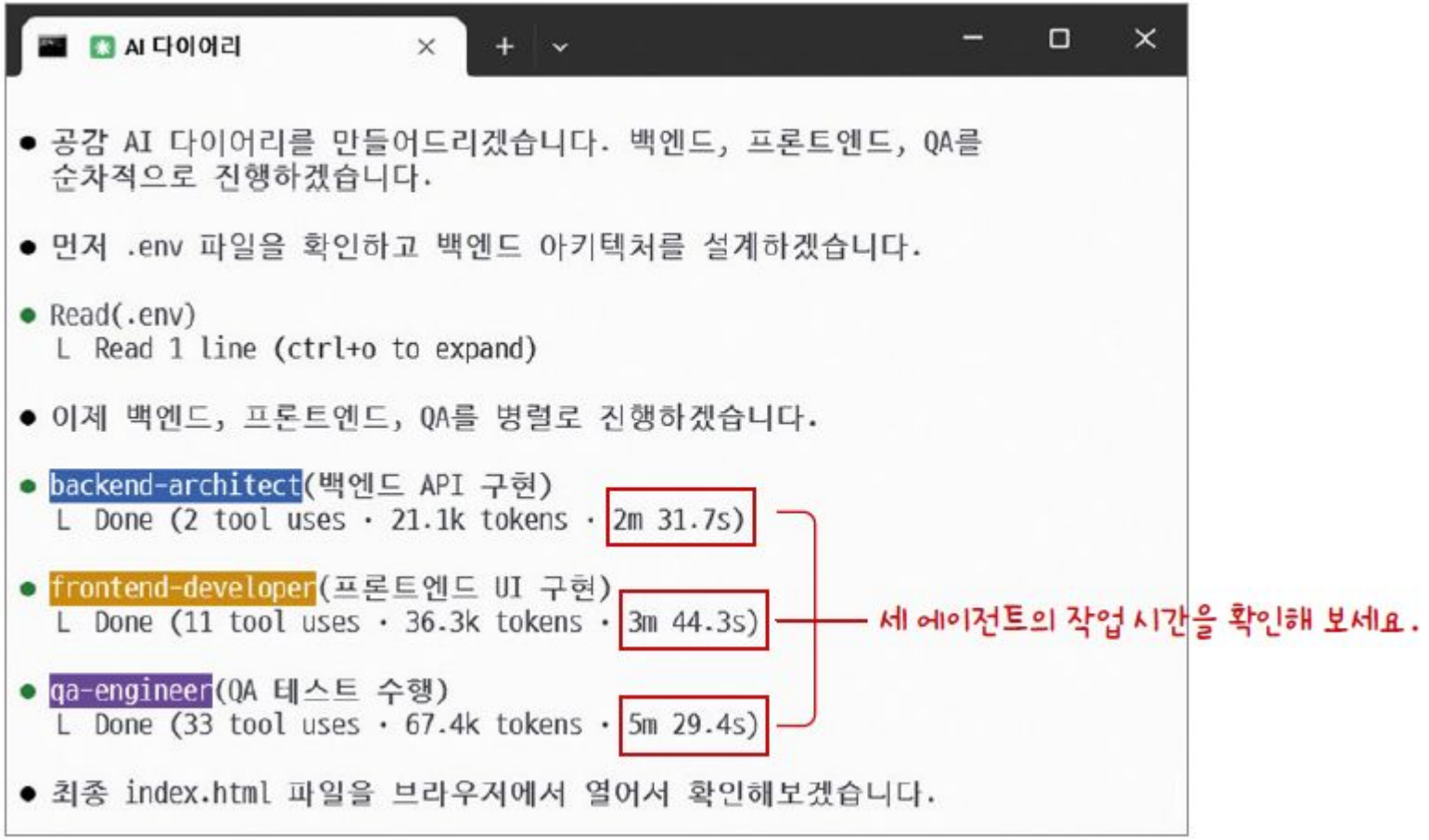
프롬프트

```
AI 공감 다이어리를 만들어 줘. 오늘 있었던 일을 한 줄로 쓰면, AI가 감정을 분석하고 공감하며 위로해 주는
일기 애플리케이션이야.
backend-architect가 OpenRouter API를 연동해서 감정 분석과 공감 메시지를 생성하는 기능을 구현해 줘.
DeepSeek V3.1 무료 모델을 사용하고, API 키는 현재 폴더의 '.env' 파일에 저장된 것을 사용해.
frontend-developer가 따뜻하고 편안한 느낌의 일기장 UI를 만든 다음,
qa-engineer가 실제로 여러 상황에서 문제없이 작동하는지 테스트해 줘. 문제를 발견하면 완전히 해결할 때까
지 수정하고, 최종 버전을 브라우저에서 바로 열 수 있는 'index.html' 파일로 만들어 줘.
```



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(16)

03 터미널에서 세 에이전트가 지시한 대로 작업하는 것을 확인



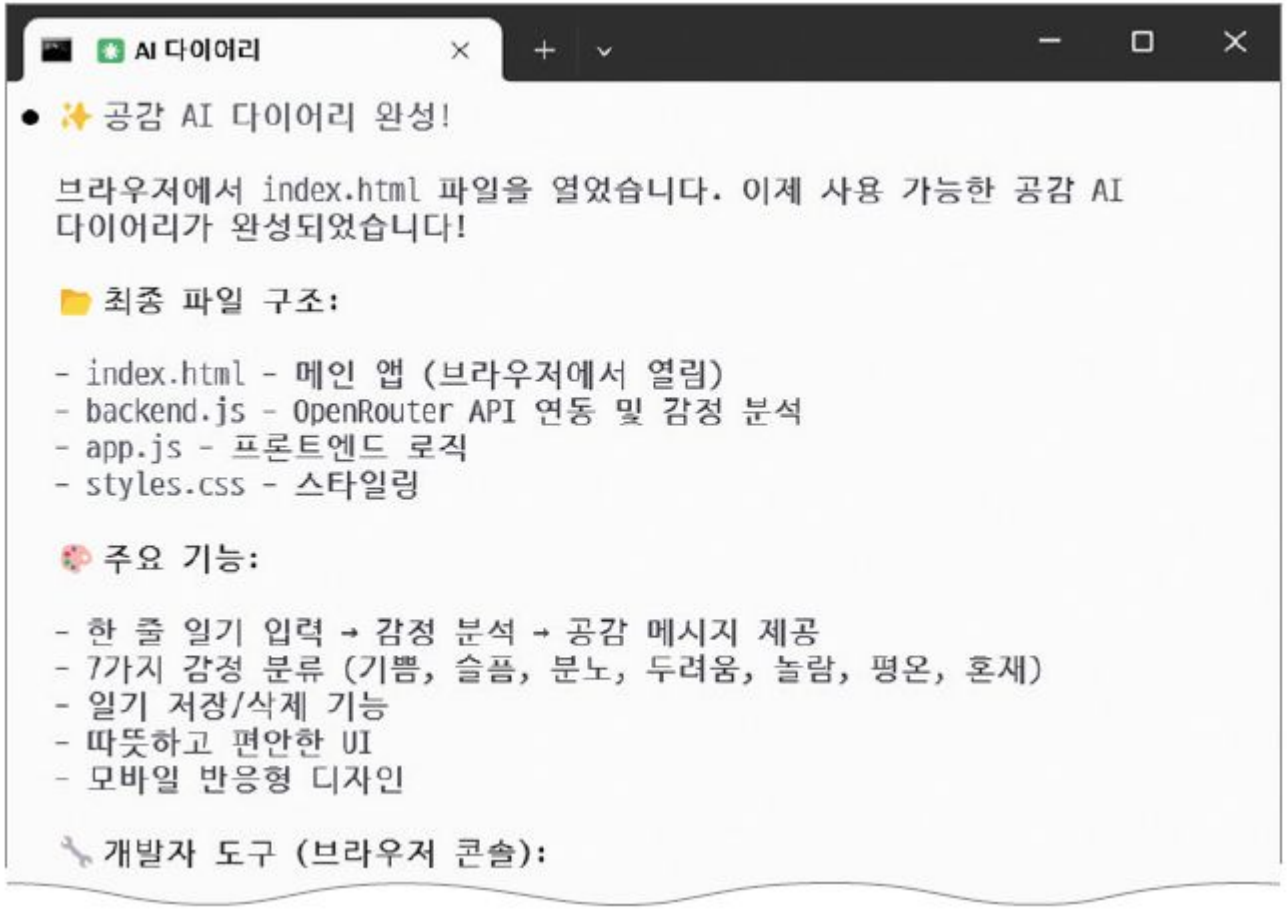
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(17)

✚ 여기서 잠깐 **에이전트 작업 시간이 서로 다른 이유**

- 각 에이전트의 작업 시간이 조금씩 다른 것은 자연스러운 현상
- 품질 보증 엔지니어 에이전트의 작업 시간이 가장 오래 걸린 이유는 앞서 다른 두 에이전트가 생성한 결과물을 모두 검토하고, 여러 시나리오를 테스트하며, 발견한 오류를 수정하는 과정까지 포함하기 때문임
- 실제 개발팀에서도 품질 보증(QA)은 가장 시간이 많이 걸리는 과정
- 만약 실행 시간이 너무 길게 느껴진다면 [Esc] 키를 눌러 중단하고, 더 간단한 테스트만 요청해 작업 범위를 줄일 수도 있음

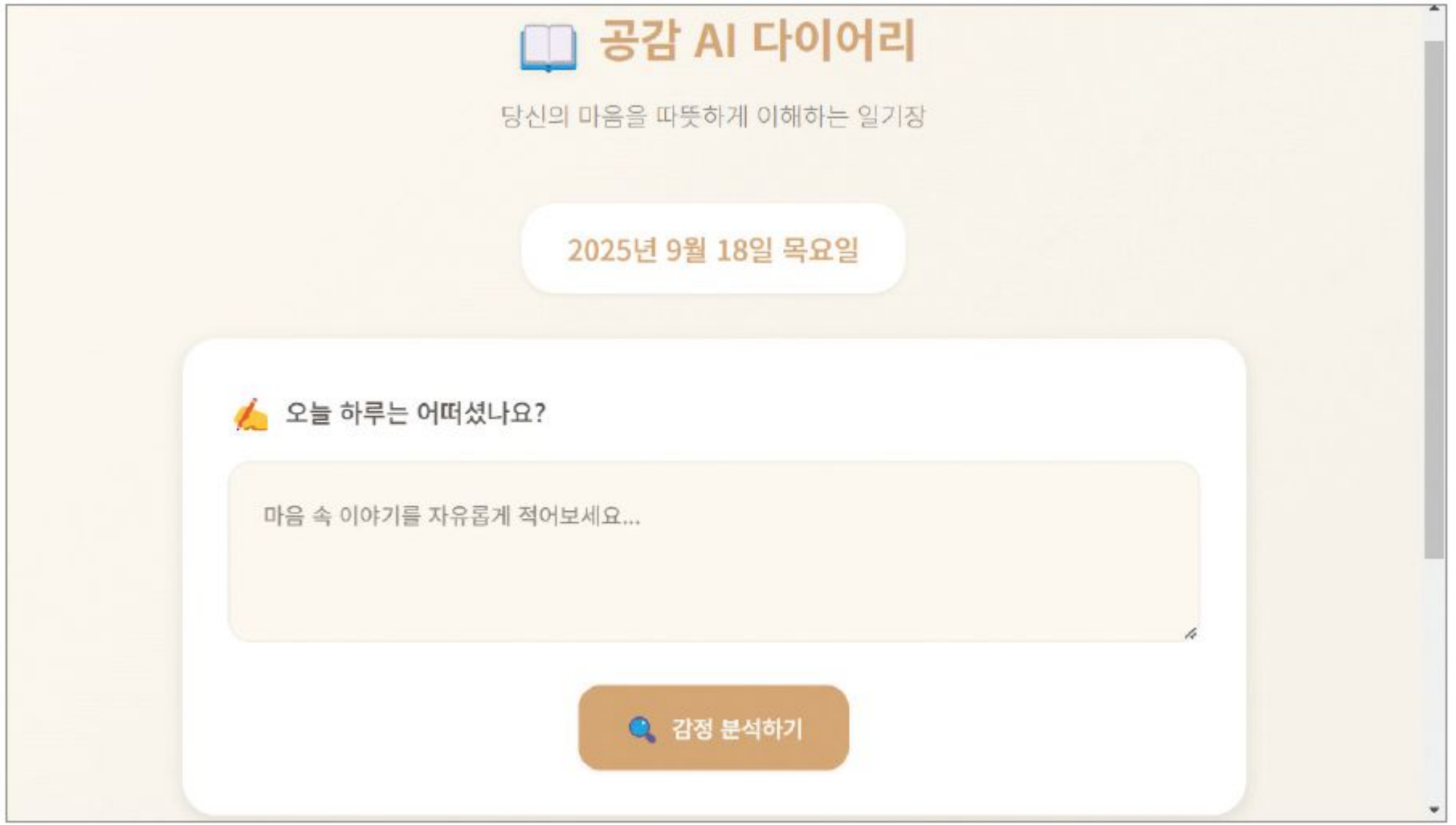
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(18)

04 최종 보고서를 보고 어떤 기능이 준비되었는지 확인



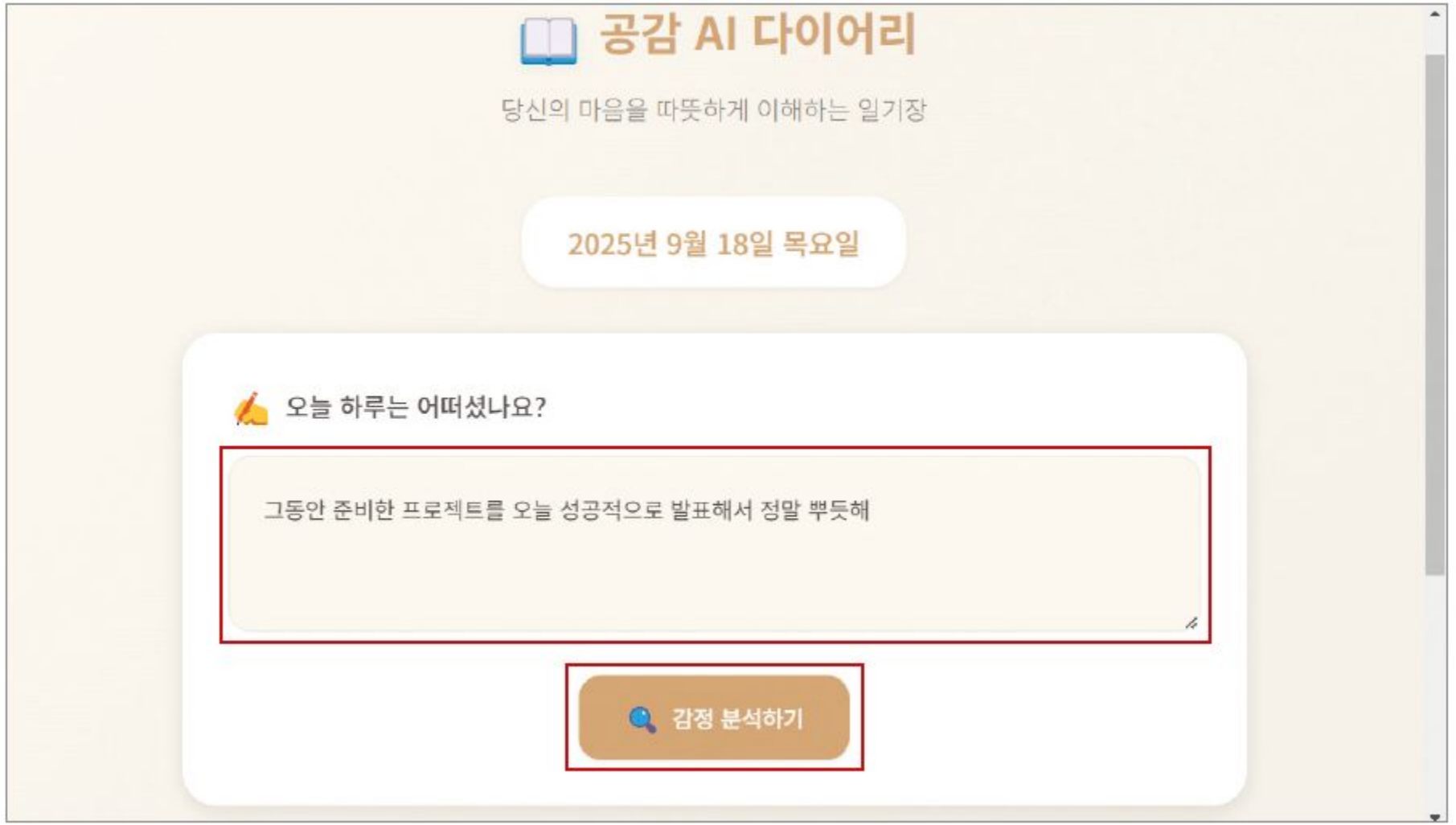
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(19)

05 ‘Study-05’ 폴더에서 ‘index.html’ 파일을 클릭해 실행 - AI 공감 다이어리가 완성된 모습



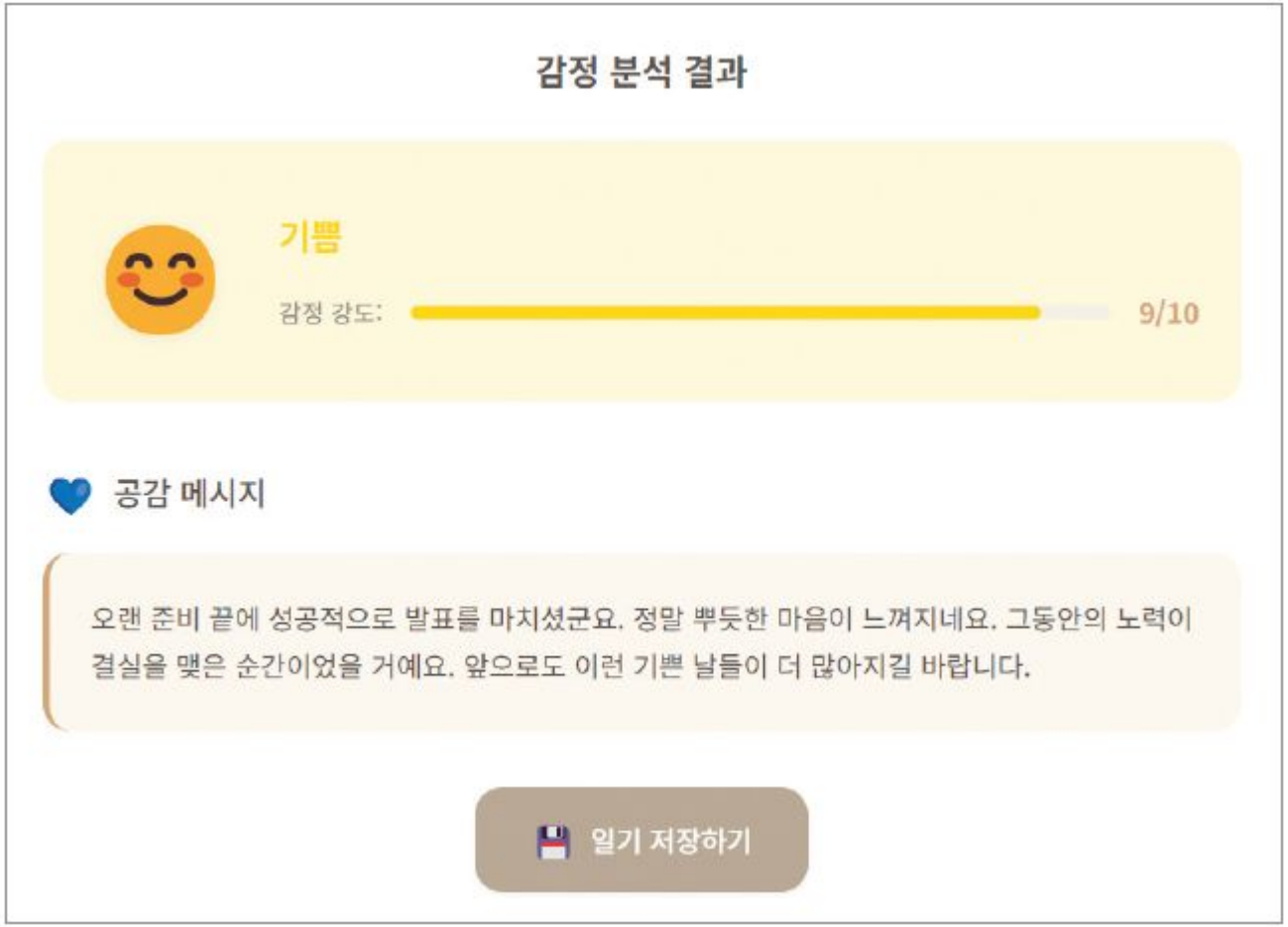
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(20)

06 애플리케이션이 텍스트 속 감정을 잘 분석하는지 테스트
다음과 같이 입력하고, [감정 분석하기] 버튼을 클릭



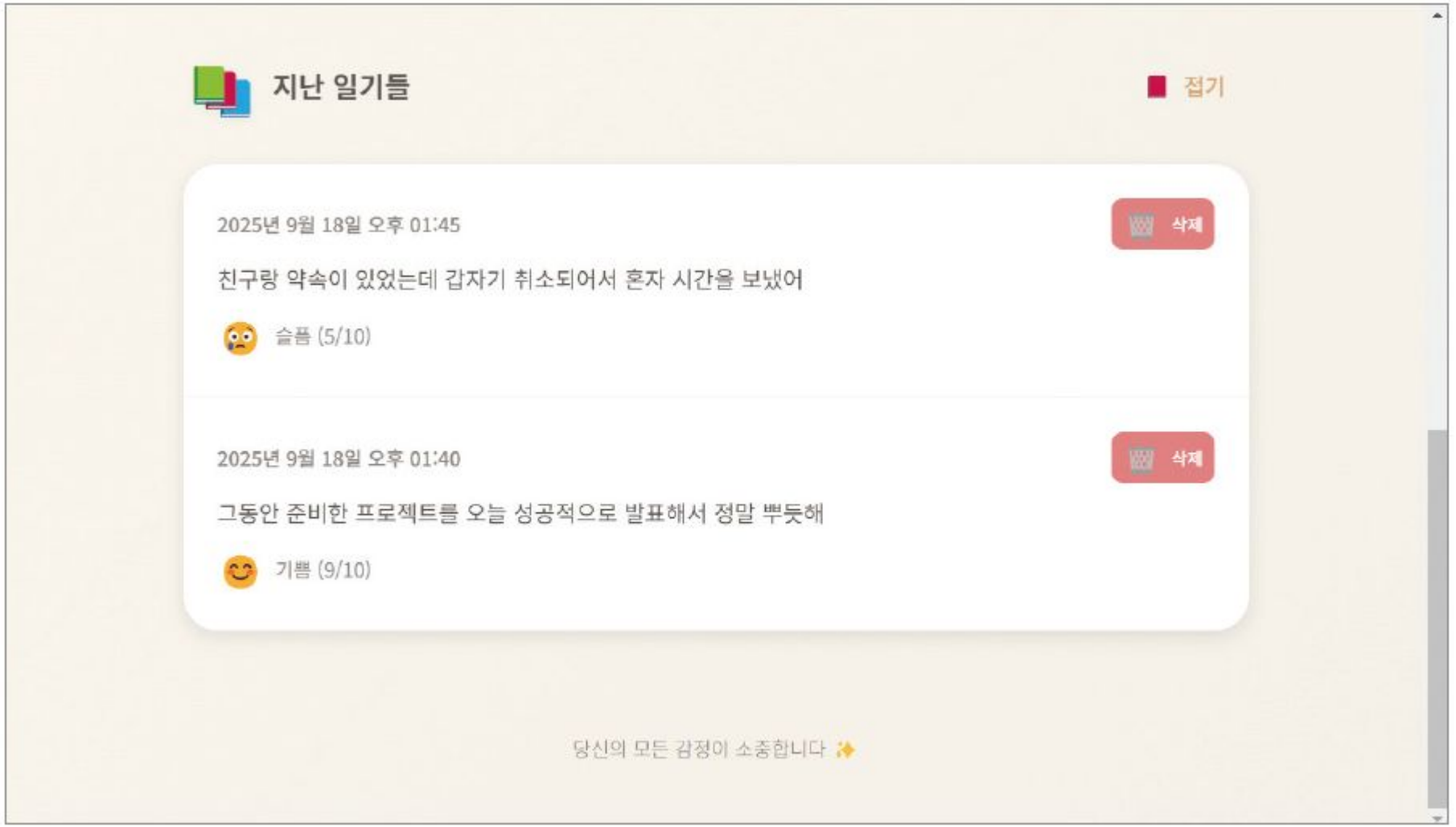
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(21)

07 뿌듯한 감정을 잘 탐지하고, 공감의 글을 덧붙여 줌



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(22)

08 작성한 감정 일기와 감정 분석 결과는 시간 순서대로 저장됨



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(23)

◦ 에이전트 팀과 완성하는 애플리케이션, 'PDF 요약 AI'

- 다섯 개의 서브에이전트를 총동원해 PDF 파일을 입력받아 텍스트를 추출하고, 이를 요약, 정리, 분석하는 애플리케이션을 만들기

  • 앞서 만들어 둔 AI 에이전트를 활용

PDF 문서 요약 AI 애플리케이션 개발을 위해 필요한 각 에이전트들의 역할과 이름,

역할	에이전트 명	작업 할당
제품 기획 관리자	product-manager-prd	PDF 문서 요약 앱의 상세 PRD와 기능 명세를 작성
백엔드 개발자	backend-architect	PDF 파일 업로드, 텍스트 추출 기능 구현
AI 통합 전문가	ai-integration-specialist	OpenRouter API를 연동해서 추출된 텍스트를 요약하는 기능 구현. 무료 모델을 사용하고, API 키는 .env 파일에 저장된 것을 사용
프런트엔드 개발자	frontend-developer	드래그&드롭 파일 업로드 UI와 요약 결과를 깔끔하게 표시하는 인터페이스를 구현
품질 보증 엔지니어	qa-engineer	다양한 PDF 형식으로 전체 시스템을 철저히 테스트하고 버그를 수정

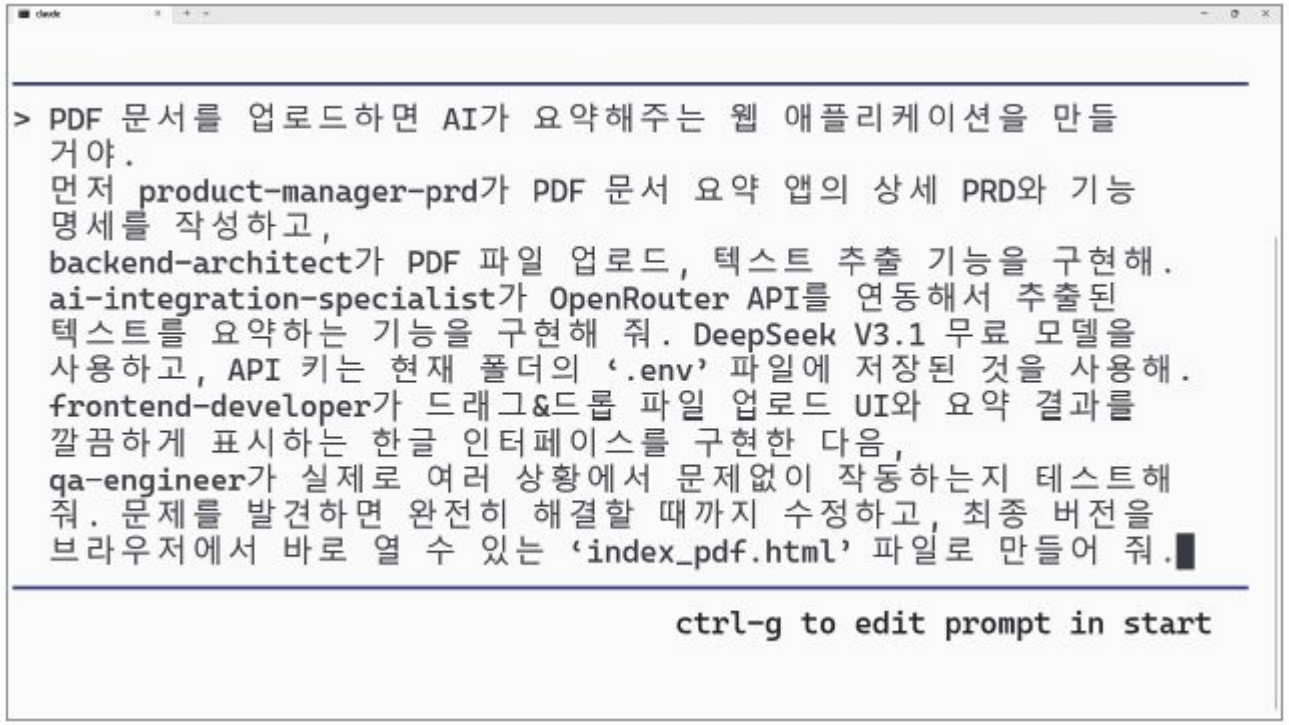
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(24)

01 클로드 코드를 실행해 'PDF 요약 AI' 애플리케이션의 기능을 설명하고, 각 서브에이전트의 역할과 구현 방식을 입력

프롬프트

```
PDF 문서를 업로드하면 AI가 요약해주는 웹 애플리케이션을 만들 거야.
먼저 product-manager-prd가 PDF 문서 요약 앱의 상세 PRD와 기능 명세를 작성하고,
backend-architect가 PDF 파일 업로드, 텍스트 추출 기능을 구현해.
ai-integration-specialist가 OpenRouter API를 연동해서 추출된 텍스트를 요약하는 기능을 구현해 줘.
DeepSeek V3.1 무료 모델을 사용하고, API 키는 현재 폴더의 '.env' 파일에 저장된 것을 사용해.
frontend-developer가 드래그&드롭 파일 업로드 UI와 요약 결과를 깔끔하게 표시하는 한글 인터페이스를 구
현한 다음,
qa-engineer가 실제로 여러 상황에서 문제없이 작동하는지 테스트해 줘. 문제를 발견하면 완전히 해결할 때까
지 수정하고, 최종 버전을 브라우저에서 바로 열 수 있는 'index_pdf.html' 파일로 만들어 줘.
```

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(25)

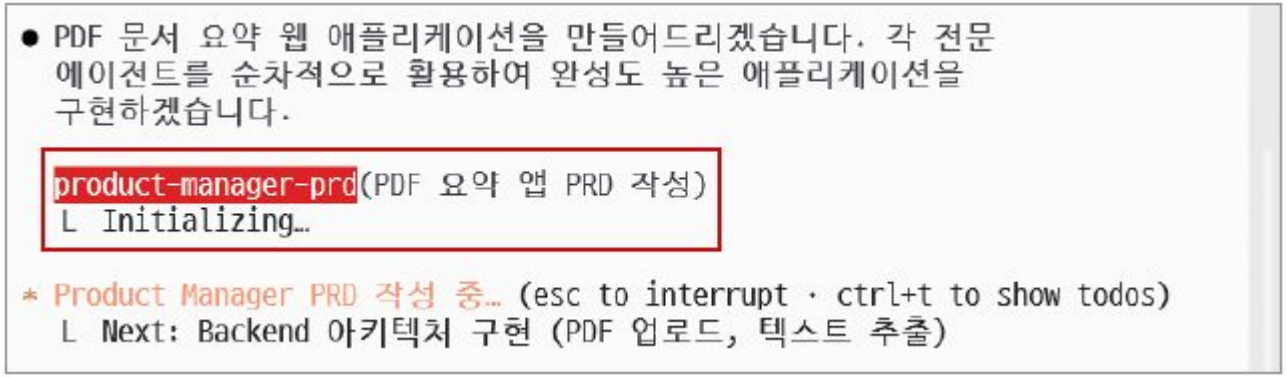


[note]

- **'AI 공감 다이어리' 애플리케이션의 실행 파일 이름과 중복되지 않도록 최종 실행 파일 이름을 'index_pdf.html'로 지정**

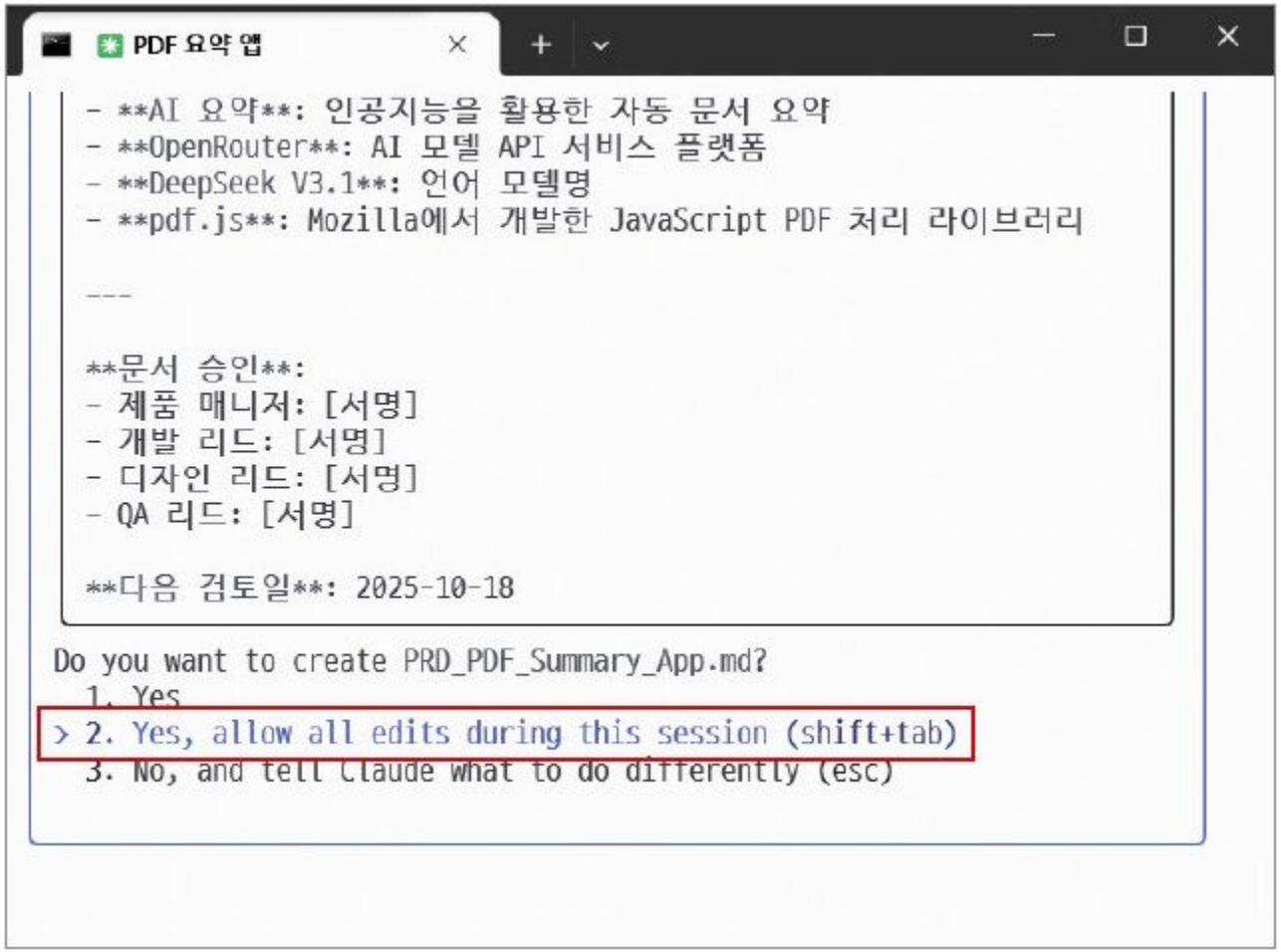
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(26)

02 제품 기획 관리자가 PRD를 만드는 과정부터 시작



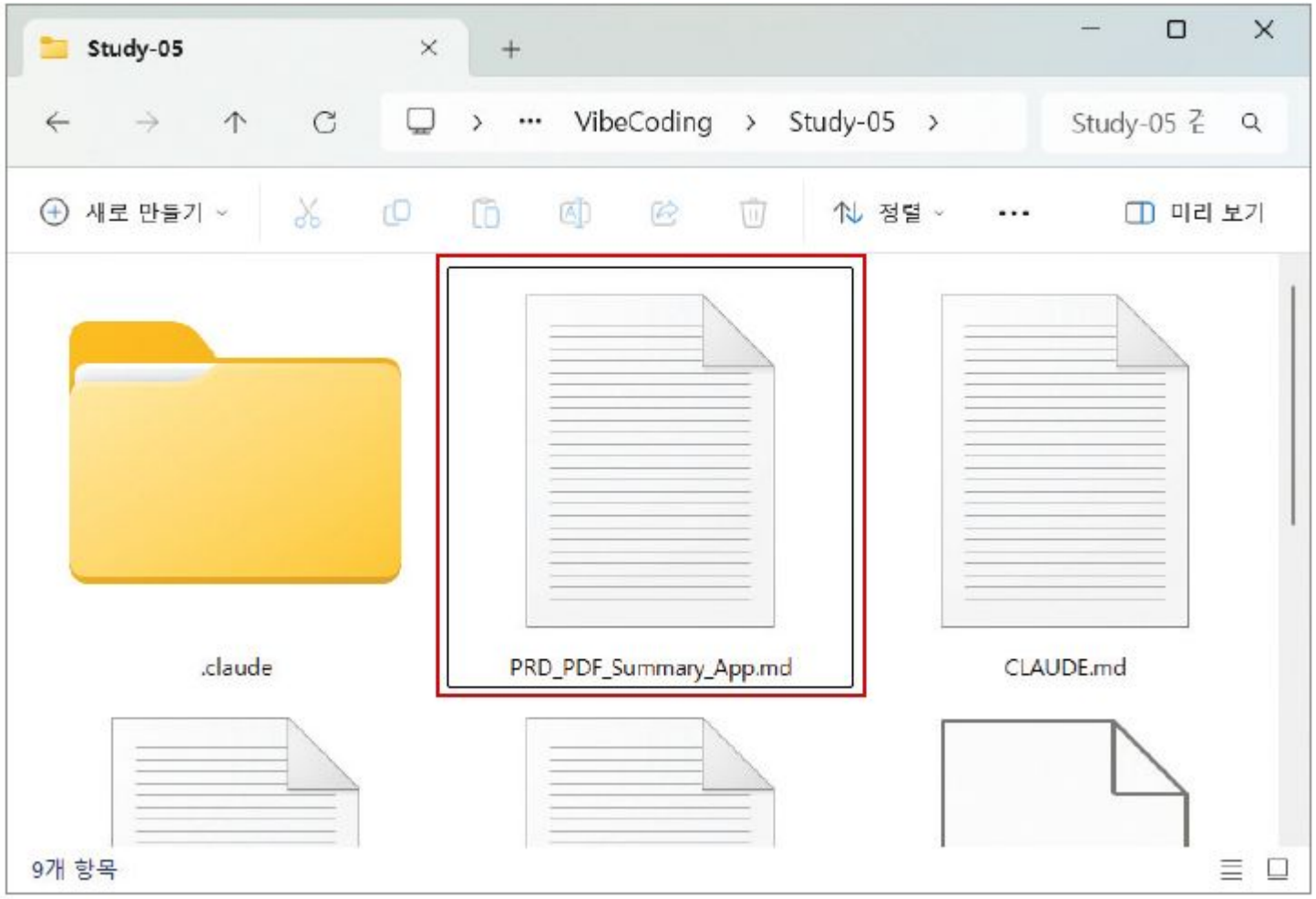
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(27)

03 PRD에 이상이 없으면 '2. Yes, allow all edits during this session'을 선택해 진행



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(28)

04 'Study-05' 폴더에 프로젝트를 위한 PRD 문서 'PRD_PDF_Summary_App.md'가 생성된 것을 확인

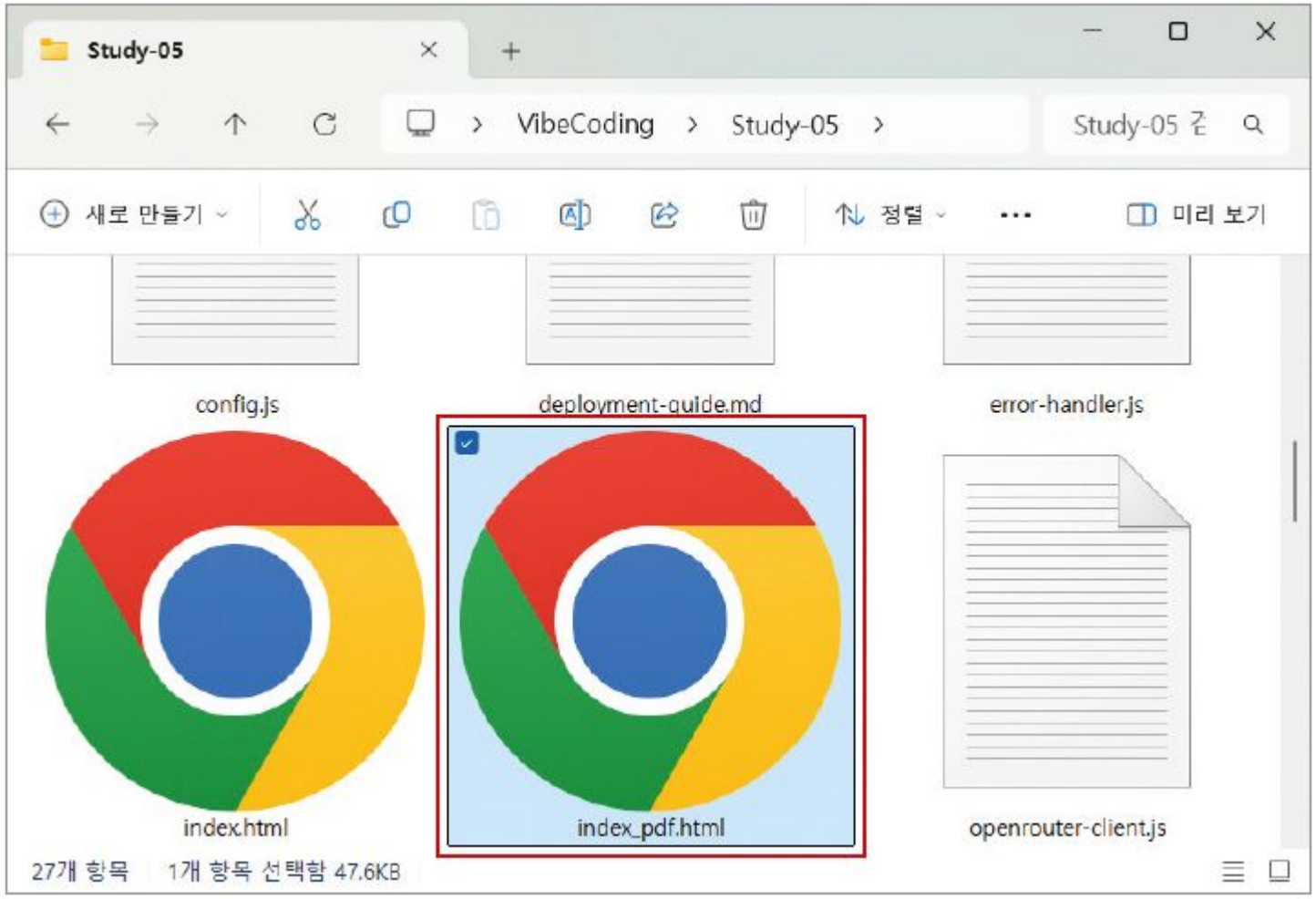


SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(29)

05 다섯 개의 에이전트가 순차적으로 작업을 수행하고, 웹 애플리케이션이 완성

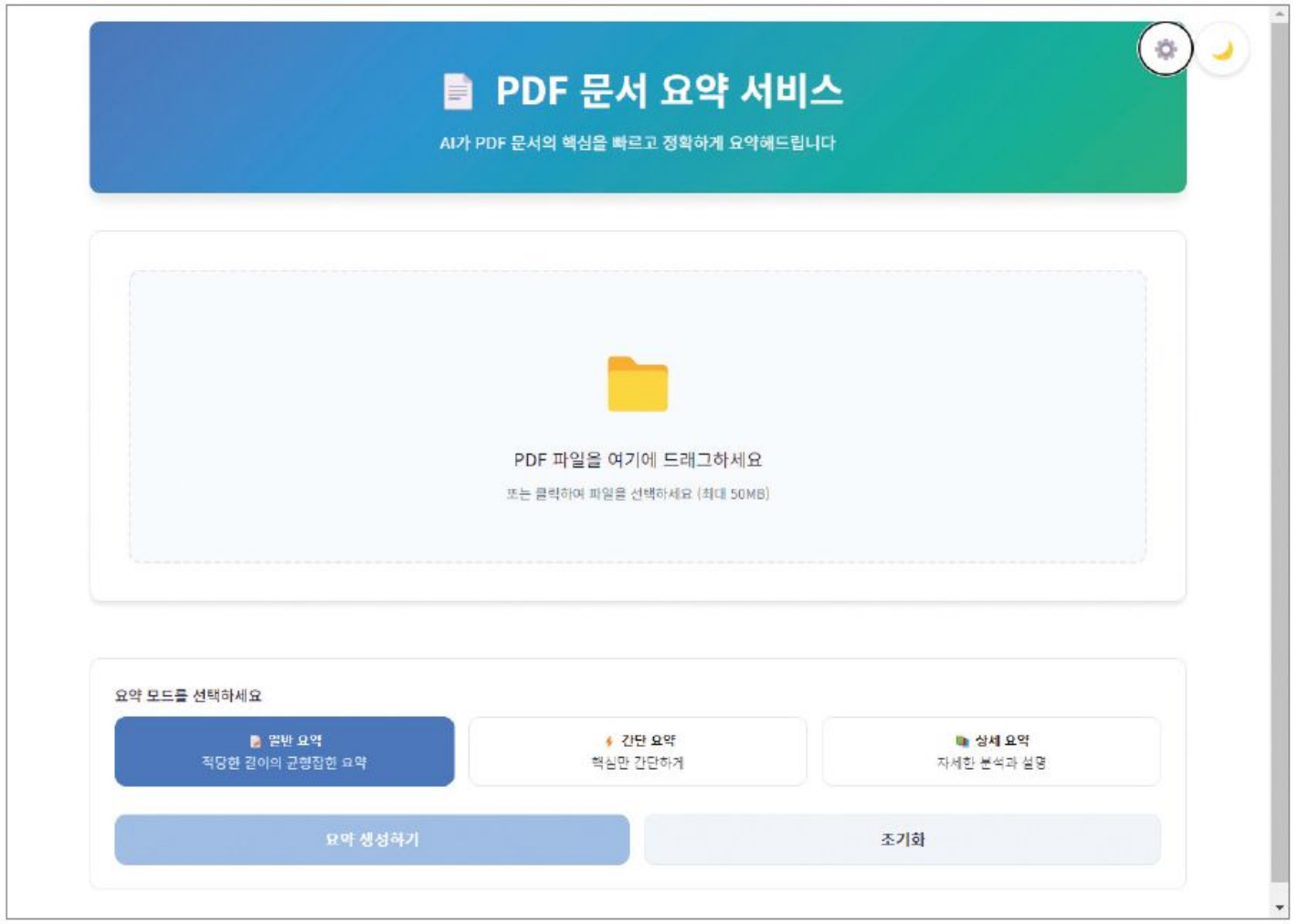
SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(30)

06 'Study-05' 폴더에 새로 생성된 'index_pdf.html' 파일을 찾아 실행



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(31)

07 이와 같이 PDF 문서 요약 서비스앱이 실행되면 완성
원하는 결과가 나오지 않았다면, 현재 상황을 클로드 코드에게 다시 입력하여 수정, 보완



SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(32)

08 테스트를 위해 깃허브 자료실(https://github.com/taehojo/vibecoding)의 '입력예제' 폴더에서 '한글맞춤법.pdf' 파일을 다운로드
애플리케이션으로 돌아와 '한글맞춤법.pdf' 파일을 업로드하고 [일반 요약] 모드를 선택한 뒤, [요약 생성하기] 버튼을 클릭

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(33)

09 PDF 파일의 내용을 요약하여 제시하는 것을 확인

SECTION 7-2 AI 에이전트로 소프트웨어 개발 자동화하기(34)

10 이번에는 [간단 요약] 모드를 선택하고 [요약 생성하기] 버튼을 클릭
조금 다른 요약 결과를 보여줌

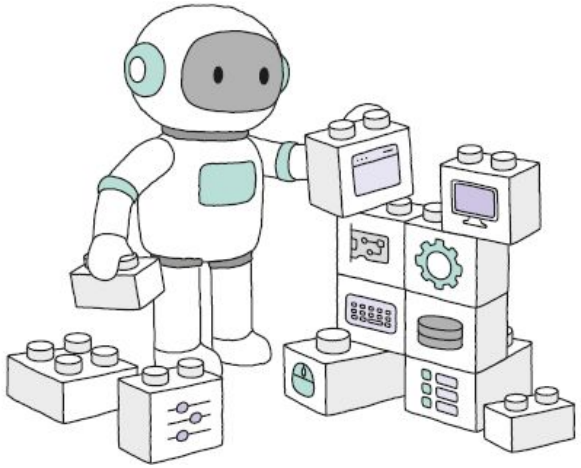
[마무리]

- 5가지 키워드로 정리하는 핵심 포인트
  - **제품 기획 관리자 -** 제품의 목표, 기능, 사용자 요구사항을 정의하고 전체 개발 일정을 관리
  - **백엔드 개발자 -** 서버, 데이터베이스 설계와 API 연동 등 애플리케이션의 핵심 로직을 구현하는 기술 담당자
  - **프런트엔드 개발자 -** 사용자가 보는 화면과 인터페이스를 설계하고 구현하는 UI 개발자
  - **품질 보증 엔지니어 -** 기능과 성능을 테스트해 버그를 수정하고 서비스의 안정성을 확보하는 검증 전문가
  - AI를 도입한 개발 과정에서 AI 모델을 연동, 최적화하기 위해 기술 통합 담당자인 **AI 통합 전문가**가 필요

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(1)

시작하기전에

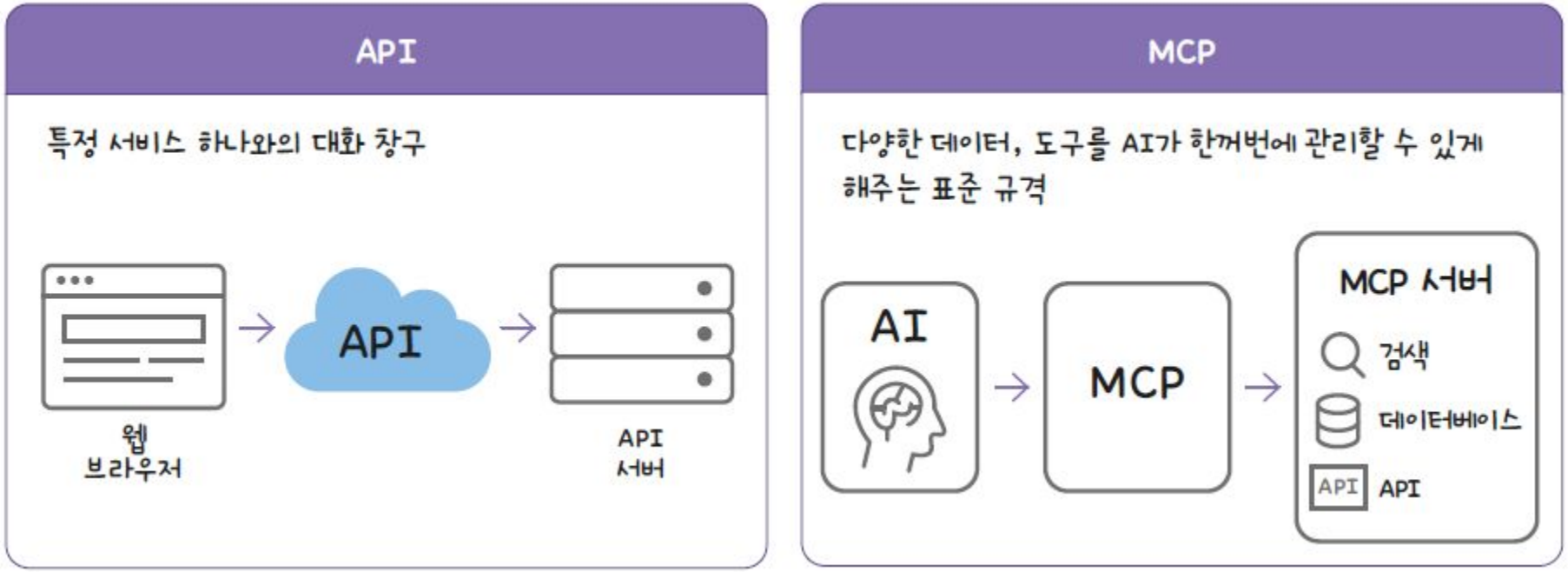
- 레고 블록을 조립할 때 기본 세트만으로는 한계가 있지만, 특별한 부품들을 추가하면 상상했던 모든 것을 만들 수 있음
- MCP는 클로드 코드에 이런 특별한 부품들을 연결해 주는 연결 장치
  - MCP는 클로드를 만든 앤트로픽(Anthropic) 사가 2024년 11월에 제안한 이후, 현재는 AI를 개발하는 전 세계 개발자들의 표준이 됨



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(2)

- MCP가 열어준 AI의 무한 확장 가능성
  - MCP(Model Context Protocol)
    - AI 모델이 외부 데이터, 도구, 서비스와 연결될 때 사용하는 표준화된 통신 규약
  - API는 서로 다른 프로그램이 데이터를 주고받을 수 있게 하는 인터페이스로, 다른 서비스의 기능을 잠시 빌려오는 방식
    - '냉장고를 부탁해' 애플리케이션을 제작하면서 오픈라우터의 API 키를 사용해 AI 모델을 가져와 이미지 인식과 텍스트 생성에 사용
  - MCP는 여러 API와 파일, 데이터베이스를 한꺼번에 통합해 연결하는 프로토콜(protocol)
    - MCP를 통해 AI 모델은 여러 외부 서비스와 데이터 소스에 직접 접근하고 이를 활용
    - API를 활용해 AI 모델에 텍스트 생성을 요청할 수는 있지만, AI 모델이 파일 시스템을 직접 조작하거나 데이터베이스를 수정하게 할 수는 없음
    - MCP를 사용하면 AI가 직접 파일 시스템에 접근하고, 데이터베이스를 조회하거나 수정할 수 있음
  - 클로드 코드는 이미 MCP 클라이언트의 기능을 내장하고 있음

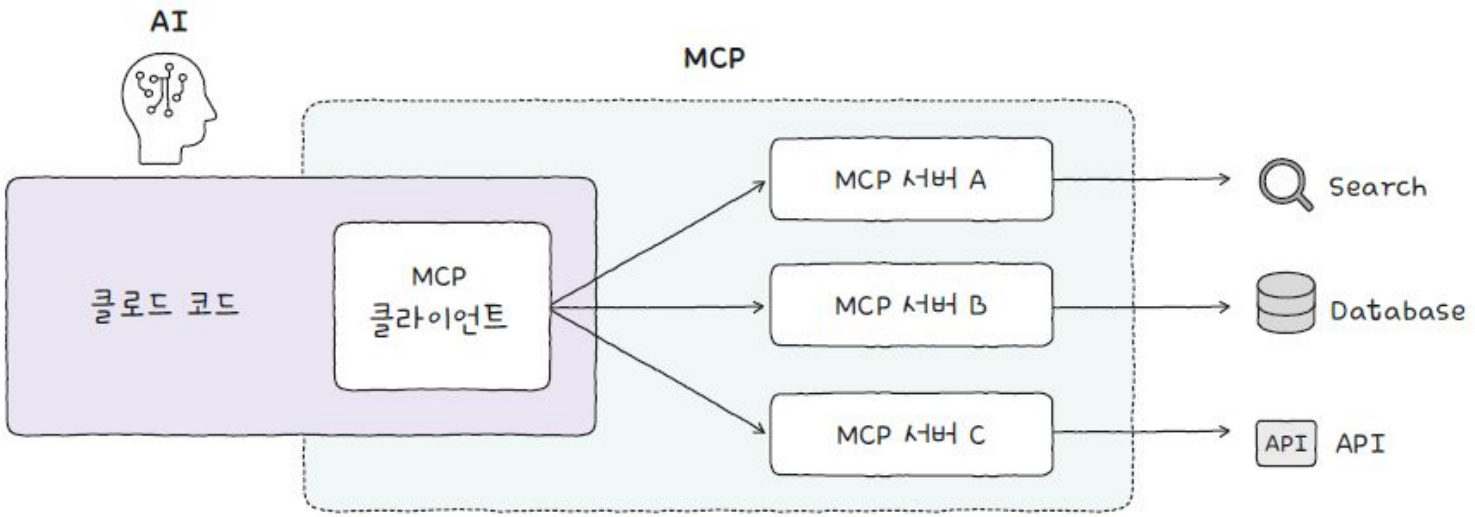
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(3)



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(4)

✚ 여기서 잠깐 **클라이언트-서버 모델**

- 클라이언트 -서버 모델(Client-Server Model, CS model)은 네트워크에서 정보를 요청하고 응답하는 구조
  - 클라이언트 - 서버를 요청하는 쪽
  - 서버 - 서비스를 제공하는 쪽
- 클로드의 MCP도 클라이언트 -서버 모델을 따름
  - 클로드가 클라이언트로서 요청을 보내고, MCP 서버가 실제 작업을 처리해 결과를 돌려주는 구조
  - 이 모델의 장점 두 가지
    1) 새로운 기능을 추가할 때 MCP 서버만 연결하면 되므로 확장이 쉬움
    2) 클로드는 작업 지시만 하고 실제 실행은 각 MCP 서버가 담당하므로 안정성이 높음
  - 단, MCP 서버가 실제로 외부 파일이나 API에 접근하므로 보안 인증과 접근 권한 설정을 꼼꼼히 해야 함



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(5)

◦ 클로드 코드와 노션 연결하기

- MCP를 사용해 문서를 작성하고 관리하는 협업 도구인 노션(Notion)에 클로드 코드를 연동해서 실제로 페이지를 생성하고 문서를 작성하기

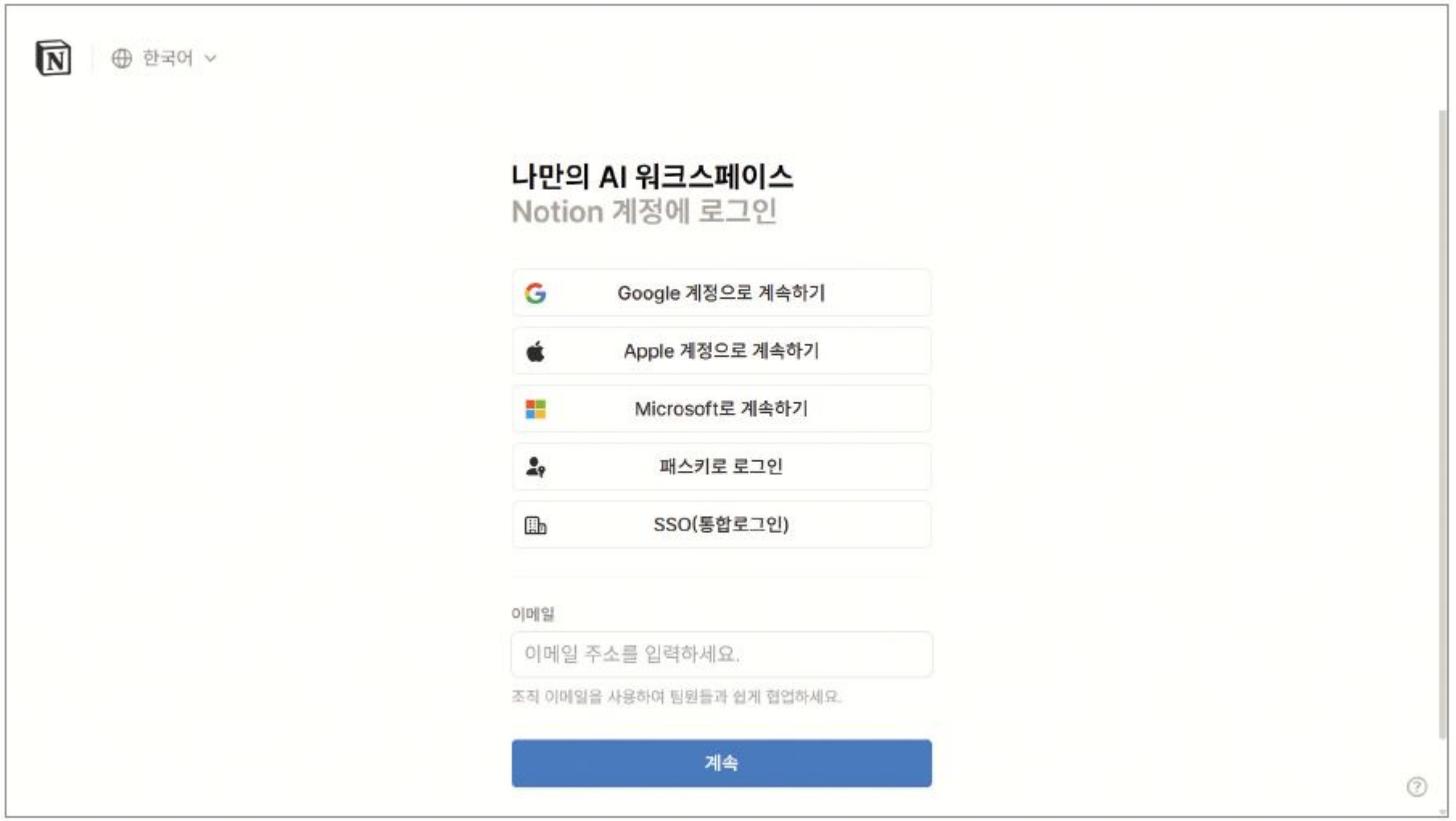
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(6)

노션 가입하기

01 웹 브라우저에서 노션 홈페이지에 접속한 뒤, 화면 우측 상단에 있는 [Notion 무료로 사용하기] 버튼을 클릭 (URL https://www.notion.com)

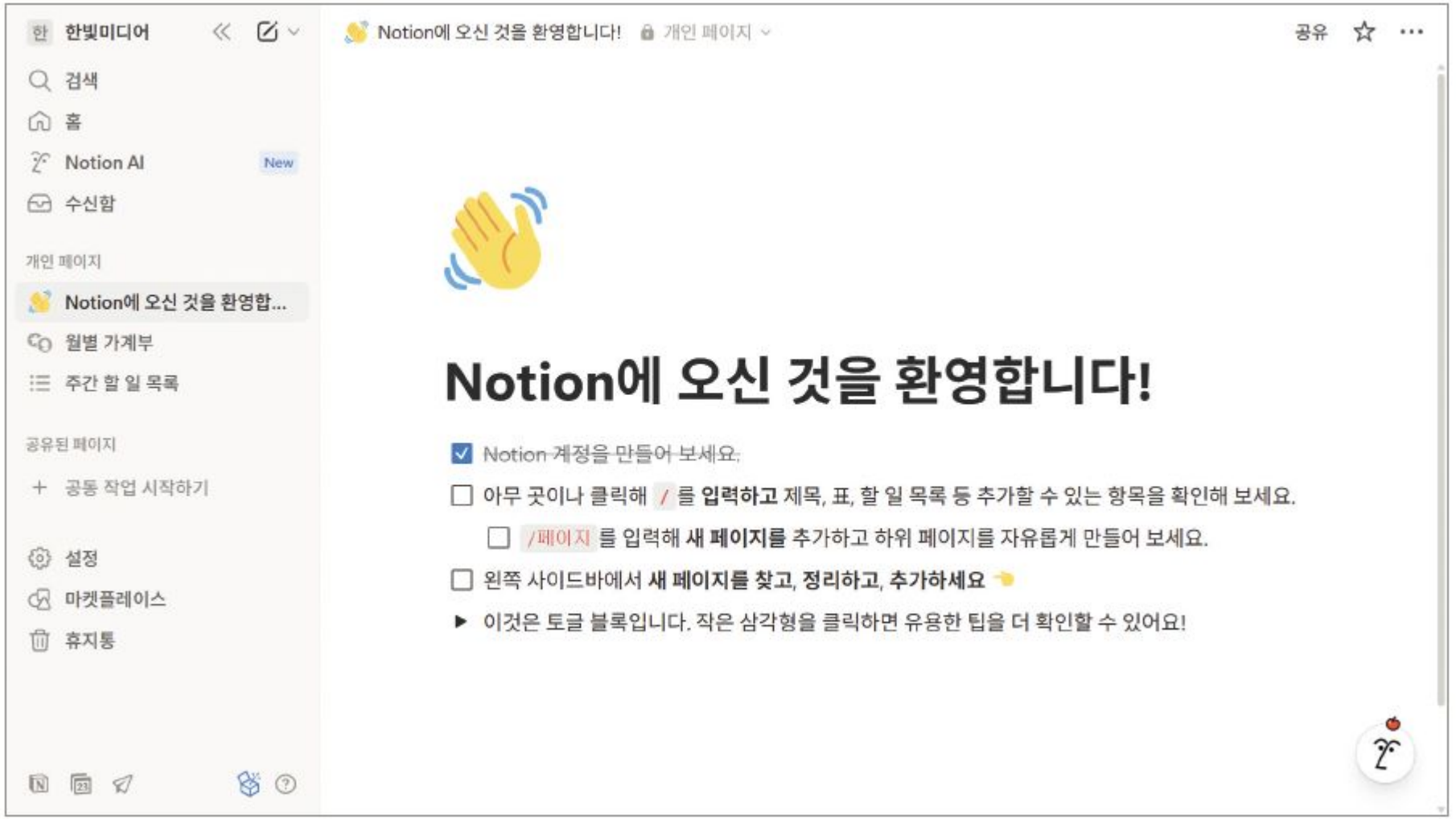
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(7)

02 계정이 있는 서비스와 연동하거나 새로 이메일 주소를 입력해 회원가입



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(8)

03 계정 생성이 완료되면 워크스페이스를 생성할 수 있음 - 이제 노션을 자유롭게 사용



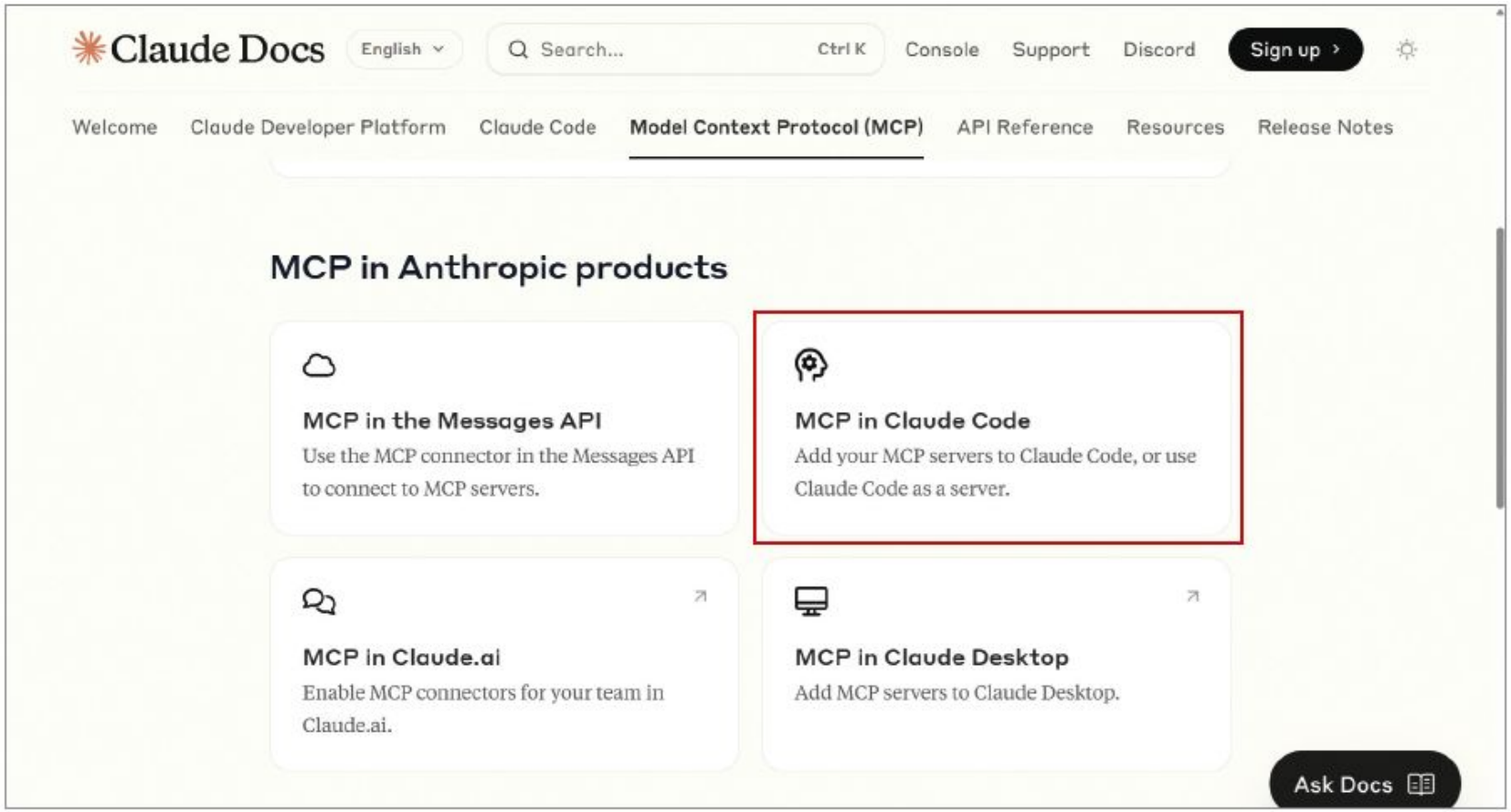
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(9)

노션 MCP 서버 설치하기

- 클로드 코드에서 노션 MCP 서버를 설치해 연동
  - 우선 데스크톱의 'VibeCoding' 폴더에 새로운 'Study-06' 폴더를 만들고, 해당 위치에서 터미널을 실행해 준비(아직 클로드를 실행하지는 말고, 터미널을 열어둔 상태)
- MCP 서버를 추가하는 과정
  1) 터미널에서 claude mcp add 명령을 이용해 원하는 MCP 서버를 설치
  2) 노션처럼 외부 서비스와 연결되는 MCP의 경우에는 계정 연동을 위한 인증 과정을 진행
  3) 클로드 코드에서 해당 MCP의 기능을 실제로 사용할 수 있도록 프롬프트를 작성해 작업을 실행
- 터미널에 설치 명령어를 입력해서 MCP 서버를 다운로드하고, 설정 파일에 등록할 것임
- MCP 서버를 다운로드할 수 있는 명령어를 클로드 코드의 공식 문서에서 찾아보기

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(10)

01 먼저 웹 브라우저에서 클로드 코드의 MCP 공식 문서에 접속
그런 다음 화면을 내려 [MCP in Claude Code]를 찾아 클릭
(URL https://docs.claude.com/en/docs/mcp)

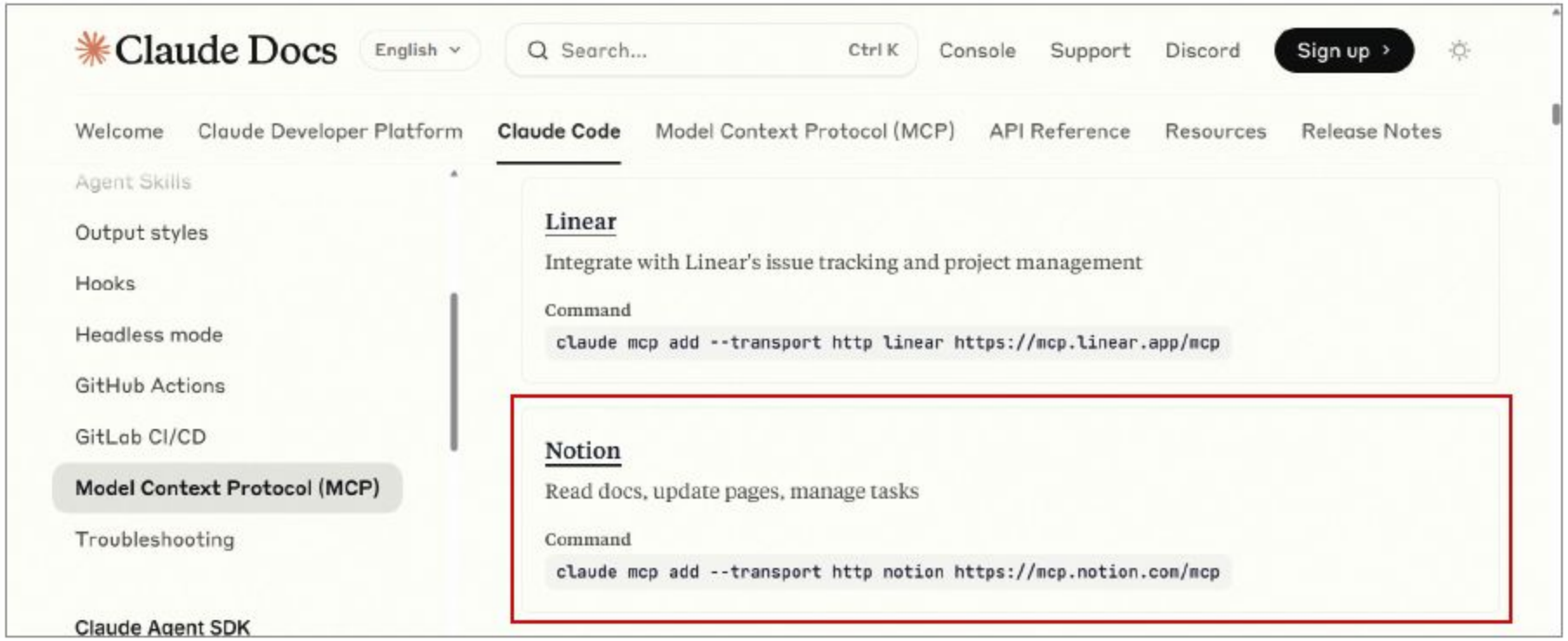


SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(11)

02 클로드 코드에 적용할 수 있는 MCP 서버 목록이 소개

화면을 스크롤해 'Project Management & Documentation' 목록에서 'Notion'을 찾기

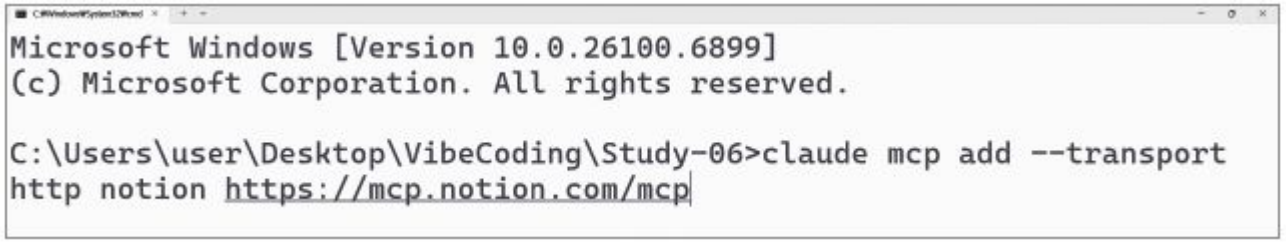
'Command' 항목에 있는 명령어 'claude mcp add --transport http notion https://mcp.notion.com/mcp'를 복사해 준비



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(12)

03 실행한 터미널에 복사한 노션 MCP 서버의 설치 명령어를 입력

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```



[note] 클로드 코드를 실행하기 전에 **MCP** 서버 설치 명령어를 먼저 입력

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(13)

04 MCP 서버가 성공적으로 추가

```
Added HTTP MCP server notion with URL: https://mcp.notion.com/mcp to local
config
File modified: C:\Users\taehj\.claude.json [project: C:\Users\taehj\OneDriv
e\Desktop\VibeCoding\Study-06]

C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude
```

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(14)

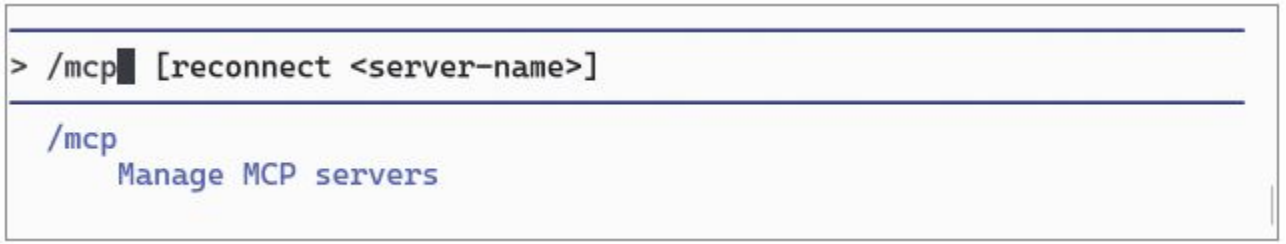
MCP 서버 인증으로 연결 완성하기

01 MCP 서버를 설치했으니 터미널에 'claude' 명령어를 입력해 클로드 코드를 실행

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(15)

02 클로드 코드 내 설치된 MCP의 목록과 서버 상태를 확인하기 위해 '/mcp' 명령어를 입력

```
/mcp
```



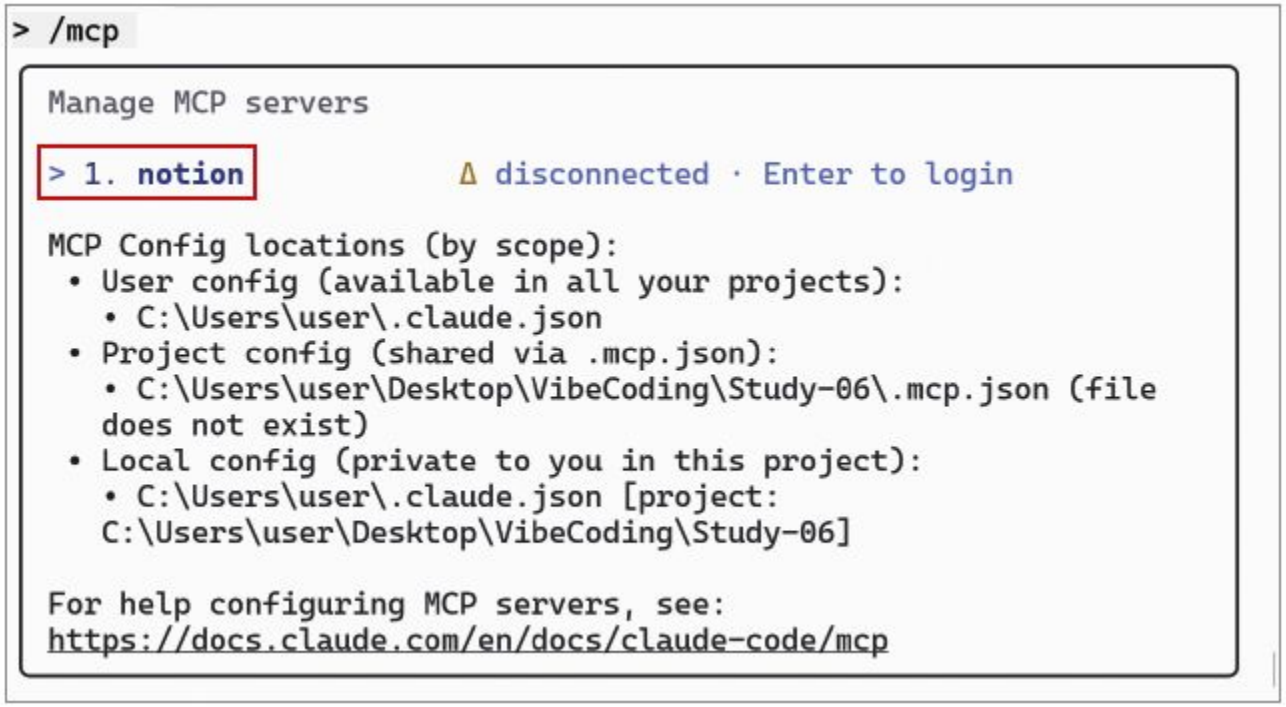
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(16)

03 MCP 서버 관리 창에서 노션 MCP가 등록되어 있는 것을 확인할 수 있음
하지만 아직 로그인하지 않아 연결되지 않은 상태
[Enter] 키를 눌러 연결을 진행

화면 해설

```
MCP 서버 관리

1. 노션                    △ 연결되어 있지 않음 • 로그인

MCP 설정 파일 위치:
  • 사용자 설정 파일(모든 프로젝트에 공통으로 적용): C:\Users\user\.claude.json
  • 프로젝트 설정 파일(특정 프로젝트에만 적용): C:\Users\user\Desktop\VibeCoding\Study-06\.mcp.
                                                json
  • 로컬 설정 파일(현재 프로젝트에만 적용): C:\Users\user\.claude.json [project: C:\Users\user\
                                            Desktop\VibeCoding\Study-06]
MCP 서버 설정 파일 관련 도움말: https://docs.claude.com/en/docs/claude-code/mcp
```

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(17)



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(18)

04 인증을 통해 클로드 코드가 노션 계정에 접근할 수 있게 설정

'1. Authenticate'를 선택하고 [Enter]

화면 해설

```
노션 MCP 서버
상태: 인증이 필요합니다.
URL: https://mcp.notion.com/mcp
설정 정보 위치: C:\Users\user\.claude.json
              C:\Users\user\Desktop\VibeCoding\Study-06
1. 인증하기
```

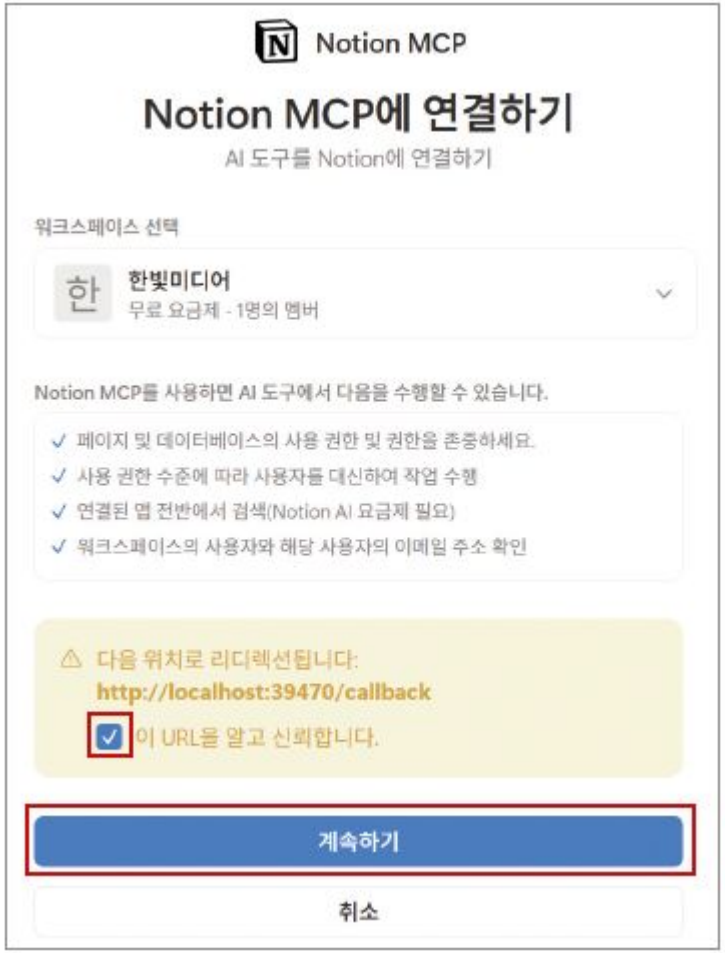
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(19)

05 잠시 기다리면 웹 브라우저에 새 창이 열림

```
Authenticating with notion…

✶  A browser window will open for authentication

If your browser doesn't open automatically, copy this URL manually:
https://mcp.notion.com/authorize?response_type=code&client_id=3e58KvQ
[redacted]
ce=https%3A%2F%2Fmcp.notion.com%2F

   Return here after authenticating in your browser. Press Esc to go
   back.
```

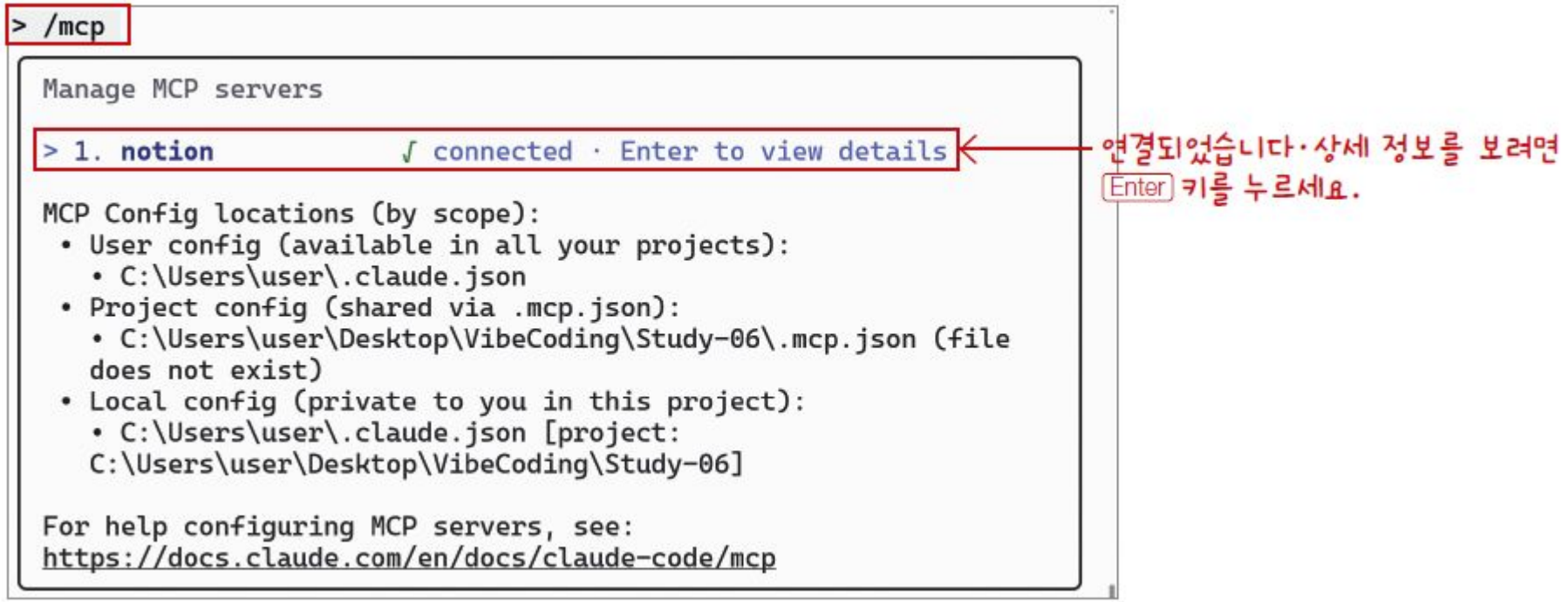
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(20)

06 MCP 서버에 노션 계정을 연결하기 위해 '이 URL을 알고 신뢰합니다.' 체크박스를 클릭한 뒤, [계속하기] 버튼을 클릭



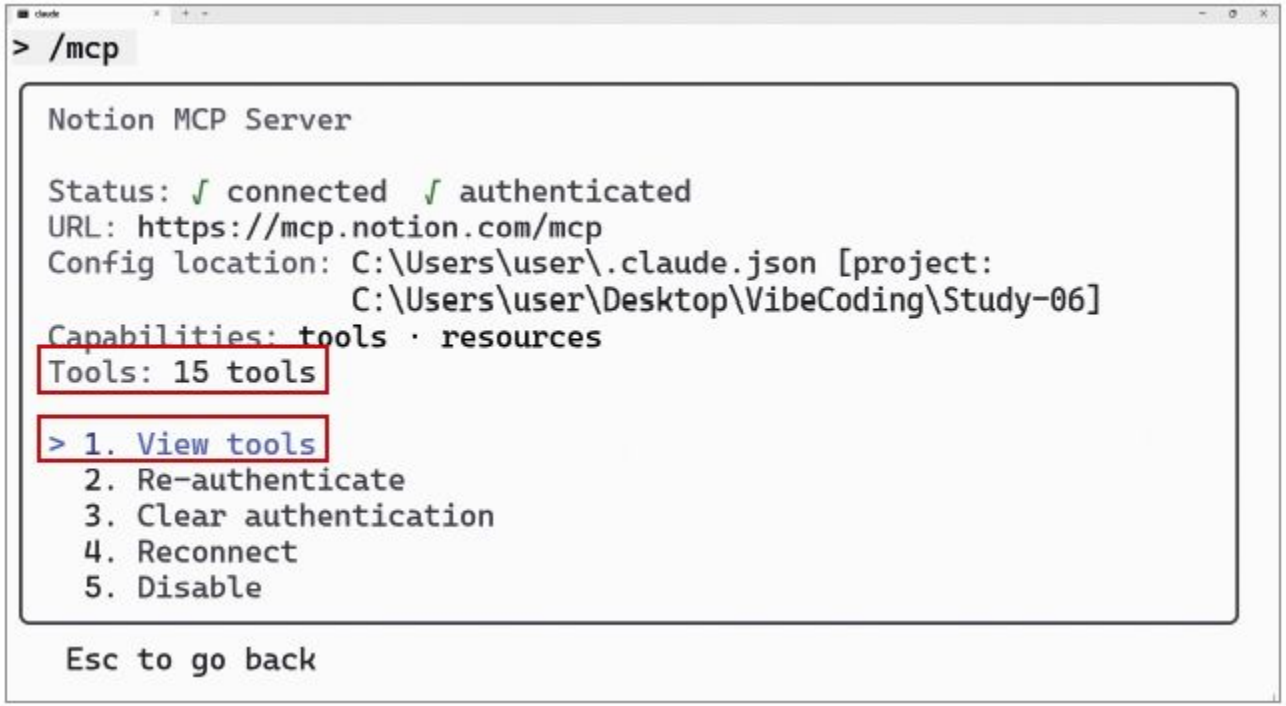
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(21)

07 ‘Authentication Successful’이라는 메시지가 보이면 인증에 성공
클로드 코드로 돌아가 다시 ‘/mcp’ 명령어를 입력
노션 MCP 서버가 연결되었음을 확인
[Enter] 키를 눌러 상세 정보를 확인



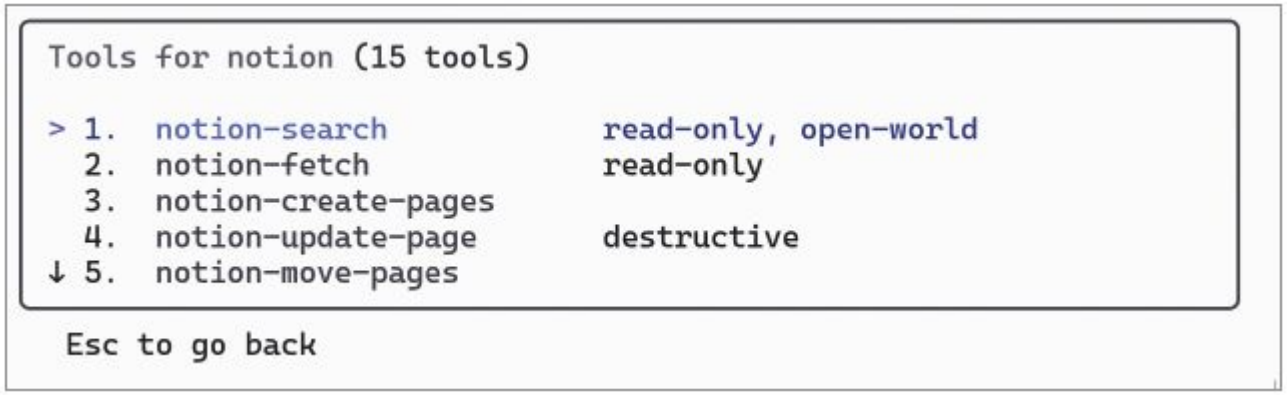
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(22)

08 노션 MCP 서버 안에는 클로드가 사용할 수 있는 15개의 작업 도구가 등록되어 있음을 확인
'1. View tools'를 선택한 뒤, [Enter]



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(23)

09 노션 MCP 서버가 제공하는 구체적인 도구 목록을 볼 수 있음
[Esc] 키를 눌러 서버 관리 창에서 나가기



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(24)

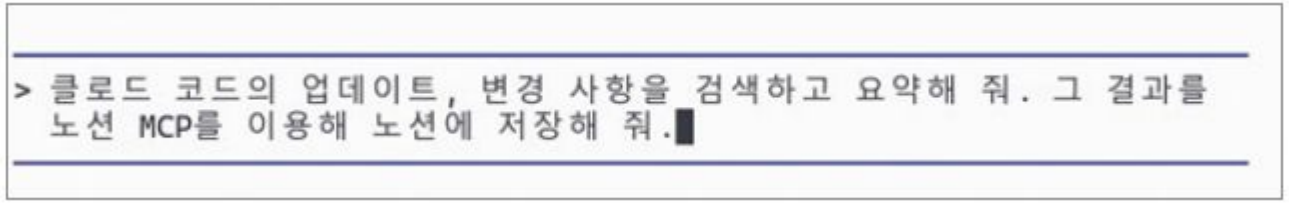
노션 MCP로 자동 리서치 노트 만들기

- 노션 MCP와 연동하여 실제 리서치 노트 자동화를 실행
  - 클로드 코드의 최신 업데이트를 검색하고 노션에 이를 요약해 업로드하도록 자동화

01 클로드 코드에 다음과 같이 프롬프트를 입력

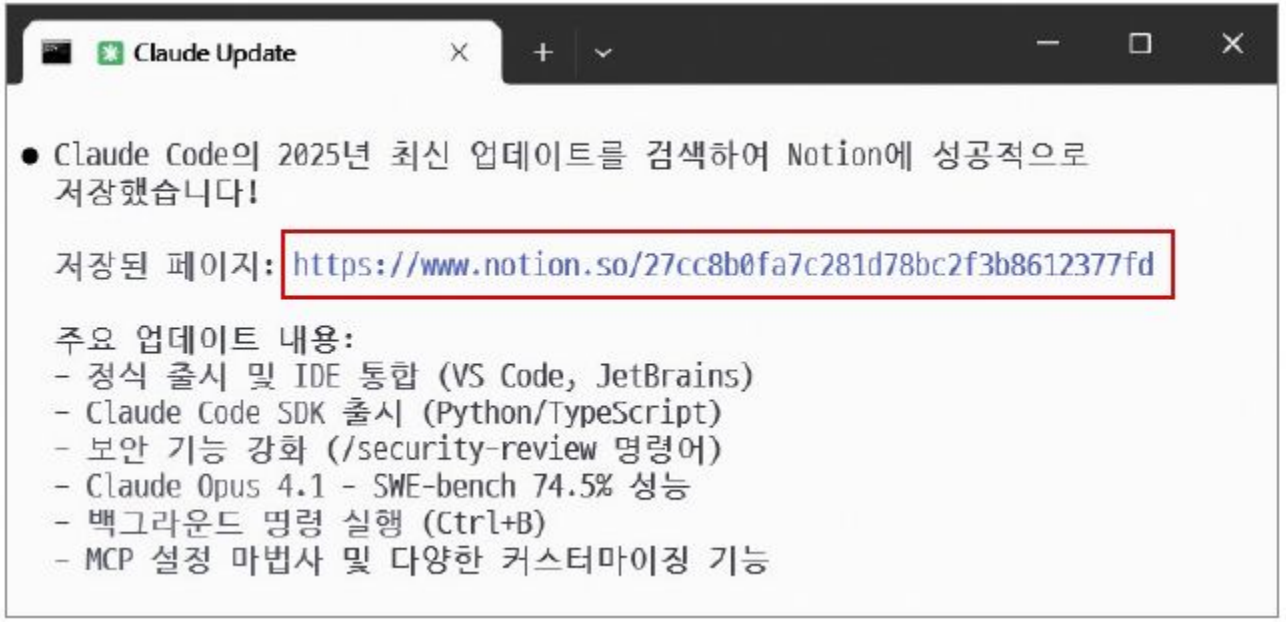
프롬프트

```
클로드 코드의 업데이트, 변경 사항을 검색하고 요약해 줘. 그 결과를 노션 MCP를 이용해 노션에 저장해 줘.
```



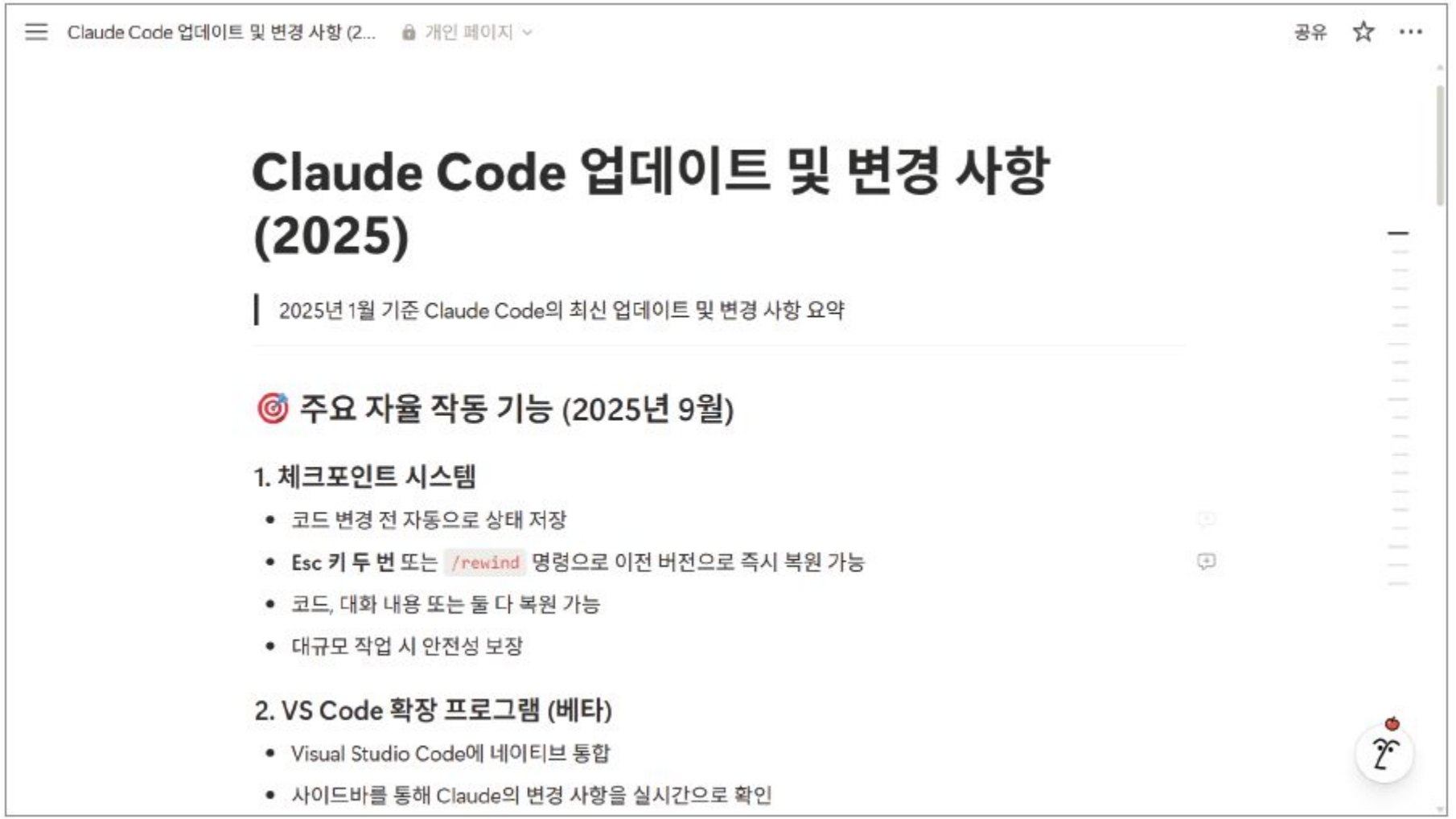
SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(25)

02 ‘1. Yes’를 눌러 계속 진행하면 이와 같이 최신 업데이트를 검색해서 노션에 성공적으로 저장했다는 메시지를 볼 수 있음 - 이제 실제로 노션에 문서가 생성되었는지 확인



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(26)

03 노션에 클로드 코드 최신 업데이트 문서가 생성되어 저장되어 있음을 볼 수 있음



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(27)

✚ 여기서 잠깐 **원하는 MCP 서버를 찾는 방법**

- MCP는 누구나 만들고 배포할 수 있기 때문에 현재에는 수천 가지가 넘는 MCP 서버들이 존재
- 클로드 코드의 MCP 공식 문서에도 몇 가지 MCP들이 있지만, 구글에서 원하는 MCP를 검색하면 더 많은 정보를 찾을 수 있음
- 다양한 MCP 서버를 카테고리별로 분류해 비교 탐색할 수 있는 플랫폼도 많으니 참고하는 것이 좋음

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(28)

◦ 순차적 추론 MCP로 더 똑똑한 AI 만들기

로컬 MCP와 원격 MCP 이해하기

- 원격 MCP - 그 중개 프로그램마저 인터넷상 외부 서버에서 실행하는 방식
  - 원격 MCP는 이미 외부 서버에 준비된 MCP에 접속해 사용하는 방식으로, 명령어 한 줄만 있으면 바로 연결
  - 인터넷을 통해 접근 가능한 클라우드 서버에 연결되며, 대부분 로그인이나 인증 과정을 거쳐 사용자의 계정과 연결

```
claude mcp add --transport http [내가 정한 이름] [서버 URL]
```

- 로컬 MCP - MCP 중개 프로그램을 내 컴퓨터 안에서 직접 실행하는 방식
  - MCP를 내 컴퓨터 안에서 직접 실행해 사용 - 인터넷 주소를 불러오는 대신 직접 실행할 명령어를 지정
  - 이미 준비된 URL을 입력하는 대신 실행할 명령어를 직접 지정해야 하기 때문에 약간의 설정이 필요하지만, 인터넷 없이도 작동하며 응답 속도도 더 빠르고 개인정보 보호에 유리
  - 명령어의 '--' 기호는 클로드 코드의 옵션과 MCP 실행 명령을 구분하는 역할

```
claude mcp add [내가 정한 이름] [옵션] -- [MCP 실행 명령]
```

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(29)

✚ 여기서 잠깐 **인증이란?**

- 외부 서비스와 MCP를 연결할 때는, 접속하려는 사람이 정말 허가된 사용자인지 확인하기 위해 인증(authentication)이 필요
- MCP 서버는 여러 사람의 데이터를 관리
  - 누가 요청을 보내는지 모른다면, 다른 사람의 정보나 파일에 잘못 접근
  - 인증을 통해 이 요청을 보낸 사람이 진짜 사용자임을 증명
- 원격 MCP는 인터넷을 통해 통신하므로 중간에서 데이터가 유출될 위험이 있음
- 인증 절차를 밟아 서버와 클라이언트가 암호화된 통신 하에 안전하게 데이터를 주고받을 수 있게 해야 함

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(30)

로컬 MCP 설치 명령어 이해하기

- 설치할 MCP는 Sequential Thinking MCP로, 앤트로픽이 만들어 배포한 공식 MCP 서버
  - 주어진 문제를 작은 단위로 분해해 순차적으로 추론하도록 돕는 도구

```
claude mcp add sequential-thinking -s local -- cmd /c npx -y
                ❶                  ❷       ❸   ❹      ❺

@modelcontextprotocol/server-sequential-thinking@latest
                ❻
```

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(31)

1. sequential-thinking
   - 클로드 코드에서 이 MCP를 구분하기 위한 이름
2. -s local
   - MCP의 적용 범위를 지정
   - ‘local’은 현재 프로젝트에만 적용
   - project로 설정하면 같은 프로젝트 팀원들과 공유
   - user로 설정하면 내 계정 전체 프로젝트에서 사용
   - 기본값은 local이므로, -s 플래그를 생략해도 동일하게 적용
3. --
   - 클로드 코드의 옵션과 인자를 나누는 기준
   - 기호를 기준으로 앞부분에는 클로드 코드 옵션이, 뒷부분에는 MCP 서버 실행 명령이 위치
4. cmd /c
   - 윈도우 환경에서 명령을 실행하고 종료하는 역할
   - 맥OS나 리눅스 환경에서는 필요하지 않으므로 이 부분을 생략해도 됨

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(32)

5. npx -y
   - Node.js 패키지를 실행
   - '-y'는 패키지가 로컬에 설치되어 있지 않을 때 자동으로 설치를 승인하도록 하는 옵션
     - 즉, 'Ok to proceed? (y)' 메시지를 자동으로 건너뜀
6. @modelcontextprotocol/server-sequential-thinking@latest
   - 불러올 MCP 패키지명
     - '@조직명/패키지명@버전' 순서로 구성
     - 즉, MCP 공식팀(modelcontextprotocol)이 배포한 Sequential Thinking MCP 패키지(server-sequential-thinking)의 최신 버전(latest)을 의미
   - 개인이 만든 MCP 서버는 '@조직명'을 생략할 수 있음
   - '@버전' 부분도 생략할 수 있으며 '@2.0'과 같이 특정한 버전 번호를 붙이면 해당 버전을 사용하게 됨

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(33)

✚ 여기서 잠깐 **이중 하이픈 '--'의 두 가지 기능**

- 명령어에서 짧은 옵션은 '-s'처럼 하이픈 한 개(-)에 한 글자를 붙여 쓰고, 긴 옵션은 '--scope'처럼 하이픈 두 개(--)에 여러 글자를 붙여 씀
- 이중 하이픈 '--' 뒤에 공백이 오면, 이것은 명령어의 옵션과 인자 사이를 구분하는 특별한 구분자가 됨

```
claude mcp add mytool --scope local -- cmd /c …
                      ①             ②
```

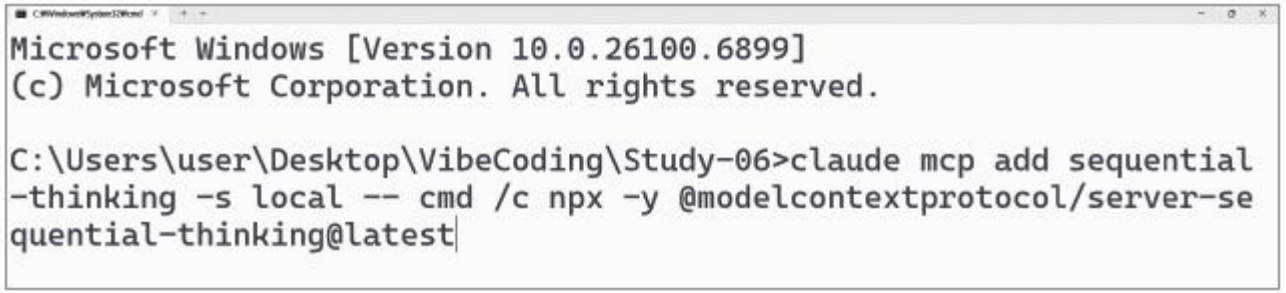
① --scope:긴 옵션 '--' 바로 뒤에 공백 없이 옵션 이름 'scope'가 붙어 있음
② '--': 구분자 '--' 뒤에 공백이 오고, 그 뒤에 'cmd /c …'는 모두 인자로 처리됨

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(34)

순차적 추론 MCP 설치하고 활용하기

01 클로드 코드에 Sequential Thinking MCP 설치 명령어를 입력해 실행

```
claude mcp add sequential-thinking -s local -- cmd /c npx -y
@modelcontextprotocol/server-sequential-thinking@latest
```



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(35)

02 설치가 끝나면 'claude mcp list'를 입력해 클로드 코드에 설치된 MCP 목록을 확인
방금 설치한 순차적 추론 MCP가 'sequential-thinking'이라는 이름으로 설치 완료

```
claude mcp list
```

```
C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp list
Checking MCP server health...

notion: https://mcp.notion.com/mcp (HTTP) - ✓ Connected
sequential-thinking: cmd /c npx -y @modelcontextprotocol/server-seq
uential-thinking@latest - ✓ Connected

C:\Users\user\Desktop\VibeCoding\Study-06>
```

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(36)

◦ 순차적 추론 MCP의 성능 확인하기

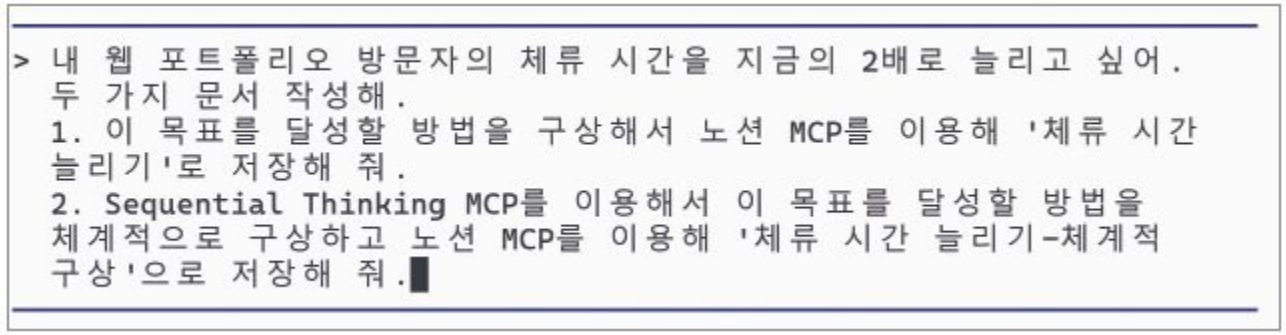
- 설치한 Sequential Thinking MCP의 추론 능력이 일반적인 추론과 얼마나 다른지 비교
- 목표 - 웹 포트폴리오의 방문자가 사이트에 머무는 시간을 두 배로 늘리는 전략 수립
- 터미널에 ‘claude’ 명령어를 입력해 클로드 코드를 실행
  - 먼저 일반적인 방식으로 ‘방문자 체류 시간을 늘리는 방법’을 요청해서 ‘체류 시간 늘리기’라는 제목으로 노션에 저장
  - Sequential Thinking MCP를 이용해 같은 주제를 더 체계적이고 단계적인 방식으로 요청한 뒤 ‘체류 시간 늘리기-체계적인 구상’이라는 이름으로 노션에 저장해 비교

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(37)

01 노션 MCP와 Sequential Thinking MCP를 사용해 목표를 달성하기 위한 방법을 구상하고 노션에 문서로 작성 - 다음과 같이 프롬프트를 입력

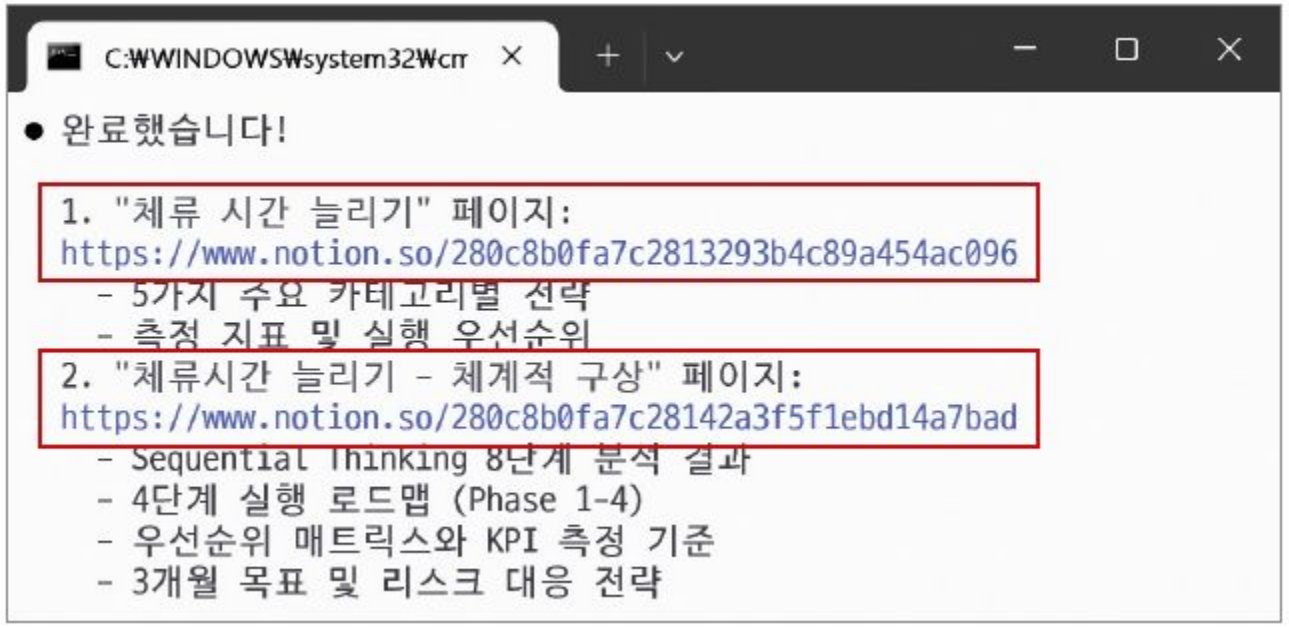
프롬프트

```
내 웹 포트폴리오 방문자의 체류 시간을 지금의 2배로 늘리고 싶어. 두 가지 문서를 작성해.
1. 이 목표를 달성할 방법을 구상해서 노션 MCP를 이용해 '체류 시간 늘리기'로 저장해 줘.
2. Sequential Thinking MCP를 이용해서 이 목표를 달성할 방법을 체계적으로 구상하고 노션 MCP를 이용해
'체류 시간 늘리기 – 체계적 구상'으로 저장해 줘.
```



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(38)

02 잠시 기다리면 완료 메시지와 함께 두 개의 노션 문서 주소를 알려줌



SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(39)

03 노션에 두 개의 새로운 페이지가 생성

SECTION 8-1 MCP 이해하고 클로드 코드와 연결하기(40)

04 두 문서의 크기를 비교

- 일반 추론의 경우 총 85줄, 순차적 추론은 총 222줄로 분량에서 차이
- 목차 역시 순차적 추론의 경우 목표, 실행 방법, 실행 로드맵, 측정 지표, 리스크 대응 등으로 더 상세

[마무리]

- 6가지 키워드로 정리하는 핵심 포인트
  - **MCP**는 AI 모델이 외부 데이터, 도구, 서비스와 연결될 때 사용하는 표준화된 통신 규약
  - **클라이언트 서버 모델**은 네트워크에서 정보를 요청하고 응답하는 구조
    - 클라이언트는 서버를 요청하는 쪽이고, 서버는 서비스를 제공하는 쪽
  - **로컬 MCP**는 MCP 중개 프로그램을 내 컴퓨터 안에서 직접 실행하는 방식
  - **원격 MCP**는 그 중개 프로그램마저 인터넷상 외부 서버에서 실행하는 방식
  - **인증**은 접속하려는 사람이 정말 허가된 사용자인지 확인하는 절차
  - 이중 하이픈 **--**은 두 글자 이상인 옵션 이름을 표시하거나, 명령어와 옵션을 구분할 때 쓰이는 기호로, 위치와 공백 사용에 따라 의미가 달라지므로 혼동에 주의

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(1)

시작하기전에

- 프로그램을 안정적으로 운영하려면 단순히 코드를 완성하는 것만으로는 충분하지 않음
- 코드가 의도한 대로 실행되는지, 오류가 없는지를 확인하는 테스트(test) 과정이 필요
  - 테스트를 통해 기능을 점검하고 새 코드를 추가해도 기존 기능이 손상되지 않게 할 수 있음
  - 특히 Playwright 같은 자동화 도구를 활용하면 반복적인 검증을 AI가 대신 수행
- 버전 관리(version control) - 프로젝트의 안정성
  - 코드의 모든 변화를 기록하고 필요할 때 이전 상태로 되돌릴 수 있어 실수나 오류로부터 안전
  - 깃허브(GitHub)를 이용하면 협업 과정에서도 각자의 변경 이력을 체계적으로 관리하며, 여러 사람이 동시에 작업해도 안정적으로 프로젝트를 유지

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(2)

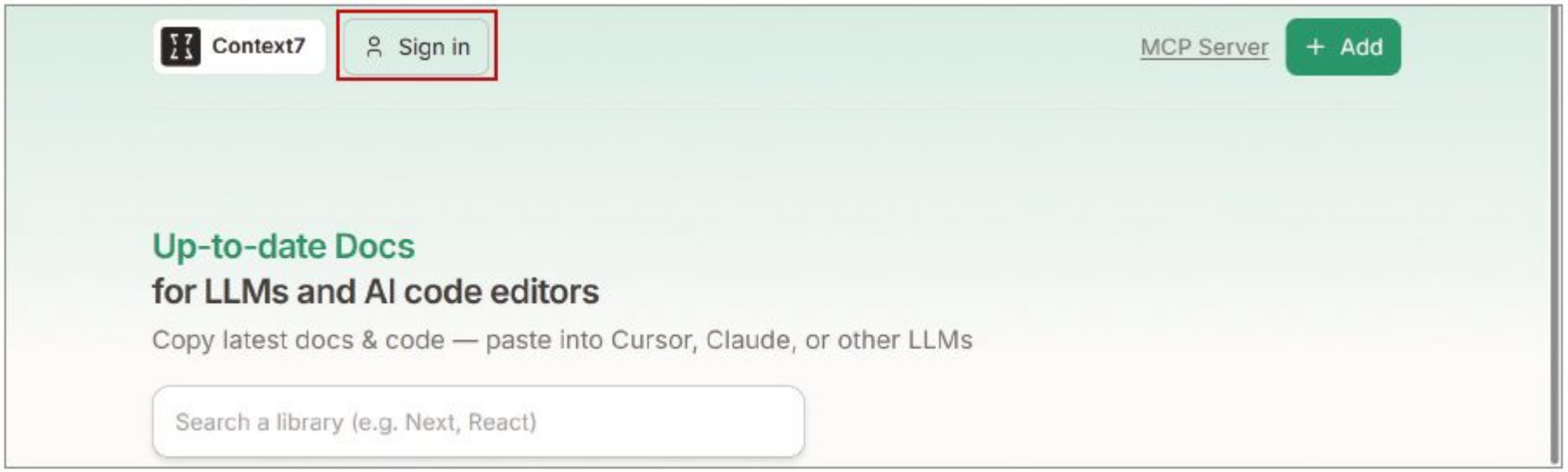
◦ 원하는 MCP를 찾는 방법

- MCP 서버를 찾는 가장 효과적인 방법은 구글 검색과 깃허브 저장소를 활용
  - 대부분의 MCP 서버는 오픈소스로 공개되어 있어 깃허브에서 찾을 수 있으며, 공식 문서를 통해 설치 방법을 확인할 수 있음
- 개발자들이 가장 많이 설치하는 두 개의 MCP를 설치해 보면서 그 과정을 학습
  - Context7: 클로드 코드가 항상 최신 문서를 참고하도록 돕는 지식 연결 도구
  - Playwright: 웹 사이트를 자동으로 테스트하고 UI를 검증하는 브라우저 제어 도구

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(3)

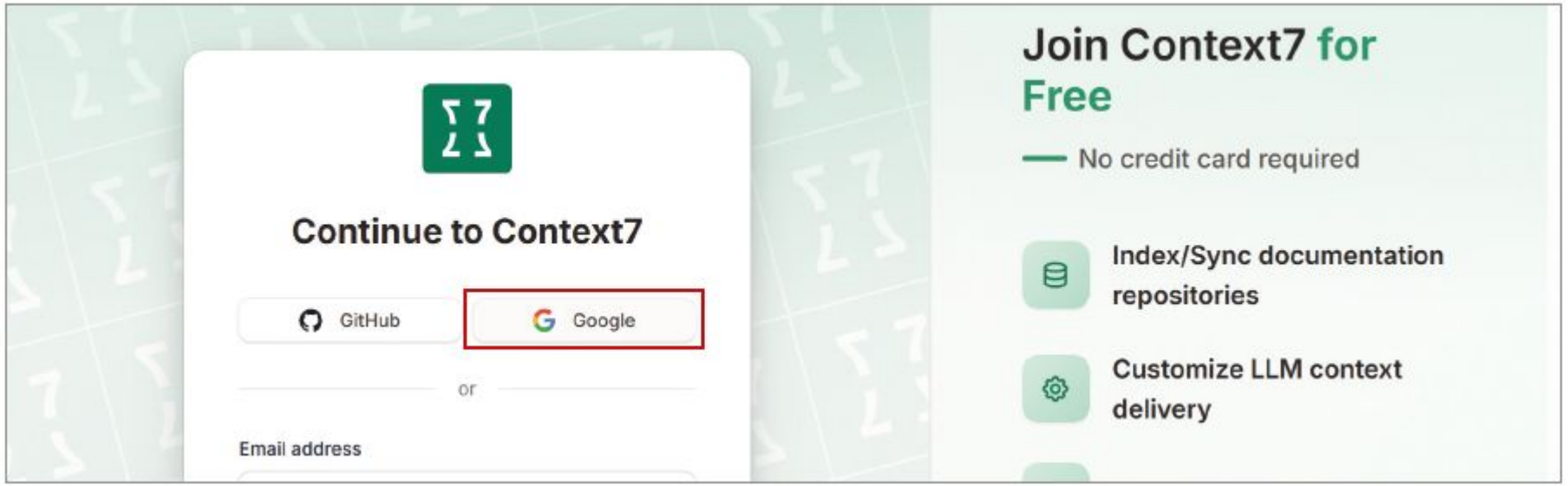
Context7 API 키 발급받기

01 먼저 Context7 홈페이지를 방문하고 [Sign in] 버튼을 눌러 회원가입을 시작 (URL https://www.context7.com)



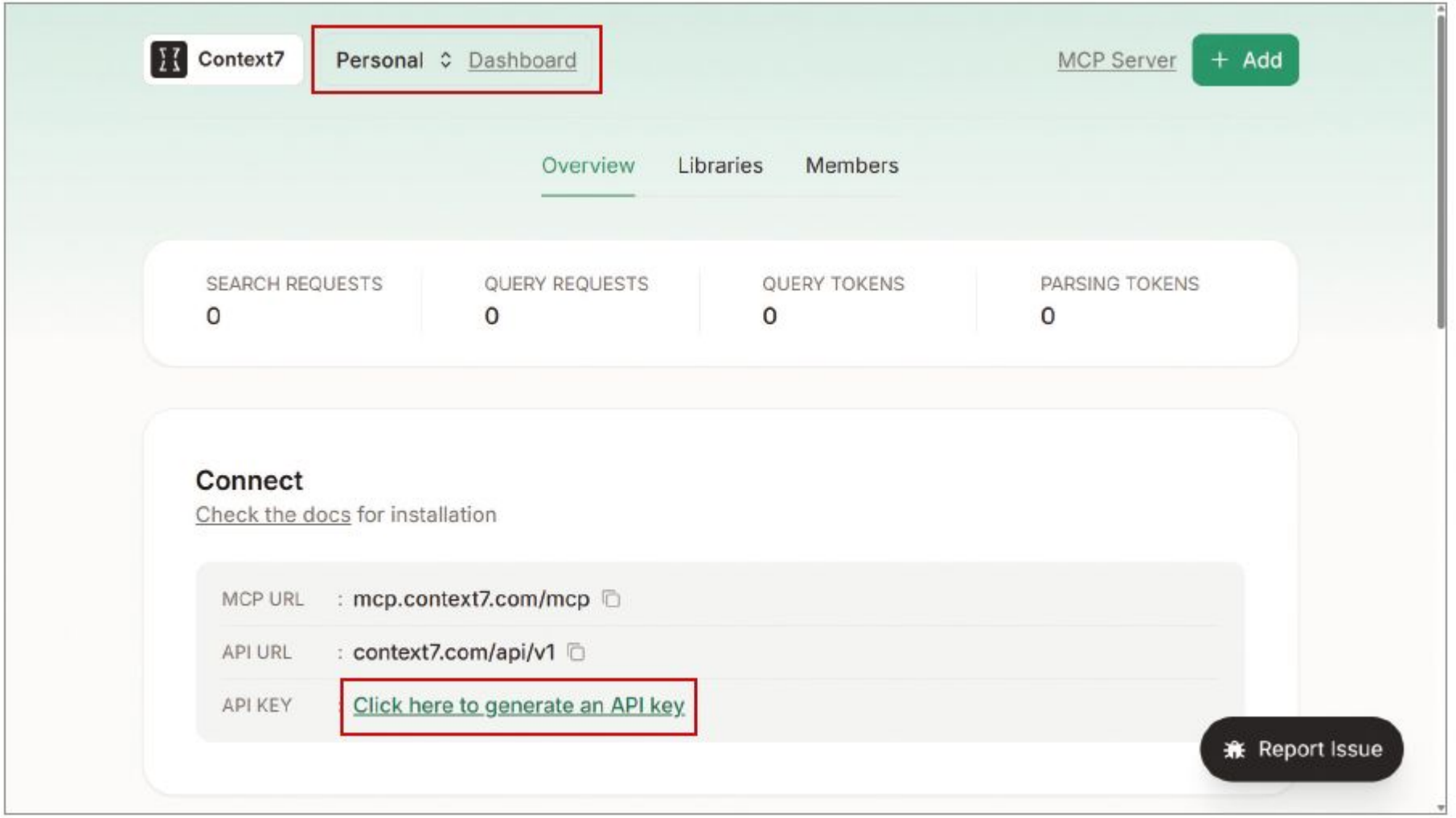
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(4)

02 Context7이 제시하는 절차에 따라 가입 - 여기서는 구글 계정을 연동



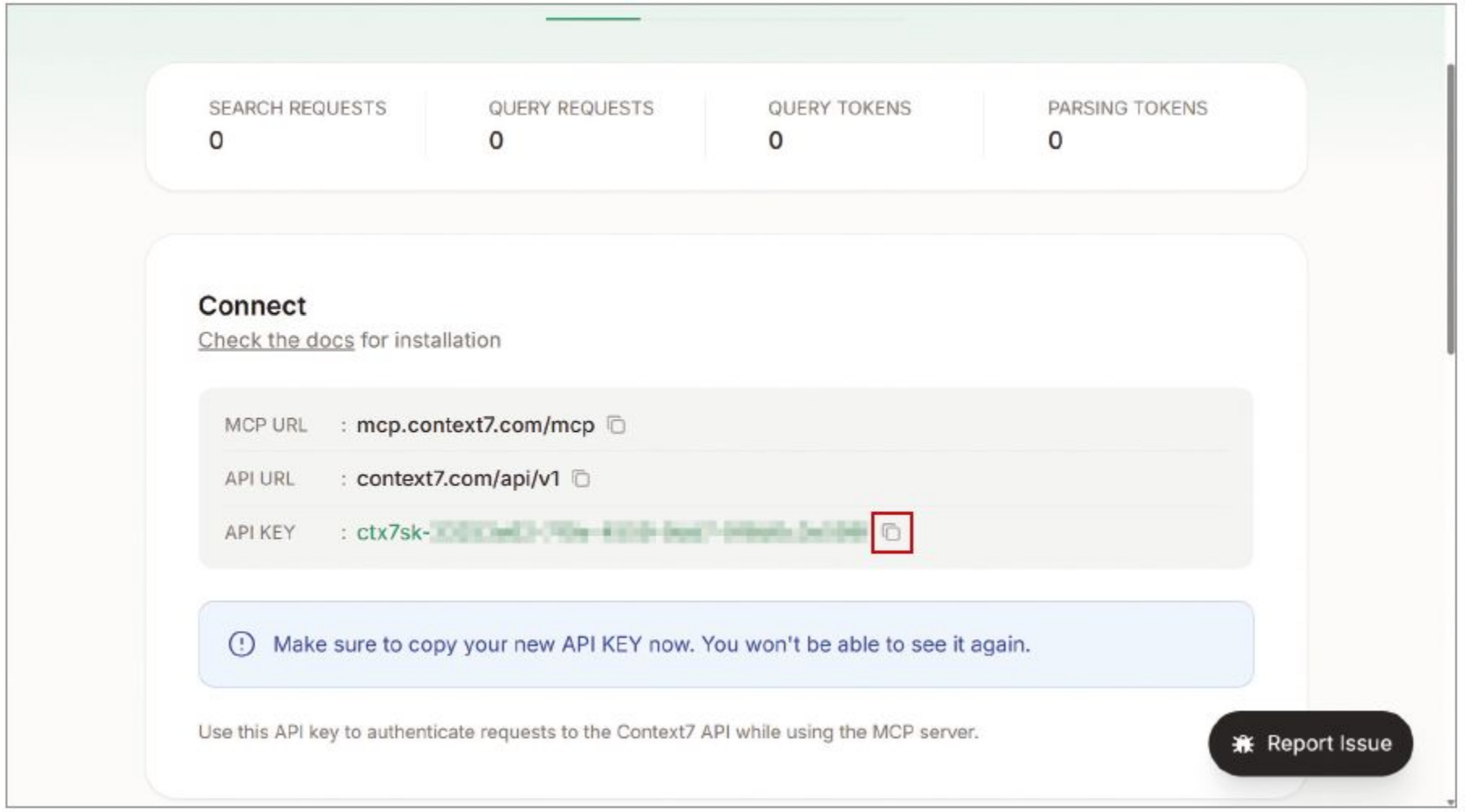
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(5)

03 로그인하면 [Personal Dashboard] 화면으로 이동
'API KEY' 항목의 'Click here to go generate an API key'를 클릭해 Context7 API 키를 발급



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(6)

04 '복사(' 아이콘을 클릭해 API 키를 복사하고 보관



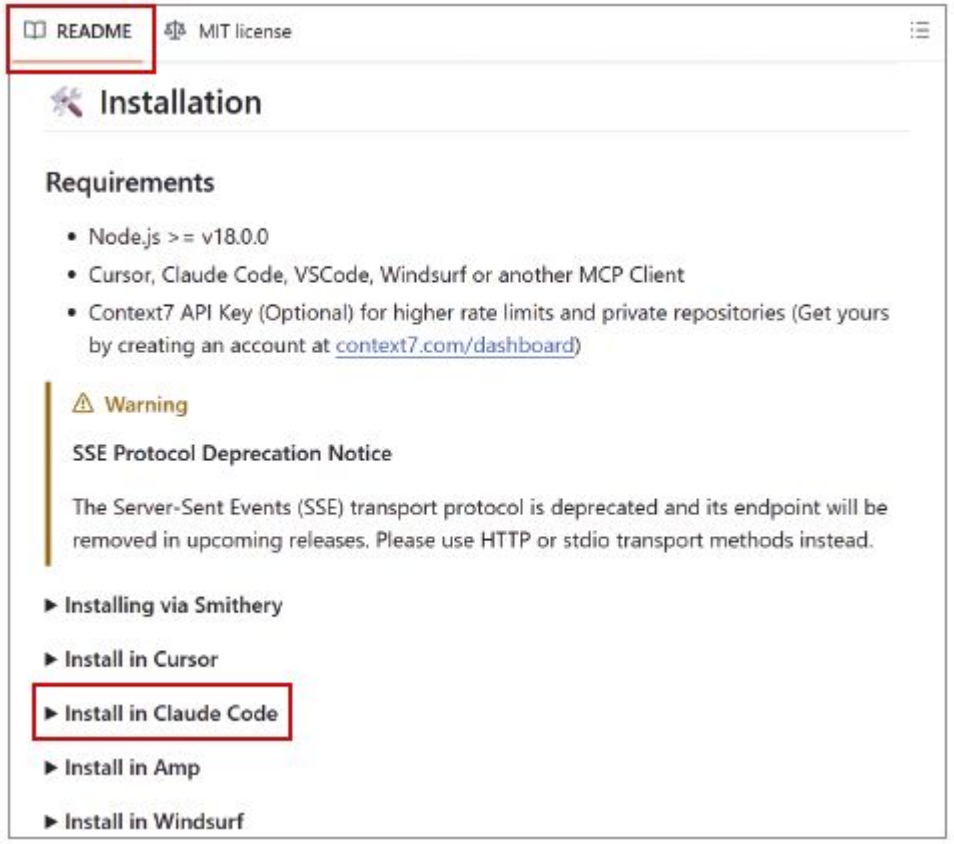
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(7)

Context7 MCP 설치하기

01 웹 브라우저에서 구글(google.com)에 접속한 뒤 검색창에 'Context7 mcp'을 입력하고 [Enter]
검색 결과에서 깃허브(GitHub)가 제공하는 문서를 클릭

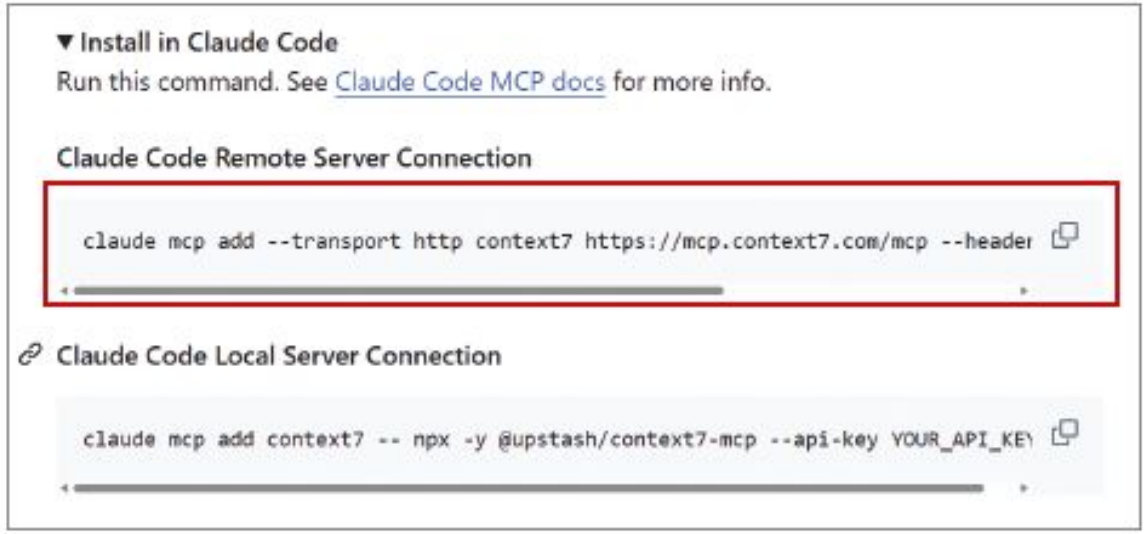
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(8)

02 Context7 MCP에 대해 소개 글 [README] 탭을 찾고 스크롤을 내리면 'Installation' 항목에서 설치 안내를 찾을 수 있음
[▶Install in Claude Code] 토글을 클릭



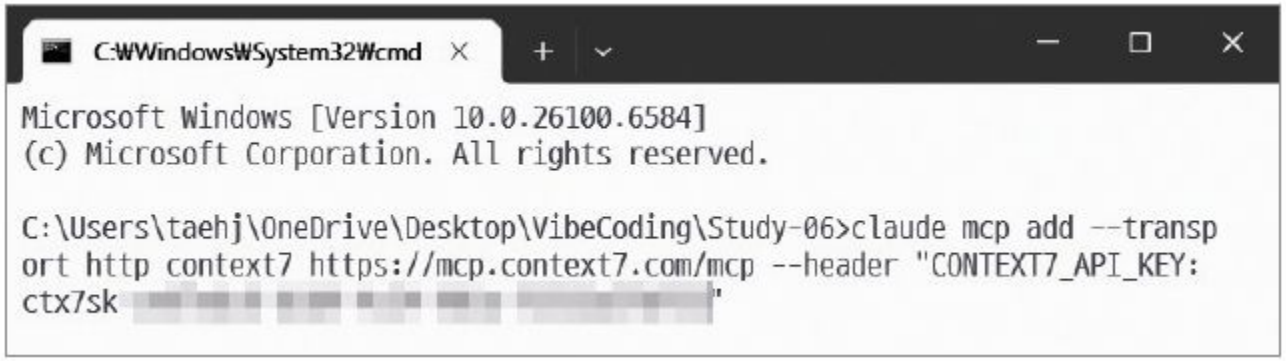
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(9)

03 클로드 코드에 MCP를 설치할 수 있는 명령어가 원격과 로컬 두 가지 방식으로 제시됨
여기서는 원격 MCP를 사용하기 위해 'Claude Code Remote Server Connection' 하위 명령어를 복사



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(10)

04 데스크톱의 'Study-06' 폴더에서 터미널을 열고 복사한 Context7 MCP 설치 명령어를 입력



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(11)

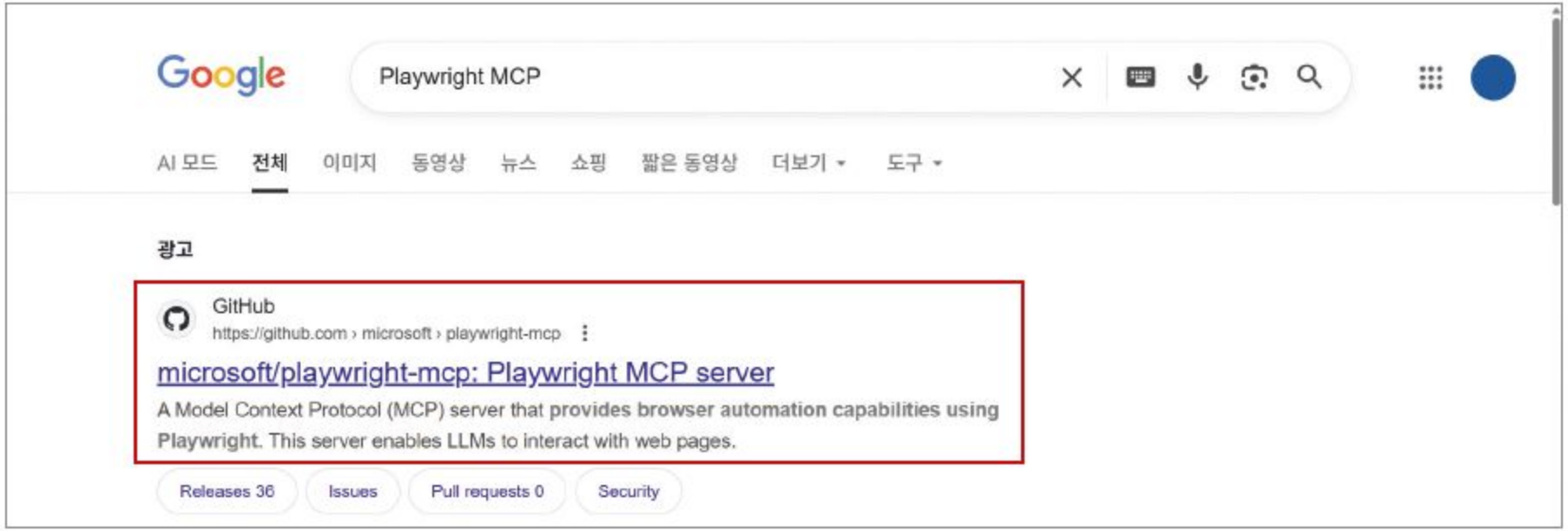
05 설치가 완료되면 'claude mcp list'를 입력해 Context7 MCP가 잘 설치되었는지 확인

```
C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude mcp list
Checking MCP server health...

notion: https://mcp.notion.com/mcp (HTTP) - ✓ Connected
sequential-thinking: cmd /c npx -y @modelcontextprotocol/server-sequential-
thinking@latest - ✓ Connected
context7: https://mcp.context7.com/mcp (HTTP) - ✓ Connected
```

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(12)

Playwright MCP 설치하기

01 Playwright의 MCP 서버주소를 얻기 위해 구글의 검색창에 'Playwright MCP'를 입력하고, 검색 결과에서 깃허브(GitHub)가 제공하는 문서를 클릭



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(13)

02 [README] 탭의 'Getting started' 항목을 찾고, [Claude Code] 토글을 클릭
Playwright MCP의 설치 명령어를 볼 수 있음

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(14)

```
claude mcp add    playwright    npx    @playwright/mcp@latest
                  ❷             ❸      ❹
❶
```

❶ 클로드 코드에 새로운 MCP 서버를 추가하는 명령어

❷ 설치할 MCP의 이름

❸ Node.js 환경에서 패키지를 바로 실행할 수 있게 해주는 명령어

❹ Playwright팀이 배포한 MCP 패키지 이름(playwright/mcp)과 버전(@latest)

Playwright를 설치에 추가할 옵션

옵션	의미	설명
`-s local`	범위 지정 옵션	현재 프로젝트 폴더에만 적용된다는 뜻
`--`	구분자	명령어와 옵션 사이의 구분자. 공백 포함 필수
`cmd /c`	윈도우 명령어 실행 옵션	윈도우 환경에서 명령어를 실행하라는 뜻

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(15)

03 클로드 코드에 Playwright MCP를 설치

'Study-06' 폴더에서 터미널을 실행한 뒤 다음과 같이 명령어를 입력

```
claude mcp add playwright -s local -- cmd /c npx @playwright/mcp@latest
```

추가한 옵션

```
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp add playwright
 -s local -- cmd /c npx @playwright/mcp@latest
```

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(16)

04 설치가 완료되면 'claude mcp list' 명령어를 통해 확인

```
claude mcp list
```

```
C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp list
Checking MCP server health...

notion: https://mcp.notion.com/mcp (HTTP) - ✓ Connected
sequential-thinking: cmd /c npx -y @modelcontextprotocol/server-seq
uential-thinking@latest - ✓ Connected
playwright: cmd /c npx @playwright/mcp@latest - ✓ Connected
```

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(17)

- 쇼핑 리스트 애플리케이션 만들고 MCP로 테스트 자동화하기
  - 웹 개발에서 가장 중요한 작업이 테스트
    - 모든 버튼이 제대로 작동하는지, 입력 필드가 올바르게 검증되는지, 데이터가 정확히 저장되는지를 매번 수동으로 확인하는 일은 시간이 많이 걸리고 실수하기 쉬움
    - Playwright MCP를 사용하면 이런 반복적인 테스트 과정을 AI에게 맡겨 자동으로 수행
  - 간단한 쇼핑 리스트 애플리케이션을 만들기
    - 쇼핑 리스트에 상품을 추가하거나 삭제하고 구매한 상품에는 체크해서 삭선하는 기능

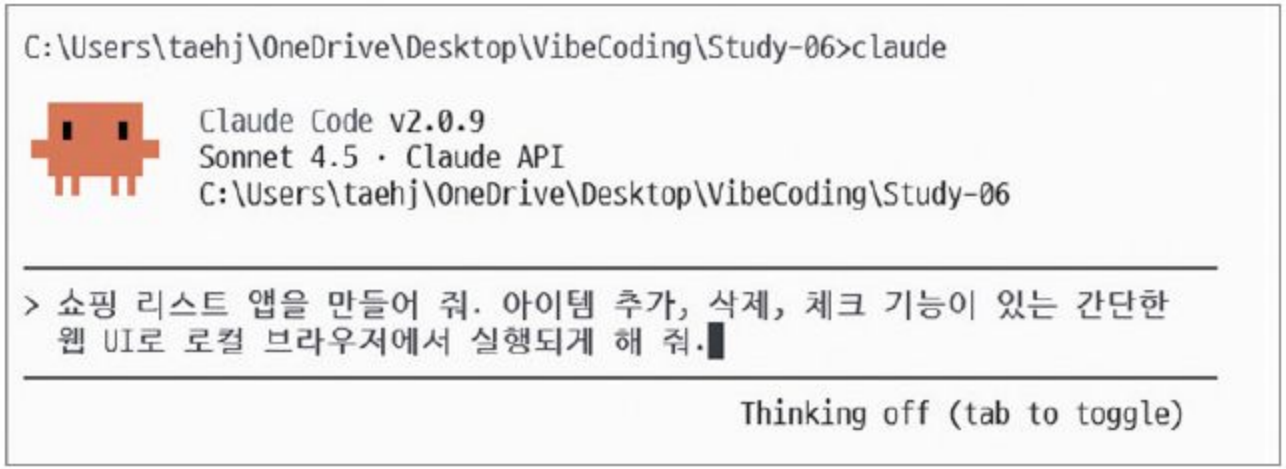
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(18)

쇼핑 리스트 앱 만들기

01 터미널에서 클로드 코드를 실행하고 다음과 같이 프롬프트를 입력

프롬프트

```
쇼핑 리스트 앱을 만들어 줘. 아이템 추가, 삭제, 체크 기능이 있는 간단한 웹 UI로 로컬 브라우저에서 실행되게 해 줘.
```



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(19)

02 작업이 완료되면 'Study-06' 폴더에 생성된 'shopping-list.html' 파일을 더블 클릭하여 실행

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(20)

03 ‘쇼핑 리스트’ 애플리케이션이 완성

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(21)

Playwright MCP로 자동 테스트 실행하기

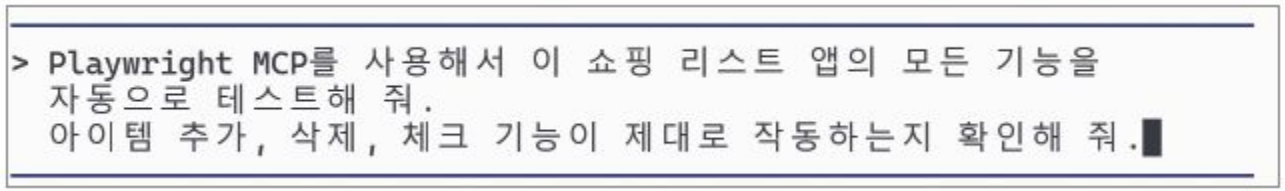
- Playwright MCP를 실행해 이 앱이 제대로 작동하는지, 자동 테스트를 실행
- '쇼핑 리스트 앱'의 주요 기능
  1. 아이템 추가,
  2. 체크 박스로 구매 완료 표시,
  3. 삭제 버튼으로 아이템 제거

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(22)

01 클로드 코드에 다음과 같이 입력해서 애플리케이션 테스트를 요청

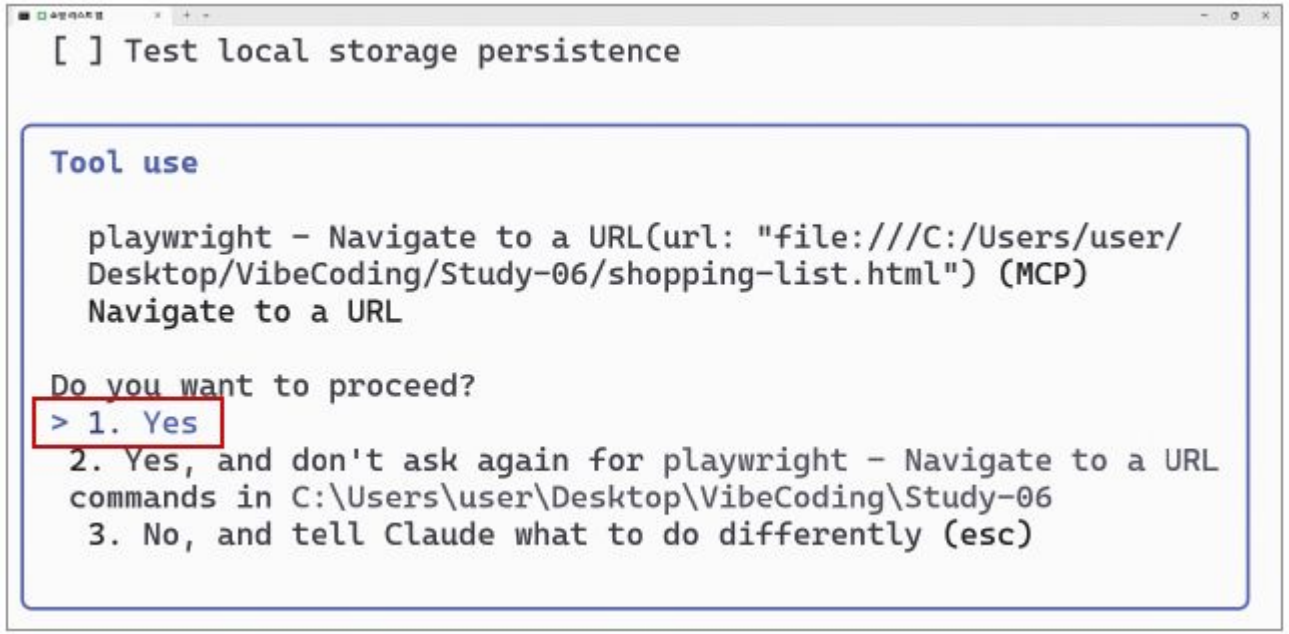
프롬프트

```
Playwright MCP를 사용해서 이 쇼핑 리스트 앱의 모든 기능을 자동으로 테스트해 줘. 아이템 추가, 삭제, 체크 기능이 제대로 작동하는지 확인해 줘.
```



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(23)

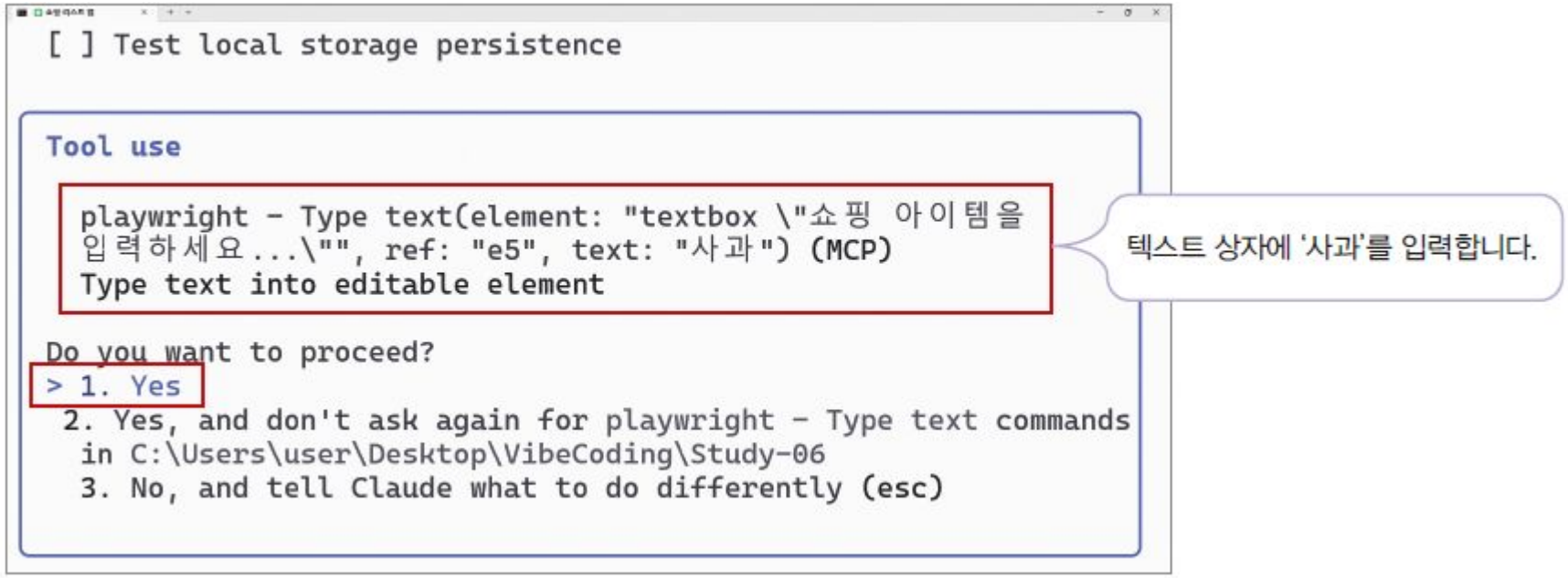
02 클로드 코드가 해당 MCP를 통해 HTML 파일을 열고 테스트 시작에 대한 승인 요청이 나타남
'1. Yes'를 눌러 계속 진행



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(24)

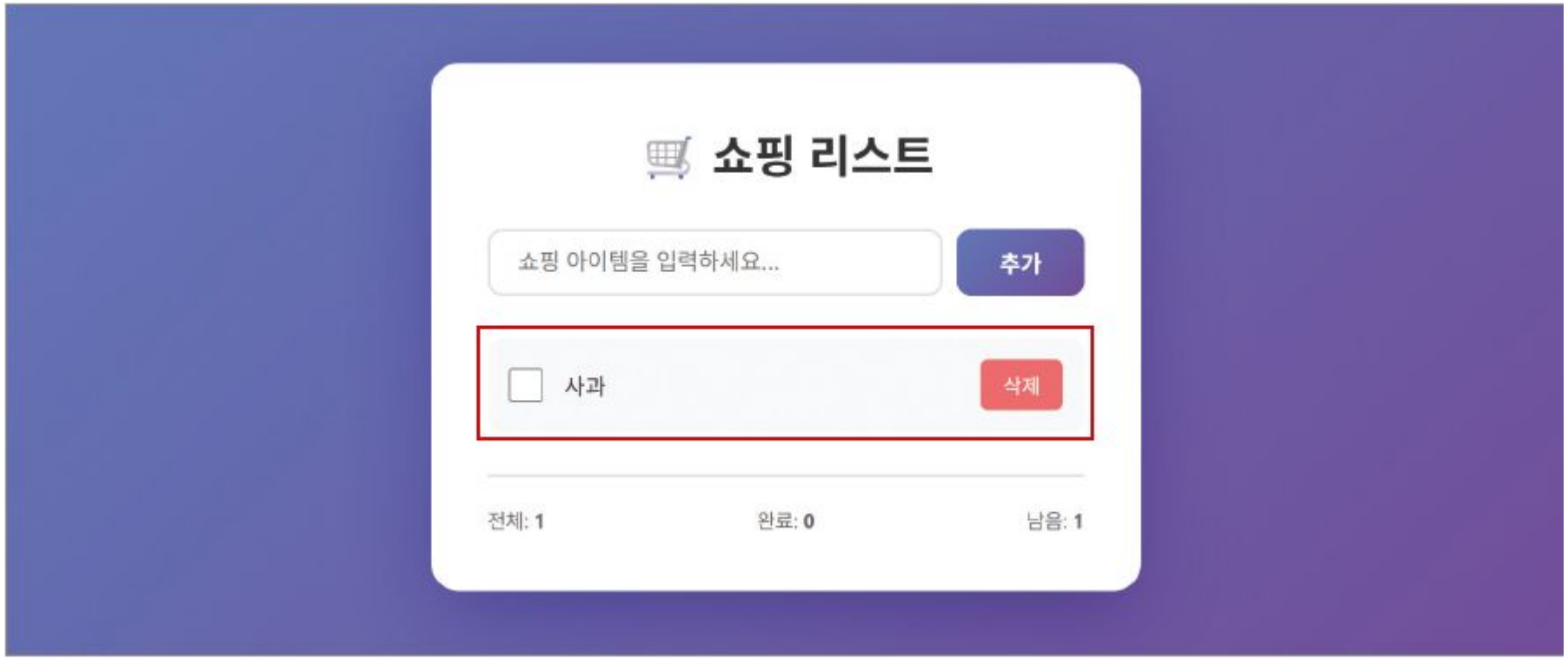
03 클로드 코드가 아이템 추가 기능이 잘 작동하는지 테스트하기 위해, 입력창에 '사과'라는 텍스트를 자동으로 입력하려고 함

'1. Yes'를 선택해 실행



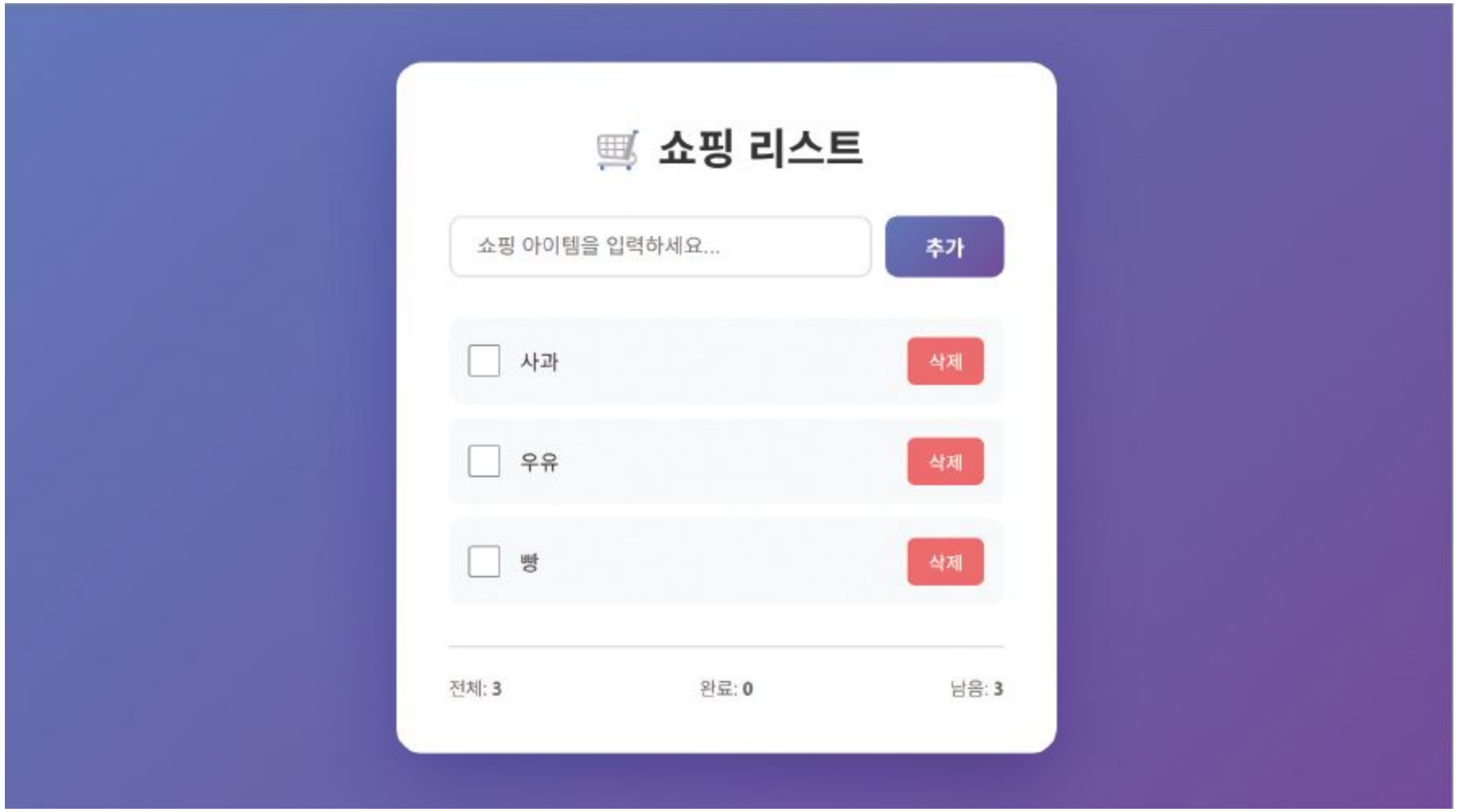
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(25)

04 브라우저가 자동으로 열리면서 쇼핑 리스트에 '사과'가 추가



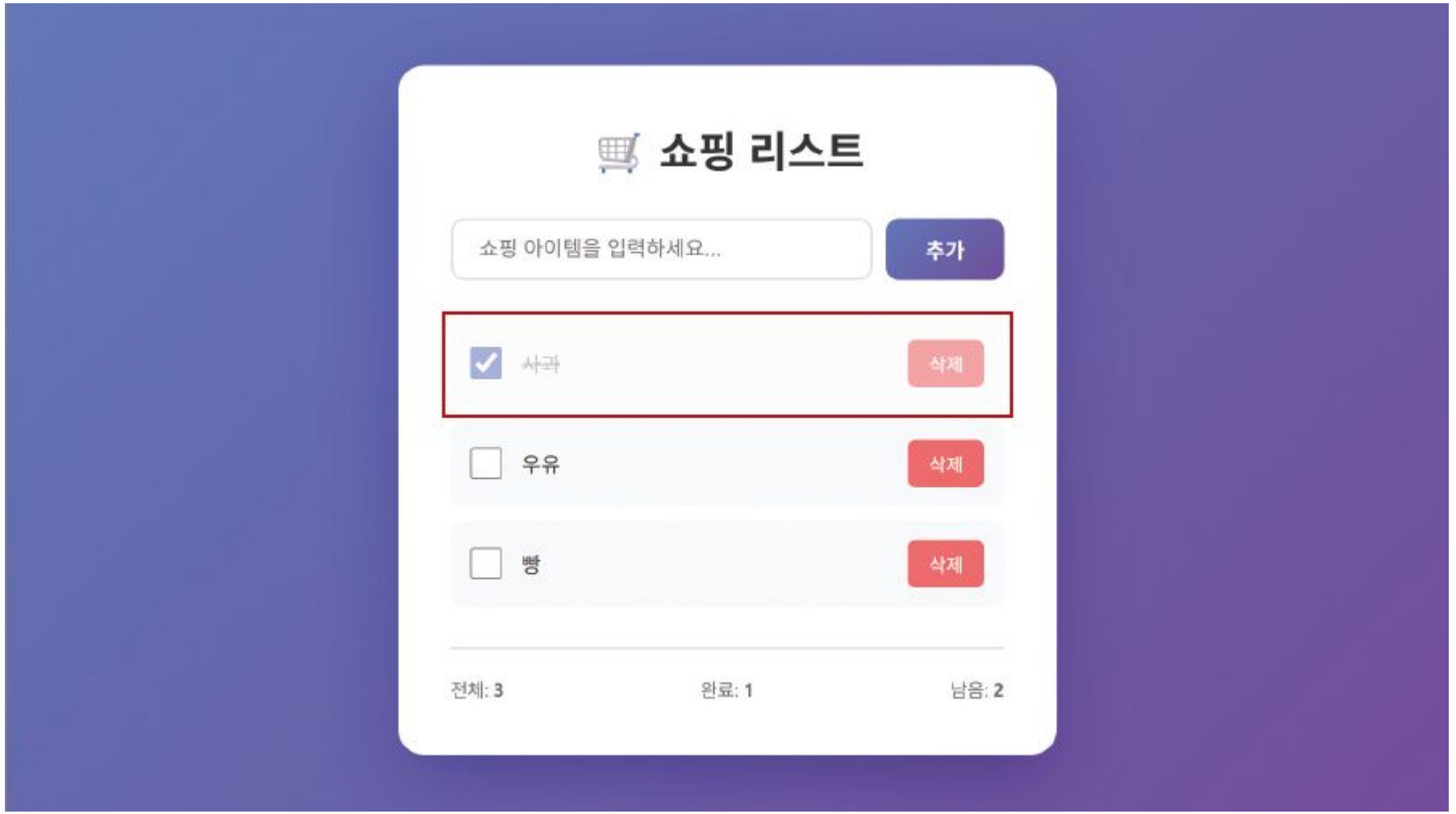
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(26)

05 클로드 코드의 Playwright MCP가 계속해서 '우유', '빵' 등 아이템을 추가해, 리스트가 제대로 작동하는지 테스트



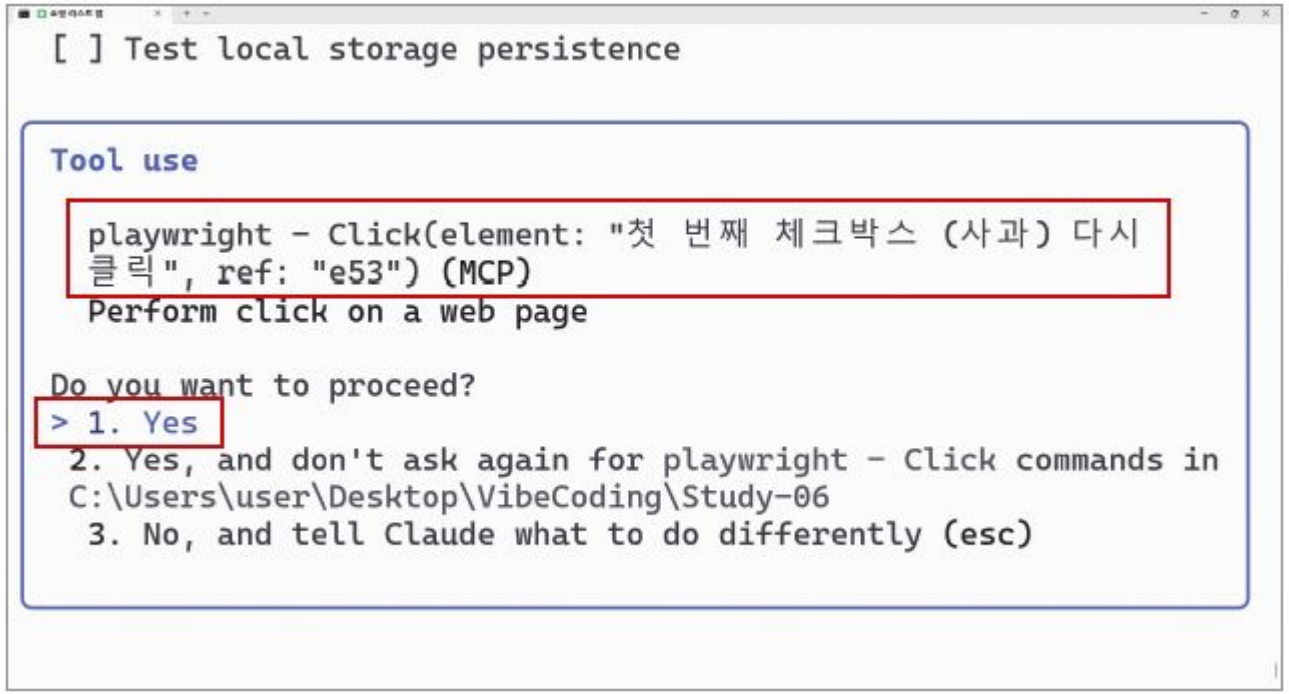
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(27)

06 구매한 아이템의 체크 박스를 클릭해서 삭선하는 기능을 점검
자동으로 체크 박스가 클릭되며 '사과' 아이템이 삭선되는 것을 확인



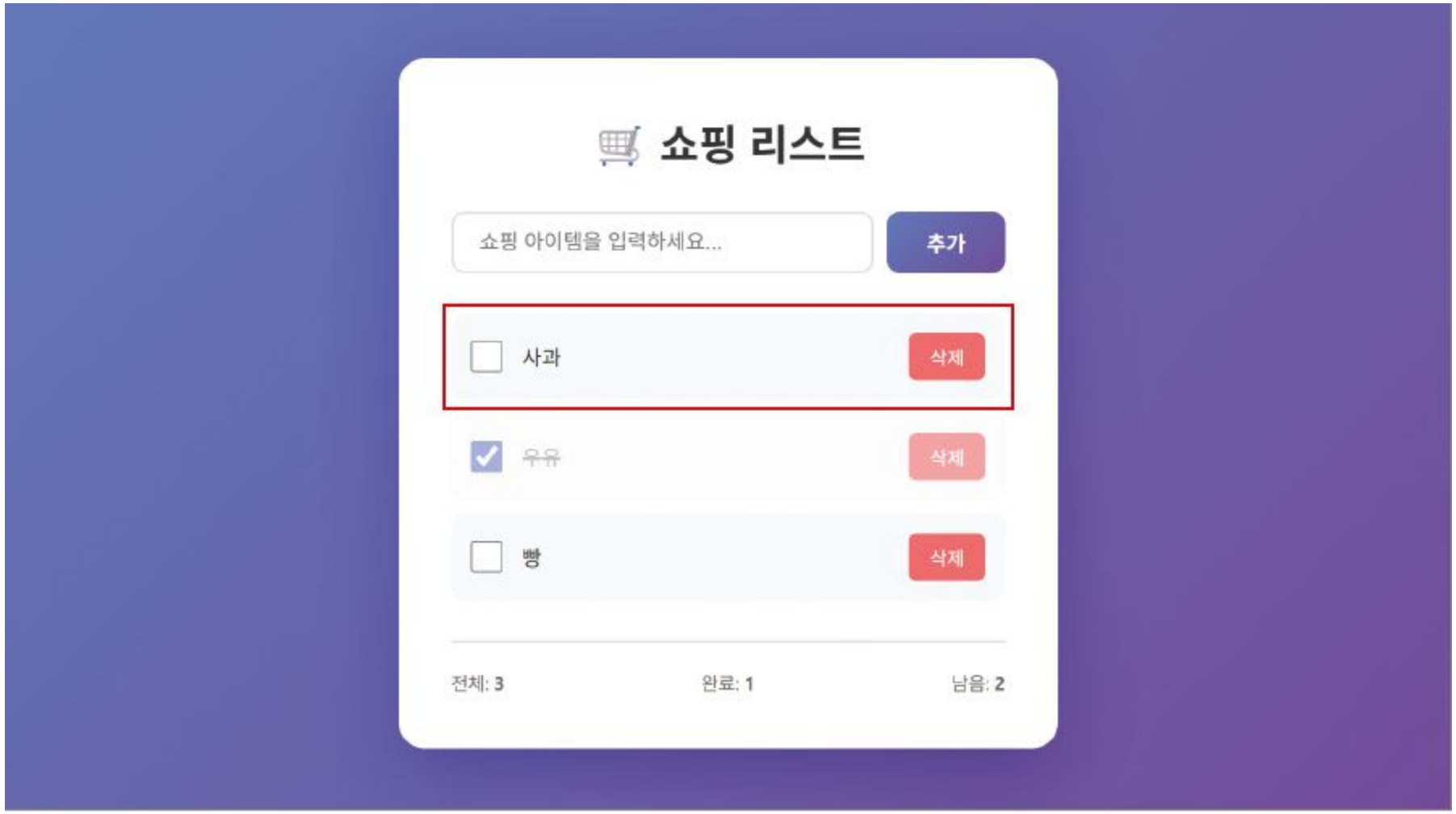
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(28)

07 삭선을 해제할 것인지 물으면 '1.Yes'를 선택해 체크 박스를 다시 클릭해서 삭선이 해제되는지 점검



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(29)

08 정상적으로 해제
체크 박스 기능 테스트 완료



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(30)

09 아이템 삭제 기능이 제대로 작동하는지 확인
쇼핑 리스트에서 '우유'와 '빵'이 자동으로 삭제

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(31)

10 Playwright MCP가 진행한 테스트를 기록하기 위해, 모든 테스트가 완료되면 현재의 화면을 스크린샷으로 남길지 질문에서 '1. Yes'를 눌러 진행

화면 해설

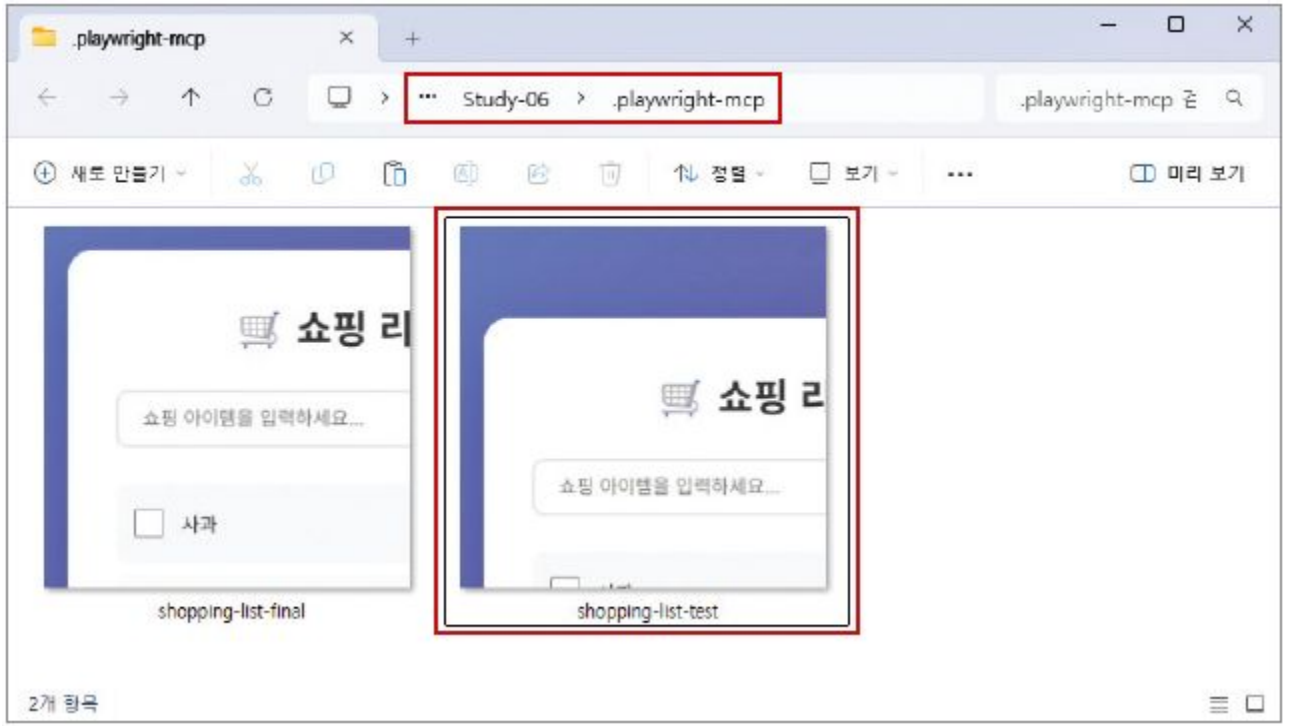
playwright - 스크린샷 촬영(파일명: shopping-list-test.png)

현재 페이지의 스크린샷을 촬영합니다.

스크린샷을 기반으로 동작을 수행할 수는 없습니다. 액션을 수행하려면 browser_snapshot 명령을 사용하세요.

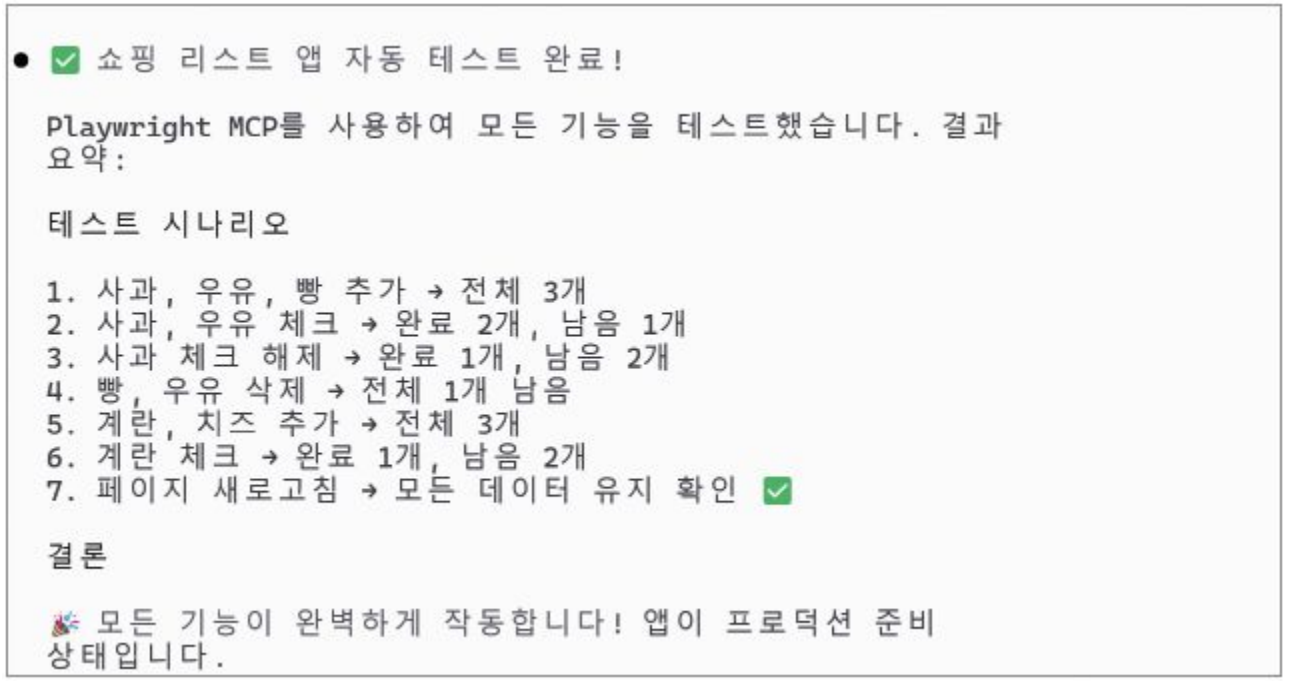
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(32)

11 'Study-06' 폴더에 테스트 폴더 '.playwright-mcp'가 생성되고, 폴더 안에는 'shopping-list-test.png' 파일이 저장
이 기능은 테스트 결과를 기록하고 실제 화면을 보고할 때 유용함



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(33)

12 모든 테스트가 정상적으로 진행되었음이 보고됨



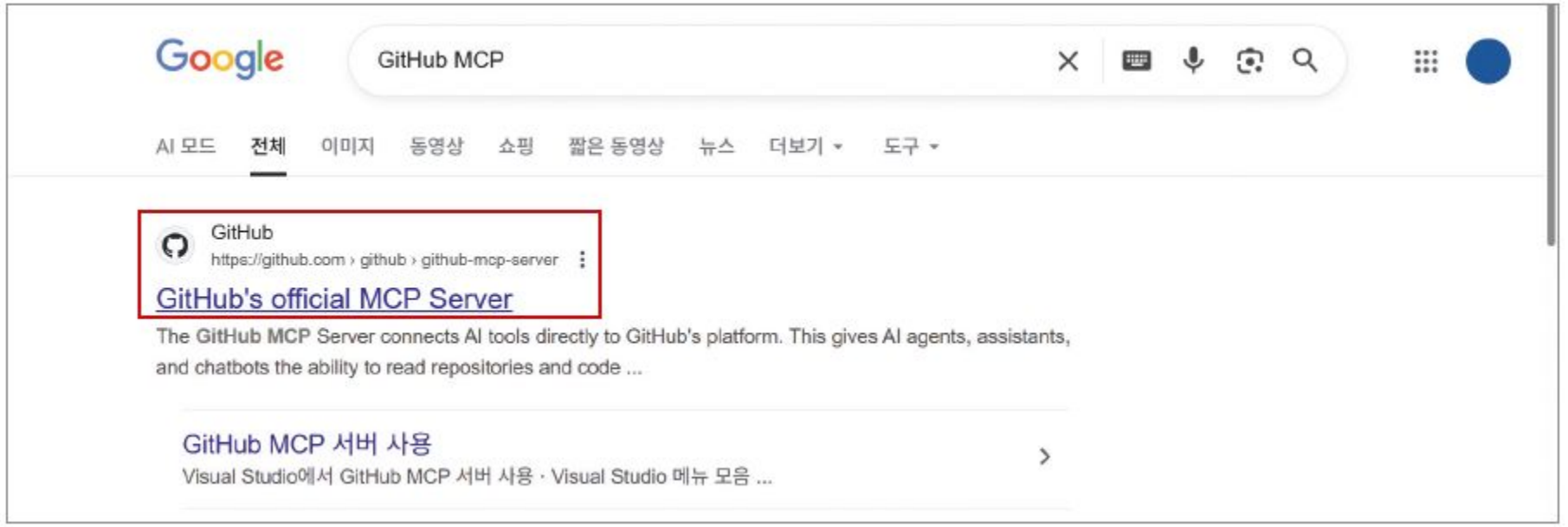
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(34)

- 깃허브 MCP로 프로젝트 영구 보관하기
  - 깃허브(GitHub)는 전 세계 개발자가 사용하는 코드 버전 관리 플랫폼
    - 클로드 코드의 MCP 서버를 이용하면 로컬 환경에서 작성한 웹 애플리케이션을 깃허브에 자동으로 업로드
  - 일반적으로 코드를 수정할 때마다 직접 깃 명령어를 입력해 변경 내용을 업로드해야 하지만, 깃허브 MCP 서버를 연결하면 클로드 코드가 이 과정을 대신 처리
    - AI가 작성한 코드를 자동으로 깃허브 저장소에 업로드하고, 버전을 관리하는 시스템을 구현
  - 깃허브 MCP 서버 주소가 필요

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(35)

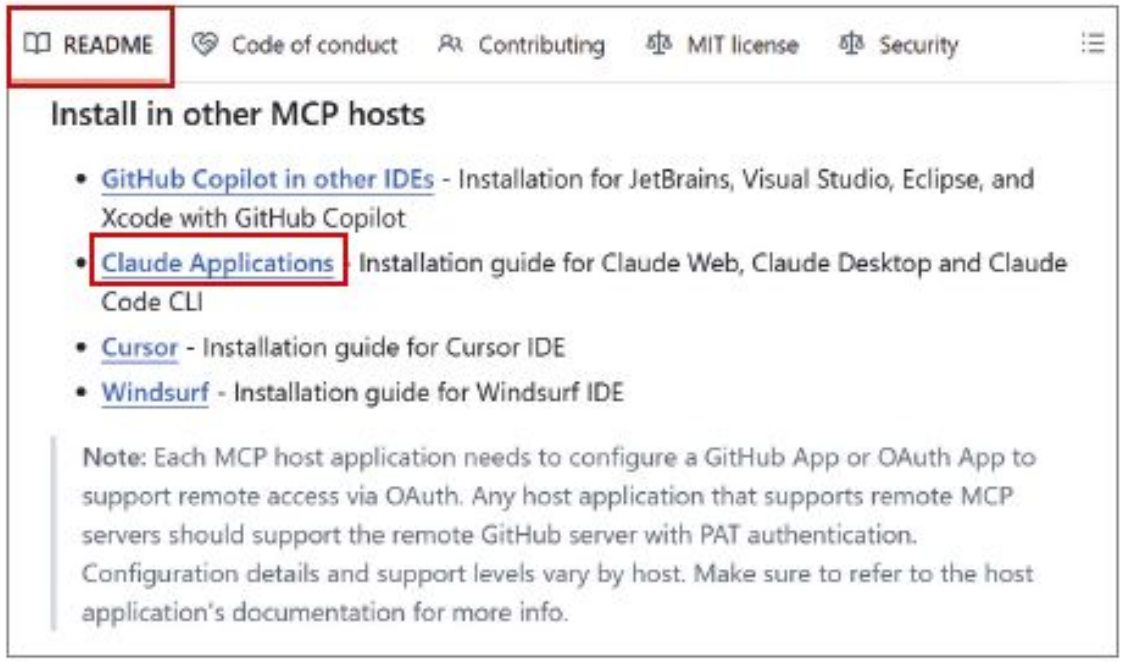
깃허브 MCP 설치 명령어 찾기

01 구글에 'GitHub MCP'를 검색한 뒤, 검색 결과 중 깃허브가 제공하는 문서를 클릭



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(36)

02 [READ ME] 탭의 'Install in other MCP hosts' 항목에서 'Claude Applications'를 클릭



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(37)

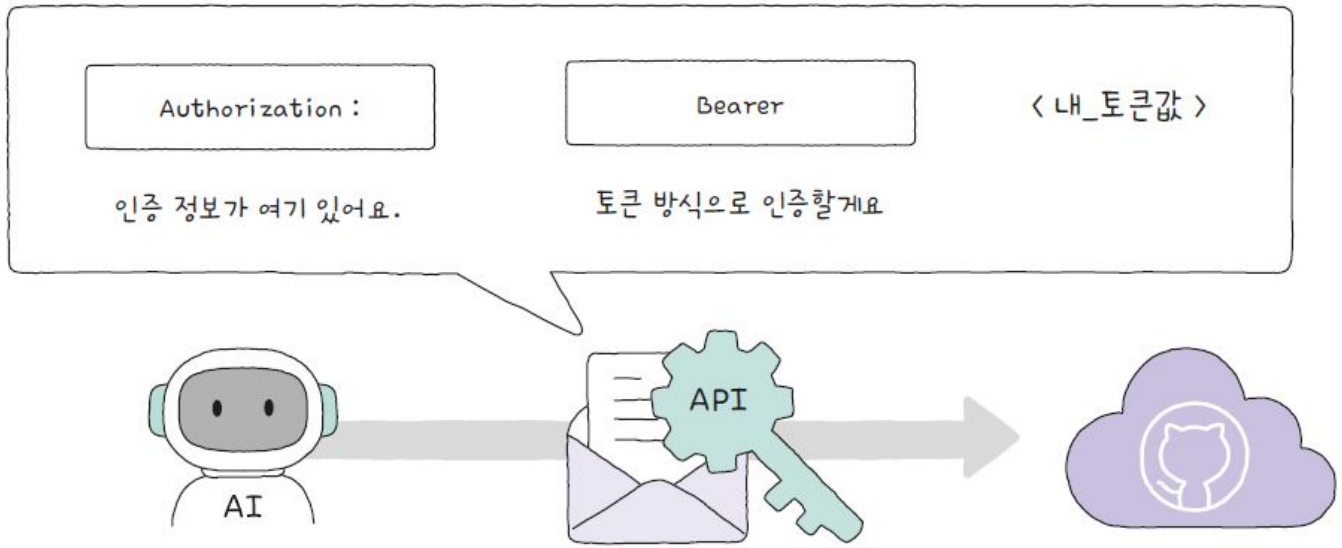
03 클로드 코드 CLI에서 원격 MCP 서버를 설치할 수 있는 명령어가 있음

- 명령어

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp -H
"Authorization: Bearer $(grep GITHUB_PAT .env | cut -d '=' -f2)"
```

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(38)

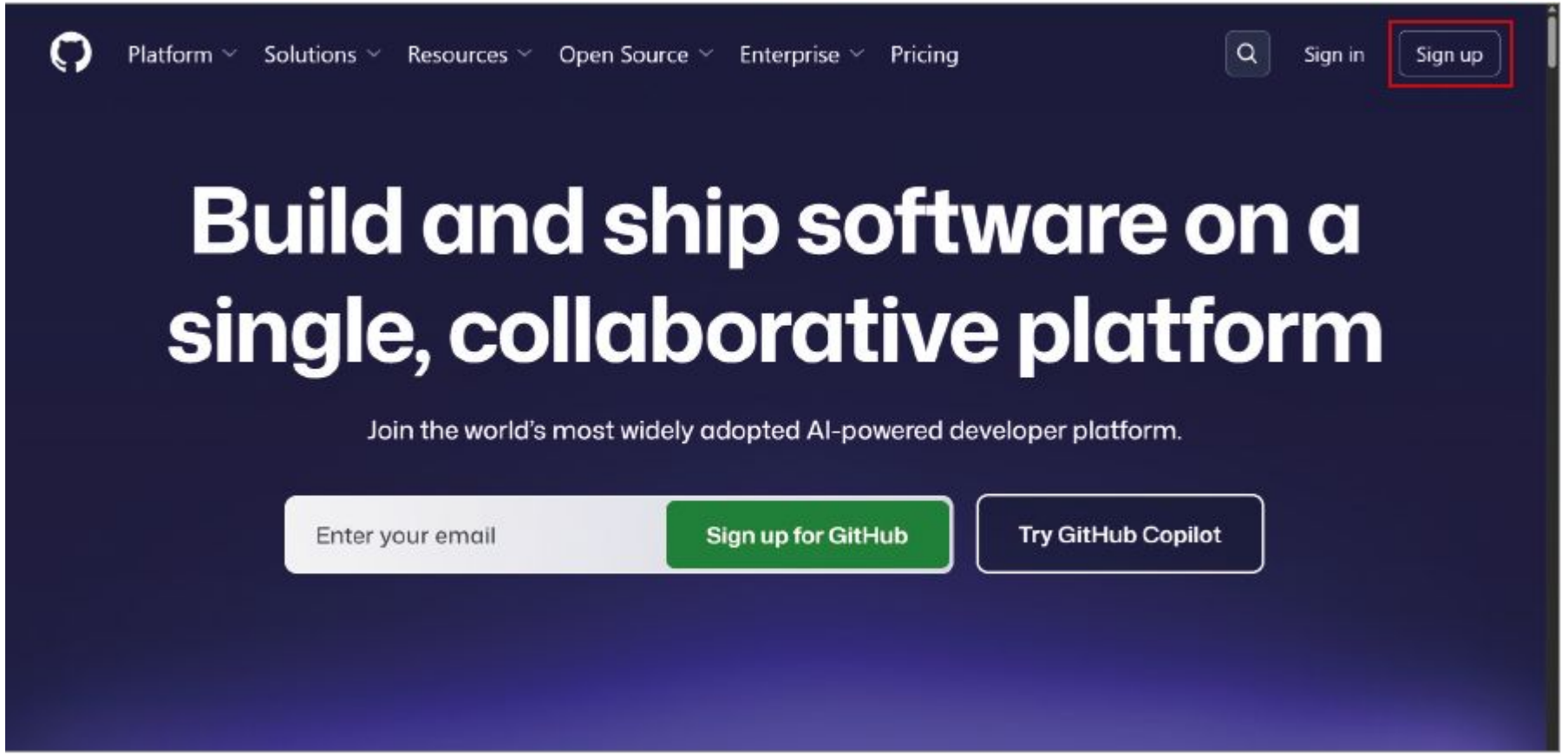
- 깃허브는 사용자 인증을 위해 API 토큰(token) 방식을 사용
- 깃허브 MCP 서버 설치 명령어에는 사용자 인증과 관련한 옵션이 포함
  - -H는 '--header'의 축약형으로, MCP 서버에 사용자 인증 데이터를 전달하기 위해 HTTP 요청에 헤더(header) 정보를 추가하라는 옵션
  - Authorization: Bearer가 깃허브가 인증을 위해 API에 접근할 때 사용하는 표준 형식. 'Authorization'은 인증 정보를 전달한다는 뜻이고, 'Bearer'는 인증 방식이 토큰 형식이라는 의미
  - $(grep GITHUB_PAT .env | ...)는 API 키가 들어갈 자리. '.env' 파일에서 'GITHUB_PAT' 변수를 포함한 줄을 찾아 명령어에 치환하라는 의미



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(39)

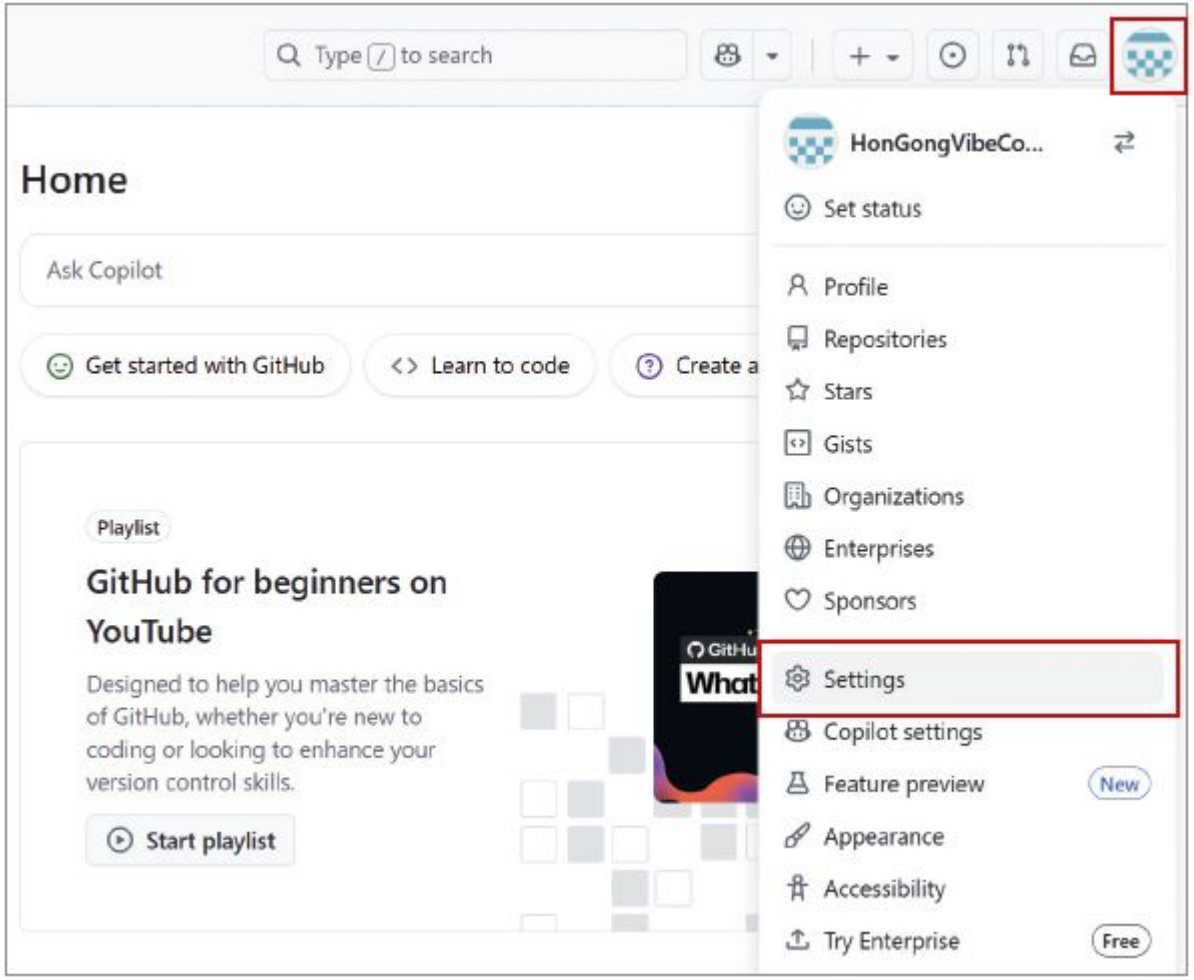
깃허브 API 토큰 발급받기

01 API 토큰을 얻기 위해 깃허브 홈페이지에 로그인하고 [Sign up] 버튼을 클릭해 회원가입을 진행 (URL https://www.github.com)



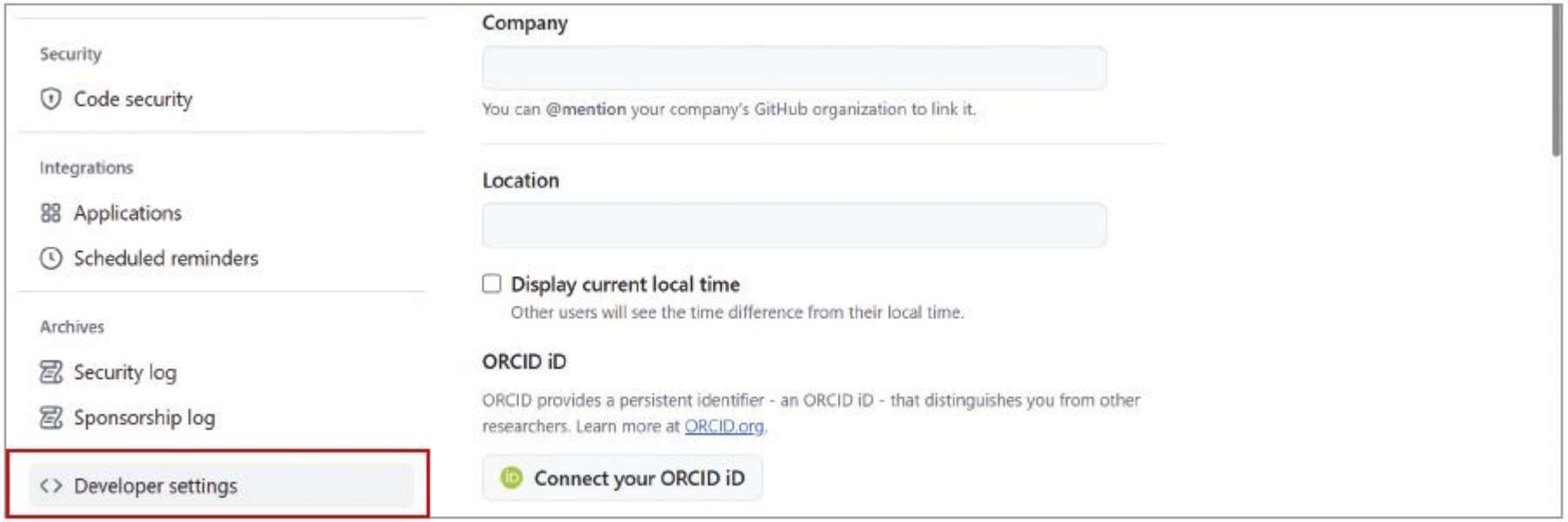
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(40)

02 깃허브에 로그인한 뒤, 오른쪽 상단의 프로필을 클릭하고 [Settings]를 클릭



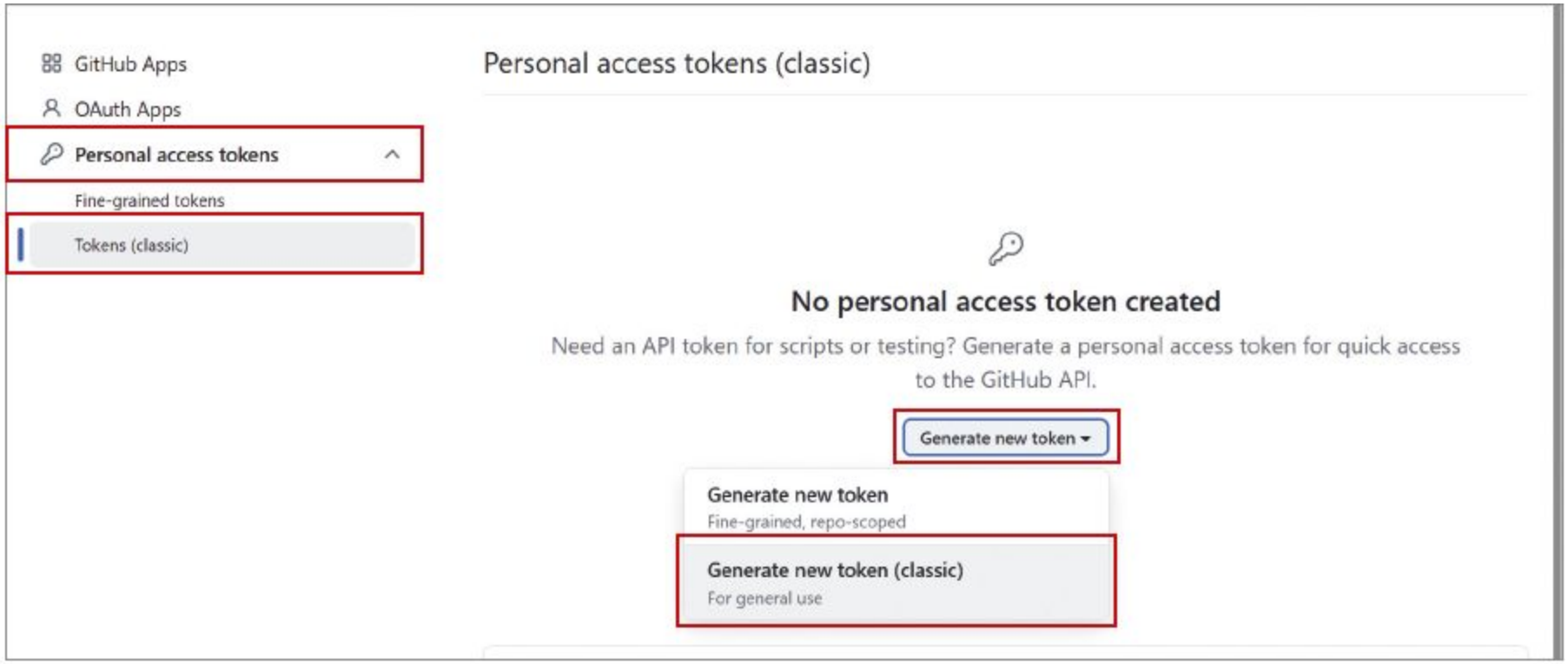
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(41)

03 왼쪽 사이드바에서 스크롤을 내려 [Developer settings] 메뉴를 클릭



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(42)

04 왼쪽 사이드바의 [GitHub Apps] 메뉴에서 [Personal access tokens]-[Tokens (classic)]을 차례로 클릭한 뒤, 화면 중앙에 [Generate new token] 드롭다운에서 [Generate new token (classic)]을 클릭



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(43)

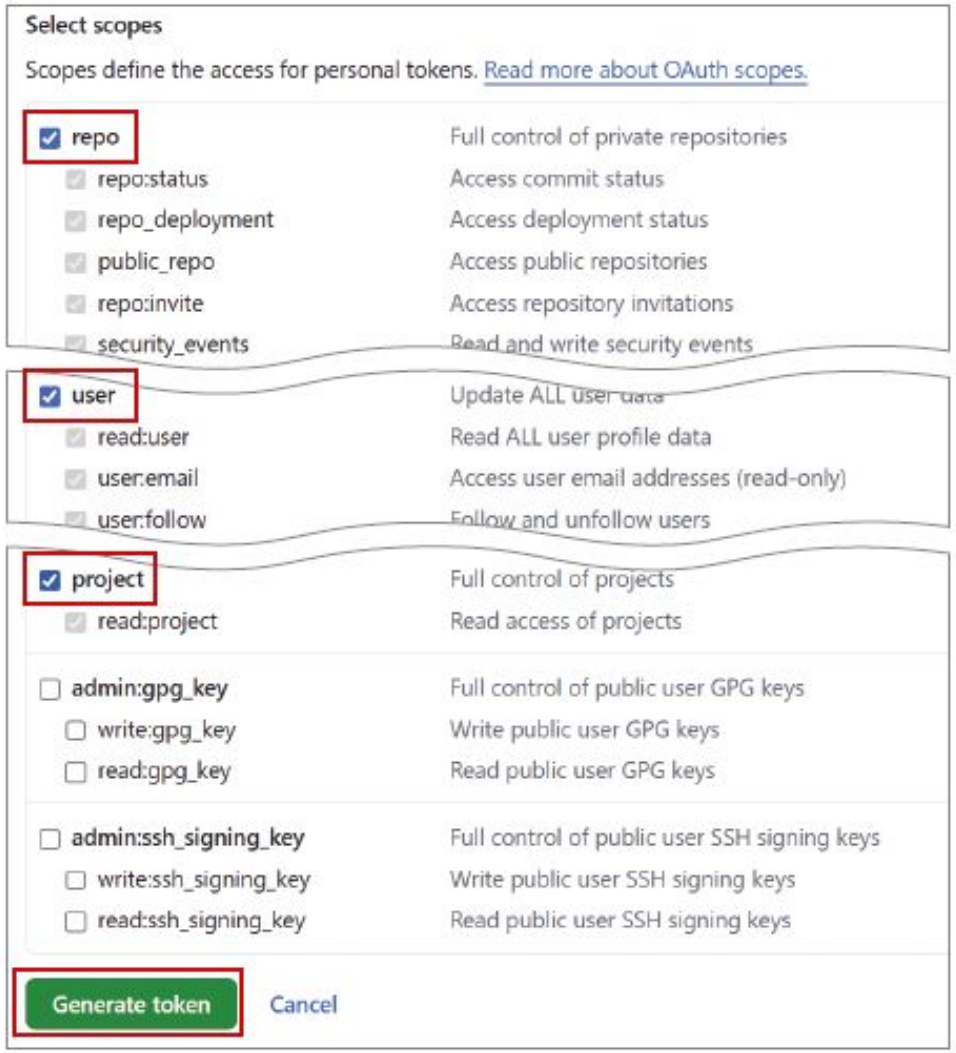
05 새 토큰을 만들 수 있음

'Note' 입력란에 'hongongvibe'라고 입력하고, [Expiration(유효 기간)]은 '30 days'로 지정

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(44)

06 'Select scopes'는 개인 액세스 토큰이 접근할 수 있는 권한 범위를 정의하는 옵션
다음 세 가지 옵션을 선택한 뒤, [Generate Token] 버튼을 클릭

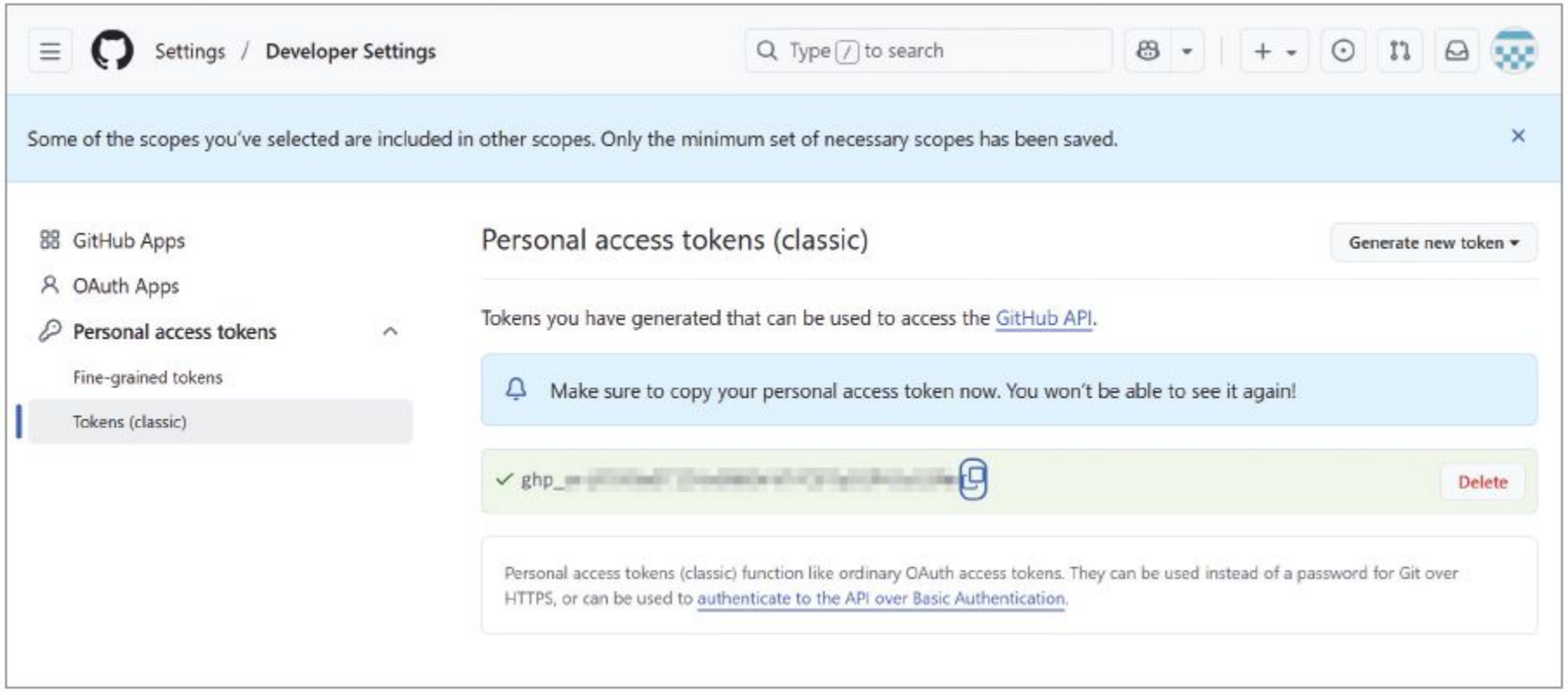
- repo: 개인 저장소 전체 제어 권한
- user: 사용자 데이터 전체 업데이트 권한
- project: 프로젝트 전체 제어 권한



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(45)

07 API 토큰이 발급

'복사(' 아이콘을 클릭하고 API 토큰을 저장

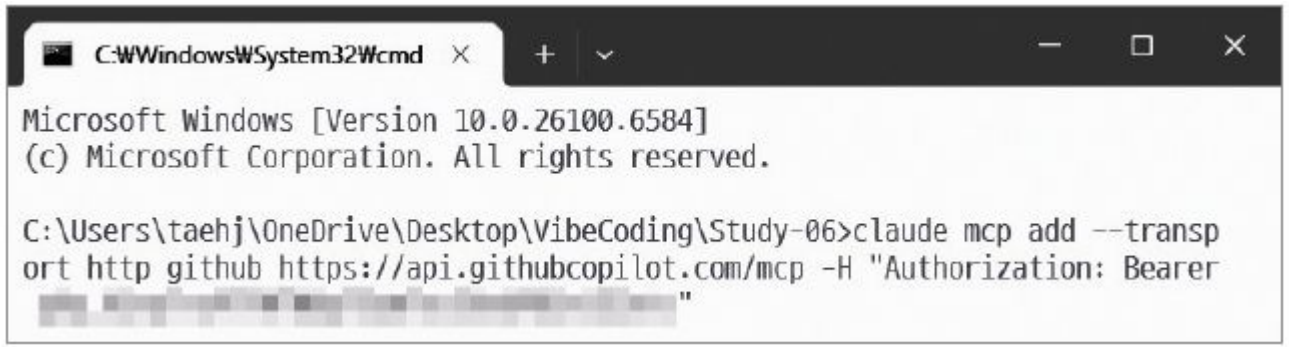


SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(46)

깃허브 MCP 설치 완료하기

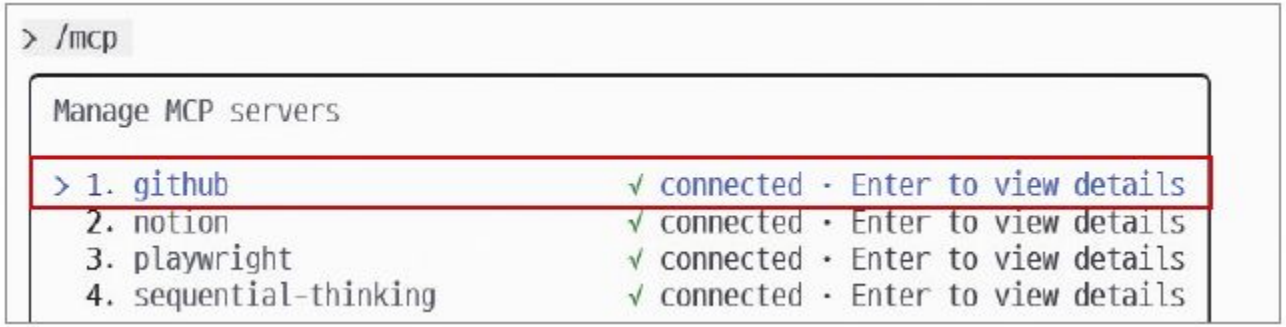
01 터미널에 다음과 같이 깃허브 MCP 설치 명령어를 입력
이때 <깃허브 API 토큰> 위치에 깃허브 API 토큰으로 대치

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp -H
"Authorization: Bearer <깃허브 API 키>"
```



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(47)

02 깃허브 MCP 서버 설치를 마치고 안내가 나오면 'claude' 명령어를 입력해 클로드 코드를 실행
'/mcp' 명령어를 통해 깃허브 MCP가 무사히 설치되었고 'connected'로 표시되는 것을 확인



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(48)

내 프로젝트 업로드하기

- 깃허브 MCP가 설치되면 이제 작업한 '쇼핑 리스트' 앱을 깃허브 저장소를 만들어 업로드
- 'Study-06' 폴더 위치에서 클로드 코드를 실행

01 다음과 같이 프롬프트를 입력

프롬프트

```
현재 폴더에 만들어진 쇼핑 리스트 앱을 깃허브에 저장하고 싶어. github mcp를 사용해서 shopping-list-app이라는 저장소를 만들고 업로드해 줘.
```

SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(49)

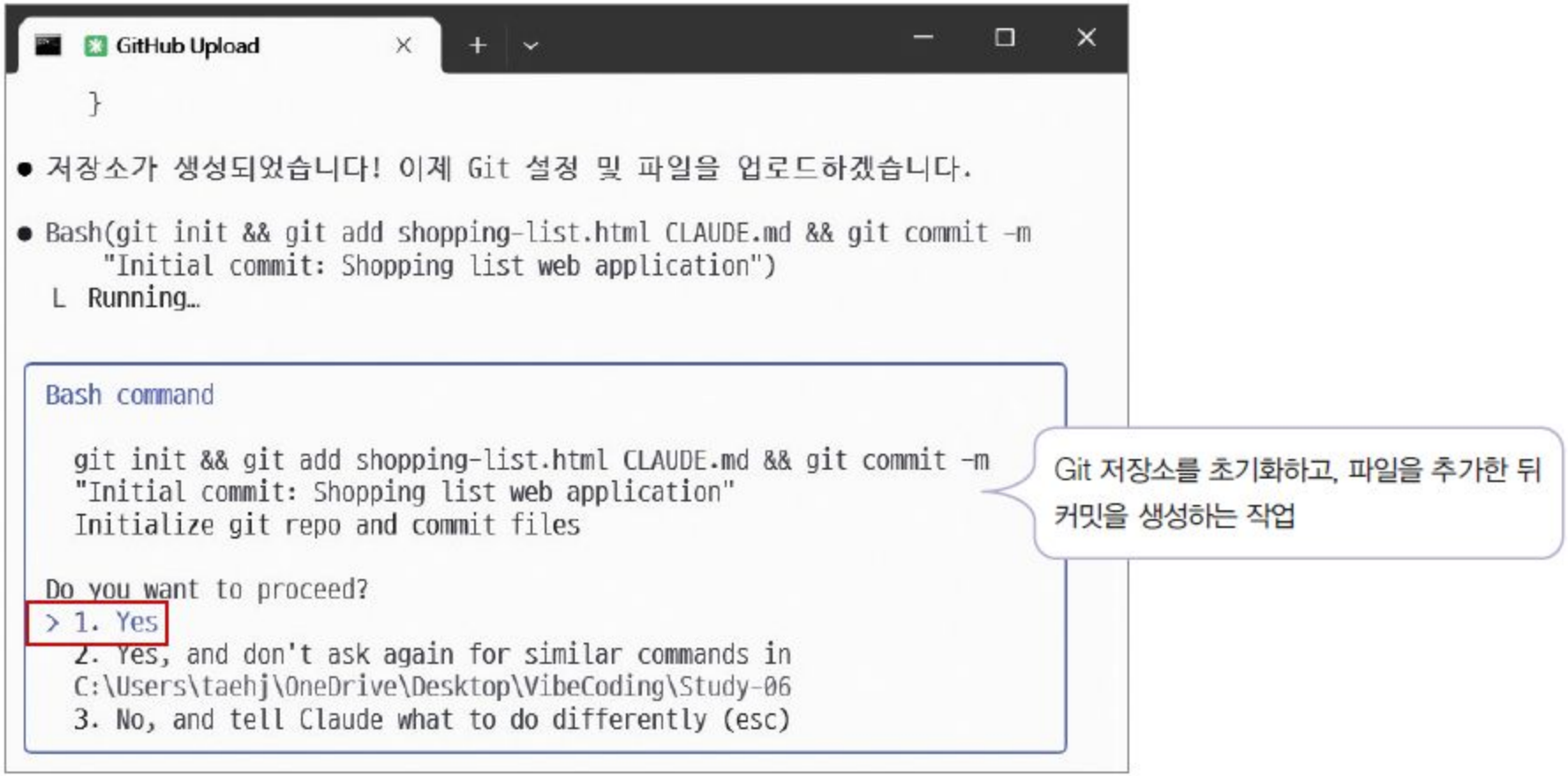
✚ 여기서 잠깐 **깃허브와 Git 명령어 이해하기**

- 깃허브는 코드를 온라인에 저장하고 관리하는 클라우드 버전 관리 플랫폼
  - 여러 사람과 함께 작업할 수도 있고, 코드의 변경 이력을 기록해서 언제든지 이전 버전으로 되돌릴 수도 있음
- 깃허브에서의 작업 흐름

1. 추가(add)
   - 깃허브에 프로젝트를 저장하기 위해서 먼저 깃허브 계정에 코드를 저장할 수 있는 원격 저장소(remote repository)를 만듦
   - 로드할 파일들을 선택해서 준비 공간(staging area)에 추가
2. 커밋(commit)
   - 준비 공간에 추가된 파일들이 확정되면 이를 전송 바로 전 단계인 로컬 저장소에 보관하는 과정을 의미
   - 변경 내용을 저장하는 단위를 의미하기도 함
   - 커밋은 코드의 현재 상태를 로컬 저장소에 저장하는 작업
3. 푸시(push)
   - 로컬 저장소에 커밋된 파일을 원격 저장소로 업로드하는 과정

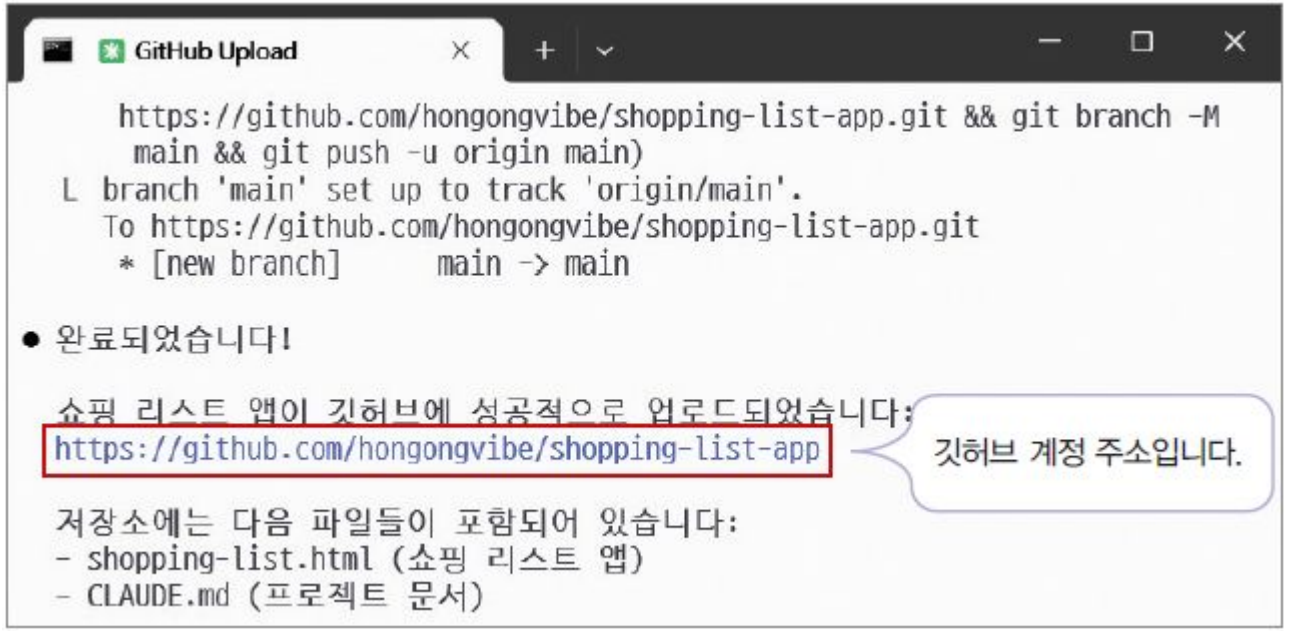
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(50)

02 클로드 코드가 저장소를 생성하고, 추가(add)와 커밋(commit) 등의 명령을 실행하기 위한 승인을 요청. '1. Yes'를 눌러 푸시(push)까지 계속 진행



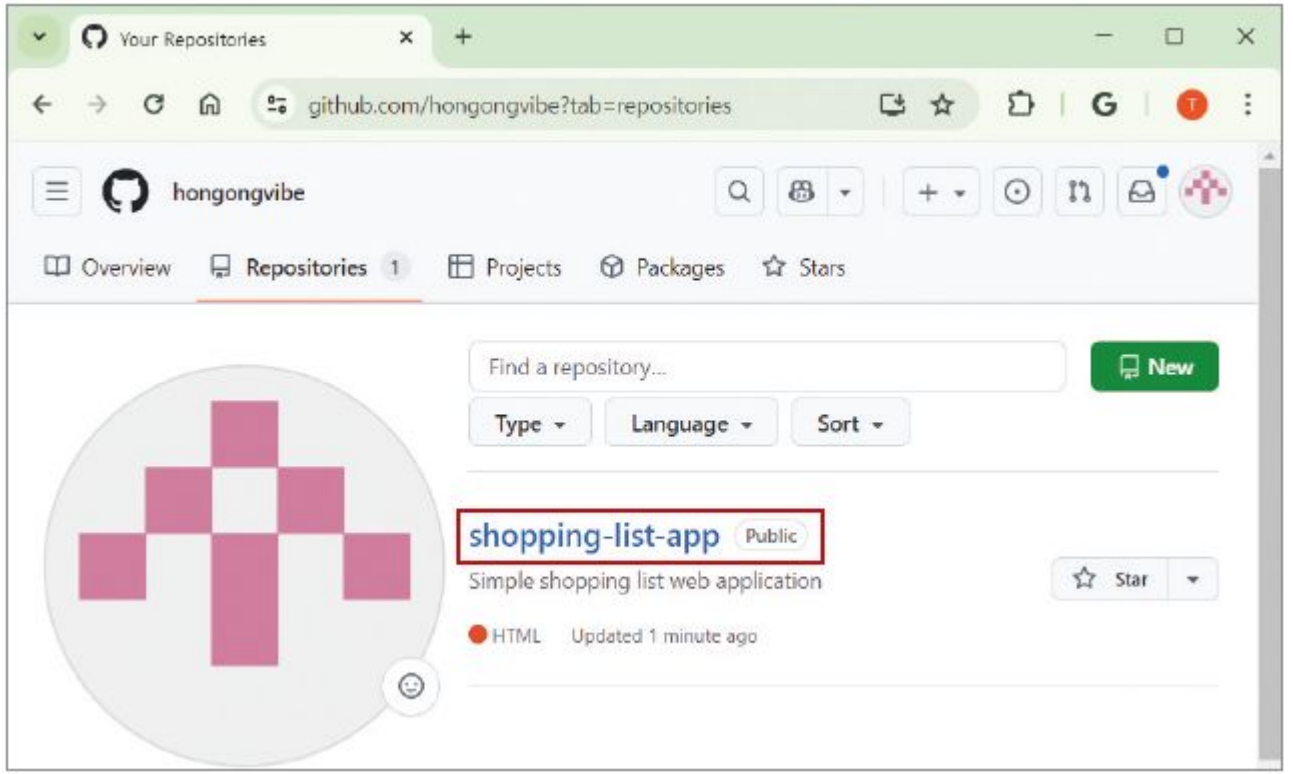
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(51)

03 쇼핑 리스트 앱이 깃허브에 업로드
URL로 이동해 깃허브에서 확인



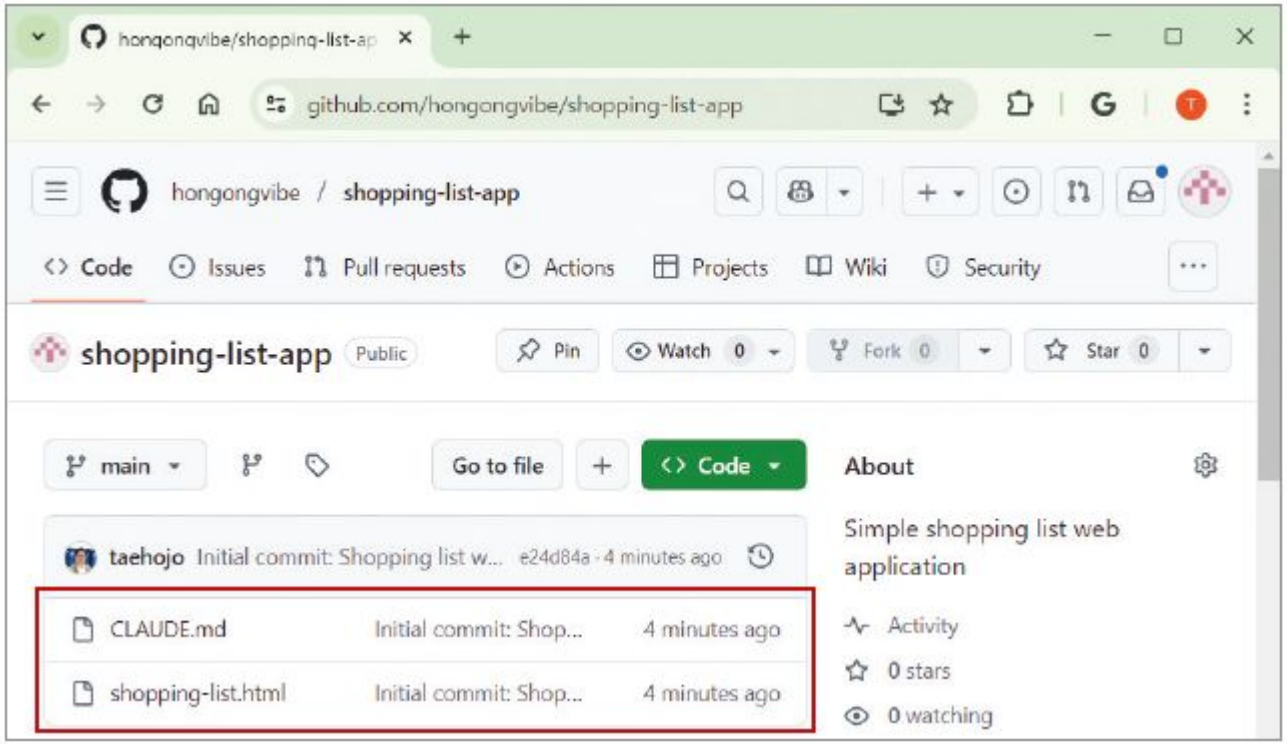
SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(52)

04 깃허브에 'shopping-list-app' 저장소가 새로 생성 - 클릭



SECTION 8-2 MCP로 구현하는 완전 자동화 개발 환경(53)

05 'CLAUDE.md'와 'shopping-list.html' 파일이 성공적으로 업로드



[마무리]

◦ 7가지 키워드로 정리하는 핵심 포인트

- **테스트**는 코드가 의도한 대로 실행되는지, 오류가 없는지를 확인하는 과정
- **버전 관리**는 코드의 변경 이력을 기록하고, 필요할 때 이전 상태로 되돌릴 수 있게 함
- **Context7**은 클로드 코드가 항상 최신 문서를 참고하도록 돕는 지식 연결 도구
- **Playwright**는 웹 사이트를 자동으로 테스트하고 UI를 검증하는 브라우저 제어 도구
- **추가(add) -** 깃허브에 업로드할 파일들을 선택해서 준비 공간에 추가하는 과정
- **커밋(commit) -** 원격 저장소에 추가된 파일이 확정되면 이를 전송 바로 전 단계인 로컬 저장소에 저장
- **푸시(push) -** 로컬 저장소에 커밋된 파일을 원격 저장소로 업로드하는 과정

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(1)

시작하기전에

- 데이터베이스(database, DB)는 많은 데이터를 체계적으로 저장하고 관리하는 시스템
  - 메모장이나 엑셀에 정보를 입력할 수는 있지만, 데이터가 많아질수록 검색이나 수정이 어려움
  - 데이터베이스는 이런 문제를 해결하기 위해 데이터를 표 형태로 구조화해 저장하고, 필요한 정보를 빠르게 찾아내거나 수정할 수 있게 도움
- 웹 애플리케이션에서는 사용자가 입력한 정보를 모두 데이터베이스에 저장
  - 앱을 종료했다 재접속해도 데이터가 유지되고, 여러 기기에서 같은 내용을 불러올 수 있음
  - 즉 데이터베이스는 앱의 기억 저장소

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(2)

◦ 나만의 웹페이지 배포하기

- 배포(deploy) - 내 애플리케이션을 외부의 인터넷에 연결된 제3의 서버에 올리는 과정
  - 애플리케이션을 배포하면 항상 켜져 있는 서버에서 앱을 대신 실행해 주기 때문에 언제, 어디서든 웹 주소, 즉 URL로 접속
- 08-2절에서 만든 쇼핑 리스트 앱을 많은 사용자가 이용할 수 있도록 인터넷에 배포

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(3)

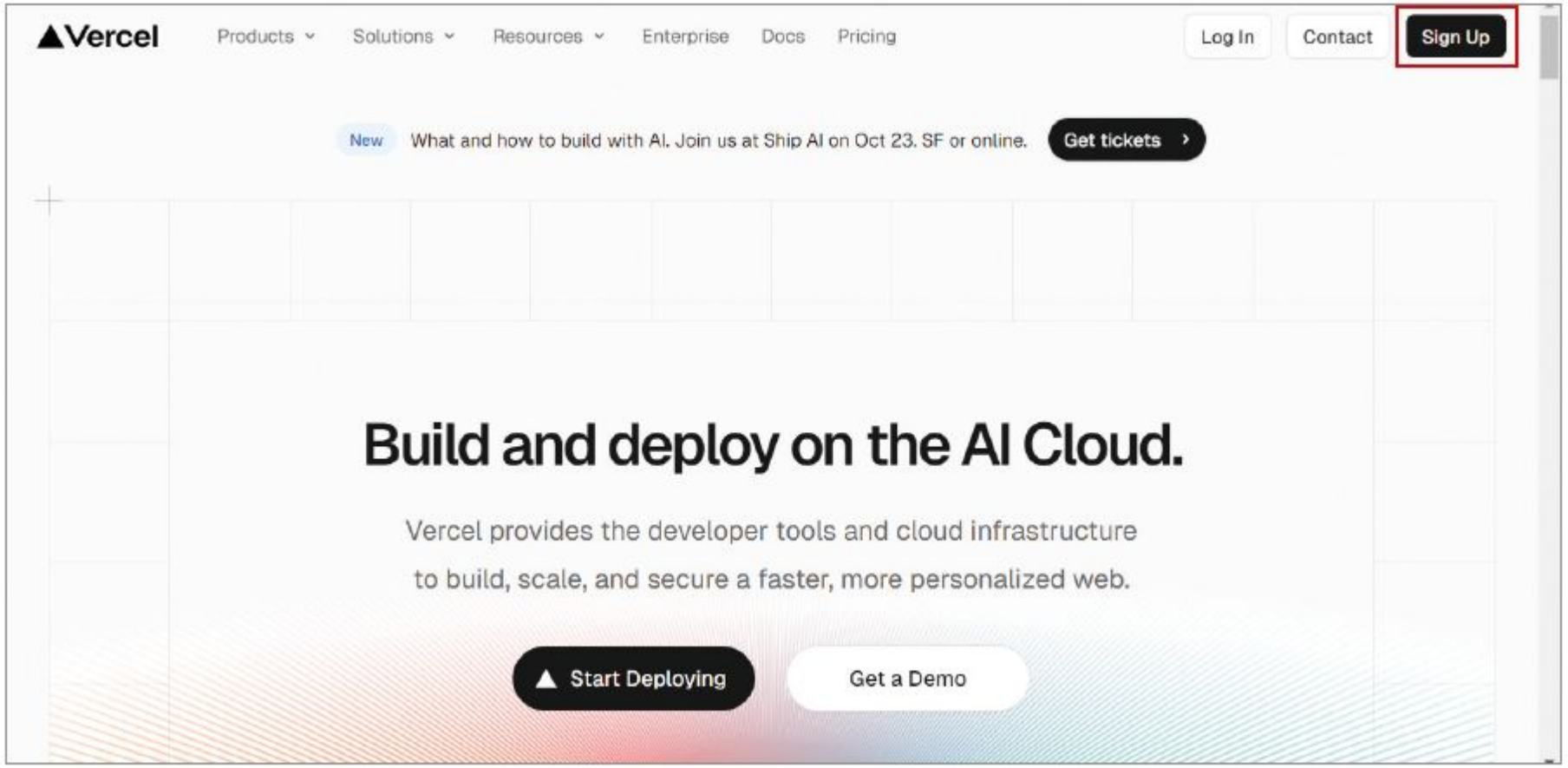
Vercel 계정 만들고 깃허브 연결하기

- Vercel - 작성한 코드를 깃허브에 올리면 자동으로 빌드(build), 배포해 주는 클라우드 플랫폼
- 간단한 웹 애플리케이션은 깃허브에 업로드하고, Vercel을 사용하면 무료로 배포까지 가능

1. 깃허브 저장소 준비
2. 로컬 환경에서 작성한 HTML 파일을 깃허브에 푸시
3. Vercel에 깃허브 인증 허가 후 배포할 저장소 선택
4. 즉시 URL 발급

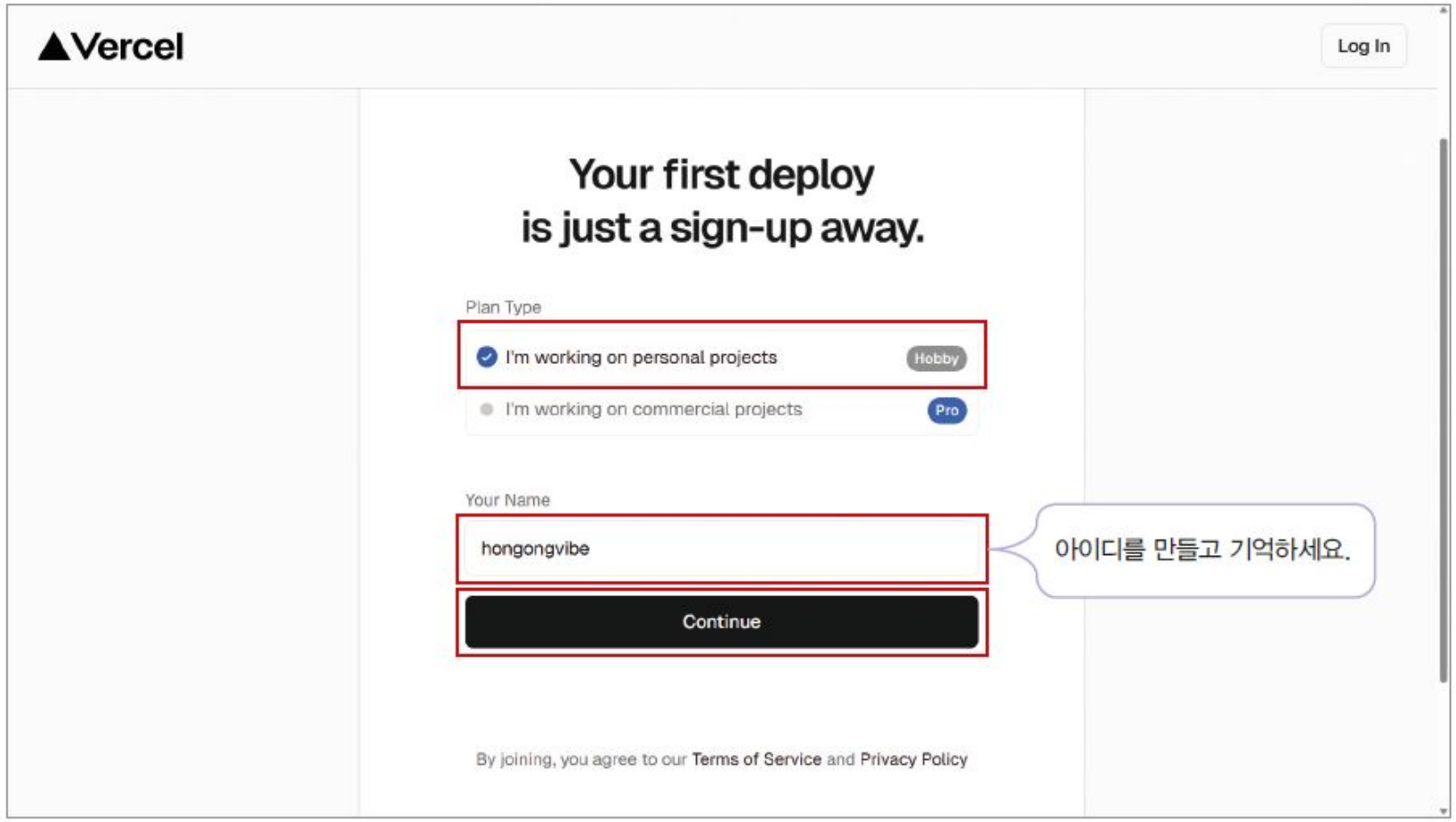
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(4)

01 Vercel 홈페이지에 접속한 뒤, [Sign Up] 버튼을 클릭해 가입을 시작
(URL https://www.vercel.com)



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(5)

02 'Plan Type(계정 유형)'은 'I'm working on personal projects(개인 프로젝트)'를 선택하고, 'Your Name(이름)' 입력란에 아이디를 입력 - 여기서는 'hongongvibe'라고 입력 [Continue] 버튼을 클릭해 진행

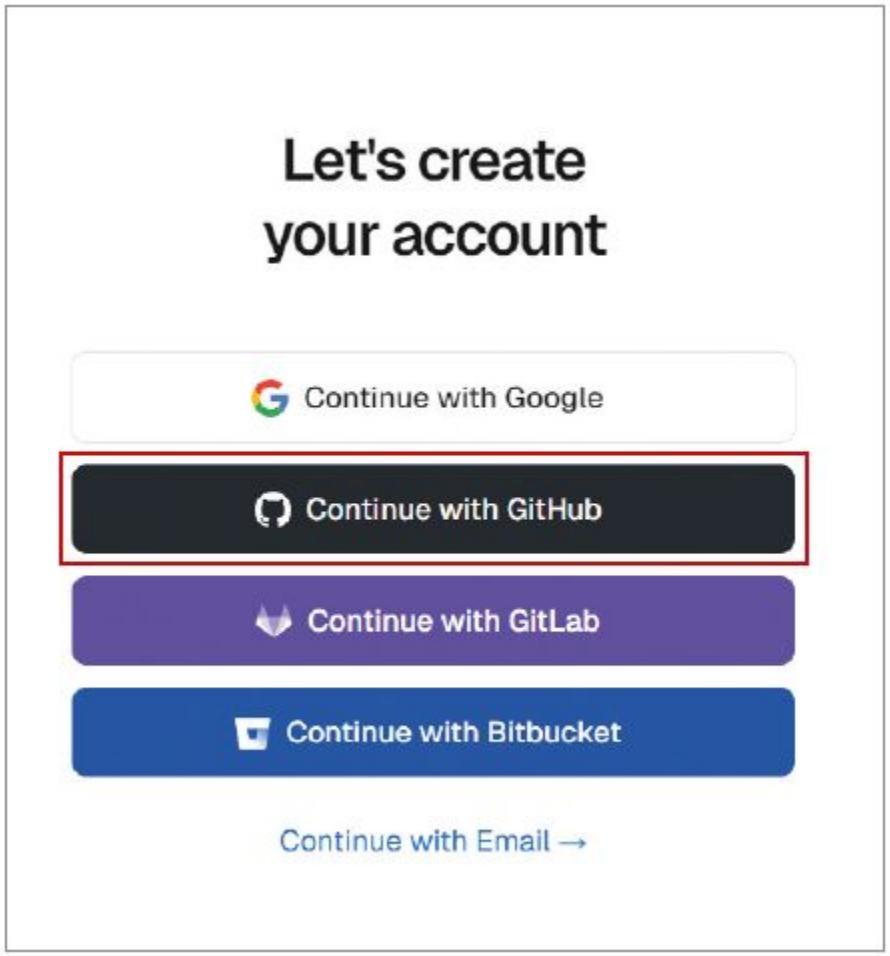


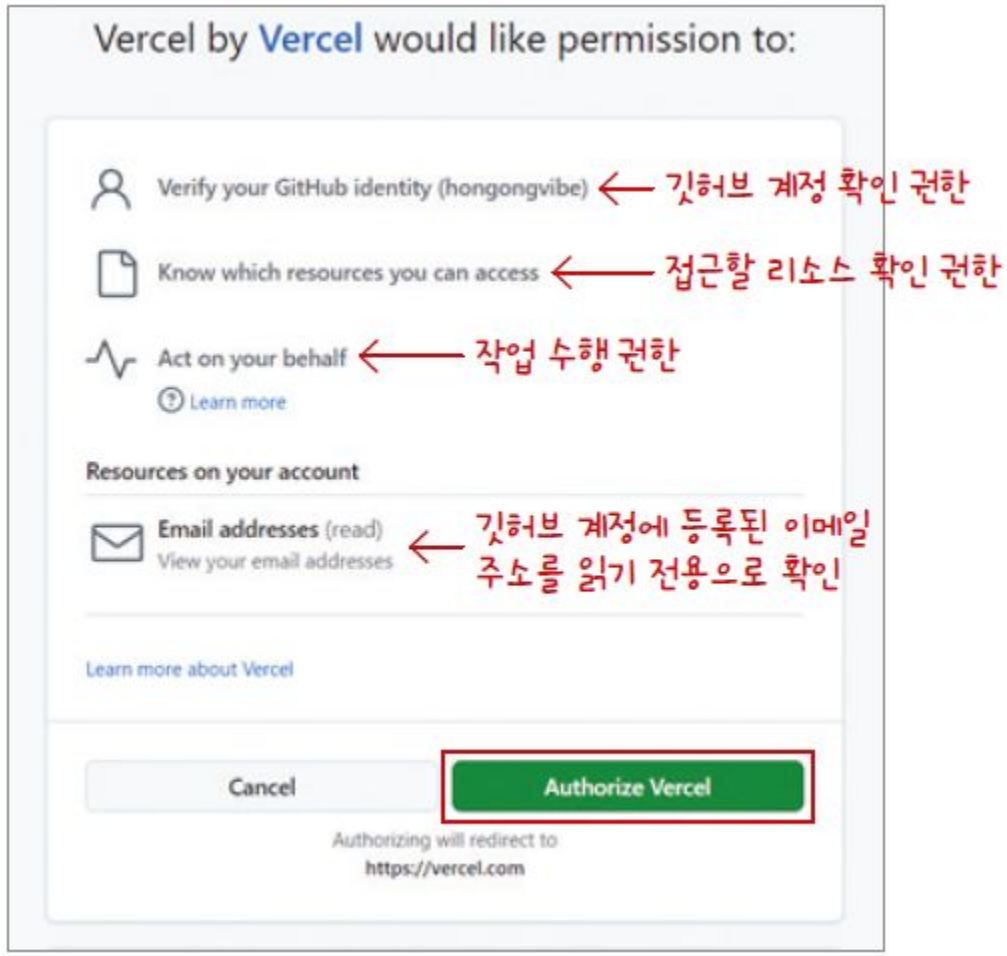
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(6)

03 Vercel 계정을 생성

여기서는 [Continue with GitHub]을 선택해 깃허브 계정과 연동

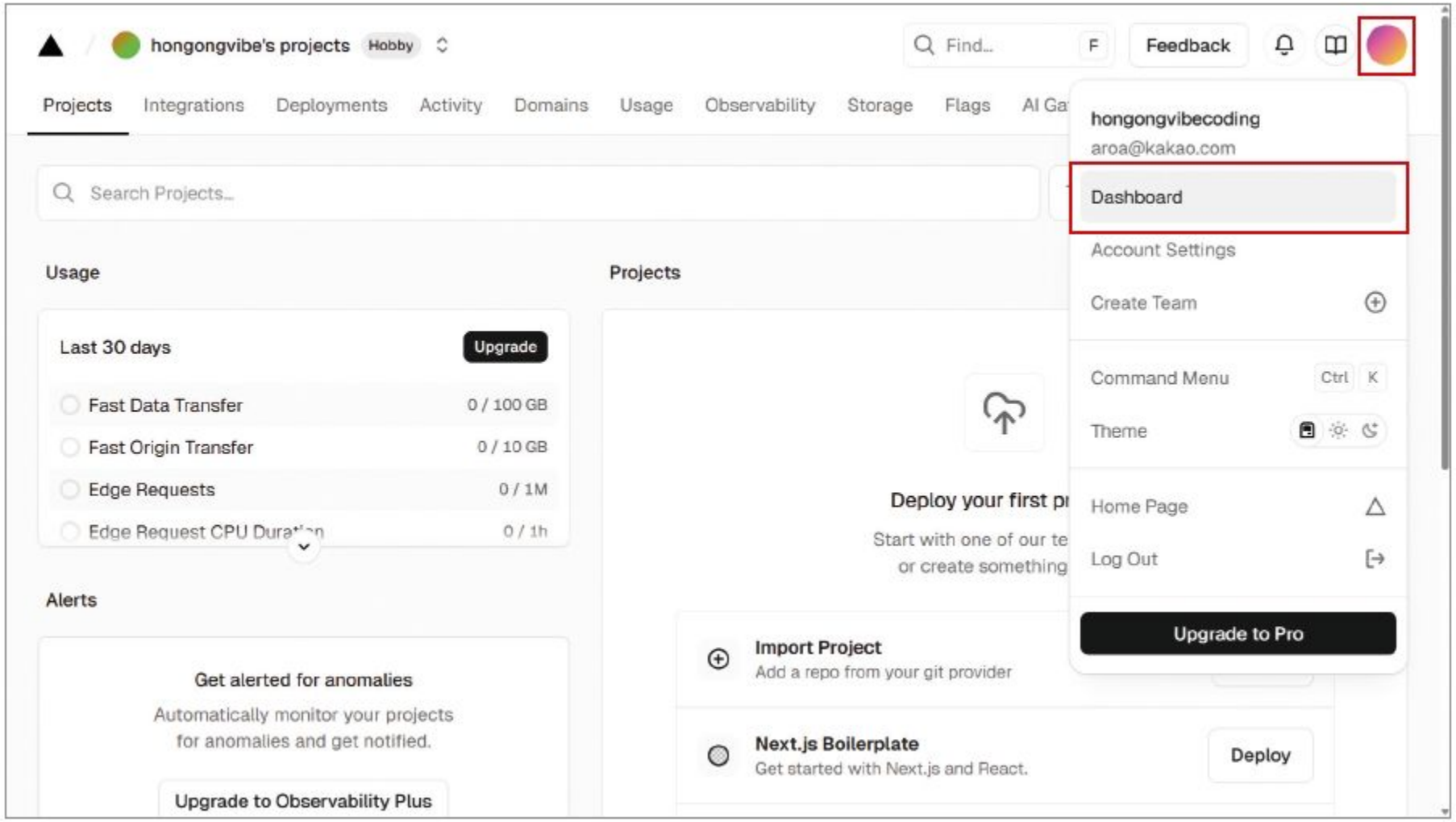
이어 열린 창에서 [Authorize Vercel] 버튼을 클릭해 인증하면 계정 생성이 완료





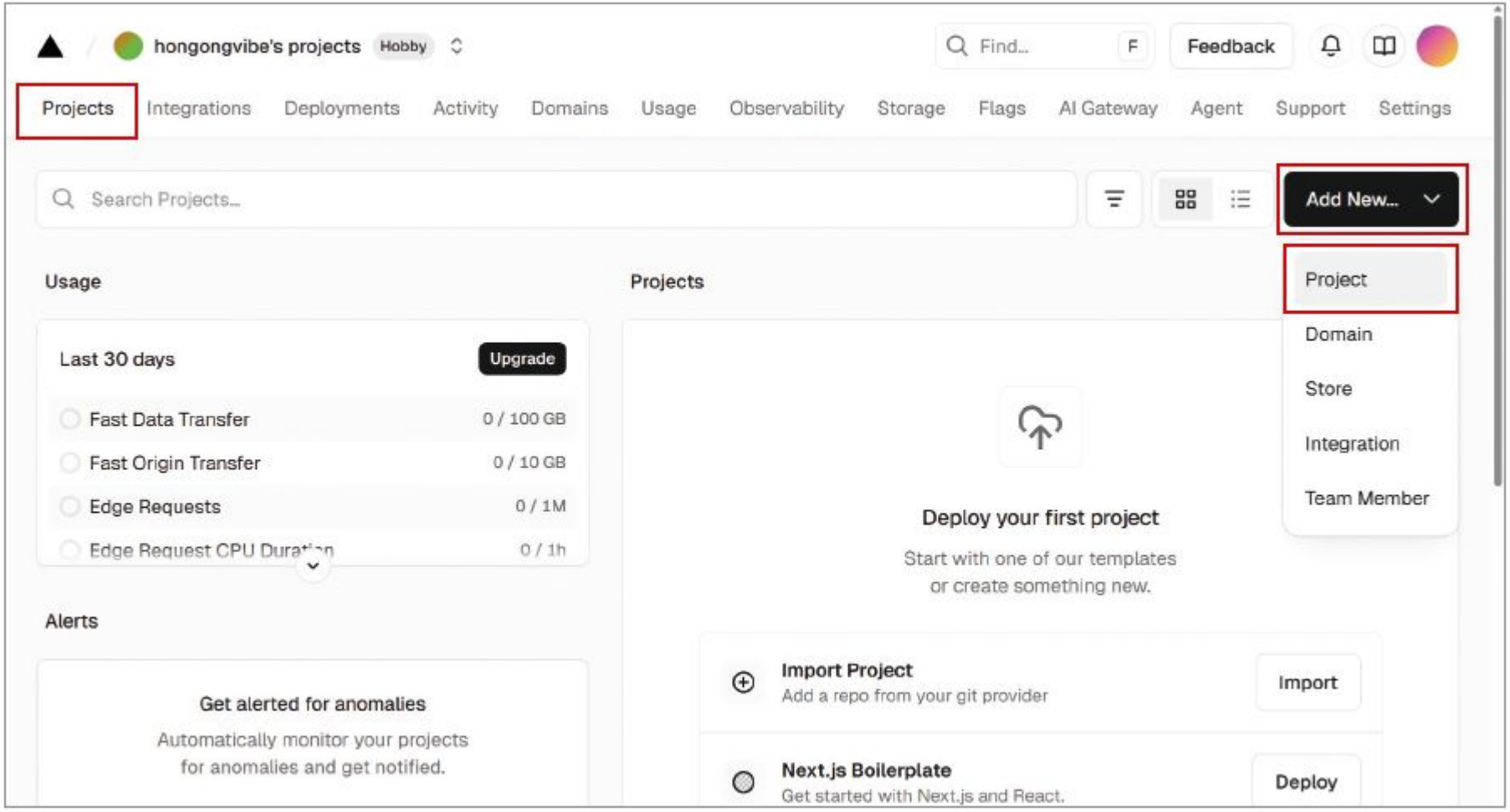
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(7)

04 Vercel에 로그인하고 우측 상단 프로필을 클릭한 뒤, [Dashboard] 메뉴를 선택



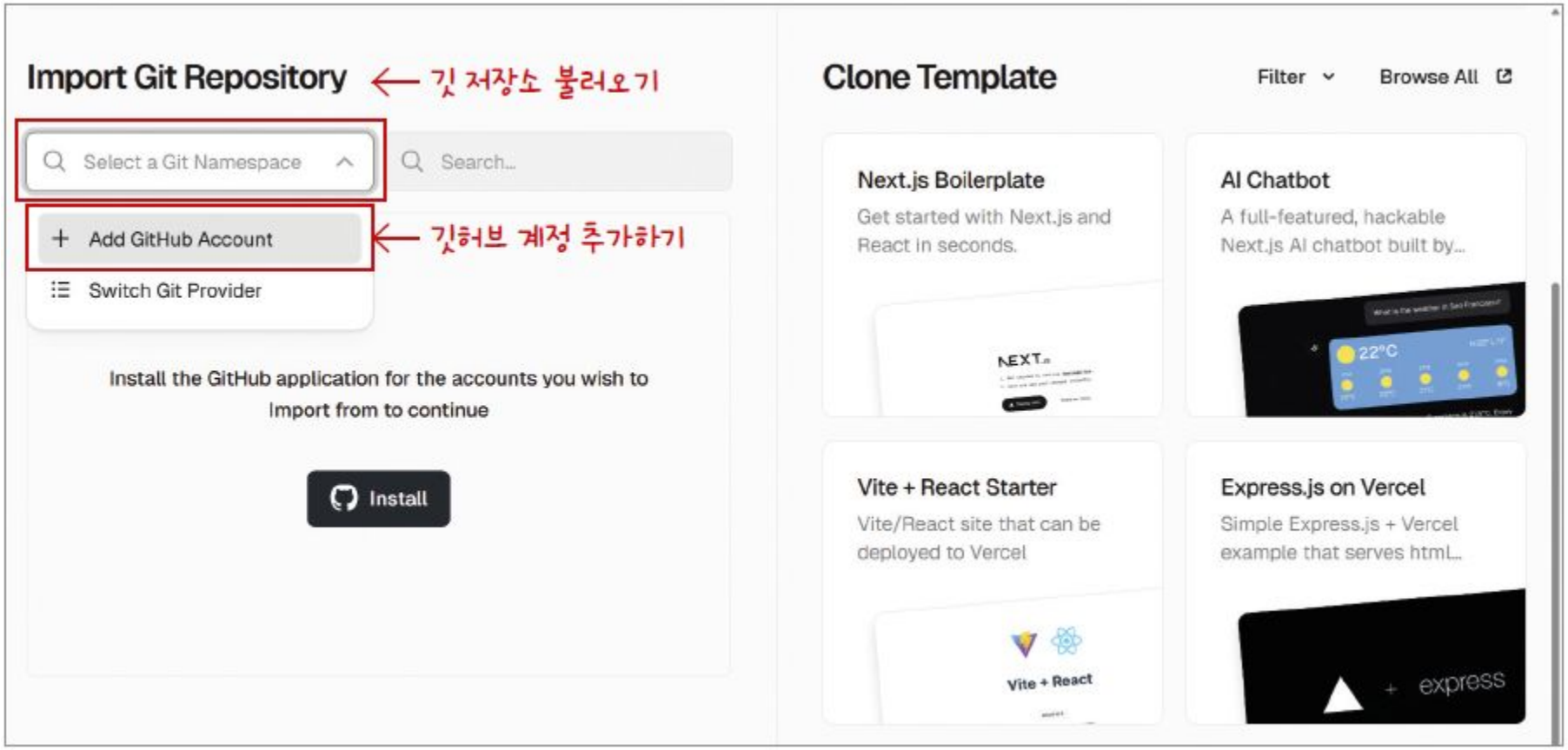
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(8)

05 [Projects] 탭이 선택된 상태에서 오른쪽 [Add New.. +] 버튼을 클릭한 뒤, 드롭다운 메뉴에서 [Project]를 선택



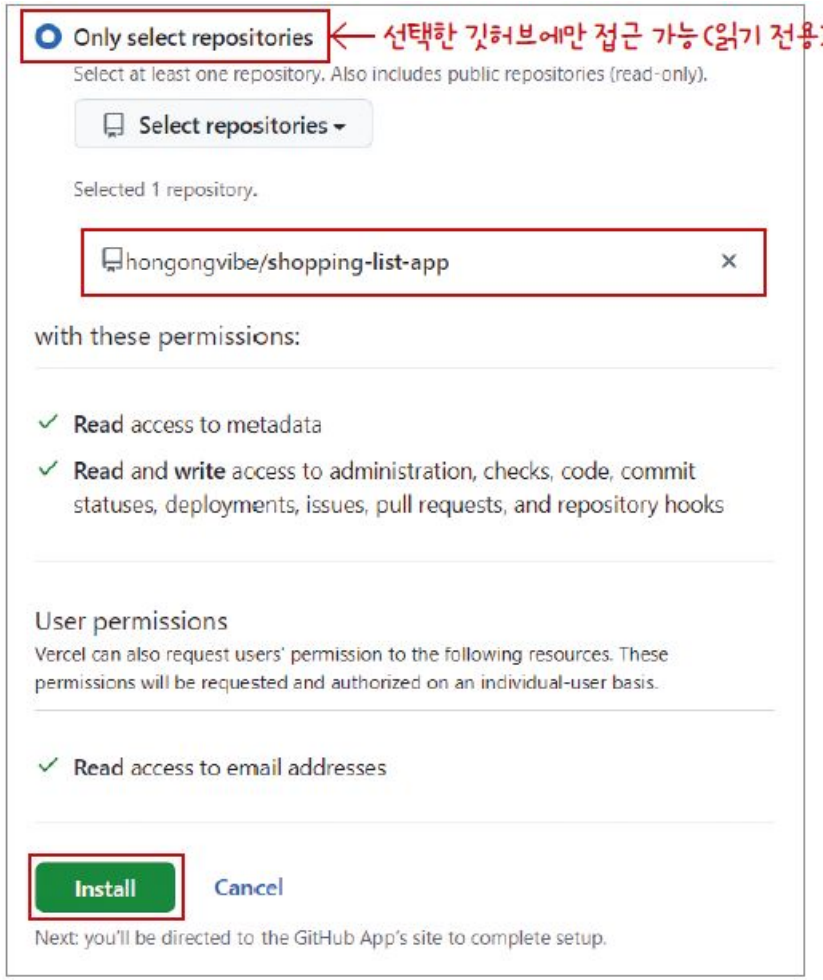
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(9)

06 'Import Git Repository'의 [Select a Git Namespace] 드롭다운 메뉴를 클릭한 뒤, 'Add GitHub Account'를 선택해 내 깃허브와 연동을 시작



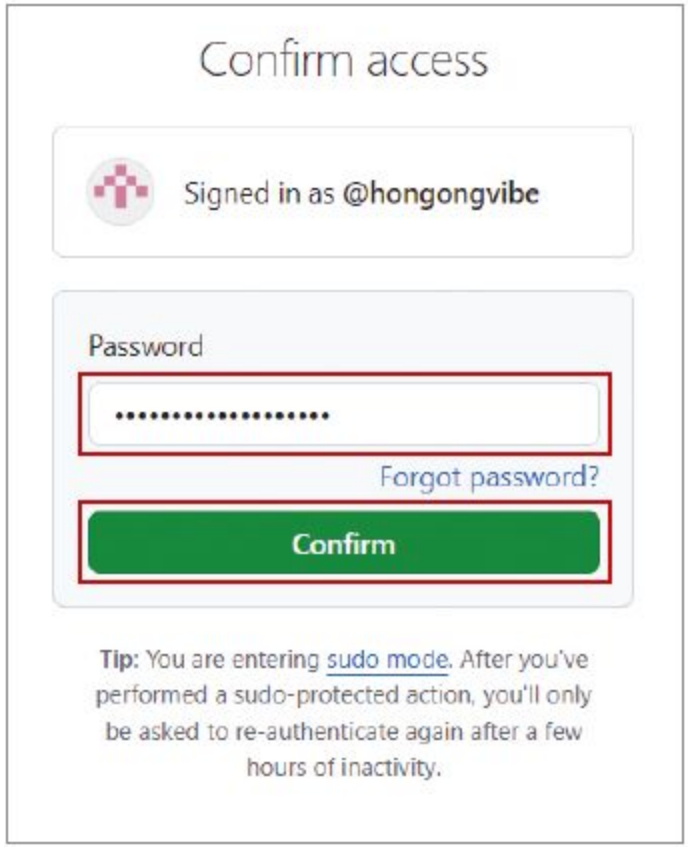
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(10)

07 선택한 깃허브 저장소에만 접근할 수 있도록 'Only select repositories'에 체크
[Select repositories] 드롭다운 버튼을 눌러 'shopping-list-app'을 선택하고,
[Install] 버튼을 눌러 Vercel을 설치



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(11)

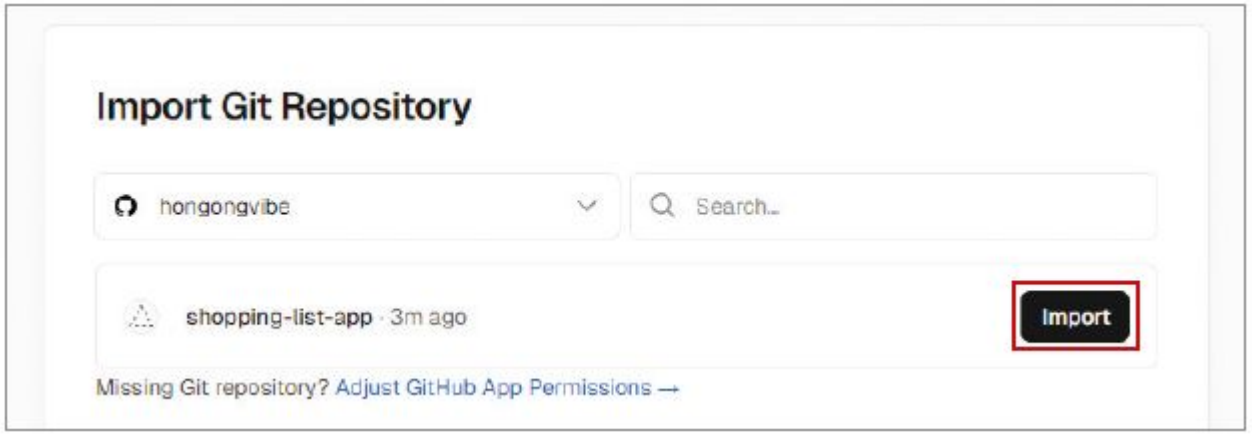
08 깃허브 비밀번호를 입력해 인증하면, 깃허브 저장소를 불러올 준비 완료



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(12)

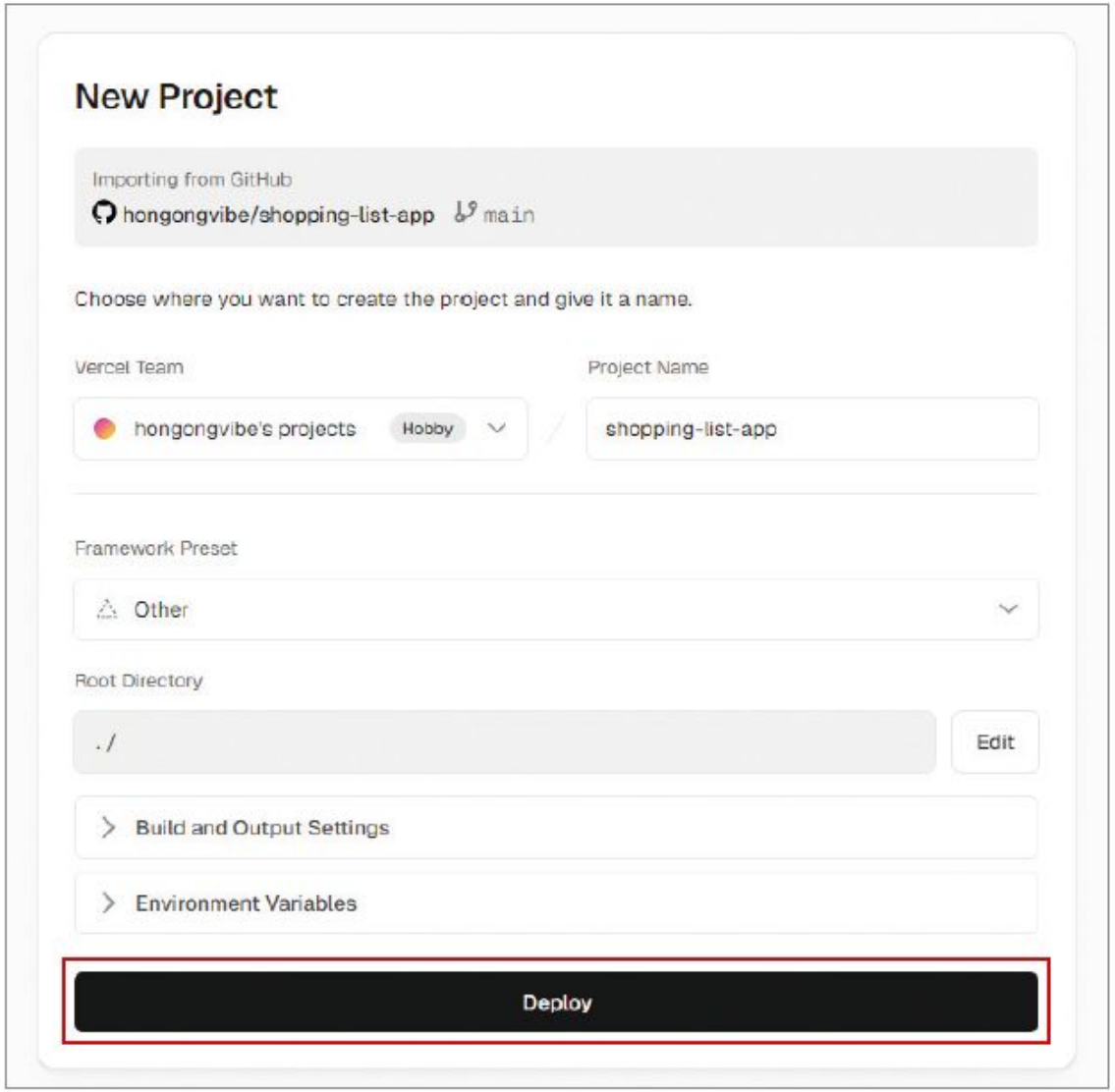
프로젝트 배포하기

01 Vercel로 돌아와 ‘New Project’ 창의 ‘Import Git Repository’ 하위에 추가된 ‘shopping-list-app’ 저장소를 확인하고, [Import] 버튼을 클릭해 불러옴



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(13)

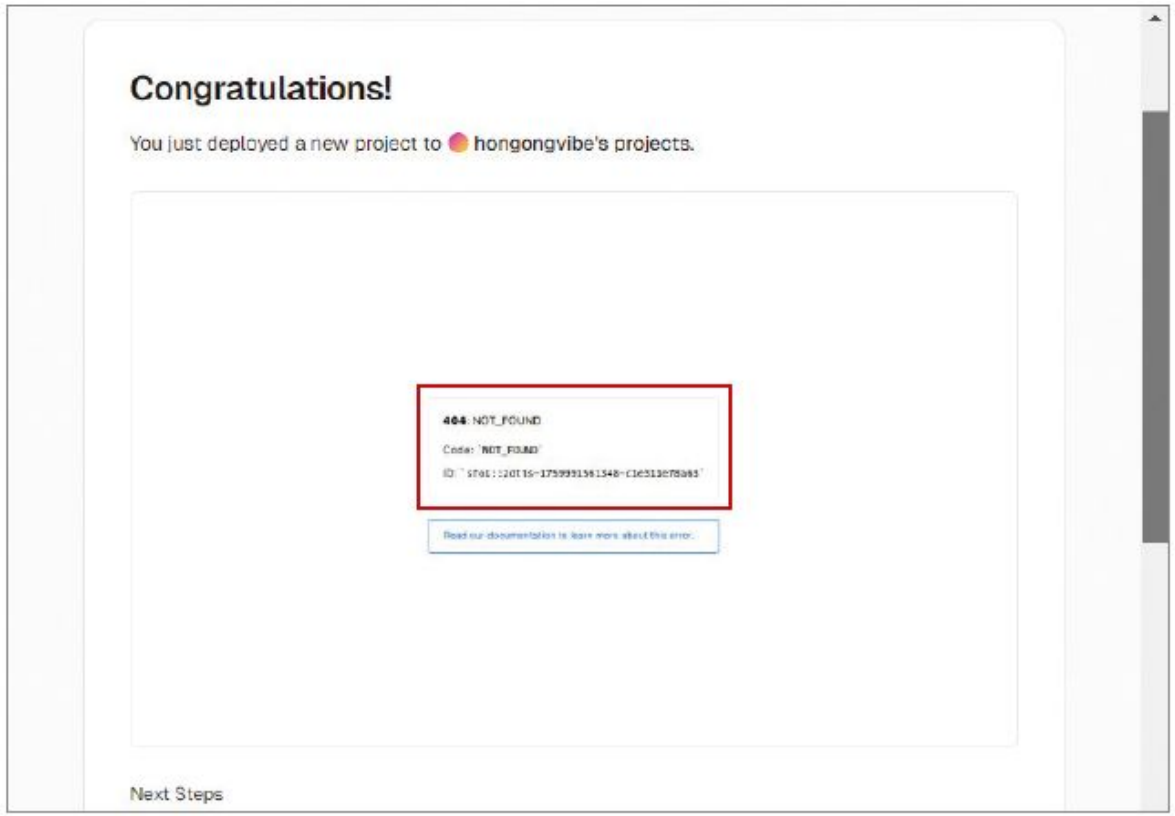
02 새로운 프로젝트를 생성하며 배포할 준비됨 - [Deploy] 버튼을 클릭



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(14)

03 잠시 후 프로젝트가 배포

- 만일 미리보기 창에 '404 Not Found'라는 오류 메시지가 보인다면, 깃허브 저장소에 아직 웹사이트의 기본 시작 페이지인 'index.html' 파일이 없기 때문임
- 쇼핑 리스트 앱이 'index.html'이 아닌 'Shopping-list.html'이라는 이름으로 만들어졌기 때문임



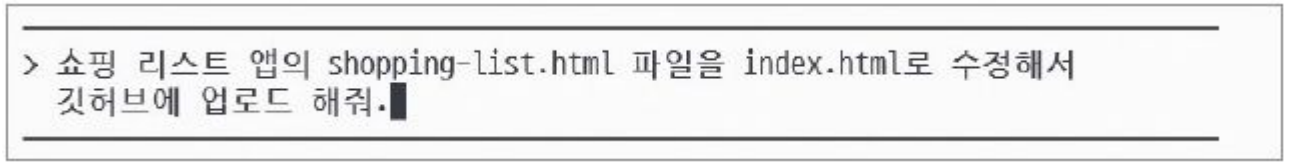
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(15)

04 클로드 코드를 이용해 'shopping-list.html' 파일의 이름을 'index.html'로 수정

클로드 코드에 다음과 같이 프롬프트를 입력

프롬프트

```
쇼핑 리스트 앱의 shopping-list.html 파일을 index.html로 수정해서 깃허브에 업로드해 줘.
```

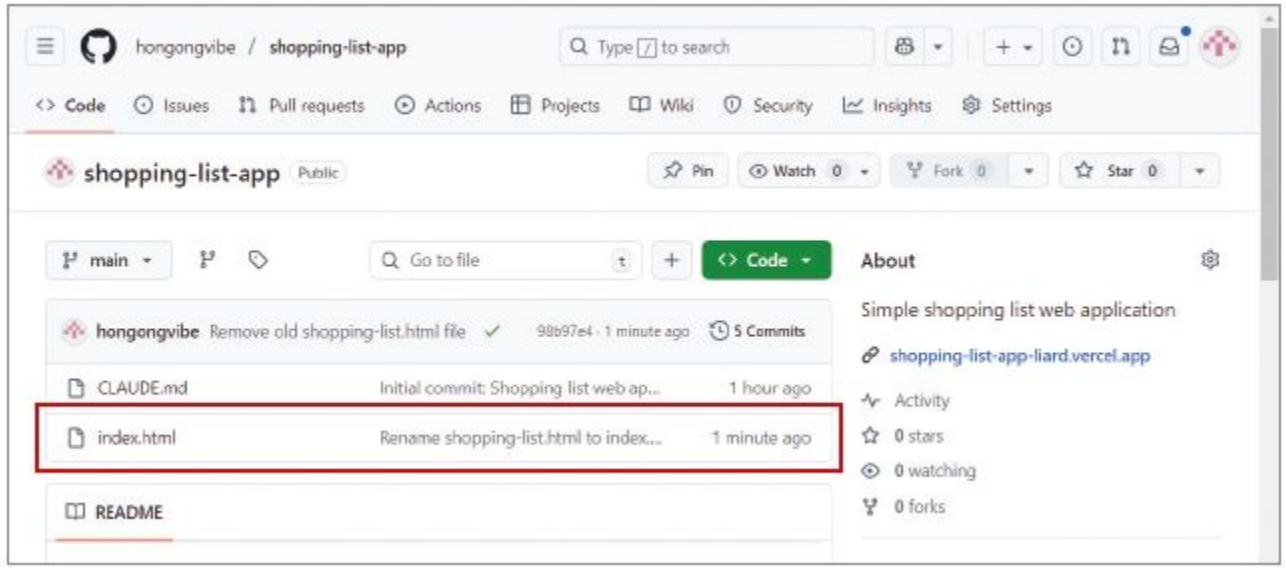


SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(16)

05 완료 메시지를 확인

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(17)

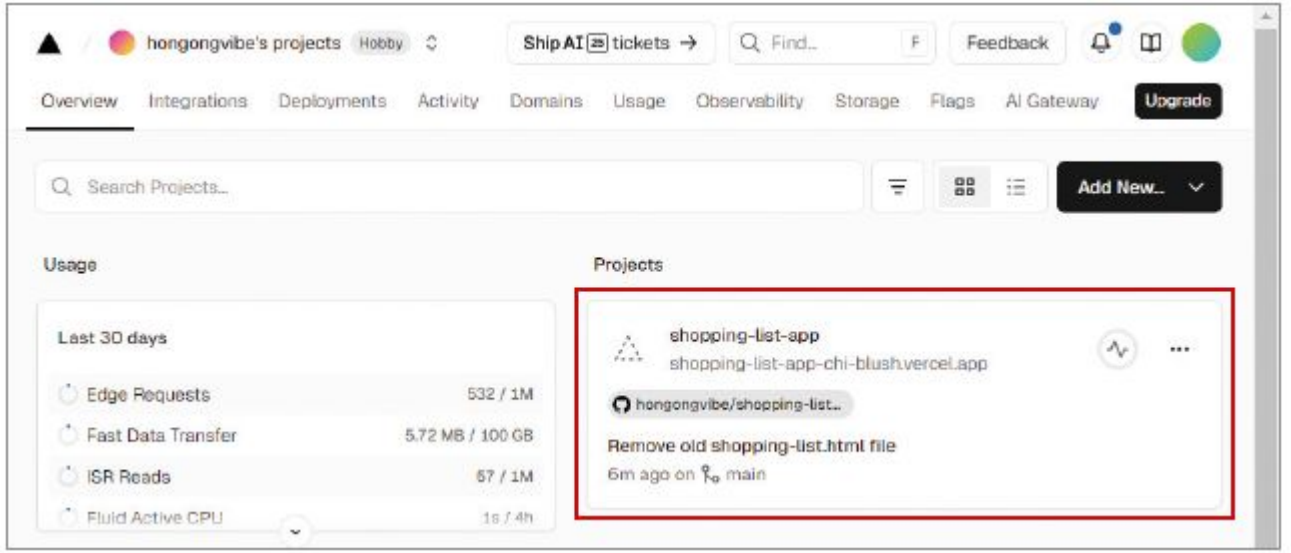
06 깃허브로 돌아가 'shopping-list.html' 파일이 'index.html'로 수정된 것을 확인



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(18)

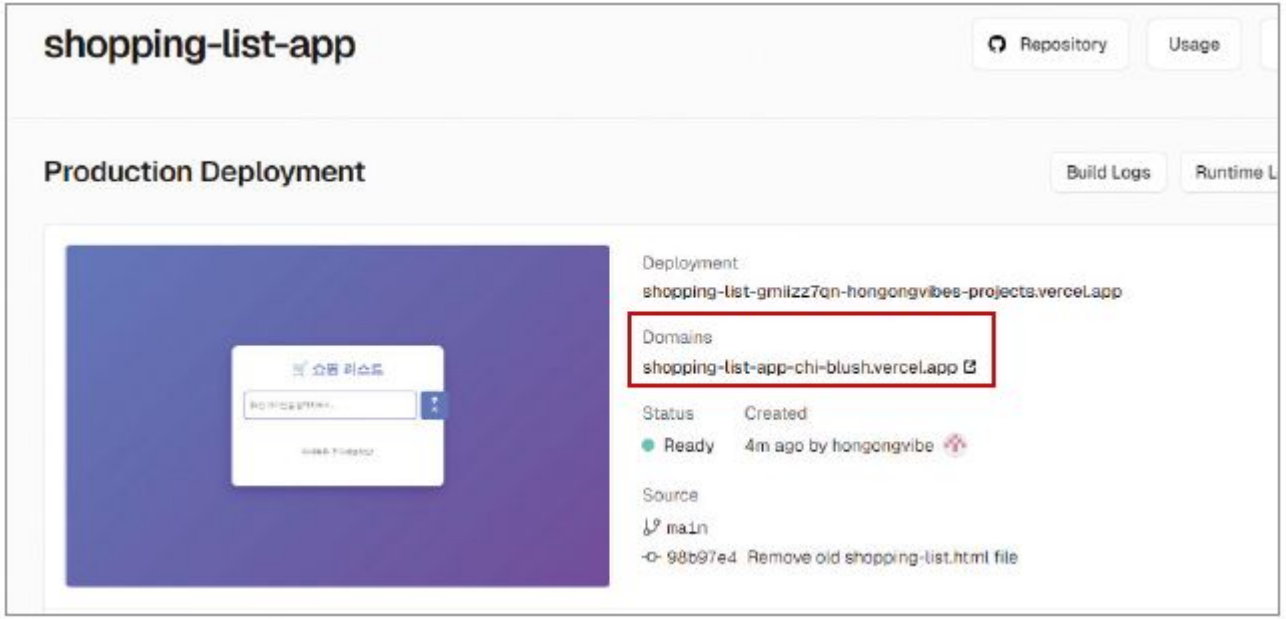
배포된 내 웹사이트 확인하기

01 Vercel의 대시보드dashboard에서 조금 전 생성된 'shopping-list-app' 프로젝트를 클릭



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(19)

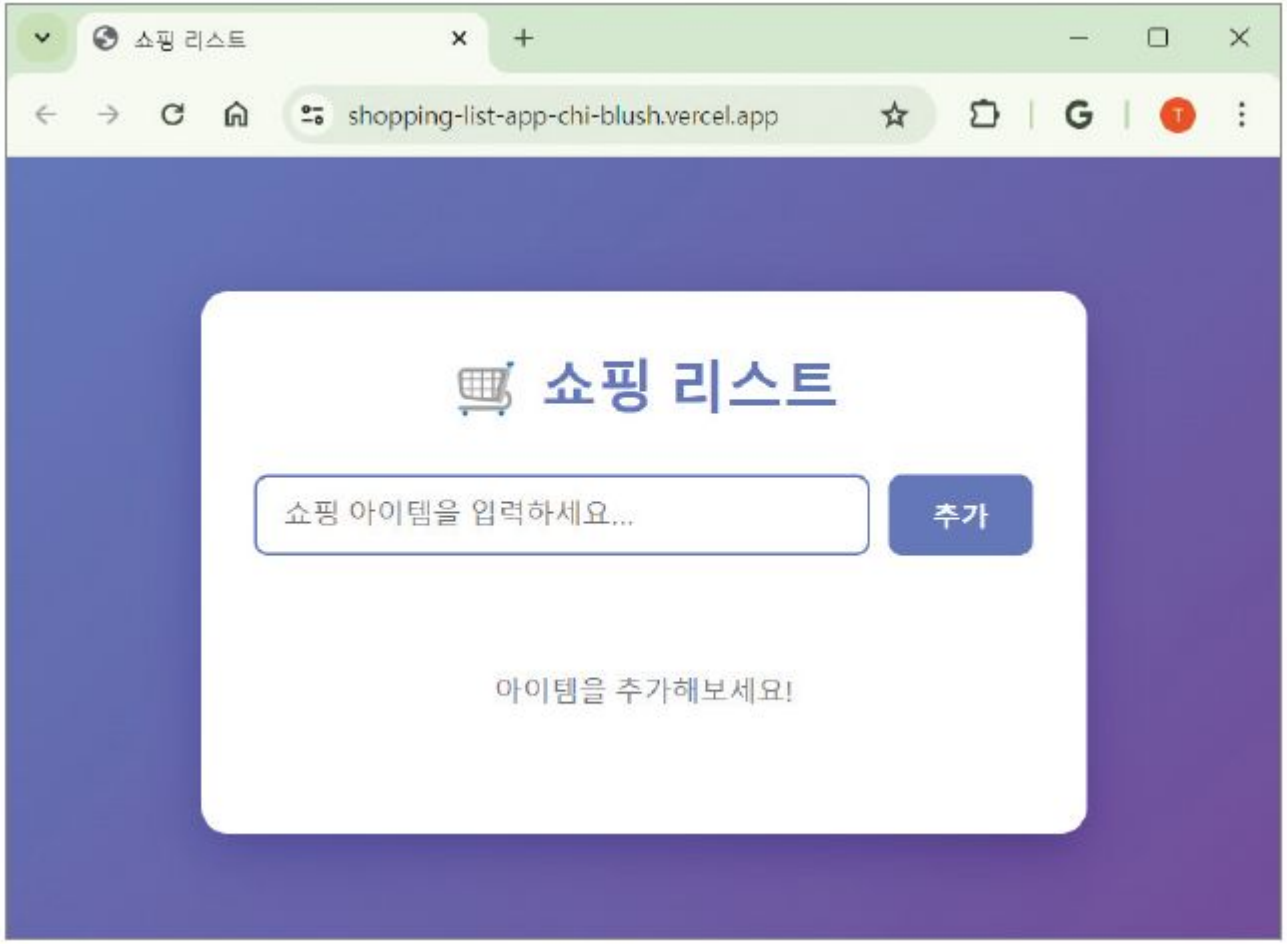
02 배포가 완료된 쇼핑 리스트 앱을 미리보기 창에서 볼 수 있음
미리보기 창 오른쪽의 'Domains' 항목에 배포된 웹 페이지의 URL이 있음
전송하기( ) 아이콘을 클릭해서 해당 주소에 방문



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(20)

03 쇼핑 리스트 앱이 인터넷에 배포

이제 전 세계의 많은 사용자들이 내가 만든 웹 페이지를 방문할 수 있음



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(21)

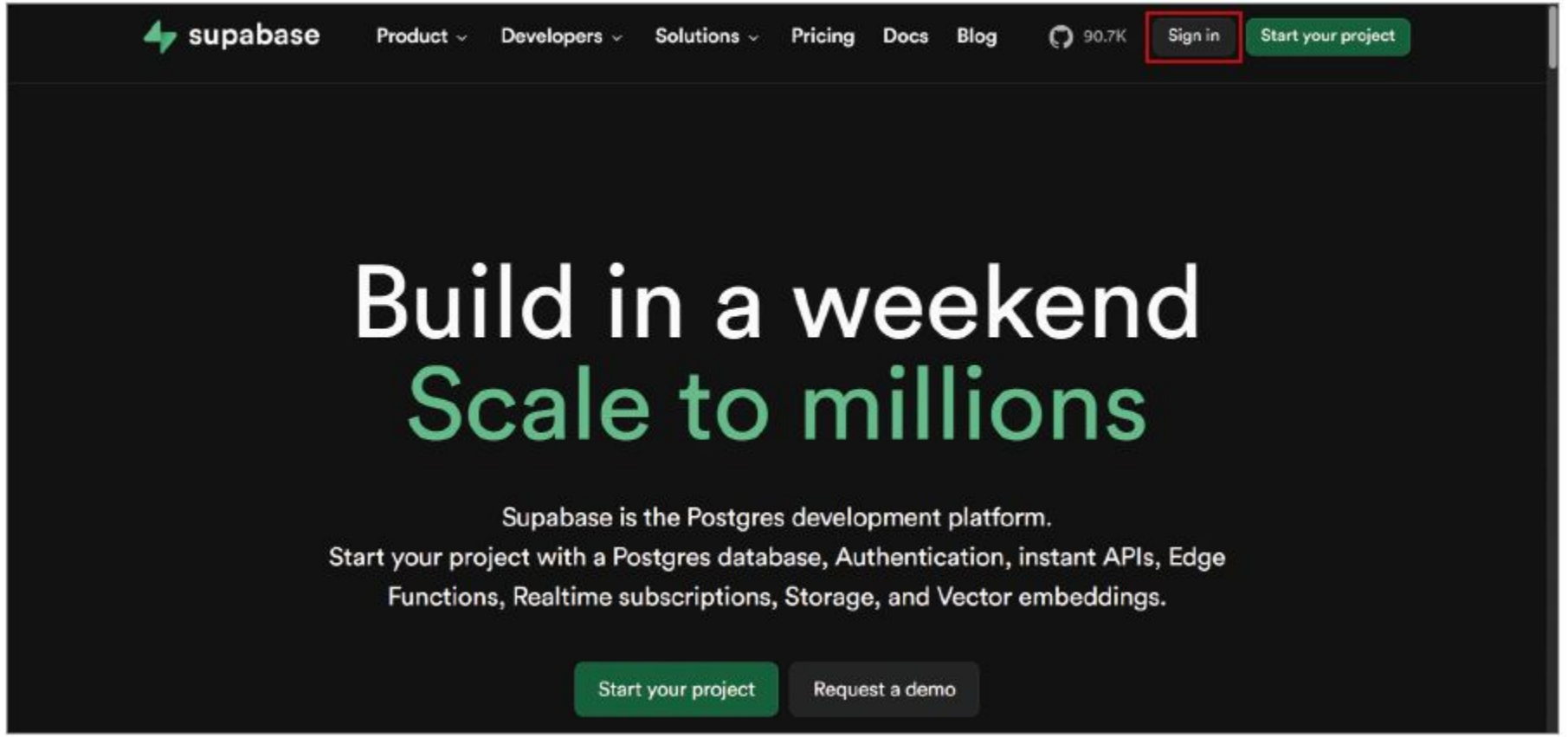
◦ 애플리케이션에 데이터베이스 연동하기

- 로컬 스토리지의 한계
  - 다른 컴퓨터나 브라우저에는 데이터가 공유되지 않고 브라우저 캐시를 지우거나 기기를 바꾸면 데이터가 사라짐 - 즉 오직 나만, 지금 이 기기에서만 데이터를 볼 수 있는 셈
- 인터넷에 연결된 외부 저장소, 클라우드 스토리지(cloud storage)가 필요
  - 데스크톱, 노트북, 스마트폰에서 같은 리스트를 확인할 수 있고, 여러 사람이 공유하거나 협업
- 일반적으로 웹 애플리케이션이 데이터를 저장하려면 Oracle, MySQL 같은 데이터베이스 프로그램을 설치하고 직접 서버를 운영
  - 설정이 복잡하고, 서버 유지와 보수에 많은 시간이 필요
- Supabase는 이런 과정을 간단히 자동화해 주는 클라우드 데이터베이스 플랫폼
  - 회원가입만 하면 별도의 설치 없이도 데이터베이스를 만들 수 있고, 웹이나 모바일 앱에서 API 형태로 데이터를 주고받을 수 있음

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(22)

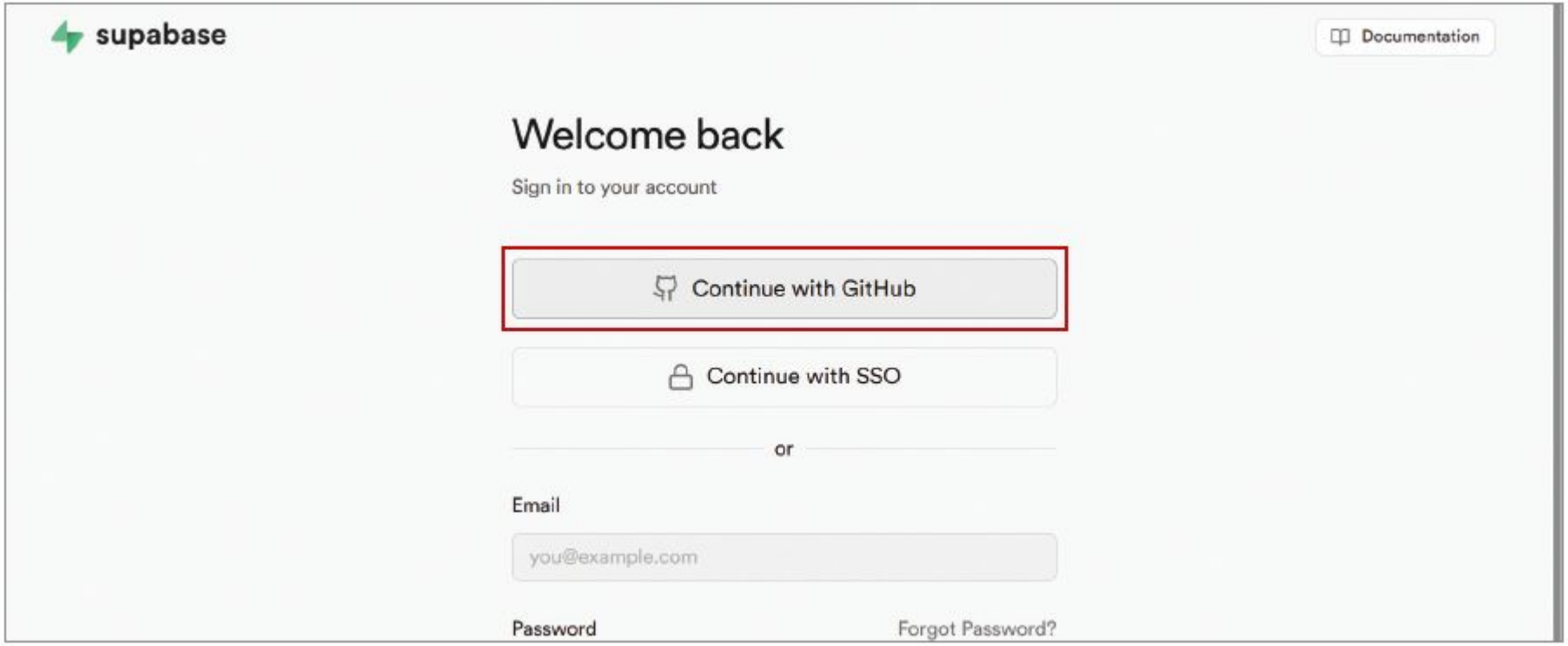
Supabase 가입하고 조직 생성하기

01 Supabase 홈페이지에 접속하고 우측 상단의 [Sign in] 버튼을 클릭해 회원가입
(URL www.supabase.com)



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(23)

02 [Continue with GitHub] 버튼을 클릭해 깃허브 계정으로 회원가입



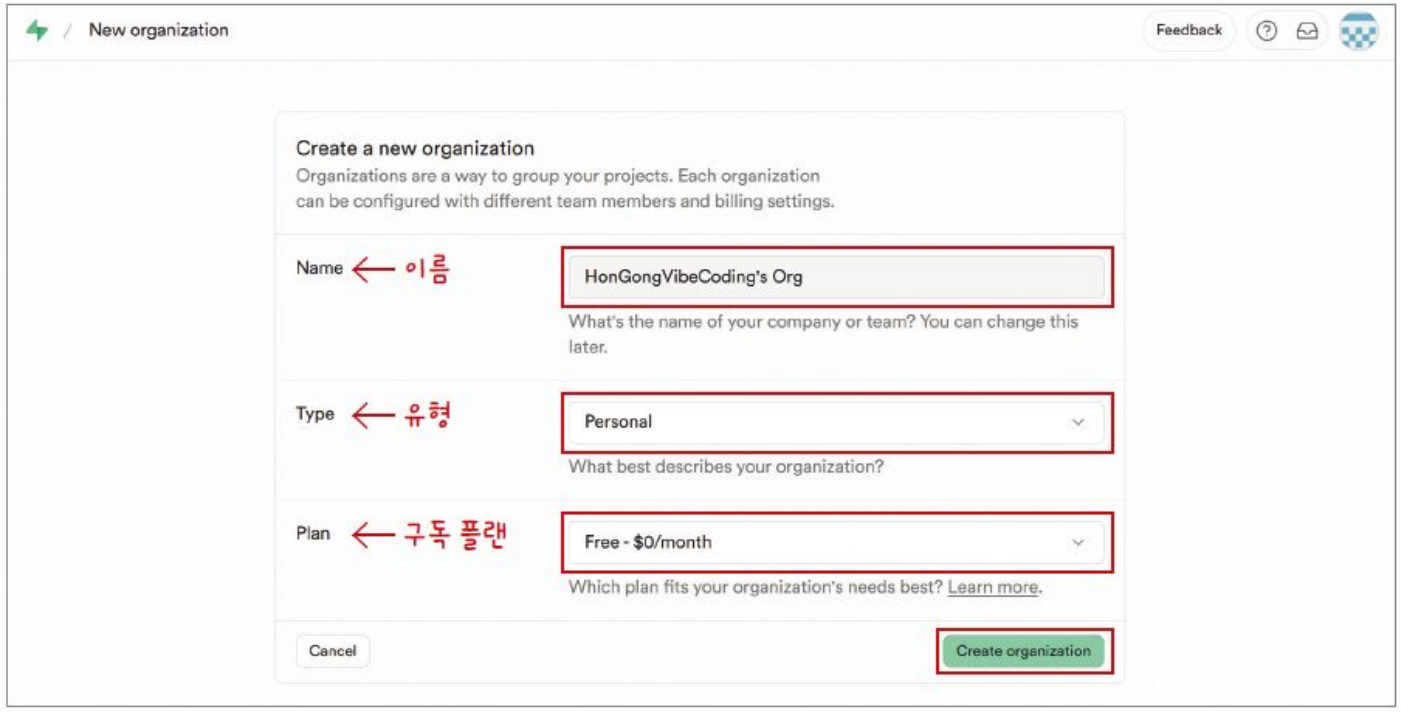
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(24)

03 'New organization' 창에서 조직(organization)을 생성

Supabase에서 조직은 여러 프로젝트를 묶어 관리하는 폴더와 같은 개념

각 항목은 다음과 같이 입력한 뒤, [Create organization] 버튼을 클릭하여 조직을 생성

- Name: HonGongVibeCoding's Org(자동 생성됨)
- Type: Personal
- Plan: Free - $0/month



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(25)

Supabase MCP 서버 설치와 API 토큰 발급

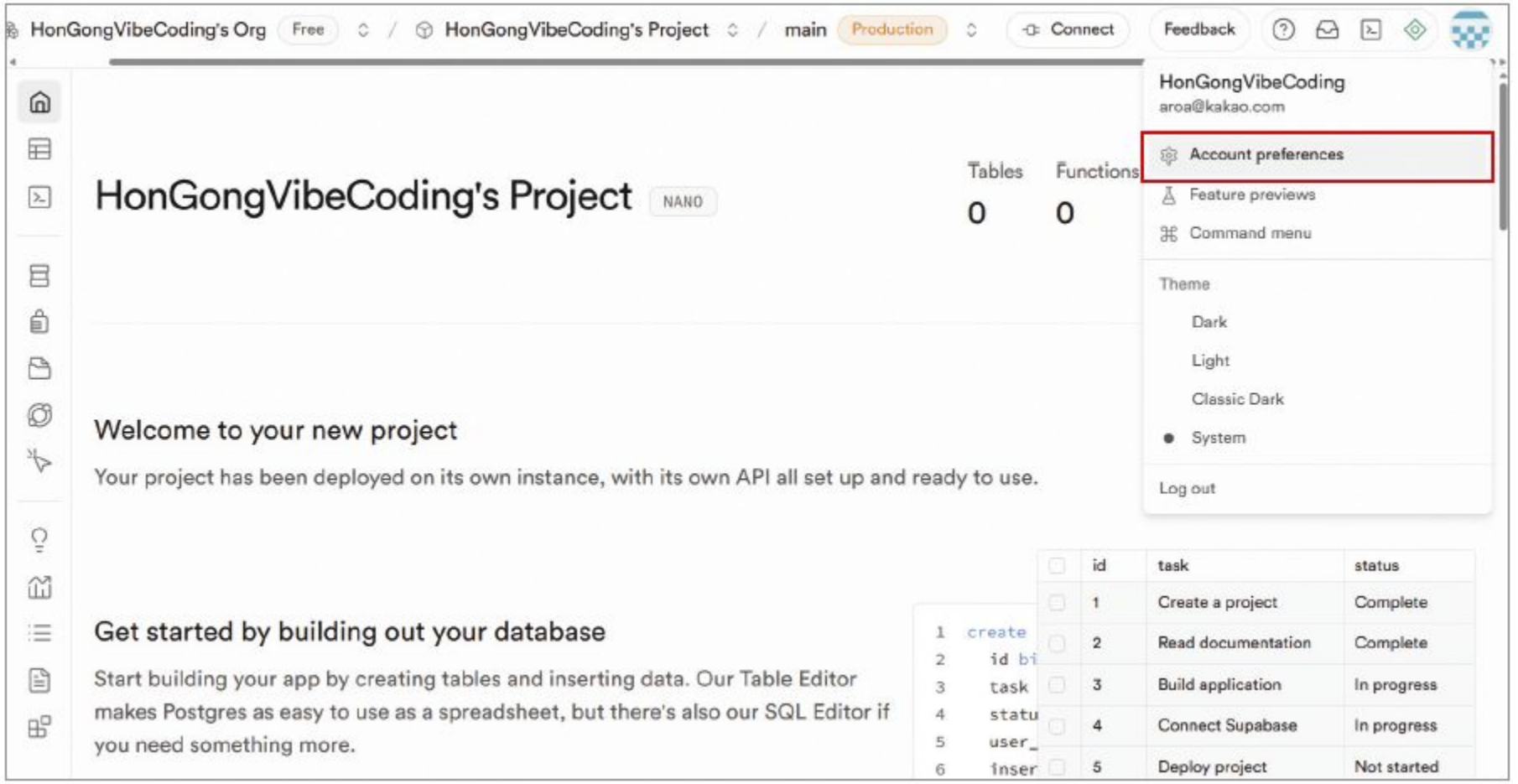
- Supabase는 로컬 MCP 서버와 원격 MCP 서버를 모두 지원 - 여기서는 로컬 MCP 서버를 설치
- 터미널을 실행하고 다음과 같이 설치 명령어를 입력한 뒤, 인증 과정을 진행

```
claude mcp add supabase -s local -e SUPABASE_ACCESS_TOKEN=<Supabase API 토큰>
-- cmd /c npx -y @supabase/mcp-server-supabase@latest
```

  - 명령어는 Supabase사(@supabase)가 만든 MCP 서버(mcp-server-supabase)를 최신 버전(@latest)으로 실행하라는 의미
  - 새로 추가된 '-e'는 '--env'의 축약형으로, MCP 서버에 전달하는 환경 변수를 설정하는 옵션
  - 여기서는 Supabase를 인증할 API 토큰이 필요한 상황이므로 토큰을 전달하는 옵션이 추가
- Supabase 홈페이지로 돌아가 API 토큰을 발급

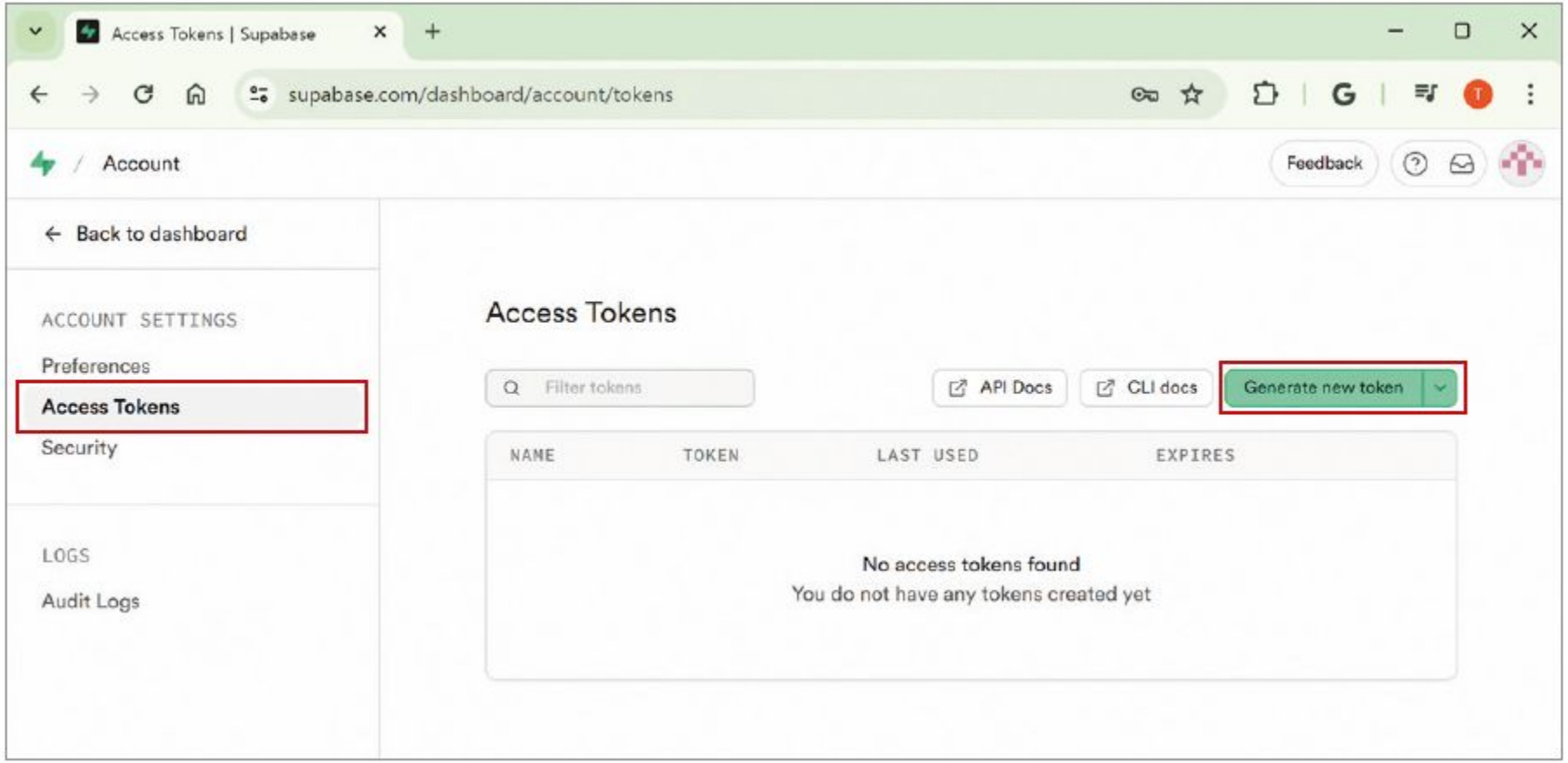
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(26)

01 Supabase 시작 페이지에서 오른쪽 상단의 프로필을 클릭한 뒤, [Account preferences] 메뉴를 선택



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(27)

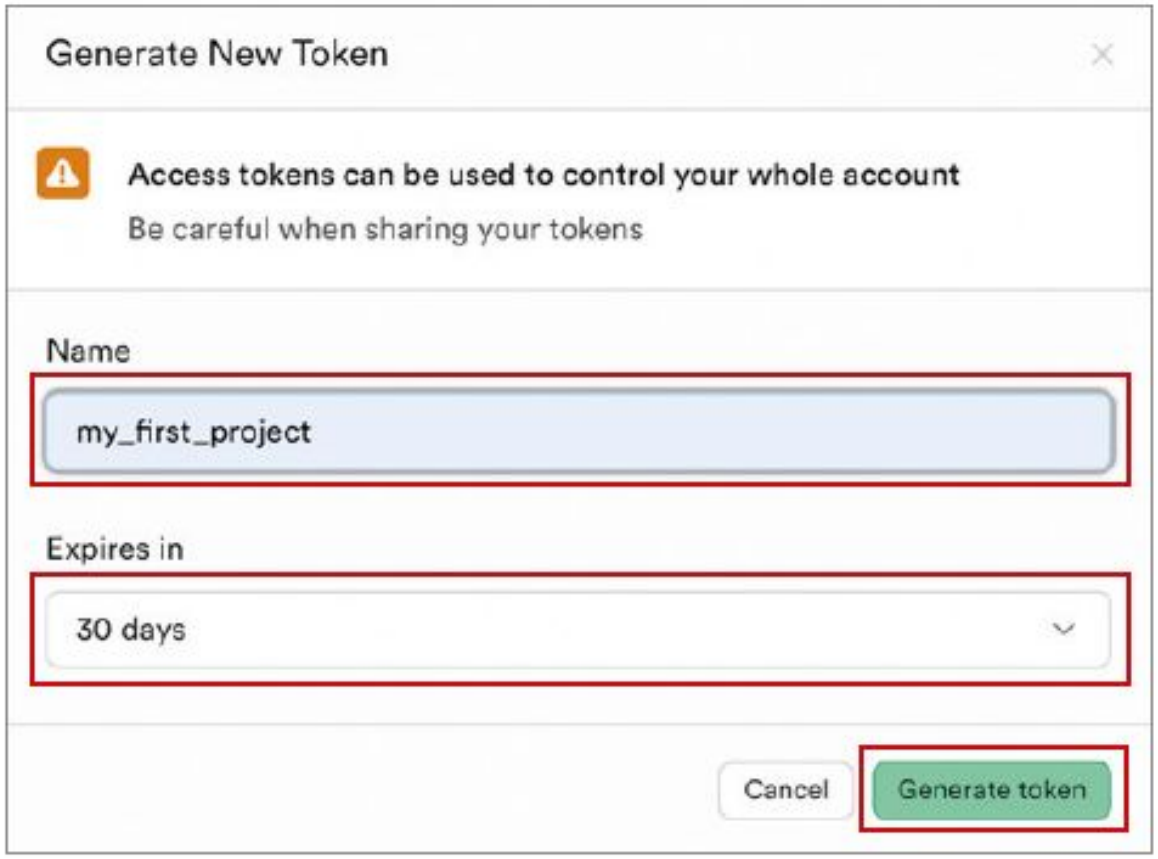
02 왼쪽 사이드바에서 [Access Tokens]를 클릭하면 토큰 관리 화면으로 이어짐
오른쪽의 [Generate new token] 버튼을 클릭하여 새로운 API 토큰을 생성



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(28)

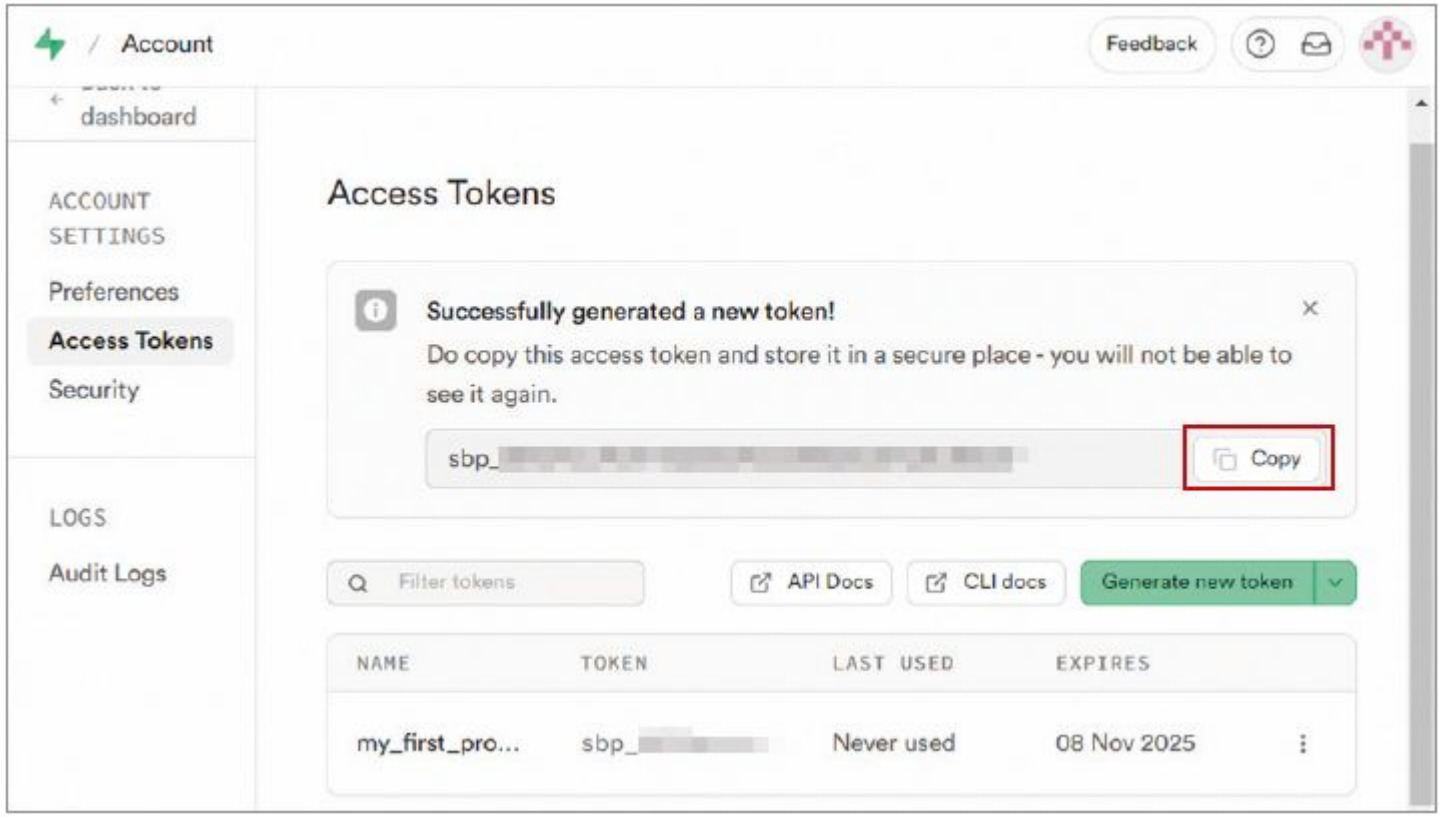
03 'Generate New Token' 창의 'Name' 입력란에는 'my_first_project'를 'Expires in(토큰 유효 기간)'은 '30 days'로 설정

[Generate token] 버튼을 클릭해 설정을 완료



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(29)

04 토큰이 성공적으로 생성. [Copy] 버튼을 클릭하여 복사한 후, 안전한 곳에 저장

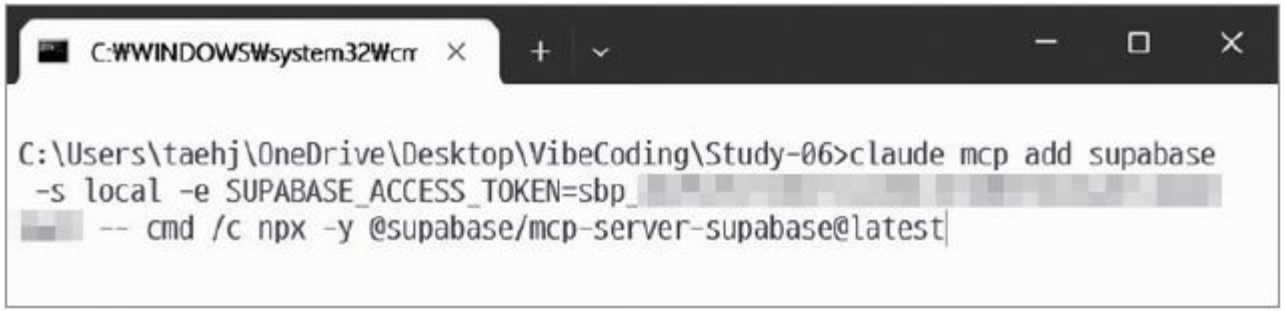


SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(30)

05 클로드 코드에서 Supabase MCP를 연결

'Study-06' 폴더에서 터미널을 실행한 후, 다음과 같이 Supabase MCP를 설치

```
claude mcp add supabase -s local -e SUPABASE_ACCESS_TOKEN=<내 API 토큰> -- cmd /c npx -y
@supabase/mcp-server-supabase@latest
```



[note] <내 API 토큰> 자리에 발급받은 Supabase API 토큰을 입력

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(31)

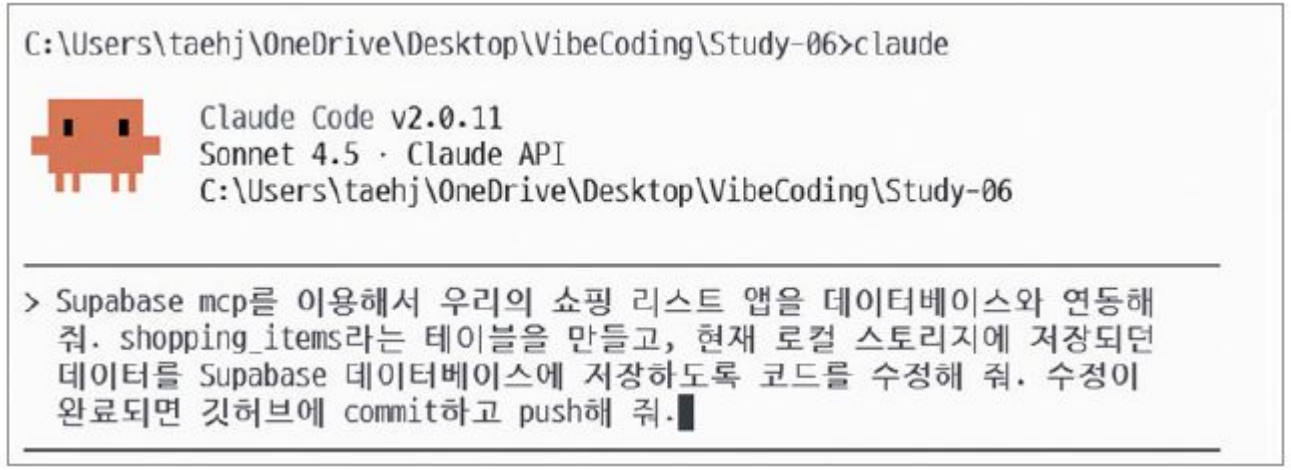
◦ 쇼핑 리스트 앱 진짜 서비스로 업그레이드하기

클로드 코드로 데이터베이스 자동 연동하기

01 클로드 코드를 실행하고 다음과 같이 입력

프롬프트

```
Supabase mcp를 이용해서 우리의 쇼핑 리스트 앱을 데이터베이스와 연동해 줘. shopping_items라는 테이블을 만들고, 현재 로컬 스토리지에 저장되던 데이터를 Supabase 데이터베이스에 저장하도록 코드를 수정해 줘. 수정이 완료되면 깃허브에 commit하고 push해 줘.
```



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(32)

✚ 여기서 잠깐 **데이터베이스의 테이블 이해하기**

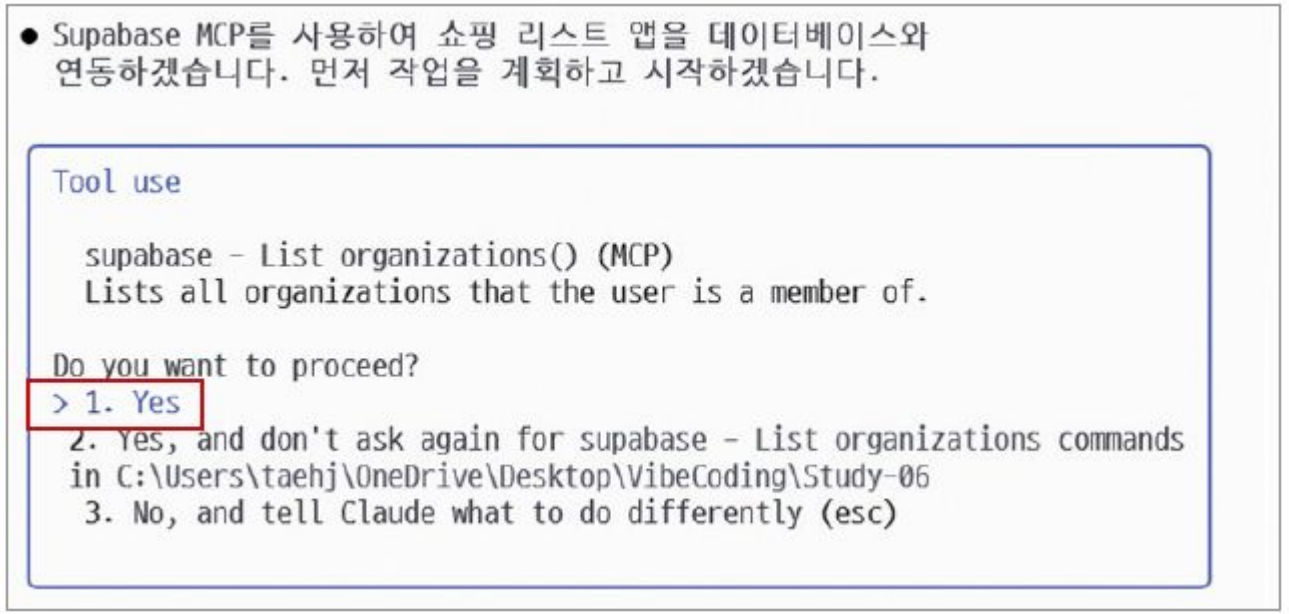
- 데이터베이스의 테이블(table)
  - 데이터를 표 형태로 저장한 구조
- 데이터베이스가 여러 엑셀 파일을 관리하는 프로그램이라면, 테이블은 데이터베이스 안에 있는 개별 엑셀 시트
  - 예) shopping_items 테이블의 행(row)에는 쇼핑 아이템이, 열(column)은 아이템의 속성이 저장
    각 열에는 고유 번호(id), 상품명(name), 구매 완료 여부(completed), 생성 시간(created_at)이 저장

id	name	completed	created_at
1	사과	False	2025-10-29 15:30:00
2	우유	True	2025-10-29 15:32:10
3	빵	false	2025-10-29 15:35:22

- 이렇게 구조화된 형태로 데이터를 저장하면 필요한 정보를 빠르게 찾고 수정할 수 있음

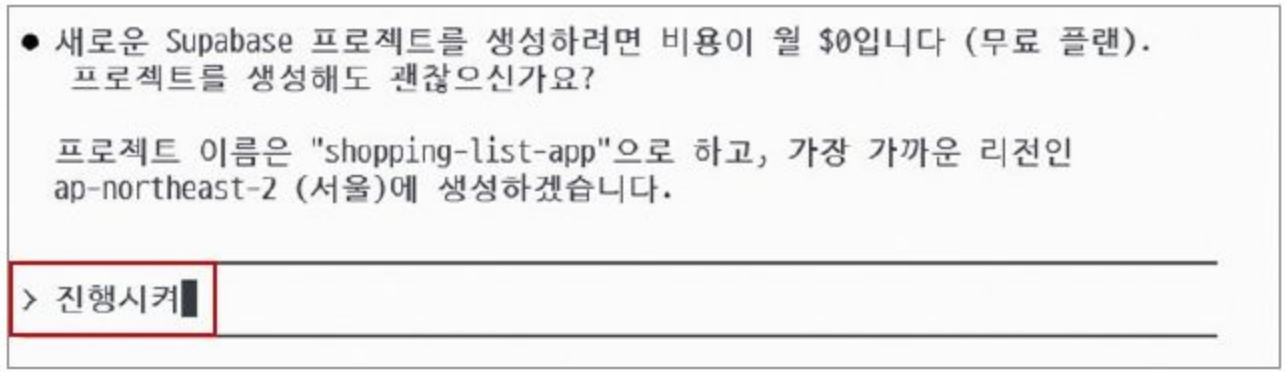
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(33)

02 Supabase MCP가 작동 - '1. Yes'를 눌러 계속 진행



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(34)

03 Supabase에 프로젝트를 새로 만든다는 메시지와 함께 무료 플랜임을 확인하는 메시지가 나옴 내용을 확인하고 계속 진행

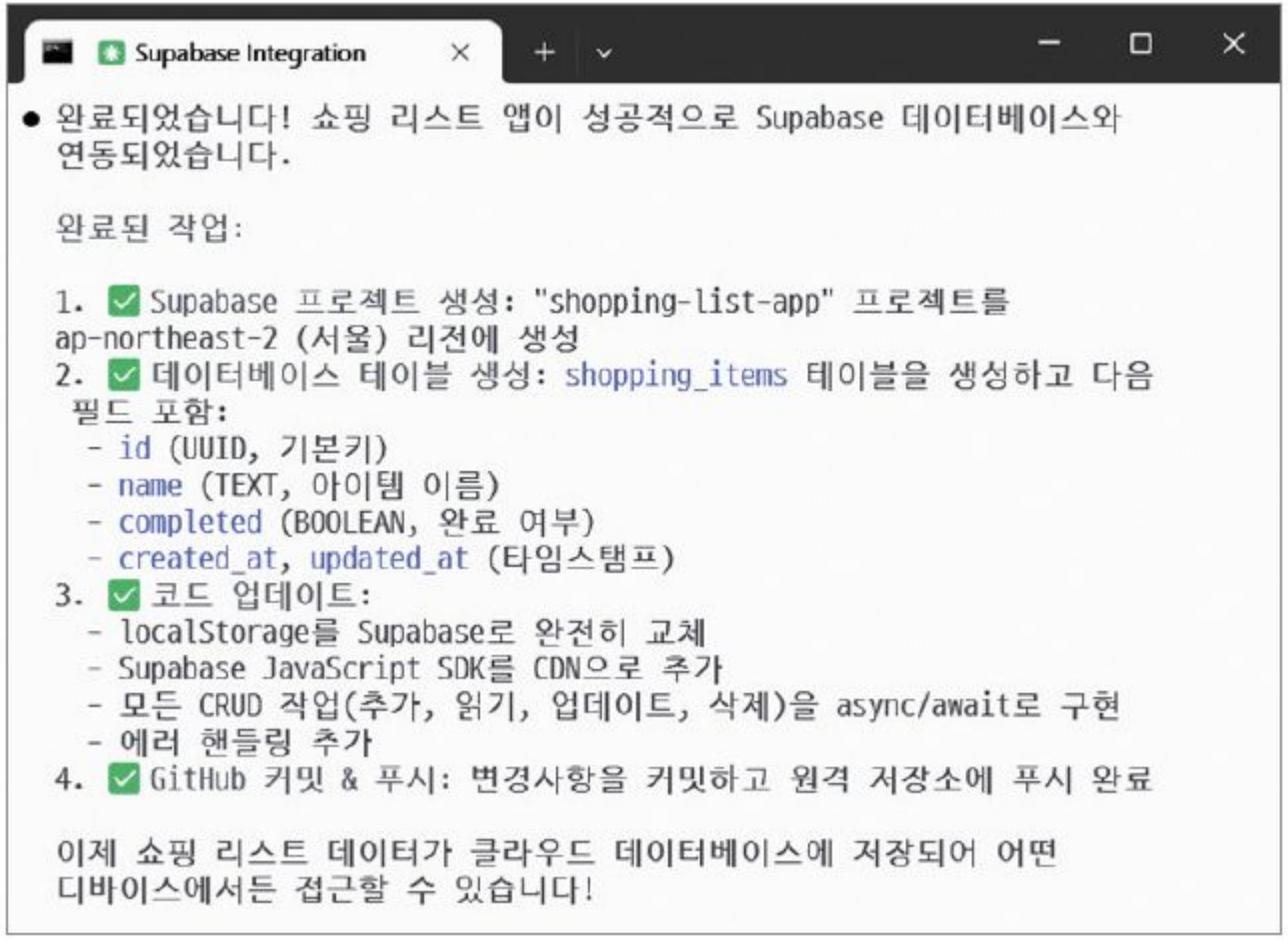


SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(35)

04 Supabase MCP를 연동할 수 있도록 'index.html' 파일을 자동으로 수정

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(36)

05 shopping_items 테이블을 생성하고 id, name 등 필요한 필드를 포함해 성공적으로 데이터베이스를 연동

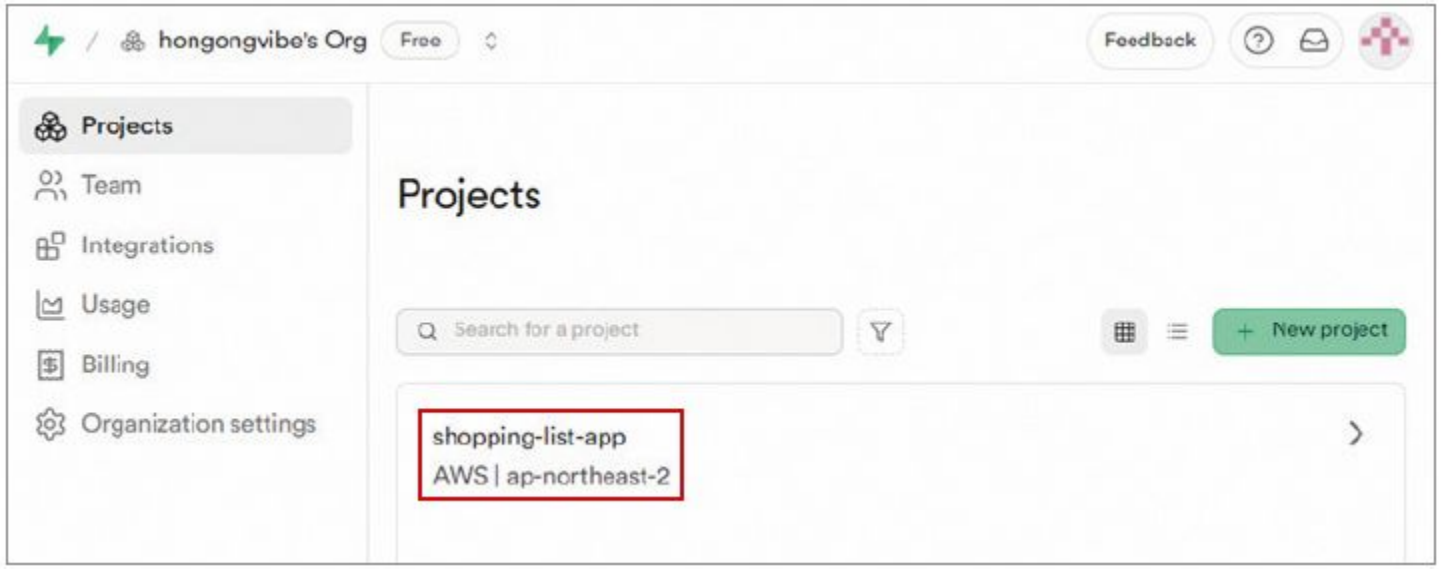


SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(37)

데이터베이스 테이블 생성 확인하기

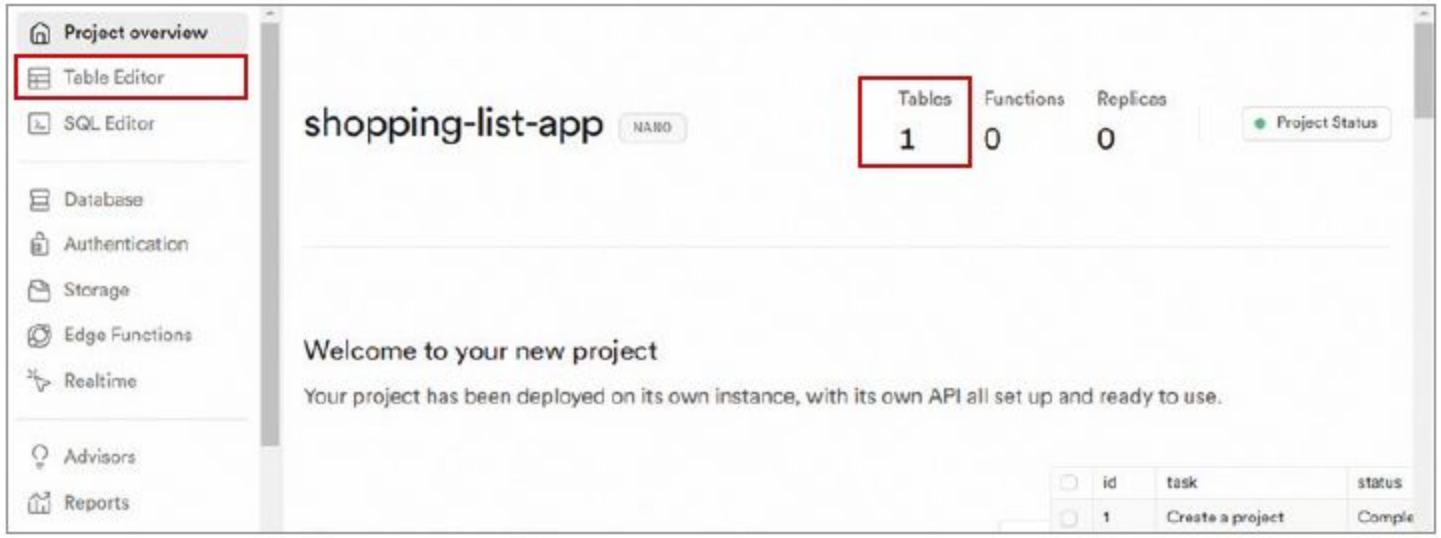
- Supabase 홈페이지에서 데이터베이스가 어떻게 생성되었는지 확인

01 대시보드에 'shopping-list-app'이라는 이름의 새 프로젝트가 생성
클릭해서 상세 정보를 확인



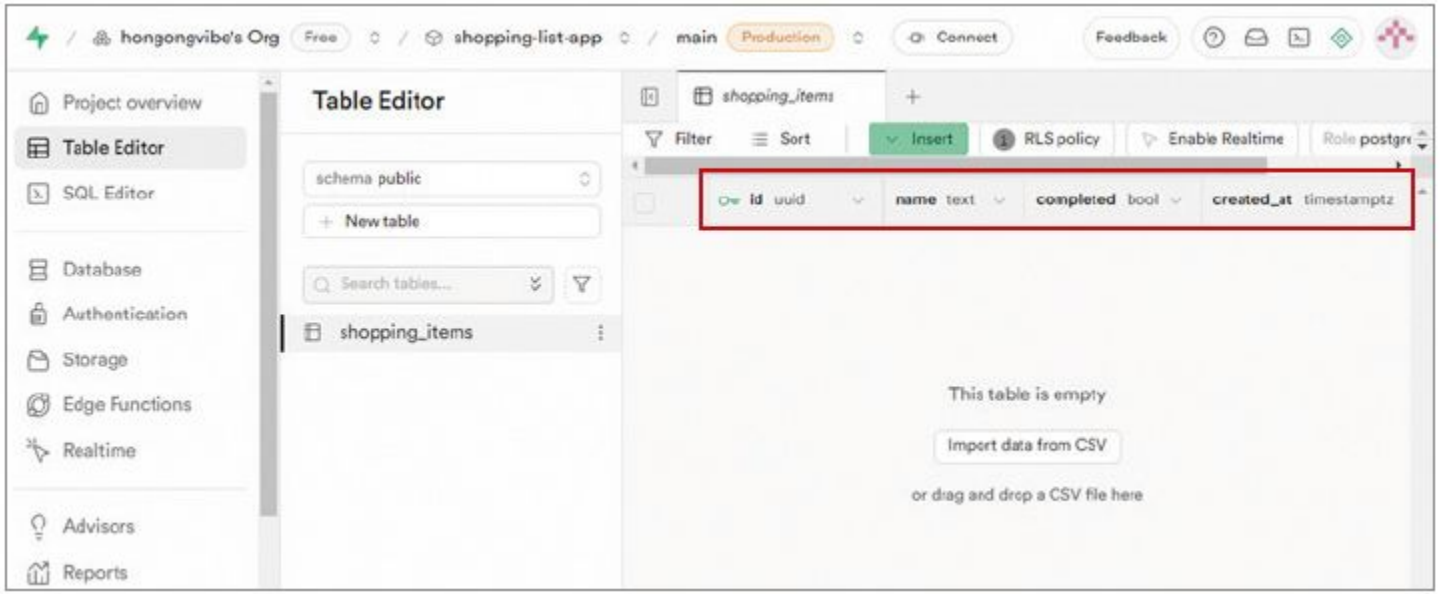
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(38)

02 우측 상단에 'Tables'에 숫자 '1'이 표시된 것으로 새로운 하나의 테이블이 추가되었음을 알 수 있음 왼쪽 사이드바에서 [Table Editor] 메뉴를 클릭



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(39)

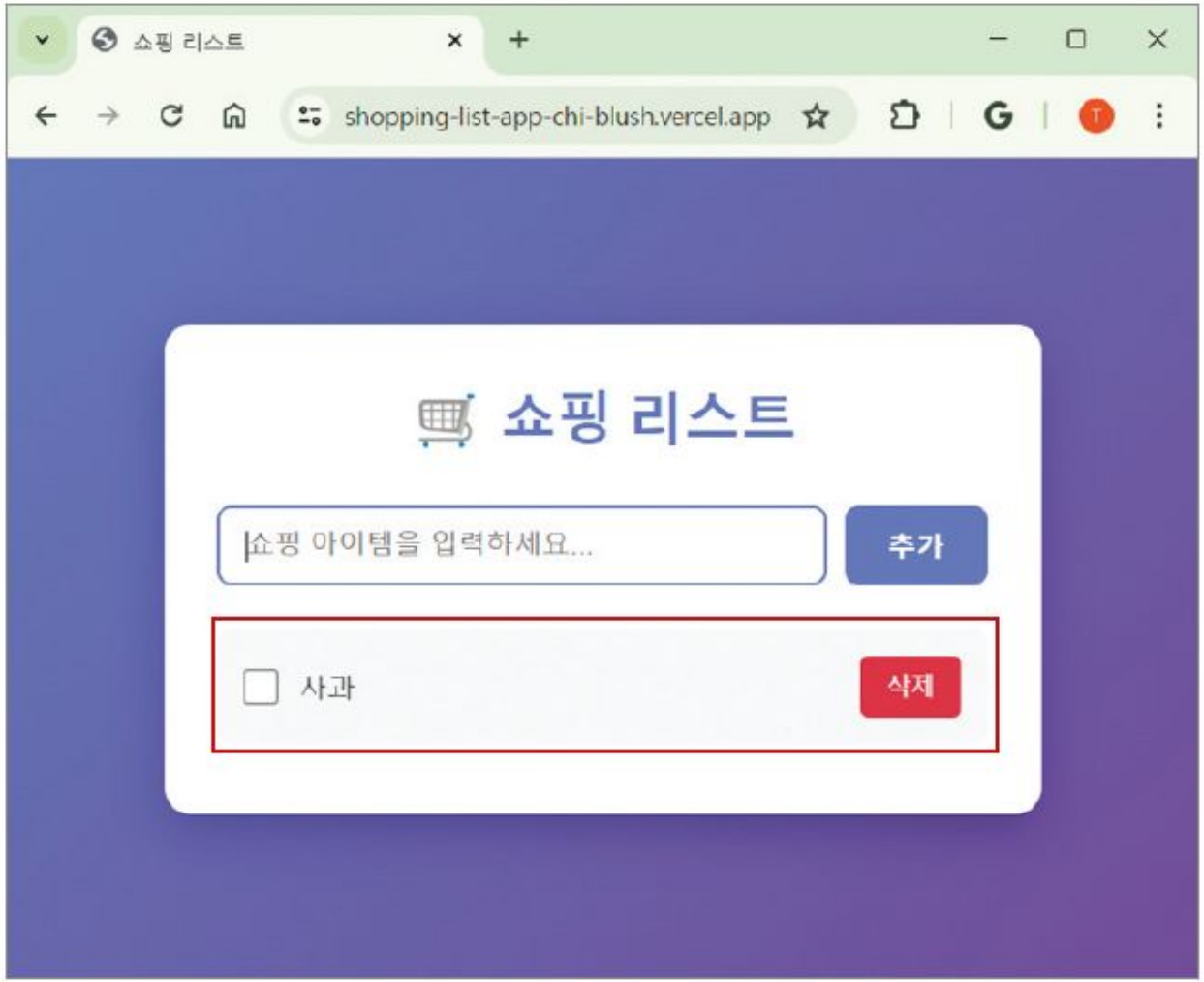
03 [Table Editor] 창에서 [shopping_items] 테이블을 클릭
id, name, completed, created _at 열(column)이 생성된 것을 확인



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(40)

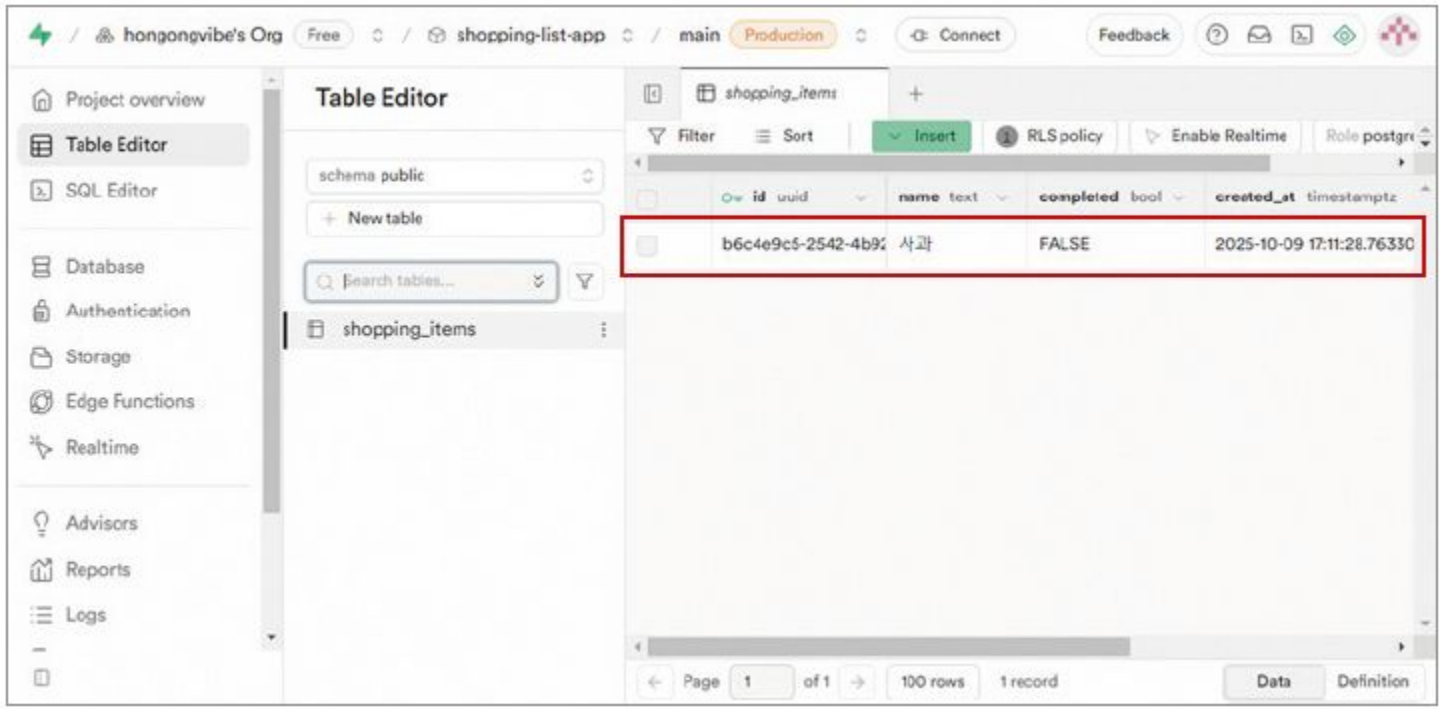
데이터베이스와의 실시간 동기화 확인하기

01 쇼핑 리스트 앱에 접속해, '사과'를 입력하고 [추가] 버튼을 클릭해 리스트에 아이템을 추가



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(41)

02 Supabase의 [Table Editor] 창을 새로고침하면 '사과' 행이 추가되었음을 확인



SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(42)

03 쇼핑 리스트 앱에서 '사과' 아이템에 체크 박스를 클릭해 완료 상태로 만듦

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(43)

04 테이블의 completed 열이 'TRUE'로 바뀜

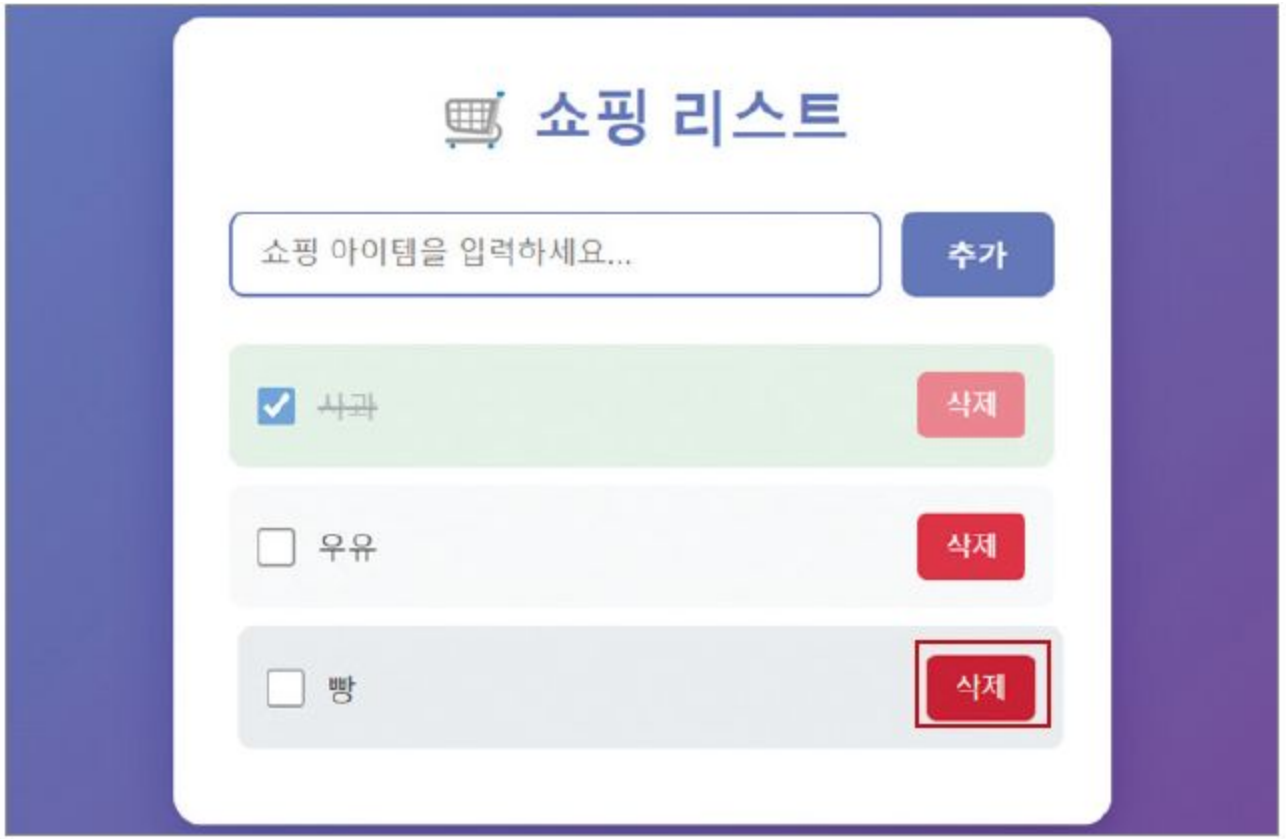
id uuid	name text	completed bool	created_at timestamptz
b6c4e9c5-2542-4b92	사과	TRUE	2025-10-09 17:11:28.76330

05 쇼핑 리스트 앱에 다른 아이템들도 추가하고, 데이터베이스의 테이블을 확인
추가한 아이템이 행에 추가됨

id uuid	name text	completed bool	created_at timestamptz
85f9d216-0c4b-4a7e-	빵	FALSE	2025-10-09 17:18:26.2059
b6c4e9c5-2542-4b92	사과	TRUE	2025-10-09 17:11:28.76330
dbd4e1ca-b348-4bef	우유	FALSE	2025-10-09 17:16:30.5716

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(44)

06 아이템 삭제 - 쇼핑 리스트 앱에서 '빵' 아이템을 삭제



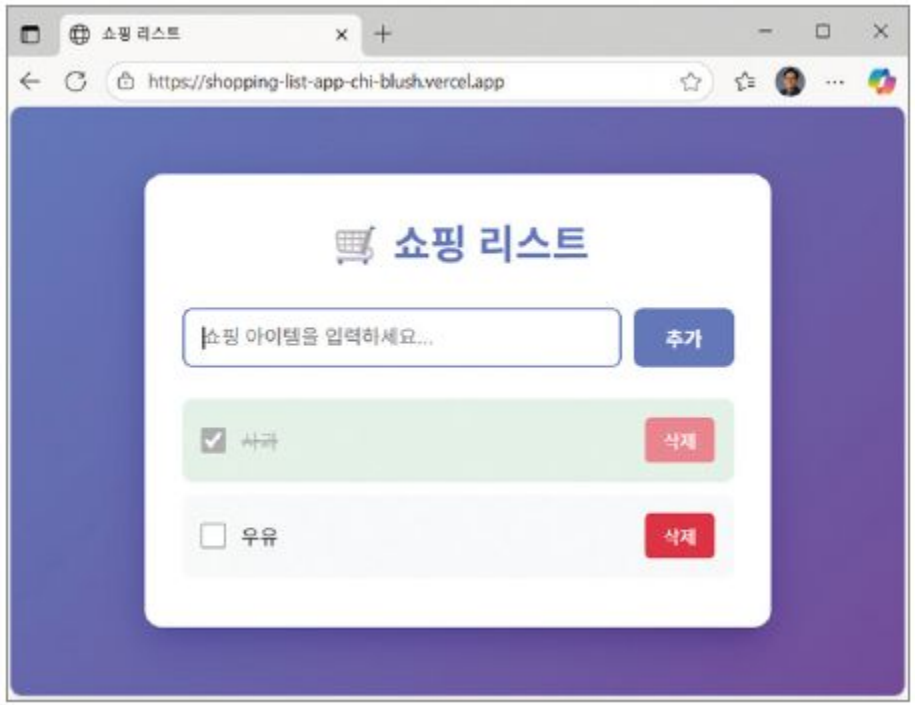
SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(45)

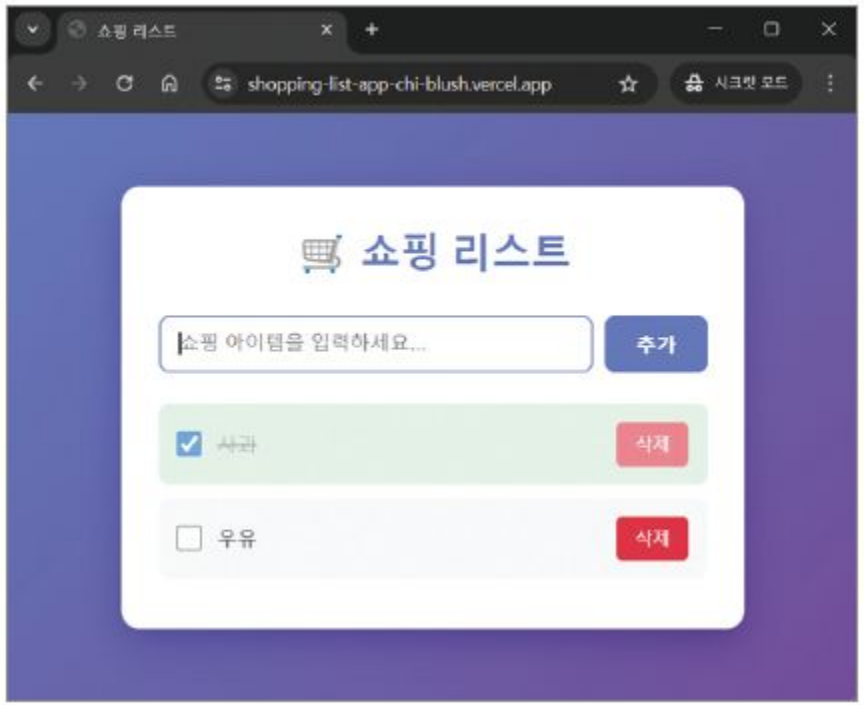
07 테이블에서 '빵' 행이 삭제됨

id uuid	name text	completed bool	created_at timestamptz
b6c4e9c5-2542-4b92	사과	TRUE	2025-10-09 17:11:28.76330
dbd4e1ca-b348-4bef	우유	FALSE	2025-10-09 17:16:30.57163

SECTION 8-3 데이터베이스 연결해 진짜 서비스 만들기(46)

08 다른 컴퓨터나 브라우저, 시크릿 모드에서도 역시 잘 작동하는 것을 확인





[마무리]

- 6가지 키워드로 정리하는 핵심 포인트
  - **배포**는 내 애플리케이션을 외부의 인터넷에 연결된 제3의 서버에 올리는 과정
  - **Vercel**은 작성한 코드를 깃허브에 올리면 자동으로 빌드(build), 배포해주는 클라우드 플랫폼
  - **로컬 스토리지**에 저장된 데이터는 다른 컴퓨터나 브라우저에는 공유되지 않고 브라우저 캐시를 지우거나 기기를 바꾸면 사라짐
  - **클라우드 스토리지**는 어디서나 동일한 데이터를 불러올 수 있는 중앙 서버 역할을 수행
  - **Supabase**는 데이터베이스 구축을 자동화해 주는 클라우드 데이터베이스 플랫폼
  - 데이터베이스의 **테이블**은 쉽게 말하면 데이터를 표 형태로 저장한 구조. **데이터베이스**가 여러 엑셀 파일을 관리하는 프로그램이라면, 테이블은 데이터베이스 안에 있는 개별 엑셀 시트와 같음